p-adics

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Introduction to the p-adics 1

2 Factory 9

3 Local Generic 107

4 p-adic Generic 131

5 p-adic Generic Nodes 149

6 p-adic Base Generic 173

7 p-adic Extension Generic 179

8 Eisenstein Extension Generic 191

9 Unramified Extension Generic 197

10 p-adic Base Leaves 203

11 p-adic Extension Leaves 213

12 Local Generic Element 217

13 p-adic Generic Element 231

14 p-adic Capped Relative Elements 273

15 p-adic Capped Absolute Elements 301

16 p-adic Fixed-Mod Element 321

17 p-adic Extension Element 343

18 p-adic `zz_pX Element` 347

19 p-adic `zz_pX` CR Element 349

20 p-adic `zz_pX` CA Element 365

21 p-adic `zz_pX` FM Element 381

22 PowComputer 397

23 PowComputer_ext 401

24 p-adic Printing 407

25 Precision Error 415

26 Lattice precision for the parents ZpLC/QpLC and ZpLF/QpLF 417

27 Miscellaneous Functions 445

28 Tools for creation of p-adic elements 451

29 Frobenius endomorphisms on p-adic fields 453

30 Witt vector rings 457

31 Witt vectors 477

32 Indices and Tables 483

Python Module Index 485

Index 487

CHAPTER

ONE

INTRODUCTION TO THE p-ADICS

This tutorial outlines what you need to know in order to use p-adics in Sage effectively.

Our goal is to create a rich structure of different options that will reflect the mathematical structures of the p-adics. This is very much a work in progress: some of the classes that we eventually intend to include have not yet been written, and some of the functionality for classes in existence has not yet been implemented. In addition, while we strive for perfect code, bugs (both subtle and not-so-subtle) continue to evade our clutches. As a user, you serve an important role. By writing non-trivial code that uses the p-adics, you both give us insight into what features are actually used and also expose problems in the code for us to fix.

Our design philosophy has been to create a robust, usable interface working first, with simple-minded implementations underneath. We want this interface to stabilize rapidly, so that users' code does not have to change. Once we get the framework in place, we can go back and work on the algorithms and implementations underneath. All of the current p-adic code is currently written in pure Python, which means that it does not have the speed advantage of compiled code. Thus our p-adics can be painfully slow at times when you're doing real computations. However, finding and fixing bugs in Python code is *far* easier than finding and fixing errors in the compiled alternative within Sage (Cython), and Python code is also faster and easier to write. We thus have significantly more functionality implemented and working than we would have if we had chosen to focus initially on speed. And at some point in the future, we will go back and improve the speed. Any code you have written on top of our p-adics will then get an immediate performance enhancement.

If you do find bugs, have feature requests or general comments, please email sage-support@groups.google.com or roed@math.harvard.edu.

1.1 Terminology and types of p-adics

To write down a general p-adic element completely would require an infinite amount of data. Since computers do not have infinite storage space, we must instead store finite approximations to elements. Thus, just as in the case of floating point numbers for representing reals, we have to store an element to a finite precision level. The different ways of doing this account for the different types of p-adics.

We can think of p-adics in two ways. First, as a projective limit of finite groups:

$$\mathbf{Z}_p = \lim_{\leftarrow n} \mathbf{Z}/p^n\mathbf{Z}.$$

Secondly, as Cauchy sequences of rationals (or integers, in the case of $\mathbf{Z}_p$) under the p-adic metric. Since we only need to consider these sequences up to equivalence, this second way of thinking of the p-adics is the same as considering power series in p with integral coefficients in the range 0 to $p-1$. If we only allow nonnegative powers of p then these power series converge to elements of $\mathbf{Z}_p$, and if we allow bounded negative powers of p then we get $\mathbf{Q}_p$.

Both of these representations give a natural way of thinking about finite approximations to a p-adic element. In the first representation, we can just stop at some point in the projective limit, giving an element of $\mathbf{Z}/p^n\mathbf{Z}$. As $\mathbf{Z}_p/p^n\mathbf{Z}_p \cong \mathbf{Z}/p^n\mathbf{Z}$, this is equivalent to specifying our element modulo $p^n\mathbf{Z}_p$.

The *absolute precision* of a finite approximation $\bar{x} \in \mathbf{Z}/p^n\mathbf{Z}$ to $x \in \mathbf{Z}_p$ is the nonnegative integer n.

In the second representation, we can achieve the same thing by truncating a series

$$a_0 + a_1 p + a_2 p^2 + \cdots$$

at p^n, yielding

$$a_0 + a_1 p + \cdots + a_{n-1} p^{n-1} + O(p^n).$$

As above, we call this n the absolute precision of our element.

Given any $x \in \mathbf{Q}_p$ with $x \neq 0$, we can write $x = p^v u$ where $v \in \mathbf{Z}$ and $u \in \mathbf{Z}_p^\times$. We could thus also store an element of $\mathbf{Q}_p$ (or $\mathbf{Z}_p$) by storing v and a finite approximation of u. This motivates the following definition: the *relative precision* of an approximation to x is defined as the absolute precision of the approximation minus the valuation of x. For example, if $x = a_k p^k + a_{k+1} p^{k+1} + \cdots + a_{n-1} p^{n-1} + O(p^n)$ then the absolute precision of x is n, the valuation of x is k and the relative precision of x is $n - k$.

There are three different representations of $\mathbf{Z}_p$ in Sage and one representation of $\mathbf{Q}_p$:

- the fixed modulus ring
- the capped absolute precision ring
- the capped relative precision ring, and
- the capped relative precision field.

1.1.1 Fixed Modulus Rings

The first, and simplest, type of $\mathbf{Z}_p$ is basically a wrapper around $\mathbf{Z}/p^n\mathbf{Z}$, providing a unified interface with the rest of the p-adics. You specify a precision, and all elements are stored to that absolute precision. If you perform an operation that would normally lose precision, the element does not track that it no longer has full precision.

The fixed modulus ring provides the lowest level of convenience, but it is also the one that has the lowest computational overhead. Once we have ironed out some bugs, the fixed modulus elements will be those most optimized for speed.

As with all of the implementations of $\mathbf{Z}_p$, one creates a new ring using the constructor `Zp`, and passing in `'fixed-mod'` for the `type` parameter. For example,

```
sage: R = Zp(5, prec = 10, type = 'fixed-mod', print_mode = 'series')
sage: R
5-adic Ring of fixed modulus 5^10
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), prec = Integer(10), type = 'fixed-mod', print_mode = 'series')
>>> R
5-adic Ring of fixed modulus 5^10
```

One can create elements as follows:

```
sage: a = R(375)
sage: a
3*5^3
sage: b = R(105)
sage: b
5 + 4*5^2
```

```
>>> from sage.all import *
>>> a = R(Integer(375))
>>> a
3*5^3
>>> b = R(Integer(105))
>>> b
5 + 4*5^2
```

Now that we have some elements, we can do arithmetic in the ring.

```
sage: a + b
5 + 4*5^2 + 3*5^3
sage: a * b
3*5^4 + 2*5^5 + 2*5^6
```

```
>>> from sage.all import *
>>> a + b
5 + 4*5^2 + 3*5^3
>>> a * b
3*5^4 + 2*5^5 + 2*5^6
```

Floor division (//) divides even though the result isn't really known to the claimed precision; note that division isn't defined:

```
sage: a // 5
3*5^2
```

```
>>> from sage.all import *
>>> a // Integer(5)
3*5^2
```

```
sage: a / 5
Traceback (most recent call last):
...
ValueError: cannot invert non-unit
```

```
>>> from sage.all import *
>>> a / Integer(5)
Traceback (most recent call last):
...
ValueError: cannot invert non-unit
```

Since elements don't actually store their actual precision, one can only divide by units:

```
sage: a / 2
4*5^3 + 2*5^4 + 2*5^5 + 2*5^6 + 2*5^7 + 2*5^8 + 2*5^9
sage: a / b
Traceback (most recent call last):
...
ValueError: cannot invert non-unit
```

```
>>> from sage.all import *
>>> a / Integer(2)
```

(continues on next page)

(continued from previous page)

```
4*5^3 + 2*5^4 + 2*5^5 + 2*5^6 + 2*5^7 + 2*5^8 + 2*5^9
>>> a / b
Traceback (most recent call last):
...
ValueError: cannot invert non-unit
```

If you want to divide by a non-unit, do it using the `//` operator:

```
sage: a // b
3*5^2 + 3*5^3 + 2*5^5 + 5^6 + 4*5^7 + 2*5^8 + 3*5^9
```

```
>>> from sage.all import *
>>> a // b
3*5^2 + 3*5^3 + 2*5^5 + 5^6 + 4*5^7 + 2*5^8 + 3*5^9
```

1.1.2 Capped Absolute Rings

The second type of implementation of $\mathbf{Z}_p$ is similar to the fixed modulus implementation, except that individual elements track their known precision. The absolute precision of each element is limited to be less than the precision cap of the ring, even if mathematically the precision of the element would be known to greater precision (see Appendix A for the reasons for the existence of a precision cap).

Once again, use `Zp` to create a capped absolute p-adic ring.

```
sage: R = Zp(5, prec = 10, type = 'capped-abs', print_mode = 'series')
sage: R
5-adic Ring with capped absolute precision 10
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), prec = Integer(10), type = 'capped-abs', print_mode = 'series')
>>> R
5-adic Ring with capped absolute precision 10
```

We can do similar things as in the fixed modulus case:

```
sage: a = R(375)
sage: a
3*5^3 + O(5^10)
sage: b = R(105)
sage: b
5 + 4*5^2 + O(5^10)
sage: a + b
5 + 4*5^2 + 3*5^3 + O(5^10)
sage: a * b
3*5^4 + 2*5^5 + 2*5^6 + O(5^10)
sage: c = a // 5
sage: c
3*5^2 + O(5^9)
```

```
>>> from sage.all import *
>>> a = R(Integer(375))
>>> a
```

(continues on next page)

(continued from previous page)

```
3*5^3 + O(5^10)
>>> b = R(Integer(105))
>>> b
5 + 4*5^2 + O(5^10)
>>> a + b
5 + 4*5^2 + 3*5^3 + O(5^10)
>>> a * b
3*5^4 + 2*5^5 + 2*5^6 + O(5^10)
>>> c = a // Integer(5)
>>> c
3*5^2 + O(5^9)
```

Note that when we divided by 5, the precision of `c` dropped. This lower precision is now reflected in arithmetic.

```
sage: c + b
5 + 2*5^2 + 5^3 + O(5^9)
```

```
>>> from sage.all import *
>>> c + b
5 + 2*5^2 + 5^3 + O(5^9)
```

Division is allowed: the element that results is a capped relative field element, which is discussed in the next section:

```
sage: 1 / (c + b)
5^-1 + 3 + 2*5 + 5^2 + 4*5^3 + 4*5^4 + 3*5^6 + O(5^7)
```

```
>>> from sage.all import *
>>> Integer(1) / (c + b)
5^-1 + 3 + 2*5 + 5^2 + 4*5^3 + 4*5^4 + 3*5^6 + O(5^7)
```

1.1.3 Capped Relative Rings and Fields

Instead of restricting the absolute precision of elements (which doesn't make much sense when elements have negative valuations), one can cap the relative precision of elements. This is analogous to floating point representations of real numbers. As in the reals, multiplication works very well: the valuations add and the relative precision of the product is the minimum of the relative precisions of the inputs. Addition, however, faces similar issues as floating point addition: relative precision is lost when lower order terms cancel.

To create a capped relative precision ring, use `Zp` as before. To create capped relative precision fields, use `Qp`.

```
sage: R = Zp(5, prec = 10, type = 'capped-rel', print_mode = 'series')
sage: R
5-adic Ring with capped relative precision 10
sage: K = Qp(5, prec = 10, type = 'capped-rel', print_mode = 'series')
sage: K
5-adic Field with capped relative precision 10
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), prec = Integer(10), type = 'capped-rel', print_mode = 'series')
>>> R
5-adic Ring with capped relative precision 10
>>> K = Qp(Integer(5), prec = Integer(10), type = 'capped-rel', print_mode = 'series')
```

(continues on next page)

(continued from previous page)

```
>>> K
5-adic Field with capped relative precision 10
```

We can do all of the same operations as in the other two cases, but precision works a bit differently: the maximum precision of an element is limited by the precision cap of the ring.

```
sage: a = R(375)
sage: a
3*5^3 + O(5^13)
sage: b = K(105)
sage: b
5 + 4*5^2 + O(5^11)
sage: a + b
5 + 4*5^2 + 3*5^3 + O(5^11)
sage: a * b
3*5^4 + 2*5^5 + 2*5^6 + O(5^14)
sage: c = a // 5
sage: c
3*5^2 + O(5^12)
sage: c + 1
1 + 3*5^2 + O(5^10)
```

```
>>> from sage.all import *
>>> a = R(Integer(375))
>>> a
3*5^3 + O(5^13)
>>> b = K(Integer(105))
>>> b
5 + 4*5^2 + O(5^11)
>>> a + b
5 + 4*5^2 + 3*5^3 + O(5^11)
>>> a * b
3*5^4 + 2*5^5 + 2*5^6 + O(5^14)
>>> c = a // Integer(5)
>>> c
3*5^2 + O(5^12)
>>> c + Integer(1)
1 + 3*5^2 + O(5^10)
```

As with the capped absolute precision rings, we can divide, yielding a capped relative precision field element.

```
sage: 1 / (c + b)
5^-1 + 3 + 2*5 + 5^2 + 4*5^3 + 4*5^4 + 3*5^6 + 2*5^7 + 5^8 + O(5^9)
```

```
>>> from sage.all import *
>>> Integer(1) / (c + b)
5^-1 + 3 + 2*5 + 5^2 + 4*5^3 + 4*5^4 + 3*5^6 + 2*5^7 + 5^8 + O(5^9)
```

1.1.4 Unramified Extensions

One can create unramified extensions of $\mathbf{Z}_p$ and $\mathbf{Q}_p$ using the functions `Zq` and `Qq`.

In addition to requiring a prime power as the first argument, `Zq` also requires a name for the generator of the residue field. One can specify this name as follows:

```
sage: R.<c> = Zq(125, prec=20); R                                               #␣
↪needs sage.libs.ntl
5-adic Unramified Extension Ring in c defined by x^3 + 3*x + 3
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), prec=Integer(20), names=('c',)); (c,) = R._first_ngens(1); R␣
↪                                                  # needs sage.libs.ntl
5-adic Unramified Extension Ring in c defined by x^3 + 3*x + 3
```

1.1.5 Eisenstein Extensions

It is also possible to create Eisenstein extensions of $\mathbf{Z}_p$ and $\mathbf{Q}_p$. In order to do so, create the ground field first:

```
sage: R = Zp(5, 2)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(2))
```

Then define the polynomial yielding the desired extension.:

```
sage: S.<x> = ZZ[]
sage: f = x^5 - 25*x^3 + 15*x - 5
```

```
>>> from sage.all import *
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) - Integer(25)*x**Integer(3) + Integer(15)*x - Integer(5)
```

Finally, use the `ext` function on the ground field to create the desired extension.:

```
sage: W.<w> = R.ext(f)                                                          #␣
↪needs sage.libs.ntl sage.rings.padics
```

```
>>> from sage.all import *
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)# needs sage.libs.ntl sage.
↪rings.padics
```

You can do arithmetic in this Eisenstein extension:

```
sage: (1 + w)^7                                                                 #␣
↪needs sage.libs.ntl sage.rings.padics
1 + 2*w + w^2 + w^5 + 3*w^6 + 3*w^7 + 3*w^8 + w^9 + O(w^10)
```

```
>>> from sage.all import *
>>> (Integer(1) + w)**Integer(7)                                                ␣
↪               # needs sage.libs.ntl sage.rings.padics
1 + 2*w + w^2 + w^5 + 3*w^6 + 3*w^7 + 3*w^8 + w^9 + O(w^10)
```

Note that the precision cap increased by a factor of 5, since the ramification index of this extension over $\mathbf{Z}_p$ is 5.

CHAPTER

TWO

FACTORY

This file contains the constructor classes and functions for p-adic rings and fields.

AUTHORS:

- David Roe

sage.rings.padics.factory.**Qp = <sage.rings.padics.factory.Qp_class object>**

A creation function for p-adic fields.

INPUT:

- `p` – integer; the p in $\mathbf{Q}_p$
- `prec` – integer (default: 20); the precision cap of the field. In the lattice capped case, `prec` can either be a pair (`relative_cap`, `absolute_cap`) or an integer (understood at relative cap). In the relaxed case, `prec` can be either a pair (`default_prec`, `halting_prec`) or an integer (understood at default precision). Except in the floating point case, individual elements keep track of their own precision. See TYPES and PRECISION below.
- `type` – string (default: `'capped-rel'`); valid types are `'capped-rel'`, `'floating-point'`, `'lattice-cap'`, `'lattice-float'`. See TYPES and PRECISION below.
- `print_mode` – string (default: `None`); valid modes are `'series'`, `'val-unit'`, `'terse'`, `'digits'`, and `'bars'`. See PRINTING below.
- `names` – string or tuple (defaults to a string representation of p); what to use whenever p is printed
- `ram_name` – string; another way to specify the name. For consistency with the `Qq` and `Zq` and extension functions.
- `print_pos` – boolean (default: `None`); whether to only use positive integers in the representations of elements. See PRINTING below.
- `print_sep` – string (default: `None`); the separator character used in the `'bars'` mode. See PRINTING below.
- `print_alphabet` – tuple (default: `None`); the encoding into digits for use in the 'digits' mode. See PRINTING below.
- `print_max_terms` – integer (default: `None`); the maximum number of terms shown. See PRINTING below.
- `show_prec` – boolean or a string (default: `None`); specify how the precision is printed. See PRINTING below.
- `check` – boolean (default: `True`); whether to check if p is prime. Non-prime input may cause seg-faults (but can also be useful for base n expansions for example).
- `label` – string (default: `None`); used for lattice precision to create parents with different lattices

OUTPUT: the corresponding p-adic field

TYPES AND PRECISION:

There are two main types of precision for a p-adic element. The first is relative precision, which gives the number of known p-adic digits:

```
sage: R = Qp(5, 20, 'capped-rel', 'series'); a = R(675); a
2*5^2 + 5^4 + O(5^22)
sage: a.precision_relative()
20
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), Integer(20), 'capped-rel', 'series'); a = R(Integer(675));␣
↪a
2*5^2 + 5^4 + O(5^22)
>>> a.precision_relative()
20
```

The second type of precision is absolute precision, which gives the power of p that this element is defined modulo:

```
sage: a.precision_absolute()
22
```

```
>>> from sage.all import *
>>> a.precision_absolute()
22
```

There are several types of p-adic fields, depending on the methods used for tracking precision. Namely, we have:

- capped relative fields (`type='capped-rel'`)
- capped absolute fields (`type='capped-abs'`)
- fixed modulus fields (`type='fixed-mod'`)
- floating point fields (`type='floating-point'`)
- lattice precision fields (`type='lattice-cap'` or `type='lattice-float'`)
- exact fields with relaxed arithmetics (`type='relaxed'`)

In the capped relative case, the relative precision of an element is restricted to be at most a certain value, specified at the creation of the field. Individual elements also store their own precision, so the effect of various arithmetic operations on precision is tracked. When you cast an exact element into a capped relative field, it truncates it to the precision cap of the field.

```
sage: R = Qp(5, 5, 'capped-rel', 'series'); a = R(4006); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(4025); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
sage: a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), Integer(5), 'capped-rel', 'series'); a = R(Integer(4006));␣
↪a
```

(continues on next page)

(continued from previous page)

```
1 + 5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(4025)); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
>>> a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

In the floating point case, elements do not track their precision, but the relative precision of elements is truncated during arithmetic to the precision cap of the field.

In the lattice case, precision on elements is tracked by a global lattice that is updated after every operation, yielding better precision behavior at the cost of higher memory and runtime usage. We refer to the documentation of the function *ZpLC()* for a small demonstration of the capabilities of this precision model.

Finally, the model for relaxed p-adics is quite different from any of the other types. In addition to storing a finite approximation, one also stores a method for increasing the precision. A quite interesting feature with relaxed p-adics is the possibility to create (in some cases) self-referent numbers, that are numbers whose n-th digit is defined by the previous ones. We refer to the documentation of the function *ZpER* for a small demonstration of the capabilities of this precision model.

PRINTING:

There are many different ways to print p-adic elements. The way elements of a given field print is controlled by options passed in at the creation of the field. There are five basic printing modes (series, val-unit, terse, digits and bars), as well as various options that either hide some information in the print representation or sometimes make print representations more compact. Note that the printing options affect whether different p-adic fields are considered equal.

1. **series**: elements are displayed as series in p.

```
sage: R = Qp(5, print_mode='series'); a = R(70700); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
sage: b = R(-70700); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='series'); a = R(Integer(70700)); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
>>> b = R(-Integer(70700)); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

print_pos controls whether negatives can be used in the coefficients of powers of p.

```
sage: S = Qp(5, print_mode='series', print_pos=False); a = S(70700); a
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
sage: b = S(-70700); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='series', print_pos=False); a =␣
↪S(Integer(70700)); a
```

(continues on next page)

(continued from previous page)

```
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
>>> b = S(-Integer(70700)); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

print_max_terms limits the number of terms that appear.

```
sage: T = Qp(5, print_mode='series', print_max_terms=4); b = R(-70700);␣
↪repr(b)
'2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)'
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='series', print_max_terms=Integer(4));␣
↪b = R(-Integer(70700)); repr(b)
'2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)'
```

names affects how the prime is printed.

```
sage: U.<p> = Qp(5); p
p + O(p^21)
```

```
>>> from sage.all import *
>>> U = Qp(Integer(5), names=('p',)); (p,) = U._first_ngens(1); p
p + O(p^21)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`) or 'bigoh'. The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: Qp(5)(6)
1 + 5 + O(5^20)
sage: Qp(5, show_prec='none')(6)
1 + 5

sage: QpFP(5)(6)
1 + 5
```

```
>>> from sage.all import *
>>> Qp(Integer(5))(Integer(6))
1 + 5 + O(5^20)
>>> Qp(Integer(5), show_prec='none')(Integer(6))
1 + 5

>>> QpFP(Integer(5))(Integer(6))
1 + 5
```

print_sep and *print_alphabet* have no effect in series mode.

Note that print options affect equality:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

2. **val-unit**: elements are displayed as p^k*u:

```
sage: R = Qp(5, print_mode='val-unit'); a = R(70700); a
5^2 * 2828 + O(5^22)
sage: b = R(-707/5); b
5^-1 * 95367431639918 + O(5^19)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='val-unit'); a = R(Integer(70700)); a
5^2 * 2828 + O(5^22)
>>> b = R(-Integer(707)/Integer(5)); b
5^-1 * 95367431639918 + O(5^19)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Qp(5, print_mode='val-unit', print_pos=False); b = S(-70700); b
5^2 * (-2828) + O(5^22)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='val-unit', print_pos=False); b = S(-
↪Integer(70700)); b
5^2 * (-2828) + O(5^22)
```

names affects how the prime is printed.

```
sage: T = Qp(5, print_mode='val-unit', names='pi'); a = T(70700); a
pi^2 * 2828 + O(pi^22)
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='val-unit', names='pi'); a =␣
↪T(Integer(70700)); a
pi^2 * 2828 + O(pi^22)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`) or 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: Qp(5, print_mode='val-unit', show_prec=False)(30)
5 * 6
```

```
>>> from sage.all import *
>>> Qp(Integer(5), print_mode='val-unit', show_prec=False)(Integer(30))
5 * 6
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

3. **terse**: elements are displayed as an integer in base 10 or the quotient of an integer by a power of p (still in base 10):

```
sage: R = Qp(5, print_mode='terse'); a = R(70700); a
70700 + O(5^22)
sage: b = R(-70700); b
2384185790944925 + O(5^22)
sage: c = R(-707/5); c
95367431639918/5 + O(5^19)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='terse'); a = R(Integer(70700)); a
70700 + O(5^22)
>>> b = R(-Integer(70700)); b
2384185790944925 + O(5^22)
>>> c = R(-Integer(707)/Integer(5)); c
95367431639918/5 + O(5^19)
```

The denominator, as of version 3.3, is always printed explicitly as a power of p, for predictability.

```
sage: d = R(707/5^2); d
707/5^2 + O(5^18)
```

```
>>> from sage.all import *
>>> d = R(Integer(707)/Integer(5)**Integer(2)); d
707/5^2 + O(5^18)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Qp(5, print_mode='terse', print_pos=False); b = S(-70700); b
-70700 + O(5^22)
sage: c = S(-707/5); c
-707/5 + O(5^19)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='terse', print_pos=False); b = S(-
↪Integer(70700)); b
-70700 + O(5^22)
>>> c = S(-Integer(707)/Integer(5)); c
-707/5 + O(5^19)
```

name affects how the name is printed.

```
sage: T.<unif> = Qp(5, print_mode='terse'); c = T(-707/5); c
95367431639918/unif + O(unif^19)
sage: d = T(-707/5^10); d
95367431639918/unif^10 + O(unif^10)
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='terse', names=('unif',)); (unif,) = T._
↪first_ngens(1); c = T(-Integer(707)/Integer(5)); c
95367431639918/unif + O(unif^19)
>>> d = T(-Integer(707)/Integer(5)**Integer(10)); d
95367431639918/unif^10 + O(unif^10)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`) or 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: Qp(5, print_mode='terse', show_prec=False)(6)
6
```

```
>>> from sage.all import *
>>> Qp(Integer(5), print_mode='terse', show_prec=False)(Integer(6))
6
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

4. **digits**: elements are displayed as a string of base p digits

   Restriction: you can only use the digits printing mode for small primes. Namely, p must be less than the length of the alphabet tuple (default alphabet has length 62).

```
sage: R = Qp(5, print_mode='digits'); a = R(70700); repr(a)
'...00000000000000004230300'
sage: b = R(-70700); repr(b)
'...44444444444444440214200'
sage: c = R(-707/5); repr(c)
'...44444444444444443413.3'
sage: d = R(-707/5^2); repr(d)
'...44444444444444441.33'
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='digits'); a = R(Integer(70700));␣
↪repr(a)
'...00000000000000004230300'
```

(continues on next page)

(continued from previous page)

```
>>> b = R(-Integer(70700)); repr(b)
'...44444444444444440214200'
>>> c = R(-Integer(707)/Integer(5)); repr(c)
'...444444444444443413.3'
>>> d = R(-Integer(707)/Integer(5)**Integer(2)); repr(d)
'...44444444444444341.33'
```

Observe that the significant 0s are printed even if they are located in front of the number. On the contrary, unknown digits located after the comma appears as question marks. The precision can therefore be read in this mode as well. Here are more examples:

```
sage: p = 7
sage: K = Qp(p, prec=10, print_mode='digits')
sage: repr(K(1))
'...0000000001'
sage: repr(K(p^2))
'...000000000100'
sage: repr(K(p^-5))
'...00000.00001'
sage: repr(K(p^-20))
'...?.??????????0000000001'
```

```
>>> from sage.all import *
>>> p = Integer(7)
>>> K = Qp(p, prec=Integer(10), print_mode='digits')
>>> repr(K(Integer(1)))
'...0000000001'
>>> repr(K(p**Integer(2)))
'...000000000100'
>>> repr(K(p**-Integer(5)))
'...00000.00001'
>>> repr(K(p**-Integer(20)))
'...?.??????????0000000001'
```

print_max_terms limits the number of digits that are printed. Note that if the valuation of the element is very negative, more digits will be printed.

```
sage: S = Qp(5, print_max_terms=4); S(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
sage: S(-707/5^2)
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
sage: S(-707/5^6)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
sage: S(-707/5^6,absprec=-2)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
sage: S(-707/5^4)
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_max_terms=Integer(4)); S(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
>>> S(-Integer(707)/Integer(5)**Integer(2))
```

(continues on next page)

(continued from previous page)

```
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
>>> S(-Integer(707)/Integer(5)**Integer(6))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
>>> S(-Integer(707)/Integer(5)**Integer(6),absprec=-Integer(2))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
>>> S(-Integer(707)/Integer(5)**Integer(4))
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

print_alphabet controls the symbols used to substitute for digits greater than 9.

Defaults to ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'):

```
sage: T = Qp(5, print_mode='digits', print_alphabet=('1','2','3','4','5
↪')); repr(T(-70700))
'...5555555555555551325311'
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='digits', print_alphabet=('1','2','3','4
↪','5')); repr(T(-Integer(70700)))
'...5555555555555551325311'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'dots' (or equivalently `True`) or 'bigoh'. The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: repr(Zp(5, print_mode='digits', show_prec=True)(6))
'...00000000000000000011'

sage: repr(Zp(5, print_mode='digits', show_prec='bigoh')(6))
'11 + O(5^20)'
```

```
>>> from sage.all import *
>>> repr(Zp(Integer(5), print_mode='digits', show_prec=True)(Integer(6)))
'...00000000000000000011'

>>> repr(Zp(Integer(5), print_mode='digits', show_prec='bigoh
↪')(Integer(6)))
'11 + O(5^20)'
```

print_pos, *name* and *print_sep* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

5. **bars**: elements are displayed as a string of base p digits with separators:

```
sage: R = Qp(5, print_mode='bars'); a = R(70700); repr(a)
'...4|2|3|0|3|0|0'
sage: b = R(-70700); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
sage: d = R(-707/5^2); repr(d)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|3|4|1|.|3|3'
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='bars'); a = R(Integer(70700)); repr(a)
'...4|2|3|0|3|0|0'
>>> b = R(-Integer(70700)); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
>>> d = R(-Integer(707)/Integer(5)**Integer(2)); repr(d)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|3|4|1|.|3|3'
```

Again, note that it's not possible to read off the precision from the representation in this mode.

print_pos controls whether the digits can be negative.

```
sage: S = Qp(5, print_mode='bars',print_pos=False); b = S(-70700); repr(b)
'...-1|0|2|2|-1|2|0|0'
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='bars',print_pos=False); b = S(-
↪Integer(70700)); repr(b)
'...-1|0|2|2|-1|2|0|0'
```

print_max_terms limits the number of digits that are printed. Note that if the valuation of the element is very negative, more digits will be printed.

```
sage: T = Qp(5, print_max_terms=4); T(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
sage: T(-707/5^2)
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
sage: T(-707/5^6)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
sage: T(-707/5^6,absprec=-2)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
sage: T(-707/5^4)
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_max_terms=Integer(4)); T(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
>>> T(-Integer(707)/Integer(5)**Integer(2))
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
>>> T(-Integer(707)/Integer(5)**Integer(6))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
>>> T(-Integer(707)/Integer(5)**Integer(6),absprec=-Integer(2))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
>>> T(-Integer(707)/Integer(5)**Integer(4))
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

print_sep controls the separation character.

```
sage: U = Qp(5, print_mode='bars', print_sep=']['); a = U(70700); repr(a)
'...4][2][3][0][3][0][0'
```

```
>>> from sage.all import *
>>> U = Qp(Integer(5), print_mode='bars', print_sep=']['); a =␣
↪U(Integer(70700)); repr(a)
'...4][2][3][0][3][0][0'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'dots' (or equivalently `True`) or 'bigoh' The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: repr(Qp(5, print_mode='bars', show_prec='bigoh')(6))
'...1|1 + O(5^20)'
```

```
>>> from sage.all import *
>>> repr(Qp(Integer(5), print_mode='bars', show_prec='bigoh')(Integer(6)))
'...1|1 + O(5^20)'
```

name and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

EXAMPLES:

```
sage: K = Qp(15, check=False); a = K(999); a
9 + 6*15 + 4*15^2 + O(15^20)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(15), check=False); a = K(Integer(999)); a
9 + 6*15 + 4*15^2 + O(15^20)
```

`sage.rings.padics.factory.`**`Zp = <sage.rings.padics.factory.Zp_class object>`**

A creation function for p-adic rings.

INPUT:

- `p` – integer; the p in $\mathbf{Z}_p$
- `prec` – integer (default: 20); the precision cap of the ring. In the lattice capped case, `prec` can either be a pair (`relative_cap`, `absolute_cap`) or an integer (understood as relative cap). In the relaxed case, `prec` can be either a pair (`default_prec`, `halting_prec`) or an integer (understood as default precision). Except for the fixed modulus and floating point cases, individual elements keep track of their own precision. See TYPES and PRECISION below.

- `type` – string (default: `'capped-rel'`); valid types are `'capped-rel'`, `'capped-abs'`, `'fixed-mod'`, `'floating-point'`, `'lattice-cap'`, `'lattice-float'`, `'relaxed'`. See TYPES and PRECISION below.
- `print_mode` – string (default: `None`); valid modes are `'series'`, `'val-unit'`, `'terse'`, `'digits'`, and `'bars'`. See PRINTING below.
- `names` – string or tuple (defaults to a string representation of p); what to use whenever p is printed
- `print_pos` – boolean (default: `None`); whether to only use positive integers in the representations of elements. See PRINTING below.
- `print_sep` – string (default: `None`); the separator character used in the `'bars'` mode. See PRINTING below.
- `print_alphabet` – tuple (default: `None`); the encoding into digits for use in the `'digits'` mode. See PRINTING below.
- `print_max_terms` – integer (default: `None`); the maximum number of terms shown. See PRINTING below.
- `show_prec` – boolean (default: `None`) whether to show the precision for elements. See PRINTING below.
- `check` – boolean (default: `True`) whether to check if p is prime. Non-prime input may cause seg-faults (but can also be useful for base n expansions for example)
- `label` – string (default: `None`); used for lattice precision to create parents with different lattices

OUTPUT: the corresponding p-adic ring

TYPES AND PRECISION:

There are two main types of precision. The first is relative precision; it gives the number of known p-adic digits:

```
sage: R = Zp(5, 20, 'capped-rel', 'series'); a = R(675); a
2*5^2 + 5^4 + O(5^22)
sage: a.precision_relative()
20
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(20), 'capped-rel', 'series'); a = R(Integer(675));␣
↪a
2*5^2 + 5^4 + O(5^22)
>>> a.precision_relative()
20
```

The second type of precision is absolute precision, which gives the power of p that this element is defined modulo:

```
sage: a.precision_absolute()
22
```

```
>>> from sage.all import *
>>> a.precision_absolute()
22
```

There are several types of p-adic rings, depending on the methods used for tracking precision. Namely, we have:

- capped relative rings (`type='capped-rel'`)
- capped absolute rings (`type='capped-abs'`)

- fixed modulus rings (`type='fixed-mod'`)
- floating point rings (`type='floating-point'`)
- lattice precision rings (`type='lattice-cap'` or `type='lattice-float'`)
- exact fields with relaxed arithmetics (`type='relaxed'`)

In the capped relative case, the relative precision of an element is restricted to be at most a certain value, specified at the creation of the field. Individual elements also store their own precision, so the effect of various arithmetic operations on precision is tracked. When you cast an exact element into a capped relative field, it truncates it to the precision cap of the field.

```
sage: R = Zp(5, 5, 'capped-rel', 'series'); a = R(4006); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(4025); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
sage: a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(5), 'capped-rel', 'series'); a = R(Integer(4006));␣
↪a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(4025)); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
>>> a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

In the capped absolute type, instead of having a cap on the relative precision of an element there is instead a cap on the absolute precision. Elements still store their own precisions, and as with the capped relative case, exact elements are truncated when cast into the ring.

```
sage: R = Zp(5, 5, 'capped-abs', 'series'); a = R(4005); a
5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(4025); b
5^2 + 2*5^3 + 5^4 + O(5^5)
sage: a * b
5^3 + 2*5^4 + O(5^5)
sage: (a * b) // 5^3
1 + 2*5 + O(5^2)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(5), 'capped-abs', 'series'); a = R(Integer(4005));␣
↪a
5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(4025)); b
5^2 + 2*5^3 + 5^4 + O(5^5)
>>> a * b
5^3 + 2*5^4 + O(5^5)
>>> (a * b) // Integer(5)**Integer(3)
1 + 2*5 + O(5^2)
```

The fixed modulus type is the leanest of the p-adic rings: it is basically just a wrapper around $\mathbf{Z}/p^n\mathbf{Z}$ providing a unified interface with the rest of the p-adics. This is the type you should use if your sole interest is speed. It does not track precision of elements.

```
sage: R = Zp(5,5,'fixed-mod','series'); a = R(4005); a
5 + 2*5^3 + 5^4
sage: a // 5
1 + 2*5^2 + 5^3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5),'fixed-mod','series'); a = R(Integer(4005)); a
5 + 2*5^3 + 5^4
>>> a // Integer(5)
1 + 2*5^2 + 5^3
```

The floating point case is similar to the fixed modulus type in that elements do not track their own precision. However, relative precision is truncated with each operation rather than absolute precision.

On the contrary, the lattice type tracks precision using lattices and automatic differentiation. It is rather slow but provides sharp (often optimal) results regarding precision. We refer to the documentation of the function *ZpLC()* for a small demonstration of the capabilities of this precision model.

Finally, the model for relaxed p-adics is quite different from any of the other types. In addition to storing a finite approximation, one also stores a method for increasing the precision. A quite interesting feature with relaxed p-adics is the possibility to create (in some cases) self-referent numbers, that are numbers whose n-th digit is defined by the previous ones. We refer to the documentation of the function *ZpER* for a small demonstration of the capabilities of this precision model.

PRINTING:

There are many different ways to print p-adic elements. The way elements of a given ring print is controlled by options passed in at the creation of the ring. There are five basic printing modes (`'series'`, `'val-unit'`, `'terse'`, `'digits'` and `'bars'`), as well as various options that either hide some information in the print representation or sometimes make print representations more compact. Note that the printing options affect whether different p-adic fields are considered equal.

1. **series**: elements are displayed as series in p.

```
sage: R = Zp(5, print_mode='series'); a = R(70700); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
sage: b = R(-70700); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='series'); a = R(Integer(70700)); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
>>> b = R(-Integer(70700)); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

print_pos controls whether negatives can be used in the coefficients of powers of p.

```
sage: S = Zp(5, print_mode='series', print_pos=False); a = S(70700); a
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
sage: b = S(-70700); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='series', print_pos=False); a =␣
↪S(Integer(70700)); a
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
>>> b = S(-Integer(70700)); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

print_max_terms limits the number of terms that appear.

```
sage: T = Zp(5, print_mode='series', print_max_terms=4); b = R(-70700); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='series', print_max_terms=Integer(4));␣
↪b = R(-Integer(70700)); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

names affects how the prime is printed.

```
sage: U.<p> = Zp(5); p
p + O(p^21)
```

```
>>> from sage.all import *
>>> U = Zp(Integer(5), names=('p',)); (p,) = U._first_ngens(1); p
p + O(p^21)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: Zp(5, show_prec=False)(6)
1 + 5
```

```
>>> from sage.all import *
>>> Zp(Integer(5), show_prec=False)(Integer(6))
1 + 5
```

print_sep and *print_alphabet* have no effect.

Note that print options affect equality:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

2. **val-unit**: elements are displayed as $p^k u$:

```
sage: R = Zp(5, print_mode='val-unit'); a = R(70700); a
5^2 * 2828 + O(5^22)
```

(continues on next page)

(continued from previous page)

```
sage: b = R(-707*5); b
5 * 95367431639918 + O(5^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='val-unit'); a = R(Integer(70700)); a
5^2 * 2828 + O(5^22)
>>> b = R(-Integer(707)*Integer(5)); b
5 * 95367431639918 + O(5^21)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Zp(5, print_mode='val-unit', print_pos=False); b = S(-70700); b
5^2 * (-2828) + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='val-unit', print_pos=False); b = S(-
↪Integer(70700)); b
5^2 * (-2828) + O(5^22)
```

names affects how the prime is printed.

```
sage: T = Zp(5, print_mode='val-unit', names='pi'); a = T(70700); a
pi^2 * 2828 + O(pi^22)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='val-unit', names='pi'); a =␣
↪T(Integer(70700)); a
pi^2 * 2828 + O(pi^22)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: Zp(5, print_mode='val-unit', show_prec=False)(30)
5 * 6
```

```
>>> from sage.all import *
>>> Zp(Integer(5), print_mode='val-unit', show_prec=False)(Integer(30))
5 * 6
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

3. **terse**: elements are displayed as an integer in base 10:

```
sage: R = Zp(5, print_mode='terse'); a = R(70700); a
70700 + O(5^22)
sage: b = R(-70700); b
2384185790944925 + O(5^22)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='terse'); a = R(Integer(70700)); a
70700 + O(5^22)
>>> b = R(-Integer(70700)); b
2384185790944925 + O(5^22)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Zp(5, print_mode='terse', print_pos=False); b = S(-70700); b
-70700 + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='terse', print_pos=False); b = S(-
↪Integer(70700)); b
-70700 + O(5^22)
```

name affects how the name is printed. Note that this interacts with the choice of shorter string for denominators.

```
sage: T.<unif> = Zp(5, print_mode='terse'); c = T(-707); c
95367431639918 + O(unif^20)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='terse', names=('unif',)); (unif,) = T._
↪first_ngens(1); c = T(-Integer(707)); c
95367431639918 + O(unif^20)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: Zp(5, print_mode='terse', show_prec=False)(30)
30
```

```
>>> from sage.all import *
>>> Zp(Integer(5), print_mode='terse', show_prec=False)(Integer(30))
30
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

4. **digits**: elements are displayed as a string of base p digits

   Restriction: you can only use the digits printing mode for small primes. Namely, p must be less than the length of the alphabet tuple (default alphabet has length 62).

```
sage: R = Zp(5, print_mode='digits'); a = R(70700); repr(a)
'...4230300'
sage: b = R(-70700); repr(b)
'...44444444444444440214200'
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='digits'); a = R(Integer(70700));␣
↪repr(a)
'...4230300'
>>> b = R(-Integer(70700)); repr(b)
'...44444444444444440214200'
```

   Note that it's not possible to read off the precision from the representation in this mode.

   print_max_terms limits the number of digits that are printed.

```
sage: S = Zp(5, print_max_terms=4); S(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_max_terms=Integer(4)); S(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

   print_alphabet controls the symbols used to substitute for digits greater than 9. Defaults to ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'):

```
sage: T = Zp(5, print_mode='digits', print_alphabet=('1','2','3','4','5'))
sage: repr(T(-70700))
'...55555555555551325311'
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='digits', print_alphabet=('1','2','3','4
↪','5'))
>>> repr(T(-Integer(70700)))
'...55555555555551325311'
```

   show_prec determines how the precision is printed. It can be either `'none'` (or equivalently `False`), `'dots'` (or equivalently `True`) or `'bigoh'`. The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: repr(Zp(5, 2, print_mode='digits', show_prec=True)(6))
'...11'
sage: repr(Zp(5, 2, print_mode='digits', show_prec='bigoh')(6))
'11 + O(5^2)'
```

```
>>> from sage.all import *
>>> repr(Zp(Integer(5), Integer(2), print_mode='digits', show_
↪prec=True)(Integer(6)))
'...11'
>>> repr(Zp(Integer(5), Integer(2), print_mode='digits', show_prec='bigoh
↪')(Integer(6)))
'11 + O(5^2)'
```

print_pos, *name* and *print_sep* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

5. **bars**: elements are displayed as a string of base p digits with separators:

```
sage: R = Zp(5, print_mode='bars'); a = R(70700); repr(a)
'...4|2|3|0|3|0|0'
sage: b = R(-70700); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='bars'); a = R(Integer(70700)); repr(a)
'...4|2|3|0|3|0|0'
>>> b = R(-Integer(70700)); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
```

Again, note that it's not possible to read off the precision from the representation in this mode.

print_pos controls whether the digits can be negative.

```
sage: S = Zp(5, print_mode='bars', print_pos=False); b = S(-70700);␣
↪repr(b)
'...-1|0|2|2|-1|2|0|0'
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='bars', print_pos=False); b = S(-
↪Integer(70700)); repr(b)
'...-1|0|2|2|-1|2|0|0'
```

print_max_terms limits the number of digits that are printed.

```
sage: T = Zp(5, print_max_terms=4); T(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_max_terms=Integer(4)); T(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

print_sep controls the separation character.

```
sage: U = Zp(5, print_mode='bars', print_sep=']['); a = U(70700); repr(a)
'...4][2][3][0][3][0][0'
```

```
>>> from sage.all import *
>>> U = Zp(Integer(5), print_mode='bars', print_sep=']['); a =␣
↪U(Integer(70700)); repr(a)
'...4][2][3][0][3][0][0'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'dots' (or equivalently `True`) or 'bigoh'. The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: repr(Zp(5, 2, print_mode='bars', show_prec=True)(6))
'...1|1'
sage: repr(Zp(5, 2, print_mode='bars', show_prec=False)(6))
'1|1'
```

```
>>> from sage.all import *
>>> repr(Zp(Integer(5), Integer(2), print_mode='bars', show_
↪prec=True)(Integer(6)))
'...1|1'
>>> repr(Zp(Integer(5), Integer(2), print_mode='bars', show_
↪prec=False)(Integer(6)))
'1|1'
```

name and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

EXAMPLES:

We allow non-prime p, but only if `check=False`. Note that some features will not work.

```
sage: K = Zp(15, check=False); a = K(999); a
9 + 6*15 + 4*15^2 + O(15^20)
```

```
>>> from sage.all import *
>>> K = Zp(Integer(15), check=False); a = K(Integer(999)); a
9 + 6*15 + 4*15^2 + O(15^20)
```

We create rings with various parameters:

```
sage: Zp(7)
7-adic Ring with capped relative precision 20
```

(continues on next page)

(continued from previous page)

```
sage: Zp(9)
Traceback (most recent call last):
...
ValueError: p must be prime
sage: Zp(17, 5)
17-adic Ring with capped relative precision 5
sage: Zp(17, 5)(-1)
16 + 16*17 + 16*17^2 + 16*17^3 + 16*17^4 + O(17^5)
```

```
>>> from sage.all import *
>>> Zp(Integer(7))
7-adic Ring with capped relative precision 20
>>> Zp(Integer(9))
Traceback (most recent call last):
...
ValueError: p must be prime
>>> Zp(Integer(17), Integer(5))
17-adic Ring with capped relative precision 5
>>> Zp(Integer(17), Integer(5))(-Integer(1))
16 + 16*17 + 16*17^2 + 16*17^3 + 16*17^4 + O(17^5)
```

It works even with a fairly huge cap:

```
sage: Zp(next_prime(10^50), 100000)
10000000000000000000000000000000000000000000000151-adic Ring
 with capped relative precision 100000
```

```
>>> from sage.all import *
>>> Zp(next_prime(Integer(10)**Integer(50)), Integer(100000))
10000000000000000000000000000000000000000000000151-adic Ring
 with capped relative precision 100000
```

We create each type of ring:

```
sage: Zp(7, 20, 'capped-rel')
7-adic Ring with capped relative precision 20
sage: Zp(7, 20, 'fixed-mod')
7-adic Ring of fixed modulus 7^20
sage: Zp(7, 20, 'capped-abs')
7-adic Ring with capped absolute precision 20
```

```
>>> from sage.all import *
>>> Zp(Integer(7), Integer(20), 'capped-rel')
7-adic Ring with capped relative precision 20
>>> Zp(Integer(7), Integer(20), 'fixed-mod')
7-adic Ring of fixed modulus 7^20
>>> Zp(Integer(7), Integer(20), 'capped-abs')
7-adic Ring with capped absolute precision 20
```

We create a capped relative ring with each print mode:

```
sage: k = Zp(7, 8, print_mode='series'); k
7-adic Ring with capped relative precision 8
sage: k(7*(19))
5*7 + 2*7^2 + O(7^9)
sage: k(7*(-19))
2*7 + 4*7^2 + 6*7^3 + 6*7^4 + 6*7^5 + 6*7^6 + 6*7^7 + 6*7^8 + O(7^9)
```

```
>>> from sage.all import *
>>> k = Zp(Integer(7), Integer(8), print_mode='series'); k
7-adic Ring with capped relative precision 8
>>> k(Integer(7)*(Integer(19)))
5*7 + 2*7^2 + O(7^9)
>>> k(Integer(7)*(-Integer(19)))
2*7 + 4*7^2 + 6*7^3 + 6*7^4 + 6*7^5 + 6*7^6 + 6*7^7 + 6*7^8 + O(7^9)
```

```
sage: k = Zp(7, print_mode='val-unit'); k
7-adic Ring with capped relative precision 20
sage: k(7*(19))
7 * 19 + O(7^21)
sage: k(7*(-19))
7 * 79792266297611982 + O(7^21)
```

```
>>> from sage.all import *
>>> k = Zp(Integer(7), print_mode='val-unit'); k
7-adic Ring with capped relative precision 20
>>> k(Integer(7)*(Integer(19)))
7 * 19 + O(7^21)
>>> k(Integer(7)*(-Integer(19)))
7 * 79792266297611982 + O(7^21)
```

```
sage: k = Zp(7, print_mode='terse'); k
7-adic Ring with capped relative precision 20
sage: k(7*(19))
133 + O(7^21)
sage: k(7*(-19))
558545864083283874 + O(7^21)
```

```
>>> from sage.all import *
>>> k = Zp(Integer(7), print_mode='terse'); k
7-adic Ring with capped relative precision 20
>>> k(Integer(7)*(Integer(19)))
133 + O(7^21)
>>> k(Integer(7)*(-Integer(19)))
558545864083283874 + O(7^21)
```

Note that p-adic rings are cached (via weak references):

```
sage: a = Zp(7); b = Zp(7)
sage: a is b
True
```

```
>>> from sage.all import *
>>> a = Zp(Integer(7)); b = Zp(Integer(7))
>>> a is b
True
```

We create some elements in various rings:

```
sage: R = Zp(5); a = R(4); a
4 + O(5^20)
sage: S = Zp(5, 10, type = 'capped-abs'); b = S(2); b
2 + O(5^10)
sage: a + b
1 + 5 + O(5^10)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5)); a = R(Integer(4)); a
4 + O(5^20)
>>> S = Zp(Integer(5), Integer(10), type = 'capped-abs'); b = S(Integer(2)); b
2 + O(5^10)
>>> a + b
1 + 5 + O(5^10)
```

sage.rings.padics.factory.**pAdicExtension = <sage.rings.padics.factory.pAdicExtension_class object>**

A class for creating extensions of p-adic rings and fields.

EXAMPLES:

```
sage: R = Zp(5,3)
sage: S.<x> = ZZ[]
sage: W.<w> = pAdicExtension(R, x^4 - 15); W
5-adic Eisenstein Extension Ring in w defined by x^4 - 15
sage: W.precision_cap()
12
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(3))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> W = pAdicExtension(R, x**Integer(4) - Integer(15), names=('w',)); (w,) = W._
↪first_ngens(1); W
5-adic Eisenstein Extension Ring in w defined by x^4 - 15
>>> W.precision_cap()
12
```

sage.rings.padics.factory.**QpCR**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create capped relative p-adic fields.

Same functionality as *Qp*. See documentation for *Qp* for a description of the input parameters.

EXAMPLES:

```
sage: QpCR(5, 40)
5-adic Field with capped relative precision 40
```

```
>>> from sage.all import *
>>> QpCR(Integer(5), Integer(40))
5-adic Field with capped relative precision 40
```

sage.rings.padics.factory.**QpER**(*p*, *prec=None*, *halt=None*, *secure=False*, **args*, ***kwds*)

A shortcut function to create relaxed p-adic fields.

See *ZpER()* for more information about this model of precision.

EXAMPLES:

```
sage: R = QpER(2); R
2-adic Field handled with relaxed arithmetics
```

```
>>> from sage.all import *
>>> R = QpER(Integer(2)); R
2-adic Field handled with relaxed arithmetics
```

sage.rings.padics.factory.**QpFP**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create floating point p-adic fields.

Same functionality as *Qp*. See documentation for *Qp* for a description of the input parameters.

EXAMPLES:

```
sage: QpFP(5, 40)
5-adic Field with floating precision 40
```

```
>>> from sage.all import *
>>> QpFP(Integer(5), Integer(40))
5-adic Field with floating precision 40
```

sage.rings.padics.factory.**QpLC**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create p-adic fields with lattice precision.

See *ZpLC()* for more information about this model of precision.

EXAMPLES:

```
sage: R = QpLC(2)
sage: R
2-adic Field with lattice-cap precision
```

```
>>> from sage.all import *
>>> R = QpLC(Integer(2))
>>> R
2-adic Field with lattice-cap precision
```

sage.rings.padics.factory.**QpLF**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create p-adic fields with lattice precision.

See *ZpLC()* for more information about this model of precision.

EXAMPLES:

```
sage: R = QpLF(2)
sage: R
2-adic Field with lattice-float precision
```

```
>>> from sage.all import *
>>> R = QpLF(Integer(2))
>>> R
2-adic Field with lattice-float precision
```

class sage.rings.padics.factory.**Qp_class**

Bases: UniqueFactory

A creation function for p-adic fields.

INPUT:

- p – integer; the p in $\mathbf{Q}_p$
- prec – integer (default: 20); the precision cap of the field. In the lattice capped case, prec can either be a pair (relative_cap, absolute_cap) or an integer (understood at relative cap). In the relaxed case, prec can be either a pair (default_prec, halting_prec) or an integer (understood at default precision). Except in the floating point case, individual elements keep track of their own precision. See TYPES and PRECISION below.
- type – string (default: 'capped-rel'); valid types are 'capped-rel', 'floating-point', 'lattice-cap', 'lattice-float'. See TYPES and PRECISION below.
- print_mode – string (default: None); valid modes are 'series', 'val-unit', 'terse', 'digits', and 'bars'. See PRINTING below.
- names – string or tuple (defaults to a string representation of p); what to use whenever p is printed
- ram_name – string; another way to specify the name. For consistency with the Qq and Zq and extension functions.
- print_pos – boolean (default: None); whether to only use positive integers in the representations of elements. See PRINTING below.
- print_sep – string (default: None); the separator character used in the 'bars' mode. See PRINTING below.
- print_alphabet – tuple (default: None); the encoding into digits for use in the 'digits' mode. See PRINTING below.
- print_max_terms – integer (default: None); the maximum number of terms shown. See PRINTING below.
- show_prec – boolean or a string (default: None); specify how the precision is printed. See PRINTING below.
- check – boolean (default: True); whether to check if p is prime. Non-prime input may cause seg-faults (but can also be useful for base n expansions for example).
- label – string (default: None); used for lattice precision to create parents with different lattices

OUTPUT: the corresponding p-adic field

TYPES AND PRECISION:

There are two main types of precision for a p-adic element. The first is relative precision, which gives the number of known p-adic digits:

```
sage: R = Qp(5, 20, 'capped-rel', 'series'); a = R(675); a
2*5^2 + 5^4 + O(5^22)
sage: a.precision_relative()
20
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), Integer(20), 'capped-rel', 'series'); a = R(Integer(675));␣
↪a
2*5^2 + 5^4 + O(5^22)
>>> a.precision_relative()
20
```

The second type of precision is absolute precision, which gives the power of p that this element is defined modulo:

```
sage: a.precision_absolute()
22
```

```
>>> from sage.all import *
>>> a.precision_absolute()
22
```

There are several types of p-adic fields, depending on the methods used for tracking precision. Namely, we have:

- capped relative fields (`type='capped-rel'`)
- capped absolute fields (`type='capped-abs'`)
- fixed modulus fields (`type='fixed-mod'`)
- floating point fields (`type='floating-point'`)
- lattice precision fields (`type='lattice-cap'` or `type='lattice-float'`)
- exact fields with relaxed arithmetics (`type='relaxed'`)

In the capped relative case, the relative precision of an element is restricted to be at most a certain value, specified at the creation of the field. Individual elements also store their own precision, so the effect of various arithmetic operations on precision is tracked. When you cast an exact element into a capped relative field, it truncates it to the precision cap of the field.

```
sage: R = Qp(5, 5, 'capped-rel', 'series'); a = R(4006); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(4025); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
sage: a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), Integer(5), 'capped-rel', 'series'); a = R(Integer(4006));␣
↪a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(4025)); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
>>> a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

In the floating point case, elements do not track their precision, but the relative precision of elements is truncated during arithmetic to the precision cap of the field.

In the lattice case, precision on elements is tracked by a global lattice that is updated after every operation, yielding better precision behavior at the cost of higher memory and runtime usage. We refer to the documentation of the function *ZpLC()* for a small demonstration of the capabilities of this precision model.

Finally, the model for relaxed p-adics is quite different from any of the other types. In addition to storing a finite approximation, one also stores a method for increasing the precision. A quite interesting feature with relaxed p-adics is the possibility to create (in some cases) self-referent numbers, that are numbers whose n-th digit is defined by the previous ones. We refer to the documentation of the function *ZpER* for a small demonstration of the capabilities of this precision model.

PRINTING:

There are many different ways to print p-adic elements. The way elements of a given field print is controlled by options passed in at the creation of the field. There are five basic printing modes (series, val-unit, terse, digits and bars), as well as various options that either hide some information in the print representation or sometimes make print representations more compact. Note that the printing options affect whether different p-adic fields are considered equal.

1. **series**: elements are displayed as series in p.

```
sage: R = Qp(5, print_mode='series'); a = R(70700); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
sage: b = R(-70700); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='series'); a = R(Integer(70700)); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
>>> b = R(-Integer(70700)); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

print_pos controls whether negatives can be used in the coefficients of powers of p.

```
sage: S = Qp(5, print_mode='series', print_pos=False); a = S(70700); a
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
sage: b = S(-70700); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='series', print_pos=False); a =␣
↪S(Integer(70700)); a
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
>>> b = S(-Integer(70700)); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

print_max_terms limits the number of terms that appear.

```
sage: T = Qp(5, print_mode='series', print_max_terms=4); b = R(-70700);␣
↪repr(b)
'2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)'
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='series', print_max_terms=Integer(4));␣
↪b = R(-Integer(70700)); repr(b)
'2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)'
```

names affects how the prime is printed.

```
sage: U.<p> = Qp(5); p
p + O(p^21)
```

```
>>> from sage.all import *
>>> U = Qp(Integer(5), names=('p',)); (p,) = U._first_ngens(1); p
p + O(p^21)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`) or 'bigoh'. The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: Qp(5)(6)
1 + 5 + O(5^20)
sage: Qp(5, show_prec='none')(6)
1 + 5

sage: QpFP(5)(6)
1 + 5
```

```
>>> from sage.all import *
>>> Qp(Integer(5))(Integer(6))
1 + 5 + O(5^20)
>>> Qp(Integer(5), show_prec='none')(Integer(6))
1 + 5

>>> QpFP(Integer(5))(Integer(6))
1 + 5
```

print_sep and *print_alphabet* have no effect in series mode.

Note that print options affect equality:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

2. **val-unit**: elements are displayed as p^k*u:

```
sage: R = Qp(5, print_mode='val-unit'); a = R(70700); a
5^2 * 2828 + O(5^22)
sage: b = R(-707/5); b
5^-1 * 95367431639918 + O(5^19)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='val-unit'); a = R(Integer(70700)); a
5^2 * 2828 + O(5^22)
>>> b = R(-Integer(707)/Integer(5)); b
5^-1 * 95367431639918 + O(5^19)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Qp(5, print_mode='val-unit', print_pos=False); b = S(-70700); b
5^2 * (-2828) + O(5^22)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='val-unit', print_pos=False); b = S(-
↪Integer(70700)); b
5^2 * (-2828) + O(5^22)
```

names affects how the prime is printed.

```
sage: T = Qp(5, print_mode='val-unit', names='pi'); a = T(70700); a
pi^2 * 2828 + O(pi^22)
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='val-unit', names='pi'); a =␣
↪T(Integer(70700)); a
pi^2 * 2828 + O(pi^22)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`) or 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: Qp(5, print_mode='val-unit', show_prec=False)(30)
5 * 6
```

```
>>> from sage.all import *
>>> Qp(Integer(5), print_mode='val-unit', show_prec=False)(Integer(30))
5 * 6
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

3. **terse**: elements are displayed as an integer in base 10 or the quotient of an integer by a power of p (still in base 10):

```
sage: R = Qp(5, print_mode='terse'); a = R(70700); a
70700 + O(5^22)
sage: b = R(-70700); b
2384185790944925 + O(5^22)
sage: c = R(-707/5); c
95367431639918/5 + O(5^19)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='terse'); a = R(Integer(70700)); a
70700 + O(5^22)
>>> b = R(-Integer(70700)); b
2384185790944925 + O(5^22)
>>> c = R(-Integer(707)/Integer(5)); c
95367431639918/5 + O(5^19)
```

The denominator, as of version 3.3, is always printed explicitly as a power of p, for predictability.

```
sage: d = R(707/5^2); d
707/5^2 + O(5^18)
```

```
>>> from sage.all import *
>>> d = R(Integer(707)/Integer(5)**Integer(2)); d
707/5^2 + O(5^18)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Qp(5, print_mode='terse', print_pos=False); b = S(-70700); b
-70700 + O(5^22)
sage: c = S(-707/5); c
-707/5 + O(5^19)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='terse', print_pos=False); b = S(-
↪Integer(70700)); b
-70700 + O(5^22)
>>> c = S(-Integer(707)/Integer(5)); c
-707/5 + O(5^19)
```

name affects how the name is printed.

```
sage: T.<unif> = Qp(5, print_mode='terse'); c = T(-707/5); c
95367431639918/unif + O(unif^19)
sage: d = T(-707/5^10); d
95367431639918/unif^10 + O(unif^10)
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='terse', names=('unif',)); (unif,) = T._
↪first_ngens(1); c = T(-Integer(707)/Integer(5)); c
95367431639918/unif + O(unif^19)
```

(continues on next page)

(continued from previous page)

```
>>> d = T(-Integer(707)/Integer(5)**Integer(10)); d
95367431639918/unif^10 + O(unif^10)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`) or 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: Qp(5, print_mode='terse', show_prec=False)(6)
6
```

```
>>> from sage.all import *
>>> Qp(Integer(5), print_mode='terse', show_prec=False)(Integer(6))
6
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

4. **digits**: elements are displayed as a string of base p digits

   Restriction: you can only use the digits printing mode for small primes. Namely, p must be less than the length of the alphabet tuple (default alphabet has length 62).

```
sage: R = Qp(5, print_mode='digits'); a = R(70700); repr(a)
'...0000000000000004230300'
sage: b = R(-70700); repr(b)
'...4444444444444440214200'
sage: c = R(-707/5); repr(c)
'...4444444444444443413.3'
sage: d = R(-707/5^2); repr(d)
'...4444444444444444341.33'
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='digits'); a = R(Integer(70700));␣
↪repr(a)
'...0000000000000004230300'
>>> b = R(-Integer(70700)); repr(b)
'...4444444444444440214200'
>>> c = R(-Integer(707)/Integer(5)); repr(c)
'...4444444444444443413.3'
>>> d = R(-Integer(707)/Integer(5)**Integer(2)); repr(d)
'...4444444444444444341.33'
```

Observe that the significant 0s are printed even if they are located in front of the number. On the contrary, unknown digits located after the comma appears as question marks. The precision can therefore be read in this mode as well. Here are more examples:

```
sage: p = 7
sage: K = Qp(p, prec=10, print_mode='digits')
sage: repr(K(1))
'...0000000001'
sage: repr(K(p^2))
'...000000000100'
sage: repr(K(p^-5))
'...00000.00001'
sage: repr(K(p^-20))
'...?.??????????0000000001'
```

```
>>> from sage.all import *
>>> p = Integer(7)
>>> K = Qp(p, prec=Integer(10), print_mode='digits')
>>> repr(K(Integer(1)))
'...0000000001'
>>> repr(K(p**Integer(2)))
'...000000000100'
>>> repr(K(p**-Integer(5)))
'...00000.00001'
>>> repr(K(p**-Integer(20)))
'...?.??????????0000000001'
```

print_max_terms limits the number of digits that are printed. Note that if the valuation of the element is very negative, more digits will be printed.

```
sage: S = Qp(5, print_max_terms=4); S(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
sage: S(-707/5^2)
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
sage: S(-707/5^6)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
sage: S(-707/5^6,absprec=-2)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
sage: S(-707/5^4)
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_max_terms=Integer(4)); S(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
>>> S(-Integer(707)/Integer(5)**Integer(2))
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
>>> S(-Integer(707)/Integer(5)**Integer(6))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
>>> S(-Integer(707)/Integer(5)**Integer(6),absprec=-Integer(2))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
>>> S(-Integer(707)/Integer(5)**Integer(4))
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

print_alphabet controls the symbols used to substitute for digits greater than 9.

Defaults to ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'):

```
sage: T = Qp(5, print_mode='digits', print_alphabet=('1','2','3','4','5
↪')); repr(T(-70700))
'...555555555555551325311'
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_mode='digits', print_alphabet=('1','2','3','4
↪','5')); repr(T(-Integer(70700)))
'...555555555555551325311'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'dots' (or equivalently `True`) or 'bigoh'. The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: repr(Zp(5, print_mode='digits', show_prec=True)(6))
'...00000000000000000011'

sage: repr(Zp(5, print_mode='digits', show_prec='bigoh')(6))
'11 + O(5^20)'
```

```
>>> from sage.all import *
>>> repr(Zp(Integer(5), print_mode='digits', show_prec=True)(Integer(6)))
'...00000000000000000011'

>>> repr(Zp(Integer(5), print_mode='digits', show_prec='bigoh
↪')(Integer(6)))
'11 + O(5^20)'
```

print_pos, *name* and *print_sep* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

5. **bars**: elements are displayed as a string of base p digits with separators:

```
sage: R = Qp(5, print_mode='bars'); a = R(70700); repr(a)
'...4|2|3|0|3|0|0'
sage: b = R(-70700); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
sage: d = R(-707/5^2); repr(d)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|3|4|1|.|3|3'
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), print_mode='bars'); a = R(Integer(70700)); repr(a)
'...4|2|3|0|3|0|0'
>>> b = R(-Integer(70700)); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
```

(continues on next page)

(continued from previous page)

```
>>> d = R(-Integer(707)/Integer(5)**Integer(2)); repr(d)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|3|4|1|.|3|3'
```

Again, note that it's not possible to read off the precision from the representation in this mode.

print_pos controls whether the digits can be negative.

```
sage: S = Qp(5, print_mode='bars',print_pos=False); b = S(-70700); repr(b)
'...-1|0|2|2|-1|2|0|0'
```

```
>>> from sage.all import *
>>> S = Qp(Integer(5), print_mode='bars',print_pos=False); b = S(-
↪Integer(70700)); repr(b)
'...-1|0|2|2|-1|2|0|0'
```

print_max_terms limits the number of digits that are printed. Note that if the valuation of the element is very negative, more digits will be printed.

```
sage: T = Qp(5, print_max_terms=4); T(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
sage: T(-707/5^2)
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
sage: T(-707/5^6)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
sage: T(-707/5^6,absprec=-2)
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
sage: T(-707/5^4)
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

```
>>> from sage.all import *
>>> T = Qp(Integer(5), print_max_terms=Integer(4)); T(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
>>> T(-Integer(707)/Integer(5)**Integer(2))
3*5^-2 + 3*5^-1 + 1 + 4*5 + ... + O(5^18)
>>> T(-Integer(707)/Integer(5)**Integer(6))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + ... + O(5^14)
>>> T(-Integer(707)/Integer(5)**Integer(6),absprec=-Integer(2))
3*5^-6 + 3*5^-5 + 5^-4 + 4*5^-3 + O(5^-2)
>>> T(-Integer(707)/Integer(5)**Integer(4))
3*5^-4 + 3*5^-3 + 5^-2 + 4*5^-1 + ... + O(5^16)
```

print_sep controls the separation character.

```
sage: U = Qp(5, print_mode='bars', print_sep=']['); a = U(70700); repr(a)
'...4][2][3][0][3][0][0'
```

```
>>> from sage.all import *
>>> U = Qp(Integer(5), print_mode='bars', print_sep=']['); a =␣
↪U(Integer(70700)); repr(a)
'...4][2][3][0][3][0][0'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'dots' (or equivalently `True`) or 'bigoh' The default is `False` for the `'floating-point'` type and `True` for

all other types.

```
sage: repr(Qp(5, print_mode='bars', show_prec='bigoh')(6))
'...1|1 + O(5^20)'
```

```
>>> from sage.all import *
>>> repr(Qp(Integer(5), print_mode='bars', show_prec='bigoh')(Integer(6)))
'...1|1 + O(5^20)'
```

name and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

EXAMPLES:

```
sage: K = Qp(15, check=False); a = K(999); a
9 + 6*15 + 4*15^2 + O(15^20)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(15), check=False); a = K(Integer(999)); a
9 + 6*15 + 4*15^2 + O(15^20)
```

create_key(*p*, *prec=None*, *type='capped-rel'*, *print_mode=None*, *names=None*, *ram_name=None*, *print_pos=None*, *print_sep=None*, *print_alphabet=None*, *print_max_terms=None*, *show_prec=None*, *check=True*, *label=None*)

Create a key from input parameters for `Qp`.

See the documentation for *Qp* for more information.

create_object(*version*, *key*)

Create an object using a given key.

See the documentation for *Qp* for more information.

`sage.rings.padics.factory.`**Qq**(*q*, *prec=None*, *type='capped-rel'*, *modulus=None*, *names=None*, *print_mode=None*, *ram_name=None*, *res_name=None*, *print_pos=None*, *print_sep=None*, *print_max_ram_terms=None*, *print_max_unram_terms=None*, *print_max_terse_terms=None*, *show_prec=None*, *check=True*, *implementation='FLINT'*)

Given a prime power $q = p^n$, return the unique unramified extension of $\mathbf{Q}_p$ of degree n.

INPUT:

- `q` – integer, list, tuple or `Factorization` object. If `q` is an integer, it is the prime power q in $\mathbf{Q}_q$. If `q` is a `Factorization` object, it is the factorization of the prime power q. As a tuple it is the pair `(p, n)`, and as a list it is a single element list `[(p, n)]`.
- `prec` – integer (default: 20); the precision cap of the field. Individual elements keep track of their own precision. See TYPES and PRECISION below.

- `type` – string (default: `'capped-rel'`); valid types are `'capped-rel'`, `'floating-point'`, `'lattice-cap'` and `'lattice-float'`. See TYPES and PRECISION below.
- `modulus` – polynomial (default: `None`); a polynomial defining an unramified extension of $\mathbf{Q}_p$. See MODULUS below.
- `names` – string or tuple (`None` is only allowed when $q = p$); the name of the generator, reducing to a generator of the residue field.
- `print_mode` – string (default: `None`); valid modes are `'series'`, `'val-unit'`, `'terse'`, and `'bars'`. See PRINTING below.
- `ram_name` – string (defaults to string representation of p if `None`). `ram_name` controls how the prime is printed. See PRINTING below.
- `res_name` – string (defaults to `None`, which corresponds to adding a `'0'` to the end of the name). Controls how elements of the residue field print.
- `print_pos` – boolean (default: `None`); whether to only use positive integers in the representations of elements. See PRINTING below.
- `print_sep` – string (default: `None`); the separator character used in the `'bars'` mode. See PRINTING below.
- `print_max_ram_terms` – integer (default: `None`) the maximum number of powers of p shown. See PRINTING below.
- `print_max_unram_terms` – integer (default: `None`); the maximum number of entries shown in a coefficient of p. See PRINTING below.
- `print_max_terse_terms` – integer (default: `None`); the maximum number of terms in the polynomial representation of an element (using `'terse'`). See PRINTING below.
- `show_prec` – boolean (default: `None`); whether to show the precision for elements. See PRINTING below.
- `check` – boolean (default: `True`); whether to check inputs

OUTPUT: the corresponding unramified p-adic field

TYPES AND PRECISION:

There are two types of precision for a p-adic element. The first is relative precision, which gives the number of known p-adic digits:

```
sage: R.<a> = Qq(25, 20, 'capped-rel', print_mode='series')
sage: b = 25*a; b
a*5^2 + O(5^22)
sage: b.precision_relative()
20
```

```
>>> from sage.all import *
>>> R = Qq(Integer(25), Integer(20), 'capped-rel', print_mode='series', names=('a
↪',)); (a,) = R._first_ngens(1)
>>> b = Integer(25)*a; b
a*5^2 + O(5^22)
>>> b.precision_relative()
20
```

The second type of precision is absolute precision, which gives the power of p that this element is defined modulo:

```
sage: b.precision_absolute()
22
```

```
>>> from sage.all import *
>>> b.precision_absolute()
22
```

There are two types of unramified p-adic fields: capped relative fields, floating point fields.

In the capped relative case, the relative precision of an element is restricted to be at most a certain value, specified at the creation of the field. Individual elements also store their own precision, so the effect of various arithmetic operations on precision is tracked. When you cast an exact element into a capped relative field, it truncates it to the precision cap of the field.

```
sage: R.<a> = Qq(9, 5, 'capped-rel', print_mode='series')
sage: b = (1+2*a)^4; b
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^5)
sage: c = R(3249); c
3^2 + 3^4 + 3^5 + 3^6 + O(3^7)
sage: b + c
2 + (2*a + 2)*3 + (2*a + 2)*3^2 + 3^4 + O(3^5)
```

```
>>> from sage.all import *
>>> R = Qq(Integer(9), Integer(5), 'capped-rel', print_mode='series', names=('a',
↪)); (a,) = R._first_ngens(1)
>>> b = (Integer(1)+Integer(2)*a)**Integer(4); b
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^5)
>>> c = R(Integer(3249)); c
3^2 + 3^4 + 3^5 + 3^6 + O(3^7)
>>> b + c
2 + (2*a + 2)*3 + (2*a + 2)*3^2 + 3^4 + O(3^5)
```

In the floating point case, elements do not track their precision, but the relative precision of elements is truncated during arithmetic to the precision cap of the field.

MODULUS:

The modulus needs to define an unramified extension of $\mathbf{Q}_p$: when it is reduced to a polynomial over $\mathbf{F}_p$ it should be irreducible.

The modulus can be given in a number of forms.

1. A **polynomial**.

   The base ring can be $\mathbf{Z}$, $\mathbf{Q}$, $\mathbf{Z}_p$, $\mathbf{Q}_p$, $\mathbf{F}_p$.

```
sage: P.<x> = ZZ[]
sage: R.<a> = Qq(27, modulus = x^3 + 2*x + 1); R.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
sage: P.<x> = QQ[]
sage: S.<a> = Qq(27, modulus = x^3 + 2*x + 1)
sage: P.<x> = Zp(3)[]
sage: T.<a> = Qq(27, modulus = x^3 + 2*x + 1)
sage: P.<x> = Qp(3)[]
sage: U.<a> = Qq(27, modulus = x^3 + 2*x + 1)
```

(continues on next page)

(continued from previous page)

```
sage: P.<x> = GF(3)[]
sage: V.<a> = Qq(27, modulus = x^3 + 2*x + 1)
```

```
>>> from sage.all import *
>>> P = ZZ['x']; (x,) = P._first_ngens(1)
>>> R = Qq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = R._first_ngens(1); R.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> S = Qq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = S._first_ngens(1)
>>> P = Zp(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> T = Qq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = T._first_ngens(1)
>>> P = Qp(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> U = Qq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = U._first_ngens(1)
>>> P = GF(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> V = Qq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = V._first_ngens(1)
```

Which form the modulus is given in has no effect on the unramified extension produced:

```
sage: R == S, S == T, T == U, U == V
(True, True, True, False)
```

```
>>> from sage.all import *
>>> R == S, S == T, T == U, U == V
(True, True, True, False)
```

unless the precision of the modulus differs. In the case of V, the modulus is only given to precision 1, so the resulting field has a precision cap of 1.

```
sage: V.precision_cap()
1
sage: U.precision_cap()
20
sage: P.<x> = Qp(3)[]
sage: modulus = x^3 + (2 + O(3^7))*x + (1 + O(3^10))
sage: modulus
(1 + O(3^20))*x^3 + (2 + O(3^7))*x + 1 + O(3^10)
sage: W.<a> = Qq(27, modulus = modulus); W.precision_cap()
7
```

```
>>> from sage.all import *
>>> V.precision_cap()
1
>>> U.precision_cap()
20
>>> P = Qp(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> modulus = x**Integer(3) + (Integer(2) + O(Integer(3)**Integer(7)))*x␣
↪+ (Integer(1) + O(Integer(3)**Integer(10)))
```

(continues on next page)

(continued from previous page)

```
>>> modulus
(1 + O(3^20))*x^3 + (2 + O(3^7))*x + 1 + O(3^10)
>>> W = Qq(Integer(27), modulus = modulus, names=('a',)); (a,) = W._first_
↪ngens(1); W.precision_cap()
7
```

2. The modulus can also be given as a **symbolic expression**.

```
sage: x = var('x')
sage: X.<a> = Qq(27, modulus = x^3 + 2*x + 1); X.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
sage: X == R
True
```

```
>>> from sage.all import *
>>> x = var('x')
>>> X = Qq(Integer(27), modulus = x**Integer(3) + Integer(2)*x + Integer(1),␣
↪names=('a',)); (a,) = X._first_ngens(1); X.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
>>> X == R
True
```

By default, the polynomial chosen is the standard lift of the generator chosen for $\mathbf{F}_q$.

```
sage: GF(125, 'a').modulus()
x^3 + 3*x + 3
sage: Y.<a> = Qq(125); Y.modulus()
(1 + O(5^20))*x^3 + O(5^20)*x^2 + (3 + O(5^20))*x + 3 + O(5^20)
```

```
>>> from sage.all import *
>>> GF(Integer(125), 'a').modulus()
x^3 + 3*x + 3
>>> Y = Qq(Integer(125), names=('a',)); (a,) = Y._first_ngens(1); Y.
↪modulus()
(1 + O(5^20))*x^3 + O(5^20)*x^2 + (3 + O(5^20))*x + 3 + O(5^20)
```

However, you can choose another polynomial if desired (as long as the reduction to $\mathbf{F}_p[x]$ is irreducible).

```
sage: P.<x> = ZZ[]
sage: Z.<a> = Qq(125, modulus = x^3 + 3*x^2 + x + 1); Z.modulus()
(1 + O(5^20))*x^3 + (3 + O(5^20))*x^2 + (1 + O(5^20))*x + 1 + O(5^20)
sage: Y == Z
False
```

```
>>> from sage.all import *
>>> P = ZZ['x']; (x,) = P._first_ngens(1)
>>> Z = Qq(Integer(125), modulus = x**Integer(3) +␣
↪Integer(3)*x**Integer(2) + x + Integer(1), names=('a',)); (a,) = Z._
↪first_ngens(1); Z.modulus()
(1 + O(5^20))*x^3 + (3 + O(5^20))*x^2 + (1 + O(5^20))*x + 1 + O(5^20)
```

(continues on next page)

(continued from previous page)

```
>>> Y == Z
False
```

PRINTING:

There are many different ways to print p-adic elements. The way elements of a given field print is controlled by options passed in at the creation of the field. There are four basic printing modes (`'series'`, `'val-unit'`, `'terse'` and `'bars'`; `'digits'` is not available), as well as various options that either hide some information in the print representation or sometimes make print representations more compact. Note that the printing options affect whether different p-adic fields are considered equal.

1. **series**: elements are displayed as series in p.

```
sage: R.<a> = Qq(9, 20, 'capped-rel', print_mode='series'); (1+2*a)^4
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^20)
sage: -3*(1+2*a)^4
3 + a*3^2 + 3^3 + (2*a + 2)*3^4 + (2*a + 2)*3^5 + (2*a + 2)*3^6 + (2*a + 2)*3^
↪7
 + (2*a + 2)*3^8 + (2*a + 2)*3^9 + (2*a + 2)*3^10 + (2*a + 2)*3^11
 + (2*a + 2)*3^12 + (2*a + 2)*3^13 + (2*a + 2)*3^14 + (2*a + 2)*3^15
 + (2*a + 2)*3^16 + (2*a + 2)*3^17 + (2*a + 2)*3^18 + (2*a + 2)*3^19
 + (2*a + 2)*3^20 + O(3^21)
sage: ~(3*a+18)
(a + 2)*3^-1 + 1 + 2*3 + (a + 1)*3^2 + 3^3 + 2*3^4 + (a + 1)*3^5 + 3^6 + 2*3^7
 + (a + 1)*3^8 + 3^9 + 2*3^10 + (a + 1)*3^11 + 3^12 + 2*3^13 + (a + 1)*3^14
 + 3^15 + 2*3^16 + (a + 1)*3^17 + 3^18 + O(3^19)
```

```
>>> from sage.all import *
>>> R = Qq(Integer(9), Integer(20), 'capped-rel', print_mode='series', names=(
↪'a',)); (a,) = R._first_ngens(1); (Integer(1)+Integer(2)*a)**Integer(4)
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^20)
>>> -Integer(3)*(Integer(1)+Integer(2)*a)**Integer(4)
3 + a*3^2 + 3^3 + (2*a + 2)*3^4 + (2*a + 2)*3^5 + (2*a + 2)*3^6 + (2*a + 2)*3^
↪7
 + (2*a + 2)*3^8 + (2*a + 2)*3^9 + (2*a + 2)*3^10 + (2*a + 2)*3^11
 + (2*a + 2)*3^12 + (2*a + 2)*3^13 + (2*a + 2)*3^14 + (2*a + 2)*3^15
 + (2*a + 2)*3^16 + (2*a + 2)*3^17 + (2*a + 2)*3^18 + (2*a + 2)*3^19
 + (2*a + 2)*3^20 + O(3^21)
>>> ~(Integer(3)*a+Integer(18))
(a + 2)*3^-1 + 1 + 2*3 + (a + 1)*3^2 + 3^3 + 2*3^4 + (a + 1)*3^5 + 3^6 + 2*3^7
 + (a + 1)*3^8 + 3^9 + 2*3^10 + (a + 1)*3^11 + 3^12 + 2*3^13 + (a + 1)*3^14
 + 3^15 + 2*3^16 + (a + 1)*3^17 + 3^18 + O(3^19)
```

print_pos controls whether negatives can be used in the coefficients of powers of p.

```
sage: S.<b> = Qq(9, print_mode='series', print_pos=False); (1+2*b)^4
-1 - b*3 - 3^2 + (b + 1)*3^3 + O(3^20)
sage: -3*(1+2*b)^4
3 + b*3^2 + 3^3 + (-b - 1)*3^4 + O(3^21)
```

```
>>> from sage.all import *
>>> S = Qq(Integer(9), print_mode='series', print_pos=False, names=('b',
```

(continues on next page)

(continued from previous page)

```
↪)); (b,) = S._first_ngens(1); (Integer(1)+Integer(2)*b)**Integer(4)
-1 - b*3 - 3^2 + (b + 1)*3^3 + O(3^20)
>>> -Integer(3)*(Integer(1)+Integer(2)*b)**Integer(4)
3 + b*3^2 + 3^3 + (-b - 1)*3^4 + O(3^21)
```

ram_name controls how the prime is printed.

```
sage: T.<d> = Qq(9, print_mode='series', ram_name='p'); 3*(1+2*d)^4
2*p + (2*d + 2)*p^2 + (2*d + 1)*p^3 + O(p^21)
```

```
>>> from sage.all import *
>>> T = Qq(Integer(9), print_mode='series', ram_name='p', names=('d',));␣
↪(d,) = T._first_ngens(1);␣
↪Integer(3)*(Integer(1)+Integer(2)*d)**Integer(4)
2*p + (2*d + 2)*p^2 + (2*d + 1)*p^3 + O(p^21)
```

print_max_ram_terms limits the number of powers of p that appear.

```
sage: U.<e> = Qq(9, print_mode='series', print_max_ram_terms=4); repr(-
↪3*(1+2*e)^4)
'3 + e*3^2 + 3^3 + (2*e + 2)*3^4 + ... + O(3^21)'
```

```
>>> from sage.all import *
>>> U = Qq(Integer(9), print_mode='series', print_max_ram_
↪terms=Integer(4), names=('e',)); (e,) = U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 + e*3^2 + 3^3 + (2*e + 2)*3^4 + ... + O(3^21)'
```

print_max_unram_terms limits the number of terms that appear in a coefficient of a power of p.

```
sage: V.<f> = Qq(128, prec = 8, print_mode='series'); repr((1+f)^9)
'(f^3 + 1) + (f^5 + f^4 + f^3 + f^2)*2 + (f^6 + f^5 + f^4 + f + 1)*2^2 +␣
↪(f^5 + f^4 + f^2 + f + 1)*2^3 + (f^6 + f^5 + f^4 + f^3 + f^2 + f + 1)*2^
↪4 + (f^5 + f^4)*2^5 + (f^6 + f^5 + f^4 + f^3 + f + 1)*2^6 + (f + 1)*2^7␣
↪+ O(2^8)'
sage: V.<f> = Qq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 3); repr((1+f)^9)
'(f^3 + 1) + (f^5 + f^4 + ... + f^2)*2 + (f^6 + f^5 + ... + 1)*2^2 + (f^5␣
↪+ f^4 + ... + 1)*2^3 + (f^6 + f^5 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^
↪6 + f^5 + ... + 1)*2^6 + (f + 1)*2^7 + O(2^8)'
sage: V.<f> = Qq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 2); repr((1+f)^9)
'(f^3 + 1) + (f^5 + ... + f^2)*2 + (f^6 + ... + 1)*2^2 + (f^5 + ... +␣
↪1)*2^3 + (f^6 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^6 + ... + 1)*2^6 +␣
↪(f + 1)*2^7 + O(2^8)'
sage: V.<f> = Qq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 1); repr((1+f)^9)
'(f^3 + ...) + (f^5 + ...)*2 + (f^6 + ...)*2^2 + (f^5 + ...)*2^3 + (f^6 +␣
↪...)*2^4 + (f^5 + ...)*2^5 + (f^6 + ...)*2^6 + (f + ...)*2^7 + O(2^8)'
sage: V.<f> = Qq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 0); repr((1+f)^9 - 1 - f^3)
'(...)*2 + (...)*2^2 + (...)*2^3 + (...)*2^4 + (...)*2^5 + (...)*2^6 + (..
```

(continues on next page)

(continued from previous page)

```
↪.)*2^7 + O(2^8)'
```

```
>>> from sage.all import *
>>> V = Qq(Integer(128), prec = Integer(8), print_mode='series', names=('f
↪',)); (f,) = V._first_ngens(1); repr((Integer(1)+f)**Integer(9))
'(f^3 + 1) + (f^5 + f^4 + f^3 + f^2)*2 + (f^6 + f^5 + f^4 + f + 1)*2^2 +␣
↪(f^5 + f^4 + f^2 + f + 1)*2^3 + (f^6 + f^5 + f^4 + f^3 + f^2 + f + 1)*2^
↪4 + (f^5 + f^4)*2^5 + (f^6 + f^5 + f^4 + f^3 + f + 1)*2^6 + (f + 1)*2^7␣
↪+ O(2^8)'
>>> V = Qq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(3), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9))
'(f^3 + 1) + (f^5 + f^4 + ... + f^2)*2 + (f^6 + f^5 + ... + 1)*2^2 + (f^5␣
↪+ f^4 + ... + 1)*2^3 + (f^6 + f^5 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^
↪6 + f^5 + ... + 1)*2^6 + (f + 1)*2^7 + O(2^8)'
>>> V = Qq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(2), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9))
'(f^3 + 1) + (f^5 + ... + f^2)*2 + (f^6 + ... + 1)*2^2 + (f^5 + ... +␣
↪1)*2^3 + (f^6 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^6 + ... + 1)*2^6 +␣
↪(f + 1)*2^7 + O(2^8)'
>>> V = Qq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(1), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9))
'(f^3 + ...) + (f^5 + ...)*2 + (f^6 + ...)*2^2 + (f^5 + ...)*2^3 + (f^6 +␣
↪...)*2^4 + (f^5 + ...)*2^5 + (f^6 + ...)*2^6 + (f + ...)*2^7 + O(2^8)'
>>> V = Qq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(0), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9) - Integer(1) - f**Integer(3))
'(...)*2 + (...)*2^2 + (...)*2^3 + (...)*2^4 + (...)*2^5 + (...)*2^6 + (..
↪.)*2^7 + O(2^8)'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: U.<e> = Qq(9, 2, show_prec=False); repr(-3*(1+2*e)^4)
'3 + e*3^2'
```

```
>>> from sage.all import *
>>> U = Qq(Integer(9), Integer(2), show_prec=False, names=('e',)); (e,) =␣
↪U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 + e*3^2'
```

print_sep and *print_max_terse_terms* have no effect.

Note that print options affect equality:

```
sage: R == S, R == T, R == U, R == V, S == T, S == U, S == V, T == U, T␣
↪== V, U == V
(False, False, False, False, False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, R == V, S == T, S == U, S == V, T == U, T ==␣
↪V, U == V
(False, False, False, False, False, False, False, False, False, False)
```

2. **val-unit**: elements are displayed as $p^k u$:

```
sage: R.<a> = Qq(9, 7, print_mode='val-unit'); b = (1+3*a)^9 - 1; b
3^3 * (15 + 64*a) + O(3^7)
sage: ~b
3^-3 * (41 + a) + O(3)
```

```
>>> from sage.all import *
>>> R = Qq(Integer(9), Integer(7), print_mode='val-unit', names=('a',)); (a,)␣
↪= R._first_ngens(1); b = (Integer(1)+Integer(3)*a)**Integer(9) - Integer(1);
↪ b
3^3 * (15 + 64*a) + O(3^7)
>>> ~b
3^-3 * (41 + a) + O(3)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S.<a> = Qq(9, 7, print_mode='val-unit', print_pos=False)
sage: b = (1+3*a)^9 - 1; b
3^3 * (15 - 17*a) + O(3^7)
sage: ~b
3^-3 * (-40 + a) + O(3)
```

```
>>> from sage.all import *
>>> S = Qq(Integer(9), Integer(7), print_mode='val-unit', print_pos=False,
↪ names=('a',)); (a,) = S._first_ngens(1)
>>> b = (Integer(1)+Integer(3)*a)**Integer(9) - Integer(1); b
3^3 * (15 - 17*a) + O(3^7)
>>> ~b
3^-3 * (-40 + a) + O(3)
```

ram_name affects how the prime is printed.

```
sage: A.<x> = Qp(next_prime(10^6), print_mode='val-unit')[]
sage: T.<a> = Qq(next_prime(10^6)^3, 4, print_mode='val-unit', ram_name='p
↪',
....:            modulus=x^3+385831*x^2+106556*x+321036)
sage: b = ~(next_prime(10^6)^2*(a^2 + a - 4)); b
p^-2 * (503009563508519137754940 + 704413692798200940253892*a
        + 968097057817740999537581*a^2) + O(p^2)
sage: b * (a^2 + a - 4)
p^-2 * 1 + O(p^2)
```

```
>>> from sage.all import *
>>> A = Qp(next_prime(Integer(10)**Integer(6)), print_mode='val-unit')['x
↪']; (x,) = A._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> T = Qq(next_prime(Integer(10)**Integer(6))**Integer(3), Integer(4),␣
↪print_mode='val-unit', ram_name='p',
...            modulus=x**Integer(3)+Integer(385831)*x**Integer(2)+Inte-
ger(106556)*x+Integer(321036), names=('a',)); (a,) = T._first_ngens(1)
>>> b = ~(next_prime(Integer(10)**Integer(6))**Integer(2)*(a**Integer(2)␣
↪+ a - Integer(4))); b
p^-2 * (503009563508519137754940 + 704413692798200940253892*a
         + 96809705781774099953758l*a^2) + O(p^2)
>>> b * (a**Integer(2) + a - Integer(4))
p^-2 * 1 + O(p^2)
```

print_max_terse_terms controls how many terms of the polynomial appear in the unit part.

```
sage: U.<a> = Qq(17^4, 6, print_mode='val-unit', print_max_terse_terms=3)
sage: b = ~(17*(a^3-a+14)); b
17^-1 * (22110411 + 11317400*a + 20656972*a^2 + ...) + O(17^5)
sage: b*17*(a^3-a+14)
1 + O(17^6)
```

```
>>> from sage.all import *
>>> U = Qq(Integer(17)**Integer(4), Integer(6), print_mode='val-unit',␣
↪print_max_terse_terms=Integer(3), names=('a',)); (a,) = U._first_
↪ngens(1)
>>> b = ~(Integer(17)*(a**Integer(3)-a+Integer(14))); b
17^-1 * (22110411 + 11317400*a + 20656972*a^2 + ...) + O(17^5)
>>> b*Integer(17)*(a**Integer(3)-a+Integer(14))
1 + O(17^6)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: U.<e> = Qq(9, 2, print_mode='val-unit', show_prec=False); repr(-
↪3*(1+2*e)^4)
'3 * (1 + 3*e)'
```

```
>>> from sage.all import *
>>> U = Qq(Integer(9), Integer(2), print_mode='val-unit', show_prec=False,
↪ names=('e',)); (e,) = U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 * (1 + 3*e)'
```

print_sep, *print_max_ram_terms* and *print_max_unram_terms* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

3. **terse**: elements are displayed as a polynomial of degree less than the degree of the extension.

```
sage: R.<a> = Qq(125, print_mode='terse')
sage: (a+5)^177
68210977979428 + 90313850704069*a + 73948093055069*a^2 + O(5^20)
sage: (a/5+1)^177
68210977979428/5^177 + 90313850704069/5^177*a + 73948093055069/5^177*a^2 +␣
↪O(5^-157)
```

```
>>> from sage.all import *
>>> R = Qq(Integer(125), print_mode='terse', names=('a',)); (a,) = R._first_
↪ngens(1)
>>> (a+Integer(5))**Integer(177)
68210977979428 + 90313850704069*a + 73948093055069*a^2 + O(5^20)
>>> (a/Integer(5)+Integer(1))**Integer(177)
68210977979428/5^177 + 90313850704069/5^177*a + 73948093055069/5^177*a^2 +␣
↪O(5^-157)
```

As of version 3.3, if coefficients of the polynomial are non-integral, they are always printed with an explicit power of p in the denominator.

```
sage: 5*a + a^2/25
5*a + 1/5^2*a^2 + O(5^18)
```

```
>>> from sage.all import *
>>> Integer(5)*a + a**Integer(2)/Integer(25)
5*a + 1/5^2*a^2 + O(5^18)
```

print_pos controls whether to use a balanced representation or not.

```
sage: (a-5)^6
22864 + 95367431627998*a + 8349*a^2 + O(5^20)
sage: S.<a> = Qq(125, print_mode='terse', print_pos=False); b = (a-5)^6; b
22864 - 12627*a + 8349*a^2 + O(5^20)
sage: (a - 1/5)^6
-20624/5^6 + 18369/5^5*a + 1353/5^3*a^2 + O(5^14)
```

```
>>> from sage.all import *
>>> (a-Integer(5))**Integer(6)
22864 + 95367431627998*a + 8349*a^2 + O(5^20)
>>> S = Qq(Integer(125), print_mode='terse', print_pos=False, names=('a',
↪)); (a,) = S._first_ngens(1); b = (a-Integer(5))**Integer(6); b
22864 - 12627*a + 8349*a^2 + O(5^20)
>>> (a - Integer(1)/Integer(5))**Integer(6)
-20624/5^6 + 18369/5^5*a + 1353/5^3*a^2 + O(5^14)
```

ram_name affects how the prime is printed.

```
sage: T.<a> = Qq(125, print_mode='terse', ram_name='p'); (a - 1/5)^6
95367431620001/p^6 + 18369/p^5*a + 1353/p^3*a^2 + O(p^14)
```

```
>>> from sage.all import *
>>> T = Qq(Integer(125), print_mode='terse', ram_name='p', names=('a',));␣
```

(continues on next page)

(continued from previous page)

```
↪(a,) = T._first_ngens(1); (a - Integer(1)/Integer(5))**Integer(6)
95367431620001/p^6 + 18369/p^5*a + 1353/p^3*a^2 + O(p^14)
```

print_max_terse_terms controls how many terms of the polynomial are shown.

```
sage: U.<a> = Qq(625, print_mode='terse', print_max_terse_terms=2); (a-1/
↪5)^6
106251/5^6 + 49994/5^5*a + ... + O(5^14)
```

```
>>> from sage.all import *
>>> U = Qq(Integer(625), print_mode='terse', print_max_terse_
↪terms=Integer(2), names=('a',)); (a,) = U._first_ngens(1); (a-
↪Integer(1)/Integer(5))**Integer(6)
106251/5^6 + 49994/5^5*a + ... + O(5^14)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: U.<e> = Qq(9, 2, print_mode='terse', show_prec=False)
sage: repr(-3*(1+2*e)^4)
'3 + 9*e'
```

```
>>> from sage.all import *
>>> U = Qq(Integer(9), Integer(2), print_mode='terse', show_prec=False,␣
↪names=('e',)); (e,) = U._first_ngens(1)
>>> repr(-Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 + 9*e'
```

print_sep, *print_max_ram_terms* and *print_max_unram_terms* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

4. **digits**: This print mode is not available when the residue field is not prime.

   It might make sense to have a dictionary for small fields, but this isn't implemented.

5. **bars**: elements are displayed in a similar fashion to series, but more compactly.

```
sage: R.<a> = Qq(125); (a+5)^6
(4*a^2 + 3*a + 4) + (3*a^2 + 2*a)*5 + (a^2 + a + 1)*5^2 + (3*a + 2)*5^3
 + (3*a^2 + a + 3)*5^4 + (2*a^2 + 3*a + 2)*5^5 + O(5^20)
sage: R.<a> = Qq(125, print_mode='bars', prec=8); repr((a+5)^6)
'...[2, 3, 2]|[3, 1, 3]|[2, 3]|[1, 1, 1]|[0, 2, 3]|[4, 3, 4]'
sage: repr((a-5)^6)
'...[0, 4]|[1, 4]|[2, 0, 2]|[1, 4, 3]|[2, 3, 1]|[4, 4, 3]|[2, 4, 4]|[4, 3, 4]'
```

```
>>> from sage.all import *
>>> R = Qq(Integer(125), names=('a',)); (a,) = R._first_ngens(1);␣
↪(a+Integer(5))**Integer(6)
(4*a^2 + 3*a + 4) + (3*a^2 + 2*a)*5 + (a^2 + a + 1)*5^2 + (3*a + 2)*5^3
 + (3*a^2 + a + 3)*5^4 + (2*a^2 + 3*a + 2)*5^5 + O(5^20)
>>> R = Qq(Integer(125), print_mode='bars', prec=Integer(8), names=('a',));␣
↪(a,) = R._first_ngens(1); repr((a+Integer(5))**Integer(6))
'...[2, 3, 2]|[3, 1, 3]|[2, 3]|[1, 1, 1]|[0, 2, 3]|[4, 3, 4]'
>>> repr((a-Integer(5))**Integer(6))
'...[0, 4]|[1, 4]|[2, 0, 2]|[1, 4, 3]|[2, 3, 1]|[4, 4, 3]|[2, 4, 4]|[4, 3, 4]'
```

Note that elements with negative valuation are shown with a decimal point at valuation 0.

```
sage: repr((a+1/5)^6)
'...[3]|[4, 1, 3]|.|[1, 2, 3]|[3, 3]|[0, 0, 3]|[0, 1]|[0, 1]|[1]'
sage: repr((a+1/5)^2)
'...[0, 0, 1]|.|[0, 2]|[1]'
```

```
>>> from sage.all import *
>>> repr((a+Integer(1)/Integer(5))**Integer(6))
'...[3]|[4, 1, 3]|.|[1, 2, 3]|[3, 3]|[0, 0, 3]|[0, 1]|[0, 1]|[1]'
>>> repr((a+Integer(1)/Integer(5))**Integer(2))
'...[0, 0, 1]|.|[0, 2]|[1]'
```

If not enough precision is known, `'?'` is used instead.

```
sage: repr((a+R(1/5,relprec=3))^7)
'...|.|?|?|?|?|[0, 1, 1]|[0, 2]|[1]'
```

```
>>> from sage.all import *
>>> repr((a+R(Integer(1)/Integer(5),relprec=Integer(3)))**Integer(7))
'...|.|?|?|?|?|[0, 1, 1]|[0, 2]|[1]'
```

Note that it's not possible to read off the precision from the representation in this mode.

```
sage: b = a + 3; repr(b)
'...[3, 1]'
sage: c = a + R(3, 4); repr(c)
'...[3, 1]'
sage: b.precision_absolute()
8
sage: c.precision_absolute()
4
```

```
>>> from sage.all import *
>>> b = a + Integer(3); repr(b)
'...[3, 1]'
>>> c = a + R(Integer(3), Integer(4)); repr(c)
'...[3, 1]'
>>> b.precision_absolute()
8
```

(continues on next page)

(continued from previous page)

```
>>> c.precision_absolute()
4
```

print_pos controls whether the digits can be negative.

```
sage: S.<a> = Qq(125, print_mode='bars', print_pos=False); repr((a-5)^6)
'...[1, -1, 1]|[2, 1, -2]|[2, 0, -2]|[-2, -1, 2]|[0, 0, -1]|[-2]|[-1, -2,␣
↪-1]'
sage: repr((a-1/5)^6)
'...[0, 1, 2]|[-1, 1, 1]|.|[-2, -1, -1]|[2, 2, 1]|[0, 0, -2]|[0, -1]|[0, -
↪1]|[1]'
```

```
>>> from sage.all import *
>>> S = Qq(Integer(125), print_mode='bars', print_pos=False, names=('a',
↪)); (a,) = S._first_ngens(1); repr((a-Integer(5))**Integer(6))
'...[1, -1, 1]|[2, 1, -2]|[2, 0, -2]|[-2, -1, 2]|[0, 0, -1]|[-2]|[-1, -2,␣
↪-1]'
>>> repr((a-Integer(1)/Integer(5))**Integer(6))
'...[0, 1, 2]|[-1, 1, 1]|.|[-2, -1, -1]|[2, 2, 1]|[0, 0, -2]|[0, -1]|[0, -
↪1]|[1]'
```

print_max_ram_terms controls the maximum number of "digits" shown. Note that this puts a cap on the relative precision, not the absolute precision.

```
sage: T.<a> = Qq(125, print_max_ram_terms=3, print_pos=False); (a-5)^6
(-a^2 - 2*a - 1) - 2*5 - a^2*5^2 + ... + O(5^20)
sage: 5*(a-5)^6 + 50
(-a^2 - 2*a - 1)*5 - a^2*5^3 + (2*a^2 - a - 2)*5^4 + ... + O(5^21)
```

```
>>> from sage.all import *
>>> T = Qq(Integer(125), print_max_ram_terms=Integer(3), print_pos=False,␣
↪names=('a',)); (a,) = T._first_ngens(1); (a-Integer(5))**Integer(6)
(-a^2 - 2*a - 1) - 2*5 - a^2*5^2 + ... + O(5^20)
>>> Integer(5)*(a-Integer(5))**Integer(6) + Integer(50)
(-a^2 - 2*a - 1)*5 - a^2*5^3 + (2*a^2 - a - 2)*5^4 + ... + O(5^21)
```

print_sep controls the separating character (`'|'` by default).

```
sage: U.<a> = Qq(625, print_mode='bars', print_sep=''); b = (a+5)^6;␣
↪repr(b)
'...[0, 1][4, 0, 2][3, 2, 2, 3][4, 2, 2, 4][0, 3][1, 1, 3][3, 1, 4, 1]'
```

```
>>> from sage.all import *
>>> U = Qq(Integer(625), print_mode='bars', print_sep='', names=('a',));␣
↪(a,) = U._first_ngens(1); b = (a+Integer(5))**Integer(6); repr(b)
'...[0, 1][4, 0, 2][3, 2, 2, 3][4, 2, 2, 4][0, 3][1, 1, 3][3, 1, 4, 1]'
```

print_max_unram_terms controls how many terms are shown in each "digit":

```
sage: with local_print_mode(U, {'max_unram_terms': 3}): repr(b)
'...[0, 1][4,..., 0, 2][3,..., 2, 3][4,..., 2, 4][0, 3][1,..., 1, 3][3,...
↪, 4, 1]'
```

(continues on next page)

(continued from previous page)

```
sage: with local_print_mode(U, {'max_unram_terms': 2}): repr(b)
'...[0, 1][4,..., 2][3,..., 3][4,..., 4][0, 3][1,..., 3][3,..., 1]'
sage: with local_print_mode(U, {'max_unram_terms': 1}): repr(b)
'...[..., 1][..., 2][..., 3][..., 4][..., 3][..., 3][..., 1]'
sage: with local_print_mode(U, {'max_unram_terms':0}): repr(b-75*a)
'...[...][...][...][...][][...][...]'
```

```
>>> from sage.all import *
>>> with local_print_mode(U, {'max_unram_terms': Integer(3)}): repr(b)
'...[0, 1][4,..., 0, 2][3,..., 2, 3][4,..., 2, 4][0, 3][1,..., 1, 3][3,...
↪, 4, 1]'
>>> with local_print_mode(U, {'max_unram_terms': Integer(2)}): repr(b)
'...[0, 1][4,..., 2][3,..., 3][4,..., 4][0, 3][1,..., 3][3,..., 1]'
>>> with local_print_mode(U, {'max_unram_terms': Integer(1)}): repr(b)
'...[..., 1][..., 2][..., 3][..., 4][..., 3][..., 3][..., 1]'
>>> with local_print_mode(U, {'max_unram_terms':Integer(0)}): repr(b-
↪Integer(75)*a)
'...[...][...][...][...][][...][...]'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'dots' (or equivalently `True`) or 'bigoh' The default is `False` for the `'floating-point'` type and `True` for all other types.

```
sage: U.<e> = Qq(9, 2, print_mode='bars', show_prec=True); repr(-
↪3*(1+2*e)^4)
'...[0, 1]|[1]|[]'
```

```
>>> from sage.all import *
>>> U = Qq(Integer(9), Integer(2), print_mode='bars', show_prec=True,␣
↪names=('e',)); (e,) = U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'...[0, 1]|[1]|[]'
```

ram_name and *print_max_terse_terms* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

EXAMPLES:

Unlike for *Qp*, you can't create `Qq(N)` when `N` is not a prime power.

However, you can use `check=False` to pass in a pair in order to not have to factor. If you do so, you need to use names explicitly rather than the `R.<a>` syntax.

```
sage: p = next_prime(2^123)
sage: k = Qp(p)
```

(continues on next page)

(continued from previous page)

```
sage: R.<x> = k[]
sage: K = Qq([(p, 5)], modulus=x^5+x+4, names='a', ram_name='p',
....:        print_pos=False, check=False)
sage: K.0^5
(-a - 4) + O(p^20)
```

```
>>> from sage.all import *
>>> p = next_prime(Integer(2)**Integer(123))
>>> k = Qp(p)
>>> R = k['x']; (x,) = R._first_ngens(1)
>>> K = Qq([(p, Integer(5))], modulus=x**Integer(5)+x+Integer(4), names='a', ram_
↪name='p',
...        print_pos=False, check=False)
>>> K.gen(0)**Integer(5)
(-a - 4) + O(p^20)
```

In tests on `sage.math.washington.edu`, the creation of `K` as above took an average of 1.58ms, while:

```
sage: K = Qq(p^5, modulus=x^5+x+4, names='a', ram_name='p',
....:        print_pos=False, check=True)
```

```
>>> from sage.all import *
>>> K = Qq(p**Integer(5), modulus=x**Integer(5)+x+Integer(4), names='a', ram_name=
↪'p',
...        print_pos=False, check=True)
```

took an average of 24.5ms. Of course, with smaller primes these savings disappear.

`sage.rings.padics.factory.`**`QqCR`**(*q*, *prec=None*, **args*, ***kwds*)

A shortcut function to create capped relative unramified p-adic fields.

Same functionality as *Qq()*. See documentation for *Qq()* for a description of the input parameters.

EXAMPLES:

```
sage: R.<a> = QqCR(25, 40); R
5-adic Unramified Extension Field in a defined by x^2 + 4*x + 2
```

```
>>> from sage.all import *
>>> R = QqCR(Integer(25), Integer(40), names=('a',)); (a,) = R._first_ngens(1); R
5-adic Unramified Extension Field in a defined by x^2 + 4*x + 2
```

`sage.rings.padics.factory.`**`QqFP`**(*q*, *prec=None*, **args*, ***kwds*)

A shortcut function to create floating point unramified p-adic fields.

Same functionality as *Qq()*. See documentation for *Qq()* for a description of the input parameters.

EXAMPLES:

```
sage: R.<a> = QqFP(25, 40); R
5-adic Unramified Extension Field in a defined by x^2 + 4*x + 2
```

```
>>> from sage.all import *
>>> R = QqFP(Integer(25), Integer(40), names=('a',)); (a,) = R._first_ngens(1); R
5-adic Unramified Extension Field in a defined by x^2 + 4*x + 2
```

sage.rings.padics.factory.**ZpCA**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create capped absolute p-adic rings.

See documentation for *Zp* for a description of the input parameters.

EXAMPLES:

```
sage: ZpCA(5, 40)
5-adic Ring with capped absolute precision 40
```

```
>>> from sage.all import *
>>> ZpCA(Integer(5), Integer(40))
5-adic Ring with capped absolute precision 40
```

sage.rings.padics.factory.**ZpCR**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create capped relative p-adic rings.

Same functionality as *Zp*. See documentation for *Zp* for a description of the input parameters.

EXAMPLES:

```
sage: ZpCR(5, 40)
5-adic Ring with capped relative precision 40
```

```
>>> from sage.all import *
>>> ZpCR(Integer(5), Integer(40))
5-adic Ring with capped relative precision 40
```

sage.rings.padics.factory.**ZpER**(*p*, *prec=None*, *halt=None*, *secure=False*, **args*, ***kwds*)

A shortcut function to create relaxed p-adic rings.

INPUT:

- `prec` – integer (default: 20); the default precision
- `halt` – integer (default: twice `prec`); the halting precision
- `secure` – boolean (default: `False`); if `False`, consider indistinguishable elements at the working precision as equal. Otherwise, raise an error.

See documentation for *Zp* for a description of the other input parameters.

A SHORT INTRODUCTION TO RELAXED p-ADICS:

The model for relaxed p-adics is quite different from any of the other types of p-adics. In addition to storing a finite approximation, one also stores a method for increasing the precision.

Relaxed p-adic rings are created by the constructor *ZpER()*:

```
sage: R = ZpER(5, print_mode='digits'); R
5-adic Ring handled with relaxed arithmetics
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5), print_mode='digits'); R
5-adic Ring handled with relaxed arithmetics
```

The precision is not capped in R:

```
sage: R.precision_cap()
+Infinity
```

```
>>> from sage.all import *
>>> R.precision_cap()
+Infinity
```

However, a default precision is fixed. This is the precision at which the elements will be printed:

```
sage: R.default_prec()
20
```

```
>>> from sage.all import *
>>> R.default_prec()
20
```

A default halting precision is also set. It is the default absolute precision at which the elements will be compared. By default, it is twice the default precision:

```
sage: R.halting_prec()
40
```

```
>>> from sage.all import *
>>> R.halting_prec()
40
```

However, both the default precision and the halting precision can be customized at the creation of the parent as follows:

```
sage: S = ZpER(5, prec=10, halt=100)
sage: S.default_prec()
10
sage: S.halting_prec()
100
```

```
>>> from sage.all import *
>>> S = ZpER(Integer(5), prec=Integer(10), halt=Integer(100))
>>> S.default_prec()
10
>>> S.halting_prec()
100
```

One creates elements as usual:

```
sage: a = R(17/42); a
...00244200244200244201
```

(continues on next page)

(continued from previous page)

```
sage: R.random_element()  # random
...21013213133412431402
```

```
>>> from sage.all import *
>>> a = R(Integer(17)/Integer(42)); a
...00244200244200244201

>>> R.random_element()  # random
...21013213133412431402
```

Here we notice that 20 digits (that is the default precision) are printed. However, the computation model is designed in order to guarantee that more digits of a will be available on demand. This feature is reflected by the fact that, when we ask for the precision of a, the software answers $+\infty$:

```
sage: a.precision_absolute()
+Infinity
```

```
>>> from sage.all import *
>>> a.precision_absolute()
+Infinity
```

Asking for more digits is achieved by the methods `at_precision_absolute` and `at_precision_relative`:

```
sage: a.at_precision_absolute(30)
...?244200244200244200244200244201
```

```
>>> from sage.all import *
>>> a.at_precision_absolute(Integer(30))
...?244200244200244200244200244201
```

As a shortcut, one can use the bracket operator:

```
sage: a[:30]
...?244200244200244200244200244201
```

```
>>> from sage.all import *
>>> a[:Integer(30)]
...?244200244200244200244200244201
```

Of course, standard operations are supported:

```
sage: b = R(42/17)
sage: a + b
...03232011214322140002
sage: a - b
...42311334324023403400
sage: a * b
...00000000000000000001
sage: a / b
...12442142113021233401
```

(continues on next page)

(continued from previous page)

```
sage: sqrt(a)
...20042333114021142101
```

```
>>> from sage.all import *
>>> b = R(Integer(42)/Integer(17))
>>> a + b
...03232011214322140002
>>> a - b
...42311334324023403400
>>> a * b
...00000000000000000001
>>> a / b
...12442142113021233401
>>> sqrt(a)
...20042333114021142101
```

We observe again that only 20 digits are printed but, as before, more digits are available on demand:

```
sage: sqrt(a)[:30]
...?14244334212004233314021142101
```

```
>>> from sage.all import *
>>> sqrt(a)[:Integer(30)]
...?14244334212004233314021142101
```

Equality tests

Checking equalities between relaxed p-adics is a bit subtle and can sometimes be puzzling at first glance.

When the parent is created with `secure=False` (which is the default), elements are compared at the current precision, or at the default halting precision if it is higher:

```
sage: a == b
False

sage: a == sqrt(a)^2
True
sage: a == sqrt(a)^2 + 5^50
True
```

```
>>> from sage.all import *
>>> a == b
False

>>> a == sqrt(a)**Integer(2)
True
>>> a == sqrt(a)**Integer(2) + Integer(5)**Integer(50)
True
```

In the above example, the halting precision is 40; it is the reason why a congruence modulo 5^50 is considered as an equality. However, if both sides of the equalities have been previously computed with more digits, those digits are taken into account. Hence comparing two elements at different times can produce different results:

```
sage: aa = sqrt(a)^2 + 5^50
sage: a == aa
True
sage: a[:60]
...?244200244200244200244200244200244200244200244200244200244201
sage: aa[:60]
...?244200244300244200244200244200244200244200244200244200244201
sage: a == aa
False
```

```
>>> from sage.all import *
>>> aa = sqrt(a)**Integer(2) + Integer(5)**Integer(50)
>>> a == aa
True
>>> a[:Integer(60)]
...?244200244200244200244200244200244200244200244200244200244201
>>> aa[:Integer(60)]
...?244200244300244200244200244200244200244200244200244200244201
>>> a == aa
False
```

This annoying situation, where the output of `a == aa` may change depending on previous computations, cannot occur when the parent is created with `secure=True`. Indeed, in this case, if the equality cannot be decided, an error is raised:

```
sage: S = ZpER(5, secure=True)
sage: u = S.random_element()
sage: uu = u + 5^50
sage: u == uu
Traceback (most recent call last):
...
PrecisionError: unable to decide equality; try to bound precision

sage: u[:60] == uu
False
```

```
>>> from sage.all import *
>>> S = ZpER(Integer(5), secure=True)
>>> u = S.random_element()
>>> uu = u + Integer(5)**Integer(50)
>>> u == uu
Traceback (most recent call last):
...
PrecisionError: unable to decide equality; try to bound precision

>>> u[:Integer(60)] == uu
False
```

Self-referent numbers

A quite interesting feature with relaxed p-adics is the possibility to create (in some cases) self-referent numbers. Here is an example. We first declare a new variable as follows:

```
sage: x = R.unknown(); x
...?.0
```

```
>>> from sage.all import *
>>> x = R.unknown(); x
...?.0
```

We then use the method `set` to define x by writing down an equation it satisfies:

```
sage: x.set(1 + 5*x^2)
True
```

```
>>> from sage.all import *
>>> x.set(Integer(1) + Integer(5)*x**Integer(2))
True
```

The variable x now contains the unique solution of the equation $x = 1 + 5x^2$:

```
sage: x
...04222412141121000211
```

```
>>> from sage.all import *
>>> x
...04222412141121000211
```

This works because the n-th digit of the right hand size of the defining equation only involves the i-th digits of x with $i < n$ (this is due to the factor 5).

As a comparison, the following does not work:

```
sage: y = R.unknown()
sage: y.set(1 + 3*y^2)
True
sage: y
...?.0
sage: y[:20]
Traceback (most recent call last):
...
RecursionError: definition looks circular
```

```
>>> from sage.all import *
>>> y = R.unknown()
>>> y.set(Integer(1) + Integer(3)*y**Integer(2))
True
>>> y
...?.0
>>> y[:Integer(20)]
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
RecursionError: definition looks circular
```

Self-referent definitions also work with systems of equations:

```
sage: u = R.unknown()
sage: v = R.unknown()
sage: w = R.unknown()
sage: u.set(1 + 2*v + 3*w^2 + 5*u*v*w)
True
sage: v.set(2 + 4*w + sqrt(1 + 5*u + 10*v + 15*w))
True
sage: w.set(3 + 25*(u*v + v*w + u*w))
True
sage: u
...31203130103131131433
sage: v
...33441043031103114240
sage: w
...30212422041102444403
```

```
>>> from sage.all import *
>>> u = R.unknown()
>>> v = R.unknown()
>>> w = R.unknown()
>>> u.set(Integer(1) + Integer(2)*v + Integer(3)*w**Integer(2) + Integer(5)*u*v*w)
True
>>> v.set(Integer(2) + Integer(4)*w + sqrt(Integer(1) + Integer(5)*u +␣
↪Integer(10)*v + Integer(15)*w))
True
>>> w.set(Integer(3) + Integer(25)*(u*v + v*w + u*w))
True
>>> u
...31203130103131131433
>>> v
...33441043031103114240
>>> w
...30212422041102444403
```

sage.rings.padics.factory.**ZpFM**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create fixed modulus p-adic rings.

See documentation for *Zp* for a description of the input parameters.

EXAMPLES:

```
sage: ZpFM(5, 40)
5-adic Ring of fixed modulus 5^40
```

```
>>> from sage.all import *
>>> ZpFM(Integer(5), Integer(40))
5-adic Ring of fixed modulus 5^40
```

sage.rings.padics.factory.**ZpFP**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create floating point p-adic rings.

Same functionality as *Zp*. See documentation for *Zp* for a description of the input parameters.

EXAMPLES:

```
sage: ZpFP(5, 40)
5-adic Ring with floating precision 40
```

```
>>> from sage.all import *
>>> ZpFP(Integer(5), Integer(40))
5-adic Ring with floating precision 40
```

sage.rings.padics.factory.**ZpLC**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create p-adic rings with lattice precision (precision is encoded by a lattice in a large vector space and tracked using automatic differentiation).

See documentation for *Zp* for a description of the input parameters.

EXAMPLES:

Below is a small demo of the features by this model of precision:

```
sage: R = ZpLC(3, print_mode='terse')
sage: R
3-adic Ring with lattice-cap precision

sage: x = R(1,10)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(3), print_mode='terse')
>>> R
3-adic Ring with lattice-cap precision

>>> x = R(Integer(1),Integer(10))
```

Of course, when we multiply by 3, we gain one digit of absolute precision:

```
sage: 3*x
3 + O(3^11)
```

```
>>> from sage.all import *
>>> Integer(3)*x
3 + O(3^11)
```

The lattice precision machinery sees this even if we decompose the computation into several steps:

```
sage: y = x+x
sage: y
2 + O(3^10)
sage: x + y
3 + O(3^11)
```

```
>>> from sage.all import *
>>> y = x+x
>>> y
2 + O(3^10)
>>> x + y
3 + O(3^11)
```

The same works for the multiplication:

```
sage: z = x^2
sage: z
1 + O(3^10)
sage: x*z
1 + O(3^11)
```

```
>>> from sage.all import *
>>> z = x**Integer(2)
>>> z
1 + O(3^10)
>>> x*z
1 + O(3^11)
```

This can be more surprising when we are working with elements given at different precisions:

```
sage: R = ZpLC(2, print_mode='terse')
sage: x = R(1,10)
sage: y = R(1,5)
sage: z = x+y; z
2 + O(2^5)
sage: t = x-y; t
O(2^5)
sage: z+t  # observe that z+t = 2*x
2 + O(2^11)
sage: z-t  # observe that z-t = 2*y
2 + O(2^6)

sage: x = R(28888,15)
sage: y = R(204,10)
sage: z = x/y; z
242 + O(2^9)
sage: z*y  # which is x
28888 + O(2^15)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), print_mode='terse')
>>> x = R(Integer(1),Integer(10))
>>> y = R(Integer(1),Integer(5))
>>> z = x+y; z
2 + O(2^5)
>>> t = x-y; t
O(2^5)
>>> z+t  # observe that z+t = 2*x
2 + O(2^11)
```

(continues on next page)

(continued from previous page)

```
>>> z-t  # observe that z-t = 2*y
2 + O(2^6)

>>> x = R(Integer(28888),Integer(15))
>>> y = R(Integer(204),Integer(10))
>>> z = x/y; z
242 + O(2^9)
>>> z*y  # which is x
28888 + O(2^15)
```

The SOMOS sequence is the sequence defined by the recurrence:

$$u_n = \frac{u_{n-1}u_{n-3} + u_{n-2}^2}{u_{n-4}}$$

It is known for its numerical instability. On the one hand, one can show that if the initial values are invertible in $\mathbf{Z}_p$ and known at precision $O(p^N)$ then all the next terms of the SOMOS sequence will be known at the same precision as well. On the other hand, because of the division, when we unroll the recurrence, we loose a lot of precision. Observe:

```
sage: R = Zp(2, 30, print_mode='terse')
sage: a,b,c,d = R(1,15), R(1,15), R(1,15), R(3,15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
4 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
13 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
55 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
21975 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
6639 + O(2^13)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
7186 + O(2^13)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
569 + O(2^13)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
253 + O(2^13)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
4149 + O(2^13)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
2899 + O(2^12)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
3072 + O(2^12)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
349 + O(2^12)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
619 + O(2^12)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
243 + O(2^12)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
3 + O(2^2)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
```

(continues on next page)

(continued from previous page)

```
2 + O(2^2)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(2), Integer(30), print_mode='terse')
>>> a,b,c,d = R(Integer(1),Integer(15)), R(Integer(1),Integer(15)), R(Integer(1),
↪Integer(15)), R(Integer(3),Integer(15))
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
4 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
13 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
55 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
21975 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
6639 + O(2^13)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
7186 + O(2^13)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
569 + O(2^13)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
253 + O(2^13)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
4149 + O(2^13)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
2899 + O(2^12)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
3072 + O(2^12)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
349 + O(2^12)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
619 + O(2^12)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
243 + O(2^12)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
3 + O(2^2)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
2 + O(2^2)
```

If instead, we use the lattice precision, everything goes well:

```
sage: R = ZpLC(2, 30, print_mode='terse')
sage: a,b,c,d = R(1,15), R(1,15), R(1,15), R(3,15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
4 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
13 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
55 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
21975 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
```

(continues on next page)

(continued from previous page)

```
23023 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
31762 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
16953 + O(2^15)
sage: a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
16637 + O(2^15)

sage: for _ in range(100):
....:     a,b,c,d = b,c,d,(b*d+c*c)/a
sage: a
15519 + O(2^15)
sage: b
32042 + O(2^15)
sage: c
17769 + O(2^15)
sage: d
20949 + O(2^15)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), Integer(30), print_mode='terse')
>>> a,b,c,d = R(Integer(1),Integer(15)), R(Integer(1),Integer(15)), R(Integer(1),
↪Integer(15)), R(Integer(3),Integer(15))
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
4 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
13 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
55 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
21975 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
23023 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
31762 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
16953 + O(2^15)
>>> a,b,c,d = b,c,d,(b*d+c*c)/a; print(d)
16637 + O(2^15)

>>> for _ in range(Integer(100)):
...     a,b,c,d = b,c,d,(b*d+c*c)/a
>>> a
15519 + O(2^15)
>>> b
32042 + O(2^15)
>>> c
17769 + O(2^15)
>>> d
20949 + O(2^15)
```

ALGORITHM:

The precision is global. It is encoded by a lattice in a huge vector space whose dimension is the number of elements having this parent. Precision is tracked using automatic differentiation techniques (see [CRV2014] and [CRV2018]).

Concretely, this precision datum is an instance of the class *PrecisionLattice*. It is attached to the parent and is created at the same time as the parent. (It is actually a bit more subtle because two different parents may share the same instance; this happens for instance for a p-adic ring and its field of fractions.)

This precision datum is accessible through the method *precision()*:

```
sage: R = ZpLC(5, print_mode='terse')
sage: prec = R.precision()
sage: prec
Precision lattice on 0 objects
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(5), print_mode='terse')
>>> prec = R.precision()
>>> prec
Precision lattice on 0 objects
```

This instance knows about all elements of the parent. It is automatically updated when a new element (of this parent) is created:

```
sage: x = R(3513,10)
sage: prec
Precision lattice on 1 object
sage: y = R(176,5)
sage: prec
Precision lattice on 2 objects
sage: z = R.random_element()
sage: prec
Precision lattice on 3 objects
```

```
>>> from sage.all import *
>>> x = R(Integer(3513),Integer(10))
>>> prec
Precision lattice on 1 object
>>> y = R(Integer(176),Integer(5))
>>> prec
Precision lattice on 2 objects
>>> z = R.random_element()
>>> prec
Precision lattice on 3 objects
```

The method *tracked_elements()* provides the list of all tracked elements:

```
sage: prec.tracked_elements()
[3513 + O(5^10), 176 + O(5^5), ...]
```

```
>>> from sage.all import *
>>> prec.tracked_elements()
[3513 + O(5^10), 176 + O(5^5), ...]
```

Similarly, when a variable is collected by the garbage collector, the precision lattice is updated. Note however that the update might be delayed. We can force it with the method *del_elements()*:

```
sage: z = 0
sage: prec  # random output, could be 2 objects if the garbage collector is fast
Precision lattice on 3 objects
sage: prec.del_elements()
sage: prec
Precision lattice on 2 objects
```

```
>>> from sage.all import *
>>> z = Integer(0)
>>> prec  # random output, could be 2 objects if the garbage collector is fast
Precision lattice on 3 objects
>>> prec.del_elements()
>>> prec
Precision lattice on 2 objects
```

The method *precision_lattice()* returns (a matrix defining) the lattice that models the precision. Here we have:

```
sage: prec.precision_lattice()
[9765625       0]
[      0    3125]
```

```
>>> from sage.all import *
>>> prec.precision_lattice()
[9765625       0]
[      0    3125]
```

Observe that $5^10 = 9765625$ and $5^5 = 3125$. The above matrix then reflects the precision on x and y.

Now, observe how the precision lattice changes while performing computations:

```
sage: x, y = 3*x+2*y, 2*(x-y)
sage: prec.del_elements()
sage: prec.precision_lattice()
[    3125 48825000]
[       0 48828125]
```

```
>>> from sage.all import *
>>> x, y = Integer(3)*x+Integer(2)*y, Integer(2)*(x-y)
>>> prec.del_elements()
>>> prec.precision_lattice()
[    3125 48825000]
[       0 48828125]
```

The matrix we get is no longer diagonal, meaning that some digits of precision are diffused among the two new elements x and y. They nevertheless show up when we compute for instance $x + y$:

```
sage: x
1516 + O(5^5)
sage: y
424 + O(5^5)
```

(continues on next page)

(continued from previous page)

```
sage: x+y
17565 + O(5^11)
```

```
>>> from sage.all import *
>>> x
1516 + O(5^5)
>>> y
424 + O(5^5)
>>> x+y
17565 + O(5^11)
```

These diffused digits of precision (which are tracked but do not appear on the printing) allow to be always sharp on precision.

NOTE:

Each elementary operation requires significant manipulations on the precision lattice and therefore is costly. Precisely:

- The creation of a new element has a cost $O(n)$ where n is the number of tracked elements.
- The destruction of one element has a cost $O(m^2)$ where m is the distance between the destroyed element and the last one. Fortunately, it seems that m tends to be small in general (the dynamics of the list of tracked elements is rather close to that of a stack).

It is nevertheless still possible to manipulate several hundred variables (e.g. square matrices of size 5 or polynomials of degree 20).

The class *PrecisionLattice* provides several features for introspection, especially concerning timings. See *history()* and *timings()* for details.

See also

ZpLF()

sage.rings.padics.factory.**ZpLF**(*p*, *prec=None*, **args*, ***kwds*)

A shortcut function to create p-adic rings where precision is encoded by a module in a large vector space.

See documentation for *Zp* for a description of the input parameters.

Note

The precision is tracked using automatic differentiation techniques (see [CRV2018] and [CRV2014]). Floating point p-adic numbers are used for the computation of the differential (which is then not exact).

EXAMPLES:

```
sage: R = ZpLF(5, 40)
sage: R
5-adic Ring with lattice-float precision
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(5), Integer(40))
```

(continues on next page)

(continued from previous page)

```
>>> R
5-adic Ring with lattice-float precision
```

See also

ZpLC()

class sage.rings.padics.factory.**Zp_class**

Bases: UniqueFactory

A creation function for p-adic rings.

INPUT:

- p – integer; the p in $\mathbf{Z}_p$
- prec – integer (default: 20); the precision cap of the ring. In the lattice capped case, prec can either be a pair (relative_cap, absolute_cap) or an integer (understood as relative cap). In the relaxed case, prec can be either a pair (default_prec, halting_prec) or an integer (understood as default precision). Except for the fixed modulus and floating point cases, individual elements keep track of their own precision. See TYPES and PRECISION below.
- type – string (default: 'capped-rel'); valid types are 'capped-rel', 'capped-abs', 'fixed-mod', 'floating-point', 'lattice-cap', 'lattice-float', 'relaxed'. See TYPES and PRECISION below.
- print_mode – string (default: None); valid modes are 'series', 'val-unit', 'terse', 'digits', and 'bars'. See PRINTING below.
- names – string or tuple (defaults to a string representation of p); what to use whenever p is printed
- print_pos – boolean (default: None); whether to only use positive integers in the representations of elements. See PRINTING below.
- print_sep – string (default: None); the separator character used in the 'bars' mode. See PRINTING below.
- print_alphabet – tuple (default: None); the encoding into digits for use in the 'digits' mode. See PRINTING below.
- print_max_terms – integer (default: None); the maximum number of terms shown. See PRINTING below.
- show_prec – boolean (default: None) whether to show the precision for elements. See PRINTING below.
- check – boolean (default: True) whether to check if p is prime. Non-prime input may cause seg-faults (but can also be useful for base n expansions for example)
- label – string (default: None); used for lattice precision to create parents with different lattices

OUTPUT: the corresponding p-adic ring

TYPES AND PRECISION:

There are two main types of precision. The first is relative precision; it gives the number of known p-adic digits:

```
sage: R = Zp(5, 20, 'capped-rel', 'series'); a = R(675); a
2*5^2 + 5^4 + O(5^22)
```

(continues on next page)

(continued from previous page)

```
sage: a.precision_relative()
20
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(20), 'capped-rel', 'series'); a = R(Integer(675));␣
↪a
2*5^2 + 5^4 + O(5^22)
>>> a.precision_relative()
20
```

The second type of precision is absolute precision, which gives the power of p that this element is defined modulo:

```
sage: a.precision_absolute()
22
```

```
>>> from sage.all import *
>>> a.precision_absolute()
22
```

There are several types of p-adic rings, depending on the methods used for tracking precision. Namely, we have:

- capped relative rings (`type='capped-rel'`)
- capped absolute rings (`type='capped-abs'`)
- fixed modulus rings (`type='fixed-mod'`)
- floating point rings (`type='floating-point'`)
- lattice precision rings (`type='lattice-cap'` or `type='lattice-float'`)
- exact fields with relaxed arithmetics (`type='relaxed'`)

In the capped relative case, the relative precision of an element is restricted to be at most a certain value, specified at the creation of the field. Individual elements also store their own precision, so the effect of various arithmetic operations on precision is tracked. When you cast an exact element into a capped relative field, it truncates it to the precision cap of the field.

```
sage: R = Zp(5, 5, 'capped-rel', 'series'); a = R(4006); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(4025); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
sage: a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(5), 'capped-rel', 'series'); a = R(Integer(4006));␣
↪a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(4025)); b
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
>>> a + b
1 + 5 + 5^2 + 4*5^3 + 2*5^4 + O(5^5)
```

In the capped absolute type, instead of having a cap on the relative precision of an element there is instead a cap on the absolute precision. Elements still store their own precisions, and as with the capped relative case, exact elements

are truncated when cast into the ring.

```
sage: R = Zp(5, 5, 'capped-abs', 'series'); a = R(4005); a
5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(4025); b
5^2 + 2*5^3 + 5^4 + O(5^5)
sage: a * b
5^3 + 2*5^4 + O(5^5)
sage: (a * b) // 5^3
1 + 2*5 + O(5^2)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(5), 'capped-abs', 'series'); a = R(Integer(4005));␣
↪a
5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(4025)); b
5^2 + 2*5^3 + 5^4 + O(5^5)
>>> a * b
5^3 + 2*5^4 + O(5^5)
>>> (a * b) // Integer(5)**Integer(3)
1 + 2*5 + O(5^2)
```

The fixed modulus type is the leanest of the p-adic rings: it is basically just a wrapper around $\mathbf{Z}/p^n\mathbf{Z}$ providing a unified interface with the rest of the p-adics. This is the type you should use if your sole interest is speed. It does not track precision of elements.

```
sage: R = Zp(5,5,'fixed-mod','series'); a = R(4005); a
5 + 2*5^3 + 5^4
sage: a // 5
1 + 2*5^2 + 5^3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5),'fixed-mod','series'); a = R(Integer(4005)); a
5 + 2*5^3 + 5^4
>>> a // Integer(5)
1 + 2*5^2 + 5^3
```

The floating point case is similar to the fixed modulus type in that elements do not track their own precision. However, relative precision is truncated with each operation rather than absolute precision.

On the contrary, the lattice type tracks precision using lattices and automatic differentiation. It is rather slow but provides sharp (often optimal) results regarding precision. We refer to the documentation of the function *ZpLC()* for a small demonstration of the capabilities of this precision model.

Finally, the model for relaxed p-adics is quite different from any of the other types. In addition to storing a finite approximation, one also stores a method for increasing the precision. A quite interesting feature with relaxed p-adics is the possibility to create (in some cases) self-referent numbers, that are numbers whose n-th digit is defined by the previous ones. We refer to the documentation of the function *ZpER* for a small demonstration of the capabilities of this precision model.

PRINTING:

There are many different ways to print p-adic elements. The way elements of a given ring print is controlled by options passed in at the creation of the ring. There are five basic printing modes (`'series'`, `'val-unit'`, `'terse'`, `'digits'` and `'bars'`), as well as various options that either hide some information in the print representation or

sometimes make print representations more compact. Note that the printing options affect whether different p-adic fields are considered equal.

1. **series**: elements are displayed as series in p.

```
sage: R = Zp(5, print_mode='series'); a = R(70700); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
sage: b = R(-70700); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='series'); a = R(Integer(70700)); a
3*5^2 + 3*5^4 + 2*5^5 + 4*5^6 + O(5^22)
>>> b = R(-Integer(70700)); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11
 + 4*5^12 + 4*5^13 + 4*5^14 + 4*5^15 + 4*5^16 + 4*5^17 + 4*5^18
 + 4*5^19 + 4*5^20 + 4*5^21 + O(5^22)
```

print_pos controls whether negatives can be used in the coefficients of powers of p.

```
sage: S = Zp(5, print_mode='series', print_pos=False); a = S(70700); a
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
sage: b = S(-70700); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='series', print_pos=False); a =␣
↪S(Integer(70700)); a
-2*5^2 + 5^3 - 2*5^4 - 2*5^5 + 5^7 + O(5^22)
>>> b = S(-Integer(70700)); b
2*5^2 - 5^3 + 2*5^4 + 2*5^5 - 5^7 + O(5^22)
```

print_max_terms limits the number of terms that appear.

```
sage: T = Zp(5, print_mode='series', print_max_terms=4); b = R(-70700); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='series', print_max_terms=Integer(4));␣
↪b = R(-Integer(70700)); b
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

names affects how the prime is printed.

```
sage: U.<p> = Zp(5); p
p + O(p^21)
```

```
>>> from sage.all import *
>>> U = Zp(Integer(5), names=('p',)); (p,) = U._first_ngens(1); p
p + O(p^21)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: Zp(5, show_prec=False)(6)
1 + 5
```

```
>>> from sage.all import *
>>> Zp(Integer(5), show_prec=False)(Integer(6))
1 + 5
```

print_sep and *print_alphabet* have no effect.

Note that print options affect equality:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

2. **val-unit**: elements are displayed as $p^k u$:

```
sage: R = Zp(5, print_mode='val-unit'); a = R(70700); a
5^2 * 2828 + O(5^22)
sage: b = R(-707*5); b
5 * 95367431639918 + O(5^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='val-unit'); a = R(Integer(70700)); a
5^2 * 2828 + O(5^22)
>>> b = R(-Integer(707)*Integer(5)); b
5 * 95367431639918 + O(5^21)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Zp(5, print_mode='val-unit', print_pos=False); b = S(-70700); b
5^2 * (-2828) + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='val-unit', print_pos=False); b = S(-
↪Integer(70700)); b
5^2 * (-2828) + O(5^22)
```

names affects how the prime is printed.

```
sage: T = Zp(5, print_mode='val-unit', names='pi'); a = T(70700); a
pi^2 * 2828 + O(pi^22)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='val-unit', names='pi'); a =␣
```

(continues on next page)

(continued from previous page)

```
↪T(Integer(70700)); a
pi^2 * 2828 + O(pi^22)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: Zp(5, print_mode='val-unit', show_prec=False)(30)
5 * 6
```

```
>>> from sage.all import *
>>> Zp(Integer(5), print_mode='val-unit', show_prec=False)(Integer(30))
5 * 6
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

3. **terse**: elements are displayed as an integer in base 10:

```
sage: R = Zp(5, print_mode='terse'); a = R(70700); a
70700 + O(5^22)
sage: b = R(-70700); b
2384185790944925 + O(5^22)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='terse'); a = R(Integer(70700)); a
70700 + O(5^22)
>>> b = R(-Integer(70700)); b
2384185790944925 + O(5^22)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S = Zp(5, print_mode='terse', print_pos=False); b = S(-70700); b
-70700 + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='terse', print_pos=False); b = S(-
↪Integer(70700)); b
-70700 + O(5^22)
```

name affects how the name is printed. Note that this interacts with the choice of shorter string for denominators.

```
sage: T.<unif> = Zp(5, print_mode='terse'); c = T(-707); c
95367431639918 + O(unif^20)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='terse', names=('unif',)); (unif,) = T._
↪first_ngens(1); c = T(-Integer(707)); c
95367431639918 + O(unif^20)
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'bigoh' (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: Zp(5, print_mode='terse', show_prec=False)(30)
30
```

```
>>> from sage.all import *
>>> Zp(Integer(5), print_mode='terse', show_prec=False)(Integer(30))
30
```

print_max_terms, *print_sep* and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

4. **digits**: elements are displayed as a string of base p digits

   Restriction: you can only use the digits printing mode for small primes. Namely, p must be less than the length of the alphabet tuple (default alphabet has length 62).

```
sage: R = Zp(5, print_mode='digits'); a = R(70700); repr(a)
'...4230300'
sage: b = R(-70700); repr(b)
'...444444444444440214200'
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='digits'); a = R(Integer(70700));␣
↪repr(a)
'...4230300'
>>> b = R(-Integer(70700)); repr(b)
'...444444444444440214200'
```

Note that it's not possible to read off the precision from the representation in this mode.

print_max_terms limits the number of digits that are printed.

```
sage: S = Zp(5, print_max_terms=4); S(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_max_terms=Integer(4)); S(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

print_alphabet controls the symbols used to substitute for digits greater than 9. Defaults to ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'):

```
sage: T = Zp(5, print_mode='digits', print_alphabet=('1','2','3','4','5'))
sage: repr(T(-70700))
'...55555555555551325311'
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_mode='digits', print_alphabet=('1','2','3','4
↪','5'))
>>> repr(T(-Integer(70700)))
'...55555555555551325311'
```

show_prec determines how the precision is printed. It can be either `'none'` (or equivalently `False`), `'dots'` (or equivalently `True`) or `'bigoh'`. The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: repr(Zp(5, 2, print_mode='digits', show_prec=True)(6))
'...11'
sage: repr(Zp(5, 2, print_mode='digits', show_prec='bigoh')(6))
'11 + O(5^2)'
```

```
>>> from sage.all import *
>>> repr(Zp(Integer(5), Integer(2), print_mode='digits', show_
↪prec=True)(Integer(6)))
'...11'
>>> repr(Zp(Integer(5), Integer(2), print_mode='digits', show_prec='bigoh
↪')(Integer(6)))
'11 + O(5^2)'
```

print_pos, *name* and *print_sep* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, S == T
(False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, S == T
(False, False, False)
```

5. **bars**: elements are displayed as a string of base p digits with separators:

```
sage: R = Zp(5, print_mode='bars'); a = R(70700); repr(a)
'...4|2|3|0|3|0|0'
sage: b = R(-70700); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='bars'); a = R(Integer(70700)); repr(a)
'...4|2|3|0|3|0|0'
>>> b = R(-Integer(70700)); repr(b)
'...4|4|4|4|4|4|4|4|4|4|4|4|4|4|4|0|2|1|4|2|0|0'
```

Again, note that it's not possible to read off the precision from the representation in this mode.

print_pos controls whether the digits can be negative.

```
sage: S = Zp(5, print_mode='bars', print_pos=False); b = S(-70700);␣
↪repr(b)
'...-1|0|2|2|-1|2|0|0'
```

```
>>> from sage.all import *
>>> S = Zp(Integer(5), print_mode='bars', print_pos=False); b = S(-
↪Integer(70700)); repr(b)
'...-1|0|2|2|-1|2|0|0'
```

print_max_terms limits the number of digits that are printed.

```
sage: T = Zp(5, print_max_terms=4); T(-70700)
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

```
>>> from sage.all import *
>>> T = Zp(Integer(5), print_max_terms=Integer(4)); T(-Integer(70700))
2*5^2 + 4*5^3 + 5^4 + 2*5^5 + ... + O(5^22)
```

print_sep controls the separation character.

```
sage: U = Zp(5, print_mode='bars', print_sep=']['); a = U(70700); repr(a)
'...4][2][3][0][3][0][0'
```

```
>>> from sage.all import *
>>> U = Zp(Integer(5), print_mode='bars', print_sep=']['); a =␣
↪U(Integer(70700)); repr(a)
'...4][2][3][0][3][0][0'
```

show_prec determines how the precision is printed. It can be either 'none' (or equivalently `False`), 'dots' (or equivalently `True`) or 'bigoh'. The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: repr(Zp(5, 2, print_mode='bars', show_prec=True)(6))
'...1|1'
sage: repr(Zp(5, 2, print_mode='bars', show_prec=False)(6))
'1|1'
```

```
>>> from sage.all import *
>>> repr(Zp(Integer(5), Integer(2), print_mode='bars', show_
↪prec=True)(Integer(6)))
'...1|1'
>>> repr(Zp(Integer(5), Integer(2), print_mode='bars', show_
```

(continues on next page)

(continued from previous page)

```
↪prec=False)(Integer(6)))
'1|1'
```

name and *print_alphabet* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

EXAMPLES:

We allow non-prime p, but only if `check=False`. Note that some features will not work.

```
sage: K = Zp(15, check=False); a = K(999); a
9 + 6*15 + 4*15^2 + O(15^20)
```

```
>>> from sage.all import *
>>> K = Zp(Integer(15), check=False); a = K(Integer(999)); a
9 + 6*15 + 4*15^2 + O(15^20)
```

We create rings with various parameters:

```
sage: Zp(7)
7-adic Ring with capped relative precision 20
sage: Zp(9)
Traceback (most recent call last):
...
ValueError: p must be prime
sage: Zp(17, 5)
17-adic Ring with capped relative precision 5
sage: Zp(17, 5)(-1)
16 + 16*17 + 16*17^2 + 16*17^3 + 16*17^4 + O(17^5)
```

```
>>> from sage.all import *
>>> Zp(Integer(7))
7-adic Ring with capped relative precision 20
>>> Zp(Integer(9))
Traceback (most recent call last):
...
ValueError: p must be prime
>>> Zp(Integer(17), Integer(5))
17-adic Ring with capped relative precision 5
>>> Zp(Integer(17), Integer(5))(-Integer(1))
16 + 16*17 + 16*17^2 + 16*17^3 + 16*17^4 + O(17^5)
```

It works even with a fairly huge cap:

```
sage: Zp(next_prime(10^50), 100000)
10000000000000000000000000000000000000000000000151-adic Ring
 with capped relative precision 100000
```

```
>>> from sage.all import *
>>> Zp(next_prime(Integer(10)**Integer(50)), Integer(100000))
10000000000000000000000000000000000000000000000151-adic Ring
 with capped relative precision 100000
```

We create each type of ring:

```
sage: Zp(7, 20, 'capped-rel')
7-adic Ring with capped relative precision 20
sage: Zp(7, 20, 'fixed-mod')
7-adic Ring of fixed modulus 7^20
sage: Zp(7, 20, 'capped-abs')
7-adic Ring with capped absolute precision 20
```

```
>>> from sage.all import *
>>> Zp(Integer(7), Integer(20), 'capped-rel')
7-adic Ring with capped relative precision 20
>>> Zp(Integer(7), Integer(20), 'fixed-mod')
7-adic Ring of fixed modulus 7^20
>>> Zp(Integer(7), Integer(20), 'capped-abs')
7-adic Ring with capped absolute precision 20
```

We create a capped relative ring with each print mode:

```
sage: k = Zp(7, 8, print_mode='series'); k
7-adic Ring with capped relative precision 8
sage: k(7*(19))
5*7 + 2*7^2 + O(7^9)
sage: k(7*(-19))
2*7 + 4*7^2 + 6*7^3 + 6*7^4 + 6*7^5 + 6*7^6 + 6*7^7 + 6*7^8 + O(7^9)
```

```
>>> from sage.all import *
>>> k = Zp(Integer(7), Integer(8), print_mode='series'); k
7-adic Ring with capped relative precision 8
>>> k(Integer(7)*(Integer(19)))
5*7 + 2*7^2 + O(7^9)
>>> k(Integer(7)*(-Integer(19)))
2*7 + 4*7^2 + 6*7^3 + 6*7^4 + 6*7^5 + 6*7^6 + 6*7^7 + 6*7^8 + O(7^9)
```

```
sage: k = Zp(7, print_mode='val-unit'); k
7-adic Ring with capped relative precision 20
sage: k(7*(19))
7 * 19 + O(7^21)
sage: k(7*(-19))
7 * 79792266297611982 + O(7^21)
```

```
>>> from sage.all import *
>>> k = Zp(Integer(7), print_mode='val-unit'); k
```

(continues on next page)

(continued from previous page)

```
7-adic Ring with capped relative precision 20
>>> k(Integer(7)*(Integer(19)))
7 * 19 + O(7^21)
>>> k(Integer(7)*(-Integer(19)))
7 * 79792266297611982 + O(7^21)
```

```
sage: k = Zp(7, print_mode='terse'); k
7-adic Ring with capped relative precision 20
sage: k(7*(19))
133 + O(7^21)
sage: k(7*(-19))
558545864083283874 + O(7^21)
```

```
>>> from sage.all import *
>>> k = Zp(Integer(7), print_mode='terse'); k
7-adic Ring with capped relative precision 20
>>> k(Integer(7)*(Integer(19)))
133 + O(7^21)
>>> k(Integer(7)*(-Integer(19)))
558545864083283874 + O(7^21)
```

Note that p-adic rings are cached (via weak references):

```
sage: a = Zp(7); b = Zp(7)
sage: a is b
True
```

```
>>> from sage.all import *
>>> a = Zp(Integer(7)); b = Zp(Integer(7))
>>> a is b
True
```

We create some elements in various rings:

```
sage: R = Zp(5); a = R(4); a
4 + O(5^20)
sage: S = Zp(5, 10, type = 'capped-abs'); b = S(2); b
2 + O(5^10)
sage: a + b
1 + 5 + O(5^10)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5)); a = R(Integer(4)); a
4 + O(5^20)
>>> S = Zp(Integer(5), Integer(10), type = 'capped-abs'); b = S(Integer(2)); b
2 + O(5^10)
>>> a + b
1 + 5 + O(5^10)
```

create_key(*p*, *prec=None*, *type='capped-rel'*, *print_mode=None*, *names=None*, *ram_name=None*, *print_pos=None*, *print_sep=None*, *print_alphabet=None*, *print_max_terms=None*, *show_prec=None*, *check=True*, *label=None*)

Create a key from input parameters for `Zp`.

See the documentation for *Zp* for more information.

create_object(*version*, *key*)

Create an object using a given key.

See the documentation for *Zp* for more information.

sage.rings.padics.factory.**Zq**(*q*, *prec=None*, *type='capped-rel'*, *modulus=None*, *names=None*, *print_mode=None*, *ram_name=None*, *res_name=None*, *print_pos=None*, *print_sep=None*, *print_max_ram_terms=None*, *print_max_unram_terms=None*, *print_max_terse_terms=None*, *show_prec=None*, *check=True*, *implementation='FLINT'*)

Given a prime power $q = p^n$, return the unique unramified extension of $\mathbf{Z}_p$ of degree n.

INPUT:

- `q` – integer, list or tuple: the prime power in $\mathbf{Q}_q$. Or a `Factorization` object, single element list `[(p, n)]` where `p` is a prime and `n` a positive integer, or the pair `(p, n)`.
- `prec` – integer (default: 20); the precision cap of the field. Individual elements keep track of their own precision. See TYPES and PRECISION below.
- `type` – string (default: `'capped-rel'`); valid types are `'capped-abs'`, `'capped-rel'`, `'fixed-mod'`, and `'floating-point'`. See TYPES and PRECISION below.
- `modulus` – polynomial (default: `None`); a polynomial defining an unramified extension of $\mathbf{Z}_p$. See MODULUS below.
- `names` – string or tuple (`None` is only allowed when $q = p$); the name of the generator, reducing to a generator of the residue field
- `print_mode` – string (default: `None`); valid modes are `'series'`, `'val-unit'`, `'terse'`, and `'bars'`. See PRINTING below.
- `ram_name` – string (defaults to string representation of p if `None`); `ram_name` controls how the prime is printed. See PRINTING below.
- `res_name` – string (defaults to `None`, which corresponds to adding a `'0'` to the end of the name); controls how elements of the residue field print.
- `print_pos` – boolean (default: `None`); whether to only use positive integers in the representations of elements. See PRINTING below.
- `print_sep` – string (default: `None`); the separator character used in the `'bars'` mode. See PRINTING below.
- `print_max_ram_terms` – integer (default: `None`); the maximum number of powers of p shown. See PRINTING below.
- `print_max_unram_terms` – integer (default: `None`) the maximum number of entries shown in a coefficient of p. See PRINTING below.
- `print_max_terse_terms` – integer (default: `None`); the maximum number of terms in the polynomial representation of an element (using `'terse'`). See PRINTING below.
- `show_prec` – boolean (default: `None`); whether to show the precision for elements. See PRINTING below.
- `check` – boolean (default: `True`) whether to check inputs
- `implementation` – string (default: `'FLINT'`); which implementation to use. `'NTL'` is the other option.

OUTPUT: the corresponding unramified p-adic ring

TYPES AND PRECISION:

There are two types of precision for a p-adic element. The first is relative precision (default), which gives the number of known p-adic digits:

```
sage: R.<a> = Zq(25, 20, 'capped-rel', print_mode='series'); b = 25*a; b
a*5^2 + O(5^22)
sage: b.precision_relative()
20
```

```
>>> from sage.all import *
>>> R = Zq(Integer(25), Integer(20), 'capped-rel', print_mode='series', names=('a
↪',)); (a,) = R._first_ngens(1); b = Integer(25)*a; b
a*5^2 + O(5^22)
>>> b.precision_relative()
20
```

The second type of precision is absolute precision, which gives the power of p that this element is defined modulo:

```
sage: b.precision_absolute()
22
```

```
>>> from sage.all import *
>>> b.precision_absolute()
22
```

There are many types of p-adic rings: capped relative rings (`type='capped-rel'`), capped absolute rings (`type='capped-abs'`), fixed modulus rings (`type='fixed-mod'`), and floating point rings (`type='floating-point'`).

In the capped relative case, the relative precision of an element is restricted to be at most a certain value, specified at the creation of the field. Individual elements also store their own precision, so the effect of various arithmetic operations on precision is tracked. When you cast an exact element into a capped relative field, it truncates it to the precision cap of the field.

```
sage: R.<a> = Zq(9, 5, 'capped-rel', print_mode='series'); b = (1+2*a)^4; b
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^5)
sage: c = R(3249); c
3^2 + 3^4 + 3^5 + 3^6 + O(3^7)
sage: b + c
2 + (2*a + 2)*3 + (2*a + 2)*3^2 + 3^4 + O(3^5)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(9), Integer(5), 'capped-rel', print_mode='series', names=('a',
↪)); (a,) = R._first_ngens(1); b = (Integer(1)+Integer(2)*a)**Integer(4); b
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^5)
>>> c = R(Integer(3249)); c
3^2 + 3^4 + 3^5 + 3^6 + O(3^7)
>>> b + c
2 + (2*a + 2)*3 + (2*a + 2)*3^2 + 3^4 + O(3^5)
```

One can invert non-units: the result is in the fraction field.

```
sage: d = ~(3*b+c); d
2*3^-1 + (a + 1) + (a + 1)*3 + a*3^3 + O(3^4)
sage: d.parent()
3-adic Unramified Extension Field in a defined by x^2 + 2*x + 2
```

```
>>> from sage.all import *
>>> d = ~(Integer(3)*b+c); d
2*3^-1 + (a + 1) + (a + 1)*3 + a*3^3 + O(3^4)
>>> d.parent()
3-adic Unramified Extension Field in a defined by x^2 + 2*x + 2
```

The capped absolute case is the same as the capped relative case, except that the cap is on the absolute precision rather than the relative precision.

```
sage: R.<a> = Zq(9, 5, 'capped-abs', print_mode='series'); b = 3*(1+2*a)^4; b
2*3 + (2*a + 2)*3^2 + (2*a + 1)*3^3 + O(3^5)
sage: c = R(3249); c
3^2 + 3^4 + O(3^5)
sage: b*c
2*3^3 + (2*a + 2)*3^4 + O(3^5)
sage: b*c >> 1
2*3^2 + (2*a + 2)*3^3 + O(3^4)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(9), Integer(5), 'capped-abs', print_mode='series', names=('a',
↪)); (a,) = R._first_ngens(1); b =␣
↪Integer(3)*(Integer(1)+Integer(2)*a)**Integer(4); b
2*3 + (2*a + 2)*3^2 + (2*a + 1)*3^3 + O(3^5)
>>> c = R(Integer(3249)); c
3^2 + 3^4 + O(3^5)
>>> b*c
2*3^3 + (2*a + 2)*3^4 + O(3^5)
>>> b*c >> Integer(1)
2*3^2 + (2*a + 2)*3^3 + O(3^4)
```

The fixed modulus case is like the capped absolute, except that individual elements don't track their precision.

```
sage: R.<a> = Zq(9, 5, 'fixed-mod', print_mode='series'); b = 3*(1+2*a)^4; b
2*3 + (2*a + 2)*3^2 + (2*a + 1)*3^3
sage: c = R(3249); c
3^2 + 3^4
sage: b*c
2*3^3 + (2*a + 2)*3^4
sage: b*c >> 1
2*3^2 + (2*a + 2)*3^3
```

```
>>> from sage.all import *
>>> R = Zq(Integer(9), Integer(5), 'fixed-mod', print_mode='series', names=('a',
↪)); (a,) = R._first_ngens(1); b =␣
↪Integer(3)*(Integer(1)+Integer(2)*a)**Integer(4); b
2*3 + (2*a + 2)*3^2 + (2*a + 1)*3^3
>>> c = R(Integer(3249)); c
```

(continues on next page)

(continued from previous page)

```
3^2 + 3^4
>>> b*c
2*3^3 + (2*a + 2)*3^4
>>> b*c >> Integer(1)
2*3^2 + (2*a + 2)*3^3
```

The floating point case is similar to the fixed modulus type in that elements do not track their own precision. However, relative precision is truncated with each operation rather than absolute precision.

MODULUS:

The modulus needs to define an unramified extension of $\mathbf{Z}_p$: when it is reduced to a polynomial over $\mathbf{F}_p$ it should be irreducible.

The modulus can be given in a number of forms.

1. A **polynomial**.

   The base ring can be $\mathbf{Z}$, $\mathbf{Q}$, $\mathbf{Z}_p$, $\mathbf{F}_p$, or anything that can be converted to $\mathbf{Z}_p$.

```
sage: P.<x> = ZZ[]
sage: R.<a> = Zq(27, modulus = x^3 + 2*x + 1); R.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
sage: P.<x> = QQ[]
sage: S.<a> = Zq(27, modulus = x^3 + 2/7*x + 1)
sage: P.<x> = Zp(3)[]
sage: T.<a> = Zq(27, modulus = x^3 + 2*x + 1)
sage: P.<x> = Qp(3)[]
sage: U.<a> = Zq(27, modulus = x^3 + 2*x + 1)
sage: P.<x> = GF(3)[]
sage: V.<a> = Zq(27, modulus = x^3 + 2*x + 1)
```

```
>>> from sage.all import *
>>> P = ZZ['x']; (x,) = P._first_ngens(1)
>>> R = Zq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = R._first_ngens(1); R.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> S = Zq(Integer(27), modulus = x**Integer(3) + Integer(2)/Integer(7)*x␣
↪+ Integer(1), names=('a',)); (a,) = S._first_ngens(1)
>>> P = Zp(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> T = Zq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = T._first_ngens(1)
>>> P = Qp(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> U = Zq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = U._first_ngens(1)
>>> P = GF(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> V = Zq(Integer(27), modulus = x**Integer(3) + Integer(2)*x +␣
↪Integer(1), names=('a',)); (a,) = V._first_ngens(1)
```

   Which form the modulus is given in has no effect on the unramified extension produced:

```
sage: R == S, R == T, T == U, U == V
(False, True, True, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, T == U, U == V
(False, True, True, False)
```

unless the modulus is different, or the precision of the modulus differs. In the case of `V`, the modulus is only given to precision `1`, so the resulting field has a precision cap of `1`.

```
sage: V.precision_cap()
1
sage: U.precision_cap()
20
sage: P.<x> = Zp(3)[]
sage: modulus = x^3 + (2 + O(3^7))*x + (1 + O(3^10))
sage: modulus
(1 + O(3^20))*x^3 + (2 + O(3^7))*x + 1 + O(3^10)
sage: W.<a> = Zq(27, modulus = modulus); W.precision_cap()
7
```

```
>>> from sage.all import *
>>> V.precision_cap()
1
>>> U.precision_cap()
20
>>> P = Zp(Integer(3))['x']; (x,) = P._first_ngens(1)
>>> modulus = x**Integer(3) + (Integer(2) + O(Integer(3)**Integer(7)))*x␣
↪+ (Integer(1) + O(Integer(3)**Integer(10)))
>>> modulus
(1 + O(3^20))*x^3 + (2 + O(3^7))*x + 1 + O(3^10)
>>> W = Zq(Integer(27), modulus = modulus, names=('a',)); (a,) = W._first_
↪ngens(1); W.precision_cap()
7
```

2. The modulus can also be given as a **symbolic expression**.

```
sage: x = var('x')
sage: X.<a> = Zq(27, modulus = x^3 + 2*x + 1); X.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
sage: X == R
True
```

```
>>> from sage.all import *
>>> x = var('x')
>>> X = Zq(Integer(27), modulus = x**Integer(3) + Integer(2)*x + Integer(1),␣
↪names=('a',)); (a,) = X._first_ngens(1); X.modulus()
(1 + O(3^20))*x^3 + O(3^20)*x^2 + (2 + O(3^20))*x + 1 + O(3^20)
>>> X == R
True
```

By default, the polynomial chosen is the standard lift of the generator chosen for $\mathbf{F}_q$.

```
sage: GF(125, 'a').modulus()
x^3 + 3*x + 3
```

(continues on next page)

(continued from previous page)

```
sage: Y.<a> = Zq(125); Y.modulus()
(1 + O(5^20))*x^3 + O(5^20)*x^2 + (3 + O(5^20))*x + 3 + O(5^20)
```

```
>>> from sage.all import *
>>> GF(Integer(125), 'a').modulus()
x^3 + 3*x + 3
>>> Y = Zq(Integer(125), names=('a',)); (a,) = Y._first_ngens(1); Y.
↪modulus()
(1 + O(5^20))*x^3 + O(5^20)*x^2 + (3 + O(5^20))*x + 3 + O(5^20)
```

However, you can choose another polynomial if desired (as long as the reduction to $\mathbf{F}_p[x]$ is irreducible).

```
sage: P.<x> = ZZ[]
sage: Z.<a> = Zq(125, modulus = x^3 + 3*x^2 + x + 1); Z.modulus()
(1 + O(5^20))*x^3 + (3 + O(5^20))*x^2 + (1 + O(5^20))*x + 1 + O(5^20)
sage: Y == Z
False
```

```
>>> from sage.all import *
>>> P = ZZ['x']; (x,) = P._first_ngens(1)
>>> Z = Zq(Integer(125), modulus = x**Integer(3) +␣
↪Integer(3)*x**Integer(2) + x + Integer(1), names=('a',)); (a,) = Z._
↪first_ngens(1); Z.modulus()
(1 + O(5^20))*x^3 + (3 + O(5^20))*x^2 + (1 + O(5^20))*x + 1 + O(5^20)
>>> Y == Z
False
```

PRINTING:

There are many different ways to print p-adic elements. The way elements of a given field print is controlled by options passed in at the creation of the field. There are four basic printing modes (`'series'`, `'val-unit'`, `'terse'` and `'bars'`; `'digits'` is not available), as well as various options that either hide some information in the print representation or sometimes make print representations more compact. Note that the printing options affect whether different p-adic fields are considered equal.

1. **series**: elements are displayed as series in p.

```
sage: R.<a> = Zq(9, 20, 'capped-rel', print_mode='series'); (1+2*a)^4
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^20)
sage: -3*(1+2*a)^4
3 + a*3^2 + 3^3 + (2*a + 2)*3^4 + (2*a + 2)*3^5 + (2*a + 2)*3^6 + (2*a + 2)*3^
↪7
  + (2*a + 2)*3^8 + (2*a + 2)*3^9 + (2*a + 2)*3^10 + (2*a + 2)*3^11 + (2*a +␣
↪2)*3^12
  + (2*a + 2)*3^13 + (2*a + 2)*3^14 + (2*a + 2)*3^15 + (2*a + 2)*3^16
  + (2*a + 2)*3^17 + (2*a + 2)*3^18 + (2*a + 2)*3^19 + (2*a + 2)*3^20 + O(3^
↪21)
sage: b = ~(3*a+18); b
(a + 2)*3^-1 + 1 + 2*3 + (a + 1)*3^2 + 3^3 + 2*3^4 + (a + 1)*3^5 + 3^6 + 2*3^7
  + (a + 1)*3^8 + 3^9 + 2*3^10 + (a + 1)*3^11 + 3^12 + 2*3^13 + (a + 1)*3^14
  + 3^15 + 2*3^16 + (a + 1)*3^17 + 3^18 + O(3^19)
sage: b.parent() is R.fraction_field()
True
```

```
>>> from sage.all import *
>>> R = Zq(Integer(9), Integer(20), 'capped-rel', print_mode='series', names=(
↪'a',)); (a,) = R._first_ngens(1); (Integer(1)+Integer(2)*a)**Integer(4)
2 + (2*a + 2)*3 + (2*a + 1)*3^2 + O(3^20)
>>> -Integer(3)*(Integer(1)+Integer(2)*a)**Integer(4)
3 + a*3^2 + 3^3 + (2*a + 2)*3^4 + (2*a + 2)*3^5 + (2*a + 2)*3^6 + (2*a + 2)*3^
↪7
  + (2*a + 2)*3^8 + (2*a + 2)*3^9 + (2*a + 2)*3^10 + (2*a + 2)*3^11 + (2*a +␣
↪2)*3^12
  + (2*a + 2)*3^13 + (2*a + 2)*3^14 + (2*a + 2)*3^15 + (2*a + 2)*3^16
  + (2*a + 2)*3^17 + (2*a + 2)*3^18 + (2*a + 2)*3^19 + (2*a + 2)*3^20 + O(3^
↪21)
>>> b = ~(Integer(3)*a+Integer(18)); b
(a + 2)*3^-1 + 1 + 2*3 + (a + 1)*3^2 + 3^3 + 2*3^4 + (a + 1)*3^5 + 3^6 + 2*3^7
  + (a + 1)*3^8 + 3^9 + 2*3^10 + (a + 1)*3^11 + 3^12 + 2*3^13 + (a + 1)*3^14
  + 3^15 + 2*3^16 + (a + 1)*3^17 + 3^18 + O(3^19)
>>> b.parent() is R.fraction_field()
True
```

print_pos controls whether negatives can be used in the coefficients of powers of p.

```
sage: S.<b> = Zq(9, print_mode='series', print_pos=False); (1+2*b)^4
-1 - b*3 - 3^2 + (b + 1)*3^3 + O(3^20)
sage: -3*(1+2*b)^4
3 + b*3^2 + 3^3 + (-b - 1)*3^4 + O(3^21)
```

```
>>> from sage.all import *
>>> S = Zq(Integer(9), print_mode='series', print_pos=False, names=('b',
↪)); (b,) = S._first_ngens(1); (Integer(1)+Integer(2)*b)**Integer(4)
-1 - b*3 - 3^2 + (b + 1)*3^3 + O(3^20)
>>> -Integer(3)*(Integer(1)+Integer(2)*b)**Integer(4)
3 + b*3^2 + 3^3 + (-b - 1)*3^4 + O(3^21)
```

ram_name controls how the prime is printed.

```
sage: T.<d> = Zq(9, print_mode='series', ram_name='p'); 3*(1+2*d)^4
2*p + (2*d + 2)*p^2 + (2*d + 1)*p^3 + O(p^21)
```

```
>>> from sage.all import *
>>> T = Zq(Integer(9), print_mode='series', ram_name='p', names=('d',));␣
↪(d,) = T._first_ngens(1);␣
↪Integer(3)*(Integer(1)+Integer(2)*d)**Integer(4)
2*p + (2*d + 2)*p^2 + (2*d + 1)*p^3 + O(p^21)
```

print_max_ram_terms limits the number of powers of p that appear.

```
sage: U.<e> = Zq(9, print_mode='series', print_max_ram_terms=4); repr(-
↪3*(1+2*e)^4)
'3 + e*3^2 + 3^3 + (2*e + 2)*3^4 + ... + O(3^21)'
```

```
>>> from sage.all import *
>>> U = Zq(Integer(9), print_mode='series', print_max_ram_
```

(continues on next page)

(continued from previous page)

```
↪terms=Integer(4), names=('e',)); (e,) = U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 + e*3^2 + 3^3 + (2*e + 2)*3^4 + ... + O(3^21)'
```

print_max_unram_terms limits the number of terms that appear in a coefficient of a power of p.

```
sage: V.<f> = Zq(128, prec = 8, print_mode='series'); repr((1+f)^9)
'(f^3 + 1) + (f^5 + f^4 + f^3 + f^2)*2 + (f^6 + f^5 + f^4 + f + 1)*2^2 +␣
↪(f^5 + f^4 + f^2 + f + 1)*2^3 + (f^6 + f^5 + f^4 + f^3 + f^2 + f + 1)*2^
↪4 + (f^5 + f^4)*2^5 + (f^6 + f^5 + f^4 + f^3 + f + 1)*2^6 + (f + 1)*2^7␣
↪+ O(2^8)'
sage: V.<f> = Zq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 3); repr((1+f)^9)
'(f^3 + 1) + (f^5 + f^4 + ... + f^2)*2 + (f^6 + f^5 + ... + 1)*2^2 + (f^5␣
↪+ f^4 + ... + 1)*2^3 + (f^6 + f^5 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^
↪6 + f^5 + ... + 1)*2^6 + (f + 1)*2^7 + O(2^8)'
sage: V.<f> = Zq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 2); repr((1+f)^9)
'(f^3 + 1) + (f^5 + ... + f^2)*2 + (f^6 + ... + 1)*2^2 + (f^5 + ... +␣
↪1)*2^3 + (f^6 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^6 + ... + 1)*2^6 +␣
↪(f + 1)*2^7 + O(2^8)'
sage: V.<f> = Zq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 1); repr((1+f)^9)
'(f^3 + ...) + (f^5 + ...)*2 + (f^6 + ...)*2^2 + (f^5 + ...)*2^3 + (f^6 +␣
↪...)*2^4 + (f^5 + ...)*2^5 + (f^6 + ...)*2^6 + (f + ...)*2^7 + O(2^8)'
sage: V.<f> = Zq(128, prec = 8, print_mode='series', print_max_unram_
↪terms = 0); repr((1+f)^9 - 1 - f^3)
'(...)*2 + (...)*2^2 + (...)*2^3 + (...)*2^4 + (...)*2^5 + (...)*2^6 + (..
↪.)*2^7 + O(2^8)'
```

```
>>> from sage.all import *
>>> V = Zq(Integer(128), prec = Integer(8), print_mode='series', names=('f
↪',)); (f,) = V._first_ngens(1); repr((Integer(1)+f)**Integer(9))
'(f^3 + 1) + (f^5 + f^4 + f^3 + f^2)*2 + (f^6 + f^5 + f^4 + f + 1)*2^2 +␣
↪(f^5 + f^4 + f^2 + f + 1)*2^3 + (f^6 + f^5 + f^4 + f^3 + f^2 + f + 1)*2^
↪4 + (f^5 + f^4)*2^5 + (f^6 + f^5 + f^4 + f^3 + f + 1)*2^6 + (f + 1)*2^7␣
↪+ O(2^8)'
>>> V = Zq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(3), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9))
'(f^3 + 1) + (f^5 + f^4 + ... + f^2)*2 + (f^6 + f^5 + ... + 1)*2^2 + (f^5␣
↪+ f^4 + ... + 1)*2^3 + (f^6 + f^5 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^
↪6 + f^5 + ... + 1)*2^6 + (f + 1)*2^7 + O(2^8)'
>>> V = Zq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(2), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9))
'(f^3 + 1) + (f^5 + ... + f^2)*2 + (f^6 + ... + 1)*2^2 + (f^5 + ... +␣
↪1)*2^3 + (f^6 + ... + 1)*2^4 + (f^5 + f^4)*2^5 + (f^6 + ... + 1)*2^6 +␣
↪(f + 1)*2^7 + O(2^8)'
>>> V = Zq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(1), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9))
```

(continues on next page)

(continued from previous page)

```
'(f^3 + ...) + (f^5 + ...)*2 + (f^6 + ...)*2^2 + (f^5 + ...)*2^3 + (f^6 +␣
↪...)*2^4 + (f^5 + ...)*2^5 + (f^6 + ...)*2^6 + (f + ...)*2^7 + O(2^8)'
>>> V = Zq(Integer(128), prec = Integer(8), print_mode='series', print_
↪max_unram_terms = Integer(0), names=('f',)); (f,) = V._first_ngens(1);␣
↪repr((Integer(1)+f)**Integer(9) - Integer(1) - f**Integer(3))
'(...)*2 + (...)*2^2 + (...)*2^3 + (...)*2^4 + (...)*2^5 + (...)*2^6 + (..
↪.)*2^7 + O(2^8)'
```

show_prec determines how the precision is printed. It can be either `'none'` (or equivalently `False`), `'bigoh'` (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: U.<e> = Zq(9, 2, show_prec=False); repr(-3*(1+2*e)^4)
'3 + e*3^2'
```

```
>>> from sage.all import *
>>> U = Zq(Integer(9), Integer(2), show_prec=False, names=('e',)); (e,) =␣
↪U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 + e*3^2'
```

print_sep and *print_max_terse_terms* have no effect.

Note that print options affect equality:

```
sage: R == S, R == T, R == U, R == V, S == T, S == U, S == V, T == U, T␣
↪== V, U == V
(False, False, False, False, False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, R == V, S == T, S == U, S == V, T == U, T ==␣
↪V, U == V
(False, False, False, False, False, False, False, False, False, False)
```

2. **val-unit**: elements are displayed as $p^k u$:

```
sage: R.<a> = Zq(9, 7, print_mode='val-unit'); b = (1+3*a)^9 - 1; b
3^3 * (15 + 64*a) + O(3^7)
sage: ~b
3^-3 * (41 + a) + O(3)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(9), Integer(7), print_mode='val-unit', names=('a',)); (a,)␣
↪= R._first_ngens(1); b = (Integer(1)+Integer(3)*a)**Integer(9) - Integer(1);
↪ b
3^3 * (15 + 64*a) + O(3^7)
>>> ~b
3^-3 * (41 + a) + O(3)
```

print_pos controls whether to use a balanced representation or not.

```
sage: S.<a> = Zq(9, 7, print_mode='val-unit', print_pos=False); b =␣
↪(1+3*a)^9 - 1; b
3^3 * (15 - 17*a) + O(3^7)
sage: ~b
3^-3 * (-40 + a) + O(3)
```

```
>>> from sage.all import *
>>> S = Zq(Integer(9), Integer(7), print_mode='val-unit', print_pos=False,
↪ names=('a',)); (a,) = S._first_ngens(1); b =␣
↪(Integer(1)+Integer(3)*a)**Integer(9) - Integer(1); b
3^3 * (15 - 17*a) + O(3^7)
>>> ~b
3^-3 * (-40 + a) + O(3)
```

ram_name affects how the prime is printed.

```
sage: A.<x> = Zp(next_prime(10^6), print_mode='val-unit')[]
sage: T.<a> = Zq(next_prime(10^6)^3, 4, print_mode='val-unit', ram_name='p
↪',
....:            modulus=x^3+385831*x^2+106556*x+321036)
sage: b = next_prime(10^6)^2*(a^2 + a - 4)^4; b
p^2 * (87996187118837557387483 + 2463488883443924184640 80*a +␣
↪135353865377533261034 9*a^2)
 + O(p^6)
sage: b * (a^2 + a - 4)^-4
p^2 * 1 + O(p^6)
```

```
>>> from sage.all import *
>>> A = Zp(next_prime(Integer(10)**Integer(6)), print_mode='val-unit')['x
↪']; (x,) = A._first_ngens(1)
>>> T = Zq(next_prime(Integer(10)**Integer(6))**Integer(3), Integer(4),␣
↪print_mode='val-unit', ram_name='p',
...             modulus=x**Integer(3)+Integer(385831)*x**Integer(2)+Inte-
ger(106556)*x+Integer(321036), names=('a',)); (a,) = T._first_ngens(1)
>>> b = next_prime(Integer(10)**Integer(6))**Integer(2)*(a**Integer(2) +␣
↪a - Integer(4))**Integer(4); b
p^2 * (87996187118837557387483 + 2463488883443924184640 80*a +␣
↪135353865377533261034 9*a^2)
 + O(p^6)
>>> b * (a**Integer(2) + a - Integer(4))**-Integer(4)
p^2 * 1 + O(p^6)
```

print_max_terse_terms controls how many terms of the polynomial appear in the unit part.

```
sage: U.<a> = Zq(17^4, 6, print_mode='val-unit', print_max_terse_terms=3)
sage: b = 17*(a^3-a+14)^6; b
17 * (12131797 + 12076378*a + 10809706*a^2 + ...) + O(17^7)
```

```
>>> from sage.all import *
>>> U = Zq(Integer(17)**Integer(4), Integer(6), print_mode='val-unit',␣
↪print_max_terse_terms=Integer(3), names=('a',)); (a,) = U._first_
↪ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> b = Integer(17)*(a**Integer(3)-a+Integer(14))**Integer(6); b
17 * (12131797 + 12076378*a + 10809706*a^2 + ...) + O(17^7)
```

show_prec determines how the precision is printed. It can be either `'none'` (or equivalently `False`), `'bigoh'` (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: U.<e> = Zq(9, 2, print_mode='val-unit', show_prec=False); repr(-
↪3*(1+2*e)^4)
'3 * (1 + 3*e)'
```

```
>>> from sage.all import *
>>> U = Zq(Integer(9), Integer(2), print_mode='val-unit', show_prec=False,
↪ names=('e',)); (e,) = U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 * (1 + 3*e)'
```

print_sep, *print_max_ram_terms* and *print_max_unram_terms* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

3. **terse**: elements are displayed as a polynomial of degree less than the degree of the extension.

```
sage: R.<a> = Zq(125, print_mode='terse')
sage: (a+5)^177
68210977979428 + 90313850704069*a + 73948093055069*a^2 + O(5^20)
sage: (a/5+1)^177
68210977979428/5^177 + 90313850704069/5^177*a + 73948093055069/5^177*a^2 +␣
↪O(5^-157)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), print_mode='terse', names=('a',)); (a,) = R._first_
↪ngens(1)
>>> (a+Integer(5))**Integer(177)
68210977979428 + 90313850704069*a + 73948093055069*a^2 + O(5^20)
>>> (a/Integer(5)+Integer(1))**Integer(177)
68210977979428/5^177 + 90313850704069/5^177*a + 73948093055069/5^177*a^2 +␣
↪O(5^-157)
```

Note that in this last computation, you get one fewer p-adic digit than one might expect. This is because `R` is capped absolute, and thus 5 is cast in with relative precision 19.

As of version 3.3, if coefficients of the polynomial are non-integral, they are always printed with an explicit power of p in the denominator.

```
sage: 5*a + a^2/25
5*a + 1/5^2*a^2 + O(5^18)
```

```
>>> from sage.all import *
>>> Integer(5)*a + a**Integer(2)/Integer(25)
5*a + 1/5^2*a^2 + O(5^18)
```

print_pos controls whether to use a balanced representation or not.

```
sage: (a-5)^6
22864 + 95367431627998*a + 8349*a^2 + O(5^20)
sage: S.<a> = Zq(125, print_mode='terse', print_pos=False); b = (a-5)^6; b
22864 - 12627*a + 8349*a^2 + O(5^20)
sage: (a - 1/5)^6
-20624/5^6 + 18369/5^5*a + 1353/5^3*a^2 + O(5^14)
```

```
>>> from sage.all import *
>>> (a-Integer(5))**Integer(6)
22864 + 95367431627998*a + 8349*a^2 + O(5^20)
>>> S = Zq(Integer(125), print_mode='terse', print_pos=False, names=('a',
↪)); (a,) = S._first_ngens(1); b = (a-Integer(5))**Integer(6); b
22864 - 12627*a + 8349*a^2 + O(5^20)
>>> (a - Integer(1)/Integer(5))**Integer(6)
-20624/5^6 + 18369/5^5*a + 1353/5^3*a^2 + O(5^14)
```

ram_name affects how the prime is printed.

```
sage: T.<a> = Zq(125, print_mode='terse', ram_name='p'); (a - 1/5)^6
95367431620001/p^6 + 18369/p^5*a + 1353/p^3*a^2 + O(p^14)
```

```
>>> from sage.all import *
>>> T = Zq(Integer(125), print_mode='terse', ram_name='p', names=('a',));␣
↪(a,) = T._first_ngens(1); (a - Integer(1)/Integer(5))**Integer(6)
95367431620001/p^6 + 18369/p^5*a + 1353/p^3*a^2 + O(p^14)
```

print_max_terse_terms controls how many terms of the polynomial are shown.

```
sage: U.<a> = Zq(625, print_mode='terse', print_max_terse_terms=2); (a-1/
↪5)^6
106251/5^6 + 49994/5^5*a + ... + O(5^14)
```

```
>>> from sage.all import *
>>> U = Zq(Integer(625), print_mode='terse', print_max_terse_
↪terms=Integer(2), names=('a',)); (a,) = U._first_ngens(1); (a-
↪Integer(1)/Integer(5))**Integer(6)
106251/5^6 + 49994/5^5*a + ... + O(5^14)
```

show_prec determines how the precision is printed. It can be either `'none'` (or equivalently `False`), `'bigoh'` (or equivalently `True`). The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: U.<e> = Zq(9, 2, print_mode='terse', show_prec=False); repr(-
↪3*(1+2*e)^4)
'3 + 9*e'
```

```
>>> from sage.all import *
>>> U = Zq(Integer(9), Integer(2), print_mode='terse', show_prec=False,␣
↪names=('e',)); (e,) = U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'3 + 9*e'
```

print_sep, *print_max_ram_terms* and *print_max_unram_terms* have no effect.

Equality again depends on the printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

4. **digits**: This print mode is not available when the residue field is not prime. It might make sense to have a dictionary for small fields, but this isn't implemented.

5. **bars**: elements are displayed in a similar fashion to series, but more compactly.

```
sage: R.<a> = Zq(125); (a+5)^6
(4*a^2 + 3*a + 4) + (3*a^2 + 2*a)*5 + (a^2 + a + 1)*5^2 + (3*a + 2)*5^3
 + (3*a^2 + a + 3)*5^4 + (2*a^2 + 3*a + 2)*5^5 + O(5^20)
sage: R.<a> = Zq(125, print_mode='bars', prec=8); repr((a+5)^6)
'...[2, 3, 2]|[3, 1, 3]|[2, 3]|[1, 1, 1]|[0, 2, 3]|[4, 3, 4]'
sage: repr((a-5)^6)
'...[0, 4]|[1, 4]|[2, 0, 2]|[1, 4, 3]|[2, 3, 1]|[4, 4, 3]|[2, 4, 4]|[4, 3, 4]'
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1);␣
↪(a+Integer(5))**Integer(6)
(4*a^2 + 3*a + 4) + (3*a^2 + 2*a)*5 + (a^2 + a + 1)*5^2 + (3*a + 2)*5^3
 + (3*a^2 + a + 3)*5^4 + (2*a^2 + 3*a + 2)*5^5 + O(5^20)
>>> R = Zq(Integer(125), print_mode='bars', prec=Integer(8), names=('a',));␣
↪(a,) = R._first_ngens(1); repr((a+Integer(5))**Integer(6))
'...[2, 3, 2]|[3, 1, 3]|[2, 3]|[1, 1, 1]|[0, 2, 3]|[4, 3, 4]'
>>> repr((a-Integer(5))**Integer(6))
'...[0, 4]|[1, 4]|[2, 0, 2]|[1, 4, 3]|[2, 3, 1]|[4, 4, 3]|[2, 4, 4]|[4, 3, 4]'
```

Note that it's not possible to read off the precision from the representation in this mode.

```
sage: b = a + 3; repr(b)
'...[3, 1]'
sage: c = a + R(3, 4); repr(c)
'...[3, 1]'
sage: b.precision_absolute()
```

(continues on next page)

(continued from previous page)

```
8
sage: c.precision_absolute()
4
```

```
>>> from sage.all import *
>>> b = a + Integer(3); repr(b)
'...[3, 1]'
>>> c = a + R(Integer(3), Integer(4)); repr(c)
'...[3, 1]'
>>> b.precision_absolute()
8
>>> c.precision_absolute()
4
```

print_pos controls whether the digits can be negative.

```
sage: S.<a> = Zq(125, print_mode='bars', print_pos=False); repr((a-5)^6)
'...[1, -1, 1]|[2, 1, -2]|[2, 0, -2]|[-2, -1, 2]|[0, 0, -1]|[-2]|[-1, -2,␣
↪-1]'
sage: repr((a-1/5)^6)
'...[0, 1, 2]|[-1, 1, 1]|.|[-2, -1, -1]|[2, 2, 1]|[0, 0, -2]|[0, -1]|[0, -
↪1]|[1]'
```

```
>>> from sage.all import *
>>> S = Zq(Integer(125), print_mode='bars', print_pos=False, names=('a',
↪)); (a,) = S._first_ngens(1); repr((a-Integer(5))**Integer(6))
'...[1, -1, 1]|[2, 1, -2]|[2, 0, -2]|[-2, -1, 2]|[0, 0, -1]|[-2]|[-1, -2,␣
↪-1]'
>>> repr((a-Integer(1)/Integer(5))**Integer(6))
'...[0, 1, 2]|[-1, 1, 1]|.|[-2, -1, -1]|[2, 2, 1]|[0, 0, -2]|[0, -1]|[0, -
↪1]|[1]'
```

print_max_ram_terms controls the maximum number of "digits" shown. Note that this puts a cap on the relative precision, not the absolute precision.

```
sage: T.<a> = Zq(125, print_max_ram_terms=3, print_pos=False); (a-5)^6
(-a^2 - 2*a - 1) - 2*5 - a^2*5^2 + ... + O(5^20)
sage: 5*(a-5)^6 + 50
(-a^2 - 2*a - 1)*5 - a^2*5^3 + (2*a^2 - a - 2)*5^4 + ... + O(5^21)
sage: (a-1/5)^6
5^-6 - a*5^-5 - a*5^-4 + ... + O(5^14)
```

```
>>> from sage.all import *
>>> T = Zq(Integer(125), print_max_ram_terms=Integer(3), print_pos=False,␣
↪names=('a',)); (a,) = T._first_ngens(1); (a-Integer(5))**Integer(6)
(-a^2 - 2*a - 1) - 2*5 - a^2*5^2 + ... + O(5^20)
>>> Integer(5)*(a-Integer(5))**Integer(6) + Integer(50)
(-a^2 - 2*a - 1)*5 - a^2*5^3 + (2*a^2 - a - 2)*5^4 + ... + O(5^21)
>>> (a-Integer(1)/Integer(5))**Integer(6)
5^-6 - a*5^-5 - a*5^-4 + ... + O(5^14)
```

print_sep controls the separating character (`'|'` by default).

```
sage: U.<a> = Zq(625, print_mode='bars', print_sep=''); b = (a+5)^6;␣
↪repr(b)
'...[0, 1][4, 0, 2][3, 2, 2, 3][4, 2, 2, 4][0, 3][1, 1, 3][3, 1, 4, 1]'
```

```
>>> from sage.all import *
>>> U = Zq(Integer(625), print_mode='bars', print_sep='', names=('a',));␣
↪(a,) = U._first_ngens(1); b = (a+Integer(5))**Integer(6); repr(b)
'...[0, 1][4, 0, 2][3, 2, 2, 3][4, 2, 2, 4][0, 3][1, 1, 3][3, 1, 4, 1]'
```

print_max_unram_terms controls how many terms are shown in each `'digit'`:

```
sage: with local_print_mode(U, {'max_unram_terms': 3}): repr(b)
'...[0, 1][4,..., 0, 2][3,..., 2, 3][4,..., 2, 4][0, 3][1,..., 1, 3][3,...
↪, 4, 1]'
sage: with local_print_mode(U, {'max_unram_terms': 2}): repr(b)
'...[0, 1][4,..., 2][3,..., 3][4,..., 4][0, 3][1,..., 3][3,..., 1]'
sage: with local_print_mode(U, {'max_unram_terms': 1}): repr(b)
'...[..., 1][..., 2][..., 3][..., 4][..., 3][..., 3][..., 1]'
sage: with local_print_mode(U, {'max_unram_terms':0}): repr(b-75*a)
'...[...][...][...][...][][...][...]'
```

```
>>> from sage.all import *
>>> with local_print_mode(U, {'max_unram_terms': Integer(3)}): repr(b)
'...[0, 1][4,..., 0, 2][3,..., 2, 3][4,..., 2, 4][0, 3][1,..., 1, 3][3,...
↪, 4, 1]'
>>> with local_print_mode(U, {'max_unram_terms': Integer(2)}): repr(b)
'...[0, 1][4,..., 2][3,..., 3][4,..., 4][0, 3][1,..., 3][3,..., 1]'
>>> with local_print_mode(U, {'max_unram_terms': Integer(1)}): repr(b)
'...[..., 1][..., 2][..., 3][..., 4][..., 3][..., 3][..., 1]'
>>> with local_print_mode(U, {'max_unram_terms':Integer(0)}): repr(b-
↪Integer(75)*a)
'...[...][...][...][...][][...][...]'
```

show_prec determines how the precision is printed. It can be either `'none'` (or equivalently `False`), `'dots'` (or equivalently `True`) or `'bigoh'`. The default is `False` for the `'floating-point'` and `'fixed-mod'` types and `True` for all other types.

```
sage: U.<e> = Zq(9, 2, print_mode='bars', show_prec='bigoh'); repr(-
↪3*(1+2*e)^4)
'[0, 1]|[1]|[] + O(3^3)'
```

```
>>> from sage.all import *
>>> U = Zq(Integer(9), Integer(2), print_mode='bars', show_prec='bigoh',␣
↪names=('e',)); (e,) = U._first_ngens(1); repr(-
↪Integer(3)*(Integer(1)+Integer(2)*e)**Integer(4))
'[0, 1]|[1]|[] + O(3^3)'
```

ram_name and *print_max_terse_terms* have no effect.

Equality depends on printing options:

```
sage: R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

```
>>> from sage.all import *
>>> R == S, R == T, R == U, S == T, S == U, T == U
(False, False, False, False, False, False)
```

EXAMPLES:

Unlike for *Zp*, you can't create `Zq(N)` when `N` is not a prime power.

However, you can use `check=False` to pass in a pair in order to not have to factor. If you do so, you need to use names explicitly rather than the `R.<a>` syntax.

```
sage: p = next_prime(2^123)
sage: k = Zp(p)
sage: R.<x> = k[]
sage: K = Zq([(p, 5)], modulus=x^5+x+4, names='a', ram_name='p',
....:        print_pos=False, check=False)
sage: K.0^5
(-a - 4) + O(p^20)
```

```
>>> from sage.all import *
>>> p = next_prime(Integer(2)**Integer(123))
>>> k = Zp(p)
>>> R = k['x']; (x,) = R._first_ngens(1)
>>> K = Zq([(p, Integer(5))], modulus=x**Integer(5)+x+Integer(4), names='a', ram_
↪name='p',
...        print_pos=False, check=False)
>>> K.gen(0)**Integer(5)
(-a - 4) + O(p^20)
```

In tests on sage.math, the creation of `K` as above took an average of 1.58ms, while:

```
sage: K = Zq(p^5, modulus=x^5+x+4, names='a', ram_name='p',
....:        print_pos=False, check=True)
```

```
>>> from sage.all import *
>>> K = Zq(p**Integer(5), modulus=x**Integer(5)+x+Integer(4), names='a', ram_name=
↪'p',
...        print_pos=False, check=True)
```

took an average of 24.5ms. Of course, with smaller primes these savings disappear.

`sage.rings.padics.factory.`**`ZqCA`**(*q*, *prec=None*, **args*, ***kwds*)

A shortcut function to create capped absolute unramified p-adic rings.

See documentation for *Zq()* for a description of the input parameters.

EXAMPLES:

```
sage: R.<a> = ZqCA(25, 40); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

```
>>> from sage.all import *
>>> R = ZqCA(Integer(25), Integer(40), names=('a',)); (a,) = R._first_ngens(1); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

sage.rings.padics.factory.**ZqCR**(*q*, *prec=None*, **args*, ***kwds*)

A shortcut function to create capped relative unramified p-adic rings.

Same functionality as *Zq()*. See documentation for *Zq()* for a description of the input parameters.

EXAMPLES:

```
sage: R.<a> = ZqCR(25, 40); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(25), Integer(40), names=('a',)); (a,) = R._first_ngens(1); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

sage.rings.padics.factory.**ZqFM**(*q*, *prec=None*, **args*, ***kwds*)

A shortcut function to create fixed modulus unramified p-adic rings.

See documentation for *Zq()* for a description of the input parameters.

EXAMPLES:

```
sage: R.<a> = ZqFM(25, 40); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(25), Integer(40), names=('a',)); (a,) = R._first_ngens(1); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

sage.rings.padics.factory.**ZqFP**(*q*, *prec=None*, **args*, ***kwds*)

A shortcut function to create floating point unramified p-adic rings.

Same functionality as *Zq()*. See documentation for *Zq()* for a description of the input parameters.

EXAMPLES:

```
sage: R.<a> = ZqFP(25, 40); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

```
>>> from sage.all import *
>>> R = ZqFP(Integer(25), Integer(40), names=('a',)); (a,) = R._first_ngens(1); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
```

sage.rings.padics.factory.**get_key_base**(*p*, *prec*, *type*, *print_mode*, *names*, *ram_name*, *print_pos*, *print_sep*, *print_alphabet*, *print_max_terms*, *show_prec*, *check*, *valid_types*, *label=None*)

This implements `create_key` for `Zp` and `Qp`: moving it here prevents code duplication.

It fills in unspecified values and checks for contradictions in the input. It also standardizes irrelevant options so that duplicate parents are not created.

EXAMPLES:

```
sage: from sage.rings.padics.factory import get_key_base
sage: get_key_base(11, 5, 'capped-rel', None, None, None, None, ':', None, None,␣
↪False, True, ['capped-rel'])
(11, 5, 'capped-rel', 'series', '11', True, '|', (), -1, 'none', None)
```

(continues on next page)

(continued from previous page)

```
sage: get_key_base(12, 5, 'capped-rel', 'digits', None, None, None, None, None,␣
↪None, True, False, ['capped-rel'])
(12,
 5,
 'capped-rel',
 'digits',
 '12',
 True,
 '|',
 ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B'),
 -1,
 'dots',
 None)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.factory import get_key_base
>>> get_key_base(Integer(11), Integer(5), 'capped-rel', None, None, None, None, ':
↪', None, None, False, True, ['capped-rel'])
(11, 5, 'capped-rel', 'series', '11', True, '|', (), -1, 'none', None)
>>> get_key_base(Integer(12), Integer(5), 'capped-rel', 'digits', None, None,␣
↪None, None, None, None, True, False, ['capped-rel'])
(12,
 5,
 'capped-rel',
 'digits',
 '12',
 True,
 '|',
 ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B'),
 -1,
 'dots',
 None)
```

sage.rings.padics.factory.**is_eisenstein**(*poly*)

Return `True` iff this monic polynomial is Eisenstein.

A polynomial is Eisenstein if it is monic, the constant term has valuation 1 and all other terms have positive valuation.

EXAMPLES:

```
sage: R = Zp(5)
sage: S.<x> = R[]
sage: from sage.rings.padics.factory import is_eisenstein
sage: f = x^4 - 75*x + 15
sage: is_eisenstein(f)
True
sage: g = x^4 + 75
sage: is_eisenstein(g)
False
sage: h = x^7 + 27*x -15
sage: is_eisenstein(h)
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> from sage.rings.padics.factory import is_eisenstein
>>> f = x**Integer(4) - Integer(75)*x + Integer(15)
>>> is_eisenstein(f)
True
>>> g = x**Integer(4) + Integer(75)
>>> is_eisenstein(g)
False
>>> h = x**Integer(7) + Integer(27)*x -Integer(15)
>>> is_eisenstein(h)
False
```

sage.rings.padics.factory.**is_unramified**(*poly*)

Return `True` iff this monic polynomial is unramified.

A polynomial is unramified if its reduction modulo the maximal ideal is irreducible.

EXAMPLES:

```
sage: R = Zp(5)
sage: S.<x> = R[]
sage: from sage.rings.padics.factory import is_unramified
sage: f = x^4 + 14*x + 9
sage: is_unramified(f)
True
sage: g = x^6 + 17*x + 6
sage: is_unramified(g)
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> from sage.rings.padics.factory import is_unramified
>>> f = x**Integer(4) + Integer(14)*x + Integer(9)
>>> is_unramified(f)
True
>>> g = x**Integer(6) + Integer(17)*x + Integer(6)
>>> is_unramified(g)
False
```

sage.rings.padics.factory.**krasner_check**(*poly*, *prec*)

Return `True` iff `poly` determines a unique isomorphism class of extensions at precision `prec`.

Currently just returns `True` (thus allowing extensions that are not defined to high enough precision in order to specify them up to isomorphism). This will change in the future.

EXAMPLES:

```
sage: from sage.rings.padics.factory import krasner_check
sage: krasner_check(1,2)  # this is a stupid example.
True
```

```
>>> from sage.all import *
>>> from sage.rings.padics.factory import krasner_check
>>> krasner_check(Integer(1),Integer(2))  # this is a stupid example.
True
```

class sage.rings.padics.factory.**pAdicExtension_class**

Bases: UniqueFactory

A class for creating extensions of p-adic rings and fields.

EXAMPLES:

```
sage: R = Zp(5,3)
sage: S.<x> = ZZ[]
sage: W.<w> = pAdicExtension(R, x^4 - 15); W
5-adic Eisenstein Extension Ring in w defined by x^4 - 15
sage: W.precision_cap()
12
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(3))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> W = pAdicExtension(R, x**Integer(4) - Integer(15), names=('w',)); (w,) = W._
↪first_ngens(1); W
5-adic Eisenstein Extension Ring in w defined by x^4 - 15
>>> W.precision_cap()
12
```

create_key_and_extra_args(*base*, *modulus*, *prec=None*, *print_mode=None*, *names=None*, *var_name=None*, *res_name=None*, *unram_name=None*, *ram_name=None*, *print_pos=None*, *print_sep=None*, *print_alphabet=None*, *print_max_ram_terms=None*, *print_max_unram_terms=None*, *print_max_terse_terms=None*, *show_prec=None*, *check=True*, *unram=False*, *implementation='FLINT'*)

Create a key from input parameters for *pAdicExtension*.

See the documentation for *Qq()* for more information.

create_object(*version*, *key*, *approx_modulus=None*, *shift_seed=None*)

Create an object using a given key.

See the documentation for *pAdicExtension* for more information.

sage.rings.padics.factory.**split**(*poly*, *prec*)

Given a polynomial `poly` and a desired precision `prec`, computes `upoly` and epoly so that the extension defined by `poly` is isomorphic to the extension defined by first taking an extension by the unramified polynomial `upoly`, and then an extension by the Eisenstein polynomial `epoly`.

We need better p-adic factoring in Sage before this function can be implemented.

EXAMPLES:

```
sage: k = Qp(13)
sage: x = polygen(k)
sage: f = x^2 + 1
sage: sage.rings.padics.factory.split(f, 10)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
NotImplementedError: Extensions by general polynomials not yet supported.
Please use an unramified or Eisenstein polynomial.
```

```
>>> from sage.all import *
>>> k = Qp(Integer(13))
>>> x = polygen(k)
>>> f = x**Integer(2) + Integer(1)
>>> sage.rings.padics.factory.split(f, Integer(10))
Traceback (most recent call last):
...
NotImplementedError: Extensions by general polynomials not yet supported.
Please use an unramified or Eisenstein polynomial.
```

sage.rings.padics.factory.**truncate_to_prec**(*poly*, *R*, *absprec*)

Truncates the unused precision off of a polynomial.

EXAMPLES:

```
sage: R = Zp(5)
sage: S.<x> = R[]
sage: from sage.rings.padics.factory import truncate_to_prec
sage: f = x^4 + (3+O(5^6))*x^3 + O(5^4)
sage: truncate_to_prec(f, R, 5)
(1 + O(5^5))*x^4 + (3 + O(5^5))*x^3 + O(5^5)*x^2 + O(5^5)*x + O(5^4)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> from sage.rings.padics.factory import truncate_to_prec
>>> f = x**Integer(4) + (Integer(3)+O(Integer(5)**Integer(6)))*x**Integer(3) +␣
↪O(Integer(5)**Integer(4))
>>> truncate_to_prec(f, R, Integer(5))
(1 + O(5^5))*x^4 + (3 + O(5^5))*x^3 + O(5^5)*x^2 + O(5^5)*x + O(5^4)
```

CHAPTER

THREE

LOCAL GENERIC

Superclass for p-adic and power series rings.

AUTHORS:

- David Roe

class sage.rings.padics.local_generic.**LocalGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: Parent

Initialize self.

EXAMPLES:

```
sage: R = Zp(5)  # indirect doctest
sage: R.precision_cap()
20
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))  # indirect doctest
>>> R.precision_cap()
20
```

In Issue #14084, the category framework has been implemented for p-adic rings:

```
sage: TestSuite(R).run()                                                    #␣
↪needs sage.geometry.polyhedron
sage: K = Qp(7)
sage: TestSuite(K).run()                                                    #␣
↪needs sage.geometry.polyhedron
```

```
>>> from sage.all import *
>>> TestSuite(R).run()                                                  # needs␣
↪sage.geometry.polyhedron
>>> K = Qp(Integer(7))
>>> TestSuite(K).run()                                                  # needs␣
↪sage.geometry.polyhedron
```

absolute_degree()

Return the degree of this extension over the prime p-adic field/ring.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                          #␣
↪needs sage.libs.ntl
sage: K.absolute_degree()                                                      #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                        #␣
↪needs sage.libs.ntl
sage: L.absolute_degree()                                                      #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.absolute_degree()                                                        #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.absolute_degree()                                                        #␣
↪needs sage.libs.ntl
2
```

absolute_e()

Return the absolute ramification index of this ring/field.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                          #␣
↪needs sage.libs.ntl
sage: K.absolute_e()                                                           #␣
↪needs sage.libs.ntl
1

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                        #␣
↪needs sage.libs.ntl
sage: L.absolute_e()                                                           #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.absolute_e()                                                             #␣
↪needs sage.libs.ntl
1

>>> R = QQ['x']; (x,) = R._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.absolute_e()                                                          #␣
↪needs sage.libs.ntl
2
```

absolute_f()

Return the degree of the residue field of this ring/field over its prime subfield.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                       #␣
↪needs sage.libs.ntl
sage: K.absolute_f()                                                        #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                     #␣
↪needs sage.libs.ntl
sage: L.absolute_f()                                                        #␣
↪needs sage.libs.ntl
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.absolute_f()                                                          #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.absolute_f()                                                          #␣
↪needs sage.libs.ntl
1
```

absolute_inertia_degree()

Return the degree of the residue field of this ring/field over its prime subfield.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                       #␣
↪needs sage.libs.ntl
sage: K.absolute_inertia_degree()                                           #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                     #␣
↪needs sage.libs.ntl
sage: L.absolute_inertia_degree()                                           #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.ntl
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.absolute_inertia_degree()                                             #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.absolute_inertia_degree()                                             #␣
↪needs sage.libs.ntl
1
```

absolute_ramification_index()

Return the absolute ramification index of this ring/field.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                       #␣
↪needs sage.libs.ntl
sage: K.absolute_ramification_index()                                       #␣
↪needs sage.libs.ntl
1

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                     #␣
↪needs sage.libs.ntl
sage: L.absolute_ramification_index()                                       #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.absolute_ramification_index()                                         #␣
↪needs sage.libs.ntl
1

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.absolute_ramification_index()                                         #␣
↪needs sage.libs.ntl
2
```

change(***kwds*)

Return a new ring with changed attributes.

INPUT:

The following arguments are applied to every ring in the tower:

- `type` – string, the precision type
- `p` – the prime of the ground ring; defining polynomials will be converted to the new base rings
- `print_mode` – string
- `print_pos` – boolean
- `print_sep` – string
- `print_alphabet` – dictionary
- `show_prec` – boolean
- `check` – boolean
- `label` – string (only for lattice precision)

The following arguments are only applied to the top ring in the tower:

- `var_name` – string
- `res_name` – string
- `unram_name` – string
- `ram_name` – string
- `names` – string
- `modulus` – polynomial

The following arguments have special behavior:

- `prec` – integer; if the precision is increased on an extension ring, the precision on the base is increased as necessary (respecting ramification). If the precision is decreased, the precision of the base is unchanged.
- `field` – boolean; if `True`, switch to a tower of fields via the fraction field If `False`, switch to a tower of rings of integers
- `q` – prime power; replace the initial unramified extension of $\mathbf{Q}_p$ or $\mathbf{Z}_p$ with an unramified extension of residue cardinality q. If the initial extension is ramified, add in an unramified extension.
- `base` – ring or field; use a specific base ring instead of recursively calling *`change()`* down the tower

See the *`constructors`* for more details on the meaning of these arguments.

EXAMPLES:

We can use this method to change the precision:

```
sage: Zp(5).change(prec=40)
5-adic Ring with capped relative precision 40
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).change(prec=Integer(40))
5-adic Ring with capped relative precision 40
```

or the precision type:

```
sage: Zp(5).change(type='capped-abs')
5-adic Ring with capped absolute precision 20
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).change(type='capped-abs')
5-adic Ring with capped absolute precision 20
```

or even the prime:

```
sage: ZpCA(3).change(p=17)
17-adic Ring with capped absolute precision 20
```

```
>>> from sage.all import *
>>> ZpCA(Integer(3)).change(p=Integer(17))
17-adic Ring with capped absolute precision 20
```

You can switch between the ring of integers and its fraction field:

```
sage: ZpCA(3).change(field=True)
3-adic Field with capped relative precision 20
```

```
>>> from sage.all import *
>>> ZpCA(Integer(3)).change(field=True)
3-adic Field with capped relative precision 20
```

You can also change print modes:

```
sage: R = Zp(5).change(prec=5, print_mode='digits')
sage: repr(-R(17))
'...13403'
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5)).change(prec=Integer(5), print_mode='digits')
>>> repr(-R(Integer(17)))
'...13403'
```

Changing print mode to 'digits' works for Eisenstein extensions:

```
sage: S.<x> = ZZ[]
sage: W.<w> = Zp(3).extension(x^4 + 9*x^2 + 3*x - 3)
sage: W.print_mode()
'series'
sage: W.change(print_mode='digits').print_mode()
'digits'
```

```
>>> from sage.all import *
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> W = Zp(Integer(3)).extension(x**Integer(4) + Integer(9)*x**Integer(2) +␣
↪Integer(3)*x - Integer(3), names=('w',)); (w,) = W._first_ngens(1)
>>> W.print_mode()
'series'
>>> W.change(print_mode='digits').print_mode()
'digits'
```

You can change extensions:

```
sage: K.<a> = QqFP(125, prec=4)
sage: K.change(q=64)
2-adic Unramified Extension Field in a defined by x^6 + x^4 + x^3 + x + 1
sage: R.<x> = QQ[]
sage: K.change(modulus = x^2 - x + 2, print_pos=False)
5-adic Unramified Extension Field in a defined by x^2 - x + 2
```

```
>>> from sage.all import *
>>> K = QqFP(Integer(125), prec=Integer(4), names=('a',)); (a,) = K._first_
↪ngens(1)
>>> K.change(q=Integer(64))
2-adic Unramified Extension Field in a defined by x^6 + x^4 + x^3 + x + 1
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> K.change(modulus = x**Integer(2) - x + Integer(2), print_pos=False)
5-adic Unramified Extension Field in a defined by x^2 - x + 2
```

and variable names:

```
sage: K.change(names='b')                                                   #␣
↪needs sage.libs.flint
5-adic Unramified Extension Field in b defined by x^3 + 3*x + 3
```

```
>>> from sage.all import *
>>> K.change(names='b')                                                     #␣
↪needs sage.libs.flint
5-adic Unramified Extension Field in b defined by x^3 + 3*x + 3
```

and precision:

```
sage: Kup = K.change(prec=8); Kup
5-adic Unramified Extension Field in a defined by x^3 + 3*x + 3
sage: Kup.precision_cap()
8
sage: Kup.base_ring()
5-adic Field with floating precision 8
```

```
>>> from sage.all import *
>>> Kup = K.change(prec=Integer(8)); Kup
5-adic Unramified Extension Field in a defined by x^3 + 3*x + 3
>>> Kup.precision_cap()
8
>>> Kup.base_ring()
5-adic Field with floating precision 8
```

If you decrease the precision, the precision of the base stays the same:

```
sage: Kdown = K.change(prec=2); Kdown
5-adic Unramified Extension Field in a defined by x^3 + 3*x + 3
sage: Kdown.precision_cap()
2
sage: Kdown.base_ring()
5-adic Field with floating precision 4
```

```
>>> from sage.all import *
>>> Kdown = K.change(prec=Integer(2)); Kdown
5-adic Unramified Extension Field in a defined by x^3 + 3*x + 3
>>> Kdown.precision_cap()
2
>>> Kdown.base_ring()
5-adic Field with floating precision 4
```

Changing the prime works for extensions:

```
sage: x = polygen(ZZ)
sage: R.<a> = Zp(5).extension(x^2 + 2)
sage: S = R.change(p=7)
sage: S.defining_polynomial(exact=True)
x^2 + 2
sage: A.<y> = Zp(5)[]
sage: R.<a> = Zp(5).extension(y^2 + 2)
sage: S = R.change(p=7)
sage: S.defining_polynomial(exact=True)
y^2 + 2
```

```
>>> from sage.all import *
>>> x = polygen(ZZ)
>>> R = Zp(Integer(5)).extension(x**Integer(2) + Integer(2), names=('a',));␣
↪(a,) = R._first_ngens(1)
>>> S = R.change(p=Integer(7))
>>> S.defining_polynomial(exact=True)
x^2 + 2
>>> A = Zp(Integer(5))['y']; (y,) = A._first_ngens(1)
>>> R = Zp(Integer(5)).extension(y**Integer(2) + Integer(2), names=('a',));␣
↪(a,) = R._first_ngens(1)
>>> S = R.change(p=Integer(7))
>>> S.defining_polynomial(exact=True)
y^2 + 2
```

```
sage: R.<a> = Zq(5^3)
sage: S = R.change(prec=50)
sage: S.defining_polynomial(exact=True)
x^3 + 3*x + 3
```

```
>>> from sage.all import *
>>> R = Zq(Integer(5)**Integer(3), names=('a',)); (a,) = R._first_ngens(1)
>>> S = R.change(prec=Integer(50))
>>> S.defining_polynomial(exact=True)
x^3 + 3*x + 3
```

Changing label for lattice precision (the precision lattice is not copied):

```
sage: R = ZpLC(37, (8,11))
sage: S = R.change(label = "change"); S
37-adic Ring with lattice-cap precision (label: change)
sage: S.change(label = "new")
37-adic Ring with lattice-cap precision (label: new)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(37), (Integer(8),Integer(11)))
>>> S = R.change(label = "change"); S
37-adic Ring with lattice-cap precision (label: change)
>>> S.change(label = "new")
37-adic Ring with lattice-cap precision (label: new)
```

defining_polynomial(*var='x'*, *exact=False*)

Return the defining polynomial of this local ring.

INPUT:

- `var` – string (default: `'x'`); the name of the variable
- `exact` – boolean (default: `False`); whether to return the underlying exact defining polynomial rather than the one with coefficients in the base ring

OUTPUT: the defining polynomial of this ring as an extension over its ground ring

EXAMPLES:

```
sage: R = Zp(3, 3, 'fixed-mod')

sage: R.defining_polynomial().parent()
Univariate Polynomial Ring in x over 3-adic Ring of fixed modulus 3^3
sage: R.defining_polynomial('foo')
foo

sage: R.defining_polynomial(exact=True).parent()
Univariate Polynomial Ring in x over Integer Ring
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(3), 'fixed-mod')

>>> R.defining_polynomial().parent()
Univariate Polynomial Ring in x over 3-adic Ring of fixed modulus 3^3
>>> R.defining_polynomial('foo')
foo

>>> R.defining_polynomial(exact=True).parent()
Univariate Polynomial Ring in x over Integer Ring
```

degree()

Return the degree of this extension.

Raise an error if the base ring/field is itself an extension.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                          #␣
↪needs sage.libs.ntl
sage: K.degree()                                                               #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
```

(continues on next page)

(continued from previous page)

```
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                    #␣
↪needs sage.libs.ntl
sage: L.degree()                                                           #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.degree()                                                             #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.degree()                                                             #␣
↪needs sage.libs.ntl
2
```

e()

Return the ramification index of this extension.

Raise an error if the base ring/field is itself an extension.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                      #␣
↪needs sage.libs.ntl
sage: K.e()                                                                #␣
↪needs sage.libs.ntl
1

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                    #␣
↪needs sage.libs.ntl
sage: L.e()                                                                #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.e()                                                                  #␣
↪needs sage.libs.ntl
1

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.e()                                                                  #␣
↪needs sage.libs.ntl
2
```

ext(**args*, ***kwds*)

Construct an extension of `self`. See *extension()* for more details.

EXAMPLES:

```
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                          #␣
↪needs sage.libs.ntl
sage: B.<t> = A.ext(x^2 + 7)                                               #␣
↪needs sage.libs.ntl
sage: B.uniformiser()                                                      #␣
↪needs sage.libs.ntl
t + O(t^21)
```

```
>>> from sage.all import *
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2) + Integer(7), names=('t',)); (t,) = B._first_
↪ngens(1)# needs sage.libs.ntl
>>> B.uniformiser()                                                        #␣
↪needs sage.libs.ntl
t + O(t^21)
```

f()

Return the degree of the residual extension.

Raise an error if the base ring/field is itself an extension.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                      #␣
↪needs sage.libs.ntl
sage: K.f()                                                                #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                    #␣
↪needs sage.libs.ntl
sage: L.f()                                                                #␣
↪needs sage.libs.ntl
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.f()                                                                  #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.f()                                                                  #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.ntl
1
```

ground_ring()

Return `self`.

Will be overridden by extensions.

INPUT:

- `self` – a local ring

OUTPUT: the ground ring of `self`, i.e., itself

EXAMPLES:

```
sage: R = Zp(3, 5, 'fixed-mod')
sage: S = Zp(3, 4, 'fixed-mod')
sage: R.ground_ring() is R
True
sage: S.ground_ring() is R
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(5), 'fixed-mod')
>>> S = Zp(Integer(3), Integer(4), 'fixed-mod')
>>> R.ground_ring() is R
True
>>> S.ground_ring() is R
False
```

ground_ring_of_tower()

Return `self`.

Will be overridden by extensions.

INPUT:

- `self` – a p-adic ring

OUTPUT: the ground ring of the tower for `self`, i.e., itself

EXAMPLES:

```
sage: R = Zp(5)
sage: R.ground_ring_of_tower()
5-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.ground_ring_of_tower()
5-adic Ring with capped relative precision 20
```

inertia_degree()

Return the degree of the residual extension.

Raise an error if the base ring/field is itself an extension.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                          #␣
↪needs sage.libs.ntl
sage: K.inertia_degree()                                                       #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                        #␣
↪needs sage.libs.ntl
sage: L.inertia_degree()                                                       #␣
↪needs sage.libs.ntl
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.inertia_degree()                                                         #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.inertia_degree()                                                         #␣
↪needs sage.libs.ntl
1
```

inertia_subring()

Return the inertia subring, i.e. `self`.

INPUT:

- `self` – a local ring

OUTPUT: the inertia subring of `self`, i.e., itself

EXAMPLES:

```
sage: R = Zp(5)
sage: R.inertia_subring()
5-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.inertia_subring()
5-adic Ring with capped relative precision 20
```

is_capped_absolute()

Return whether this p-adic ring bounds precision in a capped absolute fashion.

The absolute precision of an element is the power of p modulo which that element is defined. In a capped absolute ring, the absolute precision of elements are bounded by a constant depending on the ring.

EXAMPLES:

```
sage: R = ZpCA(5, 15)
sage: R.is_capped_absolute()
True
sage: R(5^7)
5^7 + O(5^15)
sage: S = Zp(5, 15)
sage: S.is_capped_absolute()
False
sage: S(5^7)
5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5), Integer(15))
>>> R.is_capped_absolute()
True
>>> R(Integer(5)**Integer(7))
5^7 + O(5^15)
>>> S = Zp(Integer(5), Integer(15))
>>> S.is_capped_absolute()
False
>>> S(Integer(5)**Integer(7))
5^7 + O(5^22)
```

is_capped_relative()

Return whether this p-adic ring bounds precision in a capped relative fashion.

The relative precision of an element is the power of p modulo which the unit part of that element is defined. In a capped relative ring, the relative precision of elements are bounded by a constant depending on the ring.

EXAMPLES:

```
sage: R = ZpCA(5, 15)
sage: R.is_capped_relative()
False
sage: R(5^7)
5^7 + O(5^15)
sage: S = Zp(5, 15)
sage: S.is_capped_relative()
True
sage: S(5^7)
5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5), Integer(15))
>>> R.is_capped_relative()
False
>>> R(Integer(5)**Integer(7))
5^7 + O(5^15)
>>> S = Zp(Integer(5), Integer(15))
>>> S.is_capped_relative()
True
>>> S(Integer(5)**Integer(7))
5^7 + O(5^22)
```

is_exact()

Return whether this p-adic ring is exact, i.e. `False`.

EXAMPLES:

```
sage: R = Zp(5, 3, 'fixed-mod'); R.is_exact()
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(3), 'fixed-mod'); R.is_exact()
False
```

is_fixed_mod()

Return whether this p-adic ring bounds precision in a fixed modulus fashion.

The absolute precision of an element is the power of p modulo which that element is defined. In a fixed modulus ring, the absolute precision of every element is defined to be the precision cap of the parent. This means that some operations, such as division by p, don't return a well defined answer.

EXAMPLES:

```
sage: R = ZpFM(5,15)
sage: R.is_fixed_mod()
True
sage: R(5^7,absprec=9)
5^7
sage: S = ZpCA(5, 15)
sage: S.is_fixed_mod()
False
sage: S(5^7,absprec=9)
5^7 + O(5^9)
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(15))
>>> R.is_fixed_mod()
True
>>> R(Integer(5)**Integer(7),absprec=Integer(9))
5^7
>>> S = ZpCA(Integer(5), Integer(15))
>>> S.is_fixed_mod()
False
>>> S(Integer(5)**Integer(7),absprec=Integer(9))
5^7 + O(5^9)
```

is_floating_point()

Return whether this p-adic ring bounds precision in a floating point fashion.

The relative precision of an element is the power of p modulo which the unit part of that element is defined. In a floating point ring, elements do not store precision, but arithmetic operations truncate to a relative precision depending on the ring.

EXAMPLES:

```
sage: R = ZpCR(5, 15)
sage: R.is_floating_point()
```

(continues on next page)

(continued from previous page)

```
False
sage: R(5^7)
5^7 + O(5^22)
sage: S = ZpFP(5, 15)
sage: S.is_floating_point()
True
sage: S(5^7)
5^7
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5), Integer(15))
>>> R.is_floating_point()
False
>>> R(Integer(5)**Integer(7))
5^7 + O(5^22)
>>> S = ZpFP(Integer(5), Integer(15))
>>> S.is_floating_point()
True
>>> S(Integer(5)**Integer(7))
5^7
```

is_lattice_prec()

Return whether this p-adic ring bounds precision using a lattice model.

In lattice precision, relationships between elements are stored in a precision object of the parent, which allows for optimal precision tracking at the cost of increased memory usage and runtime.

EXAMPLES:

```
sage: R = ZpCR(5, 15)
sage: R.is_lattice_prec()
False
sage: x = R(25, 8)
sage: x - x
O(5^8)
sage: S = ZpLC(5, 15)
doctest:...: FutureWarning: This class/method/function is marked as␣
↪experimental. It, its functionality or its interface might change without a␣
↪formal deprecation.
See https://github.com/sagemath/sage/issues/23505 for details.
sage: S.is_lattice_prec()
True
sage: x = S(25, 8)
sage: x - x
O(5^30)
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5), Integer(15))
>>> R.is_lattice_prec()
False
>>> x = R(Integer(25), Integer(8))
>>> x - x
```

(continues on next page)

(continued from previous page)

```
O(5^8)
>>> S = ZpLC(Integer(5), Integer(15))
doctest:...: FutureWarning: This class/method/function is marked as␣
↪experimental. It, its functionality or its interface might change without a␣
↪formal deprecation.
See https://github.com/sagemath/sage/issues/23505 for details.
>>> S.is_lattice_prec()
True
>>> x = S(Integer(25), Integer(8))
>>> x - x
O(5^30)
```

is_relaxed()

Return whether this p-adic ring bounds precision in a relaxed fashion.

In a relaxed ring, elements have mechanisms for computing themselves to greater precision.

EXAMPLES:

```
sage: R = Zp(5)
sage: R.is_relaxed()
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.is_relaxed()
False
```

maximal_unramified_subextension()

Return the maximal unramified subextension.

INPUT:

- `self` – a local ring

OUTPUT: the maximal unramified subextension of `self`

EXAMPLES:

```
sage: R = Zp(5)
sage: R.maximal_unramified_subextension()
5-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.maximal_unramified_subextension()
5-adic Ring with capped relative precision 20
```

precision_cap()

Return the precision cap for this ring.

EXAMPLES:

```
sage: R = Zp(3, 10,'fixed-mod'); R.precision_cap()
10
```

(continues on next page)

(continued from previous page)

```
sage: R = Zp(3, 10,'capped-rel'); R.precision_cap()
10
sage: R = Zp(3, 10,'capped-abs'); R.precision_cap()
10
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(10),'fixed-mod'); R.precision_cap()
10
>>> R = Zp(Integer(3), Integer(10),'capped-rel'); R.precision_cap()
10
>>> R = Zp(Integer(3), Integer(10),'capped-abs'); R.precision_cap()
10
```

Note

This will have different meanings depending on the type of local ring. For fixed modulus rings, all elements are considered modulo `self.prime()^self.precision_cap()`. For rings with an absolute cap (i.e. the class `pAdicRingCappedAbsolute`), each element has a precision that is tracked and is bounded above by `self.precision_cap()`. Rings with relative caps (e.g. the class `pAdicRingCappedRelative`) are the same except that the precision is the precision of the unit part of each element.

ramification_index()

Return the ramification index of this extension.

Raise an error if the base ring/field is itself an extension.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                          #␣
↪needs sage.libs.ntl
sage: K.ramification_index()                                                   #␣
↪needs sage.libs.ntl
1

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                        #␣
↪needs sage.libs.ntl
sage: L.ramification_index()                                                   #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.ramification_index()                                                     #␣
↪needs sage.libs.ntl
1

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
```

(continues on next page)

(continued from previous page)

```
>>> L.ramification_index()                                                    #␣
↪needs sage.libs.ntl
2
```

relative_degree()

Return the degree of this extension over its base field/ring.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                        #␣
↪needs sage.libs.ntl
sage: K.relative_degree()                                                    #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                      #␣
↪needs sage.libs.ntl
sage: L.relative_degree()                                                    #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.relative_degree()                                                      #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.relative_degree()                                                      #␣
↪needs sage.libs.ntl
2
```

relative_e()

Return the ramification index of this extension over its base ring/field.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                        #␣
↪needs sage.libs.ntl
sage: K.relative_e()                                                         #␣
↪needs sage.libs.ntl
1

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                      #␣
↪needs sage.libs.ntl
sage: L.relative_e()                                                         #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.relative_e()                                                          #␣
↪needs sage.libs.ntl
1

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.relative_e()                                                          #␣
↪needs sage.libs.ntl
2
```

relative_f()

Return the degree of the residual extension over its base ring/field.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                       #␣
↪needs sage.libs.ntl
sage: K.relative_f()                                                        #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                     #␣
↪needs sage.libs.ntl
sage: L.relative_f()                                                        #␣
↪needs sage.libs.ntl
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.relative_f()                                                          #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.relative_f()                                                          #␣
↪needs sage.libs.ntl
1
```

relative_inertia_degree()

Return the degree of the residual extension over its base ring/field.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                       #␣
↪needs sage.libs.ntl
```

(continues on next page)

(continued from previous page)

```
sage: K.relative_inertia_degree()                                           #␣
↪needs sage.libs.ntl
5

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                     #␣
↪needs sage.libs.ntl
sage: L.relative_inertia_degree()                                           #␣
↪needs sage.libs.ntl
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.relative_inertia_degree()                                             #␣
↪needs sage.libs.ntl
5

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.relative_inertia_degree()                                             #␣
↪needs sage.libs.ntl
1
```

relative_ramification_index()

Return the ramification index of this extension over its base ring/field.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                       #␣
↪needs sage.libs.ntl
sage: K.relative_ramification_index()                                       #␣
↪needs sage.libs.ntl
1

sage: R.<x> = QQ[]
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                     #␣
↪needs sage.libs.ntl
sage: L.relative_ramification_index()                                       #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.relative_ramification_index()                                         #␣
↪needs sage.libs.ntl
1

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
```

(continues on next page)

(continued from previous page)

```
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.relative_ramification_index()                                    #␣
↪needs sage.libs.ntl
2
```

residue_characteristic()

Return the characteristic of `self`'s residue field.

INPUT:

- `self` – a p-adic ring

OUTPUT: the characteristic of the residue field

EXAMPLES:

```
sage: R = Zp(3, 5, 'capped-rel'); R.residue_characteristic()
3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(5), 'capped-rel'); R.residue_characteristic()
3
```

uniformiser()

Return a uniformiser for `self`, ie a generator for the unique maximal ideal.

EXAMPLES:

```
sage: R = Zp(5)
sage: R.uniformiser()
5 + O(5^21)
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                      #␣
↪needs sage.libs.ntl
sage: B.<t> = A.ext(x^2+7)                                             #␣
↪needs sage.libs.ntl
sage: B.uniformiser()                                                  #␣
↪needs sage.libs.ntl
t + O(t^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.uniformiser()
5 + O(5^21)
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2)+Integer(7), names=('t',)); (t,) = B._first_
↪ngens(1)# needs sage.libs.ntl
>>> B.uniformiser()                                                    #␣
↪needs sage.libs.ntl
t + O(t^21)
```

uniformiser_pow(n)

Return the n-th power of the uniformiser of `self` (as an element of `self`).

EXAMPLES:

```
sage: R = Zp(5)
sage: R.uniformiser_pow(5)
5^5 + O(5^25)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.uniformiser_pow(Integer(5))
5^5 + O(5^25)
```

CHAPTER

FOUR

p-ADIC GENERIC

A generic superclass for all p-adic parents.

AUTHORS:

- David Roe
- Genya Zaytman: documentation
- David Harvey: doctests
- Julian Rueth (2013-03-16): test methods for basic arithmetic

class sage.rings.padics.padic_generic.**ResidueLiftingMap**

Bases: Morphism

Lifting map to a p-adic ring or field from its residue field or ring.

These maps must be created using the _create_ method in order to support categories correctly.

EXAMPLES:

```
sage: from sage.rings.padics.padic_generic import ResidueLiftingMap
sage: R.<a> = Zq(125); k = R.residue_field()
sage: f = ResidueLiftingMap._create_(k, R); f
Lifting morphism:
  From: Finite Field in a0 of size 5^3
  To:   5-adic Unramified Extension Ring in a defined by x^3 + 3*x + 3
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_generic import ResidueLiftingMap
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1); k = R.residue_
↪field()
>>> f = ResidueLiftingMap._create_(k, R); f
Lifting morphism:
  From: Finite Field in a0 of size 5^3
  To:   5-adic Unramified Extension Ring in a defined by x^3 + 3*x + 3
```

class sage.rings.padics.padic_generic.**ResidueReductionMap**

Bases: Morphism

Reduction map from a p-adic ring or field to its residue field or ring.

These maps must be created using the _create_ method in order to support categories correctly.

EXAMPLES:

```
sage: from sage.rings.padics.padic_generic import ResidueReductionMap
sage: R.<a> = Zq(125); k = R.residue_field()
sage: f = ResidueReductionMap._create_(R, k); f
Reduction morphism:
  From: 5-adic Unramified Extension Ring in a defined by x^3 + 3*x + 3
  To:   Finite Field in a0 of size 5^3
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_generic import ResidueReductionMap
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1); k = R.residue_
↪field()
>>> f = ResidueReductionMap._create_(R, k); f
Reduction morphism:
  From: 5-adic Unramified Extension Ring in a defined by x^3 + 3*x + 3
  To:   Finite Field in a0 of size 5^3
```

is_injective()

The reduction map is far from injective.

EXAMPLES:

```
sage: GF(5).convert_map_from(ZpCA(5)).is_injective()
False
```

```
>>> from sage.all import *
>>> GF(Integer(5)).convert_map_from(ZpCA(Integer(5))).is_injective()
False
```

is_surjective()

The reduction map is surjective.

EXAMPLES:

```
sage: GF(7).convert_map_from(Qp(7)).is_surjective()
True
```

```
>>> from sage.all import *
>>> GF(Integer(7)).convert_map_from(Qp(Integer(7))).is_surjective()
True
```

section()

Return the section from the residue ring or field back to the p-adic ring or field.

EXAMPLES:

```
sage: GF(3).convert_map_from(Zp(3)).section()
Lifting morphism:
  From: Finite Field of size 3
  To:   3-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> GF(Integer(3)).convert_map_from(Zp(Integer(3))).section()
Lifting morphism:
```

(continues on next page)

(continued from previous page)

```
From: Finite Field of size 3
To:   3-adic Ring with capped relative precision 20
```

sage.rings.padics.padic_generic.**local_print_mode**(*obj*, *print_options*, *pos=None*, *ram_name=None*)

Context manager for safely temporarily changing the print_mode of a p-adic ring/field.

EXAMPLES:

```
sage: R = Zp(5)
sage: R(45)
4*5 + 5^2 + O(5^21)
sage: with local_print_mode(R, 'val-unit'):
....:     print(R(45))
5 * 9 + O(5^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R(Integer(45))
4*5 + 5^2 + O(5^21)
>>> with local_print_mode(R, 'val-unit'):
...     print(R(Integer(45)))
5 * 9 + O(5^21)
```

Note

For more documentation see `localvars`.

class sage.rings.padics.padic_generic.**pAdicGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *LocalGeneric*

Initialize `self`.

INPUT:

- `base` – base ring
- `p` – prime
- `print_mode` – dictionary of print options
- `names` – how to print the uniformizer
- `element_class` – the class for elements of this ring

EXAMPLES:

```
sage: R = Zp(17)  # indirect doctest
```

```
>>> from sage.all import *
>>> R = Zp(Integer(17))  # indirect doctest
```

characteristic()

Return the characteristic of `self`, which is always 0.

EXAMPLES:

```
sage: R = Zp(3, 10,'fixed-mod'); R.characteristic()
0
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(10),'fixed-mod'); R.characteristic()
0
```

extension(*modulus*, *prec=None*, *names=None*, *print_mode=None*, *implementation='FLINT'*, ***kwds*)

Create an extension of this p-adic ring.

EXAMPLES:

```
sage: k = Qp(5)
sage: R.<x> = k[]
sage: l.<w> = k.extension(x^2 - 5); l
5-adic Eisenstein Extension Field in w defined by x^2 - 5
sage: F = list(Qp(19)['x'](cyclotomic_polynomial(5)).factor())[0][0]
sage: L = Qp(19).extension(F, names='a'); L
19-adic Unramified Extension Field in a defined by x^2 +␣
↪875167499621185957380638 3*x + 1
```

```
>>> from sage.all import *
>>> k = Qp(Integer(5))
>>> R = k['x']; (x,) = R._first_ngens(1)
>>> l = k.extension(x**Integer(2) - Integer(5), names=('w',)); (w,) = l._
↪first_ngens(1); l
5-adic Eisenstein Extension Field in w defined by x^2 - 5
>>> F = list(Qp(Integer(19))['x'](cyclotomic_polynomial(Integer(5))).
↪factor())[Integer(0)][Integer(0)]
>>> L = Qp(Integer(19)).extension(F, names='a'); L
19-adic Unramified Extension Field in a defined by x^2 +␣
↪875167499621185957380638 3*x + 1
```

fraction_field(*print_mode=None*)

Return the fraction field of this ring or field.

For $\mathbf{Z}_p$, this is the p-adic field with the same options, and for extensions, it is just the extension of the fraction field of the base determined by the same polynomial.

The fraction field of a capped absolute ring is capped relative, and that of a fixed modulus ring is floating point.

INPUT:

- `print_mode` – (optional) a dictionary containing print options; defaults to the same options as this ring

OUTPUT: the fraction field of this ring

EXAMPLES:

```
sage: R = Zp(5, print_mode='digits', show_prec=False)
sage: K = R.fraction_field(); K(1/3)
31313131313131313132
sage: L = R.fraction_field({'max_ram_terms':4}); L(1/3)
doctest:warning
```

(continues on next page)

(continued from previous page)

```
...
DeprecationWarning: Use the change method if you want to change print options␣
↪in fraction_field()
See https://github.com/sagemath/sage/issues/23227 for details.
3132
sage: U.<a> = Zq(17^4, 6, print_mode='val-unit', print_max_terse_terms=3)
sage: U.fraction_field()
17-adic Unramified Extension Field in a defined by x^4 + 7*x^2 + 10*x + 3
sage: U.fraction_field({"pos":False}) == U.fraction_field()
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='digits', show_prec=False)
>>> K = R.fraction_field(); K(Integer(1)/Integer(3))
3131313131313131313132
>>> L = R.fraction_field({'max_ram_terms':Integer(4)}); L(Integer(1)/
↪Integer(3))
doctest:warning
...
DeprecationWarning: Use the change method if you want to change print options␣
↪in fraction_field()
See https://github.com/sagemath/sage/issues/23227 for details.
3132
>>> U = Zq(Integer(17)**Integer(4), Integer(6), print_mode='val-unit', print_
↪max_terse_terms=Integer(3), names=('a',)); (a,) = U._first_ngens(1)
>>> U.fraction_field()
17-adic Unramified Extension Field in a defined by x^4 + 7*x^2 + 10*x + 3
>>> U.fraction_field({"pos":False}) == U.fraction_field()
False
```

frobenius_endomorphism(*n=1*)

Return the n-th power of the absolute arithmetic Frobeninus endomorphism on this field.

INPUT:

- `n` – integer (default: 1)

EXAMPLES:

```
sage: K.<a> = Qq(3^5)
sage: Frob = K.frobenius_endomorphism(); Frob
Frobenius endomorphism on 3-adic Unramified Extension
... lifting a |--> a^3 on the residue field
sage: Frob(a) == a.frobenius()
True
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism(); Frob
Frobenius endomorphism on 3-adic Unramified Extension
... lifting a |--> a^3 on the residue field
>>> Frob(a) == a.frobenius()
True
```

We can specify a power:

```
sage: K.frobenius_endomorphism(2)
Frobenius endomorphism on 3-adic Unramified Extension
... lifting a |--> a^(3^2) on the residue field
```

```
>>> from sage.all import *
>>> K.frobenius_endomorphism(Integer(2))
Frobenius endomorphism on 3-adic Unramified Extension
... lifting a |--> a^(3^2) on the residue field
```

The result is simplified if possible:

```
sage: K.frobenius_endomorphism(6)
Frobenius endomorphism on 3-adic Unramified Extension
... lifting a |--> a^3 on the residue field
sage: K.frobenius_endomorphism(5)
Identity endomorphism of 3-adic Unramified Extension ...
```

```
>>> from sage.all import *
>>> K.frobenius_endomorphism(Integer(6))
Frobenius endomorphism on 3-adic Unramified Extension
... lifting a |--> a^3 on the residue field
>>> K.frobenius_endomorphism(Integer(5))
Identity endomorphism of 3-adic Unramified Extension ...
```

Comparisons work:

```
sage: K.frobenius_endomorphism(6) == Frob
True
```

```
>>> from sage.all import *
>>> K.frobenius_endomorphism(Integer(6)) == Frob
True
```

gens()

Return a tuple of generators.

EXAMPLES:

```
sage: R = Zp(5); R.gens()
(5 + O(5^21),)
sage: Zq(25,names='a').gens()
(a + O(5^20),)
sage: S.<x> = ZZ[]; f = x^5 + 25*x -5; W.<w> = R.ext(f); W.gens()
(w + O(w^101),)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5)); R.gens()
(5 + O(5^21),)
>>> Zq(Integer(25),names='a').gens()
(a + O(5^20),)
>>> S = ZZ['x']; (x,) = S._first_ngens(1); f = x**Integer(5) + Integer(25)*x -
```

(continues on next page)

(continued from previous page)

```
↪Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1); W.gens()
(w + O(w^101),)
```

integer_ring(*print_mode=None*)

Return the ring of integers of this ring or field.

For $\mathbf{Q}_p$, this is the p-adic ring with the same options, and for extensions, it is just the extension of the ring of integers of the base determined by the same polynomial.

INPUT:

- `print_mode` – (optional) a dictionary containing print options; defaults to the same options as this ring

OUTPUT: the ring of elements of this field with nonnegative valuation

EXAMPLES:

```
sage: K = Qp(5, print_mode='digits', show_prec=False)
sage: R = K.integer_ring(); R(1/3)
31313131313131313132
sage: S = K.integer_ring({'max_ram_terms':4}); S(1/3)
doctest:warning
...
DeprecationWarning: Use the change method if you want to change print options␣
↪in integer_ring()
See https://github.com/sagemath/sage/issues/23227 for details.
3132
sage: U.<a> = Qq(17^4, 6, print_mode='val-unit', print_max_terse_terms=3)
sage: U.integer_ring()
17-adic Unramified Extension Ring in a defined by x^4 + 7*x^2 + 10*x + 3
sage: U.fraction_field({"print_mode":"terse"}) == U.fraction_field()
doctest:warning
...
DeprecationWarning: Use the change method if you want to change print options␣
↪in fraction_field()
See https://github.com/sagemath/sage/issues/23227 for details.
False
```

```
>>> from sage.all import *
>>> K = Qp(Integer(5), print_mode='digits', show_prec=False)
>>> R = K.integer_ring(); R(Integer(1)/Integer(3))
31313131313131313132
>>> S = K.integer_ring({'max_ram_terms':Integer(4)}); S(Integer(1)/Integer(3))
doctest:warning
...
DeprecationWarning: Use the change method if you want to change print options␣
↪in integer_ring()
See https://github.com/sagemath/sage/issues/23227 for details.
3132
>>> U = Qq(Integer(17)**Integer(4), Integer(6), print_mode='val-unit', print_
↪max_terse_terms=Integer(3), names=('a',)); (a,) = U._first_ngens(1)
>>> U.integer_ring()
17-adic Unramified Extension Ring in a defined by x^4 + 7*x^2 + 10*x + 3
>>> U.fraction_field({"print_mode":"terse"}) == U.fraction_field()
```

(continues on next page)

(continued from previous page)

```
doctest:warning
...
DeprecationWarning: Use the change method if you want to change print options␣
↪in fraction_field()
See https://github.com/sagemath/sage/issues/23227 for details.
False
```

ngens()

Return the number of generators of `self`.

We conventionally define this as 1: for base rings, we take a uniformizer as the generator; for extension rings, we take a root of the minimal polynomial defining the extension.

EXAMPLES:

```
sage: Zp(5).ngens()
1
sage: Zq(25,names='a').ngens()
1
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).ngens()
1
>>> Zq(Integer(25),names='a').ngens()
1
```

prime()

Return the prime, ie the characteristic of the residue field.

OUTPUT: the characteristic of the residue field

EXAMPLES:

```
sage: R = Zp(3,5,'fixed-mod')
sage: R.prime()
3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(5),'fixed-mod')
>>> R.prime()
3
```

primitive_root_of_unity(*n=None*, *order=False*)

Return a generator of the group of `n`-th roots of unity in this ring.

INPUT:

- `n` – integer or `None` (default: `None`)
- `order` – boolean (default: `False`)

OUTPUT:

A generator of the group of `n`-th roots of unity. If `n` is `None`, a generator of the full group of roots of unity is returned.

If `order` is `True`, the order of the above group is returned as well.

EXAMPLES:

```
sage: R = Zp(5, 10)
sage: zeta = R.primitive_root_of_unity(); zeta
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
sage: zeta == R.teichmuller(2)
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(10))
>>> zeta = R.primitive_root_of_unity(); zeta
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
>>> zeta == R.teichmuller(Integer(2))
True
```

Now we consider an example with non trivial p-th roots of unity:

```
sage: W = Zp(3, 2)
sage: S.<x> = W[]
sage: R.<pi> = W.extension((x+1)^6 + (x+1)^3 + 1)
sage: zeta, order = R.primitive_root_of_unity(order=True)
sage: zeta
2 + 2*pi + 2*pi^3 + 2*pi^7 + 2*pi^8 + 2*pi^9 + pi^11 + O(pi^12)
sage: order
18
sage: zeta.multiplicative_order()
18
sage: zeta, order = R.primitive_root_of_unity(24, order=True)
sage: zeta
2 + pi^3 + 2*pi^7 + 2*pi^8 + 2*pi^10 + 2*pi^11 + O(pi^12)
sage: order   # equal to gcd(18,24)
6
sage: zeta.multiplicative_order()
6
```

```
>>> from sage.all import *
>>> W = Zp(Integer(3), Integer(2))
>>> S = W['x']; (x,) = S._first_ngens(1)
>>> R = W.extension((x+Integer(1))**Integer(6) + (x+Integer(1))**Integer(3) +␣
↪Integer(1), names=('pi',)); (pi,) = R._first_ngens(1)
>>> zeta, order = R.primitive_root_of_unity(order=True)
>>> zeta
2 + 2*pi + 2*pi^3 + 2*pi^7 + 2*pi^8 + 2*pi^9 + pi^11 + O(pi^12)
>>> order
18
>>> zeta.multiplicative_order()
18
>>> zeta, order = R.primitive_root_of_unity(Integer(24), order=True)
>>> zeta
2 + pi^3 + 2*pi^7 + 2*pi^8 + 2*pi^10 + 2*pi^11 + O(pi^12)
>>> order   # equal to gcd(18,24)
6
>>> zeta.multiplicative_order()
```

(continues on next page)

(continued from previous page)

```
6
```

print_mode()

Return the current print mode as a string.

EXAMPLES:

```
sage: R = Qp(7,5, 'capped-rel')
sage: R.print_mode()
'series'
```

```
>>> from sage.all import *
>>> R = Qp(Integer(7),Integer(5), 'capped-rel')
>>> R.print_mode()
'series'
```

residue_characteristic()

Return the prime, i.e., the characteristic of the residue field.

OUTPUT: the characteristic of the residue field

EXAMPLES:

```
sage: R = Zp(3,5,'fixed-mod')
sage: R.residue_characteristic()
3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(5),'fixed-mod')
>>> R.residue_characteristic()
3
```

residue_class_field()

Return the residue class field.

EXAMPLES:

```
sage: R = Zp(3,5,'fixed-mod')
sage: k = R.residue_class_field()
sage: k
Finite Field of size 3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(5),'fixed-mod')
>>> k = R.residue_class_field()
>>> k
Finite Field of size 3
```

residue_field()

Return the residue class field.

EXAMPLES:

```
sage: R = Zp(3,5,'fixed-mod')
sage: k = R.residue_field()
sage: k
Finite Field of size 3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(5),'fixed-mod')
>>> k = R.residue_field()
>>> k
Finite Field of size 3
```

residue_ring(*n*)

Return the quotient of the ring of integers by the `n`-th power of the maximal ideal.

EXAMPLES:

```
sage: R = Zp(11)
sage: R.residue_ring(3)
Ring of integers modulo 1331
```

```
>>> from sage.all import *
>>> R = Zp(Integer(11))
>>> R.residue_ring(Integer(3))
Ring of integers modulo 1331
```

residue_system()

Return a list of elements representing all the residue classes.

EXAMPLES:

```
sage: R = Zp(3, 5,'fixed-mod')
sage: R.residue_system()
[0, 1, 2]
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(5),'fixed-mod')
>>> R.residue_system()
[0, 1, 2]
```

roots_of_unity(*n=None*)

Return all the `n`-th roots of unity in this ring.

INPUT:

- `n` – integer or `None` (default: `None`); if `None`, the full group of roots of unity is returned

EXAMPLES:

```
sage: R = Zp(5, 10)
sage: roots = R.roots_of_unity(); roots
[1 + O(5^10),
 2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 +␣
↪O(5^10),
```

(continues on next page)

(continued from previous page)

```
 3 + 3*5 + 2*5^2 + 3*5^3 + 5^4 + 2*5^6 + 5^7 + 4*5^8 + 5^9 + O(5^10)]

sage: R.roots_of_unity(10)
[1 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 +␣
↪O(5^10)]
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(10))
>>> roots = R.roots_of_unity(); roots
[1 + O(5^10),
 2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 +␣
↪O(5^10),
 3 + 3*5 + 2*5^2 + 3*5^3 + 5^4 + 2*5^6 + 5^7 + 4*5^8 + 5^9 + O(5^10)]

>>> R.roots_of_unity(Integer(10))
[1 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 +␣
↪O(5^10)]
```

In this case, the roots of unity are the Teichmüller representatives:

```
sage: R.teichmuller_system()
[1 + O(5^10),
 2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10),
 3 + 3*5 + 2*5^2 + 3*5^3 + 5^4 + 2*5^6 + 5^7 + 4*5^8 + 5^9 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 +␣
↪O(5^10)]
```

```
>>> from sage.all import *
>>> R.teichmuller_system()
[1 + O(5^10),
 2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10),
 3 + 3*5 + 2*5^2 + 3*5^3 + 5^4 + 2*5^6 + 5^7 + 4*5^8 + 5^9 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 +␣
↪O(5^10)]
```

In general, there might be more roots of unity (it happens when the ring has non trivial p-th roots of unity):

```
sage: W.<a> = Zq(3^2, 2)
sage: S.<x> = W[]
sage: R.<pi> = W.extension((x+1)^2 + (x+1) + 1)
sage: roots = R.roots_of_unity(); roots
[1 + O(pi^4),
 a + 2*a*pi + 2*a*pi^2 + a*pi^3 + O(pi^4),
 ...
 1 + pi + O(pi^4),
 a + a*pi^2 + 2*a*pi^3 + O(pi^4),
 ...
 1 + 2*pi + pi^2 + O(pi^4),
 a + a*pi + a*pi^2 + O(pi^4),
```

(continues on next page)

(continued from previous page)

```
 ...]
sage: len(roots)
24
```

```
>>> from sage.all import *
>>> W = Zq(Integer(3)**Integer(2), Integer(2), names=('a',)); (a,) = W._first_
↪ngens(1)
>>> S = W['x']; (x,) = S._first_ngens(1)
>>> R = W.extension((x+Integer(1))**Integer(2) + (x+Integer(1)) + Integer(1),␣
↪names=('pi',)); (pi,) = R._first_ngens(1)
>>> roots = R.roots_of_unity(); roots
[1 + O(pi^4),
 a + 2*a*pi + 2*a*pi^2 + a*pi^3 + O(pi^4),
 ...
 1 + pi + O(pi^4),
 a + a*pi^2 + 2*a*pi^3 + O(pi^4),
 ...
 1 + 2*pi + pi^2 + O(pi^4),
 a + a*pi + a*pi^2 + O(pi^4),
 ...]
>>> len(roots)
24
```

We check that the logarithm of each root of unity vanishes:

```
sage: for root in roots:
....:     if root.log() != 0:
....:         raise ValueError
```

```
>>> from sage.all import *
>>> for root in roots:
...     if root.log() != Integer(0):
...         raise ValueError
```

some_elements()

Return a list of elements in this ring.

This is typically used for running generic tests (see `TestSuite`).

EXAMPLES:

```
sage: Zp(2,4).some_elements()
[0, 1 + O(2^4), 2 + O(2^5), 1 + 2^2 + 2^3 + O(2^4), 2 + 2^2 + 2^3 + 2^4 + O(2^
↪5)]
```

```
>>> from sage.all import *
>>> Zp(Integer(2),Integer(4)).some_elements()
[0, 1 + O(2^4), 2 + O(2^5), 1 + 2^2 + 2^3 + O(2^4), 2 + 2^2 + 2^3 + 2^4 + O(2^
↪5)]
```

teichmuller(*x*, *prec=None*)

Return the Teichmüller representative of `x`.

- `x` – something that can be cast into `self`

OUTPUT: the Teichmüller lift of x

EXAMPLES:

```
sage: R = Zp(5, 10, 'capped-rel', 'series')
sage: R.teichmuller(2)
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
sage: R = Qp(5, 10,'capped-rel','series')
sage: R.teichmuller(2)
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
sage: R = Zp(5, 10, 'capped-abs', 'series')
sage: R.teichmuller(2)
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
sage: R = Zp(5, 10, 'fixed-mod', 'series')
sage: R.teichmuller(2)
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9

sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 +125*x - 5
sage: W.<w> = R.ext(f)
sage: y = W.teichmuller(3); y
3 + 3*w^5 + w^7 + 2*w^9 + 2*w^10 + 4*w^11 + w^12 + 2*w^13 + 3*w^15
 + 2*w^16 + 3*w^17 + w^18 + 3*w^19 + 3*w^20 + 2*w^21 + 2*w^22
 + 3*w^23 + 4*w^24 + O(w^25)
sage: y^5 == y
True
sage: g = x^3 + 3*x + 3
sage: A.<a> = R.ext(g)
sage: b = A.teichmuller(1 + 2*a - a^2); b
(4*a^2 + 2*a + 1) + 2*a*5 + (3*a^2 + 1)*5^2 + (a + 4)*5^3
 + (a^2 + a + 1)*5^4 + O(5^5)
sage: b^125 == b
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(10), 'capped-rel', 'series')
>>> R.teichmuller(Integer(2))
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
>>> R = Qp(Integer(5), Integer(10),'capped-rel','series')
>>> R.teichmuller(Integer(2))
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
>>> R = Zp(Integer(5), Integer(10), 'capped-abs', 'series')
>>> R.teichmuller(Integer(2))
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9 + O(5^10)
>>> R = Zp(Integer(5), Integer(10), 'fixed-mod', 'series')
>>> R.teichmuller(Integer(2))
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + 4*5^5 + 2*5^6 + 3*5^7 + 3*5^9

>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> y = W.teichmuller(Integer(3)); y
3 + 3*w^5 + w^7 + 2*w^9 + 2*w^10 + 4*w^11 + w^12 + 2*w^13 + 3*w^15
 + 2*w^16 + 3*w^17 + w^18 + 3*w^19 + 3*w^20 + 2*w^21 + 2*w^22
 + 3*w^23 + 4*w^24 + O(w^25)
>>> y**Integer(5) == y
True
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)
>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> b = A.teichmuller(Integer(1) + Integer(2)*a - a**Integer(2)); b
(4*a^2 + 2*a + 1) + 2*a*5 + (3*a^2 + 1)*5^2 + (a + 4)*5^3
 + (a^2 + a + 1)*5^4 + O(5^5)
>>> b**Integer(125) == b
True
```

We check that Issue #23736 is resolved:

```
sage: R.teichmuller(GF(5)(2))
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R.teichmuller(GF(Integer(5))(Integer(2)))
2 + 5 + 2*5^2 + 5^3 + 3*5^4 + O(5^5)
```

AUTHORS:

- Initial version: David Roe
- Quadratic time version: Kiran Kedlaya <kedlaya@math.mit.edu> (2007-03-27)

teichmuller_system()

Return a set of Teichmüller representatives for the invertible elements of $\mathbf{Z}/p\mathbf{Z}$.

OUTPUT:

A list of Teichmüller representatives for the invertible elements of $\mathbf{Z}/p\mathbf{Z}$.

EXAMPLES:

```
sage: R = Zp(3, 5,'fixed-mod', 'terse')
sage: R.teichmuller_system()
[1, 242]
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(5),'fixed-mod', 'terse')
>>> R.teichmuller_system()
[1, 242]
```

Check that Issue #20457 is fixed:

```
sage: F.<a> = Qq(5^2,6)
sage: F.teichmuller_system()[3]
(2*a + 2) + (4*a + 1)*5 + 4*5^2 + (2*a + 1)*5^3 + (4*a + 1)*5^4 + (2*a + 3)*5^
↪5 + O(5^6)
```

```
>>> from sage.all import *
>>> F = Qq(Integer(5)**Integer(2),Integer(6), names=('a',)); (a,) = F._first_
↪ngens(1)
>>> F.teichmuller_system()[Integer(3)]
(2*a + 2) + (4*a + 1)*5 + 4*5^2 + (2*a + 1)*5^3 + (4*a + 1)*5^4 + (2*a + 3)*5^
↪5 + O(5^6)
```

Note

Should this return 0 as well?

uniformizer_pow(n)

Return p^n, as an element of `self`.

If `n` is infinity, returns 0.

EXAMPLES:

```
sage: R = Zp(3, 5, 'fixed-mod')
sage: R.uniformizer_pow(3)
3^3
sage: R.uniformizer_pow(infinity)
0
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(5), 'fixed-mod')
>>> R.uniformizer_pow(Integer(3))
3^3
>>> R.uniformizer_pow(infinity)
0
```

valuation()

Return the p-adic valuation on this ring.

OUTPUT:

A valuation that is normalized such that the rational prime p has valuation 1.

EXAMPLES:

```
sage: K = Qp(3)
sage: R.<a> = K[]
sage: L.<a> = K.extension(a^3 - 3)
sage: v = L.valuation(); v
3-adic valuation
sage: v(3)
1
sage: L(3).valuation()
3
```

```
>>> from sage.all import *
>>> K = Qp(Integer(3))
>>> R = K['a']; (a,) = R._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> L = K.extension(a**Integer(3) - Integer(3), names=('a',)); (a,) = L._
↪first_ngens(1)
>>> v = L.valuation(); v
3-adic valuation
>>> v(Integer(3))
1
>>> L(Integer(3)).valuation()
3
```

The normalization is chosen such that the valuation restricts to the valuation on the base ring:

```
sage: v(3) == K.valuation()(3)
True
sage: v.restriction(K) == K.valuation()
True
```

```
>>> from sage.all import *
>>> v(Integer(3)) == K.valuation()(Integer(3))
True
>>> v.restriction(K) == K.valuation()
True
```

See also

`NumberField_generic.valuation()`, `Order.valuation()`

CHAPTER

FIVE

p-ADIC GENERIC NODES

This file contains a bunch of intermediate classes for the p-adic parents, allowing a function to be implemented at the right level of generality.

AUTHORS:

- David Roe

class sage.rings.padics.generic_nodes.**CappedAbsoluteGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *LocalGeneric*

is_capped_absolute()

Return whether this p-adic ring bounds precision in a capped absolute fashion.

The absolute precision of an element is the power of p modulo which that element is defined. In a capped absolute ring, the absolute precision of elements are bounded by a constant depending on the ring.

EXAMPLES:

```
sage: R = ZpCA(5, 15)
sage: R.is_capped_absolute()
True
sage: R(5^7)
5^7 + O(5^15)
sage: S = Zp(5, 15)
sage: S.is_capped_absolute()
False
sage: S(5^7)
5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5), Integer(15))
>>> R.is_capped_absolute()
True
>>> R(Integer(5)**Integer(7))
5^7 + O(5^15)
>>> S = Zp(Integer(5), Integer(15))
>>> S.is_capped_absolute()
False
>>> S(Integer(5)**Integer(7))
5^7 + O(5^22)
```

class sage.rings.padics.generic_nodes.**CappedRelativeFieldGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *CappedRelativeGeneric*

class sage.rings.padics.generic_nodes.**CappedRelativeGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *LocalGeneric*

is_capped_relative()

Return whether this p-adic ring bounds precision in a capped relative fashion.

The relative precision of an element is the power of p modulo which the unit part of that element is defined. In a capped relative ring, the relative precision of elements are bounded by a constant depending on the ring.

EXAMPLES:

```
sage: R = ZpCA(5, 15)
sage: R.is_capped_relative()
False
sage: R(5^7)
5^7 + O(5^15)
sage: S = Zp(5, 15)
sage: S.is_capped_relative()
True
sage: S(5^7)
5^7 + O(5^22)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5), Integer(15))
>>> R.is_capped_relative()
False
>>> R(Integer(5)**Integer(7))
5^7 + O(5^15)
>>> S = Zp(Integer(5), Integer(15))
>>> S.is_capped_relative()
True
>>> S(Integer(5)**Integer(7))
5^7 + O(5^22)
```

class sage.rings.padics.generic_nodes.**CappedRelativeRingGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *CappedRelativeGeneric*

class sage.rings.padics.generic_nodes.**FixedModGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *LocalGeneric*

is_fixed_mod()

Return whether this p-adic ring bounds precision in a fixed modulus fashion.

The absolute precision of an element is the power of p modulo which that element is defined. In a fixed modulus ring, the absolute precision of every element is defined to be the precision cap of the parent. This means that some operations, such as division by p, don't return a well defined answer.

EXAMPLES:

```
sage: R = ZpFM(5,15)
sage: R.is_fixed_mod()
True
sage: R(5^7,absprec=9)
5^7
sage: S = ZpCA(5, 15)
sage: S.is_fixed_mod()
False
sage: S(5^7,absprec=9)
5^7 + O(5^9)
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(15))
>>> R.is_fixed_mod()
True
>>> R(Integer(5)**Integer(7),absprec=Integer(9))
5^7
>>> S = ZpCA(Integer(5), Integer(15))
>>> S.is_fixed_mod()
False
>>> S(Integer(5)**Integer(7),absprec=Integer(9))
5^7 + O(5^9)
```

class sage.rings.padics.generic_nodes.**FloatingPointFieldGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *FloatingPointGeneric*

class sage.rings.padics.generic_nodes.**FloatingPointGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *LocalGeneric*

is_floating_point()

Return whether this p-adic ring uses a floating point precision model.

Elements in the floating point model are stored by giving a valuation and a unit part. Arithmetic is done where the unit part is truncated modulo a fixed power of the uniformizer, stored in the precision cap of the parent.

EXAMPLES:

```
sage: R = ZpFP(5,15)
sage: R.is_floating_point()
True
sage: R(5^7,absprec=9)
5^7
sage: S = ZpCR(5,15)
sage: S.is_floating_point()
False
sage: S(5^7,absprec=9)
5^7 + O(5^9)
```

```
>>> from sage.all import *
>>> R = ZpFP(Integer(5),Integer(15))
>>> R.is_floating_point()
```

(continues on next page)

(continued from previous page)

```
True
>>> R(Integer(5)**Integer(7),absprec=Integer(9))
5^7
>>> S = ZpCR(Integer(5),Integer(15))
>>> S.is_floating_point()
False
>>> S(Integer(5)**Integer(7),absprec=Integer(9))
5^7 + O(5^9)
```

class sage.rings.padics.generic_nodes.**FloatingPointRingGeneric**(*base*, *prec*, *names*, *element_class*, *category=None*)

Bases: *FloatingPointGeneric*

class sage.rings.padics.generic_nodes.**pAdicCappedAbsoluteRingGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicRingGeneric*, *CappedAbsoluteGeneric*

class sage.rings.padics.generic_nodes.**pAdicCappedRelativeFieldGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicFieldGeneric*, *CappedRelativeFieldGeneric*

class sage.rings.padics.generic_nodes.**pAdicCappedRelativeRingGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicRingGeneric*, *CappedRelativeRingGeneric*

class sage.rings.padics.generic_nodes.**pAdicFieldBaseGeneric**(*p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicBaseGeneric*, *pAdicFieldGeneric*

composite(*subfield1*, *subfield2*)

Return the composite of two subfields of `self`, i.e., the largest subfield containing both

INPUT:

- `self` – a p-adic field
- `subfield1` – a subfield
- `subfield2` – a subfield

OUTPUT: the composite of `subfield1` and `subfield2`

EXAMPLES:

```
sage: K = Qp(17); K.composite(K, K) is K
True
```

```
>>> from sage.all import *
>>> K = Qp(Integer(17)); K.composite(K, K) is K
True
```

construction(*forbid_frac_field=False*)

Return the functorial construction of `self`, namely, completion of the rational numbers with respect a given prime.

Also preserves other information that makes this field unique (e.g., precision, rounding, print mode).

INPUT:

- `forbid_frac_field` – require a completion functor rather than a fraction field functor. This is used in the *sage.rings.padics.local_generic.LocalGeneric.change()* method.

EXAMPLES:

```
sage: K = Qp(17, 8, print_mode='val-unit', print_sep='&')
sage: c, L = K.construction(); L
17-adic Ring with capped relative precision 8
sage: c
FractionField
sage: c(L)
17-adic Field with capped relative precision 8
sage: K == c(L)
True
```

```
>>> from sage.all import *
>>> K = Qp(Integer(17), Integer(8), print_mode='val-unit', print_sep='&')
>>> c, L = K.construction(); L
17-adic Ring with capped relative precision 8
>>> c
FractionField
>>> c(L)
17-adic Field with capped relative precision 8
>>> K == c(L)
True
```

We can get a completion functor by forbidding the fraction field:

```
sage: c, L = K.construction(forbid_frac_field=True); L
Rational Field
sage: c
Completion[17, prec=8]
sage: c(L)
17-adic Field with capped relative precision 8
sage: K == c(L)
True
```

```
>>> from sage.all import *
>>> c, L = K.construction(forbid_frac_field=True); L
Rational Field
>>> c
Completion[17, prec=8]
>>> c(L)
17-adic Field with capped relative precision 8
>>> K == c(L)
True
```

subfield(*list*)

Return the subfield generated by the elements in `list`.

INPUT:

- `self` – a p-adic field
- `list` – list of elements of `self`

OUTPUT: the subfield of `self` generated by the elements of `list`

EXAMPLES:

```
sage: K = Qp(17); K.subfield([K(17), K(1827)]) is K
True
```

```
>>> from sage.all import *
>>> K = Qp(Integer(17)); K.subfield([K(Integer(17)), K(Integer(1827))]) is K
True
```

subfields_of_degree(*n*)

Return the number of subfields of `self` of degree n.

INPUT:

- `self` – a p-adic field
- `n` – integer

OUTPUT:

integer – the number of subfields of degree n over `self.base_ring()`

EXAMPLES:

```
sage: K = Qp(17)
sage: K.subfields_of_degree(1)
1
```

```
>>> from sage.all import *
>>> K = Qp(Integer(17))
>>> K.subfields_of_degree(Integer(1))
1
```

class sage.rings.padics.generic_nodes.**pAdicFieldGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicGeneric*, pAdicField

is_field(*proof=True*)

Return whether this ring is actually a field, ie `True`.

EXAMPLES:

```
sage: Qp(5).is_field()
True
```

```
>>> from sage.all import *
>>> Qp(Integer(5)).is_field()
True
```

class sage.rings.padics.generic_nodes.**pAdicFixedModRingGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicRingGeneric*, *FixedModGeneric*

class sage.rings.padics.generic_nodes.**pAdicFloatingPointFieldGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicFieldGeneric*, *FloatingPointFieldGeneric*

class sage.rings.padics.generic_nodes.**pAdicFloatingPointRingGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicRingGeneric*, *FloatingPointRingGeneric*

class sage.rings.padics.generic_nodes.**pAdicLatticeGeneric**(*p*, *prec*, *print_mode*, *names*, *label=None*, *category=None*)

Bases: *pAdicGeneric*

An implementation of the p-adic rationals with lattice precision.

INPUT:

- `p` – the underlying prime number
- `prec` – the precision
- `subtype` – either `'cap'` or `'float'`, specifying the precision model used for tracking precision
- `label` – string or `None` (default: `None`)

convert_multiple(**elts*)

Convert a list of elements to this parent.

NOTE:

This function tries to be sharp on precision as much as possible. In particular, if the precision of the input elements are handled by a lattice, diffused digits of precision are preserved during the conversion.

EXAMPLES:

```
sage: R = ZpLC(2)
sage: x = R(1, 10); y = R(1, 5)
sage: x,y = x+y, x-y
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2))
>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> x,y = x+y, x-y
```

Remark that the pair (x, y) has diffused digits of precision:

```
sage: x
2 + O(2^5)
sage: y
O(2^5)
sage: x + y
```

(continues on next page)

(continued from previous page)

```
2 + O(2^11)

sage: R.precision().diffused_digits([x,y])                                    #␣
↪needs sage.geometry.polyhedron
6
```

```
>>> from sage.all import *
>>> x
2 + O(2^5)
>>> y
O(2^5)
>>> x + y
2 + O(2^11)

>>> R.precision().diffused_digits([x,y])                                      #␣
↪needs sage.geometry.polyhedron
6
```

As a consequence, if we convert `x` and `y` separately, we lose some precision:

```
sage: R2 = ZpLC(2, label='copy')
sage: x2 = R2(x); y2 = R2(y)
sage: x2
2 + O(2^5)
sage: y2
O(2^5)
sage: x2 + y2
2 + O(2^5)

sage: R2.precision().diffused_digits([x2,y2])                                 #␣
↪needs sage.geometry.polyhedron
0
```

```
>>> from sage.all import *
>>> R2 = ZpLC(Integer(2), label='copy')
>>> x2 = R2(x); y2 = R2(y)
>>> x2
2 + O(2^5)
>>> y2
O(2^5)
>>> x2 + y2
2 + O(2^5)

>>> R2.precision().diffused_digits([x2,y2])                                   #␣
↪needs sage.geometry.polyhedron
0
```

On the other hand, this issue disappears when we use multiple conversion:

```
sage: x2,y2 = R2.convert_multiple(x,y)                                        #␣
↪needs sage.geometry.polyhedron
sage: x2 + y2                                                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.padics
2 + O(2^11)

sage: R2.precision().diffused_digits([x2,y2])                                  #␣
↪needs sage.geometry.polyhedron
6
```

```
>>> from sage.all import *
>>> x2,y2 = R2.convert_multiple(x,y)                                           #␣
↪needs sage.geometry.polyhedron
>>> x2 + y2                                                                    #␣
↪needs sage.rings.padics
2 + O(2^11)

>>> R2.precision().diffused_digits([x2,y2])                                    #␣
↪needs sage.geometry.polyhedron
6
```

is_lattice_prec()

Return whether this p-adic ring bounds precision using a lattice model.

In lattice precision, relationships between elements are stored in a precision object of the parent, which allows for optimal precision tracking at the cost of increased memory usage and runtime.

EXAMPLES:

```
sage: R = ZpCR(5, 15)
sage: R.is_lattice_prec()
False
sage: x = R(25, 8)
sage: x - x
O(5^8)
sage: S = ZpLC(5, 15)
sage: S.is_lattice_prec()
True
sage: x = S(25, 8)
sage: x - x
O(5^30)
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5), Integer(15))
>>> R.is_lattice_prec()
False
>>> x = R(Integer(25), Integer(8))
>>> x - x
O(5^8)
>>> S = ZpLC(Integer(5), Integer(15))
>>> S.is_lattice_prec()
True
>>> x = S(Integer(25), Integer(8))
>>> x - x
O(5^30)
```

label()

Return the label of this parent.

NOTE:

Labels can be used to distinguish between parents with the same defining data.

They are useful in the lattice precision framework in order to limit the size of the lattice modeling the precision (which is roughly the number of elements having this parent).

Elements of a parent with some label do not coerce to a parent with a different label. However conversions are allowed.

EXAMPLES:

```
sage: R = ZpLC(5)
sage: R.label()  # no label by default

sage: R = ZpLC(5, label='mylabel')
sage: R.label()
'mylabel'
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(5))
>>> R.label()  # no label by default

>>> R = ZpLC(Integer(5), label='mylabel')
>>> R.label()
'mylabel'
```

Labels are typically useful to isolate computations. For example, assume that we first want to do some calculations with matrices:

```
sage: R = ZpLC(5, label='matrices')
sage: M = random_matrix(R, 4, 4)                                                #␣
↪needs sage.geometry.polyhedron
sage: d = M.determinant()                                                       #␣
↪needs sage.geometry.polyhedron
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(5), label='matrices')
>>> M = random_matrix(R, Integer(4), Integer(4))                                 ␣
↪              # needs sage.geometry.polyhedron
>>> d = M.determinant()                                                        #␣
↪needs sage.geometry.polyhedron
```

Now, if we want to do another unrelated computation, we can use a different label:

```
sage: R = ZpLC(5, label='polynomials')
sage: S.<x> = PolynomialRing(R)
sage: P = (x-1)*(x-2)*(x-3)*(x-4)*(x-5)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(5), label='polynomials')
>>> S = PolynomialRing(R, names=('x',)); (x,) = S._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> P = (x-Integer(1))*(x-Integer(2))*(x-Integer(3))*(x-Integer(4))*(x-
↪Integer(5))
```

Without labels, the software would have modeled the precision on the matrices and on the polynomials using the same lattice (manipulating a lattice of higher dimension can have a significant impact on performance).

precision()

Return the lattice precision object attached to this parent.

EXAMPLES:

```
sage: R = ZpLC(5, label='precision')
sage: R.precision()
Precision lattice on 0 objects (label: precision)

sage: x = R(1, 10); y = R(1, 5)
sage: R.precision()
Precision lattice on 2 objects (label: precision)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(5), label='precision')
>>> R.precision()
Precision lattice on 0 objects (label: precision)

>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> R.precision()
Precision lattice on 2 objects (label: precision)
```

See also

PrecisionLattice

precision_cap()

Return the relative precision cap for this ring if it is finite. Otherwise return the absolute precision cap.

EXAMPLES:

```
sage: R = ZpLC(3)
sage: R.precision_cap()
20
sage: R.precision_cap_relative()
20

sage: R = ZpLC(3, prec=(infinity,20))
sage: R.precision_cap()
20
sage: R.precision_cap_relative()
+Infinity
sage: R.precision_cap_absolute()
20
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(3))
>>> R.precision_cap()
20
>>> R.precision_cap_relative()
20

>>> R = ZpLC(Integer(3), prec=(infinity,Integer(20)))
>>> R.precision_cap()
20
>>> R.precision_cap_relative()
+Infinity
>>> R.precision_cap_absolute()
20
```

See also

precision_cap_relative(), *precision_cap_absolute()*

precision_cap_absolute()

Return the absolute precision cap for this ring.

EXAMPLES:

```
sage: R = ZpLC(3)
sage: R.precision_cap_absolute()
40

sage: R = ZpLC(3, prec=(infinity,20))
sage: R.precision_cap_absolute()
20
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(3))
>>> R.precision_cap_absolute()
40

>>> R = ZpLC(Integer(3), prec=(infinity,Integer(20)))
>>> R.precision_cap_absolute()
20
```

See also

precision_cap(), *precision_cap_relative()*

precision_cap_relative()

Return the relative precision cap for this ring.

EXAMPLES:

```
sage: R = ZpLC(3)
sage: R.precision_cap_relative()
20

sage: R = ZpLC(3, prec=(infinity,20))
sage: R.precision_cap_relative()
+Infinity
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(3))
>>> R.precision_cap_relative()
20

>>> R = ZpLC(Integer(3), prec=(infinity,Integer(20)))
>>> R.precision_cap_relative()
+Infinity
```

See also

precision_cap(), *precision_cap_absolute()*

class sage.rings.padics.generic_nodes.**pAdicRelaxedGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicGeneric*

Generic class for relaxed p-adics.

INPUT:

- `p` – the underlying prime number
- `prec` – the default precision

an_element(*unbounded=False*)

Return an element in this ring.

EXAMPLES:

```
sage: R = ZpER(7, prec=5)                                                   #␣
↪needs sage.libs.flint
sage: R.an_element()                                                        #␣
↪needs sage.libs.flint
7 + O(7^5)
sage: R.an_element(unbounded=True)                                          #␣
↪needs sage.libs.flint
7 + ...
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(7), prec=Integer(5))                                    ␣
↪              # needs sage.libs.flint
>>> R.an_element()                                                         #␣
↪needs sage.libs.flint
7 + O(7^5)
```

(continues on next page)

(continued from previous page)

```
>>> R.an_element(unbounded=True)                                              #␣
↪needs sage.libs.flint
7 + ...
```

default_prec()

Return the default precision of this relaxed p-adic ring.

The default precision is mostly used for printing: it is the number of digits which are printed for unbounded elements (that is elements having infinite absolute precision).

EXAMPLES:

```
sage: R = ZpER(5, print_mode='digits')
sage: R.default_prec()
20
sage: R(1/17)
...34024323104201213403
sage: S = ZpER(5, prec=10, print_mode='digits')
sage: S.default_prec()
10
sage: S(1/17)
...4201213403
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5), print_mode='digits')
>>> R.default_prec()
20
>>> R(Integer(1)/Integer(17))
...34024323104201213403
>>> S = ZpER(Integer(5), prec=Integer(10), print_mode='digits')
>>> S.default_prec()
10
>>> S(Integer(1)/Integer(17))
...4201213403
```

halting_prec()

Return the default halting precision of this relaxed p-adic ring.

The halting precision is the precision at which elements of this parent are compared (unless more digits have been previously computed). By default, it is twice the default precision.

EXAMPLES:

```
sage: R = ZpER(5, print_mode='digits')                                      #␣
↪needs sage.libs.flint
sage: R.halting_prec()                                                      #␣
↪needs sage.libs.flint
40
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5), print_mode='digits')                                ␣
↪     # needs sage.libs.flint
>>> R.halting_prec()                                                        #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.flint
40
```

is_relaxed()

Return whether this p-adic ring is relaxed.

EXAMPLES:

```
sage: R = Zp(5)
sage: R.is_relaxed()
False
sage: S = ZpER(5)                                                    #␣
↪needs sage.libs.flint
sage: S.is_relaxed()                                                 #␣
↪needs sage.libs.flint
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.is_relaxed()
False
>>> S = ZpER(Integer(5))                                                ␣
↪     # needs sage.libs.flint
>>> S.is_relaxed()                                                  #␣
↪needs sage.libs.flint
True
```

is_secure()

Return `False` if this p-adic relaxed ring is not secure (i.e., if indistinguishable elements at the working precision are considered as equal); `True` otherwise (in which case, an error is raised when equality cannot be decided).

EXAMPLES:

```
sage: R = ZpER(5)
sage: R.is_secure()
False
sage: x = R(20/21)
sage: y = x + 5^50
sage: x == y
True

sage: S = ZpER(5, secure=True)
sage: S.is_secure()
True
sage: x = S(20/21)
sage: y = x + 5^50
sage: x == y
Traceback (most recent call last):
...
PrecisionError: unable to decide equality; try to bound precision
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5))
>>> R.is_secure()
False
>>> x = R(Integer(20)/Integer(21))
>>> y = x + Integer(5)**Integer(50)
>>> x == y
True

>>> S = ZpER(Integer(5), secure=True)
>>> S.is_secure()
True
>>> x = S(Integer(20)/Integer(21))
>>> y = x + Integer(5)**Integer(50)
>>> x == y
Traceback (most recent call last):
...
PrecisionError: unable to decide equality; try to bound precision
```

precision_cap()

Return the precision cap of this p-adic ring, which is infinite in the case of relaxed rings.

EXAMPLES:

```
sage: R = ZpER(5)                                                              #␣
↪needs sage.libs.flint
sage: R.precision_cap()                                                        #␣
↪needs sage.libs.flint
+Infinity
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5))                                                        ␣
↪     # needs sage.libs.flint
>>> R.precision_cap()                                                         #␣
↪needs sage.libs.flint
+Infinity
```

random_element(*integral=False*, *prec=None*)

Return a random element in this ring.

INPUT:

- `integral` – boolean (default: `False`); if `True`, return a random element in the ring of integers of this ring
- `prec` – integer or `None` (default: `None`); if given, bound the precision of the output to `prec`

EXAMPLES:

```
sage: R = ZpER(5, prec=10)                                                     #␣
↪needs sage.libs.flint
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5), prec=Integer(10))                                      ␣
↪              # needs sage.libs.flint
```

By default, this method returns a unbounded element:

```
sage: a = R.random_element()                                                #␣
↪needs sage.libs.flint
sage: a  # random                                                           #␣
↪needs sage.libs.flint
4 + 3*5 + 3*5^2 + 5^3 + 3*5^4 + 2*5^5 + 2*5^6 + 5^7 + 5^9 + ...
sage: a.precision_absolute()                                                #␣
↪needs sage.libs.flint
+Infinity
```

```
>>> from sage.all import *
>>> a = R.random_element()                                                  #␣
↪needs sage.libs.flint
>>> a  # random                                                             #␣
↪needs sage.libs.flint
4 + 3*5 + 3*5^2 + 5^3 + 3*5^4 + 2*5^5 + 2*5^6 + 5^7 + 5^9 + ...
>>> a.precision_absolute()                                                  #␣
↪needs sage.libs.flint
+Infinity
```

The precision can be bounded by passing in a precision:

```
sage: b = R.random_element(prec=15)                                         #␣
↪needs sage.libs.flint
sage: b  # random                                                           #␣
↪needs sage.libs.flint
2 + 3*5^2 + 5^3 + 3*5^4 + 5^5 + 3*5^6 + 3*5^8 + 3*5^9 + 4*5^10
 + 5^11 + 4*5^12 + 5^13 + 2*5^14 + O(5^15)
sage: b.precision_absolute()                                                #␣
↪needs sage.libs.flint
15
```

```
>>> from sage.all import *
>>> b = R.random_element(prec=Integer(15))                                  ␣
↪     # needs sage.libs.flint
>>> b  # random                                                             #␣
↪needs sage.libs.flint
2 + 3*5^2 + 5^3 + 3*5^4 + 5^5 + 3*5^6 + 3*5^8 + 3*5^9 + 4*5^10
 + 5^11 + 4*5^12 + 5^13 + 2*5^14 + O(5^15)
>>> b.precision_absolute()                                                  #␣
↪needs sage.libs.flint
15
```

some_elements(*unbounded=False*)

Return a list of elements in this ring.

This is typically used for running generic tests (see `TestSuite`).

EXAMPLES:

```
sage: R = ZpER(7, prec=5)                                                   #␣
↪needs sage.libs.flint
sage: R.some_elements()                                                     #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.flint
[O(7^5),
 1 + O(7^5),
 7 + O(7^5),
 7 + O(7^5),
 1 + 5*7 + 3*7^2 + 6*7^3 + O(7^5),
 7 + 6*7^2 + 6*7^3 + 6*7^4 + O(7^5)]

sage: R.some_elements(unbounded=True)                                    #␣
↪needs sage.libs.flint
[0,
 1 + ...,
 7 + ...,
 7 + ...,
 1 + 5*7 + 3*7^2 + 6*7^3 + ...,
 7 + 6*7^2 + 6*7^3 + 6*7^4 + ...]
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(7), prec=Integer(5))                                ␣
↪              # needs sage.libs.flint
>>> R.some_elements()                                                   #␣
↪needs sage.libs.flint
[O(7^5),
 1 + O(7^5),
 7 + O(7^5),
 7 + O(7^5),
 1 + 5*7 + 3*7^2 + 6*7^3 + O(7^5),
 7 + 6*7^2 + 6*7^3 + 6*7^4 + O(7^5)]

>>> R.some_elements(unbounded=True)                                     #␣
↪needs sage.libs.flint
[0,
 1 + ...,
 7 + ...,
 7 + ...,
 1 + 5*7 + 3*7^2 + 6*7^3 + ...,
 7 + 6*7^2 + 6*7^3 + 6*7^4 + ...]
```

teichmuller(x)

Return the Teichmuller representative of x.

EXAMPLES:

```
sage: R = ZpER(5, print_mode='digits')                                   #␣
↪needs sage.libs.flint
sage: R.teichmuller(2)                                                   #␣
↪needs sage.libs.flint
...40423140223032431212
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5), print_mode='digits')                            ␣
↪     # needs sage.libs.flint
```

(continues on next page)

(continued from previous page)

```
>>> R.teichmuller(Integer(2))                                                  ␣
↪     # needs sage.libs.flint
...40423140223032431212
```

teichmuller_system()

Return a set of teichmuller representatives for the invertible elements of $\mathbf{Z}/p\mathbf{Z}$.

EXAMPLES:

```
sage: R = ZpER(7, print_mode='digits')                                       #␣
↪needs sage.libs.flint
sage: R.teichmuller_system()                                                 #␣
↪needs sage.libs.flint
[...0000000000000000001,
 ...1641212544342620 3642,
 ...1641212544342620 3643,
 ...5025454122324046 3024,
 ...5025454122324046 3025,
 ...6666666666666666666]
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(7), print_mode='digits')                                 ␣
↪     # needs sage.libs.flint
>>> R.teichmuller_system()                                                  #␣
↪needs sage.libs.flint
[...0000000000000000001,
 ...1641212544342620 3642,
 ...1641212544342620 3643,
 ...5025454122324046 3024,
 ...5025454122324046 3025,
 ...6666666666666666666]
```

unknown(*start_val=0*, *digits=None*)

Return a self-referent number in this ring.

INPUT:

- `start_val` – integer (default: 0); a lower bound on the valuation of the returned element
- `digits` – an element, a list, or `None` (default: `None`); the first digit or the list of the digits of the returned element

NOTE:

Self-referent numbers are numbers whose digits are defined in terms of the previous ones. This method is used to declare a self-referent number (and optionally, to set its first digits). The definition of the number itself will be given afterwords using to method `sage.rings.padics.relaxed_template.RelaxedElement_unknown.set` of the element.

EXAMPLES:

sage: R = ZpER(5, prec=10)

We declare a self-referent number:

```
sage: a = R.unknown()
```

```
>>> from sage.all import *
>>> a = R.unknown()
```

So far, we do not know anything on a (except that it has nonnegative valuation):

```
sage: a
O(5^0)
```

```
>>> from sage.all import *
>>> a
O(5^0)
```

We can now use the method `sage.rings.padics.relaxed_template.RelaxedElement_unknown.set` to define a. Below, for example, we say that the digits of a have to agree with the digits of $1 + 5a$. Note that the factor 5 shifts the digits; the n-th digit of a is then defined by the previous ones:

```
sage: a.set(1 + 5*a)
True
```

```
>>> from sage.all import *
>>> a.set(Integer(1) + Integer(5)*a)
True
```

After this, a contains the solution of the equation $a = 1 + 5a$, that is $a = -1/4$:

```
sage: a
1 + 5 + 5^2 + 5^3 + 5^4 + 5^5 + 5^6 + 5^7 + 5^8 + 5^9 + ...
```

```
>>> from sage.all import *
>>> a
1 + 5 + 5^2 + 5^3 + 5^4 + 5^5 + 5^6 + 5^7 + 5^8 + 5^9 + ...
```

Here is another example with an equation of degree 2:

```
sage: b = R.unknown()
sage: b.set(1 - 5*b^2)
True
sage: b
1 + 4*5 + 5^2 + 3*5^4 + 4*5^6 + 4*5^8 + 2*5^9 + ...
sage: (sqrt(R(21)) - 1) / 10
1 + 4*5 + 5^2 + 3*5^4 + 4*5^6 + 4*5^8 + 2*5^9 + ...
```

```
>>> from sage.all import *
>>> b = R.unknown()
>>> b.set(Integer(1) - Integer(5)*b**Integer(2))
True
>>> b
1 + 4*5 + 5^2 + 3*5^4 + 4*5^6 + 4*5^8 + 2*5^9 + ...
>>> (sqrt(R(Integer(21))) - Integer(1)) / Integer(10)
1 + 4*5 + 5^2 + 3*5^4 + 4*5^6 + 4*5^8 + 2*5^9 + ...
```

Cross self-referent definitions are also allowed:

```
sage: u = R.unknown()
sage: v = R.unknown()
sage: w = R.unknown()
sage: u.set(1 + 2*v + 3*w^2 + 5*u*v*w)
True
sage: v.set(2 + 4*w + sqrt(1 + 5*u + 10*v + 15*w))
True
sage: w.set(3 + 25*(u*v + v*w + u*w))
True
sage: u
3 + 3*5 + 4*5^2 + 5^3 + 3*5^4 + 5^5 + 5^6 + 3*5^7 + 5^8 + 3*5^9 + ...
sage: v
4*5 + 2*5^2 + 4*5^3 + 5^4 + 5^5 + 3*5^6 + 5^8 + 5^9 + ...
sage: w
3 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 2*5^6 + 5^8 + 5^9 + ...
```

```
>>> from sage.all import *
>>> u = R.unknown()
>>> v = R.unknown()
>>> w = R.unknown()
>>> u.set(Integer(1) + Integer(2)*v + Integer(3)*w**Integer(2) +␣
↪Integer(5)*u*v*w)
True
>>> v.set(Integer(2) + Integer(4)*w + sqrt(Integer(1) + Integer(5)*u +␣
↪Integer(10)*v + Integer(15)*w))
True
>>> w.set(Integer(3) + Integer(25)*(u*v + v*w + u*w))
True
>>> u
3 + 3*5 + 4*5^2 + 5^3 + 3*5^4 + 5^5 + 5^6 + 3*5^7 + 5^8 + 3*5^9 + ...
>>> v
4*5 + 2*5^2 + 4*5^3 + 5^4 + 5^5 + 3*5^6 + 5^8 + 5^9 + ...
>>> w
3 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 2*5^6 + 5^8 + 5^9 + ...
```

class sage.rings.padics.generic_nodes.**pAdicRingBaseGeneric**(*p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicBaseGeneric*, *pAdicRingGeneric*

construction(*forbid_frac_field=False*)

Return the functorial construction of `self`, namely, completion of the rational numbers with respect to a given prime.

Also preserves other information that makes this field unique (e.g., precision, rounding, print mode).

INPUT:

- `forbid_frac_field` – ignored, for compatibility with other p-adic types

EXAMPLES:

```
sage: K = Zp(17, 8, print_mode='val-unit', print_sep='&')
sage: c, L = K.construction(); L
Integer Ring
```

(continues on next page)

(continued from previous page)

```
sage: c(L)
17-adic Ring with capped relative precision 8
sage: K == c(L)
True
```

```
>>> from sage.all import *
>>> K = Zp(Integer(17), Integer(8), print_mode='val-unit', print_sep='&')
>>> c, L = K.construction(); L
Integer Ring
>>> c(L)
17-adic Ring with capped relative precision 8
>>> K == c(L)
True
```

random_element(*algorithm='default'*)

Return a random element of `self`, optionally using the `algorithm` argument to decide how it generates the element. Algorithms currently implemented:

- `'default'`: Choose a_i, $i \geq 0$, randomly between 0 and $p-1$ until a nonzero choice is made. Then continue choosing a_i randomly between 0 and $p-1$ until we reach precision_cap, and return $\sum a_i p^i$.

EXAMPLES:

```
sage: Zp(5,6).random_element().parent() is Zp(5,6)
True
sage: ZpCA(5,6).random_element().parent() is ZpCA(5,6)
True
sage: ZpFM(5,6).random_element().parent() is ZpFM(5,6)
True
```

```
>>> from sage.all import *
>>> Zp(Integer(5),Integer(6)).random_element().parent() is Zp(Integer(5),
↪Integer(6))
True
>>> ZpCA(Integer(5),Integer(6)).random_element().parent() is ZpCA(Integer(5),
↪Integer(6))
True
>>> ZpFM(Integer(5),Integer(6)).random_element().parent() is ZpFM(Integer(5),
↪Integer(6))
True
```

class sage.rings.padics.generic_nodes.**pAdicRingGeneric**(*base*, *p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicGeneric*, pAdicRing

is_field(*proof=True*)

Return whether this ring is actually a field, ie `False`.

EXAMPLES:

```
sage: Zp(5).is_field()
False
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).is_field()
False
```

krull_dimension()

Return the Krull dimension of self, i.e. 1.

INPUT:

- `self` – a p-adic ring

OUTPUT: the Krull dimension of `self`. Since `self` is a p-adic ring, this is 1.

EXAMPLES:

```
sage: Zp(5).krull_dimension()
1
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).krull_dimension()
1
```

CHAPTER

SIX

p-ADIC BASE GENERIC

A superclass for implementations of $\mathbf{Z}_p$ and $\mathbf{Q}_p$.

AUTHORS:

- David Roe

class sage.rings.padics.padic_base_generic.**pAdicBaseGeneric**(*p*, *prec*, *print_mode*, *names*, *element_class*, *category=None*)

Bases: *pAdicGeneric*

Initialization.

absolute_discriminant()

Return the absolute discriminant of this p-adic ring.

EXAMPLES:

```
sage: Zp(5).absolute_discriminant()
1
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).absolute_discriminant()
1
```

discriminant(*K=None*)

Return the discriminant of this p-adic ring over `K`.

INPUT:

- `self` – a p-adic ring
- `K` – a sub-ring of `self` or `None` (default: `None`)

OUTPUT:

integer; the discriminant of this ring over `K` (or the absolute discriminant if `K` is `None`)

EXAMPLES:

```
sage: Zp(5).discriminant()
1
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).discriminant()
1
```

exact_field()

Return the rational field.

For compatibility with extensions of p-adics.

EXAMPLES:

```
sage: Zp(5).exact_field()
Rational Field
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).exact_field()
Rational Field
```

exact_ring()

Return the integer ring.

EXAMPLES:

```
sage: Zp(5).exact_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).exact_ring()
Integer Ring
```

gen(n=*0*)

Return the n-th generator of this extension. For base rings/fields, we consider the generator to be the prime.

EXAMPLES:

```
sage: R = Zp(5); R.gen()
5 + O(5^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5)); R.gen()
5 + O(5^21)
```

has_pth_root()

Return whether or not $\mathbf{Z}_p$ has a primitive p-th root of unity.

EXAMPLES:

```
sage: Zp(2).has_pth_root()
True
sage: Zp(17).has_pth_root()
False
```

```
>>> from sage.all import *
>>> Zp(Integer(2)).has_pth_root()
True
>>> Zp(Integer(17)).has_pth_root()
False
```

has_root_of_unity(n)

Return whether or not $\mathbf{Z}_p$ has a primitive n-th root of unity.

INPUT:

- `self` – a p-adic ring
- `n` – integer

OUTPUT:

boolean; whether `self` has primitive n-th root of unity

EXAMPLES:

```
sage: R=Zp(37)
sage: R.has_root_of_unity(12)
True
sage: R.has_root_of_unity(11)
False
```

```
>>> from sage.all import *
>>> R=Zp(Integer(37))
>>> R.has_root_of_unity(Integer(12))
True
>>> R.has_root_of_unity(Integer(11))
False
```

is_abelian()

Return whether the Galois group is abelian, i.e. `True`. #should this be automorphism group?

EXAMPLES:

```
sage: R = Zp(3, 10,'fixed-mod'); R.is_abelian()
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(10),'fixed-mod'); R.is_abelian()
True
```

is_isomorphic(*ring*)

Return whether `self` and `ring` are isomorphic, i.e. whether `ring` is an implementation of $\mathbf{Z}_p$ for the same prime as `self`.

INPUT:

- `self` – a p-adic ring
- `ring` – a ring

OUTPUT: `boolean` – whether `ring` is an implementation of ZZ_p` for the same prime as `self`

EXAMPLES:

```
sage: R = Zp(5, 15, print_mode='digits'); S = Zp(5, 44, print_max_terms=4); R.
↪is_isomorphic(S)
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(15), print_mode='digits'); S = Zp(Integer(5),␣
↪Integer(44), print_max_terms=Integer(4)); R.is_isomorphic(S)
True
```

is_normal()

Return whether or not this is a normal extension, i.e. `True`.

EXAMPLES:

```
sage: R = Zp(3, 10,'fixed-mod'); R.is_normal()
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(10),'fixed-mod'); R.is_normal()
True
```

modulus(*exact=False*)

Return the polynomial defining this extension.

For compatibility with extension fields; we define the modulus to be $x - 1$.

INPUT:

- `exact` – boolean (default: `False`); whether to return a polynomial with integer entries

EXAMPLES:

```
sage: Zp(5).modulus(exact=True)
x
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).modulus(exact=True)
x
```

plot(*max_points=2500*, ***args*)

Create a visualization of this p-adic ring as a fractal similar to a generalization of the Sierpi'nski triangle.

The resulting image attempts to capture the algebraic and topological characteristics of $\mathbf{Z}_p$.

INPUT:

- `max_points` – the maximum number or points to plot, which controls the depth of recursion (default: 2500)
- `**args` – color, size, etc. that are passed to the underlying point graphics objects

REFERENCES:

- Cuoco, A. "Visualizing the p-adic Integers", The American Mathematical Monthly, Vol. 98, No. 4 (Apr., 1991), pp. 355-364

EXAMPLES:

```
sage: Zp(3).plot()                                                          #␣
↪needs sage.plot
Graphics object consisting of 1 graphics primitive
sage: Zp(5).plot(max_points=625)                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.plot
Graphics object consisting of 1 graphics primitive
sage: Zp(23).plot(rgbcolor=(1,0,0))                                    #␣
↪needs sage.plot
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> Zp(Integer(3)).plot()                                              ␣
↪     # needs sage.plot
Graphics object consisting of 1 graphics primitive
>>> Zp(Integer(5)).plot(max_points=Integer(625))                       ␣
↪              # needs sage.plot
Graphics object consisting of 1 graphics primitive
>>> Zp(Integer(23)).plot(rgbcolor=(Integer(1),Integer(0),Integer(0)))  ␣
↪                                 # needs sage.plot
Graphics object consisting of 1 graphics primitive
```

uniformizer()

Return a uniformizer for this ring.

EXAMPLES:

```
sage: R = Zp(3,5,'fixed-mod', 'series')
sage: R.uniformizer()
3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(5),'fixed-mod', 'series')
>>> R.uniformizer()
3
```

uniformizer_pow(n)

Return the n-th power of the uniformizer of `self` (as an element of `self`).

EXAMPLES:

```
sage: R = Zp(5)
sage: R.uniformizer_pow(5)
5^5 + O(5^25)
sage: R.uniformizer_pow(infinity)
0
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> R.uniformizer_pow(Integer(5))
5^5 + O(5^25)
>>> R.uniformizer_pow(infinity)
0
```

zeta(n=*None*)

Return a generator of the group of roots of unity.

INPUT:

- `self` – a p-adic ring
- `n` – integer or `None` (default: `None`)

OUTPUT:

`element`; a generator of the n-th roots of unity, or a generator of the full group of roots of unity if `n` is `None`

EXAMPLES:

```
sage: R = Zp(37,5)
sage: R.zeta(12)
8 + 24*37 + 37^2 + 29*37^3 + 23*37^4 + O(37^5)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(37),Integer(5))
>>> R.zeta(Integer(12))
8 + 24*37 + 37^2 + 29*37^3 + 23*37^4 + O(37^5)
```

zeta_order()

Return the order of the group of roots of unity.

EXAMPLES:

```
sage: R = Zp(37); R.zeta_order()
36
sage: Zp(2).zeta_order()
2
```

```
>>> from sage.all import *
>>> R = Zp(Integer(37)); R.zeta_order()
36
>>> Zp(Integer(2)).zeta_order()
2
```

CHAPTER

SEVEN

p-ADIC EXTENSION GENERIC

A common superclass for all extensions of Qp and Zp.

AUTHORS:

- David Roe

class sage.rings.padics.padic_extension_generic.**DefPolyConversion**

Bases: Morphism

Conversion map between p-adic rings/fields with the same defining polynomial.

INPUT:

- R – a p-adic extension ring or field
- S – a p-adic extension ring or field with the same defining polynomial

EXAMPLES:

```
sage: R.<a> = Zq(125, print_mode='terse')
sage: S = R.change(prec = 15, type='floating-point')
sage: a - 1
95367431640624 + a + O(5^20)
sage: S(a - 1)
30517578124 + a + O(5^15)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), print_mode='terse', names=('a',)); (a,) = R._first_
↪ngens(1)
>>> S = R.change(prec = Integer(15), type='floating-point')
>>> a - Integer(1)
95367431640624 + a + O(5^20)
>>> S(a - Integer(1))
30517578124 + a + O(5^15)
```

```
sage: R.<a> = Zq(125, print_mode='terse')
sage: S = R.change(prec = 15, type='floating-point')
sage: f = S.convert_map_from(R)
sage: TestSuite(f).run()
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), print_mode='terse', names=('a',)); (a,) = R._first_
↪ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> S = R.change(prec = Integer(15), type='floating-point')
>>> f = S.convert_map_from(R)
>>> TestSuite(f).run()
```

class sage.rings.padics.padic_extension_generic.**MapFreeModuleToOneStep**

Bases: *pAdicModuleIsomorphism*

The isomorphism from the underlying module of a one-step p-adic extension to the extension.

EXAMPLES:

```
sage: K.<a> = Qq(125)
sage: V, fr, to = K.free_module()
sage: TestSuite(fr).run(skip=['_test_nonzero_equal'])  # skipped since Qq(125)␣
↪doesn't have dimension()
```

```
>>> from sage.all import *
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> V, fr, to = K.free_module()
>>> TestSuite(fr).run(skip=['_test_nonzero_equal'])  # skipped since Qq(125) doesn
↪'t have dimension()
```

class sage.rings.padics.padic_extension_generic.**MapFreeModuleToTwoStep**

Bases: *pAdicModuleIsomorphism*

The isomorphism from the underlying module of a two-step p-adic extension to the extension.

EXAMPLES:

```
sage: K.<a> = Qq(125)
sage: R.<x> = ZZ[]
sage: L.<b> = K.extension(x^2 - 5*x + 5)
sage: V, fr, to = L.free_module(base=Qp(5))
sage: TestSuite(fr).run(skip=['_test_nonzero_equal'])  # skipped since L doesn't␣
↪have dimension()
```

```
>>> from sage.all import *
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> L = K.extension(x**Integer(2) - Integer(5)*x + Integer(5), names=('b',)); (b,
↪) = L._first_ngens(1)
>>> V, fr, to = L.free_module(base=Qp(Integer(5)))
>>> TestSuite(fr).run(skip=['_test_nonzero_equal'])  # skipped since L doesn't␣
↪have dimension()
```

class sage.rings.padics.padic_extension_generic.**MapOneStepToFreeModule**

Bases: *pAdicModuleIsomorphism*

The isomorphism from a one-step p-adic extension to its underlying free module.

EXAMPLES:

```
sage: K.<a> = Qq(125)
sage: V, fr, to = K.free_module()
sage: TestSuite(to).run()
```

```
>>> from sage.all import *
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> V, fr, to = K.free_module()
>>> TestSuite(to).run()
```

class sage.rings.padics.padic_extension_generic.**MapTwoStepToFreeModule**

Bases: *pAdicModuleIsomorphism*

The isomorphism from a two-step p-adic extension to its underlying free module.

EXAMPLES:

```
sage: K.<a> = Qq(125)
sage: R.<x> = ZZ[]
sage: L.<b> = K.extension(x^2 - 5*x + 5)
sage: V, fr, to = L.free_module(base=Qp(5))
sage: TestSuite(to).run()
```

```
>>> from sage.all import *
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> L = K.extension(x**Integer(2) - Integer(5)*x + Integer(5), names=('b',)); (b,
↪) = L._first_ngens(1)
>>> V, fr, to = L.free_module(base=Qp(Integer(5)))
>>> TestSuite(to).run()
```

class sage.rings.padics.padic_extension_generic.**pAdicExtensionGeneric**(*poly*, *prec*, *print_mode*, *names*, *element_class*)

Bases: *pAdicGeneric*

Initialization.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 +125*x - 5
sage: W.<w> = R.ext(f)  # indirect doctest
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)# indirect doctest
```

construction(*forbid_frac_field=False*)

Return the functorial construction of this ring, namely, the algebraic extension of the base ring defined by the given polynomial.

Also preserves other information that makes this ring unique (e.g. precision, rounding, print mode).

INPUT:

- `forbid_frac_field` – require a completion functor rather than a fraction field functor. This is used in the *sage.rings.padics.local_generic.LocalGeneric.change()* method.

EXAMPLES:

```
sage: R.<a> = Zq(25, 8, print_mode='val-unit')
sage: c, R0 = R.construction(); R0
5-adic Ring with capped relative precision 8
sage: c(R0)
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
sage: c(R0) == R
True
```

```
>>> from sage.all import *
>>> R = Zq(Integer(25), Integer(8), print_mode='val-unit', names=('a',)); (a,
↪) = R._first_ngens(1)
>>> c, R0 = R.construction(); R0
5-adic Ring with capped relative precision 8
>>> c(R0)
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
>>> c(R0) == R
True
```

For a field, by default we return a fraction field functor.

```
sage: K.<a> = Qq(25, 8)
sage: c, R = K.construction(); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
sage: c
FractionField
```

```
>>> from sage.all import *
>>> K = Qq(Integer(25), Integer(8), names=('a',)); (a,) = K._first_ngens(1)
>>> c, R = K.construction(); R
5-adic Unramified Extension Ring in a defined by x^2 + 4*x + 2
>>> c
FractionField
```

If you prefer an extension functor, you can use the `forbit_frac_field` keyword:

```
sage: c, R = K.construction(forbid_frac_field=True); R
5-adic Field with capped relative precision 8
sage: c
AlgebraicExtensionFunctor
sage: c(R) is K
True
```

```
>>> from sage.all import *
>>> c, R = K.construction(forbid_frac_field=True); R
5-adic Field with capped relative precision 8
>>> c
AlgebraicExtensionFunctor
>>> c(R) is K
True
```

defining_polynomial(*var=None*, *exact=False*)

Return the polynomial defining this extension.

INPUT:

- `var` – string (default: `'x'`); the name of the variable
- **`exact` – boolean (default: `False`); whether to return the underlying exact** defining polynomial rather than the one with coefficients in the base ring

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: W.defining_polynomial()
(1 + O(5^5))*x^5 + O(5^6)*x^4 + (3*5^2 + O(5^6))*x^3 + (2*5 + 4*5^2 + 4*5^3 +␣
↪4*5^4 + 4*5^5 + O(5^6))*x^2 + (5^3 + O(5^6))*x + 4*5 + 4*5^2 + 4*5^3 + 4*5^
↪4 + 4*5^5 + O(5^6)
sage: W.defining_polynomial(exact=True)
x^5 + 75*x^3 - 15*x^2 + 125*x - 5

sage: W.defining_polynomial(var='y', exact=True)
y^5 + 75*y^3 - 15*y^2 + 125*y - 5
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> W.defining_polynomial()
(1 + O(5^5))*x^5 + O(5^6)*x^4 + (3*5^2 + O(5^6))*x^3 + (2*5 + 4*5^2 + 4*5^3 +␣
↪4*5^4 + 4*5^5 + O(5^6))*x^2 + (5^3 + O(5^6))*x + 4*5 + 4*5^2 + 4*5^3 + 4*5^
↪4 + 4*5^5 + O(5^6)
>>> W.defining_polynomial(exact=True)
x^5 + 75*x^3 - 15*x^2 + 125*x - 5

>>> W.defining_polynomial(var='y', exact=True)
y^5 + 75*y^3 - 15*y^2 + 125*y - 5
```

See also

modulus() exact_field()

`exact_field``()`

Return a number field with the same defining polynomial.

Note that this method always returns a field, even for a p-adic ring.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 +125*x - 5
sage: W.<w> = R.ext(f)
```

(continues on next page)

(continued from previous page)

```
sage: W.exact_field()
Number Field in w with defining polynomial x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> W.exact_field()
Number Field in w with defining polynomial x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

See also

defining_polynomial() modulus()

exact_ring()

Return the order with the same defining polynomial.

Will raise a ValueError if the coefficients of the defining polynomial are not integral.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 +125*x - 5
sage: W.<w> = R.ext(f)
sage: W.exact_ring()
Order generated by w in Number Field in w with defining polynomial x^5 + 75*x^
↪3 - 15*x^2 + 125*x - 5

sage: T = Zp(5,5)
sage: U.<z> = T[]
sage: g = 2*z^4 + 1
sage: V.<v> = T.ext(g)
sage: V.exact_ring()
Traceback (most recent call last):
...
ValueError: each generator must be integral
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> W.exact_ring()
Order generated by w in Number Field in w with defining polynomial x^5 + 75*x^
↪3 - 15*x^2 + 125*x - 5

>>> T = Zp(Integer(5),Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> U = T['z']; (z,) = U._first_ngens(1)
>>> g = Integer(2)*z**Integer(4) + Integer(1)
>>> V = T.ext(g, names=('v',)); (v,) = V._first_ngens(1)
>>> V.exact_ring()
Traceback (most recent call last):
...
ValueError: each generator must be integral
```

free_module(*base=None*, *basis=None*, *map=True*)

Return a free module V over a specified base ring together with maps to and from V.

INPUT:

- `base` – a subring R so that this ring/field is isomorphic to a finite-rank free R-module V
- `basis` – a basis for this ring/field over the base
- `map` – boolean (default: `True`); whether to return R-linear maps to and from V

OUTPUT:

- A finite-rank free R-module V
- An R-module isomorphism from V to this ring/field (only included if `map` is `True`)
- An R-module isomorphism from this ring/field to V (only included if `map` is `True`)

EXAMPLES:

```
sage: R.<x> = ZZ[]
sage: K.<a> = Qq(125)
sage: L.<pi> = K.extension(x^2-5)
sage: V, from_V, to_V = K.free_module()
sage: W, from_W, to_W = L.free_module()
sage: W0, from_W0, to_W0 = L.free_module(base=Qp(5))
sage: to_V(a + O(5^7))
(O(5^7), 1 + O(5^7), O(5^7))
sage: to_W(a)
(a + O(5^20), O(5^20))
sage: to_W0(a + O(5^7))
(O(5^7), 1 + O(5^7), O(5^7), O(5^7), O(5^7), O(5^7))
sage: to_W(pi)
(O(5^21), 1 + O(5^20))
sage: to_W0(pi + O(pi^11))
(O(5^6), O(5^6), O(5^6), 1 + O(5^5), O(5^5), O(5^5))

sage: X, from_X, to_X = K.free_module(K)
sage: to_X(a)
(a + O(5^20))
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> L = K.extension(x**Integer(2)-Integer(5), names=('pi',)); (pi,) = L._
↪first_ngens(1)
>>> V, from_V, to_V = K.free_module()
```

(continues on next page)

(continued from previous page)

```
>>> W, from_W, to_W = L.free_module()
>>> W0, from_W0, to_W0 = L.free_module(base=Qp(Integer(5)))
>>> to_V(a + O(Integer(5)**Integer(7)))
(O(5^7), 1 + O(5^7), O(5^7))
>>> to_W(a)
(a + O(5^20), O(5^20))
>>> to_W0(a + O(Integer(5)**Integer(7)))
(O(5^7), 1 + O(5^7), O(5^7), O(5^7), O(5^7), O(5^7))
>>> to_W(pi)
(O(5^21), 1 + O(5^20))
>>> to_W0(pi + O(pi**Integer(11)))
(O(5^6), O(5^6), O(5^6), 1 + O(5^5), O(5^5), O(5^5))

>>> X, from_X, to_X = K.free_module(K)
>>> to_X(a)
(a + O(5^20))
```

ground_ring()

Return the ring of which this ring is an extension.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 +125*x - 5
sage: W.<w> = R.ext(f)
sage: W.ground_ring()
5-adic Ring with capped relative precision 5
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> W.ground_ring()
5-adic Ring with capped relative precision 5
```

ground_ring_of_tower()

Return the p-adic base ring of which this is ultimately an extension.

Currently this function is identical to `ground_ring()`, since relative extensions have not yet been implemented.

EXAMPLES:

```
sage: Qq(27,30,names='a').ground_ring_of_tower()
3-adic Field with capped relative precision 30
```

```
>>> from sage.all import *
>>> Qq(Integer(27),Integer(30),names='a').ground_ring_of_tower()
3-adic Field with capped relative precision 30
```

modulus(*exact=False*)

Return the polynomial defining this extension.

INPUT:

- `exact` – boolean (default: `False`); whether to return the underlying exact defining polynomial rather than the one with coefficients in the base ring

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 +125*x - 5
sage: W.<w> = R.ext(f)
sage: W.modulus()
(1 + O(5^5))*x^5 + O(5^6)*x^4 + (3*5^2 + O(5^6))*x^3 + (2*5 + 4*5^2 + 4*5^3 +␣
↪4*5^4 + 4*5^5 + O(5^6))*x^2 + (5^3 + O(5^6))*x + 4*5 + 4*5^2 + 4*5^3 + 4*5^
↪4 + 4*5^5 + O(5^6)
sage: W.modulus(exact=True)
x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> W.modulus()
(1 + O(5^5))*x^5 + O(5^6)*x^4 + (3*5^2 + O(5^6))*x^3 + (2*5 + 4*5^2 + 4*5^3 +␣
↪4*5^4 + 4*5^5 + O(5^6))*x^2 + (5^3 + O(5^6))*x + 4*5 + 4*5^2 + 4*5^3 + 4*5^
↪4 + 4*5^5 + O(5^6)
>>> W.modulus(exact=True)
x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

See also

defining_polynomial() exact_field()

polynomial_ring()

Return the polynomial ring of which this is a quotient.

EXAMPLES:

```
sage: Qq(27,30,names='a').polynomial_ring()
Univariate Polynomial Ring in x over 3-adic Field with capped relative␣
↪precision 30
```

```
>>> from sage.all import *
>>> Qq(Integer(27),Integer(30),names='a').polynomial_ring()
Univariate Polynomial Ring in x over 3-adic Field with capped relative␣
↪precision 30
```

random_element()

Return a random element of `self`.

This is done by picking a random element of the ground ring self.degree() times, then treating those elements as coefficients of a polynomial in self.gen().

EXAMPLES:

```
sage: R.<a> = Zq(125, 5)
sage: R.random_element().parent() is R
True
sage: R = Zp(5,3); S.<x> = ZZ[]; f = x^5 + 25*x^2 - 5; W.<w> = R.ext(f)
sage: W.random_element().parent() is W
True
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), Integer(5), names=('a',)); (a,) = R._first_ngens(1)
>>> R.random_element().parent() is R
True
>>> R = Zp(Integer(5),Integer(3)); S = ZZ['x']; (x,) = S._first_ngens(1); f =␣
↪x**Integer(5) + Integer(25)*x**Integer(2) - Integer(5); W = R.ext(f, names=(
↪'w',)); (w,) = W._first_ngens(1)
>>> W.random_element().parent() is W
True
```

class sage.rings.padics.padic_extension_generic.**pAdicModuleIsomorphism**

Bases: Map

A base class for various isomorphisms between p-adic rings/fields and free modules.

EXAMPLES:

```
sage: K.<a> = Qq(125)
sage: V, fr, to = K.free_module()
sage: from sage.rings.padics.padic_extension_generic import pAdicModuleIsomorphism
sage: isinstance(fr, pAdicModuleIsomorphism)
True
```

```
>>> from sage.all import *
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> V, fr, to = K.free_module()
>>> from sage.rings.padics.padic_extension_generic import pAdicModuleIsomorphism
>>> isinstance(fr, pAdicModuleIsomorphism)
True
```

is_injective()

EXAMPLES:

```
sage: K.<a> = Qq(125)
sage: V, fr, to = K.free_module()
sage: fr.is_injective()
True
```

```
>>> from sage.all import *
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> V, fr, to = K.free_module()
>>> fr.is_injective()
True
```

is_surjective()

EXAMPLES:

```
sage: K.<a> = Qq(125)
sage: V, fr, to = K.free_module()
sage: fr.is_surjective()
True
```

```
>>> from sage.all import *
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> V, fr, to = K.free_module()
>>> fr.is_surjective()
True
```

CHAPTER

EIGHT

EISENSTEIN EXTENSION GENERIC

This file implements the shared functionality for Eisenstein extensions.

AUTHORS:

- David Roe

class sage.rings.padics.eisenstein_extension_generic.**EisensteinExtensionGeneric**(*poly*, *prec*, *print_mode*, *names*, *element_class*)

Bases: *pAdicExtensionGeneric*

Initialize `self`.

EXAMPLES:

```
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                          #␣
↪needs sage.libs.ntl
sage: B.<t> = A.ext(x^2+7)  # indirect doctest                             #␣
↪needs sage.libs.ntl sage.rings.padics
```

```
>>> from sage.all import *
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2)+Integer(7), names=('t',)); (t,) = B._first_ngens(1)#␣
↪indirect doctest                                  # needs sage.libs.ntl sage.rings.
↪padics
```

absolute_e()

Return the absolute ramification index of this ring or field.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                      #␣
↪needs sage.libs.ntl
sage: K.absolute_e()                                                       #␣
↪needs sage.libs.ntl
1

sage: x = polygen(ZZ, 'x')
```

(continues on next page)

(continued from previous page)

```
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                    #␣
↪needs sage.libs.ntl
sage: L.absolute_e()                                                       #␣
↪needs sage.libs.ntl
2
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.absolute_e()                                                         #␣
↪needs sage.libs.ntl
1

>>> x = polygen(ZZ, 'x')
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.absolute_e()                                                         #␣
↪needs sage.libs.ntl
2
```

gen(*n=0*)

Return a generator for `self` as an extension of its ground ring.

EXAMPLES:

```
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                          #␣
↪needs sage.libs.ntl
sage: B.<t> = A.ext(x^2 + 7)                                               #␣
↪needs sage.libs.ntl
sage: B.gen()                                                              #␣
↪needs sage.libs.ntl
t + O(t^21)
```

```
>>> from sage.all import *
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2) + Integer(7), names=('t',)); (t,) = B._first_
↪ngens(1)# needs sage.libs.ntl
>>> B.gen()                                                                #␣
↪needs sage.libs.ntl
t + O(t^21)
```

inertia_subring()

Return the inertia subring.

Since an Eisenstein extension is totally ramified, this is just the ground field.

EXAMPLES:

```
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                          #␣
↪needs sage.libs.ntl
```

(continues on next page)

(continued from previous page)

```
sage: B.<t> = A.ext(x^2 + 7)                                                #␣
↪needs sage.libs.ntl
sage: B.inertia_subring()                                                   #␣
↪needs sage.libs.ntl
7-adic Ring with capped relative precision 10
```

```
>>> from sage.all import *
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2) + Integer(7), names=('t',)); (t,) = B._first_
↪ngens(1)# needs sage.libs.ntl
>>> B.inertia_subring()                                                     #␣
↪needs sage.libs.ntl
7-adic Ring with capped relative precision 10
```

residue_class_field()

Return the residue class field.

INPUT:

- `self` – a p-adic ring

OUTPUT: the residue field

EXAMPLES:

```
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                           #␣
↪needs sage.libs.ntl
sage: B.<t> = A.ext(x^2 + 7)                                                #␣
↪needs sage.libs.ntl
sage: B.residue_class_field()                                               #␣
↪needs sage.libs.ntl
Finite Field of size 7
```

```
>>> from sage.all import *
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2) + Integer(7), names=('t',)); (t,) = B._first_
↪ngens(1)# needs sage.libs.ntl
>>> B.residue_class_field()                                                 #␣
↪needs sage.libs.ntl
Finite Field of size 7
```

residue_ring(n)

Return the quotient of the ring of integers by the n-th power of its maximal ideal.

EXAMPLES:

```
sage: S.<x> = ZZ[]
sage: W.<w> = Zp(5).extension(x^2 - 5)                                      #␣
↪needs sage.libs.ntl
sage: W.residue_ring(1)                                                     #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.ntl
Ring of integers modulo 5
```

```
>>> from sage.all import *
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> W = Zp(Integer(5)).extension(x**Integer(2) - Integer(5), names=('w',));␣
↪(w,) = W._first_ngens(1)# needs sage.libs.ntl
>>> W.residue_ring(Integer(1))                                                ␣
↪     # needs sage.libs.ntl
Ring of integers modulo 5
```

The following requires implementing more general Artinian rings:

```
sage: W.residue_ring(2)                                                      #␣
↪needs sage.libs.ntl
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> W.residue_ring(Integer(2))                                               ␣
↪     # needs sage.libs.ntl
Traceback (most recent call last):
...
NotImplementedError
```

uniformizer()

Return the uniformizer of `self`, i.e., a generator for the unique maximal ideal.

EXAMPLES:

```
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                            #␣
↪needs sage.libs.ntl
sage: B.<t> = A.ext(x^2 + 7)                                                 #␣
↪needs sage.libs.ntl
sage: B.uniformizer()                                                        #␣
↪needs sage.libs.ntl
t + O(t^21)
```

```
>>> from sage.all import *
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2) + Integer(7), names=('t',)); (t,) = B._first_
↪ngens(1)# needs sage.libs.ntl
>>> B.uniformizer()                                                          #␣
↪needs sage.libs.ntl
t + O(t^21)
```

uniformizer_pow(n)

Return the n-th power of the uniformizer of `self` (as an element of `self`).

EXAMPLES:

```
sage: A = Zp(7,10)
sage: S.<x> = A[]                                                              #␣
↪needs sage.libs.ntl
sage: B.<t> = A.ext(x^2 + 7)                                                   #␣
↪needs sage.libs.ntl
sage: B.uniformizer_pow(5)                                                     #␣
↪needs sage.libs.ntl
t^5 + O(t^25)
```

```
>>> from sage.all import *
>>> A = Zp(Integer(7),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)# needs sage.libs.ntl
>>> B = A.ext(x**Integer(2) + Integer(7), names=('t',)); (t,) = B._first_
↪ngens(1)# needs sage.libs.ntl
>>> B.uniformizer_pow(Integer(5))                                              ␣
↪     # needs sage.libs.ntl
t^5 + O(t^25)
```

CHAPTER

NINE

UNRAMIFIED EXTENSION GENERIC

This file implements the shared functionality for unramified extensions.

AUTHORS:

- David Roe

class sage.rings.padics.unramified_extension_generic.**UnramifiedExtensionGeneric**(*poly*, *prec*, *print_mode*, *names*, *element_class*)

Bases: *pAdicExtensionGeneric*

An unramified extension of $\mathbf{Q}_p$ or $\mathbf{Z}_p$.

absolute_f()

Return the degree of the residue field of this ring/field over its prime subfield.

EXAMPLES:

```
sage: K.<a> = Qq(3^5)                                                    #␣
↪needs sage.libs.ntl
sage: K.absolute_f()                                                     #␣
↪needs sage.libs.ntl
5

sage: x = polygen(ZZ, 'x')
sage: L.<pi> = Qp(3).extension(x^2 - 3)                                  #␣
↪needs sage.libs.ntl
sage: L.absolute_f()                                                     #␣
↪needs sage.libs.ntl
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(3)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)#␣
↪needs sage.libs.ntl
>>> K.absolute_f()                                                       #␣
↪needs sage.libs.ntl
5

>>> x = polygen(ZZ, 'x')
>>> L = Qp(Integer(3)).extension(x**Integer(2) - Integer(3), names=('pi',));␣
```

(continues on next page)

(continued from previous page)

```
↪(pi,) = L._first_ngens(1)# needs sage.libs.ntl
>>> L.absolute_f()                                                    #␣
↪needs sage.libs.ntl
1
```

discriminant(*K=None*)

Return the discriminant of `self` over the subring K.

INPUT:

- `K` – a subring/subfield (defaults to the base ring)

EXAMPLES:

```
sage: R.<a> = Zq(125)                                                 #␣
↪needs sage.libs.ntl
sage: R.discriminant()                                                #␣
↪needs sage.libs.ntl
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1)# needs sage.
↪libs.ntl
>>> R.discriminant()                                                  #␣
↪needs sage.libs.ntl
Traceback (most recent call last):
...
NotImplementedError
```

gen(*n=0*)

Return a generator for this unramified extension.

This is an element that satisfies the polynomial defining this extension. Such an element will reduce to a generator of the corresponding residue field extension.

EXAMPLES:

```
sage: R.<a> = Zq(125); R.gen()                                        #␣
↪needs sage.libs.ntl
a + O(5^20)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1); R.gen()    ␣
↪                                        # needs sage.libs.ntl
a + O(5^20)
```

has_pth_root()

Return whether or not $\mathbf{Z}_p$ has a primitive p-th root of unity.

Since adjoining a p-th root of unity yields a totally ramified extension, `self` will contain one if and only if the ground ring does.

INPUT:

- `self` – a p-adic ring

OUTPUT: boolean; whether `self` has primitive p-th root of unity

EXAMPLES:

```
sage: R.<a> = Zq(1024); R.has_pth_root()                                    #␣
↪needs sage.libs.ntl
True
sage: R.<a> = Zq(17^5); R.has_pth_root()                                    #␣
↪needs sage.libs.ntl
False
```

```
>>> from sage.all import *
>>> R = Zq(Integer(1024), names=('a',)); (a,) = R._first_ngens(1); R.has_pth_
↪root()                                    # needs sage.libs.ntl
True
>>> R = Zq(Integer(17)**Integer(5), names=('a',)); (a,) = R._first_ngens(1);␣
↪R.has_pth_root()                                    # needs sage.libs.ntl
False
```

has_root_of_unity(*n*)

Return whether or not $\mathbf{Z}_p$ has a primitive n-th root of unity.

INPUT:

- `self` – a p-adic ring
- `n` – integer

OUTPUT: boolean

EXAMPLES:

```
sage: R.<a> = Zq(37^8)
sage: R.has_root_of_unity(144)
True
sage: R.has_root_of_unity(89)
True
sage: R.has_root_of_unity(11)
False
```

```
>>> from sage.all import *
>>> R = Zq(Integer(37)**Integer(8), names=('a',)); (a,) = R._first_ngens(1)
>>> R.has_root_of_unity(Integer(144))
True
>>> R.has_root_of_unity(Integer(89))
True
>>> R.has_root_of_unity(Integer(11))
False
```

is_galois(*K=None*)

Return `True` if this extension is Galois.

Every unramified extension is Galois.

INPUT:

- K – a subring/subfield (defaults to the base ring)

EXAMPLES:

```
sage: R.<a> = Zq(125); R.is_galois()                                          #␣
↪needs sage.libs.ntl
True
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1); R.is_
↪galois()                                          # needs sage.libs.ntl
True
```

residue_class_field()

Return the residue class field.

EXAMPLES:

```
sage: R.<a> = Zq(125); R.residue_class_field()                                #␣
↪needs sage.libs.ntl
Finite Field in a0 of size 5^3
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1); R.residue_
↪class_field()                                 # needs sage.libs.ntl
Finite Field in a0 of size 5^3
```

residue_ring(n)

Return the quotient of the ring of integers by the n-th power of its maximal ideal.

EXAMPLES:

```
sage: R.<a> = Zq(125)                                                         #␣
↪needs sage.libs.ntl
sage: R.residue_ring(1)                                                       #␣
↪needs sage.libs.ntl
Finite Field in a0 of size 5^3
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1)# needs sage.
↪libs.ntl
>>> R.residue_ring(Integer(1))                                                 ␣
↪     # needs sage.libs.ntl
Finite Field in a0 of size 5^3
```

The following requires implementing more general Artinian rings:

```
sage: R.residue_ring(2)                                                       #␣
↪needs sage.libs.ntl
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R.residue_ring(Integer(2))
↪     # needs sage.libs.ntl
Traceback (most recent call last):
...
NotImplementedError
```

uniformizer()

Return a uniformizer for this extension.

Since this extension is unramified, a uniformizer for the ground ring will also be a uniformizer for this extension.

EXAMPLES:

```
sage: R.<a> = ZqCR(125)
↪needs sage.libs.ntl
sage: R.uniformizer()
↪needs sage.libs.ntl
5 + O(5^21)
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(125), names=('a',)); (a,) = R._first_ngens(1)# needs
↪sage.libs.ntl
>>> R.uniformizer()
↪needs sage.libs.ntl
5 + O(5^21)
```

uniformizer_pow(n)

Return the n-th power of the uniformizer of `self` (as an element of `self`).

EXAMPLES:

```
sage: R.<a> = ZqCR(125)
↪needs sage.libs.ntl
sage: R.uniformizer_pow(5)
↪needs sage.libs.ntl
5^5 + O(5^25)
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(125), names=('a',)); (a,) = R._first_ngens(1)# needs
↪sage.libs.ntl
>>> R.uniformizer_pow(Integer(5))
↪     # needs sage.libs.ntl
5^5 + O(5^25)
```

CHAPTER

TEN

p-ADIC BASE LEAVES

Implementations of $\mathbf{Z}_p$ and $\mathbf{Q}_p$

AUTHORS:

- David Roe
- Genya Zaytman: documentation
- David Harvey: doctests
- William Stein: doctest updates

EXAMPLES:

p-adic rings and fields are examples of inexact structures, as the reals are. That means that elements cannot generally be stored exactly: to do so would take an infinite amount of storage. Instead, we store an approximation to the elements with varying precision.

There are two types of precision for a p-adic element. The first is relative precision, which gives the number of known p-adic digits:

```
sage: R = Qp(5, 20, 'capped-rel', 'series'); a = R(675); a
2*5^2 + 5^4 + O(5^22)
sage: a.precision_relative()
20
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), Integer(20), 'capped-rel', 'series'); a = R(Integer(675)); a
2*5^2 + 5^4 + O(5^22)
>>> a.precision_relative()
20
```

The second type of precision is absolute precision, which gives the power of p that this element is stored modulo:

```
sage: a.precision_absolute()
22
```

```
>>> from sage.all import *
>>> a.precision_absolute()
22
```

The number of times that p divides the element is called the valuation, and can be accessed with the methods *valuation()* and *ordp()*:

sage: a.valuation() 2

The following relationship holds:

`self.valuation() + self.precision_relative() == self.precision_absolute()`.

sage: a.valuation() + a.precision_relative() == a.precision_absolute() True

In the capped relative case, the relative precision of an element is restricted to be at most a certain value, specified at the creation of the field. Individual elements also store their own precision, so the effect of various arithmetic operations on precision is tracked. When you cast an exact element into a capped relative field, it truncates it to the precision cap of the field.:

```
sage: R = Qp(5, 5); a = R(4006); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(17/3); b
4 + 2*5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)
sage: c = R(4025); c
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
sage: a + b
4*5 + 3*5^2 + 3*5^3 + 4*5^4 + O(5^5)
sage: a + b + c
4*5 + 4*5^2 + 5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), Integer(5)); a = R(Integer(4006)); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(17)/Integer(3)); b
4 + 2*5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)
>>> c = R(Integer(4025)); c
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
>>> a + b
4*5 + 3*5^2 + 3*5^3 + 4*5^4 + O(5^5)
>>> a + b + c
4*5 + 4*5^2 + 5^4 + O(5^5)
```

```
sage: R = Zp(5, 5, 'capped-rel', 'series'); a = R(4006); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(17/3); b
4 + 2*5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)
sage: c = R(4025); c
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
sage: a + b
4*5 + 3*5^2 + 3*5^3 + 4*5^4 + O(5^5)
sage: a + b + c
4*5 + 4*5^2 + 5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(5), 'capped-rel', 'series'); a = R(Integer(4006)); a
1 + 5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(17)/Integer(3)); b
4 + 2*5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)
>>> c = R(Integer(4025)); c
5^2 + 2*5^3 + 5^4 + 5^5 + O(5^7)
>>> a + b
4*5 + 3*5^2 + 3*5^3 + 4*5^4 + O(5^5)
```

(continues on next page)

(continued from previous page)

```
>>> a + b + c
4*5 + 4*5^2 + 5^4 + O(5^5)
```

In the capped absolute type, instead of having a cap on the relative precision of an element there is instead a cap on the absolute precision. Elements still store their own precisions, and as with the capped relative case, exact elements are truncated when cast into the ring.:

```
sage: R = ZpCA(5, 5); a = R(4005); a
5 + 2*5^3 + 5^4 + O(5^5)
sage: b = R(4025); b
5^2 + 2*5^3 + 5^4 + O(5^5)
sage: a * b
5^3 + 2*5^4 + O(5^5)
sage: (a * b) // 5^3
1 + 2*5 + O(5^2)
sage: type((a * b) // 5^3)
<class 'sage.rings.padics.padic_capped_absolute_element.pAdicCappedAbsoluteElement'>
sage: (a * b) / 5^3
1 + 2*5 + O(5^2)
sage: type((a * b) / 5^3)
<class 'sage.rings.padics.padic_capped_relative_element.pAdicCappedRelativeElement'>
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5), Integer(5)); a = R(Integer(4005)); a
5 + 2*5^3 + 5^4 + O(5^5)
>>> b = R(Integer(4025)); b
5^2 + 2*5^3 + 5^4 + O(5^5)
>>> a * b
5^3 + 2*5^4 + O(5^5)
>>> (a * b) // Integer(5)**Integer(3)
1 + 2*5 + O(5^2)
>>> type((a * b) // Integer(5)**Integer(3))
<class 'sage.rings.padics.padic_capped_absolute_element.pAdicCappedAbsoluteElement'>
>>> (a * b) / Integer(5)**Integer(3)
1 + 2*5 + O(5^2)
>>> type((a * b) / Integer(5)**Integer(3))
<class 'sage.rings.padics.padic_capped_relative_element.pAdicCappedRelativeElement'>
```

The fixed modulus type is the leanest of the p-adic rings: it is basically just a wrapper around $\mathbf{Z}/p^n\mathbf{Z}$ providing a unified interface with the rest of the p-adics. This is the type you should use if your primary interest is in speed (though it's not all that much faster than other p-adic types). It does not track precision of elements.:

```
sage: R = ZpFM(5, 5); a = R(4005); a
5 + 2*5^3 + 5^4
sage: a // 5
1 + 2*5^2 + 5^3
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5), Integer(5)); a = R(Integer(4005)); a
5 + 2*5^3 + 5^4
>>> a // Integer(5)
1 + 2*5^2 + 5^3
```

p-adic rings and fields should be created using the creation functions *Zp* and *Qp* as above. This will ensure that there is only one instance of $\mathbf{Z}_p$ and $\mathbf{Q}_p$ of a given type, p, print mode and precision. It also saves typing very long class names.:

```
sage: Qp(17,10)
17-adic Field with capped relative precision 10
sage: R = Qp(7, prec = 20, print_mode = 'val-unit'); S = Qp(7, prec = 20, print_mode␣
↪= 'val-unit'); R is S
True
sage: Qp(2)
2-adic Field with capped relative precision 20
```

```
>>> from sage.all import *
>>> Qp(Integer(17),Integer(10))
17-adic Field with capped relative precision 10
>>> R = Qp(Integer(7), prec = Integer(20), print_mode = 'val-unit'); S =␣
↪Qp(Integer(7), prec = Integer(20), print_mode = 'val-unit'); R is S
True
>>> Qp(Integer(2))
2-adic Field with capped relative precision 20
```

Once one has a p-adic ring or field, one can cast elements into it in the standard way. Integers, ints, longs, Rationals, other p-adic types, pari p-adics and elements of $\mathbf{Z}/p^n\mathbf{Z}$ can all be cast into a p-adic field.:

```
sage: R = Qp(5, 5, 'capped-rel','series'); a = R(16); a
1 + 3*5 + O(5^5)
sage: b = R(23/15); b
5^-1 + 3 + 3*5 + 5^2 + 3*5^3 + O(5^4)
sage: S = Zp(5, 5, 'fixed-mod','val-unit'); c = S(Mod(75,125)); c
5^2 * 3
sage: R(c)
3*5^2 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5), Integer(5), 'capped-rel','series'); a = R(Integer(16)); a
1 + 3*5 + O(5^5)
>>> b = R(Integer(23)/Integer(15)); b
5^-1 + 3 + 3*5 + 5^2 + 3*5^3 + O(5^4)
>>> S = Zp(Integer(5), Integer(5), 'fixed-mod','val-unit'); c = S(Mod(Integer(75),
↪Integer(125))); c
5^2 * 3
>>> R(c)
3*5^2 + O(5^5)
```

In the previous example, since fixed-mod elements don't keep track of their precision, we assume that it has the full precision of the ring. This is why you have to cast manually here.

While you can cast explicitly as above, the chains of automatic coercion are more restricted. As always in Sage, the following arrows are transitive and the diagram is commutative.:

```
int -> long -> Integer -> Zp capped-rel -> Zp capped_abs -> IntegerMod
Integer -> Zp fixed-mod -> IntegerMod
Integer -> Zp capped-abs -> Qp capped-rel
```

In addition, there are arrows within each type. For capped relative and capped absolute rings and fields, these arrows go from lower precision cap to higher precision cap. This works since elements track their own precision: choosing the parent

with higher precision cap means that precision is less likely to be truncated unnecessarily. For fixed modulus parents, the arrow goes from higher precision cap to lower. The fact that elements do not track precision necessitates this choice in order to not produce incorrect results.

class sage.rings.padics.padic_base_leaves.**pAdicFieldCappedRelative**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicFieldBaseGeneric*, *pAdicCappedRelativeFieldGeneric*

An implementation of p-adic fields with capped relative precision.

EXAMPLES:

```
sage: K = Qp(17, 1000000)  # indirect doctest
sage: K = Qp(101)  # indirect doctest
```

```
>>> from sage.all import *
>>> K = Qp(Integer(17), Integer(1000000))  # indirect doctest
>>> K = Qp(Integer(101))  # indirect doctest
```

random_element(*algorithm='default'*)

Return a random element of `self`, optionally using the `algorithm` argument to decide how it generates the element. Algorithms currently implemented:

- `'default'`: Choose an integer k using the standard distribution on the integers. Then choose an integer a uniformly in the range $0 \leq a < p^N$ where N is the precision cap of `self`. Return `self(p^k * a, absprec = k + self.precision_cap())`.

EXAMPLES:

```
sage: Qp(17,6).random_element().parent() is Qp(17,6)
True
```

```
>>> from sage.all import *
>>> Qp(Integer(17),Integer(6)).random_element().parent() is Qp(Integer(17),
↪Integer(6))
True
```

class sage.rings.padics.padic_base_leaves.**pAdicFieldFloatingPoint**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicFieldBaseGeneric*, *pAdicFloatingPointFieldGeneric*

An implementation of the p-adic rationals with floating point precision.

class sage.rings.padics.padic_base_leaves.**pAdicFieldLattice**(*p*, *prec*, *subtype*, *print_mode*, *names*, *label=None*, *category=None*)

Bases: *pAdicLatticeGeneric*, *pAdicFieldBaseGeneric*

An implementation of the p-adic numbers with lattice precision.

INPUT:

- `p` – prime
- `prec` – precision cap, given as a pair (`relative_cap`, `absolute_cap`)
- `subtype` – either `'cap'` or `'float'`
- `print_mode` – dictionary with print options
- `names` – how to print the prime

- `label` – the label of this ring

See also

label()

EXAMPLES:

```
sage: R = QpLC(next_prime(10^60))  # indirect doctest
doctest:...: FutureWarning: This class/method/function is marked as experimental.
It, its functionality or its interface might change without a formal deprecation.
See https://github.com/sagemath/sage/issues/23505 for details.
sage: type(R)
<class 'sage.rings.padics.padic_base_leaves.pAdicFieldLattice_with_category'>

sage: R = QpLC(2,label='init')  # indirect doctest
sage: R
2-adic Field with lattice-cap precision (label: init)
```

```
>>> from sage.all import *
>>> R = QpLC(next_prime(Integer(10)**Integer(60)))  # indirect doctest
doctest:...: FutureWarning: This class/method/function is marked as experimental.
It, its functionality or its interface might change without a formal deprecation.
See https://github.com/sagemath/sage/issues/23505 for details.
>>> type(R)
<class 'sage.rings.padics.padic_base_leaves.pAdicFieldLattice_with_category'>

>>> R = QpLC(Integer(2),label='init')  # indirect doctest
>>> R
2-adic Field with lattice-cap precision (label: init)
```

random_element(*prec=None*, *integral=False*)

Return a random element of this ring.

INPUT:

- `prec` – integer or `None` (default); the absolute precision of the generated random element
- `integral` – boolean (default: `False`); if `True`, return an element in the ring of integers

EXAMPLES:

```
sage: K = QpLC(2)
sage: K.random_element()   # not tested, known bug (see :issue:`32126`)
2^-8 + 2^-7 + 2^-6 + 2^-5 + 2^-3 + 1 + 2^2 + 2^3 + 2^5 + O(2^12)
sage: K.random_element(integral=True)     # random
2^3 + 2^4 + 2^5 + 2^6 + 2^7 + 2^10 + 2^11 + 2^14 + 2^15 + 2^16
 + 2^17 + 2^18 + 2^19 + O(2^20)

sage: K.random_element(prec=10)     # random
2^(-3) + 1 + 2 + 2^4 + 2^8 + O(2^10)
```

```
>>> from sage.all import *
>>> K = QpLC(Integer(2))
```

(continues on next page)

(continued from previous page)

```
>>> K.random_element()   # not tested, known bug (see :issue:`32126`)
2^-8 + 2^-7 + 2^-6 + 2^-5 + 2^-3 + 1 + 2^2 + 2^3 + 2^5 + O(2^12)
>>> K.random_element(integral=True)     # random
2^3 + 2^4 + 2^5 + 2^6 + 2^7 + 2^10 + 2^11 + 2^14 + 2^15 + 2^16
 + 2^17 + 2^18 + 2^19 + O(2^20)

>>> K.random_element(prec=Integer(10))     # random
2^(-3) + 1 + 2 + 2^4 + 2^8 + O(2^10)
```

If the given precision is higher than the internal cap of the parent, then the cap is used:

```
sage: K.precision_cap_relative()
20
sage: K.random_element(prec=100)     # random
2^5 + 2^8 + 2^11 + 2^12 + 2^14 + 2^18 + 2^20 + 2^24 + O(2^25)
```

```
>>> from sage.all import *
>>> K.precision_cap_relative()
20
>>> K.random_element(prec=Integer(100))     # random
2^5 + 2^8 + 2^11 + 2^12 + 2^14 + 2^18 + 2^20 + 2^24 + O(2^25)
```

class sage.rings.padics.padic_base_leaves.**pAdicFieldRelaxed**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicRelaxedGeneric*, *pAdicFieldBaseGeneric*

An implementation of relaxed arithmetics over $\mathbf{Q}_p$.

INPUT:

- `p` – prime
- `prec` – default precision
- `print_mode` – dictionary with print options
- `names` – how to print the prime

EXAMPLES:

```
sage: R = QpER(5)  # indirect doctest                                        #␣
↪needs sage.libs.flint
sage: type(R)                                                               #␣
↪needs sage.libs.flint
<class 'sage.rings.padics.padic_base_leaves.pAdicFieldRelaxed_with_category'>
```

```
>>> from sage.all import *
>>> R = QpER(Integer(5))  # indirect doctest                                 ␣
↪     # needs sage.libs.flint
>>> type(R)                                                                 #␣
↪needs sage.libs.flint
<class 'sage.rings.padics.padic_base_leaves.pAdicFieldRelaxed_with_category'>
```

class sage.rings.padics.padic_base_leaves.**pAdicRingCappedAbsolute**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicRingBaseGeneric*, *pAdicCappedAbsoluteRingGeneric*

An implementation of the p-adic integers with capped absolute precision.

class sage.rings.padics.padic_base_leaves.**pAdicRingCappedRelative**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicRingBaseGeneric*, *pAdicCappedRelativeRingGeneric*

An implementation of the p-adic integers with capped relative precision.

class sage.rings.padics.padic_base_leaves.**pAdicRingFixedMod**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicRingBaseGeneric*, *pAdicFixedModRingGeneric*

An implementation of the p-adic integers using fixed modulus.

class sage.rings.padics.padic_base_leaves.**pAdicRingFloatingPoint**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicRingBaseGeneric*, *pAdicFloatingPointRingGeneric*

An implementation of the p-adic integers with floating point precision.

class sage.rings.padics.padic_base_leaves.**pAdicRingLattice**(*p*, *prec*, *subtype*, *print_mode*, *names*, *label=None*, *category=None*)

Bases: *pAdicLatticeGeneric*, *pAdicRingBaseGeneric*

An implementation of the p-adic integers with lattice precision.

INPUT:

- `p` – prime
- `prec` – precision cap, given as a pair (`relative_cap`, `absolute_cap`)
- `subtype` – either `'cap'` or `'float'`
- `print_mode` – dictionary with print options
- `names` – how to print the prime
- `label` – the label of this ring

See also

label()

EXAMPLES:

```
sage: R = ZpLC(next_prime(10^60))  # indirect doctest
doctest:...: FutureWarning: This class/method/function is marked as experimental.
It, its functionality or its interface might change without a formal deprecation.
See https://github.com/sagemath/sage/issues/23505 for details.
sage: type(R)
<class 'sage.rings.padics.padic_base_leaves.pAdicRingLattice_with_category'>

sage: R = ZpLC(2, label='init')  # indirect doctest
sage: R
2-adic Ring with lattice-cap precision (label: init)
```

```
>>> from sage.all import *
>>> R = ZpLC(next_prime(Integer(10)**Integer(60)))  # indirect doctest
doctest:...: FutureWarning: This class/method/function is marked as experimental.
It, its functionality or its interface might change without a formal deprecation.
See https://github.com/sagemath/sage/issues/23505 for details.
>>> type(R)
<class 'sage.rings.padics.padic_base_leaves.pAdicRingLattice_with_category'>

>>> R = ZpLC(Integer(2), label='init')  # indirect doctest
>>> R
2-adic Ring with lattice-cap precision (label: init)
```

random_element(*prec=None*)

Return a random element of this ring.

INPUT:

- `prec` – integer or `None` (default); the absolute precision of the generated random element

EXAMPLES:

```
sage: R = ZpLC(2)
sage: R.random_element()     # random
2^3 + 2^4 + 2^5 + 2^6 + 2^7 + 2^10 + 2^11 + 2^14 + 2^15 + 2^16
 + 2^17 + 2^18 + 2^19 + 2^21 + O(2^23)

sage: R.random_element(prec=10)     # random
1 + 2^3 + 2^4 + 2^7 + O(2^10)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2))
>>> R.random_element()     # random
2^3 + 2^4 + 2^5 + 2^6 + 2^7 + 2^10 + 2^11 + 2^14 + 2^15 + 2^16
 + 2^17 + 2^18 + 2^19 + 2^21 + O(2^23)

>>> R.random_element(prec=Integer(10))     # random
1 + 2^3 + 2^4 + 2^7 + O(2^10)
```

class sage.rings.padics.padic_base_leaves.**pAdicRingRelaxed**(*p*, *prec*, *print_mode*, *names*, *category=None*)

Bases: *pAdicRelaxedGeneric*, *pAdicRingBaseGeneric*

An implementation of relaxed arithmetics over $\mathbf{Z}_p$.

INPUT:

- `p` – prime
- `prec` – default precision
- `print_mode` – dictionary with print options
- `names` – how to print the prime

EXAMPLES:

```
sage: R = ZpER(5)  # indirect doctest                                          #␣
↪needs sage.libs.flint
sage: type(R)                                                                  #␣
↪needs sage.libs.flint
<class 'sage.rings.padics.padic_base_leaves.pAdicRingRelaxed_with_category'>
```

```
>>> from sage.all import *
>>> R = ZpER(Integer(5))  # indirect doctest                                    ␣
↪     # needs sage.libs.flint
>>> type(R)                                                                   #␣
↪needs sage.libs.flint
<class 'sage.rings.padics.padic_base_leaves.pAdicRingRelaxed_with_category'>
```

CHAPTER

ELEVEN

p-ADIC EXTENSION LEAVES

The final classes for extensions of $\mathbf{Z}_p$ and $\mathbf{Q}_p$ (i.e., classes that are not just designed to be inherited from).

AUTHORS:

- David Roe

class sage.rings.padics.padic_extension_leaves.**EisensteinExtensionFieldCappedRelative**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='NTL'*)

Bases: *EisensteinExtensionGeneric*, *pAdicCappedRelativeFieldGeneric*

class sage.rings.padics.padic_extension_leaves.**EisensteinExtensionRingCappedAbsolute**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation*)

Bases: *EisensteinExtensionGeneric*, *pAdicCappedAbsoluteRingGeneric*

class sage.rings.padics.padic_extension_leaves.**EisensteinExtensionRingCappedRelative**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='NTL'*)

Bases: *EisensteinExtensionGeneric*, *pAdicCappedRelativeRingGeneric*

class sage.rings.padics.padic_extension_leaves.**EisensteinExtensionRingFixedMod**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='NTL'*)

Bases: *EisensteinExtensionGeneric*, *pAdicFixedModRingGeneric*

fraction_field()

Eisenstein extensions with fixed modulus do not support fraction fields.

EXAMPLES:

```
sage: S.<x> = ZZ[]
sage: R.<a> = ZpFM(5).extension(x^2 - 5)                                #␣
↪needs sage.libs.ntl
sage: R.fraction_field()                                                #␣
↪needs sage.libs.ntl
Traceback (most recent call last):
...
TypeError: This implementation of the p-adic ring
does not support fields of fractions.
```

```
>>> from sage.all import *
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> R = ZpFM(Integer(5)).extension(x**Integer(2) - Integer(5), names=('a',));␣
↪(a,) = R._first_ngens(1)# needs sage.libs.ntl
>>> R.fraction_field()                                                  #␣
↪needs sage.libs.ntl
Traceback (most recent call last):
...
TypeError: This implementation of the p-adic ring
does not support fields of fractions.
```

class sage.rings.padics.padic_extension_leaves.**UnramifiedExtensionFieldCappedRelative**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='FLINT'*)

Bases: *UnramifiedExtensionGeneric*, *pAdicCappedRelativeFieldGeneric*

class sage.rings.padics.padic_extension_leaves.**UnramifiedExtensionFieldFloatingPoint**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='FLINT'*)

Bases: *UnramifiedExtensionGeneric*, *pAdicFloatingPointFieldGeneric*

class sage.rings.padics.padic_extension_leaves.**UnramifiedExtensionRingCappedAbsolute**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='FLINT'*)

Bases: *UnramifiedExtensionGeneric*, *pAdicCappedAbsoluteRingGeneric*

class sage.rings.padics.padic_extension_leaves.**UnramifiedExtensionRingCappedRelative**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='FLINT'*)

Bases: *UnramifiedExtensionGeneric*, *pAdicCappedRelativeRingGeneric*

class sage.rings.padics.padic_extension_leaves.**UnramifiedExtensionRingFixedMod**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='FLINT'*)

Bases: *UnramifiedExtensionGeneric*, *pAdicFixedModRingGeneric*

class sage.rings.padics.padic_extension_leaves.**UnramifiedExtensionRingFloatingPoint**(*exact_modulus*, *poly*, *prec*, *print_mode*, *shift_seed*, *names*, *implementation='FLINT'*)

Bases: *UnramifiedExtensionGeneric*, *pAdicFloatingPointRingGeneric*

CHAPTER

TWELVE

LOCAL GENERIC ELEMENT

This file contains a common superclass for p-adic elements and power series elements.

AUTHORS:

- David Roe: initial version
- Julian Rueth (2012-10-15, 2014-06-25, 2017-08-04): added inverse_of_unit(); improved add_bigoh(); added _test_expansion()

class sage.rings.padics.local_generic_element.**LocalGenericElement**

Bases: CommutativeRingElement

add_bigoh(*absprec*)

Return a copy of this element with absolute precision decreased to absprec.

INPUT:

- absprec – integer or positive infinity

EXAMPLES:

```
sage: K = QpCR(3,4)
sage: o = K(1); o
1 + O(3^4)
sage: o.add_bigoh(2)
1 + O(3^2)
sage: o.add_bigoh(-5)
O(3^-5)
```

```
>>> from sage.all import *
>>> K = QpCR(Integer(3),Integer(4))
>>> o = K(Integer(1)); o
1 + O(3^4)
>>> o.add_bigoh(Integer(2))
1 + O(3^2)
>>> o.add_bigoh(-Integer(5))
O(3^-5)
```

One cannot use add_bigoh to lift to a higher precision; this can be accomplished with lift_to_precision():

```
sage: o.add_bigoh(5)
1 + O(3^4)
```

```
>>> from sage.all import *
>>> o.add_bigoh(Integer(5))
1 + O(3^4)
```

Negative values of `absprec` return an element in the fraction field of the element's parent:

```
sage: R = ZpCA(3,4)
sage: R(3).add_bigoh(-5)
O(3^-5)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(3),Integer(4))
>>> R(Integer(3)).add_bigoh(-Integer(5))
O(3^-5)
```

For fixed-mod elements this method truncates the element:

```
sage: R = ZpFM(3,4)
sage: R(3).add_bigoh(1)
0
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(3),Integer(4))
>>> R(Integer(3)).add_bigoh(Integer(1))
0
```

If `absprec` exceeds the precision of the element, then this method has no effect:

```
sage: R(3).add_bigoh(5)
3
```

```
>>> from sage.all import *
>>> R(Integer(3)).add_bigoh(Integer(5))
3
```

A negative value for `absprec` returns an element in the fraction field:

```
sage: R(3).add_bigoh(-1).parent()
3-adic Field with floating precision 4
```

```
>>> from sage.all import *
>>> R(Integer(3)).add_bigoh(-Integer(1)).parent()
3-adic Field with floating precision 4
```

euclidean_degree()

Return the degree of this element as an element of a Euclidean domain.

EXAMPLES:

For a field, this is always zero except for the zero element:

```
sage: K = Qp(2)
sage: K.one().euclidean_degree()
```

(continues on next page)

(continued from previous page)

```
0
sage: K.gen().euclidean_degree()
0
sage: K.zero().euclidean_degree()
Traceback (most recent call last):
...
ValueError: euclidean degree not defined for the zero element
```

```
>>> from sage.all import *
>>> K = Qp(Integer(2))
>>> K.one().euclidean_degree()
0
>>> K.gen().euclidean_degree()
0
>>> K.zero().euclidean_degree()
Traceback (most recent call last):
...
ValueError: euclidean degree not defined for the zero element
```

For a ring which is not a field, this is the valuation of the element:

```
sage: R = Zp(2)
sage: R.one().euclidean_degree()
0
sage: R.gen().euclidean_degree()
1
sage: R.zero().euclidean_degree()
Traceback (most recent call last):
...
ValueError: euclidean degree not defined for the zero element
```

```
>>> from sage.all import *
>>> R = Zp(Integer(2))
>>> R.one().euclidean_degree()
0
>>> R.gen().euclidean_degree()
1
>>> R.zero().euclidean_degree()
Traceback (most recent call last):
...
ValueError: euclidean degree not defined for the zero element
```

inverse_of_unit()

Return the inverse of `self` if `self` is a unit.

OUTPUT: an element in the same ring as `self`

EXAMPLES:

```
sage: R = ZpCA(3,5)
sage: a = R(2); a
2 + O(3^5)
```

(continues on next page)

(continued from previous page)

```
sage: b = a.inverse_of_unit(); b
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(3),Integer(5))
>>> a = R(Integer(2)); a
2 + O(3^5)
>>> b = a.inverse_of_unit(); b
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
```

A `ZeroDivisionError` is raised if an element has no inverse in the ring:

```
sage: R(3).inverse_of_unit()
Traceback (most recent call last):
...
ZeroDivisionError: inverse of 3 + O(3^5) does not exist
```

```
>>> from sage.all import *
>>> R(Integer(3)).inverse_of_unit()
Traceback (most recent call last):
...
ZeroDivisionError: inverse of 3 + O(3^5) does not exist
```

Unlike the usual inverse of an element, the result is in the same ring as `self` and not just in its fraction field:

```
sage: c = ~a; c
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
sage: a.parent()
3-adic Ring with capped absolute precision 5
sage: b.parent()
3-adic Ring with capped absolute precision 5
sage: c.parent()
3-adic Field with capped relative precision 5
```

```
>>> from sage.all import *
>>> c = ~a; c
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
>>> a.parent()
3-adic Ring with capped absolute precision 5
>>> b.parent()
3-adic Ring with capped absolute precision 5
>>> c.parent()
3-adic Field with capped relative precision 5
```

For fields this does of course not make any difference:

```
sage: R = QpCR(3,5)
sage: a = R(2)
sage: b = a.inverse_of_unit()
sage: c = ~a
sage: a.parent()
3-adic Field with capped relative precision 5
```

(continues on next page)

(continued from previous page)

```
sage: b.parent()
3-adic Field with capped relative precision 5
sage: c.parent()
3-adic Field with capped relative precision 5
```

```
>>> from sage.all import *
>>> R = QpCR(Integer(3),Integer(5))
>>> a = R(Integer(2))
>>> b = a.inverse_of_unit()
>>> c = ~a
>>> a.parent()
3-adic Field with capped relative precision 5
>>> b.parent()
3-adic Field with capped relative precision 5
>>> c.parent()
3-adic Field with capped relative precision 5
```

is_integral()

Return whether `self` is an integral element.

INPUT:

- `self` – a local ring element

OUTPUT: boolean; whether `self` is an integral element

EXAMPLES:

```
sage: R = Qp(3,20)
sage: a = R(7/3); a.is_integral()
False
sage: b = R(7/5); b.is_integral()
True
```

```
>>> from sage.all import *
>>> R = Qp(Integer(3),Integer(20))
>>> a = R(Integer(7)/Integer(3)); a.is_integral()
False
>>> b = R(Integer(7)/Integer(5)); b.is_integral()
True
```

is_padic_unit()

Return whether `self` is a p-adic unit. That is, whether it has zero valuation.

INPUT:

- `self` – a local ring element

OUTPUT: boolean; whether `self` is a unit

EXAMPLES:

```
sage: R = Zp(3,20,'capped-rel'); K = Qp(3,20,'capped-rel')
sage: R(0).is_padic_unit()
False
```

(continues on next page)

(continued from previous page)

```
sage: R(1).is_padic_unit()
True
sage: R(2).is_padic_unit()
True
sage: R(3).is_padic_unit()
False
sage: Qp(5,5)(5).is_padic_unit()
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(20),'capped-rel'); K = Qp(Integer(3),
↪Integer(20),'capped-rel')
>>> R(Integer(0)).is_padic_unit()
False
>>> R(Integer(1)).is_padic_unit()
True
>>> R(Integer(2)).is_padic_unit()
True
>>> R(Integer(3)).is_padic_unit()
False
>>> Qp(Integer(5),Integer(5))(Integer(5)).is_padic_unit()
False
```

is_unit()

Return whether `self` is a unit.

INPUT:

- `self` – a local ring element

OUTPUT: boolean; whether `self` is a unit

Note

For fields all nonzero elements are units. For DVR's, only those elements of valuation 0 are. An older implementation ignored the case of fields, and returned always the negation of self.valuation()==0. This behavior is now supported with self.is_padic_unit().

EXAMPLES:

```
sage: R = Zp(3,20,'capped-rel'); K = Qp(3,20,'capped-rel')
sage: R(0).is_unit()
False
sage: R(1).is_unit()
True
sage: R(2).is_unit()
True
sage: R(3).is_unit()
False
sage: Qp(5,5)(5).is_unit()  # Note that 5 is invertible in `QQ_5`, even if it␣
↪has positive valuation!
True
```

(continues on next page)

(continued from previous page)

```
sage: Qp(5,5)(5).is_padic_unit()
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(20),'capped-rel'); K = Qp(Integer(3),
↪Integer(20),'capped-rel')
>>> R(Integer(0)).is_unit()
False
>>> R(Integer(1)).is_unit()
True
>>> R(Integer(2)).is_unit()
True
>>> R(Integer(3)).is_unit()
False
>>> Qp(Integer(5),Integer(5))(Integer(5)).is_unit()  # Note that 5 is␣
↪invertible in `QQ_5`, even if it has positive valuation!
True
>>> Qp(Integer(5),Integer(5))(Integer(5)).is_padic_unit()
False
```

normalized_valuation()

Return the normalized valuation of this local ring element, i.e., the valuation divided by the absolute ramification index.

INPUT:

- `self` – a local ring element

OUTPUT: rational; the normalized valuation of `self`

EXAMPLES:

```
sage: Q7 = Qp(7)
sage: R.<x> = Q7[]                                                          #␣
↪needs sage.libs.ntl
sage: F.<z> = Q7.ext(x^3+7*x+7)                                             #␣
↪needs sage.libs.ntl
sage: z.normalized_valuation()                                              #␣
↪needs sage.libs.ntl
1/3
```

```
>>> from sage.all import *
>>> Q7 = Qp(Integer(7))
>>> R = Q7['x']; (x,) = R._first_ngens(1)# needs sage.libs.ntl
>>> F = Q7.ext(x**Integer(3)+Integer(7)*x+Integer(7), names=('z',)); (z,) = F.
↪_first_ngens(1)# needs sage.libs.ntl
>>> z.normalized_valuation()                                                #␣
↪needs sage.libs.ntl
1/3
```

quo_rem(*other*, *integral=False*)

Return the quotient with remainder of the division of this element by `other`.

INPUT:

- `other` – an element in the same ring
- `integral` – if `True`, use integral-style remainders even when the parent is a field. Namely, the remainder will have no terms in its p-adic expansion above the valuation of `other`.

EXAMPLES:

```
sage: R = Zp(3, 5)
sage: R(12).quo_rem(R(2))
(2*3 + O(3^6), 0)
sage: R(2).quo_rem(R(12))
(O(3^4), 2 + O(3^5))

sage: K = Qp(3, 5)
sage: K(12).quo_rem(K(2))
(2*3 + O(3^6), 0)
sage: K(2).quo_rem(K(12))
(2*3^-1 + 1 + 3 + 3^2 + 3^3 + O(3^4), 0)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(5))
>>> R(Integer(12)).quo_rem(R(Integer(2)))
(2*3 + O(3^6), 0)
>>> R(Integer(2)).quo_rem(R(Integer(12)))
(O(3^4), 2 + O(3^5))

>>> K = Qp(Integer(3), Integer(5))
>>> K(Integer(12)).quo_rem(K(Integer(2)))
(2*3 + O(3^6), 0)
>>> K(Integer(2)).quo_rem(K(Integer(12)))
(2*3^-1 + 1 + 3 + 3^2 + 3^3 + O(3^4), 0)
```

You can get the same behavior for fields as for rings by using integral=True:

```
sage: K(12).quo_rem(K(2), integral=True)
(2*3 + O(3^6), 0)
sage: K(2).quo_rem(K(12), integral=True)
(O(3^4), 2 + O(3^5))
```

```
>>> from sage.all import *
>>> K(Integer(12)).quo_rem(K(Integer(2)), integral=True)
(2*3 + O(3^6), 0)
>>> K(Integer(2)).quo_rem(K(Integer(12)), integral=True)
(O(3^4), 2 + O(3^5))
```

slice(*i*, *j*, *k=1*, *lift_mode='simple'*)

Return the sum of the $pi^{i+l\cdot k}$ terms of the series expansion of this element, where pi is the uniformizer, for $i + l \cdot k$ between `i` and `j-1` inclusive, and nonnegative integers l. Behaves analogously to the slice function for lists.

INPUT:

- `i` – integer; if set to `None`, the sum will start with the first nonzero term of the series
- `j` – integer; if set to `None` or ∞, this method behaves as if it was set to the absolute precision of this element

- k – (default: 1) a positive integer

EXAMPLES:

```
sage: R = Zp(5, 6, 'capped-rel')
sage: a = R(1/2); a
3 + 2*5 + 2*5^2 + 2*5^3 + 2*5^4 + 2*5^5 + O(5^6)
sage: a.slice(2, 4)
2*5^2 + 2*5^3 + O(5^4)
sage: a.slice(1, 6, 2)
2*5 + 2*5^3 + 2*5^5 + O(5^6)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(6), 'capped-rel')
>>> a = R(Integer(1)/Integer(2)); a
3 + 2*5 + 2*5^2 + 2*5^3 + 2*5^4 + 2*5^5 + O(5^6)
>>> a.slice(Integer(2), Integer(4))
2*5^2 + 2*5^3 + O(5^4)
>>> a.slice(Integer(1), Integer(6), Integer(2))
2*5 + 2*5^3 + 2*5^5 + O(5^6)
```

The step size k has to be positive:

```
sage: a.slice(0, 3, 0)
Traceback (most recent call last):
...
ValueError: slice step must be positive
sage: a.slice(0, 3, -1)
Traceback (most recent call last):
...
ValueError: slice step must be positive
```

```
>>> from sage.all import *
>>> a.slice(Integer(0), Integer(3), Integer(0))
Traceback (most recent call last):
...
ValueError: slice step must be positive
>>> a.slice(Integer(0), Integer(3), -Integer(1))
Traceback (most recent call last):
...
ValueError: slice step must be positive
```

If i exceeds j, then the result will be zero, with the precision given by j:

```
sage: a.slice(5, 4)
O(5^4)
sage: a.slice(6, 5)
O(5^5)
```

```
>>> from sage.all import *
>>> a.slice(Integer(5), Integer(4))
O(5^4)
>>> a.slice(Integer(6), Integer(5))
O(5^5)
```

However, the precision cannot exceed the precision of the element:

```
sage: a.slice(101,100)
O(5^6)
sage: a.slice(0,5,2)
3 + 2*5^2 + 2*5^4 + O(5^5)
sage: a.slice(0,6,2)
3 + 2*5^2 + 2*5^4 + O(5^6)
sage: a.slice(0,7,2)
3 + 2*5^2 + 2*5^4 + O(5^6)
```

```
>>> from sage.all import *
>>> a.slice(Integer(101),Integer(100))
O(5^6)
>>> a.slice(Integer(0),Integer(5),Integer(2))
3 + 2*5^2 + 2*5^4 + O(5^5)
>>> a.slice(Integer(0),Integer(6),Integer(2))
3 + 2*5^2 + 2*5^4 + O(5^6)
>>> a.slice(Integer(0),Integer(7),Integer(2))
3 + 2*5^2 + 2*5^4 + O(5^6)
```

If start is left blank, it is set to the valuation:

```
sage: K = Qp(5, 6)
sage: x = K(1/25 + 5); x
5^-2 + 5 + O(5^4)
sage: x.slice(None, 3)
5^-2 + 5 + O(5^3)
sage: x[:3]
doctest:warning
...
DeprecationWarning: __getitem__ is changing to match the behavior of number␣
↪fields. Please use expansion instead.
See https://github.com/sagemath/sage/issues/14825 for details.
5^-2 + 5 + O(5^3)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(5), Integer(6))
>>> x = K(Integer(1)/Integer(25) + Integer(5)); x
5^-2 + 5 + O(5^4)
>>> x.slice(None, Integer(3))
5^-2 + 5 + O(5^3)
>>> x[:Integer(3)]
doctest:warning
...
DeprecationWarning: __getitem__ is changing to match the behavior of number␣
↪fields. Please use expansion instead.
See https://github.com/sagemath/sage/issues/14825 for details.
5^-2 + 5 + O(5^3)
```

sqrt(*extend=True*, *all=False*, *algorithm=None*)

Return the square root of this element.

INPUT:

- `self` – a p-adic element
- `extend` – boolean (default: `True`); if `True`, return a square root in an extension if necessary; if `False` and no root exists in the given ring or field, raise a `ValueError`.
- `all` – boolean (default: `False`); if `True`, return a list of all square roots
- `algorithm` – `'pari'`, `'sage'` or `None` (default: `None`); Sage provides an implementation for any extension of Q_p whereas only square roots over Q_p is implemented in Pari; the default is `'pari'` if the ground field is Q_p, `'sage'` otherwise.

OUTPUT: the square root or the list of all square roots of this element

Note

The square root is chosen (resp. the square roots are ordered) in a deterministic way, which is compatible with change of precision.

EXAMPLES:

```
sage: R = Zp(3, 20)
sage: sqrt(R(0))
0

sage: sqrt(R(1))
1 + O(3^20)

sage: R(2).sqrt(extend=False)
Traceback (most recent call last):
...
ValueError: element is not a square

sage: s = sqrt(R(4)); -s
2 + O(3^20)

sage: s = sqrt(R(9)); s
3 + O(3^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(20))
>>> sqrt(R(Integer(0)))
0

>>> sqrt(R(Integer(1)))
1 + O(3^20)

>>> R(Integer(2)).sqrt(extend=False)
Traceback (most recent call last):
...
ValueError: element is not a square

>>> s = sqrt(R(Integer(4))); -s
2 + O(3^20)
```

(continues on next page)

(continued from previous page)

```
>>> s = sqrt(R(Integer(9))); s
3 + O(3^21)
```

Over the 2-adics, the precision of the square root is less than the input:

```
sage: R2 = Zp(2, 20)
sage: sqrt(R2(1))
1 + O(2^19)
sage: sqrt(R2(4))
2 + O(2^20)

sage: R.<t> = Zq(2^10, 10)                                                  #␣
↪needs sage.libs.ntl
sage: u = 1 + 8*t                                                           #␣
↪needs sage.libs.ntl
sage: sqrt(u)                                                               #␣
↪needs sage.libs.ntl
1 + t*2^2 + t^2*2^3 + t^2*2^4 + (t^4 + t^3 + t^2)*2^5 + (t^4 + t^2)*2^6
  + (t^5 + t^2)*2^7 + (t^6 + t^5 + t^4 + t^2)*2^8 + O(2^9)

sage: R.<a> = Zp(2).extension(x^3 - 2)
sage: u = R(1 + a^4 + a^5 + a^7 + a^8, 10); u
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
sage: v = sqrt(u); v                                                        #␣
↪needs sage.libs.ntl
1 + a^2 + a^4 + a^6 + O(a^7)
```

```
>>> from sage.all import *
>>> R2 = Zp(Integer(2), Integer(20))
>>> sqrt(R2(Integer(1)))
1 + O(2^19)
>>> sqrt(R2(Integer(4)))
2 + O(2^20)

>>> R = Zq(Integer(2)**Integer(10), Integer(10), names=('t',)); (t,) = R._
↪first_ngens(1)# needs sage.libs.ntl
>>> u = Integer(1) + Integer(8)*t                                           ␣
↪              # needs sage.libs.ntl
>>> sqrt(u)                                                                #␣
↪needs sage.libs.ntl
1 + t*2^2 + t^2*2^3 + t^2*2^4 + (t^4 + t^3 + t^2)*2^5 + (t^4 + t^2)*2^6
  + (t^5 + t^2)*2^7 + (t^6 + t^5 + t^4 + t^2)*2^8 + O(2^9)

>>> R = Zp(Integer(2)).extension(x**Integer(3) - Integer(2), names=('a',));␣
↪(a,) = R._first_ngens(1)
>>> u = R(Integer(1) + a**Integer(4) + a**Integer(5) + a**Integer(7) +␣
↪a**Integer(8), Integer(10)); u
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
>>> v = sqrt(u); v                                                         #␣
↪needs sage.libs.ntl
1 + a^2 + a^4 + a^6 + O(a^7)
```

However, observe that the precision increases to its original value when we recompute the square of the square

root:

```
sage: v^2                                                                   #␣
↪needs sage.libs.ntl
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
```

```
>>> from sage.all import *
>>> v**Integer(2)                                                           ␣
↪       # needs sage.libs.ntl
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
```

If the input does not have enough precision in order to determine if the given element has a square root in the ground field, an error is raised:

```
sage: R(1, 6).sqrt()
Traceback (most recent call last):
...
PrecisionError: not enough precision to be sure that this element has a␣
↪square root

sage: R(1, 7).sqrt()
1 + O(a^4)

sage: R(1+a^6, 7).sqrt(extend=False)
Traceback (most recent call last):
...
ValueError: element is not a square
```

```
>>> from sage.all import *
>>> R(Integer(1), Integer(6)).sqrt()
Traceback (most recent call last):
...
PrecisionError: not enough precision to be sure that this element has a␣
↪square root

>>> R(Integer(1), Integer(7)).sqrt()
1 + O(a^4)

>>> R(Integer(1)+a**Integer(6), Integer(7)).sqrt(extend=False)
Traceback (most recent call last):
...
ValueError: element is not a square
```

In particular, an error is raised when we try to compute the square root of an inexact

CHAPTER

THIRTEEN

p-ADIC GENERIC ELEMENT

Elements of p-adic Rings and Fields

AUTHORS:

- David Roe
- Genya Zaytman: documentation
- David Harvey: doctests
- Julian Rueth: fixes for exp() and log(), implemented gcd, xgcd

sage.rings.padics.padic_generic_element.**dwork_mahler_coeffs**(*R*, *bd=20*)

Compute Dwork's formula for Mahler coefficients of p-adic Gamma.

This is called internally when one computes Gamma for a p-adic integer. Normally there is no need to call it directly.

INPUT:

- `R` – p-adic ring in which to compute
- `bd` – integer; number of terms in the expansion to use

OUTPUT: a list of p-adic integers

EXAMPLES:

```
sage: from sage.rings.padics.padic_generic_element import dwork_mahler_coeffs,␣
↪evaluate_dwork_mahler
sage: R = Zp(3)
sage: v = dwork_mahler_coeffs(R)
sage: x = R(1/7)
sage: evaluate_dwork_mahler(v, x, 3, 20, 1)
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
sage: x.dwork_expansion(a=1)  # Same result
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_generic_element import dwork_mahler_coeffs,␣
↪evaluate_dwork_mahler
>>> R = Zp(Integer(3))
>>> v = dwork_mahler_coeffs(R)
>>> x = R(Integer(1)/Integer(7))
```

(continues on next page)

(continued from previous page)

```
>>> evaluate_dwork_mahler(v, x, Integer(3), Integer(20), Integer(1))
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
>>> x.dwork_expansion(a=Integer(1))  # Same result
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
```

sage.rings.padics.padic_generic_element.**evaluate_dwork_mahler**(*v*, *x*, *p*, *bd*, *a*)

Evaluate Dwork's Mahler series for p-adic Gamma.

EXAMPLES:

```
sage: from sage.rings.padics.padic_generic_element import dwork_mahler_coeffs,␣
↪evaluate_dwork_mahler
sage: R = Zp(3)
sage: v = dwork_mahler_coeffs(R)
sage: x = R(1/7)
sage: evaluate_dwork_mahler(v, x, 3, 20, 1)
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
sage: x.dwork_expansion(a=1)  # Same result
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_generic_element import dwork_mahler_coeffs,␣
↪evaluate_dwork_mahler
>>> R = Zp(Integer(3))
>>> v = dwork_mahler_coeffs(R)
>>> x = R(Integer(1)/Integer(7))
>>> evaluate_dwork_mahler(v, x, Integer(3), Integer(20), Integer(1))
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
>>> x.dwork_expansion(a=Integer(1))  # Same result
2 + 2*3 + 3^2 + 3^3 + 3^4 + 3^5 + 2*3^6 + 2*3^7 + 2*3^8 + 2*3^9 + 2*3^11
 + 2*3^12 + 3^13 + 3^14 + 2*3^16 + 3^17 + 3^19 + O(3^20)
```

sage.rings.padics.padic_generic_element.**gauss_table**(*p*, *f*, *prec*, *use_longs*)

Compute a table of Gauss sums using the Gross-Koblitz formula.

This is used in the computation of L-functions of hypergeometric motives. The Gross-Koblitz formula is used as in $sage.rings.padics.misc.gauss_sum$, but further unpacked for efficiency.

INPUT:

- `p` – prime
- `f`, `prec` – positive integers
- **`use_longs` – boolean; if `True`, computations are done in C `long long`** integers rather than Sage p-adics, and the results are returned as a Python array rather than a list.

OUTPUT:

A list of length $q - 1 = p^f - 1$. The entries are p-adic units created with absolute precision `prec`.

EXAMPLES:

```
sage: from sage.rings.padics.padic_generic_element import gauss_table
sage: gauss_table(2,2,4,False)
[1 + 2 + 2^2 + 2^3, 1 + 2 + 2^2 + 2^3, 1 + 2 + 2^2 + 2^3]
sage: gauss_table(3,2,4,False)[3]
2 + 3 + 2*3^2
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_generic_element import gauss_table
>>> gauss_table(Integer(2),Integer(2),Integer(4),False)
[1 + 2 + 2^2 + 2^3, 1 + 2 + 2^2 + 2^3, 1 + 2 + 2^2 + 2^3]
>>> gauss_table(Integer(3),Integer(2),Integer(4),False)[Integer(3)]
2 + 3 + 2*3^2
```

class sage.rings.padics.padic_generic_element.**pAdicGenericElement**

Bases: *LocalGenericElement*

abs(*prec=None*)

Return the p-adic absolute value of `self`.

This is normalized so that the absolute value of p is $1/p$.

INPUT:

- `prec` – integer; the precision of the real field in which the answer is returned. If `None`, returns a rational for absolutely unramified fields, or a real with 53 bits of precision for ramified fields.

EXAMPLES:

```
sage: a = Qp(5)(15); a.abs()
1/5
sage: a.abs(53)
0.20000000000000000
sage: Qp(7)(0).abs()
0
sage: Qp(7)(0).abs(prec=20)
0.00000
```

```
>>> from sage.all import *
>>> a = Qp(Integer(5))(Integer(15)); a.abs()
1/5
>>> a.abs(Integer(53))
0.20000000000000000
>>> Qp(Integer(7))(Integer(0)).abs()
0
>>> Qp(Integer(7))(Integer(0)).abs(prec=Integer(20))
0.00000
```

An unramified extension:

```
sage: R = Zp(5,5)
sage: P.<x> = PolynomialRing(R)
sage: Z25.<u> = R.ext(x^2 - 3)
sage: u.abs()
1
```

(continues on next page)

(continued from previous page)

```
sage: (u^24-1).abs()
1/5
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> P = PolynomialRing(R, names=('x',)); (x,) = P._first_ngens(1)
>>> Z25 = R.ext(x**Integer(2) - Integer(3), names=('u',)); (u,) = Z25._first_
↪ngens(1)
>>> u.abs()
1
>>> (u**Integer(24)-Integer(1)).abs()
1/5
```

A ramified extension:

```
sage: W.<w> = R.ext(x^5 + 75*x^3 - 15*x^2 + 125*x - 5)
sage: w.abs()
0.724779663677696
sage: W(0).abs()
0.000000000000000
```

```
>>> from sage.all import *
>>> W = R.ext(x**Integer(5) + Integer(75)*x**Integer(3) -␣
↪Integer(15)*x**Integer(2) + Integer(125)*x - Integer(5), names=('w',)); (w,
↪) = W._first_ngens(1)
>>> w.abs()
0.724779663677696
>>> W(Integer(0)).abs()
0.000000000000000
```

additive_order(*prec=None*)

Return the additive order of this element truncated at precision `prec`.

INPUT:

- `prec` – integer or `None` (default: `None`)

OUTPUT: the additive order of this element

EXAMPLES:

```
sage: R = Zp(7, 4, 'capped-rel', 'series'); a = R(7^3); a.additive_order(3)
1
sage: a.additive_order(4)
+Infinity
sage: R = Zp(7, 4, 'fixed-mod', 'series'); a = R(7^5); a.additive_order(6)
1
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7), Integer(4), 'capped-rel', 'series'); a =␣
↪R(Integer(7)**Integer(3)); a.additive_order(Integer(3))
1
>>> a.additive_order(Integer(4))
```

(continues on next page)

(continued from previous page)

```
+Infinity
>>> R = Zp(Integer(7), Integer(4), 'fixed-mod', 'series'); a =␣
↪R(Integer(7)**Integer(5)); a.additive_order(Integer(6))
1
```

algdep(*n*)

alias of *algebraic_dependency()*.

algebraic_dependency(*n*)

Return a polynomial of degree at most n which is approximately satisfied by this number.

Note that the returned polynomial need not be irreducible, and indeed usually will not be if this number is a good approximation to an algebraic number of degree less than n.

ALGORITHM: Uses the PARI C-library pari:algdep command.

INPUT:

- `self` – a p-adic element
- `n` – integer

OUTPUT:

polynomial; degree n polynomial approximately satisfied by `self`

EXAMPLES:

```
sage: K = Qp(3,20,'capped-rel','series'); R = Zp(3,20,'capped-rel','series')
sage: a = K(7/19); a
1 + 2*3 + 3^2 + 3^3 + 2*3^4 + 2*3^5 + 3^8 + 2*3^9 + 3^11 + 3^12
  + 2*3^15 + 2*3^16 + 3^17 + 2*3^19 + O(3^20)
sage: a.algebraic_dependency(1)
19*x - 7
sage: K2 = Qp(7,20,'capped-rel')
sage: b = K2.zeta(); b.algebraic_dependency(2)
x^2 - x + 1
sage: K2 = Qp(11,20,'capped-rel')
sage: b = K2.zeta(); b.algebraic_dependency(4)
x^4 - x^3 + x^2 - x + 1
sage: a = R(7/19); a
1 + 2*3 + 3^2 + 3^3 + 2*3^4 + 2*3^5 + 3^8 + 2*3^9 + 3^11 + 3^12
  + 2*3^15 + 2*3^16 + 3^17 + 2*3^19 + O(3^20)
sage: a.algebraic_dependency(1)
19*x - 7
sage: R2 = Zp(7,20,'capped-rel')
sage: b = R2.zeta(); b.algebraic_dependency(2)
x^2 - x + 1
sage: R2 = Zp(11,20,'capped-rel')
sage: b = R2.zeta(); b.algebraic_dependency(4)
x^4 - x^3 + x^2 - x + 1
```

```
>>> from sage.all import *
>>> K = Qp(Integer(3),Integer(20),'capped-rel','series'); R = Zp(Integer(3),
↪Integer(20),'capped-rel','series')
>>> a = K(Integer(7)/Integer(19)); a
```

(continues on next page)

(continued from previous page)

```
1 + 2*3 + 3^2 + 3^3 + 2*3^4 + 2*3^5 + 3^8 + 2*3^9 + 3^11 + 3^12
  + 2*3^15 + 2*3^16 + 3^17 + 2*3^19 + O(3^20)
>>> a.algebraic_dependency(Integer(1))
19*x - 7
>>> K2 = Qp(Integer(7),Integer(20),'capped-rel')
>>> b = K2.zeta(); b.algebraic_dependency(Integer(2))
x^2 - x + 1
>>> K2 = Qp(Integer(11),Integer(20),'capped-rel')
>>> b = K2.zeta(); b.algebraic_dependency(Integer(4))
x^4 - x^3 + x^2 - x + 1
>>> a = R(Integer(7)/Integer(19)); a
1 + 2*3 + 3^2 + 3^3 + 2*3^4 + 2*3^5 + 3^8 + 2*3^9 + 3^11 + 3^12
  + 2*3^15 + 2*3^16 + 3^17 + 2*3^19 + O(3^20)
>>> a.algebraic_dependency(Integer(1))
19*x - 7
>>> R2 = Zp(Integer(7),Integer(20),'capped-rel')
>>> b = R2.zeta(); b.algebraic_dependency(Integer(2))
x^2 - x + 1
>>> R2 = Zp(Integer(11),Integer(20),'capped-rel')
>>> b = R2.zeta(); b.algebraic_dependency(Integer(4))
x^4 - x^3 + x^2 - x + 1
```

artin_hasse_exp(*prec=None*, *algorithm=None*)

Return the Artin-Hasse exponential of this element.

INPUT:

- `prec` – integer or `None` (default: `None`); the desired precision on the result; if `None`, the precision is derived from the precision on the input
- `algorithm` – `'direct'`, `'series'`, `'newton'` or `None` (default)

  The direct algorithm computes the Artin-Hasse exponential of x, namely `AH(x)` as

  $$AH(x) = \exp(x + \frac{x^p}{p} + \frac{x^{p^2}}{p^2} + \dots$$

  It runs roughly as fast as the computation of the exponential (since the computation of the argument is not that costly).

  The series algorithm computes the series defining the Artin-Hasse exponential and evaluates it.

  The `'newton'` algorithm solves the equation

  $$\log(AH(x)) = x + \frac{x^p}{p} + \frac{x^{p^2}}{p^2} + \dots$$

  using a Newton scheme. It runs roughly as fast as the computation of the logarithm.

  By default, we use the `'direct'` algorithm if a fast algorithm for computing the exponential is available. If not, we use the `'newton'` algorithm if a fast algorithm for computing the logarithm is available. Otherwise we switch to the `'series'` algorithm.

OUTPUT: the Artin-Hasse exponential of this element

See Wikipedia article Artin-Hasse_exponential for more information.

EXAMPLES:

```
sage: x = Zp(5)(45/7)
sage: y = x.artin_hasse_exp(); y
1 + 2*5 + 4*5^2 + 3*5^3 + 5^7 + 2*5^8 + 3*5^10 + 2*5^11 + 2*5^12 +
2*5^13 + 5^14 + 3*5^17 + 2*5^18 + 2*5^19 + O(5^20)

sage: y * (-x).artin_hasse_exp()
1 + O(5^20)
```

```
>>> from sage.all import *
>>> x = Zp(Integer(5))(Integer(45)/Integer(7))
>>> y = x.artin_hasse_exp(); y
1 + 2*5 + 4*5^2 + 3*5^3 + 5^7 + 2*5^8 + 3*5^10 + 2*5^11 + 2*5^12 +
2*5^13 + 5^14 + 3*5^17 + 2*5^18 + 2*5^19 + O(5^20)

>>> y * (-x).artin_hasse_exp()
1 + O(5^20)
```

The function respects your precision:

```
sage: x = Zp(3,30)(45/7)
sage: x.artin_hasse_exp()
1 + 2*3^2 + 3^4 + 2*3^5 + 3^6 + 2*3^7 + 2*3^8 + 3^9 + 2*3^10 + 3^11 +
3^13 + 2*3^15 + 2*3^16 + 2*3^17 + 3^19 + 3^20 + 2*3^21 + 3^23 + 3^24 +
3^26 + 3^27 + 2*3^28 + O(3^30)
```

```
>>> from sage.all import *
>>> x = Zp(Integer(3),Integer(30))(Integer(45)/Integer(7))
>>> x.artin_hasse_exp()
1 + 2*3^2 + 3^4 + 2*3^5 + 3^6 + 2*3^7 + 2*3^8 + 3^9 + 2*3^10 + 3^11 +
3^13 + 2*3^15 + 2*3^16 + 2*3^17 + 3^19 + 3^20 + 2*3^21 + 3^23 + 3^24 +
3^26 + 3^27 + 2*3^28 + O(3^30)
```

Unless you tell it not to:

```
sage: x = Zp(3,30)(45/7)
sage: x.artin_hasse_exp()
1 + 2*3^2 + 3^4 + 2*3^5 + 3^6 + 2*3^7 + 2*3^8 + 3^9 + 2*3^10 + 3^11 +
3^13 + 2*3^15 + 2*3^16 + 2*3^17 + 3^19 + 3^20 + 2*3^21 + 3^23 + 3^24 +
3^26 + 3^27 + 2*3^28 + O(3^30)
sage: x.artin_hasse_exp(10)
1 + 2*3^2 + 3^4 + 2*3^5 + 3^6 + 2*3^7 + 2*3^8 + 3^9 + O(3^10)
```

```
>>> from sage.all import *
>>> x = Zp(Integer(3),Integer(30))(Integer(45)/Integer(7))
>>> x.artin_hasse_exp()
1 + 2*3^2 + 3^4 + 2*3^5 + 3^6 + 2*3^7 + 2*3^8 + 3^9 + 2*3^10 + 3^11 +
3^13 + 2*3^15 + 2*3^16 + 2*3^17 + 3^19 + 3^20 + 2*3^21 + 3^23 + 3^24 +
3^26 + 3^27 + 2*3^28 + O(3^30)
>>> x.artin_hasse_exp(Integer(10))
1 + 2*3^2 + 3^4 + 2*3^5 + 3^6 + 2*3^7 + 2*3^8 + 3^9 + O(3^10)
```

For precision 1 the function just returns 1 since the exponential is always a 1-unit:

```
sage: x = Zp(3).random_element()
sage: while x.dist(0) >= 1:
....:     x = Zp(3).random_element()
sage: x.artin_hasse_exp(1)
1 + O(3)
```

```
>>> from sage.all import *
>>> x = Zp(Integer(3)).random_element()
>>> while x.dist(Integer(0)) >= Integer(1):
...     x = Zp(Integer(3)).random_element()
>>> x.artin_hasse_exp(Integer(1))
1 + O(3)
```

AUTHORS:

- Mitchell Owen, Sebastian Pancrantz (2012-02): initial version.
- Xavier Caruso (2018-08): extend to any p-adic rings and fields and implement several algorithms.

dwork_expansion(*bd=20*, *a=0*)

Return the value of a function defined by Dwork.

Used to compute the p-adic Gamma function, see *gamma()*.

INPUT:

- `bd` – integer (default: 20); precision bound
- `a` – integer (default: 0); offset parameter

OUTPUT: a p-adic integer

Note

This is based on GP code written by Fernando Rodriguez Villegas (http://www.ma.utexas.edu/cnt/cnt-frames.html). William Stein sped it up for GP (http://sage.math.washington.edu/home/wstein/www/home/wbhart/pari-2.4.2.alpha/src/basemath/trans2.c). The output is a p-adic integer from Dwork's expansion, used to compute the p-adic gamma function as in [RV2007] section 6.2. The coefficients of the expansion are now cached to speed up multiple evaluation, as in the trace formula for hypergeometric motives.

EXAMPLES:

```
sage: R = Zp(17)
sage: x = R(5+3*17+13*17^2+6*17^3+12*17^5+10*17^(14)+5*17^(17)+O(17^(19)))
sage: x.dwork_expansion(18)
16 + 7*17 + 11*17^2 + 4*17^3 + 8*17^4 + 10*17^5 + 11*17^6 + 6*17^7
+ 17^8 + 8*17^10 + 13*17^11 + 9*17^12 + 15*17^13  + 2*17^14 + 6*17^15
+ 7*17^16 + 6*17^17 + O(17^18)

sage: R = Zp(5)
sage: x = R(3*5^2+4*5^3+1*5^4+2*5^5+1*5^(10)+O(5^(20)))
sage: x.dwork_expansion()
4 + 4*5 + 4*5^2 + 4*5^3 + 2*5^4 + 4*5^5 + 5^7 + 3*5^9 + 4*5^10 + 3*5^11
+ 5^13 + 4*5^14 + 2*5^15 + 2*5^16 + 2*5^17 + 3*5^18 + O(5^20)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(17))
>>> x = R(Integer(5)+Integer(3)*Integer(17)+Integer(13)*Integer(17)**Integer(2
↪)+Integer(6)*Integer(17)**Integer(3)+Integer(12)*Integer(17)**Integer(5)+Int
↪eger(10)*Integer(17)**(Integer(14))+Integer(5)*Integer(17)**(Integer(17))+O(
↪Integer(17)**(Integer(19))))
>>> x.dwork_expansion(Integer(18))
16 + 7*17 + 11*17^2 + 4*17^3 + 8*17^4 + 10*17^5 + 11*17^6 + 6*17^7
+ 17^8 + 8*17^10 + 13*17^11 + 9*17^12 + 15*17^13  + 2*17^14 + 6*17^15
+ 7*17^16 + 6*17^17 + O(17^18)

>>> R = Zp(Integer(5))
>>> x = R(Integer(3)*Integer(5)**Integer(2)+Integer(4)*Integer(5)**Integer(3)+
↪Integer(1)*Integer(5)**Integer(4)+Integer(2)*Integer(5)**Integer(5)+Integer(
↪1)*Integer(5)**(Integer(10))+O(Integer(5)**(Integer(20))))
>>> x.dwork_expansion()
4 + 4*5 + 4*5^2 + 4*5^3 + 2*5^4 + 4*5^5 + 5^7 + 3*5^9 + 4*5^10 + 3*5^11
+ 5^13 + 4*5^14 + 2*5^15 + 2*5^16 + 2*5^17 + 3*5^18 + O(5^20)
```

exp(*aprec=None*, *algorithm=None*)

Compute the p-adic exponential of this element if the exponential series converges.

INPUT:

- `aprec` – integer or `None` (default: `None`); if specified, computes only up to the indicated precision
- `algorithm` – `'generic'`, `'binary_splitting'`, `'newton'` or `None` (default)

  The `'generic'` algorithm evaluates naively the series defining the exponential, namely

  $$\exp(x) = 1 + x + x^2/2 + x^3/6 + x^4/24 + \cdots .$$

  Its binary complexity is quadratic with respect to the precision.

  The `'binary_splitting'` algorithm is faster, it has a quasi-linear complexity.

  The `'newton'` algorithms solve the equation $\log(x) =$ `self` using a Newton scheme. It runs roughly as fast as the computation of the logarithm.

  By default, we use the `'binary_splitting'` if it is available. If it is not, we use the `'newton'` algorithm if a fast algorithm for computing the logarithm is available. Otherwise we switch to the `'generic'` algorithm.

EXAMPLES:

log() and *exp()* are inverse to each other:

```
sage: Z13 = Zp(13, 10)
sage: a = Z13(14); a
1 + 13 + O(13^10)
sage: a.log().exp()
1 + 13 + O(13^10)
```

```
>>> from sage.all import *
>>> Z13 = Zp(Integer(13), Integer(10))
>>> a = Z13(Integer(14)); a
1 + 13 + O(13^10)
```

(continues on next page)

(continued from previous page)

```
>>> a.log().exp()
1 + 13 + O(13^10)
```

An error occurs if this is called with an element for which the exponential series does not converge:

```
sage: Z13.one().exp()
Traceback (most recent call last):
...
ValueError: Exponential does not converge for that input.
```

```
>>> from sage.all import *
>>> Z13.one().exp()
Traceback (most recent call last):
...
ValueError: Exponential does not converge for that input.
```

The next few examples illustrate precision when computing p-adic exponentials:

```
sage: R = Zp(5,10)
sage: e = R(2*5 + 2*5**2 + 4*5**3 + 3*5**4
....:         + 5**5 + 3*5**7 + 2*5**8 + 4*5**9).add_bigoh(10); e
2*5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^5 + 3*5^7 + 2*5^8 + 4*5^9 + O(5^10)
sage: e.exp()*R.teichmuller(4)
4 + 2*5 + 3*5^3 + O(5^10)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(10))
>>> e = R(Integer(2)*Integer(5) + Integer(2)*Integer(5)**Integer(2) +␣
↪Integer(4)*Integer(5)**Integer(3) + Integer(3)*Integer(5)**Integer(4)
...         + Integer(5)**Integer(5) + Integer(3)*Integer(5)**Integer(7) +␣
↪Integer(2)*Integer(5)**Integer(8) + Integer(4)*Integer(5)**Integer(9)).add_
↪bigoh(Integer(10)); e
2*5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^5 + 3*5^7 + 2*5^8 + 4*5^9 + O(5^10)
>>> e.exp()*R.teichmuller(Integer(4))
4 + 2*5 + 3*5^3 + O(5^10)
```

```
sage: K = Qp(5,10)
sage: e = K(2*5 + 2*5**2 + 4*5**3 + 3*5**4
....:         + 5**5 + 3*5**7 + 2*5**8 + 4*5**9).add_bigoh(10); e
2*5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^5 + 3*5^7 + 2*5^8 + 4*5^9 + O(5^10)
sage: e.exp()*K.teichmuller(4)
4 + 2*5 + 3*5^3 + O(5^10)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(5),Integer(10))
>>> e = K(Integer(2)*Integer(5) + Integer(2)*Integer(5)**Integer(2) +␣
↪Integer(4)*Integer(5)**Integer(3) + Integer(3)*Integer(5)**Integer(4)
...         + Integer(5)**Integer(5) + Integer(3)*Integer(5)**Integer(7) +␣
↪Integer(2)*Integer(5)**Integer(8) + Integer(4)*Integer(5)**Integer(9)).add_
↪bigoh(Integer(10)); e
2*5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^5 + 3*5^7 + 2*5^8 + 4*5^9 + O(5^10)
```

(continues on next page)

(continued from previous page)

```
>>> e.exp()*K.teichmuller(Integer(4))
4 + 2*5 + 3*5^3 + O(5^10)
```

Logarithms and exponentials in extension fields. First, in an Eisenstein extension:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^4 + 15*x^2 + 625*x - 5
sage: W.<w> = R.ext(f)
sage: z = 1 + w^2 + 4*w^7; z
1 + w^2 + 4*w^7 + O(w^20)
sage: z.log().exp()
1 + w^2 + 4*w^7 + O(w^20)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(4) + Integer(15)*x**Integer(2) + Integer(625)*x -␣
↪Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = Integer(1) + w**Integer(2) + Integer(4)*w**Integer(7); z
1 + w^2 + 4*w^7 + O(w^20)
>>> z.log().exp()
1 + w^2 + 4*w^7 + O(w^20)
```

Now an unramified example:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: g = x^3 + 3*x + 3
sage: A.<a> = R.ext(g)
sage: b = 1 + 5*(1 + a^2) + 5^3*(3 + 2*a); b
1 + (a^2 + 1)*5 + (2*a + 3)*5^3 + O(5^5)
sage: b.log().exp()
1 + (a^2 + 1)*5 + (2*a + 3)*5^3 + O(5^5)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)
>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> b = Integer(1) + Integer(5)*(Integer(1) + a**Integer(2)) +␣
↪Integer(5)**Integer(3)*(Integer(3) + Integer(2)*a); b
1 + (a^2 + 1)*5 + (2*a + 3)*5^3 + O(5^5)
>>> b.log().exp()
1 + (a^2 + 1)*5 + (2*a + 3)*5^3 + O(5^5)
```

AUTHORS:

- Genya Zaytman (2007-02-15)
- Amnon Besser, Marc Masdeu (2012-02-23): Complete rewrite
- Julian Rueth (2013-02-14): Added doctests, fixed some corner cases

- Xavier Caruso (2017-06): Added binary splitting and Newton algorithms

gamma(*algorithm='pari'*)

Return the value of the p-adic Gamma function.

INPUT:

- `algorithm` – string. Can be set to `'pari'` to call the PARI function, or `'sage'` to call the function implemented in Sage. The default is `'pari'` since PARI is about 10 times faster than Sage.

OUTPUT: a p-adic integer

> **Note**
>
> This is based on GP code written by Fernando Rodriguez Villegas (http://www.ma.utexas.edu/cnt/cnt-frames.html). William Stein sped it up for GP (http://sage.math.washington.edu/home/wstein/www/home/wbhart/pari-2.4.2.alpha/src/basemath/trans2.c). The `'sage'` version uses dwork_expansion() to compute the p-adic gamma function of `self` as in [RV2007] section 6.2.

EXAMPLES:

This example illustrates `x.gamma()` for x a p-adic unit:

```
sage: R = Zp(7)
sage: x = R(2+3*7^2+4*7^3+O(7^20))
sage: x.gamma('pari')
1 + 2*7^2 + 4*7^3 + 5*7^4 + 3*7^5 + 7^8 + 7^9 + 4*7^10 + 3*7^12
+ 7^13 + 5*7^14 + 3*7^15 + 2*7^16 + 2*7^17 + 5*7^18 + 4*7^19 + O(7^20)
sage: x.gamma('sage')
1 + 2*7^2 + 4*7^3 + 5*7^4 + 3*7^5 + 7^8 + 7^9 + 4*7^10 + 3*7^12
+ 7^13 + 5*7^14 + 3*7^15 + 2*7^16 + 2*7^17 + 5*7^18 + 4*7^19 + O(7^20)
sage: x.gamma('pari') == x.gamma('sage')
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7))
>>> x = R(Integer(2)+Integer(3)*Integer(7)**Integer(2)+Integer(4)*Integer(7)**
↪Integer(3)+O(Integer(7)**Integer(20)))
>>> x.gamma('pari')
1 + 2*7^2 + 4*7^3 + 5*7^4 + 3*7^5 + 7^8 + 7^9 + 4*7^10 + 3*7^12
+ 7^13 + 5*7^14 + 3*7^15 + 2*7^16 + 2*7^17 + 5*7^18 + 4*7^19 + O(7^20)
>>> x.gamma('sage')
1 + 2*7^2 + 4*7^3 + 5*7^4 + 3*7^5 + 7^8 + 7^9 + 4*7^10 + 3*7^12
+ 7^13 + 5*7^14 + 3*7^15 + 2*7^16 + 2*7^17 + 5*7^18 + 4*7^19 + O(7^20)
>>> x.gamma('pari') == x.gamma('sage')
True
```

Now `x.gamma()` for x a p-adic integer but not a unit:

```
sage: R = Zp(17)
sage: x = R(17+17^2+3*17^3+12*17^8+O(17^13))
sage: x.gamma('pari')
1 + 12*17 + 13*17^2 + 13*17^3 + 10*17^4 + 7*17^5 + 16*17^7
+ 13*17^9 + 4*17^10 + 9*17^11 + 17^12 + O(17^13)
sage: x.gamma('sage')
```

(continues on next page)

(continued from previous page)

```
1 + 12*17 + 13*17^2 + 13*17^3 + 10*17^4 + 7*17^5 + 16*17^7
+ 13*17^9 + 4*17^10 + 9*17^11 + 17^12 + O(17^13)
sage: x.gamma('pari') == x.gamma('sage')
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(17))
>>> x = R(Integer(17)+Integer(17)**Integer(2)+Integer(3)*Integer(17)**Integer(
↪3)+Integer(12)*Integer(17)**Integer(8)+O(Integer(17)**Integer(13)))
>>> x.gamma('pari')
1 + 12*17 + 13*17^2 + 13*17^3 + 10*17^4 + 7*17^5 + 16*17^7
+ 13*17^9 + 4*17^10 + 9*17^11 + 17^12 + O(17^13)
>>> x.gamma('sage')
1 + 12*17 + 13*17^2 + 13*17^3 + 10*17^4 + 7*17^5 + 16*17^7
+ 13*17^9 + 4*17^10 + 9*17^11 + 17^12 + O(17^13)
>>> x.gamma('pari') == x.gamma('sage')
True
```

Finally, this function is not defined if x is not a p-adic integer:

```
sage: K = Qp(7)
sage: x = K(7^-5 + 2*7^-4 + 5*7^-3 + 2*7^-2 + 3*7^-1 + 3 + 3*7
....:       + 7^3 + 4*7^4 + 5*7^5 + 6*7^8 + 3*7^9 + 6*7^10 + 5*7^11 + 6*7^12
....:       + 3*7^13 + 5*7^14 + O(7^15))
sage: x.gamma()
Traceback (most recent call last):
...
ValueError: The p-adic gamma function only works on elements of Zp
```

```
>>> from sage.all import *
>>> K = Qp(Integer(7))
>>> x = K(Integer(7)**-Integer(5) + Integer(2)*Integer(7)**-Integer(4) +␣
↪Integer(5)*Integer(7)**-Integer(3) + Integer(2)*Integer(7)**-Integer(2) +␣
↪Integer(3)*Integer(7)**-Integer(1) + Integer(3) + Integer(3)*Integer(7)
...       + Integer(7)**Integer(3) + Integer(4)*Integer(7)**Integer(4) +␣
↪Integer(5)*Integer(7)**Integer(5) + Integer(6)*Integer(7)**Integer(8) +␣
↪Integer(3)*Integer(7)**Integer(9) + Integer(6)*Integer(7)**Integer(10) +␣
↪Integer(5)*Integer(7)**Integer(11) + Integer(6)*Integer(7)**Integer(12)
...       + Integer(3)*Integer(7)**Integer(13) +␣
↪Integer(5)*Integer(7)**Integer(14) + O(Integer(7)**Integer(15)))
>>> x.gamma()
Traceback (most recent call last):
...
ValueError: The p-adic gamma function only works on elements of Zp
```

gcd(*other*)

Return a greatest common divisor of `self` and `other`.

INPUT:

- `other` – an element in the same ring as `self`

AUTHORS:

- Julian Rueth (2012-10-19): initial version

Note

Since the elements are only given with finite precision, their greatest common divisor is in general not unique (not even up to units). For example $O(3)$ is a representative for the elements 0 and 3 in the 3-adic ring $\mathbf{Z}_3$. The greatest common divisor of $O(3)$ and $O(3)$ could be (among others) 3 or 0 which have different valuation. The algorithm implemented here, will return an element of minimal valuation among the possible greatest common divisors.

EXAMPLES:

The greatest common divisor is either zero or a power of the uniformizing parameter:

```
sage: R = Zp(3)
sage: R.zero().gcd(R.zero())
0
sage: R(3).gcd(9)
3 + O(3^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3))
>>> R.zero().gcd(R.zero())
0
>>> R(Integer(3)).gcd(Integer(9))
3 + O(3^21)
```

A nonzero result is always lifted to the maximal precision possible in the ring:

```
sage: a = R(3,2); a
3 + O(3^2)
sage: b = R(9,3); b
3^2 + O(3^3)
sage: a.gcd(b)
3 + O(3^21)
sage: a.gcd(0)
3 + O(3^21)
```

```
>>> from sage.all import *
>>> a = R(Integer(3),Integer(2)); a
3 + O(3^2)
>>> b = R(Integer(9),Integer(3)); b
3^2 + O(3^3)
>>> a.gcd(b)
3 + O(3^21)
>>> a.gcd(Integer(0))
3 + O(3^21)
```

If both elements are zero, then the result is zero with the precision set to the smallest of their precisions:

```
sage: a = R.zero(); a
0
sage: b = R(0,2); b
```

(continues on next page)

(continued from previous page)

```
O(3^2)
sage: a.gcd(b)
O(3^2)
```

```
>>> from sage.all import *
>>> a = R.zero(); a
0
>>> b = R(Integer(0),Integer(2)); b
O(3^2)
>>> a.gcd(b)
O(3^2)
```

One could argue that it is mathematically correct to return $9 + O(3^{22})$ instead. However, this would lead to some confusing behaviour:

```
sage: alternative_gcd = R(9,22); alternative_gcd
3^2 + O(3^22)
sage: a.is_zero()
True
sage: b.is_zero()
True
sage: alternative_gcd.is_zero()
False
```

```
>>> from sage.all import *
>>> alternative_gcd = R(Integer(9),Integer(22)); alternative_gcd
3^2 + O(3^22)
>>> a.is_zero()
True
>>> b.is_zero()
True
>>> alternative_gcd.is_zero()
False
```

If exactly one element is zero, then the result depends on the valuation of the other element:

```
sage: R(0,3).gcd(3^4)
O(3^3)
sage: R(0,4).gcd(3^4)
O(3^4)
sage: R(0,5).gcd(3^4)
3^4 + O(3^24)
```

```
>>> from sage.all import *
>>> R(Integer(0),Integer(3)).gcd(Integer(3)**Integer(4))
O(3^3)
>>> R(Integer(0),Integer(4)).gcd(Integer(3)**Integer(4))
O(3^4)
>>> R(Integer(0),Integer(5)).gcd(Integer(3)**Integer(4))
3^4 + O(3^24)
```

Over a field, the greatest common divisor is either zero (possibly with finite precision) or one:

```
sage: K = Qp(3)
sage: K(3).gcd(0)
1 + O(3^20)
sage: K.zero().gcd(0)
0
sage: K.zero().gcd(K(0,2))
O(3^2)
sage: K(3).gcd(4)
1 + O(3^20)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(3))
>>> K(Integer(3)).gcd(Integer(0))
1 + O(3^20)
>>> K.zero().gcd(Integer(0))
0
>>> K.zero().gcd(K(Integer(0),Integer(2)))
O(3^2)
>>> K(Integer(3)).gcd(Integer(4))
1 + O(3^20)
```

is_prime()

Return whether this element is prime in its parent.

EXAMPLES:

```
sage: A = Zp(2)
sage: A(1).is_prime()
False
sage: A(2).is_prime()
True

sage: K = A.fraction_field()
sage: K(2).is_prime()
False
```

```
>>> from sage.all import *
>>> A = Zp(Integer(2))
>>> A(Integer(1)).is_prime()
False
>>> A(Integer(2)).is_prime()
True

>>> K = A.fraction_field()
>>> K(Integer(2)).is_prime()
False
```

```
sage: x = polygen(ZZ, 'x')
sage: B.<pi> = A.extension(x^5 - 2)
sage: pi.is_prime()
True
sage: B(2).is_prime()
False
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> B = A.extension(x**Integer(5) - Integer(2), names=('pi',)); (pi,) = B._
↪first_ngens(1)
>>> pi.is_prime()
True
>>> B(Integer(2)).is_prime()
False
```

is_square()

Return whether this element is a square.

INPUT:

- `self` – a p-adic element

EXAMPLES:

```
sage: R = Zp(3,20,'capped-rel')
sage: R(0).is_square()
True
sage: R(1).is_square()
True
sage: R(2).is_square()
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3),Integer(20),'capped-rel')
>>> R(Integer(0)).is_square()
True
>>> R(Integer(1)).is_square()
True
>>> R(Integer(2)).is_square()
False
```

is_squarefree()

Return whether this element is squarefree, i.e., whether there exists no non-unit g such that g^2 divides this element.

EXAMPLES:

The zero element is never squarefree:

```
sage: K = Qp(2)
sage: K.zero().is_squarefree()
False
```

```
>>> from sage.all import *
>>> K = Qp(Integer(2))
>>> K.zero().is_squarefree()
False
```

In p-adic rings, only elements of valuation at most 1 are squarefree:

```
sage: R = Zp(2)
sage: R(1).is_squarefree()
True
sage: R(2).is_squarefree()
True
sage: R(4).is_squarefree()
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(2))
>>> R(Integer(1)).is_squarefree()
True
>>> R(Integer(2)).is_squarefree()
True
>>> R(Integer(4)).is_squarefree()
False
```

This works only if the precision is known sufficiently well:

```
sage: R(0,1).is_squarefree()
Traceback (most recent call last):
...
PrecisionError: element not known to sufficient precision to decide␣
↪squarefreeness
sage: R(0,2).is_squarefree()
False
sage: R(1,1).is_squarefree()
True
```

```
>>> from sage.all import *
>>> R(Integer(0),Integer(1)).is_squarefree()
Traceback (most recent call last):
...
PrecisionError: element not known to sufficient precision to decide␣
↪squarefreeness
>>> R(Integer(0),Integer(2)).is_squarefree()
False
>>> R(Integer(1),Integer(1)).is_squarefree()
True
```

For fields we are not so strict about the precision and treat inexact zeros as the zero element:

```
K(0,0).is_squarefree()
False
```

log(*p_branch=None*, *pi_branch=None*, *aprec=None*, *change_frac=False*, *algorithm=None*)

Compute the p-adic logarithm of this element.

The usual power series for the logarithm with values in the additive group of a p-adic ring only converges for 1-units (units congruent to 1 modulo p). However, there is a unique extension of the logarithm to a homomorphism defined on all the units: If $u = a \cdot v$ is a unit with $v \equiv 1 \pmod{p}$ and a a Teichmuller representative, then we define $log(u) = log(v)$. This is the correct extension because the units U split as a product $U = V \times \langle w \rangle$, where V is the subgroup of 1-units and w is a fundamental root of unity. The $\langle w \rangle$ factor is torsion, so must go to 0 under any homomorphism to the fraction field, which is a torsion free group.

INPUT:

- `p_branch` – an element in the base ring or its fraction field; the implementation will choose the branch of the logarithm which sends p to `branch`
- `pi_branch` – an element in the base ring or its fraction field; the implementation will choose the branch of the logarithm which sends the uniformizer to `branch`; you may specify at most one of `p_branch` and `pi_branch`, and must specify one of them if this element is not a unit
- `aprec` – integer or `None` (default: `None`); if not `None`, then the result will only be correct to precision `aprec`
- `change_frac` – in general the codomain of the logarithm should be in the p-adic field, however, for most neighborhoods of 1, it lies in the ring of integers. This flag decides if the codomain should be the same as the input (default) or if it should change to the fraction field of the input.
- `algorithm` – `'generic'`, `'binary_splitting'` or `None` (default) The generic algorithm evaluates naively the series defining the log, namely

  $$\log(1-x) = -x - 1/2x^2 - 1/3x^3 - 1/4x^4 - 1/5x^5 - \cdots .$$

  Its binary complexity is quadratic with respect to the precision.

  The `'binary_splitting'` algorithm is faster, it has a quasi-linear complexity. By default, we use `'binary_splitting'` if it is available. Otherwise we switch to the `'generic'` algorithm.

Note

What some other systems do:

- PARI: Seems to define the logarithm for units not congruent to 1 as we do.
- MAGMA: Only implements logarithm for 1-units (version 2.19-2)

Todo

There is a soft-linear time algorithm for logarithm described by Dan Bernstein at http://cr.yp.to/lineartime/multapps-20041007.pdf

EXAMPLES:

```
sage: Z13 = Zp(13, 10)
sage: a = Z13(14); a
1 + 13 + O(13^10)
sage: a.log()
13 + 6*13^2 + 2*13^3 + 5*13^4 + 10*13^6 + 13^7 + 11*13^8 + 8*13^9 + O(13^10)

sage: Q13 = Qp(13, 10)
sage: a = Q13(14); a
1 + 13 + O(13^10)
sage: a.log()
13 + 6*13^2 + 2*13^3 + 5*13^4 + 10*13^6 + 13^7 + 11*13^8 + 8*13^9 + O(13^10)
```

```
>>> from sage.all import *
>>> Z13 = Zp(Integer(13), Integer(10))
```

(continues on next page)

(continued from previous page)

```
>>> a = Z13(Integer(14)); a
1 + 13 + O(13^10)
>>> a.log()
13 + 6*13^2 + 2*13^3 + 5*13^4 + 10*13^6 + 13^7 + 11*13^8 + 8*13^9 + O(13^10)

>>> Q13 = Qp(Integer(13), Integer(10))
>>> a = Q13(Integer(14)); a
1 + 13 + O(13^10)
>>> a.log()
13 + 6*13^2 + 2*13^3 + 5*13^4 + 10*13^6 + 13^7 + 11*13^8 + 8*13^9 + O(13^10)
```

Note that the relative precision decreases when we take log. Precisely the absolute precision on `log(a)` agrees with the relative precision on `a` thanks to the relation $d\log(a) = da/a$.

The call `log(a)` works as well:

```
sage: log(a)
13 + 6*13^2 + 2*13^3 + 5*13^4 + 10*13^6 + 13^7 + 11*13^8 + 8*13^9 + O(13^10)
sage: log(a) == a.log()
True
```

```
>>> from sage.all import *
>>> log(a)
13 + 6*13^2 + 2*13^3 + 5*13^4 + 10*13^6 + 13^7 + 11*13^8 + 8*13^9 + O(13^10)
>>> log(a) == a.log()
True
```

The logarithm is not only defined for 1-units:

```
sage: R = Zp(5,10)
sage: a = R(2)
sage: a.log()
2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(10))
>>> a = R(Integer(2))
>>> a.log()
2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
```

If you want to take the logarithm of a non-unit you must specify either `p_branch` or `pi_branch`:

```
sage: b = R(5)
sage: b.log()
Traceback (most recent call last):
...
ValueError: you must specify a branch of the logarithm for non-units
sage: b.log(p_branch=4)
4 + O(5^10)
sage: c = R(10)
sage: c.log(p_branch=4)
4 + 2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
```

```
>>> from sage.all import *
>>> b = R(Integer(5))
>>> b.log()
Traceback (most recent call last):
...
ValueError: you must specify a branch of the logarithm for non-units
>>> b.log(p_branch=Integer(4))
4 + O(5^10)
>>> c = R(Integer(10))
>>> c.log(p_branch=Integer(4))
4 + 2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
```

The branch parameters are only relevant for elements of nonzero valuation:

```
sage: a.log(p_branch=0)
2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
sage: a.log(p_branch=1)
2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
```

```
>>> from sage.all import *
>>> a.log(p_branch=Integer(0))
2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
>>> a.log(p_branch=Integer(1))
2*5 + 3*5^2 + 2*5^3 + 4*5^4 + 2*5^6 + 2*5^7 + 4*5^8 + 2*5^9 + O(5^10)
```

Logarithms can also be computed in extension fields. First, in an Eisenstein extension:

```
sage: R = Zp(5,5)
sage: S.<x> = ZZ[]
sage: f = x^4 + 15*x^2 + 625*x - 5
sage: W.<w> = R.ext(f)
sage: z = 1 + w^2 + 4*w^7; z
1 + w^2 + 4*w^7 + O(w^20)
sage: z.log()
w^2 + 2*w^4 + 3*w^6 + 4*w^7 + w^9 + 4*w^10 + 4*w^11 + 4*w^12
 + 3*w^14 + w^15 + w^17 + 3*w^18 + 3*w^19 + O(w^20)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(4) + Integer(15)*x**Integer(2) + Integer(625)*x -␣
↪Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = Integer(1) + w**Integer(2) + Integer(4)*w**Integer(7); z
1 + w^2 + 4*w^7 + O(w^20)
>>> z.log()
w^2 + 2*w^4 + 3*w^6 + 4*w^7 + w^9 + 4*w^10 + 4*w^11 + 4*w^12
 + 3*w^14 + w^15 + w^17 + 3*w^18 + 3*w^19 + O(w^20)
```

In an extension, there will usually be a difference between specifying `p_branch` and `pi_branch`:

```
sage: b = W(5)
sage: b.log()
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: you must specify a branch of the logarithm for non-units
sage: b.log(p_branch=0)
O(w^20)
sage: b.log(p_branch=w)
w + O(w^20)
sage: b.log(pi_branch=0)
3*w^2 + 2*w^4 + 2*w^6 + 3*w^8 + 4*w^10 + w^13 + w^14 + 2*w^15
 + 2*w^16 + w^18 + 4*w^19 + O(w^20)
sage: b.unit_part().log()
3*w^2 + 2*w^4 + 2*w^6 + 3*w^8 + 4*w^10 + w^13 + w^14 + 2*w^15
 + 2*w^16 + w^18 + 4*w^19 + O(w^20)
sage: y = w^2 * 4*w^7; y
4*w^9 + O(w^29)
sage: y.log(p_branch=0)
2*w^2 + 2*w^4 + 2*w^6 + 2*w^8 + w^10 + w^12 + 4*w^13 + 4*w^14 + 3*w^15
 + 4*w^16 + 4*w^17 + w^18 + 4*w^19 + O(w^20)
sage: y.log(p_branch=w)
w + 2*w^2 + 2*w^4 + 4*w^5 + 2*w^6 + 2*w^7 + 2*w^8 + 4*w^9 + w^10
 + 3*w^11 + w^12 + 4*w^14 + 4*w^16 + 2*w^17 + w^19 + O(w^20)
```

```
>>> from sage.all import *
>>> b = W(Integer(5))
>>> b.log()
Traceback (most recent call last):
...
ValueError: you must specify a branch of the logarithm for non-units
>>> b.log(p_branch=Integer(0))
O(w^20)
>>> b.log(p_branch=w)
w + O(w^20)
>>> b.log(pi_branch=Integer(0))
3*w^2 + 2*w^4 + 2*w^6 + 3*w^8 + 4*w^10 + w^13 + w^14 + 2*w^15
 + 2*w^16 + w^18 + 4*w^19 + O(w^20)
>>> b.unit_part().log()
3*w^2 + 2*w^4 + 2*w^6 + 3*w^8 + 4*w^10 + w^13 + w^14 + 2*w^15
 + 2*w^16 + w^18 + 4*w^19 + O(w^20)
>>> y = w**Integer(2) * Integer(4)*w**Integer(7); y
4*w^9 + O(w^29)
>>> y.log(p_branch=Integer(0))
2*w^2 + 2*w^4 + 2*w^6 + 2*w^8 + w^10 + w^12 + 4*w^13 + 4*w^14 + 3*w^15
 + 4*w^16 + 4*w^17 + w^18 + 4*w^19 + O(w^20)
>>> y.log(p_branch=w)
w + 2*w^2 + 2*w^4 + 4*w^5 + 2*w^6 + 2*w^7 + 2*w^8 + 4*w^9 + w^10
 + 3*w^11 + w^12 + 4*w^14 + 4*w^16 + 2*w^17 + w^19 + O(w^20)
```

Check that log is multiplicative:

```
sage: y.log(p_branch=0) + z.log() - (y*z).log(p_branch=0)
O(w^20)
```

```
>>> from sage.all import *
>>> y.log(p_branch=Integer(0)) + z.log() - (y*z).log(p_branch=Integer(0))
O(w^20)
```

Now an unramified example:

```
sage: g = x^3 + 3*x + 3
sage: A.<a> = R.ext(g)
sage: b = 1 + 5*(1 + a^2) + 5^3*(3 + 2*a)
sage: b.log()
(a^2 + 1)*5 + (3*a^2 + 4*a + 2)*5^2 + (3*a^2 + 2*a)*5^3
 + (3*a^2 + 2*a + 2)*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)
>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> b = Integer(1) + Integer(5)*(Integer(1) + a**Integer(2)) +␣
↪Integer(5)**Integer(3)*(Integer(3) + Integer(2)*a)
>>> b.log()
(a^2 + 1)*5 + (3*a^2 + 4*a + 2)*5^2 + (3*a^2 + 2*a)*5^3
 + (3*a^2 + 2*a + 2)*5^4 + O(5^5)
```

Check that log is multiplicative:

```
sage: c = 3 + 5^2*(2 + 4*a)
sage: b.log() + c.log() - (b*c).log()
O(5^5)
```

```
>>> from sage.all import *
>>> c = Integer(3) + Integer(5)**Integer(2)*(Integer(2) + Integer(4)*a)
>>> b.log() + c.log() - (b*c).log()
O(5^5)
```

We illustrate the effect of the `precision` argument:

```
sage: R = ZpCA(7,10)
sage: x = R(41152263); x
5 + 3*7^2 + 4*7^3 + 3*7^4 + 5*7^5 + 6*7^6 + 7^9 + O(7^10)
sage: x.log(aprec = 5)
7 + 3*7^2 + 4*7^3 + 3*7^4 + O(7^5)
sage: x.log(aprec = 7)
7 + 3*7^2 + 4*7^3 + 3*7^4 + 7^5 + 3*7^6 + O(7^7)
sage: x.log()
7 + 3*7^2 + 4*7^3 + 3*7^4 + 7^5 + 3*7^6 + 7^7 + 3*7^8 + 4*7^9 + O(7^10)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(7),Integer(10))
>>> x = R(Integer(41152263)); x
5 + 3*7^2 + 4*7^3 + 3*7^4 + 5*7^5 + 6*7^6 + 7^9 + O(7^10)
>>> x.log(aprec = Integer(5))
7 + 3*7^2 + 4*7^3 + 3*7^4 + O(7^5)
>>> x.log(aprec = Integer(7))
```

(continues on next page)

(continued from previous page)

```
7 + 3*7^2 + 4*7^3 + 3*7^4 + 7^5 + 3*7^6 + O(7^7)
>>> x.log()
7 + 3*7^2 + 4*7^3 + 3*7^4 + 7^5 + 3*7^6 + 7^7 + 3*7^8 + 4*7^9 + O(7^10)
```

The logarithm is not defined for zero:

```
sage: R.zero().log()
Traceback (most recent call last):
...
ValueError: logarithm is not defined at zero
```

```
>>> from sage.all import *
>>> R.zero().log()
Traceback (most recent call last):
...
ValueError: logarithm is not defined at zero
```

For elements in a p-adic ring, the logarithm will be returned in the same ring:

```
sage: x = R(2)
sage: x.log().parent()
7-adic Ring with capped absolute precision 10
sage: x = R(14)
sage: x.log(p_branch=0).parent()
7-adic Ring with capped absolute precision 10
```

```
>>> from sage.all import *
>>> x = R(Integer(2))
>>> x.log().parent()
7-adic Ring with capped absolute precision 10
>>> x = R(Integer(14))
>>> x.log(p_branch=Integer(0)).parent()
7-adic Ring with capped absolute precision 10
```

This is not possible if the logarithm has negative valuation:

```
sage: R = ZpCA(3,10)
sage: S.<x> = R[]
sage: f = x^3 - 3
sage: W.<w> = R.ext(f)
sage: w.log(p_branch=2)
Traceback (most recent call last):
...
ValueError: logarithm is not integral, use change_frac=True
to obtain a result in the fraction field
sage: w.log(p_branch=2, change_frac=True)
2*w^-3 + O(w^24)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(3),Integer(10))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(3) - Integer(3)
```

(continues on next page)

(continued from previous page)

```
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> w.log(p_branch=Integer(2))
Traceback (most recent call last):
...
ValueError: logarithm is not integral, use change_frac=True
to obtain a result in the fraction field
>>> w.log(p_branch=Integer(2), change_frac=True)
2*w^-3 + O(w^24)
```

AUTHORS:

- William Stein: initial version
- David Harvey (2006-09-13): corrected subtle precision bug (need to take denominators into account! – see Issue #53)
- Genya Zaytman (2007-02-14): adapted to new p-adic class
- Amnon Besser, Marc Masdeu (2012-02-21): complete rewrite, valid for generic p-adic rings.
- Soroosh Yazdani (2013-02-1): Fixed a precision issue in `_log_generic`. This should really fix the issue with divisions.
- Julian Rueth (2013-02-14): Added doctests, some changes for capped-absolute implementations.
- Xavier Caruso (2017-06): Added binary splitting type algorithms over Qp

minimal_polynomial(*name='x'*, *base=None*)

Return the minimal polynomial of this element over `base`.

INPUT:

- `name` – string (default: `'x'`); the name of the variable
- `base` – a ring (default: the base ring of the parent); the base ring over which the minimal polynomial is computed

EXAMPLES:

```
sage: Zp(5,5)(1/3).minimal_polynomial('x')
(1 + O(5^5))*x + 3 + 5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)

sage: Zp(5,5)(1/3).minimal_polynomial('foo')
(1 + O(5^5))*foo + 3 + 5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> Zp(Integer(5),Integer(5))(Integer(1)/Integer(3)).minimal_polynomial('x')
(1 + O(5^5))*x + 3 + 5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)

>>> Zp(Integer(5),Integer(5))(Integer(1)/Integer(3)).minimal_polynomial('foo')
(1 + O(5^5))*foo + 3 + 5 + 3*5^2 + 5^3 + 3*5^4 + O(5^5)
```

```
sage: K.<a> = QqCR(2^3,5)
sage: S.<x> = K[]
sage: L.<pi> = K.extension(x^4 - 2*a)
sage: pi.minimal_polynomial()
(1 + O(2^5))*x^4 + a*2 + a*2^2 + a*2^3 + a*2^4 + a*2^5 + O(2^6)
```

(continues on next page)

(continued from previous page)

```
sage: (pi^2).minimal_polynomial()
(1 + O(2^5))*x^2 + a*2 + a*2^2 + a*2^3 + a*2^4 + a*2^5 + O(2^6)
sage: (1/pi).minimal_polynomial()
(1 + O(2^5))*x^4 + (a^2 + 1)*2^-1 + O(2^4)
sage: elt = L.random_element()
sage: P = elt.minimal_polynomial()  # not tested, known bug (see␣
↪:issue:`32111`)
sage: P(elt) == 0  # not tested
True
```

```
>>> from sage.all import *
>>> K = QqCR(Integer(2)**Integer(3),Integer(5), names=('a',)); (a,) = K._
↪first_ngens(1)
>>> S = K['x']; (x,) = S._first_ngens(1)
>>> L = K.extension(x**Integer(4) - Integer(2)*a, names=('pi',)); (pi,) = L._
↪first_ngens(1)
>>> pi.minimal_polynomial()
(1 + O(2^5))*x^4 + a*2 + a*2^2 + a*2^3 + a*2^4 + a*2^5 + O(2^6)
>>> (pi**Integer(2)).minimal_polynomial()
(1 + O(2^5))*x^2 + a*2 + a*2^2 + a*2^3 + a*2^4 + a*2^5 + O(2^6)
>>> (Integer(1)/pi).minimal_polynomial()
(1 + O(2^5))*x^4 + (a^2 + 1)*2^-1 + O(2^4)
>>> elt = L.random_element()
>>> P = elt.minimal_polynomial()  # not tested, known bug (see :issue:`32111`)
>>> P(elt) == Integer(0)  # not tested
True
```

multiplicative_order(*prec=None*)

Return the multiplicative order of `self`, where `self` is considered to be one if it is one modulo p^{prec}.

INPUT:

- `self` – a p-adic element
- `prec` – integer

OUTPUT: integer; the multiplicative order of `self`

EXAMPLES:

```
sage: K = Qp(5,20,'capped-rel')
sage: K(-1).multiplicative_order(20)
2
sage: K(1).multiplicative_order(20)
1
sage: K(2).multiplicative_order(20)
+Infinity
sage: K(5).multiplicative_order(20)
+Infinity
sage: K(1/5).multiplicative_order(20)
+Infinity
sage: K.zeta().multiplicative_order(20)
4
```

```
>>> from sage.all import *
>>> K = Qp(Integer(5),Integer(20),'capped-rel')
>>> K(-Integer(1)).multiplicative_order(Integer(20))
2
>>> K(Integer(1)).multiplicative_order(Integer(20))
1
>>> K(Integer(2)).multiplicative_order(Integer(20))
+Infinity
>>> K(Integer(5)).multiplicative_order(Integer(20))
+Infinity
>>> K(Integer(1)/Integer(5)).multiplicative_order(Integer(20))
+Infinity
>>> K.zeta().multiplicative_order(Integer(20))
4
```

Over unramified extensions:

```
sage: L1.<a> = Qq(5^3)
sage: c = L1.teichmuller(a)
sage: c.multiplicative_order()
124
sage: c^124
1 + O(5^20)
```

```
>>> from sage.all import *
>>> L1 = Qq(Integer(5)**Integer(3), names=('a',)); (a,) = L1._first_ngens(1)
>>> c = L1.teichmuller(a)
>>> c.multiplicative_order()
124
>>> c**Integer(124)
1 + O(5^20)
```

Over totally ramified extensions:

```
sage: x = polygen(ZZ, 'x')
sage: L2.<pi> = Qp(5).extension(x^4 + 5*x^3 + 10*x^2 + 10*x + 5)
sage: u = 1 + pi
sage: u.multiplicative_order()
5
sage: v = L2.teichmuller(2)
sage: v.multiplicative_order()
4
sage: (u*v).multiplicative_order()
20
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> L2 = Qp(Integer(5)).extension(x**Integer(4) + Integer(5)*x**Integer(3) +␣
↪Integer(10)*x**Integer(2) + Integer(10)*x + Integer(5), names=('pi',)); (pi,
↪) = L2._first_ngens(1)
>>> u = Integer(1) + pi
>>> u.multiplicative_order()
5
```

(continues on next page)

(continued from previous page)

```
>>> v = L2.teichmuller(Integer(2))
>>> v.multiplicative_order()
4
>>> (u*v).multiplicative_order()
20
```

norm(*base=None*)

Return the norm of this p-adic element over `base`.

Warning

This is not the p-adic absolute value. This is a field theoretic norm down to a base ring. If you want the p-adic absolute value, use the method *abs()* instead.

INPUT:

- `base` – a subring of the parent (default: base ring)

OUTPUT: the norm of this p-adic element over the given base

EXAMPLES:

```
sage: Zp(5)(5).norm()
5 + O(5^21)
```

```
>>> from sage.all import *
>>> Zp(Integer(5))(Integer(5)).norm()
5 + O(5^21)
```

```
sage: K.<a> = QqCR(2^3,5)
sage: S.<x> = K[]
sage: L.<pi> = K.extension(x^4 - 2*a)
sage: pi.norm()  # norm over K
a*2 + a*2^2 + a*2^3 + a*2^4 + a*2^5 + O(2^6)
sage: (pi^2).norm()
a^2*2^2 + O(2^7)
sage: pi.norm()^2
a^2*2^2 + O(2^7)
```

```
>>> from sage.all import *
>>> K = QqCR(Integer(2)**Integer(3),Integer(5), names=('a',)); (a,) = K._
↪first_ngens(1)
>>> S = K['x']; (x,) = S._first_ngens(1)
>>> L = K.extension(x**Integer(4) - Integer(2)*a, names=('pi',)); (pi,) = L._
↪first_ngens(1)
>>> pi.norm()  # norm over K
a*2 + a*2^2 + a*2^3 + a*2^4 + a*2^5 + O(2^6)
>>> (pi**Integer(2)).norm()
a^2*2^2 + O(2^7)
>>> pi.norm()**Integer(2)
a^2*2^2 + O(2^7)
```

nth_root(*n*, *all=False*)

Return the n-th root of this element.

INPUT:

- `n` – integer
- `all` – boolean (default: `False`); if `True`, return all n-th roots of this element, instead of just one

EXAMPLES:

```
sage: A = Zp(5,10)
sage: x = A(61376); x
1 + 5^3 + 3*5^4 + 4*5^5 + 3*5^6 + O(5^10)
sage: y = x.nth_root(4); y
2 + 5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^6 + O(5^10)
sage: y^4 == x
True

sage: x.nth_root(4, all=True)
[2 + 5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^6 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^4 + 3*5^5 + 5^6 + 3*5^7 + 5^8 + 5^9 + O(5^10),
 3 + 3*5 + 2*5^2 + 5^4 + 4*5^5 + 3*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + O(5^10),
 1 + 4*5^3 + 5^5 + 3*5^6 + 5^7 + 3*5^8 + 3*5^9 + O(5^10)]
```

```
>>> from sage.all import *
>>> A = Zp(Integer(5),Integer(10))
>>> x = A(Integer(61376)); x
1 + 5^3 + 3*5^4 + 4*5^5 + 3*5^6 + O(5^10)
>>> y = x.nth_root(Integer(4)); y
2 + 5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^6 + O(5^10)
>>> y**Integer(4) == x
True

>>> x.nth_root(Integer(4), all=True)
[2 + 5 + 2*5^2 + 4*5^3 + 3*5^4 + 5^6 + O(5^10),
 4 + 4*5 + 4*5^2 + 4*5^4 + 3*5^5 + 5^6 + 3*5^7 + 5^8 + 5^9 + O(5^10),
 3 + 3*5 + 2*5^2 + 5^4 + 4*5^5 + 3*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + O(5^10),
 1 + 4*5^3 + 5^5 + 3*5^6 + 5^7 + 3*5^8 + 3*5^9 + O(5^10)]
```

When n is divisible by the underlying prime p, we are losing precision (which is consistent with the fact that raising to the p-th power increases precision):

```
sage: z = x.nth_root(5); z
1 + 5^2 + 3*5^3 + 2*5^4 + 5^5 + 3*5^7 + 2*5^8 + O(5^9)
sage: z^5
1 + 5^3 + 3*5^4 + 4*5^5 + 3*5^6 + O(5^10)
```

```
>>> from sage.all import *
>>> z = x.nth_root(Integer(5)); z
1 + 5^2 + 3*5^3 + 2*5^4 + 5^5 + 3*5^7 + 2*5^8 + O(5^9)
>>> z**Integer(5)
1 + 5^3 + 3*5^4 + 4*5^5 + 3*5^6 + O(5^10)
```

Everything works over extensions as well:

```
sage: W.<a> = Zq(5^3)
sage: S.<x> = W[]
sage: R.<pi> = W.extension(x^7 - 5)
sage: R(5).nth_root(7)
pi + O(pi^141)
sage: R(5).nth_root(7, all=True)
[pi + O(pi^141)]
```

```
>>> from sage.all import *
>>> W = Zq(Integer(5)**Integer(3), names=('a',)); (a,) = W._first_ngens(1)
>>> S = W['x']; (x,) = S._first_ngens(1)
>>> R = W.extension(x**Integer(7) - Integer(5), names=('pi',)); (pi,) = R._
↪first_ngens(1)
>>> R(Integer(5)).nth_root(Integer(7))
pi + O(pi^141)
>>> R(Integer(5)).nth_root(Integer(7), all=True)
[pi + O(pi^141)]
```

An error is raised if the given element is not an n-th power in the ring:

```
sage: R(5).nth_root(11)
Traceback (most recent call last):
...
ValueError: this element is not a nth power
```

```
>>> from sage.all import *
>>> R(Integer(5)).nth_root(Integer(11))
Traceback (most recent call last):
...
ValueError: this element is not a nth power
```

Similarly, when precision on the input is too small, an error is raised:

```
sage: x = R(1,6); x
1 + O(pi^6)
sage: x.nth_root(5)
Traceback (most recent call last):
...
PrecisionError: not enough precision to be sure that this element is a nth␣
↪power
```

```
>>> from sage.all import *
>>> x = R(Integer(1),Integer(6)); x
1 + O(pi^6)
>>> x.nth_root(Integer(5))
Traceback (most recent call last):
...
PrecisionError: not enough precision to be sure that this element is a nth␣
↪power
```

Check that Issue #30314 is fixed:

```
sage: K = Qp(29)
sage: x = polygen(K)
sage: L.<a> = K.extension(x^2 - 29)
sage: L(4).nth_root(2)
2 + O(a^40)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(29))
>>> x = polygen(K)
>>> L = K.extension(x**Integer(2) - Integer(29), names=('a',)); (a,) = L._
↪first_ngens(1)
>>> L(Integer(4)).nth_root(Integer(2))
2 + O(a^40)
```

ordp(*p=None*)

Return the valuation of `self`, normalized so that the valuation of p is 1.

INPUT:

- `self` – a p-adic element
- `p` – a prime (default: `None`); if specified, will make sure that `p == self.parent().prime()`

Note

The optional argument p is used for consistency with the valuation methods on integers and rationals.

OUTPUT:

integer – the valuation of `self`, normalized so that the valuation of p is 1

EXAMPLES:

```
sage: R = Zp(5,20,'capped-rel')
sage: R(0).ordp()
+Infinity
sage: R(1).ordp()
0
sage: R(2).ordp()
0
sage: R(5).ordp()
1
sage: R(10).ordp()
1
sage: R(25).ordp()
2
sage: R(50).ordp()
2
sage: R(1/2).ordp()
0
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(20),'capped-rel')
```

(continues on next page)

(continued from previous page)

```
>>> R(Integer(0)).ordp()
+Infinity
>>> R(Integer(1)).ordp()
0
>>> R(Integer(2)).ordp()
0
>>> R(Integer(5)).ordp()
1
>>> R(Integer(10)).ordp()
1
>>> R(Integer(25)).ordp()
2
>>> R(Integer(50)).ordp()
2
>>> R(Integer(1)/Integer(2)).ordp()
0
```

polylog(*n*, *p_branch=0*)

Return $Li_n(\text{self})$, the n-th p-adic polylogarithm of this element.

INPUT:

- `n` – nonnegative integer
- `p_branch` – an element in the base ring or its fraction field; the implementation will choose the branch of the logarithm which sends p to `p_branch`

EXAMPLES:

The n-th polylogarithm of -1 is 0 for even n:

```
sage: Qp(13)(-1).polylog(6) == 0
True
```

```
>>> from sage.all import *
>>> Qp(Integer(13))(-Integer(1)).polylog(Integer(6)) == Integer(0)
True
```

We can check some identities, for example those mentioned in [DCW2016]:

```
sage: x = Qp(7, prec=30)(1/3)
sage: (x^2).polylog(4) - 8*x.polylog(4) - 8*(-x).polylog(4) == 0
True
```

```
>>> from sage.all import *
>>> x = Qp(Integer(7), prec=Integer(30))(Integer(1)/Integer(3))
>>> (x**Integer(2)).polylog(Integer(4)) - Integer(8)*x.polylog(Integer(4)) -␣
↪Integer(8)*(-x).polylog(Integer(4)) == Integer(0)
True
```

```
sage: x = Qp(5, prec=30)(4)
sage: x.polylog(2) + (1/x).polylog(2) + x.log(0)**2/2 == 0
True
```

```
>>> from sage.all import *
>>> x = Qp(Integer(5), prec=Integer(30))(Integer(4))
>>> x.polylog(Integer(2)) + (Integer(1)/x).polylog(Integer(2)) + x.
↪log(Integer(0))**Integer(2)/Integer(2) == Integer(0)
True
```

```
sage: x = Qp(11, prec=30)(2)
sage: x.polylog(2) + (1-x).polylog(2) + x.log(0)**2*(1-x).log(0) == 0
True
```

```
>>> from sage.all import *
>>> x = Qp(Integer(11), prec=Integer(30))(Integer(2))
>>> x.polylog(Integer(2)) + (Integer(1)-x).polylog(Integer(2)) + x.
↪log(Integer(0))**Integer(2)*(Integer(1)-x).log(Integer(0)) == Integer(0)
True
```

$Li_1(z) = -\log(1-z)$ for $|z| < 1$:

```
sage: Qp(5)(10).polylog(1) == -Qp(5)(1-10).log(0)
True
```

```
>>> from sage.all import *
>>> Qp(Integer(5))(Integer(10)).polylog(Integer(1)) == -
↪Qp(Integer(5))(Integer(1)-Integer(10)).log(Integer(0))
True
```

The dilogarithm of 1 is zero:

```
sage: Qp(5)(1).polylog(2)
O(5^20)
```

```
>>> from sage.all import *
>>> Qp(Integer(5))(Integer(1)).polylog(Integer(2))
O(5^20)
```

The cubing relation holds for the trilogarithm at 1:

```
sage: K = Qp(7)
sage: z = K.zeta(3)
sage: -8*K(1).polylog(3) == 9*(K(z).polylog(3) + K(z^2).polylog(3))
True
```

```
>>> from sage.all import *
>>> K = Qp(Integer(7))
>>> z = K.zeta(Integer(3))
>>> -Integer(8)*K(Integer(1)).polylog(Integer(3)) == Integer(9)*(K(z).
↪polylog(Integer(3)) + K(z**Integer(2)).polylog(Integer(3)))
True
```

The polylogarithm of 0 is 0:

```
sage: Qp(11)(0).polylog(7)
0
```

```
>>> from sage.all import *
>>> Qp(Integer(11))(Integer(0)).polylog(Integer(7))
0
```

Only polylogarithms for positive n are defined:

```
sage: Qp(11)(2).polylog(-1)
Traceback (most recent call last):
...
ValueError: polylogarithm only implemented for n at least 0
```

```
>>> from sage.all import *
>>> Qp(Integer(11))(Integer(2)).polylog(-Integer(1))
Traceback (most recent call last):
...
ValueError: polylogarithm only implemented for n at least 0
```

Check that Issue #29222 is fixed:

```
sage: K = Qp(7)
sage: print(K(1 + 7^11).polylog(4))
6*7^14 + 3*7^15 + 7^16 + 7^17 + O(7^18)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(7))
>>> print(K(Integer(1) + Integer(7)**Integer(11)).polylog(Integer(4)))
6*7^14 + 3*7^15 + 7^16 + 7^17 + O(7^18)
```

ALGORITHM:

The algorithm of Besser-de Jeu, as described in [BdJ2008] is used.

AUTHORS:

- Jennifer Balakrishnan - Initial implementation
- Alex J. Best (2017-07-21) - Extended to other residue disks

Todo

- Implement for extensions.
- Use the change method to create K from `self.parent()`.

rational_reconstruction()

Return a rational approximation to this p-adic number.

This will raise an `ArithmeticError` if there are no valid approximations to the unit part with numerator and denominator bounded by `sqrt(p^absprec / 2)`.

See also

rational

OUTPUT: rational; an approximation to `self`

EXAMPLES:

```
sage: R = Zp(5,20,'capped-rel')
sage: for i in range(11):
....:     for j in range(1,10):
....:         if j == 5:
....:             continue
....:         assert i/j == R(i/j).rational_reconstruction()
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(20),'capped-rel')
>>> for i in range(Integer(11)):
...     for j in range(Integer(1),Integer(10)):
...         if j == Integer(5):
...             continue
...         assert i/j == R(i/j).rational_reconstruction()
```

square_root(*extend=True*, *all=False*, *algorithm=None*)

Return the square root of this p-adic number.

INPUT:

- `self` – a p-adic element
- `extend` – boolean (default: `True`); if `True`, return a square root in an extension if necessary; if `False` and no root exists in the given ring or field, raise a `ValueError`.
- `all` – boolean (default: `False`); if `True`, return a list of all square roots
- `algorithm` – `'pari'`, `'sage'` or `None` (default: `None`); Sage provides an implementation for any extension of $\mathbf{Q}_p$, whereas only square roots over $\mathbf{Q}_p$ are implemented in PARI. The default is `'pari'` if the ground field is $\mathbf{Q}_p$, `'sage'` otherwise.

OUTPUT:

The square root or the list of all square roots of this p-adic number.

NOTE:

The square root is chosen (resp. the square roots are ordered) in a deterministic way.

EXAMPLES:

```
sage: R = Zp(3, 20)
sage: R(0).square_root()
0

sage: R(1).square_root()
1 + O(3^20)

sage: R(2).square_root(extend=False)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: element is not a square

sage: -R(4).square_root()
2 + O(3^20)

sage: R(9).square_root()
3 + O(3^21)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3), Integer(20))
>>> R(Integer(0)).square_root()
0

>>> R(Integer(1)).square_root()
1 + O(3^20)

>>> R(Integer(2)).square_root(extend=False)
Traceback (most recent call last):
...
ValueError: element is not a square

>>> -R(Integer(4)).square_root()
2 + O(3^20)

>>> R(Integer(9)).square_root()
3 + O(3^21)
```

When $p = 2$, the precision of the square root is less than the input:

```
sage: R2 = Zp(2, 20)
sage: R2(1).square_root()
1 + O(2^19)
sage: R2(4).square_root()
2 + O(2^20)

sage: R.<t> = Zq(2^10, 10)
sage: u = 1 + 8*t
sage: u.square_root()
1 + t*2^2 + t^2*2^3 + t^2*2^4 + (t^4 + t^3 + t^2)*2^5 + (t^4 + t^2)*2^6
 + (t^5 + t^2)*2^7 + (t^6 + t^5 + t^4 + t^2)*2^8 + O(2^9)

sage: x = polygen(ZZ, 'x')
sage: R.<a> = Zp(2).extension(x^3 - 2)
sage: u = R(1 + a^4 + a^5 + a^7 + a^8, 10); u
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
sage: v = u.square_root(); v
1 + a^2 + a^4 + a^6 + O(a^7)
```

```
>>> from sage.all import *
>>> R2 = Zp(Integer(2), Integer(20))
>>> R2(Integer(1)).square_root()
```

(continues on next page)

(continued from previous page)

```
1 + O(2^19)
>>> R2(Integer(4)).square_root()
2 + O(2^20)

>>> R = Zq(Integer(2)**Integer(10), Integer(10), names=('t',)); (t,) = R._
↪first_ngens(1)
>>> u = Integer(1) + Integer(8)*t
>>> u.square_root()
1 + t*2^2 + t^2*2^3 + t^2*2^4 + (t^4 + t^3 + t^2)*2^5 + (t^4 + t^2)*2^6
 + (t^5 + t^2)*2^7 + (t^6 + t^5 + t^4 + t^2)*2^8 + O(2^9)

>>> x = polygen(ZZ, 'x')
>>> R = Zp(Integer(2)).extension(x**Integer(3) - Integer(2), names=('a',));␣
↪(a,) = R._first_ngens(1)
>>> u = R(Integer(1) + a**Integer(4) + a**Integer(5) + a**Integer(7) +␣
↪a**Integer(8), Integer(10)); u
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
>>> v = u.square_root(); v
1 + a^2 + a^4 + a^6 + O(a^7)
```

However, observe that the precision increases to its original value when we recompute the square of the square root:

```
sage: v^2
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
```

```
>>> from sage.all import *
>>> v**Integer(2)
1 + a^4 + a^5 + a^7 + a^8 + O(a^10)
```

If the input does not have enough precision in order to determine if the given element has a square root in the ground field, an error is raised:

```
sage: R(1, 6).square_root()
Traceback (most recent call last):
...
PrecisionError: not enough precision to be sure that this element has a␣
↪square root
sage: R(1, 7).square_root()
1 + O(a^4)
sage: R(1+a^6, 7).square_root(extend=False)
Traceback (most recent call last):
...
ValueError: element is not a square
```

```
>>> from sage.all import *
>>> R(Integer(1), Integer(6)).square_root()
Traceback (most recent call last):
...
PrecisionError: not enough precision to be sure that this element has a␣
↪square root
>>> R(Integer(1), Integer(7)).square_root()
```

(continues on next page)

(continued from previous page)

```
1 + O(a^4)
>>> R(Integer(1)+a**Integer(6), Integer(7)).square_root(extend=False)
Traceback (most recent call last):
...
ValueError: element is not a square
```

In particular, an error is raised when we try to compute the square root of an inexact zero.

str(*mode=None*)

Return a string representation of `self`.

EXAMPLES:

```
sage: Zp(5,5,print_mode='bars')(1/3).str()[3:]
'1|3|1|3|2'
```

```
>>> from sage.all import *
>>> Zp(Integer(5),Integer(5),print_mode='bars')(Integer(1)/Integer(3)).
↪str()[Integer(3):]
'1|3|1|3|2'
```

trace(*base=None*)

Return the trace of this p-adic element over the base ring.

INPUT:

- `base` – a subring of the parent (default: base ring)

OUTPUT: the trace of this p-adic element over the given base

EXAMPLES:

```
sage: Zp(5,5)(5).trace()
5 + O(5^6)

sage: K.<a> = QqCR(2^3,7)
sage: S.<x> = K[]
sage: L.<pi> = K.extension(x^4 - 4*a*x^3 + 2*a)
sage: pi.trace()  # trace over K
a*2^2 + O(2^8)
sage: (pi+1).trace()
(a + 1)*2^2 + O(2^7)
```

```
>>> from sage.all import *
>>> Zp(Integer(5),Integer(5))(Integer(5)).trace()
5 + O(5^6)

>>> K = QqCR(Integer(2)**Integer(3),Integer(7), names=('a',)); (a,) = K._
↪first_ngens(1)
>>> S = K['x']; (x,) = S._first_ngens(1)
>>> L = K.extension(x**Integer(4) - Integer(4)*a*x**Integer(3) + Integer(2)*a,
↪ names=('pi',)); (pi,) = L._first_ngens(1)
>>> pi.trace()  # trace over K
a*2^2 + O(2^8)
```

(continues on next page)

(continued from previous page)

```
>>> (pi+Integer(1)).trace()
(a + 1)*2^2 + O(2^7)
```

val_unit()

Return `(self.valuation(), self.unit_part())`. To be overridden in derived classes.

EXAMPLES:

```
sage: Zp(5,5)(5).val_unit()
(1, 1 + O(5^5))
```

```
>>> from sage.all import *
>>> Zp(Integer(5),Integer(5))(Integer(5)).val_unit()
(1, 1 + O(5^5))
```

valuation(*p=None*)

Return the valuation of this element.

INPUT:

- `self` – a p-adic element
- `p` – a prime (default: `None`); if specified, will make sure that `p == self.parent().prime()`

Note

The optional argument p is used for consistency with the valuation methods on integers and rationals.

OUTPUT: integer; the valuation of `self`

EXAMPLES:

```
sage: R = Zp(17, 4,'capped-rel')
sage: a = R(2*17^2)
sage: a.valuation()
2
sage: R = Zp(5, 4,'capped-rel')
sage: R(0).valuation()
+Infinity
```

```
>>> from sage.all import *
>>> R = Zp(Integer(17), Integer(4),'capped-rel')
>>> a = R(Integer(2)*Integer(17)**Integer(2))
>>> a.valuation()
2
>>> R = Zp(Integer(5), Integer(4),'capped-rel')
>>> R(Integer(0)).valuation()
+Infinity
```

xgcd(*other*)

Compute the extended gcd of this element and `other`.

INPUT:

- `other` – an element in the same ring

OUTPUT:

A tuple `r`, `s`, `t` such that `r` is a greatest common divisor of this element and `other` and `r = s*self + t*other`.

AUTHORS:

- Julian Rueth (2012-10-19): initial version

Note

Since the elements are only given with finite precision, their greatest common divisor is in general not unique (not even up to units). For example $O(3)$ is a representative for the elements 0 and 3 in the 3-adic ring $\mathbf{Z}_3$. The greatest common divisor of $O(3)$ and $O(3)$ could be (among others) 3 or 0 which have different valuation. The algorithm implemented here, will return an element of minimal valuation among the possible greatest common divisors.

EXAMPLES:

The greatest common divisor is either zero or a power of the uniformizing parameter:

```
sage: R = Zp(3)
sage: R.zero().xgcd(R.zero())
(0, 1 + O(3^20), 0)
sage: R(3).xgcd(9)
(3 + O(3^21), 1 + O(3^20), 0)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3))
>>> R.zero().xgcd(R.zero())
(0, 1 + O(3^20), 0)
>>> R(Integer(3)).xgcd(Integer(9))
(3 + O(3^21), 1 + O(3^20), 0)
```

Unlike for *gcd()*, the result is not lifted to the maximal precision possible in the ring; it is such that `r = s*self + t*other` holds true:

```
sage: a = R(3,2); a
3 + O(3^2)
sage: b = R(9,3); b
3^2 + O(3^3)
sage: a.xgcd(b)
(3 + O(3^2), 1 + O(3), 0)
sage: a.xgcd(0)
(3 + O(3^2), 1 + O(3), 0)
```

```
>>> from sage.all import *
>>> a = R(Integer(3),Integer(2)); a
3 + O(3^2)
>>> b = R(Integer(9),Integer(3)); b
3^2 + O(3^3)
>>> a.xgcd(b)
```

(continues on next page)

(continued from previous page)

```
(3 + O(3^2), 1 + O(3), 0)
>>> a.xgcd(Integer(0))
(3 + O(3^2), 1 + O(3), 0)
```

If both elements are zero, then the result is zero with the precision set to the smallest of their precisions:

```
sage: a = R.zero(); a
0
sage: b = R(0,2); b
O(3^2)
sage: a.xgcd(b)
(O(3^2), 0, 1 + O(3^20))
```

```
>>> from sage.all import *
>>> a = R.zero(); a
0
>>> b = R(Integer(0),Integer(2)); b
O(3^2)
>>> a.xgcd(b)
(O(3^2), 0, 1 + O(3^20))
```

If only one element is zero, then the result depends on its precision:

```
sage: R(9).xgcd(R(0,1))
(O(3), 0, 1 + O(3^20))
sage: R(9).xgcd(R(0,2))
(O(3^2), 0, 1 + O(3^20))
sage: R(9).xgcd(R(0,3))
(3^2 + O(3^22), 1 + O(3^20), 0)
sage: R(9).xgcd(R(0,4))
(3^2 + O(3^22), 1 + O(3^20), 0)
```

```
>>> from sage.all import *
>>> R(Integer(9)).xgcd(R(Integer(0),Integer(1)))
(O(3), 0, 1 + O(3^20))
>>> R(Integer(9)).xgcd(R(Integer(0),Integer(2)))
(O(3^2), 0, 1 + O(3^20))
>>> R(Integer(9)).xgcd(R(Integer(0),Integer(3)))
(3^2 + O(3^22), 1 + O(3^20), 0)
>>> R(Integer(9)).xgcd(R(Integer(0),Integer(4)))
(3^2 + O(3^22), 1 + O(3^20), 0)
```

Over a field, the greatest common divisor is either zero (possibly with finite precision) or one:

```
sage: K = Qp(3)
sage: K(3).xgcd(0)
(1 + O(3^20), 3^-1 + O(3^19), 0)
sage: K.zero().xgcd(0)
(0, 1 + O(3^20), 0)
sage: K.zero().xgcd(K(0,2))
(O(3^2), 0, 1 + O(3^20))
sage: K(3).xgcd(4)
```

(continues on next page)

(continued from previous page)

```
(1 + O(3^20), 3^-1 + O(3^19), 0)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(3))
>>> K(Integer(3)).xgcd(Integer(0))
(1 + O(3^20), 3^-1 + O(3^19), 0)
>>> K.zero().xgcd(Integer(0))
(0, 1 + O(3^20), 0)
>>> K.zero().xgcd(K(Integer(0),Integer(2)))
(O(3^2), 0, 1 + O(3^20))
>>> K(Integer(3)).xgcd(Integer(4))
(1 + O(3^20), 3^-1 + O(3^19), 0)
```

CHAPTER

FOURTEEN

p-ADIC CAPPED RELATIVE ELEMENTS

Elements of p-adic Rings with Capped Relative Precision

AUTHORS:

- David Roe: initial version, rewriting to use templates (2012-3-1)
- Genya Zaytman: documentation
- David Harvey: doctests

class sage.rings.padics.padic_capped_relative_element.**CRElement**

Bases: *pAdicTemplateElement*

add_bigoh(*absprec*)

Return a new element with absolute precision decreased to `absprec`.

INPUT:

- `absprec` – integer or infinity

OUTPUT:

an equal element with precision set to the minimum of `self`'s precision and `absprec`

EXAMPLES:

```
sage: R = Zp(7,4,'capped-rel','series'); a = R(8); a.add_bigoh(1)
1 + O(7)
sage: b = R(0); b.add_bigoh(3)
O(7^3)
sage: R = Qp(7,4); a = R(8); a.add_bigoh(1)
1 + O(7)
sage: b = R(0); b.add_bigoh(3)
O(7^3)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'capped-rel','series'); a = R(Integer(8)); a.
↪add_bigoh(Integer(1))
1 + O(7)
>>> b = R(Integer(0)); b.add_bigoh(Integer(3))
O(7^3)
>>> R = Qp(Integer(7),Integer(4)); a = R(Integer(8)); a.add_bigoh(Integer(1))
1 + O(7)
>>> b = R(Integer(0)); b.add_bigoh(Integer(3))
O(7^3)
```

The precision never increases:

```
sage: R(4).add_bigoh(2).add_bigoh(4)
4 + O(7^2)
```

```
>>> from sage.all import *
>>> R(Integer(4)).add_bigoh(Integer(2)).add_bigoh(Integer(4))
4 + O(7^2)
```

Another example that illustrates that the precision does not increase:

```
sage: k = Qp(3,5)
sage: a = k(1234123412/3^70); a
2*3^-70 + 3^-69 + 3^-68 + 3^-67 + O(3^-65)
sage: a.add_bigoh(2)
2*3^-70 + 3^-69 + 3^-68 + 3^-67 + O(3^-65)

sage: k = Qp(5,10)
sage: a = k(1/5^3 + 5^2); a
5^-3 + 5^2 + O(5^7)
sage: a.add_bigoh(2)
5^-3 + O(5^2)
sage: a.add_bigoh(-1)
5^-3 + O(5^-1)
```

```
>>> from sage.all import *
>>> k = Qp(Integer(3),Integer(5))
>>> a = k(Integer(1234123412)/Integer(3)**Integer(70)); a
2*3^-70 + 3^-69 + 3^-68 + 3^-67 + O(3^-65)
>>> a.add_bigoh(Integer(2))
2*3^-70 + 3^-69 + 3^-68 + 3^-67 + O(3^-65)

>>> k = Qp(Integer(5),Integer(10))
>>> a = k(Integer(1)/Integer(5)**Integer(3) + Integer(5)**Integer(2)); a
5^-3 + 5^2 + O(5^7)
>>> a.add_bigoh(Integer(2))
5^-3 + O(5^2)
>>> a.add_bigoh(-Integer(1))
5^-3 + O(5^-1)
```

is_equal_to(*_right*, *absprec=None*)

Return whether `self` is equal to `right` modulo π^{absprec}.

If `absprec` is `None`, returns `True` if `self` and `right` are equal to the minimum of their precisions.

INPUT:

- `right` – a p-adic element
- `absprec` – integer, infinity, or `None`

EXAMPLES:

```
sage: R = Zp(5, 10); a = R(0); b = R(0, 3); c = R(75, 5)
sage: aa = a + 625; bb = b + 625; cc = c + 625
```

(continues on next page)

(continued from previous page)

```
sage: a.is_equal_to(aa), a.is_equal_to(aa, 4), a.is_equal_to(aa, 5)
(False, True, False)
sage: a.is_equal_to(aa, 15)
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

sage: a.is_equal_to(a, 50000)
True

sage: a.is_equal_to(b), a.is_equal_to(b, 2)
(True, True)
sage: a.is_equal_to(b, 5)
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

sage: b.is_equal_to(b, 5)
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

sage: b.is_equal_to(bb, 3)
True
sage: b.is_equal_to(bb, 4)
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

sage: c.is_equal_to(b, 2), c.is_equal_to(b, 3)
(True, False)
sage: c.is_equal_to(b, 4)
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

sage: c.is_equal_to(cc, 2), c.is_equal_to(cc, 4), c.is_equal_to(cc, 5)
(True, True, False)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(10)); a = R(Integer(0)); b = R(Integer(0),␣
↪Integer(3)); c = R(Integer(75), Integer(5))
>>> aa = a + Integer(625); bb = b + Integer(625); cc = c + Integer(625)
>>> a.is_equal_to(aa), a.is_equal_to(aa, Integer(4)), a.is_equal_to(aa,␣
↪Integer(5))
(False, True, False)
>>> a.is_equal_to(aa, Integer(15))
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

>>> a.is_equal_to(a, Integer(50000))
```

(continues on next page)

(continued from previous page)

```
True

>>> a.is_equal_to(b), a.is_equal_to(b, Integer(2))
(True, True)
>>> a.is_equal_to(b, Integer(5))
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

>>> b.is_equal_to(b, Integer(5))
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

>>> b.is_equal_to(bb, Integer(3))
True
>>> b.is_equal_to(bb, Integer(4))
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

>>> c.is_equal_to(b, Integer(2)), c.is_equal_to(b, Integer(3))
(True, False)
>>> c.is_equal_to(b, Integer(4))
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision

>>> c.is_equal_to(cc, Integer(2)), c.is_equal_to(cc, Integer(4)), c.is_equal_
↪to(cc, Integer(5))
(True, True, False)
```

is_zero(*absprec=None*)

Determine whether this element is zero modulo π^{absprec}.

If `absprec` is `None`, returns `True` if this element is indistinguishable from zero.

INPUT:

- `absprec` – integer, infinity, or `None`

EXAMPLES:

```
sage: R = Zp(5); a = R(0); b = R(0,5); c = R(75)
sage: a.is_zero(), a.is_zero(6)
(True, True)
sage: b.is_zero(), b.is_zero(5)
(True, True)
sage: c.is_zero(), c.is_zero(2), c.is_zero(3)
(False, True, False)
sage: b.is_zero(6)
Traceback (most recent call last):
...
PrecisionError: not enough precision to determine if element is zero
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5)); a = R(Integer(0)); b = R(Integer(0),Integer(5)); c =␣
↪R(Integer(75))
>>> a.is_zero(), a.is_zero(Integer(6))
(True, True)
>>> b.is_zero(), b.is_zero(Integer(5))
(True, True)
>>> c.is_zero(), c.is_zero(Integer(2)), c.is_zero(Integer(3))
(False, True, False)
>>> b.is_zero(Integer(6))
Traceback (most recent call last):
...
PrecisionError: not enough precision to determine if element is zero
```

polynomial(*var='x'*)

Return a polynomial over the base ring that yields this element when evaluated at the generator of the parent.

INPUT:

- `var` – string, the variable name for the polynomial

EXAMPLES:

```
sage: K.<a> = Qq(5^3)
sage: a.polynomial()
(1 + O(5^20))*x + O(5^20)
sage: a.polynomial(var='y')
(1 + O(5^20))*y + O(5^20)
sage: (5*a^2 + K(25, 4)).polynomial()
(5 + O(5^4))*x^2 + O(5^4)*x + 5^2 + O(5^4)
```

```
>>> from sage.all import *
>>> K = Qq(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> a.polynomial()
(1 + O(5^20))*x + O(5^20)
>>> a.polynomial(var='y')
(1 + O(5^20))*y + O(5^20)
>>> (Integer(5)*a**Integer(2) + K(Integer(25), Integer(4))).polynomial()
(5 + O(5^4))*x^2 + O(5^4)*x + 5^2 + O(5^4)
```

precision_absolute()

Return the absolute precision of this element.

This is the power of the maximal ideal modulo which this element is defined.

EXAMPLES:

```
sage: R = Zp(7,3,'capped-rel'); a = R(7); a.precision_absolute()
4
sage: R = Qp(7,3); a = R(7); a.precision_absolute()
4
sage: R(7^-3).precision_absolute()
0

sage: R(0).precision_absolute()
```

(continues on next page)

(continued from previous page)

```
+Infinity
sage: R(0,7).precision_absolute()
7
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(3),'capped-rel'); a = R(Integer(7)); a.
↪precision_absolute()
4
>>> R = Qp(Integer(7),Integer(3)); a = R(Integer(7)); a.precision_absolute()
4
>>> R(Integer(7)**-Integer(3)).precision_absolute()
0

>>> R(Integer(0)).precision_absolute()
+Infinity
>>> R(Integer(0),Integer(7)).precision_absolute()
7
```

precision_relative()

Return the relative precision of this element.

This is the power of the maximal ideal modulo which the unit part of `self` is defined.

EXAMPLES:

```
sage: R = Zp(7,3,'capped-rel'); a = R(7); a.precision_relative()
3
sage: R = Qp(7,3); a = R(7); a.precision_relative()
3
sage: a = R(7^-2, -1); a.precision_relative()
1
sage: a
7^-2 + O(7^-1)

sage: R(0).precision_relative()
0
sage: R(0,7).precision_relative()
0
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(3),'capped-rel'); a = R(Integer(7)); a.
↪precision_relative()
3
>>> R = Qp(Integer(7),Integer(3)); a = R(Integer(7)); a.precision_relative()
3
>>> a = R(Integer(7)**-Integer(2), -Integer(1)); a.precision_relative()
1
>>> a
7^-2 + O(7^-1)

>>> R(Integer(0)).precision_relative()
0
```

(continues on next page)

(continued from previous page)

```
>>> R(Integer(0),Integer(7)).precision_relative()
0
```

unit_part()

Return u, where this element is $\pi^v u$.

EXAMPLES:

```
sage: R = Zp(17,4,'capped-rel')
sage: a = R(18*17)
sage: a.unit_part()
1 + 17 + O(17^4)
sage: type(a)
<class 'sage.rings.padics.padic_capped_relative_element.
↪pAdicCappedRelativeElement'>
sage: R = Qp(17,4,'capped-rel')
sage: a = R(18*17)
sage: a.unit_part()
1 + 17 + O(17^4)
sage: type(a)
<class 'sage.rings.padics.padic_capped_relative_element.
↪pAdicCappedRelativeElement'>
sage: a = R(2*17^2); a
2*17^2 + O(17^6)
sage: a.unit_part()
2 + O(17^4)
sage: b=1/a; b
9*17^-2 + 8*17^-1 + 8 + 8*17 + O(17^2)
sage: b.unit_part()
9 + 8*17 + 8*17^2 + 8*17^3 + O(17^4)
sage: Zp(5)(75).unit_part()
3 + O(5^20)

sage: R(0).unit_part()
Traceback (most recent call last):
...
ValueError: unit part of 0 not defined
sage: R(0,7).unit_part()
O(17^0)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(17),Integer(4),'capped-rel')
>>> a = R(Integer(18)*Integer(17))
>>> a.unit_part()
1 + 17 + O(17^4)
>>> type(a)
<class 'sage.rings.padics.padic_capped_relative_element.
↪pAdicCappedRelativeElement'>
>>> R = Qp(Integer(17),Integer(4),'capped-rel')
>>> a = R(Integer(18)*Integer(17))
>>> a.unit_part()
1 + 17 + O(17^4)
```

(continues on next page)

(continued from previous page)

```
>>> type(a)
<class 'sage.rings.padics.padic_capped_relative_element.
↪pAdicCappedRelativeElement'>
>>> a = R(Integer(2)*Integer(17)**Integer(2)); a
2*17^2 + O(17^6)
>>> a.unit_part()
2 + O(17^4)
>>> b=Integer(1)/a; b
9*17^-2 + 8*17^-1 + 8 + 8*17 + O(17^2)
>>> b.unit_part()
9 + 8*17 + 8*17^2 + 8*17^3 + O(17^4)
>>> Zp(Integer(5))(Integer(75)).unit_part()
3 + O(5^20)

>>> R(Integer(0)).unit_part()
Traceback (most recent call last):
...
ValueError: unit part of 0 not defined
>>> R(Integer(0),Integer(7)).unit_part()
O(17^0)
```

val_unit(*p=None*)

Return a pair `(self.valuation(), self.unit_part())`.

INPUT:

- `p` – a prime (default: `None`); if specified, will make sure that `p == self.parent().prime()`

> **Note**
>
> The optional argument `p` is used for consistency with the valuation methods on integers and rationals.

EXAMPLES:

```
sage: R = Zp(5); a = R(75, 20); a
3*5^2 + O(5^20)
sage: a.val_unit()
(2, 3 + O(5^18))
sage: R(0).val_unit()
Traceback (most recent call last):
...
ValueError: unit part of 0 not defined
sage: R(0, 10).val_unit()
(10, O(5^0))
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5)); a = R(Integer(75), Integer(20)); a
3*5^2 + O(5^20)
>>> a.val_unit()
(2, 3 + O(5^18))
>>> R(Integer(0)).val_unit()
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: unit part of 0 not defined
>>> R(Integer(0), Integer(10)).val_unit()
(10, O(5^0))
```

class sage.rings.padics.padic_capped_relative_element.**ExpansionIter**

Bases: object

An iterator over a p-adic expansion.

This class should not be instantiated directly, but instead using `expansion`.

INPUT:

- `elt` – the p-adic element
- `prec` – the number of terms to be emitted
- `mode` – either `simple_mode`, `smallest_mode` or `teichmuller_mode`

EXAMPLES:

```
sage: E = Zp(5,4)(373).expansion()
sage: I = iter(E)  # indirect doctest
sage: type(I)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIter'>
```

```
>>> from sage.all import *
>>> E = Zp(Integer(5),Integer(4))(Integer(373)).expansion()
>>> I = iter(E)  # indirect doctest
>>> type(I)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIter'>
```

class sage.rings.padics.padic_capped_relative_element.**ExpansionIterable**

Bases: object

An iterable storing a p-adic expansion of an element.

This class should not be instantiated directly, but instead using *expansion()*.

INPUT:

- `elt` – the p-adic element
- `prec` – the number of terms to be emitted
- `val_shift` – how many zeros to add at the beginning of the expansion, or the number of initial terms to truncate (if negative)
- `mode` – one of the following:
  - `'simple_mode'`
  - `'smallest_mode'`
  - `'teichmuller_mode'`

EXAMPLES:

```
sage: E = Zp(5,4)(373).expansion()  # indirect doctest
sage: type(E)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIterable'>
```

```
>>> from sage.all import *
>>> E = Zp(Integer(5),Integer(4))(Integer(373)).expansion()  # indirect doctest
>>> type(E)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIterable'>
```

class sage.rings.padics.padic_capped_relative_element.**PowComputer_**

Bases: *PowComputer_base*

A PowComputer for a capped-relative p-adic ring or field.

sage.rings.padics.padic_capped_relative_element.**base_p_list**(*n*, *pos*, *prime_pow*)

Return a base-p list of digits of `n`.

INPUT:

- `n` – a positive `Integer`
- `pos` – boolean; if `True`, then returns the standard base p expansion, otherwise the digits lie in the range $-p/2$ to $p/2$
- `prime_pow` – a *PowComputer* giving the prime

EXAMPLES:

```
sage: from sage.rings.padics.padic_capped_relative_element import base_p_list
sage: base_p_list(192837, True, Zp(5).prime_pow)
[2, 2, 3, 2, 3, 1, 2, 2]
sage: 2 + 2*5 + 3*5^2 + 2*5^3 + 3*5^4 + 5^5 + 2*5^6 + 2*5^7
192837
sage: base_p_list(192837, False, Zp(5).prime_pow)
[2, 2, -2, -2, -1, 2, 2, 2]
sage: 2 + 2*5 - 2*5^2 - 2*5^3 - 5^4 + 2*5^5 + 2*5^6 + 2*5^7
192837
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_capped_relative_element import base_p_list
>>> base_p_list(Integer(192837), True, Zp(Integer(5)).prime_pow)
[2, 2, 3, 2, 3, 1, 2, 2]
>>> Integer(2) + Integer(2)*Integer(5) + Integer(3)*Integer(5)**Integer(2) +␣
↪Integer(2)*Integer(5)**Integer(3) + Integer(3)*Integer(5)**Integer(4) +␣
↪Integer(5)**Integer(5) + Integer(2)*Integer(5)**Integer(6) +␣
↪Integer(2)*Integer(5)**Integer(7)
192837
>>> base_p_list(Integer(192837), False, Zp(Integer(5)).prime_pow)
[2, 2, -2, -2, -1, 2, 2, 2]
>>> Integer(2) + Integer(2)*Integer(5) - Integer(2)*Integer(5)**Integer(2) -␣
↪Integer(2)*Integer(5)**Integer(3) - Integer(5)**Integer(4) +␣
↪Integer(2)*Integer(5)**Integer(5) + Integer(2)*Integer(5)**Integer(6) +␣
↪Integer(2)*Integer(5)**Integer(7)
192837
```

class sage.rings.padics.padic_capped_relative_element.**pAdicCappedRelativeElement**

Bases: *CRElement*

Construct new element with given parent and value.

INPUT:

- `x` – value to coerce into a capped relative ring or field
- `absprec` – maximum number of digits of absolute precision
- `relprec` – maximum number of digits of relative precision

EXAMPLES:

```
sage: R = Zp(5, 10, 'capped-rel')
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(10), 'capped-rel')
```

Construct from integers:

```
sage: R(3)
3 + O(5^10)
sage: R(75)
3*5^2 + O(5^12)
sage: R(0)
0
sage: R(-1)
4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + O(5^10)
sage: R(-5)
4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 +␣
↪O(5^11)
sage: R(-7*25)
3*5^2 + 3*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11 +␣
↪O(5^12)
```

```
>>> from sage.all import *
>>> R(Integer(3))
3 + O(5^10)
>>> R(Integer(75))
3*5^2 + O(5^12)
>>> R(Integer(0))
0
>>> R(-Integer(1))
4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + O(5^10)
>>> R(-Integer(5))
4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 +␣
↪O(5^11)
>>> R(-Integer(7)*Integer(25))
3*5^2 + 3*5^3 + 4*5^4 + 4*5^5 + 4*5^6 + 4*5^7 + 4*5^8 + 4*5^9 + 4*5^10 + 4*5^11 +␣
↪O(5^12)
```

Construct from rationals:

```
sage: R(1/2)
3 + 2*5 + 2*5^2 + 2*5^3 + 2*5^4 + 2*5^5 + 2*5^6 + 2*5^7 + 2*5^8 + 2*5^9 + O(5^10)
sage: R(-7875/874)
3*5^3 + 2*5^4 + 2*5^5 + 5^6 + 3*5^7 + 2*5^8 + 3*5^10 + 3*5^11 + 3*5^12 + O(5^13)
sage: R(15/425)
Traceback (most recent call last):
...
ValueError: p divides the denominator
```

```
>>> from sage.all import *
>>> R(Integer(1)/Integer(2))
3 + 2*5 + 2*5^2 + 2*5^3 + 2*5^4 + 2*5^5 + 2*5^6 + 2*5^7 + 2*5^8 + 2*5^9 + O(5^10)
>>> R(-Integer(7875)/Integer(874))
3*5^3 + 2*5^4 + 2*5^5 + 5^6 + 3*5^7 + 2*5^8 + 3*5^10 + 3*5^11 + 3*5^12 + O(5^13)
>>> R(Integer(15)/Integer(425))
Traceback (most recent call last):
...
ValueError: p divides the denominator
```

Construct from IntegerMod:

```
sage: R(Integers(125)(3))
3 + O(5^3)
sage: R(Integers(5)(3))
3 + O(5)
sage: R(Integers(5^30)(3))
3 + O(5^10)
sage: R(Integers(5^30)(1+5^23))
1 + O(5^10)
sage: R(Integers(49)(3))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 49
```

```
>>> from sage.all import *
>>> R(Integers(Integer(125))(Integer(3)))
3 + O(5^3)
>>> R(Integers(Integer(5))(Integer(3)))
3 + O(5)
>>> R(Integers(Integer(5)**Integer(30))(Integer(3)))
3 + O(5^10)
>>> R(Integers(Integer(5)**Integer(30))(Integer(1)+Integer(5)**Integer(23)))
1 + O(5^10)
>>> R(Integers(Integer(49))(Integer(3)))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 49
```

```
sage: R(Integers(48)(3))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 48
```

```
>>> from sage.all import *
>>> R(Integers(Integer(48))(Integer(3)))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 48
```

Some other conversions:

```
sage: R(R(5))
5 + O(5^11)
```

```
>>> from sage.all import *
>>> R(R(Integer(5)))
5 + O(5^11)
```

Construct from Pari objects:

```
sage: R = Zp(5)
sage: x = pari(123123) ; R(x)                                                   #␣
↪needs sage.libs.pari
3 + 4*5 + 4*5^2 + 4*5^3 + 5^4 + 4*5^5 + 2*5^6 + 5^7 + O(5^20)
sage: R(pari(R(5252)))
2 + 2*5^3 + 3*5^4 + 5^5 + O(5^20)
sage: R = Zp(5,prec=5)
sage: R(pari(-1))
4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5)
sage: pari(R(-1))                                                               #␣
↪needs sage.libs.pari
4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5)
sage: pari(R(0))                                                                #␣
↪needs sage.libs.pari
0
sage: R(pari(R(0,5)))
O(5^5)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5))
>>> x = pari(Integer(123123)) ; R(x)                                            ␣
↪     # needs sage.libs.pari
3 + 4*5 + 4*5^2 + 4*5^3 + 5^4 + 4*5^5 + 2*5^6 + 5^7 + O(5^20)
>>> R(pari(R(Integer(5252))))
2 + 2*5^3 + 3*5^4 + 5^5 + O(5^20)
>>> R = Zp(Integer(5),prec=Integer(5))
>>> R(pari(-Integer(1)))
4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5)
>>> pari(R(-Integer(1)))                                                        ␣
↪     # needs sage.libs.pari
4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5)
>>> pari(R(Integer(0)))                                                         ␣
↪     # needs sage.libs.pari
0
>>> R(pari(R(Integer(0),Integer(5))))
O(5^5)
```

Todo

doctests for converting from other types of p-adic rings

lift()

Return an integer or rational congruent to `self` modulo `self`'s precision. If a rational is returned, its denominator will equal `p^ordp(self)`.

EXAMPLES:

```
sage: R = Zp(7,4,'capped-rel'); a = R(8); a.lift()
8
sage: R = Qp(7,4); a = R(8); a.lift()
8
sage: R = Qp(7,4); a = R(8/7); a.lift()
8/7
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'capped-rel'); a = R(Integer(8)); a.lift()
8
>>> R = Qp(Integer(7),Integer(4)); a = R(Integer(8)); a.lift()
8
>>> R = Qp(Integer(7),Integer(4)); a = R(Integer(8)/Integer(7)); a.lift()
8/7
```

residue(*absprec=1*, *field=None*, *check_prec=True*)

Reduce this element modulo p^{absprec}.

INPUT:

- `absprec` – nonnegative integer (default: 1)
- `field` – boolean (default: `None`); whether to return an element of $\mathbf{F}_p$ or $\mathbf{Z}/p\mathbf{Z}$
- `check_prec` – boolean (default: `True`); whether to raise an error if this element has insufficient precision to determine the reduction

OUTPUT:

This element reduced modulo p^{absprec} as an element of $\mathbf{Z}/p^{\text{absprec}}\mathbf{Z}$.

EXAMPLES:

```
sage: R = Zp(7,4)
sage: a = R(8)
sage: a.residue(1)
1
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4))
>>> a = R(Integer(8))
>>> a.residue(Integer(1))
1
```

This is different from applying `% p^n` which returns an element in the same ring:

```
sage: b = a.residue(2); b
8
sage: b.parent()
Ring of integers modulo 49
sage: c = a % 7^2; c
1 + 7 + O(7^4)
sage: c.parent()
7-adic Ring with capped relative precision 4
```

```
>>> from sage.all import *
>>> b = a.residue(Integer(2)); b
8
>>> b.parent()
Ring of integers modulo 49
>>> c = a % Integer(7)**Integer(2); c
1 + 7 + O(7^4)
>>> c.parent()
7-adic Ring with capped relative precision 4
```

For elements in a field, application of `% p^n` always returns zero, the remainder of the division by `p^n`:

```
sage: K = Qp(7,4)
sage: a = K(8)
sage: a.residue(2)
8
sage: a % 7^2
1 + 7 + O(7^4)

sage: b = K(1/7)
sage: b.residue()
Traceback (most recent call last):
...
ValueError: element must have nonnegative valuation in order to compute␣
↪residue
```

```
>>> from sage.all import *
>>> K = Qp(Integer(7),Integer(4))
>>> a = K(Integer(8))
>>> a.residue(Integer(2))
8
>>> a % Integer(7)**Integer(2)
1 + 7 + O(7^4)

>>> b = K(Integer(1)/Integer(7))
>>> b.residue()
Traceback (most recent call last):
...
ValueError: element must have nonnegative valuation in order to compute␣
↪residue
```

See also

mod

class sage.rings.padics.padic_capped_relative_element.**pAdicCoercion_CR_frac_field**

Bases: RingHomomorphism

The canonical inclusion of $\mathbf{Z}_q$ into its fraction field.

EXAMPLES:

```
sage: R.<a> = ZqCR(27, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R); f
Ring morphism:
  From: 3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(27), implementation='FLINT', names=('a',)); (a,) = R._first_
↪ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R); f
Ring morphism:
  From: 3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
```

is_injective()

Return whether this map is injective.

EXAMPLES:

```
sage: R.<a> = ZqCR(9, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f.is_injective()
True
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(9), implementation='FLINT', names=('a',)); (a,) = R._
↪first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f.is_injective()
True
```

is_surjective()

Return whether this map is surjective.

EXAMPLES:

```
sage: R.<a> = ZqCR(9, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
```

(continues on next page)

(continued from previous page)

```
sage: f.is_surjective()
False
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(9), implementation='FLINT', names=('a',)); (a,) = R._
↪first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f.is_surjective()
False
```

section()

Return a map back to the ring that converts elements of nonnegative valuation.

EXAMPLES:

```
sage: R.<a> = ZqCR(27, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f(K.gen())
a + O(3^20)
sage: f.section()
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(27), implementation='FLINT', names=('a',)); (a,) = R._
↪first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f(K.gen())
a + O(3^20)
>>> f.section()
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

class sage.rings.padics.padic_capped_relative_element.**pAdicCoercion_QQ_CR**

Bases: RingHomomorphism

The canonical inclusion from the rationals to a capped relative field.

EXAMPLES:

```
sage: f = Qp(5).coerce_map_from(QQ); f
Ring morphism:
  From: Rational Field
  To:   5-adic Field with capped relative precision 20
```

```
>>> from sage.all import *
>>> f = Qp(Integer(5)).coerce_map_from(QQ); f
Ring morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Rational Field
  To:   5-adic Field with capped relative precision 20
```

section()

Return a map back to the rationals that approximates an element by a rational number.

EXAMPLES:

```
sage: f = Qp(5).coerce_map_from(QQ).section()
sage: f(Qp(5)(1/4))
1/4
sage: f(Qp(5)(1/5))
1/5
```

```
>>> from sage.all import *
>>> f = Qp(Integer(5)).coerce_map_from(QQ).section()
>>> f(Qp(Integer(5))(Integer(1)/Integer(4)))
1/4
>>> f(Qp(Integer(5))(Integer(1)/Integer(5)))
1/5
```

class sage.rings.padics.padic_capped_relative_element.**pAdicCoercion_ZZ_CR**

Bases: RingHomomorphism

The canonical inclusion from the integer ring to a capped relative ring.

EXAMPLES:

```
sage: f = Zp(5).coerce_map_from(ZZ); f
Ring morphism:
  From: Integer Ring
  To:   5-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> f = Zp(Integer(5)).coerce_map_from(ZZ); f
Ring morphism:
  From: Integer Ring
  To:   5-adic Ring with capped relative precision 20
```

section()

Return a map back to the ring of integers that approximates an element by an integer.

EXAMPLES:

```
sage: f = Zp(5).coerce_map_from(ZZ).section()
sage: f(Zp(5)(-1)) - 5^20
-1
```

```
>>> from sage.all import *
>>> f = Zp(Integer(5)).coerce_map_from(ZZ).section()
>>> f(Zp(Integer(5))(-Integer(1))) - Integer(5)**Integer(20)
-1
```

class sage.rings.padics.padic_capped_relative_element.**pAdicConvert_CR_QQ**

Bases: RingMap

The map from the capped relative ring back to the rationals that returns a rational approximation of its input.

EXAMPLES:

```
sage: f = Qp(5).coerce_map_from(QQ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Field with capped relative precision 20
  To:   Rational Field
```

```
>>> from sage.all import *
>>> f = Qp(Integer(5)).coerce_map_from(QQ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Field with capped relative precision 20
  To:   Rational Field
```

class sage.rings.padics.padic_capped_relative_element.**pAdicConvert_CR_ZZ**

Bases: RingMap

The map from a capped relative ring back to the ring of integers that returns the smallest nonnegative integer approximation to its input which is accurate up to the precision.

Raises a ValueError, if the input is not in the closure of the image of the integers.

EXAMPLES:

```
sage: f = Zp(5).coerce_map_from(ZZ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Ring with capped relative precision 20
  To:   Integer Ring
```

```
>>> from sage.all import *
>>> f = Zp(Integer(5)).coerce_map_from(ZZ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Ring with capped relative precision 20
  To:   Integer Ring
```

class sage.rings.padics.padic_capped_relative_element.**pAdicConvert_CR_frac_field**

Bases: Morphism

The section of the inclusion from $\mathbf{Z}_q$ to its fraction field.

EXAMPLES:

```
sage: R.<a> = ZqCR(27, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = R.convert_map_from(K); f
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(27), implementation='FLINT', names=('a',)); (a,) = R._first_
↪ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> K = R.fraction_field()
>>> f = R.convert_map_from(K); f
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

class sage.rings.padics.padic_capped_relative_element.**pAdicConvert_QQ_CR**

Bases: Morphism

The inclusion map from the rationals to a capped relative ring that is defined on all elements with nonnegative p-adic valuation.

EXAMPLES:

```
sage: f = Zp(5).convert_map_from(QQ); f
Generic morphism:
  From: Rational Field
  To:   5-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> f = Zp(Integer(5)).convert_map_from(QQ); f
Generic morphism:
  From: Rational Field
  To:   5-adic Ring with capped relative precision 20
```

section()

Return the map back to the rationals that returns the smallest nonnegative integer approximation to its input which is accurate up to the precision.

EXAMPLES:

```
sage: f = Zp(5,4).convert_map_from(QQ).section()
sage: f(Zp(5,4)(-1))
-1
```

```
>>> from sage.all import *
>>> f = Zp(Integer(5),Integer(4)).convert_map_from(QQ).section()
>>> f(Zp(Integer(5),Integer(4))(-Integer(1)))
-1
```

class sage.rings.padics.padic_capped_relative_element.**pAdicTemplateElement**

Bases: *pAdicGenericElement*

A class for common functionality among the p-adic template classes.

INPUT:

- parent – a local ring or field
- x – data defining this element. Various types are supported, including ints, Integers, Rationals, PARI p-adics, integers mod p^k and other Sage p-adics.
- absprec – a cap on the absolute precision of this element
- relprec – a cap on the relative precision of this element

EXAMPLES:

```
sage: Zp(17)(17^3, 8, 4)
17^3 + O(17^7)
```

```
>>> from sage.all import *
>>> Zp(Integer(17))(Integer(17)**Integer(3), Integer(8), Integer(4))
17^3 + O(17^7)
```

expansion(*n=None*, *lift_mode='simple'*, *start_val=None*)

Return the coefficients in a π-adic expansion. If this is a field element, start at $\pi^{\text{valuation}}$, if a ring element at π^0.

For each lift mode, this function returns a list of a_i so that this element can be expressed as

$$\pi^v \cdot \sum_{i=0}^{\infty} a_i \pi^i,$$

where v is the valuation of this element when the parent is a field, and $v = 0$ otherwise.

Different lift modes affect the choice of a_i. When `lift_mode` is `'simple'`, the resulting a_i will be non-negative: if the residue field is $\mathbf{F}_p$ then they will be integers with $0 \leq a_i < p$; otherwise they will be a list of integers in the same range giving the coefficients of a polynominant in the indeterminant representing the maximal unramified subextension.

Choosing `lift_mode` as `'smallest'` is similar to `'simple'`, but uses a balanced representation $-p/2 < a_i \leq p/2$.

Finally, setting `lift_mode = 'teichmuller'` will yield Teichmuller representatives for the a_i: $a_i^q = a_i$. In this case the a_i will lie in the ring of integers of the maximal unramified subextension of the parent of this element.

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion. Can also accept a slice (see *slice()*).
- `lift_mode` – `'simple'`, `'smallest'` or `'teichmuller'` (default: `'simple'`)
- `start_val` – start at this valuation rather than the default (0 or the valuation of this element)

OUTPUT:

- If `n` is `None`, an iterable giving a π-adic expansion of this element. For base elements the contents will be integers if `lift_mode` is `'simple'` or `'smallest'`, and elements of `self.parent()` if `lift_mode` is `'teichmuller'`.
- If `n` is an integer, the coefficient of π^n in the π-adic expansion of this element.

Note

Use slice operators to get a particular range.

EXAMPLES:

```
sage: R = Zp(7,6); a = R(12837162817); a
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
sage: E = a.expansion(); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
```

(continues on next page)

(continued from previous page)

```
sage: list(E)
[3, 4, 4, 0, 4, 0]
sage: sum([c * 7^i for i, c in enumerate(E)]) == a
True
sage: E = a.expansion(lift_mode='smallest'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (balanced)
sage: list(E)
[3, -3, -2, 1, -3, 1]
sage: sum([c * 7^i for i, c in enumerate(E)]) == a
True
sage: E = a.expansion(lift_mode='teichmuller'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (teichmuller)
sage: list(E)
[3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + O(7^6),
0,
5 + 2*7 + 3*7^3 + O(7^4),
1 + O(7^3),
3 + 4*7 + O(7^2),
5 + O(7)]
sage: sum(c * 7^i for i, c in enumerate(E))
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(6)); a = R(Integer(12837162817)); a
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
>>> E = a.expansion(); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
>>> list(E)
[3, 4, 4, 0, 4, 0]
>>> sum([c * Integer(7)**i for i, c in enumerate(E)]) == a
True
>>> E = a.expansion(lift_mode='smallest'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (balanced)
>>> list(E)
[3, -3, -2, 1, -3, 1]
>>> sum([c * Integer(7)**i for i, c in enumerate(E)]) == a
True
>>> E = a.expansion(lift_mode='teichmuller'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (teichmuller)
>>> list(E)
[3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + O(7^6),
0,
5 + 2*7 + 3*7^3 + O(7^4),
1 + O(7^3),
3 + 4*7 + O(7^2),
5 + O(7)]
>>> sum(c * Integer(7)**i for i, c in enumerate(E))
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
```

If the element has positive valuation then the list will start with some zeros:

```
sage: a = R(7^3 * 17)
```

(continues on next page)

(continued from previous page)

```
sage: E = a.expansion(); E
7-adic expansion of 3*7^3 + 2*7^4 + O(7^9)
sage: list(E)
[0, 0, 0, 3, 2, 0, 0, 0, 0]
```

```
>>> from sage.all import *
>>> a = R(Integer(7)**Integer(3) * Integer(17))
>>> E = a.expansion(); E
7-adic expansion of 3*7^3 + 2*7^4 + O(7^9)
>>> list(E)
[0, 0, 0, 3, 2, 0, 0, 0, 0]
```

The expansion of 0 is truncated:

```
sage: E = R(0, 7).expansion(); E
7-adic expansion of O(7^7)
sage: len(E)
0
sage: list(E)
[]
```

```
>>> from sage.all import *
>>> E = R(Integer(0), Integer(7)).expansion(); E
7-adic expansion of O(7^7)
>>> len(E)
0
>>> list(E)
[]
```

In fields, on the other hand, the expansion starts at the valuation:

```
sage: R = Qp(7,4); a = R(6*7+7**2); E = a.expansion(); E
7-adic expansion of 6*7 + 7^2 + O(7^5)
sage: list(E)
[6, 1, 0, 0]
sage: list(a.expansion(lift_mode='smallest'))
[-1, 2, 0, 0]
sage: list(a.expansion(lift_mode='teichmuller'))
[6 + 6*7 + 6*7^2 + 6*7^3 + O(7^4),
2 + 4*7 + 6*7^2 + O(7^3),
3 + 4*7 + O(7^2),
3 + O(7)]
```

```
>>> from sage.all import *
>>> R = Qp(Integer(7),Integer(4)); a =␣
↪R(Integer(6)*Integer(7)+Integer(7)**Integer(2)); E = a.expansion(); E
7-adic expansion of 6*7 + 7^2 + O(7^5)
>>> list(E)
[6, 1, 0, 0]
>>> list(a.expansion(lift_mode='smallest'))
[-1, 2, 0, 0]
>>> list(a.expansion(lift_mode='teichmuller'))
```

(continues on next page)

(continued from previous page)

```
[6 + 6*7 + 6*7^2 + 6*7^3 + O(7^4),
2 + 4*7 + 6*7^2 + O(7^3),
3 + 4*7 + O(7^2),
3 + O(7)]
```

You can ask for a specific entry in the expansion:

```
sage: a.expansion(1)
6
sage: a.expansion(1, lift_mode='smallest')
-1
sage: a.expansion(2, lift_mode='teichmuller')
2 + 4*7 + 6*7^2 + O(7^3)
```

```
>>> from sage.all import *
>>> a.expansion(Integer(1))
6
>>> a.expansion(Integer(1), lift_mode='smallest')
-1
>>> a.expansion(Integer(2), lift_mode='teichmuller')
2 + 4*7 + 6*7^2 + O(7^3)
```

lift_to_precision(*absprec=None*)

Return another element of the same parent with absolute precision at least `absprec`, congruent to this p-adic element modulo the precision of this element.

INPUT:

- `absprec` – integer or `None` (default: `None`); the absolute precision of the result. If `None`, lifts to the maximum precision allowed.

Note

If setting `absprec` that high would violate the precision cap, raises a precision error. Note that the new digits will not necessarily be zero.

EXAMPLES:

```
sage: R = ZpCA(17)
sage: R(-1,2).lift_to_precision(10)
16 + 16*17 + O(17^10)
sage: R(1,15).lift_to_precision(10)
1 + O(17^15)
sage: R(1,15).lift_to_precision(30)
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
sage: R(-1,2).lift_to_precision().precision_absolute() == R.precision_cap()
True

sage: R = Zp(5); c = R(17,3); c.lift_to_precision(8)
2 + 3*5 + O(5^8)
```

(continues on next page)

(continued from previous page)

```
sage: c.lift_to_precision().precision_relative() == R.precision_cap()
True
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(17))
>>> R(-Integer(1),Integer(2)).lift_to_precision(Integer(10))
16 + 16*17 + O(17^10)
>>> R(Integer(1),Integer(15)).lift_to_precision(Integer(10))
1 + O(17^15)
>>> R(Integer(1),Integer(15)).lift_to_precision(Integer(30))
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
>>> R(-Integer(1),Integer(2)).lift_to_precision().precision_absolute() == R.
↪precision_cap()
True

>>> R = Zp(Integer(5)); c = R(Integer(17),Integer(3)); c.lift_to_
↪precision(Integer(8))
2 + 3*5 + O(5^8)
>>> c.lift_to_precision().precision_relative() == R.precision_cap()
True
```

Fixed modulus elements don't raise errors:

```
sage: R = ZpFM(5); a = R(5); a.lift_to_precision(7)
5
sage: a.lift_to_precision(10000)
5
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5)); a = R(Integer(5)); a.lift_to_precision(Integer(7))
5
>>> a.lift_to_precision(Integer(10000))
5
```

residue(*absprec=1*, *field=None*, *check_prec=True*)

Reduce this element modulo p^{absprec}.

INPUT:

- `absprec` – `0` or `1`
- `field` – boolean (default: `None`); for precision 1, whether to return an element of the residue field or a residue ring. Currently unused.
- `check_prec` – boolean (default: `True`); whether to raise an error if this element has insufficient precision to determine the reduction. Errors are never raised for fixed-mod or floating-point types.

OUTPUT:

This element reduced modulo p^{absprec} as an element of the residue field or the null ring.

EXAMPLES:

```
sage: R.<a> = Zq(27, 4)
sage: (3 + 3*a).residue()
0
sage: (a + 1).residue()
a0 + 1
```

```
>>> from sage.all import *
>>> R = Zq(Integer(27), Integer(4), names=('a',)); (a,) = R._first_ngens(1)
>>> (Integer(3) + Integer(3)*a).residue()
0
>>> (a + Integer(1)).residue()
a0 + 1
```

teichmuller_expansion(*n=None*)

Return an iterator over coefficients $a_0, a_1, \ldots, a_n$ such that

- $a_i^q = a_i$, where q is the cardinality of the residue field,
- this element can be expressed as

$$\pi^v \cdot \sum_{i=0}^{\infty} a_i \pi^i$$

where v is the valuation of this element when the parent is a field, and $v = 0$ otherwise.

- if $a_i \neq 0$, the precision of a_i is i less than the precision of this element (relative in the case that the parent is a field, absolute otherwise)

Note

The coefficients will lie in the ring of integers of the maximal unramified subextension.

INPUT:

- `n` – integer (default: `None`); if given, returns the coefficient of π^n in the expansion

EXAMPLES:

For fields, the expansion starts at the valuation:

```
sage: R = Qp(5,5); list(R(70).teichmuller_expansion())
[4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5),
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4),
2 + 5 + 2*5^2 + O(5^3),
1 + O(5^2),
4 + O(5)]
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5),Integer(5)); list(R(Integer(70)).teichmuller_
↪expansion())
[4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5),
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4),
2 + 5 + 2*5^2 + O(5^3),
1 + O(5^2),
4 + O(5)]
```

But if you specify `n`, you get the coefficient of π^n:

```
sage: R(70).teichmuller_expansion(2)
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4)
```

```
>>> from sage.all import *
>>> R(Integer(70)).teichmuller_expansion(Integer(2))
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4)
```

unit_part()

Return the unit part of this element.

This is the p-adic element u in the same ring so that this element is $\pi^v u$, where π is a uniformizer and v is the valuation of this element.

EXAMPLES:

```
sage: R.<a> = Zq(125)
sage: (5*a).unit_part()
a + O(5^20)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1)
>>> (Integer(5)*a).unit_part()
a + O(5^20)
```

sage.rings.padics.padic_capped_relative_element.**unpickle_cre_v2**(*cls*, *parent*, *unit*, *ordp*, *relprec*)

Unpickles a capped relative element.

EXAMPLES:

```
sage: from sage.rings.padics.padic_capped_relative_element import unpickle_cre_v2
sage: R = Zp(5); a = R(85,6)
sage: b = unpickle_cre_v2(a.__class__, R, 17, 1, 5)
sage: a == b
True
sage: a.precision_relative() == b.precision_relative()
True
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_capped_relative_element import unpickle_cre_v2
>>> R = Zp(Integer(5)); a = R(Integer(85),Integer(6))
>>> b = unpickle_cre_v2(a.__class__, R, Integer(17), Integer(1), Integer(5))
>>> a == b
True
>>> a.precision_relative() == b.precision_relative()
True
```

sage.rings.padics.padic_capped_relative_element.**unpickle_pcre_v1**(*R*, *unit*, *ordp*, *relprec*)

Unpickles a capped relative element.

EXAMPLES:

```
sage: from sage.rings.padics.padic_capped_relative_element import unpickle_pcre_v1
sage: R = Zp(5)
sage: a = unpickle_pcre_v1(R, 17, 2, 5); a
2*5^2 + 3*5^3 + O(5^7)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_capped_relative_element import unpickle_pcre_v1
>>> R = Zp(Integer(5))
>>> a = unpickle_pcre_v1(R, Integer(17), Integer(2), Integer(5)); a
2*5^2 + 3*5^3 + O(5^7)
```

CHAPTER

FIFTEEN

p-ADIC CAPPED ABSOLUTE ELEMENTS

Elements of p-adic Rings with Absolute Precision Cap

AUTHORS:

- David Roe
- Genya Zaytman: documentation
- David Harvey: doctests

class sage.rings.padics.padic_capped_absolute_element.**CAElement**

Bases: *pAdicTemplateElement*

add_bigoh(*absprec*)

Return a new element with absolute precision decreased to `absprec`. The precision never increases.

INPUT:

- `absprec` – integer or infinity

OUTPUT: `self` with precision set to the minimum of `self`'s precision and `prec`

EXAMPLES:

```
sage: R = Zp(7,4,'capped-abs','series'); a = R(8); a.add_bigoh(1)
1 + O(7)

sage: k = ZpCA(3,5)
sage: a = k(41); a
2 + 3 + 3^2 + 3^3 + O(3^5)
sage: a.add_bigoh(7)
2 + 3 + 3^2 + 3^3 + O(3^5)
sage: a.add_bigoh(3)
2 + 3 + 3^2 + O(3^3)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'capped-abs','series'); a = R(Integer(8)); a.
↪add_bigoh(Integer(1))
1 + O(7)

>>> k = ZpCA(Integer(3),Integer(5))
>>> a = k(Integer(41)); a
2 + 3 + 3^2 + 3^3 + O(3^5)
>>> a.add_bigoh(Integer(7))
2 + 3 + 3^2 + 3^3 + O(3^5)
```

(continues on next page)

(continued from previous page)

```
>>> a.add_bigoh(Integer(3))
2 + 3 + 3^2 + O(3^3)
```

is_equal_to(*_right*, *absprec=None*)

Determine whether the inputs are equal modulo π^{absprec}.

INPUT:

- `right` – a p-adic element with the same parent
- `absprec` – integer, infinity, or `None`

EXAMPLES:

```
sage: R = ZpCA(2, 6)
sage: R(13).is_equal_to(R(13))
True
sage: R(13).is_equal_to(R(13+2^10))
True
sage: R(13).is_equal_to(R(17), 2)
True
sage: R(13).is_equal_to(R(17), 5)
False
sage: R(13).is_equal_to(R(13+2^10),absprec=10)
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(2), Integer(6))
>>> R(Integer(13)).is_equal_to(R(Integer(13)))
True
>>> R(Integer(13)).is_equal_to(R(Integer(13)+Integer(2)**Integer(10)))
True
>>> R(Integer(13)).is_equal_to(R(Integer(17)), Integer(2))
True
>>> R(Integer(13)).is_equal_to(R(Integer(17)), Integer(5))
False
>>> R(Integer(13)).is_equal_to(R(Integer(13)+Integer(2)**Integer(10)),
↪absprec=Integer(10))
Traceback (most recent call last):
...
PrecisionError: elements not known to enough precision
```

is_zero(*absprec=None*)

Determine whether this element is zero modulo π^{absprec}.

If `absprec is None`, returns `True` if this element is indistinguishable from zero.

INPUT:

- `absprec` – integer, infinity, or `None`

EXAMPLES:

```
sage: R = ZpCA(17, 6)
sage: R(0).is_zero()
True
sage: R(17^6).is_zero()
True
sage: R(17^2).is_zero(absprec=2)
True
sage: R(17^6).is_zero(absprec=10)
Traceback (most recent call last):
...
PrecisionError: not enough precision to determine if element is zero
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(17), Integer(6))
>>> R(Integer(0)).is_zero()
True
>>> R(Integer(17)**Integer(6)).is_zero()
True
>>> R(Integer(17)**Integer(2)).is_zero(absprec=Integer(2))
True
>>> R(Integer(17)**Integer(6)).is_zero(absprec=Integer(10))
Traceback (most recent call last):
...
PrecisionError: not enough precision to determine if element is zero
```

polynomial(*var='x'*)

Return a polynomial over the base ring that yields this element when evaluated at the generator of the parent.

INPUT:

- `var` – string; the variable name for the polynomial

EXAMPLES:

```
sage: R.<a> = ZqCA(5^3)
sage: a.polynomial()
(1 + O(5^20))*x + O(5^20)
sage: a.polynomial(var='y')
(1 + O(5^20))*y + O(5^20)
sage: (5*a^2 + R(25, 4)).polynomial()
(5 + O(5^4))*x^2 + O(5^4)*x + 5^2 + O(5^4)
```

```
>>> from sage.all import *
>>> R = ZqCA(Integer(5)**Integer(3), names=('a',)); (a,) = R._first_ngens(1)
>>> a.polynomial()
(1 + O(5^20))*x + O(5^20)
>>> a.polynomial(var='y')
(1 + O(5^20))*y + O(5^20)
>>> (Integer(5)*a**Integer(2) + R(Integer(25), Integer(4))).polynomial()
(5 + O(5^4))*x^2 + O(5^4)*x + 5^2 + O(5^4)
```

precision_absolute()

The absolute precision of this element.

This is the power of the maximal ideal modulo which this element is defined.

EXAMPLES:

```
sage: R = Zp(7,4,'capped-abs'); a = R(7); a.precision_absolute()
4
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'capped-abs'); a = R(Integer(7)); a.
↪precision_absolute()
4
```

precision_relative()

The relative precision of this element.

This is the power of the maximal ideal modulo which the unit part of this element is defined.

EXAMPLES:

```
sage: R = Zp(7,4,'capped-abs'); a = R(7); a.precision_relative()
3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'capped-abs'); a = R(Integer(7)); a.
↪precision_relative()
3
```

unit_part()

Return the unit part of this element.

EXAMPLES:

```
sage: R = Zp(17,4,'capped-abs', 'val-unit')
sage: a = R(18*17)
sage: a.unit_part()
18 + O(17^3)
sage: type(a)
<class 'sage.rings.padics.padic_capped_absolute_element.
↪pAdicCappedAbsoluteElement'>
sage: R(0).unit_part()
O(17^0)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(17),Integer(4),'capped-abs', 'val-unit')
>>> a = R(Integer(18)*Integer(17))
>>> a.unit_part()
18 + O(17^3)
>>> type(a)
<class 'sage.rings.padics.padic_capped_absolute_element.
↪pAdicCappedAbsoluteElement'>
>>> R(Integer(0)).unit_part()
O(17^0)
```

val_unit()

Return a 2-tuple, the first element set to the valuation of this element, and the second to the unit part of this element.

For a zero element, the unit part is `O(p^0)`.

EXAMPLES:

```
sage: R = ZpCA(5)
sage: a = R(75, 6); b = a - a
sage: a.val_unit()
(2, 3 + O(5^4))
sage: b.val_unit()
(6, O(5^0))
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5))
>>> a = R(Integer(75), Integer(6)); b = a - a
>>> a.val_unit()
(2, 3 + O(5^4))
>>> b.val_unit()
(6, O(5^0))
```

class sage.rings.padics.padic_capped_absolute_element.**ExpansionIter**

Bases: `object`

An iterator over a p-adic expansion.

This class should not be instantiated directly, but instead using `expansion`.

INPUT:

- `elt` – the p-adic element
- `prec` – the number of terms to be emitted
- `mode` – either `simple_mode`, `smallest_mode` or `teichmuller_mode`

EXAMPLES:

```
sage: E = Zp(5,4)(373).expansion()
sage: I = iter(E)  # indirect doctest
sage: type(I)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIter'>
```

```
>>> from sage.all import *
>>> E = Zp(Integer(5),Integer(4))(Integer(373)).expansion()
>>> I = iter(E)  # indirect doctest
>>> type(I)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIter'>
```

class sage.rings.padics.padic_capped_absolute_element.**ExpansionIterable**

Bases: `object`

An iterable storing a p-adic expansion of an element.

This class should not be instantiated directly, but instead using *`expansion()`*.

INPUT:

- `elt` – the p-adic element
- `prec` – the number of terms to be emitted

- `val_shift` – how many zeros to add at the beginning of the expansion, or the number of initial terms to truncate (if negative)
- `mode` – one of the following:
  - `'simple_mode'`
  - `'smallest_mode'`
  - `'teichmuller_mode'`

EXAMPLES:

```
sage: E = Zp(5,4)(373).expansion()  # indirect doctest
sage: type(E)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIterable'>
```

```
>>> from sage.all import *
>>> E = Zp(Integer(5),Integer(4))(Integer(373)).expansion()  # indirect doctest
>>> type(E)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIterable'>
```

class sage.rings.padics.padic_capped_absolute_element.**PowComputer_**

Bases: *PowComputer_base*

A PowComputer for a capped-absolute p-adic ring.

sage.rings.padics.padic_capped_absolute_element.**make_pAdicCappedAbsoluteElement**(*parent*, *x*, *absprec*)

Unpickles a capped absolute element.

EXAMPLES:

```
sage: from sage.rings.padics.padic_capped_absolute_element import make_
↪pAdicCappedAbsoluteElement
sage: R = ZpCA(5)
sage: a = make_pAdicCappedAbsoluteElement(R, 17*25, 5); a
2*5^2 + 3*5^3 + O(5^5)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_capped_absolute_element import make_
↪pAdicCappedAbsoluteElement
>>> R = ZpCA(Integer(5))
>>> a = make_pAdicCappedAbsoluteElement(R, Integer(17)*Integer(25), Integer(5)); a
2*5^2 + 3*5^3 + O(5^5)
```

class sage.rings.padics.padic_capped_absolute_element.**pAdicCappedAbsoluteElement**

Bases: *CAElement*

Construct new element with given parent and value.

INPUT:

- `x` – value to coerce into a capped absolute ring
- `absprec` – maximum number of digits of absolute precision
- `relprec` – maximum number of digits of relative precision

EXAMPLES:

```
sage: R = ZpCA(3, 5)
sage: R(2)
2 + O(3^5)
sage: R(2, absprec=2)
2 + O(3^2)
sage: R(3, relprec=2)
3 + O(3^3)
sage: R(Qp(3)(10))
1 + 3^2 + O(3^5)
sage: R(pari(6))
2*3 + O(3^5)
sage: R(pari(1/2))
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
sage: R(1/2)
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
sage: R(mod(-1, 3^7))
2 + 2*3 + 2*3^2 + 2*3^3 + 2*3^4 + O(3^5)
sage: R(mod(-1, 3^2))
2 + 2*3 + O(3^2)
sage: R(3 + O(3^2))
3 + O(3^2)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(3), Integer(5))
>>> R(Integer(2))
2 + O(3^5)
>>> R(Integer(2), absprec=Integer(2))
2 + O(3^2)
>>> R(Integer(3), relprec=Integer(2))
3 + O(3^3)
>>> R(Qp(Integer(3))(Integer(10)))
1 + 3^2 + O(3^5)
>>> R(pari(Integer(6)))
2*3 + O(3^5)
>>> R(pari(Integer(1)/Integer(2)))
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
>>> R(Integer(1)/Integer(2))
2 + 3 + 3^2 + 3^3 + 3^4 + O(3^5)
>>> R(mod(-Integer(1), Integer(3)**Integer(7)))
2 + 2*3 + 2*3^2 + 2*3^3 + 2*3^4 + O(3^5)
>>> R(mod(-Integer(1), Integer(3)**Integer(2)))
2 + 2*3 + O(3^2)
>>> R(Integer(3) + O(Integer(3)**Integer(2)))
3 + O(3^2)
```

lift()

EXAMPLES:

```
sage: R = ZpCA(3)
sage: R(10).lift()
10
sage: R(-1).lift()
3486784400
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(3))
>>> R(Integer(10)).lift()
10
>>> R(-Integer(1)).lift()
3486784400
```

multiplicative_order()

Return the minimum possible multiplicative order of this element.

OUTPUT:

The multiplicative order of `self`. This is the minimum multiplicative order of all elements of $\mathbf{Z}_p$ lifting `self` to infinite precision.

EXAMPLES:

```
sage: R = ZpCA(7, 6)
sage: R(1/3)
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + 4*7^5 + O(7^6)
sage: R(1/3).multiplicative_order()
+Infinity
sage: R(7).multiplicative_order()
+Infinity
sage: R(1).multiplicative_order()
1
sage: R(-1).multiplicative_order()
2
sage: R.teichmuller(3).multiplicative_order()
6
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(7), Integer(6))
>>> R(Integer(1)/Integer(3))
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + 4*7^5 + O(7^6)
>>> R(Integer(1)/Integer(3)).multiplicative_order()
+Infinity
>>> R(Integer(7)).multiplicative_order()
+Infinity
>>> R(Integer(1)).multiplicative_order()
1
>>> R(-Integer(1)).multiplicative_order()
2
>>> R.teichmuller(Integer(3)).multiplicative_order()
6
```

residue(*absprec=1*, *field=None*, *check_prec=True*)

Reduces `self` modulo p^{absprec}.

INPUT:

- `absprec` – nonnegative integer (default: 1)
- `field` – boolean (default: `None`); whether to return an element of GF(p) or Zmod(p)

- check_prec – boolean (default: True); whether to raise an error if this element has insufficient precision to determine the reduction

OUTPUT:

This element reduced modulo p^{absprec} as an element of $\mathbf{Z}/p^{\text{absprec}}\mathbf{Z}$

EXAMPLES:

```
sage: R = Zp(7,10,'capped-abs')
sage: a = R(8)
sage: a.residue(1)
1
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(10),'capped-abs')
>>> a = R(Integer(8))
>>> a.residue(Integer(1))
1
```

This is different from applying % p^n which returns an element in the same ring:

```
sage: b = a.residue(2); b
8
sage: b.parent()
Ring of integers modulo 49
sage: c = a % 7^2; c
1 + 7 + O(7^10)
sage: c.parent()
7-adic Ring with capped absolute precision 10
```

```
>>> from sage.all import *
>>> b = a.residue(Integer(2)); b
8
>>> b.parent()
Ring of integers modulo 49
>>> c = a % Integer(7)**Integer(2); c
1 + 7 + O(7^10)
>>> c.parent()
7-adic Ring with capped absolute precision 10
```

Note that reduction of c dropped to the precision of the unit part of 7^2, see _mod_:

```
sage: R(7^2).unit_part()
1 + O(7^8)
```

```
>>> from sage.all import *
>>> R(Integer(7)**Integer(2)).unit_part()
1 + O(7^8)
```

See also

mod

class sage.rings.padics.padic_capped_absolute_element.**pAdicCoercion_CA_frac_field**

Bases: RingHomomorphism

The canonical inclusion of Zq into its fraction field.

EXAMPLES:

```
sage: R.<a> = ZqCA(27, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R); f
Ring morphism:
  From: 3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqCA(Integer(27), implementation='FLINT', names=('a',)); (a,) = R._first_
↪ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R); f
Ring morphism:
  From: 3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
```

is_injective()

Return whether this map is injective.

EXAMPLES:

```
sage: R.<a> = ZqCA(9, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f.is_injective()
True
```

```
>>> from sage.all import *
>>> R = ZqCA(Integer(9), implementation='FLINT', names=('a',)); (a,) = R._
↪first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f.is_injective()
True
```

is_surjective()

Return whether this map is surjective.

EXAMPLES:

```
sage: R.<a> = ZqCA(9, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f.is_surjective()
False
```

```
>>> from sage.all import *
>>> R = ZqCA(Integer(9), implementation='FLINT', names=('a',)); (a,) = R._
↪first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f.is_surjective()
False
```

section()

Return a map back to the ring that converts elements of nonnegative valuation.

EXAMPLES:

```
sage: R.<a> = ZqCA(27, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f(K.gen())
a + O(3^20)
sage: f.section()
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqCA(Integer(27), implementation='FLINT', names=('a',)); (a,) = R._
↪first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f(K.gen())
a + O(3^20)
>>> f.section()
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

class sage.rings.padics.padic_capped_absolute_element.**pAdicCoercion_ZZ_CA**

Bases: RingHomomorphism

The canonical inclusion from the ring of integers to a capped absolute ring.

EXAMPLES:

```
sage: f = ZpCA(5).coerce_map_from(ZZ); f
Ring morphism:
  From: Integer Ring
  To:   5-adic Ring with capped absolute precision 20
```

```
>>> from sage.all import *
>>> f = ZpCA(Integer(5)).coerce_map_from(ZZ); f
Ring morphism:
  From: Integer Ring
  To:   5-adic Ring with capped absolute precision 20
```

section()

Return a map back to the ring of integers that approximates an element by an integer.

EXAMPLES:

```
sage: f = ZpCA(5).coerce_map_from(ZZ).section()
sage: f(ZpCA(5)(-1)) - 5^20
-1
```

```
>>> from sage.all import *
>>> f = ZpCA(Integer(5)).coerce_map_from(ZZ).section()
>>> f(ZpCA(Integer(5))(-Integer(1))) - Integer(5)**Integer(20)
-1
```

class sage.rings.padics.padic_capped_absolute_element.**pAdicConvert_CA_ZZ**

Bases: RingMap

The map from a capped absolute ring back to the ring of integers that returns the smallest nonnegative integer approximation to its input which is accurate up to the precision.

Raises a ValueError if the input is not in the closure of the image of the ring of integers.

EXAMPLES:

```
sage: f = ZpCA(5).coerce_map_from(ZZ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Ring with capped absolute precision 20
  To:   Integer Ring
```

```
>>> from sage.all import *
>>> f = ZpCA(Integer(5)).coerce_map_from(ZZ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Ring with capped absolute precision 20
  To:   Integer Ring
```

class sage.rings.padics.padic_capped_absolute_element.**pAdicConvert_CA_frac_field**

Bases: Morphism

The section of the inclusion from $\mathbf{Z}_q$ to its fraction field.

EXAMPLES:

```
sage: R.<a> = ZqCA(27, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = R.convert_map_from(K); f
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqCA(Integer(27), implementation='FLINT', names=('a',)); (a,) = R._first_
↪ngens(1)
>>> K = R.fraction_field()
>>> f = R.convert_map_from(K); f
Generic morphism:
```

(continues on next page)

(continued from previous page)

```
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

class sage.rings.padics.padic_capped_absolute_element.**pAdicConvert_QQ_CA**

Bases: Morphism

The inclusion map from the rationals to a capped absolute ring that is defined on all elements with nonnegative p-adic valuation.

EXAMPLES:

```
sage: f = ZpCA(5).convert_map_from(QQ); f
Generic morphism:
  From: Rational Field
  To:   5-adic Ring with capped absolute precision 20
```

```
>>> from sage.all import *
>>> f = ZpCA(Integer(5)).convert_map_from(QQ); f
Generic morphism:
  From: Rational Field
  To:   5-adic Ring with capped absolute precision 20
```

class sage.rings.padics.padic_capped_absolute_element.**pAdicTemplateElement**

Bases: *pAdicGenericElement*

A class for common functionality among the p-adic template classes.

INPUT:

- `parent` – a local ring or field
- `x` – data defining this element. Various types are supported, including ints, Integers, Rationals, PARI p-adics, integers mod p^k and other Sage p-adics.
- `absprec` – a cap on the absolute precision of this element
- `relprec` – a cap on the relative precision of this element

EXAMPLES:

```
sage: Zp(17)(17^3, 8, 4)
17^3 + O(17^7)
```

```
>>> from sage.all import *
>>> Zp(Integer(17))(Integer(17)**Integer(3), Integer(8), Integer(4))
17^3 + O(17^7)
```

expansion(*n=None*, *lift_mode='simple'*, *start_val=None*)

Return the coefficients in a π-adic expansion. If this is a field element, start at $\pi^{\text{valuation}}$, if a ring element at π^0.

For each lift mode, this function returns a list of a_i so that this element can be expressed as

$$\pi^v \cdot \sum_{i=0}^{\infty} a_i \pi^i,$$

where v is the valuation of this element when the parent is a field, and $v = 0$ otherwise.

Different lift modes affect the choice of a_i. When `lift_mode` is `'simple'`, the resulting a_i will be non-negative: if the residue field is $\mathbf{F}_p$ then they will be integers with $0 \leq a_i < p$; otherwise they will be a list of integers in the same range giving the coefficients of a polynomial in the indeterminant representing the maximal unramified subextension.

Choosing `lift_mode` as `'smallest'` is similar to `'simple'`, but uses a balanced representation $-p/2 < a_i \leq p/2$.

Finally, setting `lift_mode = 'teichmuller'` will yield Teichmuller representatives for the a_i: $a_i^q = a_i$. In this case the a_i will lie in the ring of integers of the maximal unramified subextension of the parent of this element.

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion. Can also accept a slice (see *slice()*).
- `lift_mode` – `'simple'`, `'smallest'` or `'teichmuller'` (default: `'simple'`)
- `start_val` – start at this valuation rather than the default (0 or the valuation of this element)

OUTPUT:

- If `n` is `None`, an iterable giving a π-adic expansion of this element. For base elements the contents will be integers if `lift_mode` is `'simple'` or `'smallest'`, and elements of `self.parent()` if `lift_mode` is `'teichmuller'`.
- If `n` is an integer, the coefficient of π^n in the π-adic expansion of this element.

Note

Use slice operators to get a particular range.

EXAMPLES:

```
sage: R = Zp(7,6); a = R(12837162817); a
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
sage: E = a.expansion(); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
sage: list(E)
[3, 4, 4, 0, 4, 0]
sage: sum([c * 7^i for i, c in enumerate(E)]) == a
True
sage: E = a.expansion(lift_mode='smallest'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (balanced)
sage: list(E)
[3, -3, -2, 1, -3, 1]
sage: sum([c * 7^i for i, c in enumerate(E)]) == a
True
sage: E = a.expansion(lift_mode='teichmuller'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (teichmuller)
sage: list(E)
[3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + O(7^6),
0,
5 + 2*7 + 3*7^3 + O(7^4),
1 + O(7^3),
3 + 4*7 + O(7^2),
```

(continues on next page)

(continued from previous page)

```
5 + O(7)]
sage: sum(c * 7^i for i, c in enumerate(E))
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(6)); a = R(Integer(12837162817)); a
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
>>> E = a.expansion(); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
>>> list(E)
[3, 4, 4, 0, 4, 0]
>>> sum([c * Integer(7)**i for i, c in enumerate(E)]) == a
True
>>> E = a.expansion(lift_mode='smallest'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (balanced)
>>> list(E)
[3, -3, -2, 1, -3, 1]
>>> sum([c * Integer(7)**i for i, c in enumerate(E)]) == a
True
>>> E = a.expansion(lift_mode='teichmuller'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (teichmuller)
>>> list(E)
[3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + O(7^6),
0,
5 + 2*7 + 3*7^3 + O(7^4),
1 + O(7^3),
3 + 4*7 + O(7^2),
5 + O(7)]
>>> sum(c * Integer(7)**i for i, c in enumerate(E))
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
```

If the element has positive valuation then the list will start with some zeros:

```
sage: a = R(7^3 * 17)
sage: E = a.expansion(); E
7-adic expansion of 3*7^3 + 2*7^4 + O(7^9)
sage: list(E)
[0, 0, 0, 3, 2, 0, 0, 0, 0]
```

```
>>> from sage.all import *
>>> a = R(Integer(7)**Integer(3) * Integer(17))
>>> E = a.expansion(); E
7-adic expansion of 3*7^3 + 2*7^4 + O(7^9)
>>> list(E)
[0, 0, 0, 3, 2, 0, 0, 0, 0]
```

The expansion of 0 is truncated:

```
sage: E = R(0, 7).expansion(); E
7-adic expansion of O(7^7)
sage: len(E)
0
```

(continues on next page)

(continued from previous page)

```
sage: list(E)
[]
```

```
>>> from sage.all import *
>>> E = R(Integer(0), Integer(7)).expansion(); E
7-adic expansion of O(7^7)
>>> len(E)
0
>>> list(E)
[]
```

In fields, on the other hand, the expansion starts at the valuation:

```
sage: R = Qp(7,4); a = R(6*7+7**2); E = a.expansion(); E
7-adic expansion of 6*7 + 7^2 + O(7^5)
sage: list(E)
[6, 1, 0, 0]
sage: list(a.expansion(lift_mode='smallest'))
[-1, 2, 0, 0]
sage: list(a.expansion(lift_mode='teichmuller'))
[6 + 6*7 + 6*7^2 + 6*7^3 + O(7^4),
2 + 4*7 + 6*7^2 + O(7^3),
3 + 4*7 + O(7^2),
3 + O(7)]
```

```
>>> from sage.all import *
>>> R = Qp(Integer(7),Integer(4)); a =␣
↪R(Integer(6)*Integer(7)+Integer(7)**Integer(2)); E = a.expansion(); E
7-adic expansion of 6*7 + 7^2 + O(7^5)
>>> list(E)
[6, 1, 0, 0]
>>> list(a.expansion(lift_mode='smallest'))
[-1, 2, 0, 0]
>>> list(a.expansion(lift_mode='teichmuller'))
[6 + 6*7 + 6*7^2 + 6*7^3 + O(7^4),
2 + 4*7 + 6*7^2 + O(7^3),
3 + 4*7 + O(7^2),
3 + O(7)]
```

You can ask for a specific entry in the expansion:

```
sage: a.expansion(1)
6
sage: a.expansion(1, lift_mode='smallest')
-1
sage: a.expansion(2, lift_mode='teichmuller')
2 + 4*7 + 6*7^2 + O(7^3)
```

```
>>> from sage.all import *
>>> a.expansion(Integer(1))
6
>>> a.expansion(Integer(1), lift_mode='smallest')
```

(continues on next page)

(continued from previous page)

```
-1
>>> a.expansion(Integer(2), lift_mode='teichmuller')
2 + 4*7 + 6*7^2 + O(7^3)
```

lift_to_precision(*absprec=None*)

Return another element of the same parent with absolute precision at least `absprec`, congruent to this p-adic element modulo the precision of this element.

INPUT:

- `absprec` – integer or `None` (default: `None`); the absolute precision of the result. If `None`, lifts to the maximum precision allowed.

Note

If setting `absprec` that high would violate the precision cap, raises a precision error. Note that the new digits will not necessarily be zero.

EXAMPLES:

```
sage: R = ZpCA(17)
sage: R(-1,2).lift_to_precision(10)
16 + 16*17 + O(17^10)
sage: R(1,15).lift_to_precision(10)
1 + O(17^15)
sage: R(1,15).lift_to_precision(30)
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
sage: R(-1,2).lift_to_precision().precision_absolute() == R.precision_cap()
True

sage: R = Zp(5); c = R(17,3); c.lift_to_precision(8)
2 + 3*5 + O(5^8)
sage: c.lift_to_precision().precision_relative() == R.precision_cap()
True
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(17))
>>> R(-Integer(1),Integer(2)).lift_to_precision(Integer(10))
16 + 16*17 + O(17^10)
>>> R(Integer(1),Integer(15)).lift_to_precision(Integer(10))
1 + O(17^15)
>>> R(Integer(1),Integer(15)).lift_to_precision(Integer(30))
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
>>> R(-Integer(1),Integer(2)).lift_to_precision().precision_absolute() == R.
↪precision_cap()
True

>>> R = Zp(Integer(5)); c = R(Integer(17),Integer(3)); c.lift_to_
```

(continues on next page)

(continued from previous page)

```
↪precision(Integer(8))
2 + 3*5 + O(5^8)
>>> c.lift_to_precision().precision_relative() == R.precision_cap()
True
```

Fixed modulus elements don't raise errors:

```
sage: R = ZpFM(5); a = R(5); a.lift_to_precision(7)
5
sage: a.lift_to_precision(10000)
5
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5)); a = R(Integer(5)); a.lift_to_precision(Integer(7))
5
>>> a.lift_to_precision(Integer(10000))
5
```

residue(*absprec=1*, *field=None*, *check_prec=True*)

Reduce this element modulo p^{absprec}.

INPUT:

- `absprec` – `0` or `1`
- `field` – boolean (default: `None`); for precision 1, whether to return an element of the residue field or a residue ring. Currently unused.
- `check_prec` – boolean (default: `True`); whether to raise an error if this element has insufficient precision to determine the reduction. Errors are never raised for fixed-mod or floating-point types.

OUTPUT:

This element reduced modulo p^{absprec} as an element of the residue field or the null ring.

EXAMPLES:

```
sage: R.<a> = Zq(27, 4)
sage: (3 + 3*a).residue()
0
sage: (a + 1).residue()
a0 + 1
```

```
>>> from sage.all import *
>>> R = Zq(Integer(27), Integer(4), names=('a',)); (a,) = R._first_ngens(1)
>>> (Integer(3) + Integer(3)*a).residue()
0
>>> (a + Integer(1)).residue()
a0 + 1
```

teichmuller_expansion(*n=None*)

Return an iterator over coefficients $a_0, a_1, \ldots, a_n$ such that

- $a_i^q = a_i$, where q is the cardinality of the residue field,
- this element can be expressed as

$$\pi^v \cdot \sum_{i=0}^{\infty} a_i \pi^i$$

where v is the valuation of this element when the parent is a field, and $v = 0$ otherwise.

- if $a_i \neq 0$, the precision of a_i is i less than the precision of this element (relative in the case that the parent is a field, absolute otherwise)

Note

The coefficients will lie in the ring of integers of the maximal unramified subextension.

INPUT:

- `n` – integer (default: `None`); if given, returns the coefficient of π^n in the expansion

EXAMPLES:

For fields, the expansion starts at the valuation:

```
sage: R = Qp(5,5); list(R(70).teichmuller_expansion())
[4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5),
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4),
2 + 5 + 2*5^2 + O(5^3),
1 + O(5^2),
4 + O(5)]
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5),Integer(5)); list(R(Integer(70)).teichmuller_
↪expansion())
[4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5),
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4),
2 + 5 + 2*5^2 + O(5^3),
1 + O(5^2),
4 + O(5)]
```

But if you specify `n`, you get the coefficient of π^n:

```
sage: R(70).teichmuller_expansion(2)
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4)
```

```
>>> from sage.all import *
>>> R(Integer(70)).teichmuller_expansion(Integer(2))
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4)
```

unit_part()

Return the unit part of this element.

This is the p-adic element u in the same ring so that this element is $\pi^v u$, where π is a uniformizer and v is the valuation of this element.

EXAMPLES:

```
sage: R.<a> = Zq(125)
sage: (5*a).unit_part()
a + O(5^20)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1)
>>> (Integer(5)*a).unit_part()
a + O(5^20)
```

sage.rings.padics.padic_capped_absolute_element.**unpickle_cae_v2**(*cls*, *parent*, *value*, *absprec*)

Unpickle capped absolute elements.

INPUT:

- `cls` – the class of the capped absolute element
- `parent` – a p-adic ring
- `value` – a Python object wrapping a celement, of the kind accepted by the cunpickle function
- `absprec` – a Python int or Sage integer

EXAMPLES:

```
sage: from sage.rings.padics.padic_capped_absolute_element import unpickle_cae_v2,
↪ pAdicCappedAbsoluteElement
sage: R = ZpCA(5,8)
sage: a = unpickle_cae_v2(pAdicCappedAbsoluteElement, R, 42, int(6)); a
2 + 3*5 + 5^2 + O(5^6)
sage: a.parent() is R
True
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_capped_absolute_element import unpickle_cae_v2,␣
↪pAdicCappedAbsoluteElement
>>> R = ZpCA(Integer(5),Integer(8))
>>> a = unpickle_cae_v2(pAdicCappedAbsoluteElement, R, Integer(42),␣
↪int(Integer(6))); a
2 + 3*5 + 5^2 + O(5^6)
>>> a.parent() is R
True
```

CHAPTER

SIXTEEN

p-ADIC FIXED-MOD ELEMENT

Elements of p-adic Rings with Fixed Modulus

AUTHORS:

- David Roe
- Genya Zaytman: documentation
- David Harvey: doctests

class sage.rings.padics.padic_fixed_mod_element.**ExpansionIter**

Bases: object

An iterator over a p-adic expansion.

This class should not be instantiated directly, but instead using `expansion`.

INPUT:

- `elt` – the p-adic element
- `prec` – the number of terms to be emitted
- `mode` – either `simple_mode`, `smallest_mode` or `teichmuller_mode`

EXAMPLES:

```
sage: E = Zp(5,4)(373).expansion()
sage: I = iter(E)  # indirect doctest
sage: type(I)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIter'>
```

```
>>> from sage.all import *
>>> E = Zp(Integer(5),Integer(4))(Integer(373)).expansion()
>>> I = iter(E)  # indirect doctest
>>> type(I)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIter'>
```

class sage.rings.padics.padic_fixed_mod_element.**ExpansionIterable**

Bases: object

An iterable storing a p-adic expansion of an element.

This class should not be instantiated directly, but instead using *expansion()*.

INPUT:

- `elt` – the p-adic element

- `prec` – the number of terms to be emitted
- `val_shift` – how many zeros to add at the beginning of the expansion, or the number of initial terms to truncate (if negative)
- `mode` – one of the following:
  - `'simple_mode'`
  - `'smallest_mode'`
  - `'teichmuller_mode'`

EXAMPLES:

```
sage: E = Zp(5,4)(373).expansion()  # indirect doctest
sage: type(E)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIterable'>
```

```
>>> from sage.all import *
>>> E = Zp(Integer(5),Integer(4))(Integer(373)).expansion()  # indirect doctest
>>> type(E)
<class 'sage.rings.padics.padic_capped_relative_element.ExpansionIterable'>
```

class sage.rings.padics.padic_fixed_mod_element.**FMElement**

Bases: *pAdicTemplateElement*

add_bigoh(*absprec*)

Return a new element truncated modulo π^{absprec}.

INPUT:

- `absprec` – integer or infinity

OUTPUT: a new element truncated modulo π^{absprec}

EXAMPLES:

```
sage: R = Zp(7,4,'fixed-mod','series'); a = R(8); a.add_bigoh(1)
1
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'fixed-mod','series'); a = R(Integer(8)); a.
↪add_bigoh(Integer(1))
1
```

is_equal_to(*_right*, *absprec=None*)

Return whether this element is equal to `right` modulo p^{absprec}.

If `absprec` is `None`, returns if `self == 0`.

INPUT:

- `right` – a p-adic element with the same parent
- `absprec` – positive integer or `None` (default: `None`)

EXAMPLES:

```
sage: R = ZpFM(2, 6)
sage: R(13).is_equal_to(R(13))
True
sage: R(13).is_equal_to(R(13+2^10))
True
sage: R(13).is_equal_to(R(17), 2)
True
sage: R(13).is_equal_to(R(17), 5)
False
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(2), Integer(6))
>>> R(Integer(13)).is_equal_to(R(Integer(13)))
True
>>> R(Integer(13)).is_equal_to(R(Integer(13)+Integer(2)**Integer(10)))
True
>>> R(Integer(13)).is_equal_to(R(Integer(17)), Integer(2))
True
>>> R(Integer(13)).is_equal_to(R(Integer(17)), Integer(5))
False
```

is_zero(*absprec=None*)

Return whether `self` is zero modulo π^{absprec}.

INPUT:

- `absprec` – integer

EXAMPLES:

```
sage: R = ZpFM(17, 6)
sage: R(0).is_zero()
True
sage: R(17^6).is_zero()
True
sage: R(17^2).is_zero(absprec=2)
True
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(17), Integer(6))
>>> R(Integer(0)).is_zero()
True
>>> R(Integer(17)**Integer(6)).is_zero()
True
>>> R(Integer(17)**Integer(2)).is_zero(absprec=Integer(2))
True
```

polynomial(*var='x'*)

Return a polynomial over the base ring that yields this element when evaluated at the generator of the parent.

INPUT:

- `var` – string, the variable name for the polynomial

EXAMPLES:

```
sage: R.<a> = ZqFM(5^3)
sage: a.polynomial()
x
sage: a.polynomial(var='y')
y
sage: (5*a^2 + 25).polynomial()
5*x^2 + 5^2
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(5)**Integer(3), names=('a',)); (a,) = R._first_ngens(1)
>>> a.polynomial()
x
>>> a.polynomial(var='y')
y
>>> (Integer(5)*a**Integer(2) + Integer(25)).polynomial()
5*x^2 + 5^2
```

precision_absolute()

The absolute precision of this element.

EXAMPLES:

```
sage: R = Zp(7,4,'fixed-mod'); a = R(7); a.precision_absolute()
4
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'fixed-mod'); a = R(Integer(7)); a.precision_
↪absolute()
4
```

precision_relative()

The relative precision of this element.

EXAMPLES:

```
sage: R = Zp(7,4,'fixed-mod'); a = R(7); a.precision_relative()
3
sage: a = R(0); a.precision_relative()
0
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'fixed-mod'); a = R(Integer(7)); a.precision_
↪relative()
3
>>> a = R(Integer(0)); a.precision_relative()
0
```

unit_part()

Return the unit part of `self`.

If the valuation of `self` is positive, then the high digits of the result will be zero.

EXAMPLES:

```
sage: R = Zp(17, 4, 'fixed-mod')
sage: R(5).unit_part()
5
sage: R(18*17).unit_part()
1 + 17
sage: R(0).unit_part()
0
sage: type(R(5).unit_part())
<class 'sage.rings.padics.padic_fixed_mod_element.pAdicFixedModElement'>
sage: R = ZpFM(5, 5); a = R(75); a.unit_part()
3
```

```
>>> from sage.all import *
>>> R = Zp(Integer(17), Integer(4), 'fixed-mod')
>>> R(Integer(5)).unit_part()
5
>>> R(Integer(18)*Integer(17)).unit_part()
1 + 17
>>> R(Integer(0)).unit_part()
0
>>> type(R(Integer(5)).unit_part())
<class 'sage.rings.padics.padic_fixed_mod_element.pAdicFixedModElement'>
>>> R = ZpFM(Integer(5), Integer(5)); a = R(Integer(75)); a.unit_part()
3
```

val_unit()

Return a 2-tuple, the first element set to the valuation of `self`, and the second to the unit part of `self`.

If `self == 0`, then the unit part is `O(p^self.parent().precision_cap())`.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: a = R(75); b = a - a
sage: a.val_unit()
(2, 3)
sage: b.val_unit()
(5, 0)
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> a = R(Integer(75)); b = a - a
>>> a.val_unit()
(2, 3)
>>> b.val_unit()
(5, 0)
```

class sage.rings.padics.padic_fixed_mod_element.**PowComputer_**

Bases: *PowComputer_base*

A PowComputer for a fixed-modulus p-adic ring.

sage.rings.padics.padic_fixed_mod_element.**make_pAdicFixedModElement**(*parent*, *value*)

Unpickles a fixed modulus element.

EXAMPLES:

```
sage: from sage.rings.padics.padic_fixed_mod_element import make_
↪pAdicFixedModElement
sage: R = ZpFM(5)
sage: a = make_pAdicFixedModElement(R, 17*25); a
2*5^2 + 3*5^3
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_fixed_mod_element import make_
↪pAdicFixedModElement
>>> R = ZpFM(Integer(5))
>>> a = make_pAdicFixedModElement(R, Integer(17)*Integer(25)); a
2*5^2 + 3*5^3
```

class sage.rings.padics.padic_fixed_mod_element.**pAdicCoercion_FM_frac_field**

Bases: RingHomomorphism

The canonical inclusion of $\mathbf{Z}_q$ into its fraction field.

EXAMPLES:

```
sage: R.<a> = ZqFM(27, implementation='FLINT')
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R); f
Ring morphism:
  From: 3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(27), implementation='FLINT', names=('a',)); (a,) = R._first_
↪ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R); f
Ring morphism:
  From: 3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
```

is_injective()

Return whether this map is injective.

EXAMPLES:

```
sage: R.<a> = ZqFM(9)
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f.is_injective()
True
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(9), names=('a',)); (a,) = R._first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f.is_injective()
True
```

is_surjective()

Return whether this map is surjective.

EXAMPLES:

```
sage: R.<a> = ZqFM(9)
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f.is_surjective()
False
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(9), names=('a',)); (a,) = R._first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f.is_surjective()
False
```

section()

Return a map back to the ring that converts elements of nonnegative valuation.

EXAMPLES:

```
sage: R.<a> = ZqFM(27)
sage: K = R.fraction_field()
sage: f = K.coerce_map_from(R)
sage: f.section()(K.gen())
a
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(27), names=('a',)); (a,) = R._first_ngens(1)
>>> K = R.fraction_field()
>>> f = K.coerce_map_from(R)
>>> f.section()(K.gen())
a
```

class sage.rings.padics.padic_fixed_mod_element.**pAdicCoercion_ZZ_FM**

Bases: RingHomomorphism

The canonical inclusion from **Z** to a fixed modulus ring.

EXAMPLES:

```
sage: f = ZpFM(5).coerce_map_from(ZZ); f
Ring morphism:
  From: Integer Ring
  To:   5-adic Ring of fixed modulus 5^20
```

```
>>> from sage.all import *
>>> f = ZpFM(Integer(5)).coerce_map_from(ZZ); f
Ring morphism:
  From: Integer Ring
  To:   5-adic Ring of fixed modulus 5^20
```

section()

Return a map back to $\mathbf{Z}$ that approximates an element of this p-adic ring by an integer.

EXAMPLES:

```
sage: f = ZpFM(5).coerce_map_from(ZZ).section()
sage: f(ZpFM(5)(-1)) - 5^20
-1
```

```
>>> from sage.all import *
>>> f = ZpFM(Integer(5)).coerce_map_from(ZZ).section()
>>> f(ZpFM(Integer(5))(-Integer(1))) - Integer(5)**Integer(20)
-1
```

class sage.rings.padics.padic_fixed_mod_element.**pAdicConvert_FM_ZZ**

Bases: RingMap

The map from a fixed modulus ring back to $\mathbf{Z}$ that returns the smallest nonnegative integer approximation to its input which is accurate up to the precision.

If the input is not in the closure of the image of $\mathbf{Z}$, raises a ValueError.

EXAMPLES:

```
sage: f = ZpFM(5).coerce_map_from(ZZ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Ring of fixed modulus 5^20
  To:   Integer Ring
```

```
>>> from sage.all import *
>>> f = ZpFM(Integer(5)).coerce_map_from(ZZ).section(); f
Set-theoretic ring morphism:
  From: 5-adic Ring of fixed modulus 5^20
  To:   Integer Ring
```

class sage.rings.padics.padic_fixed_mod_element.**pAdicConvert_FM_frac_field**

Bases: Morphism

The section of the inclusion from $\mathbf{Z}_q$ to its fraction field.

EXAMPLES:

```
sage: R.<a> = ZqFM(27)
sage: K = R.fraction_field()
sage: f = R.convert_map_from(K); f
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(27), names=('a',)); (a,) = R._first_ngens(1)
>>> K = R.fraction_field()
>>> f = R.convert_map_from(K); f
Generic morphism:
  From: 3-adic Unramified Extension Field in a defined by x^3 + 2*x + 1
  To:   3-adic Unramified Extension Ring in a defined by x^3 + 2*x + 1
```

class sage.rings.padics.padic_fixed_mod_element.**pAdicConvert_QQ_FM**

Bases: Morphism

The inclusion map from **Q** to a fixed modulus ring that is defined on all elements with nonnegative p-adic valuation.

EXAMPLES:

```
sage: f = ZpFM(5).convert_map_from(QQ); f
Generic morphism:
  From: Rational Field
  To:   5-adic Ring of fixed modulus 5^20
```

```
>>> from sage.all import *
>>> f = ZpFM(Integer(5)).convert_map_from(QQ); f
Generic morphism:
  From: Rational Field
  To:   5-adic Ring of fixed modulus 5^20
```

class sage.rings.padics.padic_fixed_mod_element.**pAdicFixedModElement**

Bases: *FMElement*

INPUT:

- `parent` – a `pAdicRingFixedMod` object
- `x` – input data to be converted into the parent
- `absprec` – ignored; for compatibility with other p-adic rings
- `relprec` – ignored; for compatibility with other p-adic rings

Note

The following types are currently supported for x:

- Integers
- Rationals – denominator must be relatively prime to p
- FixedMod p-adics
- Elements of `IntegerModRing(p^k)` for `k` less than or equal to the modulus

The following types should be supported eventually:

- Finite precision p-adics
- Lazy p-adics
- Elements of local extensions of THIS p-adic ring that actually lie in $\mathbf{Z}_p$

EXAMPLES:

```
sage: R = Zp(5, 20, 'fixed-mod', 'terse')
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), Integer(20), 'fixed-mod', 'terse')
```

Construct from integers:

```
sage: R(3)
3
sage: R(75)
75
sage: R(0)
0

sage: R(-1)
95367431640624
sage: R(-5)
95367431640620
```

```
>>> from sage.all import *
>>> R(Integer(3))
3
>>> R(Integer(75))
75
>>> R(Integer(0))
0

>>> R(-Integer(1))
95367431640624
>>> R(-Integer(5))
95367431640620
```

Construct from rationals:

```
sage: R(1/2)
47683715820313
sage: R(-7875/874)
9493096742250
sage: R(15/425)
Traceback (most recent call last):
...
ValueError: p divides denominator
```

```
>>> from sage.all import *
>>> R(Integer(1)/Integer(2))
47683715820313
>>> R(-Integer(7875)/Integer(874))
9493096742250
>>> R(Integer(15)/Integer(425))
Traceback (most recent call last):
...
ValueError: p divides denominator
```

Construct from IntegerMod:

```
sage: R(Integers(125)(3))
3
sage: R(Integers(5)(3))
3
sage: R(Integers(5^30)(3))
```

(continues on next page)

(continued from previous page)

```
3
sage: R(Integers(5^30)(1+5^23))
1
sage: R(Integers(49)(3))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 49

sage: R(Integers(48)(3))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 48
```

```
>>> from sage.all import *
>>> R(Integers(Integer(125))(Integer(3)))
3
>>> R(Integers(Integer(5))(Integer(3)))
3
>>> R(Integers(Integer(5)**Integer(30))(Integer(3)))
3
>>> R(Integers(Integer(5)**Integer(30))(Integer(1)+Integer(5)**Integer(23)))
1
>>> R(Integers(Integer(49))(Integer(3)))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 49

>>> R(Integers(Integer(48))(Integer(3)))
Traceback (most recent call last):
...
TypeError: p does not divide modulus 48
```

Some other conversions:

```
sage: R(R(5))
5
```

```
>>> from sage.all import *
>>> R(R(Integer(5)))
5
```

Todo

doctests for converting from other types of p-adic rings

lift()

Return an integer congruent to `self` modulo the precision.

> **Warning**
>
> Since fixed modulus elements don't track their precision, the result may not be correct modulo $i^{\text{prec}_{\text{cap}}}$ if the element was defined by constructions that lost precision.

EXAMPLES:

```
sage: R = Zp(7,4,'fixed-mod'); a = R(8); a.lift()
8
sage: type(a.lift())
<class 'sage.rings.integer.Integer'>
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'fixed-mod'); a = R(Integer(8)); a.lift()
8
>>> type(a.lift())
<class 'sage.rings.integer.Integer'>
```

multiplicative_order()

Return the minimum possible multiplicative order of `self`.

OUTPUT:

an integer – the multiplicative order of this element. This is the minimum multiplicative order of all elements of $\mathbf{Z}_p$ lifting this element to infinite precision.

EXAMPLES:

```
sage: R = ZpFM(7, 6)
sage: R(1/3)
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + 4*7^5
sage: R(1/3).multiplicative_order()
+Infinity
sage: R(7).multiplicative_order()
+Infinity
sage: R(1).multiplicative_order()
1
sage: R(-1).multiplicative_order()
2
sage: R.teichmuller(3).multiplicative_order()
6
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(7), Integer(6))
>>> R(Integer(1)/Integer(3))
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + 4*7^5
>>> R(Integer(1)/Integer(3)).multiplicative_order()
+Infinity
>>> R(Integer(7)).multiplicative_order()
+Infinity
>>> R(Integer(1)).multiplicative_order()
1
>>> R(-Integer(1)).multiplicative_order()
```

(continues on next page)

(continued from previous page)

```
2
>>> R.teichmuller(Integer(3)).multiplicative_order()
6
```

residue(*absprec=1*, *field=None*, *check_prec=False*)

Reduce `self` modulo p^{absprec}.

INPUT:

- `absprec` – integer (default: 1)
- `field` – boolean (default: `None`); whether to return an element of GF(p) or Zmod(p)
- `check_prec` – boolean (default: `False`); no effect (for compatibility with other types)

OUTPUT:

This element reduced modulo p^{absprec} as an element of $\mathbf{Z}/p^{\text{absprec}}\mathbf{Z}$.

EXAMPLES:

```
sage: R = Zp(7,4,'fixed-mod')
sage: a = R(8)
sage: a.residue(1)
1
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'fixed-mod')
>>> a = R(Integer(8))
>>> a.residue(Integer(1))
1
```

This is different from applying `% p^n` which returns an element in the same ring:

```
sage: b = a.residue(2); b
8
sage: b.parent()
Ring of integers modulo 49
sage: c = a % 7^2; c
1 + 7
sage: c.parent()
7-adic Ring of fixed modulus 7^4
```

```
>>> from sage.all import *
>>> b = a.residue(Integer(2)); b
8
>>> b.parent()
Ring of integers modulo 49
>>> c = a % Integer(7)**Integer(2); c
1 + 7
>>> c.parent()
7-adic Ring of fixed modulus 7^4
```

See also

mod

class sage.rings.padics.padic_fixed_mod_element.**pAdicTemplateElement**

Bases: *pAdicGenericElement*

A class for common functionality among the p-adic template classes.

INPUT:

- `parent` – a local ring or field
- `x` – data defining this element. Various types are supported, including ints, Integers, Rationals, PARI p-adics, integers mod p^k and other Sage p-adics.
- `absprec` – a cap on the absolute precision of this element
- `relprec` – a cap on the relative precision of this element

EXAMPLES:

```
sage: Zp(17)(17^3, 8, 4)
17^3 + O(17^7)
```

```
>>> from sage.all import *
>>> Zp(Integer(17))(Integer(17)**Integer(3), Integer(8), Integer(4))
17^3 + O(17^7)
```

expansion(*n=None*, *lift_mode='simple'*, *start_val=None*)

Return the coefficients in a π-adic expansion. If this is a field element, start at $\pi^{\text{valuation}}$, if a ring element at π^0.

For each lift mode, this function returns a list of a_i so that this element can be expressed as

$$\pi^v \cdot \sum_{i=0}^{\infty} a_i \pi^i,$$

where v is the valuation of this element when the parent is a field, and $v = 0$ otherwise.

Different lift modes affect the choice of a_i. When `lift_mode` is `'simple'`, the resulting a_i will be non-negative: if the residue field is $\mathbf{F}_p$ then they will be integers with $0 \leq a_i < p$; otherwise they will be a list of integers in the same range giving the coefficients of a polynomial in the indeterminant representing the maximal unramified subextension.

Choosing `lift_mode` as `'smallest'` is similar to `'simple'`, but uses a balanced representation $-p/2 < a_i \leq p/2$.

Finally, setting `lift_mode = 'teichmuller'` will yield Teichmuller representatives for the a_i: $a_i^q = a_i$. In this case the a_i will lie in the ring of integers of the maximal unramified subextension of the parent of this element.

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion. Can also accept a slice (see *slice()*).
- `lift_mode` – `'simple'`, `'smallest'` or `'teichmuller'` (default: `'simple'`)
- `start_val` – start at this valuation rather than the default (0 or the valuation of this element)

OUTPUT:

- If `n` is `None`, an iterable giving a π-adic expansion of this element. For base elements the contents will be integers if `lift_mode` is `'simple'` or `'smallest'`, and elements of `self.parent()` if `lift_mode` is `'teichmuller'`.
- If `n` is an integer, the coefficient of π^n in the π-adic expansion of this element.

Note

Use slice operators to get a particular range.

EXAMPLES:

```
sage: R = Zp(7,6); a = R(12837162817); a
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
sage: E = a.expansion(); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
sage: list(E)
[3, 4, 4, 0, 4, 0]
sage: sum([c * 7^i for i, c in enumerate(E)]) == a
True
sage: E = a.expansion(lift_mode='smallest'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (balanced)
sage: list(E)
[3, -3, -2, 1, -3, 1]
sage: sum([c * 7^i for i, c in enumerate(E)]) == a
True
sage: E = a.expansion(lift_mode='teichmuller'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (teichmuller)
sage: list(E)
[3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + O(7^6),
0,
5 + 2*7 + 3*7^3 + O(7^4),
1 + O(7^3),
3 + 4*7 + O(7^2),
5 + O(7)]
sage: sum(c * 7^i for i, c in enumerate(E))
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(6)); a = R(Integer(12837162817)); a
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
>>> E = a.expansion(); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
>>> list(E)
[3, 4, 4, 0, 4, 0]
>>> sum([c * Integer(7)**i for i, c in enumerate(E)]) == a
True
>>> E = a.expansion(lift_mode='smallest'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (balanced)
>>> list(E)
[3, -3, -2, 1, -3, 1]
```

(continues on next page)

(continued from previous page)

```
>>> sum([c * Integer(7)**i for i, c in enumerate(E)]) == a
True
>>> E = a.expansion(lift_mode='teichmuller'); E
7-adic expansion of 3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6) (teichmuller)
>>> list(E)
[3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + O(7^6),
0,
5 + 2*7 + 3*7^3 + O(7^4),
1 + O(7^3),
3 + 4*7 + O(7^2),
5 + O(7)]
>>> sum(c * Integer(7)**i for i, c in enumerate(E))
3 + 4*7 + 4*7^2 + 4*7^4 + O(7^6)
```

If the element has positive valuation then the list will start with some zeros:

```
sage: a = R(7^3 * 17)
sage: E = a.expansion(); E
7-adic expansion of 3*7^3 + 2*7^4 + O(7^9)
sage: list(E)
[0, 0, 0, 3, 2, 0, 0, 0, 0]
```

```
>>> from sage.all import *
>>> a = R(Integer(7)**Integer(3) * Integer(17))
>>> E = a.expansion(); E
7-adic expansion of 3*7^3 + 2*7^4 + O(7^9)
>>> list(E)
[0, 0, 0, 3, 2, 0, 0, 0, 0]
```

The expansion of 0 is truncated:

```
sage: E = R(0, 7).expansion(); E
7-adic expansion of O(7^7)
sage: len(E)
0
sage: list(E)
[]
```

```
>>> from sage.all import *
>>> E = R(Integer(0), Integer(7)).expansion(); E
7-adic expansion of O(7^7)
>>> len(E)
0
>>> list(E)
[]
```

In fields, on the other hand, the expansion starts at the valuation:

```
sage: R = Qp(7,4); a = R(6*7+7**2); E = a.expansion(); E
7-adic expansion of 6*7 + 7^2 + O(7^5)
sage: list(E)
[6, 1, 0, 0]
```

(continues on next page)

(continued from previous page)

```
sage: list(a.expansion(lift_mode='smallest'))
[-1, 2, 0, 0]
sage: list(a.expansion(lift_mode='teichmuller'))
[6 + 6*7 + 6*7^2 + 6*7^3 + O(7^4),
2 + 4*7 + 6*7^2 + O(7^3),
3 + 4*7 + O(7^2),
3 + O(7)]
```

```
>>> from sage.all import *
>>> R = Qp(Integer(7),Integer(4)); a =␣
↪R(Integer(6)*Integer(7)+Integer(7)**Integer(2)); E = a.expansion(); E
7-adic expansion of 6*7 + 7^2 + O(7^5)
>>> list(E)
[6, 1, 0, 0]
>>> list(a.expansion(lift_mode='smallest'))
[-1, 2, 0, 0]
>>> list(a.expansion(lift_mode='teichmuller'))
[6 + 6*7 + 6*7^2 + 6*7^3 + O(7^4),
2 + 4*7 + 6*7^2 + O(7^3),
3 + 4*7 + O(7^2),
3 + O(7)]
```

You can ask for a specific entry in the expansion:

```
sage: a.expansion(1)
6
sage: a.expansion(1, lift_mode='smallest')
-1
sage: a.expansion(2, lift_mode='teichmuller')
2 + 4*7 + 6*7^2 + O(7^3)
```

```
>>> from sage.all import *
>>> a.expansion(Integer(1))
6
>>> a.expansion(Integer(1), lift_mode='smallest')
-1
>>> a.expansion(Integer(2), lift_mode='teichmuller')
2 + 4*7 + 6*7^2 + O(7^3)
```

lift_to_precision(*absprec=None*)

Return another element of the same parent with absolute precision at least `absprec`, congruent to this p-adic element modulo the precision of this element.

INPUT:

- `absprec` – integer or `None` (default: `None`); the absolute precision of the result. If `None`, lifts to the maximum precision allowed.

Note

If setting `absprec` that high would violate the precision cap, raises a precision error. Note that the new digits will not necessarily be zero.

EXAMPLES:

```
sage: R = ZpCA(17)
sage: R(-1,2).lift_to_precision(10)
16 + 16*17 + O(17^10)
sage: R(1,15).lift_to_precision(10)
1 + O(17^15)
sage: R(1,15).lift_to_precision(30)
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
sage: R(-1,2).lift_to_precision().precision_absolute() == R.precision_cap()
True

sage: R = Zp(5); c = R(17,3); c.lift_to_precision(8)
2 + 3*5 + O(5^8)
sage: c.lift_to_precision().precision_relative() == R.precision_cap()
True
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(17))
>>> R(-Integer(1),Integer(2)).lift_to_precision(Integer(10))
16 + 16*17 + O(17^10)
>>> R(Integer(1),Integer(15)).lift_to_precision(Integer(10))
1 + O(17^15)
>>> R(Integer(1),Integer(15)).lift_to_precision(Integer(30))
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
>>> R(-Integer(1),Integer(2)).lift_to_precision().precision_absolute() == R.
↪precision_cap()
True

>>> R = Zp(Integer(5)); c = R(Integer(17),Integer(3)); c.lift_to_
↪precision(Integer(8))
2 + 3*5 + O(5^8)
>>> c.lift_to_precision().precision_relative() == R.precision_cap()
True
```

Fixed modulus elements don't raise errors:

```
sage: R = ZpFM(5); a = R(5); a.lift_to_precision(7)
5
sage: a.lift_to_precision(10000)
5
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5)); a = R(Integer(5)); a.lift_to_precision(Integer(7))
5
>>> a.lift_to_precision(Integer(10000))
5
```

residue(*absprec=1*, *field=None*, *check_prec=True*)

Reduce this element modulo p^{absprec}.

INPUT:

- `absprec` – 0 or 1
- `field` – boolean (default: `None`); for precision 1, whether to return an element of the residue field or a residue ring. Currently unused.
- `check_prec` – boolean (default: `True`); whether to raise an error if this element has insufficient precision to determine the reduction. Errors are never raised for fixed-mod or floating-point types.

OUTPUT:

This element reduced modulo p^{absprec} as an element of the residue field or the null ring.

EXAMPLES:

```
sage: R.<a> = Zq(27, 4)
sage: (3 + 3*a).residue()
0
sage: (a + 1).residue()
a0 + 1
```

```
>>> from sage.all import *
>>> R = Zq(Integer(27), Integer(4), names=('a',)); (a,) = R._first_ngens(1)
>>> (Integer(3) + Integer(3)*a).residue()
0
>>> (a + Integer(1)).residue()
a0 + 1
```

teichmuller_expansion(*n=None*)

Return an iterator over coefficients $a_0, a_1, \ldots, a_n$ such that

- $a_i^q = a_i$, where q is the cardinality of the residue field,
- this element can be expressed as

$$\pi^v \cdot \sum_{i=0}^{\infty} a_i \pi^i$$

where v is the valuation of this element when the parent is a field, and $v = 0$ otherwise.

- if $a_i \neq 0$, the precision of a_i is i less than the precision of this element (relative in the case that the parent is a field, absolute otherwise)

Note

The coefficients will lie in the ring of integers of the maximal unramified subextension.

INPUT:

- `n` – integer (default: `None`); if given, returns the coefficient of π^n in the expansion

EXAMPLES:

For fields, the expansion starts at the valuation:

```
sage: R = Qp(5,5); list(R(70).teichmuller_expansion())
[4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5),
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4),
```

(continues on next page)

(continued from previous page)

```
2 + 5 + 2*5^2 + O(5^3),
1 + O(5^2),
4 + O(5)]
```

```
>>> from sage.all import *
>>> R = Qp(Integer(5),Integer(5)); list(R(Integer(70)).teichmuller_
↪expansion())
[4 + 4*5 + 4*5^2 + 4*5^3 + 4*5^4 + O(5^5),
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4),
2 + 5 + 2*5^2 + O(5^3),
1 + O(5^2),
4 + O(5)]
```

But if you specify `n`, you get the coefficient of π^n:

```
sage: R(70).teichmuller_expansion(2)
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4)
```

```
>>> from sage.all import *
>>> R(Integer(70)).teichmuller_expansion(Integer(2))
3 + 3*5 + 2*5^2 + 3*5^3 + O(5^4)
```

unit_part()

Return the unit part of this element.

This is the p-adic element u in the same ring so that this element is $\pi^v u$, where π is a uniformizer and v is the valuation of this element.

EXAMPLES:

```
sage: R.<a> = Zq(125)
sage: (5*a).unit_part()
a + O(5^20)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(125), names=('a',)); (a,) = R._first_ngens(1)
>>> (Integer(5)*a).unit_part()
a + O(5^20)
```

sage.rings.padics.padic_fixed_mod_element.**unpickle_fme_v2**(*cls*, *parent*, *value*)

Unpickles a fixed-mod element.

EXAMPLES:

```
sage: from sage.rings.padics.padic_fixed_mod_element import pAdicFixedModElement,␣
↪unpickle_fme_v2
sage: R = ZpFM(5)
sage: a = unpickle_fme_v2(pAdicFixedModElement, R, 17*25); a
2*5^2 + 3*5^3
sage: a.parent() is R
True
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_fixed_mod_element import pAdicFixedModElement,␣
↪unpickle_fme_v2
>>> R = ZpFM(Integer(5))
>>> a = unpickle_fme_v2(pAdicFixedModElement, R, Integer(17)*Integer(25)); a
2*5^2 + 3*5^3
>>> a.parent() is R
True
```

CHAPTER

SEVENTEEN

p-ADIC EXTENSION ELEMENT

A common superclass for all elements of extension rings and field of $\mathbf{Z}_p$ and $\mathbf{Q}_p$.

AUTHORS:

- David Roe (2007): initial version
- Julian Rueth (2012-10-18): added residue

class sage.rings.padics.padic_ext_element.**pAdicExtElement**

Bases: *pAdicGenericElement*

frobenius(*arithmetic=True*)

Return the image of this element under the Frobenius automorphism applied to its parent.

INPUT:

- `arithmetic` – whether to apply the arithmetic Frobenius (acting by raising to the p-th power on the residue field). If `False` is provided, the image of geometric Frobenius (raising to the $(1/p)$-th power on the residue field) will be returned instead.

EXAMPLES:

```
sage: R.<a> = Zq(5^4,3)
sage: a.frobenius()
(a^3 + a^2 + 3*a) + (3*a + 1)*5 + (2*a^3 + 2*a^2 + 2*a)*5^2 + O(5^3)
sage: f = R.defining_polynomial()
sage: f(a)
O(5^3)
sage: f(a.frobenius())
O(5^3)
sage: for i in range(4): a = a.frobenius()
sage: a
a + O(5^3)

sage: K.<a> = Qq(7^3,4)
sage: b = (a+1)/7
sage: c = b.frobenius(); c
(3*a^2 + 5*a + 1)*7^-1 + (6*a^2 + 6*a + 6) + (4*a^2 + 3*a + 4)*7 + (6*a^2 + a␣
↪+ 6)*7^2 + O(7^3)
sage: c.frobenius().frobenius()
(a + 1)*7^-1 + O(7^3)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(5)**Integer(4),Integer(3), names=('a',)); (a,) = R._first_
```

(continues on next page)

(continued from previous page)

```
↪ngens(1)
>>> a.frobenius()
(a^3 + a^2 + 3*a) + (3*a + 1)*5 + (2*a^3 + 2*a^2 + 2*a)*5^2 + O(5^3)
>>> f = R.defining_polynomial()
>>> f(a)
O(5^3)
>>> f(a.frobenius())
O(5^3)
>>> for i in range(Integer(4)): a = a.frobenius()
>>> a
a + O(5^3)

>>> K = Qq(Integer(7)**Integer(3),Integer(4), names=('a',)); (a,) = K._first_
↪ngens(1)
>>> b = (a+Integer(1))/Integer(7)
>>> c = b.frobenius(); c
(3*a^2 + 5*a + 1)*7^-1 + (6*a^2 + 6*a + 6) + (4*a^2 + 3*a + 4)*7 + (6*a^2 + a␣
↪+ 6)*7^2 + O(7^3)
>>> c.frobenius().frobenius()
(a + 1)*7^-1 + O(7^3)
```

An error will be raised if the parent of `self` is a ramified extension:

```
sage: x = polygen(ZZ, 'x')
sage: K.<a> = Qp(5).extension(x^2 - 5)
sage: a.frobenius()
Traceback (most recent call last):
...
NotImplementedError: Frobenius automorphism only implemented for unramified␣
↪extensions
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> K = Qp(Integer(5)).extension(x**Integer(2) - Integer(5), names=('a',));␣
↪(a,) = K._first_ngens(1)
>>> a.frobenius()
Traceback (most recent call last):
...
NotImplementedError: Frobenius automorphism only implemented for unramified␣
↪extensions
```

residue(*absprec=1*, *field=None*, *check_prec=True*)

Reduces this element modulo π^{absprec}.

INPUT:

- `absprec` – nonnegative integer (default: 1)
- `field` – boolean (default: `None`); for precision 1, whether to return an element of the residue field or a residue ring. Currently unused.
- `check_prec` – boolean (default: `True`); whether to raise an error if this element has insufficient precision to determine the reduction. Errors are never raised for fixed-mod or floating-point types.

OUTPUT: this element reduced modulo π^{absprec}

If `absprec` is zero, then as an element of $\mathbf{Z}/(1)$.

If `absprec` is one, then as an element of the residue field.

Note

Only implemented for `absprec` less than or equal to one.

AUTHORS:

- Julian Rueth (2012-10-18): initial version

EXAMPLES:

Unramified case:

```
sage: R = ZpCA(3,5)
sage: S.<a> = R[]
sage: W.<a> = R.extension(a^2 + 9*a + 1)
sage: (a + 1).residue(1)
a0 + 1
sage: a.residue(2)
Traceback (most recent call last):
...
NotImplementedError: reduction modulo p^n with n>1
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(3),Integer(5))
>>> S = R['a']; (a,) = S._first_ngens(1)
>>> W = R.extension(a**Integer(2) + Integer(9)*a + Integer(1), names=('a',));␣
↪(a,) = W._first_ngens(1)
>>> (a + Integer(1)).residue(Integer(1))
a0 + 1
>>> a.residue(Integer(2))
Traceback (most recent call last):
...
NotImplementedError: reduction modulo p^n with n>1
```

Eisenstein case:

```
sage: R = ZpCA(3,5)
sage: S.<a> = R[]
sage: W.<a> = R.extension(a^2 + 9*a + 3)
sage: (a + 1).residue(1)
1
sage: a.residue(2)
Traceback (most recent call last):
...
NotImplementedError: residue() not implemented in extensions for absprec␣
↪larger than one
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(3),Integer(5))
>>> S = R['a']; (a,) = S._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> W = R.extension(a**Integer(2) + Integer(9)*a + Integer(3), names=('a',));␣
↪(a,) = W._first_ngens(1)
>>> (a + Integer(1)).residue(Integer(1))
1
>>> a.residue(Integer(2))
Traceback (most recent call last):
...
NotImplementedError: residue() not implemented in extensions for absprec␣
↪larger than one
```

CHAPTER

EIGHTEEN

p-ADIC ZZ_PX ELEMENT

A common superclass implementing features shared by all elements that use NTL's ZZ_pX as the fundamental data type.

AUTHORS:

- David Roe

class sage.rings.padics.padic_ZZ_pX_element.**pAdicZZpXElement**

Bases: *pAdicExtElement*

Initialization.

EXAMPLES:

```
sage: A = Zp(next_prime(50000),10)
sage: S.<x> = A[]
sage: B.<t> = A.ext(x^2 + next_prime(50000))  # indirect doctest
```

```
>>> from sage.all import *
>>> A = Zp(next_prime(Integer(50000)),Integer(10))
>>> S = A['x']; (x,) = S._first_ngens(1)
>>> B = A.ext(x**Integer(2) + next_prime(Integer(50000)), names=('t',)); (t,) = B.
↪_first_ngens(1)# indirect doctest
```

norm(*base=None*)

Return the absolute or relative norm of this element.

> **Note**
>
> This is not the p-adic absolute value. This is a field theoretic norm down to a ground ring. If you want the p-adic absolute value, use the abs() function instead.

If base is given then base must be a subfield of the parent L of self, in which case the norm is the relative norm from L to base.

In all other cases, the norm is the absolute norm down to $\mathbf{Q}_p$ or $\mathbf{Z}_p$.

EXAMPLES:

```
sage: R = ZpCR(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
```

(continues on next page)

(continued from previous page)

```
sage: ((1+2*w)^5).norm()
1 + 5^2 + O(5^5)
sage: ((1+2*w)).norm()^5
1 + 5^2 + O(5^5)
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> ((Integer(1)+Integer(2)*w)**Integer(5)).norm()
1 + 5^2 + O(5^5)
>>> ((Integer(1)+Integer(2)*w)).norm()**Integer(5)
1 + 5^2 + O(5^5)
```

trace(*base=None*)

Return the absolute or relative trace of this element.

If `base` is given then `base` must be a subfield of the parent L of `self`, in which case the norm is the relative norm from L to `base`.

In all other cases, the norm is the absolute norm down to $\mathbf{Q}_p$ or $\mathbf{Z}_p$.

EXAMPLES:

```
sage: R = ZpCR(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = (2+3*w)^7
sage: b = (6+w^3)^5
sage: a.trace()
3*5 + 2*5^2 + 3*5^3 + 2*5^4 + O(5^5)
sage: a.trace() + b.trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
sage: (a+b).trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = (Integer(2)+Integer(3)*w)**Integer(7)
>>> b = (Integer(6)+w**Integer(3))**Integer(5)
>>> a.trace()
3*5 + 2*5^2 + 3*5^3 + 2*5^4 + O(5^5)
>>> a.trace() + b.trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
>>> (a+b).trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
```

CHAPTER

NINETEEN

p-ADIC ZZ_PX CR ELEMENT

This file implements elements of Eisenstein and unramified extensions of $\mathbf{Z}_p$ and $\mathbf{Q}_p$ with capped relative precision.

For the parent class see *sage.rings.padics.padic_extension_leaves*.

The underlying implementation is through NTL's ZZ_pX class. Each element contains the following data:

- ordp – long; a power of the uniformizer to scale the unit by. For unramified extensions this uniformizer is p, for Eisenstein extensions it is not. A value equal to the maximum value of a long indicates that the element is an exact zero.
- relprec – long; a signed integer giving the precision to which this element is defined. For nonzero relprec, the absolute value gives the power of the uniformizer modulo which the unit is defined. A positive value indicates that the element is normalized (ie unit is actually a unit: in the case of Eisenstein extensions the constant term is not divisible by p, in the case of unramified extensions that there is at least one coefficient that is not divisible by p). A negative value indicates that the element may or may not be normalized. A zero value indicates that the element is zero to some precision. If so, ordp gives the absolute precision of the element. If ordp is greater than maxordp, then the element is an exact zero.
- unit – ZZ_pX_c; an ntl ZZ_pX storing the unit part The variable x is the uniformizer in the case of Eisenstein extensions. If the element is not normalized, the unit may or may not actually be a unit. This ZZ_pX is created with global ntl modulus determined by the absolute value of relprec. If relprec is 0, unit **is not initialized**, or destructed if normalized and found to be zero. Otherwise, let r be relprec and e be the ramification index over $\mathbf{Q}_p$ or $\mathbf{Z}_p$. Then the modulus of unit is given by $p^{ceil(r/e)}$. Note that all kinds of problems arise if you try to mix moduli. ZZ_pX_conv_modulus gives a semi-safe way to convert between different moduli without having to pass through ZZX.
- prime_pow (some subclass of *PowComputer_ZZ_pX*) – a class, identical among all elements with the same parent, holding common data.
  - prime_pow.deg – the degree of the extension
  - prime_pow.e – the ramification index
  - prime_pow.f – the inertia degree
  - prime_pow.prec_cap – the unramified precision cap. For Eisenstein extensions this is the smallest power of p that is zero.
  - prime_pow.ram_prec_cap – the ramified precision cap. For Eisenstein extensions this will be the smallest power of x that is indistinguishable from zero.
  - prime_pow.pow_ZZ_tmp, prime_pow.pow_mpz_t_tmp``, prime_pow.pow_Integer – functions for accessing powers of p. The first two return pointers. See *sage.rings.padics.pow_computer_ext* for examples and important warnings.

- `prime_pow.get_context`, `prime_pow.get_context_capdiv`, `prime_pow.get_top_context` – obtain an `ntl_ZZ_pContext_class` corresponding to p^n. The capdiv version divides by `prime_pow.e` as appropriate. `top_context` corresponds to $p^{\text{prec_cap}}$.
- `prime_pow.restore_context`, `prime_pow.restore_context_capdiv`, `prime_pow.restore_top_context` – restores the given context.
- `prime_pow.get_modulus`, `get_modulus_capdiv`, `get_top_modulus` – returns a `ZZ_pX_Modulus_c*` pointing to a polynomial modulus defined modulo p^n (appropriately divided by `prime_pow.e` in the capdiv case).

EXAMPLES:

An Eisenstein extension:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f); W
5-adic Eisenstein Extension Ring in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: z = (1+w)^5; z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16 +␣
↪4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
sage: y = z >> 1; y
w^4 + w^5 + 2*w^6 + 4*w^7 + 3*w^9 + w^11 + 4*w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^19 + w^20 + 4*w^23 + O(w^24)
sage: y.valuation()
4
sage: y.precision_relative()
20
sage: y.precision_absolute()
24
sage: z - (y << 1)
1 + O(w^25)
sage: (1/w)^12+w
w^-12 + w + O(w^13)
sage: (1/w).parent()
5-adic Eisenstein Extension Field in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1); W
5-adic Eisenstein Extension Ring in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
>>> z = (Integer(1)+w)**Integer(5); z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16 +␣
↪4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
>>> y = z >> Integer(1); y
w^4 + w^5 + 2*w^6 + 4*w^7 + 3*w^9 + w^11 + 4*w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^19 + w^20 + 4*w^23 + O(w^24)
>>> y.valuation()
4
>>> y.precision_relative()
```

(continues on next page)

(continued from previous page)

```
20
>>> y.precision_absolute()
24
>>> z - (y << Integer(1))
1 + O(w^25)
>>> (Integer(1)/w)**Integer(12)+w
w^-12 + w + O(w^13)
>>> (Integer(1)/w).parent()
5-adic Eisenstein Extension Field in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

Unramified extensions:

```
sage: g = x^3 + 3*x + 3
sage: A.<a> = R.ext(g)
sage: z = (1+a)^5; z
(2*a^2 + 4*a) + (3*a^2 + 3*a + 1)*5 + (4*a^2 + 3*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^3 +␣
↪(4*a^2 + 4*a + 4)*5^4 + O(5^5)
sage: z - 1 - 5*a - 10*a^2 - 10*a^3 - 5*a^4 - a^5
O(5^5)
sage: y = z >> 1; y
(3*a^2 + 3*a + 1) + (4*a^2 + 3*a + 4)*5 + (4*a^2 + 4*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^
↪3 + O(5^4)
sage: 1/a
(3*a^2 + 4) + (a^2 + 4)*5 + (3*a^2 + 4)*5^2 + (a^2 + 4)*5^3 + (3*a^2 + 4)*5^4 + O(5^5)
sage: FFp = R.residue_field()
sage: R(FFp(3))
3 + O(5)
sage: QQq.<zz> = Qq(25,4)
sage: QQq(FFp(3))
3 + O(5)
sage: FFq = QQq.residue_field(); QQq(FFq(3))
3 + O(5)
sage: zz0 = FFq.gen(); QQq(zz0^2)
(zz + 3) + O(5)
```

```
>>> from sage.all import *
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)
>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> z = (Integer(1)+a)**Integer(5); z
(2*a^2 + 4*a) + (3*a^2 + 3*a + 1)*5 + (4*a^2 + 3*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^3 +␣
↪(4*a^2 + 4*a + 4)*5^4 + O(5^5)
>>> z - Integer(1) - Integer(5)*a - Integer(10)*a**Integer(2) -␣
↪Integer(10)*a**Integer(3) - Integer(5)*a**Integer(4) - a**Integer(5)
O(5^5)
>>> y = z >> Integer(1); y
(3*a^2 + 3*a + 1) + (4*a^2 + 3*a + 4)*5 + (4*a^2 + 4*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^
↪3 + O(5^4)
>>> Integer(1)/a
(3*a^2 + 4) + (a^2 + 4)*5 + (3*a^2 + 4)*5^2 + (a^2 + 4)*5^3 + (3*a^2 + 4)*5^4 + O(5^5)
>>> FFp = R.residue_field()
>>> R(FFp(Integer(3)))
3 + O(5)
```

(continues on next page)

(continued from previous page)

```
>>> QQq = Qq(Integer(25),Integer(4), names=('zz',)); (zz,) = QQq._first_ngens(1)
>>> QQq(FFp(Integer(3)))
3 + O(5)
>>> FFq = QQq.residue_field(); QQq(FFq(Integer(3)))
3 + O(5)
>>> zz0 = FFq.gen(); QQq(zz0**Integer(2))
(zz + 3) + O(5)
```

Different printing modes:

```
sage: R = Zp(5, print_mode='digits'); S.<x> = R[]; f = x^5 + 75*x^3 - 15*x^2 + 125*x -
↪5; W.<w> = R.ext(f)
sage: z = (1+w)^5; repr(z)
'...411040311321031044222131124200011101120110200202330321433201121440323201314400140
↪444441030421100001'
sage: R = Zp(5, print_mode='bars'); S.<x> = R[]; g = x^3 + 3*x + 3; A.<a> = R.ext(g)
sage: z = (1+a)^5; repr(z)
'...[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 3, 4]|[1, 3, 3]|[0, 4, 2]'
sage: R = Zp(5, print_mode='terse'); S.<x> = R[]; f = x^5 + 75*x^3 - 15*x^2 + 125*x -
↪5; W.<w> = R.ext(f)
sage: z = (1+w)^5; z
6 + 95367431640505*w + 25*w^2 + 95367431640560*w^3 + 5*w^4 + O(w^100)
sage: R = Zp(5, print_mode='val-unit'); S.<x> = R[]; f = x^5 + 75*x^3 - 15*x^2 +␣
↪125*x -5; W.<w> = R.ext(f)
sage: y = (1+w)^5 - 1; y
w^5 * (2090041 + 19073486126901*w + 1258902*w^2 + 674*w^3 + 16785*w^4) + O(w^100)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5), print_mode='digits'); S = R['x']; (x,) = S._first_ngens(1); f␣
↪= x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); repr(z)
'...411040311321031044222131124200011101120110200202330321433201121440323201314400140
↪444441030421100001'
>>> R = Zp(Integer(5), print_mode='bars'); S = R['x']; (x,) = S._first_ngens(1); g =␣
↪x**Integer(3) + Integer(3)*x + Integer(3); A = R.ext(g, names=('a',)); (a,) = A._
↪first_ngens(1)
>>> z = (Integer(1)+a)**Integer(5); repr(z)
'...[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 3, 4]|[1, 3, 3]|[0, 4, 2]'
>>> R = Zp(Integer(5), print_mode='terse'); S = R['x']; (x,) = S._first_ngens(1); f =␣
↪x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); z
6 + 95367431640505*w + 25*w^2 + 95367431640560*w^3 + 5*w^4 + O(w^100)
>>> R = Zp(Integer(5), print_mode='val-unit'); S = R['x']; (x,) = S._first_ngens(1);␣
↪f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> y = (Integer(1)+w)**Integer(5) - Integer(1); y
```

(continues on next page)

(continued from previous page)

```
w^5 * (2090041 + 19073486126901*w + 1258902*w^2 + 674*w^3 + 16785*w^4) + O(w^100)
```

You can get at the underlying ntl unit:

```
sage: z._ntl_rep()
[6 95367431640505 25 95367431640560 5]
sage: y._ntl_rep()
[2090041 19073486126901 1258902 674 16785]
sage: y._ntl_rep_abs()
([5 95367431640505 25 95367431640560 5], 0)
```

```
>>> from sage.all import *
>>> z._ntl_rep()
[6 95367431640505 25 95367431640560 5]
>>> y._ntl_rep()
[2090041 19073486126901 1258902 674 16785]
>>> y._ntl_rep_abs()
([5 95367431640505 25 95367431640560 5], 0)
```

Note

If you get an error `internal error: can't grow this _ntl_gbigint`, it indicates that moduli are being mixed inappropriately somewhere.

For example, when calling a function with a `ZZ_pX_c` as an argument, it copies. If the modulus is not set to the modulus of the `ZZ_pX_c`, you can get errors.

AUTHORS:

- David Roe (2008-01-01): initial version
- Robert Harron (2011-09): fixes/enhancements
- Julian Rueth (2014-05-09): enable caching through `_cache_key`

`sage.rings.padics.padic_ZZ_pX_CR_element.`**`make_ZZpXCRElement`**(*parent*, *unit*, *ordp*, *relprec*, *version*)

Unpickling.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: y = W(775, 19); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
sage: loads(dumps(y))  # indirect doctest
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)

sage: from sage.rings.padics.padic_ZZ_pX_CR_element import make_ZZpXCRElement
sage: make_ZZpXCRElement(W, y._ntl_rep(), 3, 9, 0)
w^3 + 4*w^5 + 2*w^7 + w^8 + 2*w^9 + 4*w^10 + w^11 + O(w^12)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> y = W(Integer(775), Integer(19)); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
>>> loads(dumps(y))  # indirect doctest
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)

>>> from sage.rings.padics.padic_ZZ_pX_CR_element import make_ZZpXCRElement
>>> make_ZZpXCRElement(W, y._ntl_rep(), Integer(3), Integer(9), Integer(0))
w^3 + 4*w^5 + 2*w^7 + w^8 + 2*w^9 + 4*w^10 + w^11 + O(w^12)
```

class sage.rings.padics.padic_ZZ_pX_CR_element.**pAdicZZpXCRElement**

Bases: *pAdicZZpXElement*

Create an element of a capped relative precision, unramified or Eisenstein extension of $\mathbf{Z}_p$ or $\mathbf{Q}_p$.

INPUT:

- `parent` – either an `EisensteinRingCappedRelative` or `UnramifiedRingCappedRelative`
- `x` – integer; rational, p-adic element, polynomial, list, integer_mod, pari int/frac/poly_t/pol_mod, an `ntl_ZZ_pX`, an `ntl_ZZ`, an `ntl_ZZ_p`, an `ntl_ZZX`, or something convertible into parent.residue_field()
- `absprec` – an upper bound on the absolute precision of the element created
- `relprec` – an upper bound on the relative precision of the element created
- `empty` – whether to return after initializing to zero (without setting the valuation)

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: z = (1+w)^5; z  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
sage: W(pari('3 + O(5^3)'))
3 + O(w^15)
sage: W(R(3,3))
3 + O(w^15)
sage: W.<w> = R.ext(x^625 + 915*x^17 - 95)
sage: W(3)
3 + O(w^3125)
sage: W(w, 14)
w + O(w^14)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); z  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
>>> W(pari('3 + O(5^3)'))
3 + O(w^15)
>>> W(R(Integer(3),Integer(3)))
3 + O(w^15)
>>> W = R.ext(x**Integer(625) + Integer(915)*x**Integer(17) - Integer(95), names=(
↪'w',)); (w,) = W._first_ngens(1)
>>> W(Integer(3))
3 + O(w^3125)
>>> W(w, Integer(14))
w + O(w^14)
```

expansion(*n=None*, *lift_mode='simple'*)

Return a list giving a series representation of `self`.

- If `lift_mode == 'simple'` or `'smallest'`, the returned list will consist of integers (in the Eisenstein case) or a list of lists of integers (in the unramified case). `self` can be reconstructed as a sum of elements of the list times powers of the uniformiser (in the Eisenstein case), or as a sum of powers of the p times polynomials in the generator (in the unramified case).
  - If `lift_mode == 'simple'`, all integers will be in the interval $[0, p-1]$.
  - If `lift_mode == 'smallest'` they will be in the interval $[(1-p)/2, p/2]$.
- If `lift_mode == 'teichmuller'`, returns a list of `pAdicZZpXCRElements`, all of which are Teichmuller representatives and such that `self` is the sum of that list times powers of the uniformizer.

Note that zeros are truncated from the returned list if `self.parent()` is a field, so you must use the `valuation` function to fully reconstruct `self`.

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: y = W(775, 19); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
sage: (y>>9).expansion()
[0, 1, 0, 4, 0, 2, 1, 2, 4, 1]
sage: (y>>9).expansion(lift_mode='smallest')
[0, 1, 0, -1, 0, 2, 1, 2, 0, 1]
sage: w^10 - w^12 + 2*w^14 + w^15 + 2*w^16 + w^18 + O(w^19)
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
sage: g = x^3 + 3*x + 3
sage: A.<a> = R.ext(g)
sage: y = 75 + 45*a + 1200*a^2; y
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^6)
```

(continues on next page)

(continued from previous page)

```
sage: E = y.expansion(); E
5-adic expansion of 4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^6)
sage: list(E)
[[], [0, 4], [3, 1, 3], [0, 0, 4], [0, 0, 1], []]
sage: list(y.expansion(lift_mode='smallest'))
[[], [0, -1], [-2, 2, -2], [1], [0, 0, 2], []]
sage: 5*((-2*5 + 25) + (-1 + 2*5)*a + (-2*5 + 2*125)*a^2)
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^6)
sage: list(W(0).expansion())
[]
sage: list(W(0,4).expansion())
[]
sage: list(A(0,4).expansion())
[]
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> y = W(Integer(775), Integer(19)); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
>>> (y>>Integer(9)).expansion()
[0, 1, 0, 4, 0, 2, 1, 2, 4, 1]
>>> (y>>Integer(9)).expansion(lift_mode='smallest')
[0, 1, 0, -1, 0, 2, 1, 2, 0, 1]
>>> w**Integer(10) - w**Integer(12) + Integer(2)*w**Integer(14) +␣
↪w**Integer(15) + Integer(2)*w**Integer(16) + w**Integer(18) +␣
↪O(w**Integer(19))
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)
>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> y = Integer(75) + Integer(45)*a + Integer(1200)*a**Integer(2); y
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^6)
>>> E = y.expansion(); E
5-adic expansion of 4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^6)
>>> list(E)
[[], [0, 4], [3, 1, 3], [0, 0, 4], [0, 0, 1], []]
>>> list(y.expansion(lift_mode='smallest'))
[[], [0, -1], [-2, 2, -2], [1], [0, 0, 2], []]
>>> Integer(5)*((-Integer(2)*Integer(5) + Integer(25)) + (-Integer(1) +␣
↪Integer(2)*Integer(5))*a + (-Integer(2)*Integer(5) +␣
↪Integer(2)*Integer(125))*a**Integer(2))
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^6)
>>> list(W(Integer(0)).expansion())
[]
>>> list(W(Integer(0),Integer(4)).expansion())
[]
>>> list(A(Integer(0),Integer(4)).expansion())
[]
```

is_equal_to(*right*, *absprec=None*)

Return whether this element is equal to `right` modulo `self.uniformizer()^absprec`.

If `absprec` is `None`, checks whether this element is equal to `right` modulo the lower of their two precisions.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(47); b = W(47 + 25)
sage: a.is_equal_to(b)
False
sage: a.is_equal_to(b, 7)
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(47)); b = W(Integer(47) + Integer(25))
>>> a.is_equal_to(b)
False
>>> a.is_equal_to(b, Integer(7))
True
```

is_zero(*absprec=None*)

Return whether the valuation of this element is at least `absprec`. If `absprec` is `None`, checks if this element is indistinguishable from zero.

If this element is an inexact zero of valuation less than `absprec`, raises a *PrecisionError*.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: O(w^189).is_zero()
True
sage: W(0).is_zero()
True
sage: a = W(675)
sage: a.is_zero()
False
sage: a.is_zero(7)
True
sage: a.is_zero(21)
False
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> O(w**Integer(189)).is_zero()
True
>>> W(Integer(0)).is_zero()
True
>>> a = W(Integer(675))
>>> a.is_zero()
False
>>> a.is_zero(Integer(7))
True
>>> a.is_zero(Integer(21))
False
```

lift_to_precision(*absprec=None*)

Return a `pAdicZZpXCRElement` congruent to this element but with absolute precision at least `absprec`.

INPUT:

- `absprec` – (default: `None`) the absolute precision of the result. If `None`, lifts to the maximum precision allowed.

Note

If setting `absprec` that high would violate the precision cap, raises a precision error. If `self` is an inexact zero and `absprec` is greater than the maximum allowed valuation, raises an error.

Note that the new digits will not necessarily be zero.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(345, 17); a
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + O(w^
↪17)
sage: b = a.lift_to_precision(19); b
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15
 + w^17 + 2*w^18 + O(w^19)
sage: c = a.lift_to_precision(24); c
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15
 + w^17 + 2*w^18 + 4*w^19 + 4*w^20 + 2*w^21 + 4*w^23 + O(w^24)
sage: a._ntl_rep()
[19 35 118 60 121]
sage: b._ntl_rep()
[19 35 118 60 121]
sage: c._ntl_rep()
[19 35 118 60 121]
```

(continues on next page)

(continued from previous page)

```
sage: a.lift_to_precision().precision_relative() == W.precision_cap()
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(345), Integer(17)); a
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + O(w^
↪17)
>>> b = a.lift_to_precision(Integer(19)); b
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15
 + w^17 + 2*w^18 + O(w^19)
>>> c = a.lift_to_precision(Integer(24)); c
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15
 + w^17 + 2*w^18 + 4*w^19 + 4*w^20 + 2*w^21 + 4*w^23 + O(w^24)
>>> a._ntl_rep()
[19 35 118 60 121]
>>> b._ntl_rep()
[19 35 118 60 121]
>>> c._ntl_rep()
[19 35 118 60 121]
>>> a.lift_to_precision().precision_relative() == W.precision_cap()
True
```

matrix_mod_pn()

Return the matrix of right multiplication by the element on the power basis $1, x, x^2, \ldots, x^{d-1}$ for this extension field. Thus the *rows* of this matrix give the images of each of the x^i. The entries of the matrices are `IntegerMod()` elements, defined modulo $p^{N/e}$ where N is the absolute precision of this element (unless this element is zero to arbitrary precision; in that case the entries are integer zeros.)

Raises an error if this element has negative valuation.

EXAMPLES:

```
sage: R = ZpCR(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = (3+w)^7
sage: a.matrix_mod_pn()
[2757  333 1068  725 2510]
[  50 1507  483  318  725]
[ 500   50 3007 2358  318]
[1590 1375 1695 1032 2358]
[2415  590 2370 2970 1032]
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = (Integer(3)+w)**Integer(7)
>>> a.matrix_mod_pn()
[2757  333 1068  725 2510]
[  50 1507  483  318  725]
[ 500   50 3007 2358  318]
[1590 1375 1695 1032 2358]
[2415  590 2370 2970 1032]
```

polynomial(*var='x'*)

Return a polynomial over the base ring that yields this element when evaluated at the generator of the parent.

INPUT:

- `var` – string, the variable name for the polynomial

EXAMPLES:

```
sage: S.<x> = ZZ[]
sage: W.<w> = Zp(5).extension(x^2 - 5)
sage: (w + W(5, 7)).polynomial()
(1 + O(5^3))*x + 5 + O(5^4)
```

```
>>> from sage.all import *
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> W = Zp(Integer(5)).extension(x**Integer(2) - Integer(5), names=('w',));␣
↪(w,) = W._first_ngens(1)
>>> (w + W(Integer(5), Integer(7))).polynomial()
(1 + O(5^3))*x + 5 + O(5^4)
```

precision_absolute()

Return the absolute precision of this element, i.e., the power of the uniformizer modulo which this element is defined.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75, 19); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
sage: a.valuation()
10
sage: a.precision_absolute()
19
sage: a.precision_relative()
9
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
sage: (a.unit_part() - 3).precision_absolute()
9
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75), Integer(19)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
>>> a.valuation()
10
>>> a.precision_absolute()
19
>>> a.precision_relative()
9
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
>>> (a.unit_part() - Integer(3)).precision_absolute()
9
```

precision_relative()

Return the relative precision of this element, i.e., the power of the uniformizer modulo which the unit part of `self` is defined.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75, 19); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
sage: a.valuation()
10
sage: a.precision_absolute()
19
sage: a.precision_relative()
9
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75), Integer(19)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
>>> a.valuation()
10
>>> a.precision_absolute()
19
>>> a.precision_relative()
```

(continues on next page)

(continued from previous page)

```
9
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

teichmuller_expansion(*n=None*)

Return a list $[a_0, a_1, \ldots, a_n]$ such that:

- $a_i^q = a_i$
- `self.unit_part()` = $\sum_{i=0}^{n} a_i \pi^i$, where π is a uniformizer of `self.parent()`
- if $a_i \neq 0$, the absolute precision of a_i is `self.precision_relative() - i`

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion

EXAMPLES:

```
sage: R.<a> = ZqCR(5^4,4)
sage: E = a.teichmuller_expansion(); E
5-adic expansion of a + O(5^4) (teichmuller)
sage: list(E)
[a + (2*a^3 + 2*a^2 + 3*a + 4)*5 + (4*a^3 + 3*a^2 + 3*a + 2)*5^2
   + (4*a^2 + 2*a + 2)*5^3 + O(5^4),
 (3*a^3 + 3*a^2 + 2*a + 1) + (a^3 + 4*a^2 + 1)*5 + (a^2 + 4*a + 4)*5^2 + O(5^
↪3),
 (4*a^3 + 2*a^2 + a + 1) + (2*a^3 + 2*a^2 + 2*a + 4)*5 + O(5^2),
 (a^3 + a^2 + a + 4) + O(5)]
sage: sum([c * 5^i for i, c in enumerate(E)])
a + O(5^4)
sage: all(c^625 == c for c in E)
True

sage: S.<x> = ZZ[]
sage: f = x^3 - 98*x + 7
sage: W.<w> = ZpCR(7,3).ext(f)
sage: b = (1+w)^5; L = b.teichmuller_expansion(); L
[1 + O(w^9), 5 + 5*w^3 + w^6 + 4*w^7 + O(w^8), 3 + 3*w^3 + O(w^7),
 3 + 3*w^3 + O(w^6), O(w^5), 4 + 5*w^3 + O(w^4), 3 + O(w^3),
 6 + O(w^2), 6 + O(w)]
sage: sum([w^i*L[i] for i in range(9)]) == b
True
sage: all(L[i]^(7^3) == L[i] for i in range(9))
True

sage: L = W(3).teichmuller_expansion(); L
[3 + 3*w^3 + w^7 + O(w^9), O(w^8), O(w^7), 4 + 5*w^3 + O(w^6),
 O(w^5), O(w^4), 3 + O(w^3), 6 + O(w^2)]
sage: sum([w^i*L[i] for i in range(len(L))])
3 + O(w^9)
```

```
>>> from sage.all import *
>>> R = ZqCR(Integer(5)**Integer(4),Integer(4), names=('a',)); (a,) = R._
↪first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> E = a.teichmuller_expansion(); E
5-adic expansion of a + O(5^4) (teichmuller)
>>> list(E)
[a + (2*a^3 + 2*a^2 + 3*a + 4)*5 + (4*a^3 + 3*a^2 + 3*a + 2)*5^2
   + (4*a^2 + 2*a + 2)*5^3 + O(5^4),
 (3*a^3 + 3*a^2 + 2*a + 1) + (a^3 + 4*a^2 + 1)*5 + (a^2 + 4*a + 4)*5^2 + O(5^
↪3),
 (4*a^3 + 2*a^2 + a + 1) + (2*a^3 + 2*a^2 + 2*a + 4)*5 + O(5^2),
 (a^3 + a^2 + a + 4) + O(5)]
>>> sum([c * Integer(5)**i for i, c in enumerate(E)])
a + O(5^4)
>>> all(c**Integer(625) == c for c in E)
True

>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(3) - Integer(98)*x + Integer(7)
>>> W = ZpCR(Integer(7),Integer(3)).ext(f, names=('w',)); (w,) = W._first_
↪ngens(1)
>>> b = (Integer(1)+w)**Integer(5); L = b.teichmuller_expansion(); L
[1 + O(w^9), 5 + 5*w^3 + w^6 + 4*w^7 + O(w^8), 3 + 3*w^3 + O(w^7),
 3 + 3*w^3 + O(w^6), O(w^5), 4 + 5*w^3 + O(w^4), 3 + O(w^3),
 6 + O(w^2), 6 + O(w)]
>>> sum([w**i*L[i] for i in range(Integer(9))]) == b
True
>>> all(L[i]**(Integer(7)**Integer(3)) == L[i] for i in range(Integer(9)))
True

>>> L = W(Integer(3)).teichmuller_expansion(); L
[3 + 3*w^3 + w^7 + O(w^9), O(w^8), O(w^7), 4 + 5*w^3 + O(w^6),
 O(w^5), O(w^4), 3 + O(w^3), 6 + O(w^2)]
>>> sum([w**i*L[i] for i in range(len(L))])
3 + O(w^9)
```

unit_part()

Return the unit part of this element, ie `self / uniformizer^(self.valuation())`.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75, 19); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
sage: a.valuation()
10
sage: a.precision_absolute()
19
sage: a.precision_relative()
9
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75), Integer(19)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
>>> a.valuation()
10
>>> a.precision_absolute()
19
>>> a.precision_relative()
9
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

CHAPTER

TWENTY

p-ADIC ZZ_PX CA ELEMENT

This file implements elements of Eisenstein and unramified extensions of Zp with capped absolute precision.

For the parent class see padic_extension_leaves.pyx.

The underlying implementation is through NTL's ZZ_pX class. Each element contains the following data:

- absprec – long; an integer giving the precision to which this element is defined. This is the power of the uniformizer modulo which the element is well defined.
- value – ZZ_pX_c; an ntl ZZ_pX storing the value. The variable x is the uniformizer in the case of Eisenstein extensions This ZZ_pX is created with global ntl modulus determined by absprec. Let a be absprec and e be the ramification index over $\mathbf{Q}_p$ or $\mathbf{Z}_p$. Then the modulus is given by $p^{ceil(a/e)}$. Note that all kinds of problems arise if you try to mix moduli. ZZ_pX_conv_modulus gives a semi-safe way to convert between different moduli without having to pass through ZZX.
- prime_pow – (some subclass of PowComputer_ZZ_pX) a class, identical among all elements with the same parent, holding common data.
  - prime_pow.deg – the degree of the extension
  - prime_pow.e – the ramification index
  - prime_pow.f – the inertia degree
  - prime_pow.prec_cap – the unramified precision cap. For Eisenstein extensions this is the smallest power of p that is zero.
  - prime_pow.ram_prec_cap – the ramified precision cap. For Eisenstein extensions this will be the smallest power of x that is indistinguishable from zero.
  - prime_pow.pow_ZZ_tmp, prime_pow.pow_mpz_t_tmp``, prime_pow.pow_Integer – functions for accessing powers of p. The first two return pointers. See sage/rings/padics/pow_computer_ext for examples and important warnings.
  - prime_pow.get_context, prime_pow.get_context_capdiv, prime_pow.get_top_context – obtain an ntl_ZZ_pContext_class corresponding to p^n. The capdiv version divides by prime_pow.e as appropriate. top_context corresponds to $p^{\text{prec_cap}}$.
  - prime_pow.restore_context, prime_pow.restore_context_capdiv, prime_pow.restore_top_context – restores the given context.
  - prime_pow.get_modulus, get_modulus_capdiv, get_top_modulus – returns a ZZ_pX_Modulus_c* pointing to a polynomial modulus defined modulo p^n (appropriately divided by prime_pow.e in the capdiv case).

EXAMPLES:

An Eisenstein extension:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f); W
5-adic Eisenstein Extension Ring in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: z = (1+w)^5; z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15
  + 4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
sage: y = z >> 1; y
w^4 + w^5 + 2*w^6 + 4*w^7 + 3*w^9 + w^11 + 4*w^12 + 4*w^13 + 4*w^14 + 4*w^15
  + 4*w^16 + 4*w^19 + w^20 + 4*w^23 + O(w^24)
sage: y.valuation()
4
sage: y.precision_relative()
20
sage: y.precision_absolute()
24
sage: z - (y << 1)
1 + O(w^25)
sage: (1/w)^12+w
w^-12 + w + O(w^12)
sage: (1/w).parent()
5-adic Eisenstein Extension Field in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1); W
5-adic Eisenstein Extension Ring in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
>>> z = (Integer(1)+w)**Integer(5); z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15
  + 4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
>>> y = z >> Integer(1); y
w^4 + w^5 + 2*w^6 + 4*w^7 + 3*w^9 + w^11 + 4*w^12 + 4*w^13 + 4*w^14 + 4*w^15
  + 4*w^16 + 4*w^19 + w^20 + 4*w^23 + O(w^24)
>>> y.valuation()
4
>>> y.precision_relative()
20
>>> y.precision_absolute()
24
>>> z - (y << Integer(1))
1 + O(w^25)
>>> (Integer(1)/w)**Integer(12)+w
w^-12 + w + O(w^12)
>>> (Integer(1)/w).parent()
5-adic Eisenstein Extension Field in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
```

An unramified extension:

```
sage: g = x^3 + 3*x + 3
sage: A.<a> = R.ext(g)
sage: z = (1+a)^5; z
(2*a^2 + 4*a) + (3*a^2 + 3*a + 1)*5 + (4*a^2 + 3*a + 4)*5^2
+ (4*a^2 + 4*a + 4)*5^3 + (4*a^2 + 4*a + 4)*5^4 + O(5^5)
sage: z - 1 - 5*a - 10*a^2 - 10*a^3 - 5*a^4 - a^5
O(5^5)
sage: y = z >> 1; y
(3*a^2 + 3*a + 1) + (4*a^2 + 3*a + 4)*5 + (4*a^2 + 4*a + 4)*5^2
+ (4*a^2 + 4*a + 4)*5^3 + O(5^4)
sage: 1/a
(3*a^2 + 4) + (a^2 + 4)*5 + (3*a^2 + 4)*5^2 + (a^2 + 4)*5^3 + (3*a^2 + 4)*5^4 + O(5^5)
sage: FFA = A.residue_field()
sage: a0 = FFA.gen(); A(a0^3)
(2*a + 2) + O(5)
```

```
>>> from sage.all import *
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)
>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> z = (Integer(1)+a)**Integer(5); z
(2*a^2 + 4*a) + (3*a^2 + 3*a + 1)*5 + (4*a^2 + 3*a + 4)*5^2
+ (4*a^2 + 4*a + 4)*5^3 + (4*a^2 + 4*a + 4)*5^4 + O(5^5)
>>> z - Integer(1) - Integer(5)*a - Integer(10)*a**Integer(2) -␣
↪Integer(10)*a**Integer(3) - Integer(5)*a**Integer(4) - a**Integer(5)
O(5^5)
>>> y = z >> Integer(1); y
(3*a^2 + 3*a + 1) + (4*a^2 + 3*a + 4)*5 + (4*a^2 + 4*a + 4)*5^2
+ (4*a^2 + 4*a + 4)*5^3 + O(5^4)
>>> Integer(1)/a
(3*a^2 + 4) + (a^2 + 4)*5 + (3*a^2 + 4)*5^2 + (a^2 + 4)*5^3 + (3*a^2 + 4)*5^4 + O(5^5)
>>> FFA = A.residue_field()
>>> a0 = FFA.gen(); A(a0**Integer(3))
(2*a + 2) + O(5)
```

Different printing modes:

```
sage: R = ZpCA(5, print_mode='digits'); S.<x> = ZZ[]; f = x^5 + 75*x^3 - 15*x^2 +␣
↪125*x -5; W.<w> = R.ext(f)
sage: z = (1+w)^5; repr(z)
'...411040311321031044222131124200011101120110200202330321433201121440323201314400140
↪444441030421100001'
sage: R = ZpCA(5, print_mode='bars'); S.<x> = ZZ[]; g = x^3 + 3*x + 3; A.<a> = R.
↪ext(g)
sage: z = (1+a)^5; repr(z)
'...[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 3, 4]|[1, 3, 3]|[0, 4, 2]'
sage: R = ZpCA(5, print_mode='terse'); S.<x> = ZZ[]; f = x^5 + 75*x^3 - 15*x^2 +␣
↪125*x -5; W.<w> = R.ext(f)
sage: z = (1+w)^5; z
6 + 95367431640505*w + 25*w^2 + 95367431640560*w^3 + 5*w^4 + O(w^100)
sage: R = ZpCA(5, print_mode='val-unit'); S.<x> = ZZ[]; f = x^5 + 75*x^3 - 15*x^2 +␣
↪125*x -5; W.<w> = R.ext(f)
```

(continues on next page)

(continued from previous page)

```
sage: y = (1+w)^5 - 1; y
w^5 * (2090041 + 19073486126901*w + 1258902*w^2 + 674*w^3 + 16785*w^4) + O(w^100)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5), print_mode='digits'); S = ZZ['x']; (x,) = S._first_ngens(1);␣
↪f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); repr(z)
'...411040311321031044222131124200011101120110200202330321433201121440323201314400140
↪4444410304211000 01'
>>> R = ZpCA(Integer(5), print_mode='bars'); S = ZZ['x']; (x,) = S._first_ngens(1); g␣
↪= x**Integer(3) + Integer(3)*x + Integer(3); A = R.ext(g, names=('a',)); (a,) = A._
↪first_ngens(1)
>>> z = (Integer(1)+a)**Integer(5); repr(z)
'...[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 3, 4]|[1, 3, 3]|[0, 4, 2]'
>>> R = ZpCA(Integer(5), print_mode='terse'); S = ZZ['x']; (x,) = S._first_ngens(1);␣
↪f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); z
6 + 95367431640505*w + 25*w^2 + 95367431640560*w^3 + 5*w^4 + O(w^100)
>>> R = ZpCA(Integer(5), print_mode='val-unit'); S = ZZ['x']; (x,) = S._first_
↪ngens(1); f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> y = (Integer(1)+w)**Integer(5) - Integer(1); y
w^5 * (2090041 + 19073486126901*w + 1258902*w^2 + 674*w^3 + 16785*w^4) + O(w^100)
```

You can get at the underlying ntl representation:

```
sage: z._ntl_rep()
[6 95367431640505 25 95367431640560 5]
sage: y._ntl_rep()
[5 95367431640505 25 95367431640560 5]
sage: y._ntl_rep_abs()
([5 95367431640505 25 95367431640560 5], 0)
```

```
>>> from sage.all import *
>>> z._ntl_rep()
[6 95367431640505 25 95367431640560 5]
>>> y._ntl_rep()
[5 95367431640505 25 95367431640560 5]
>>> y._ntl_rep_abs()
([5 95367431640505 25 95367431640560 5], 0)
```

Note

If you get an error `internal error: can't grow this _ntl_gbigint`, it indicates that moduli are being mixed inappropriately somewhere.

For example, when calling a function with a `ZZ_pX_c` as an argument, it copies. If the modulus is not set to the

modulus of the ZZ_pX_c, you can get errors.

AUTHORS:

- David Roe (2008-01-01): initial version
- Robert Harron (2011-09): fixes/enhancements
- Julian Rueth (2012-10-15): fixed an initialization bug

sage.rings.padics.padic_ZZ_pX_CA_element.**make_ZZpXCAElement**(*parent*, *value*, *absprec*, *version*)

For pickling. Makes a pAdicZZpXCAElement with given parent, value, absprec.

EXAMPLES:

```
sage: from sage.rings.padics.padic_ZZ_pX_CA_element import make_ZZpXCAElement
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: make_ZZpXCAElement(W, ntl.ZZ_pX([3,2,4],5^3),13,0)
3 + 2*w + 4*w^2 + O(w^13)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_ZZ_pX_CA_element import make_ZZpXCAElement
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> make_ZZpXCAElement(W, ntl.ZZ_pX([Integer(3),Integer(2),Integer(4)],
↪Integer(5)**Integer(3)),Integer(13),Integer(0))
3 + 2*w + 4*w^2 + O(w^13)
```

class sage.rings.padics.padic_ZZ_pX_CA_element.**pAdicZZpXCAElement**

Bases: *pAdicZZpXElement*

Create an element of a capped absolute precision, unramified or Eisenstein extension of Zp or Qp.

INPUT:

- parent – either an EisensteinRingCappedAbsolute or UnramifiedRingCappedAbsolute
- x – integer, rational, p-adic element, polynomial, list, integer_mod, pari int/frac/poly_t/pol_mod, an ntl_ZZ_pX, an ntl_ZZ, an ntl_ZZ_p, an ntl_ZZX, or something convertible into parent.residue_field()
- absprec – an upper bound on the absolute precision of the element created
- relprec – an upper bound on the relative precision of the element created
- empty – whether to return after initializing to zero

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
```

(continues on next page)

(continued from previous page)

```
sage: z = (1+w)^5; z  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15
  + 4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
sage: W(R(3,3))
3 + O(w^15)
sage: W(pari('3 + O(5^3)'))
3 + O(w^15)
sage: W(w, 14)
w + O(w^14)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); z  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15
  + 4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
>>> W(R(Integer(3),Integer(3)))
3 + O(w^15)
>>> W(pari('3 + O(5^3)'))
3 + O(w^15)
>>> W(w, Integer(14))
w + O(w^14)
```

expansion(*n=None*, *lift_mode='simple'*)

Return a list giving a series representation of `self`.

- If `lift_mode == 'simple'` or `'smallest'`, the returned list will consist of integers (in the Eisenstein case) or a list of lists of integers (in the unramified case). `self` can be reconstructed as a sum of elements of the list times powers of the uniformiser (in the Eisenstein case), or as a sum of powers of p times polynomials in the generator (in the unramified case).
  - If `lift_mode == 'simple'`, all integers will be in the interval $[0, p-1]$
  - If `lift_mod == 'smallest'` they will be in the interval $[(1-p)/2, p/2]$.
- If `lift_mode == 'teichmuller'`, returns a list of `pAdicZZpXCAElements`, all of which are Teichmuller representatives and such that `self` is the sum of that list times powers of the uniformizer.

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: y = W(775, 19); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
sage: (y>>9).expansion()
[0, 1, 0, 4, 0, 2, 1, 2, 4, 1]
```

(continues on next page)

(continued from previous page)

```
sage: (y>>9).expansion(lift_mode='smallest')
[0, 1, 0, -1, 0, 2, 1, 2, 0, 1]
sage: w^10 - w^12 + 2*w^14 + w^15 + 2*w^16 + w^18 + O(w^19)
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
sage: g = x^3 + 3*x + 3

sage: A.<a> = R.ext(g)
sage: y = 75 + 45*a + 1200*a^2; y
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^5)
sage: E = y.expansion(); E
5-adic expansion of 4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^5)
sage: list(E)
[[], [0, 4], [3, 1, 3], [0, 0, 4], [0, 0, 1]]
sage: list(y.expansion(lift_mode='smallest'))
[[], [0, -1], [-2, 2, -2], [1], [0, 0, 2]]
sage: 5*((-2*5 + 25) + (-1 + 2*5)*a + (-2*5 + 2*125)*a^2)
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^5)
sage: W(0).expansion()
[]
sage: list(A(0,4).expansion())
[]
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> y = W(Integer(775), Integer(19)); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
>>> (y>>Integer(9)).expansion()
[0, 1, 0, 4, 0, 2, 1, 2, 4, 1]
>>> (y>>Integer(9)).expansion(lift_mode='smallest')
[0, 1, 0, -1, 0, 2, 1, 2, 0, 1]
>>> w**Integer(10) - w**Integer(12) + Integer(2)*w**Integer(14) +␣
↪w**Integer(15) + Integer(2)*w**Integer(16) + w**Integer(18) +␣
↪O(w**Integer(19))
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + O(w^19)
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)

>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> y = Integer(75) + Integer(45)*a + Integer(1200)*a**Integer(2); y
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^5)
>>> E = y.expansion(); E
5-adic expansion of 4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^5)
>>> list(E)
[[], [0, 4], [3, 1, 3], [0, 0, 4], [0, 0, 1]]
>>> list(y.expansion(lift_mode='smallest'))
[[], [0, -1], [-2, 2, -2], [1], [0, 0, 2]]
>>> Integer(5)*((-Integer(2)*Integer(5) + Integer(25)) + (-Integer(1) +␣
↪Integer(2)*Integer(5))*a + (-Integer(2)*Integer(5) +␣
↪Integer(2)*Integer(125))*a**Integer(2))
```

(continues on next page)

(continued from previous page)

```
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4 + O(5^5)
>>> W(Integer(0)).expansion()
[]
>>> list(A(Integer(0),Integer(4)).expansion())
[]
```

Check that Issue #25879 has been resolved:

```
sage: K = ZpCA(3,5)
sage: R.<a> = K[]
sage: L.<a> = K.extension(a^2 - 3)
sage: a.residue()
0
```

```
>>> from sage.all import *
>>> K = ZpCA(Integer(3),Integer(5))
>>> R = K['a']; (a,) = R._first_ngens(1)
>>> L = K.extension(a**Integer(2) - Integer(3), names=('a',)); (a,) = L._
↪first_ngens(1)
>>> a.residue()
0
```

is_equal_to(*right*, *absprec=None*)

Return whether `self` is equal to `right` modulo `self.uniformizer()^absprec`.

If `absprec` is `None`, returns if `self` is equal to `right` modulo the lower of their two precisions.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(47); b = W(47 + 25)
sage: a.is_equal_to(b)
False
sage: a.is_equal_to(b, 7)
True
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(47)); b = W(Integer(47) + Integer(25))
>>> a.is_equal_to(b)
False
>>> a.is_equal_to(b, Integer(7))
True
```

is_zero(*absprec=None*)

Return whether the valuation of `self` is at least `absprec`.

If `absprec` is `None`, returns if `self` is indistinguishable from zero.

If `self` is an inexact zero of valuation less than `absprec`, raises a *PrecisionError*.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: O(w^189).is_zero()
True
sage: W(0).is_zero()
True
sage: a = W(675)
sage: a.is_zero()
False
sage: a.is_zero(7)
True
sage: a.is_zero(21)
False
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> O(w**Integer(189)).is_zero()
True
>>> W(Integer(0)).is_zero()
True
>>> a = W(Integer(675))
>>> a.is_zero()
False
>>> a.is_zero(Integer(7))
True
>>> a.is_zero(Integer(21))
False
```

lift_to_precision(*absprec=None*)

Return a `pAdicZZpXCAElement` congruent to `self` but with absolute precision at least `absprec`.

INPUT:

- `absprec` – (default: `None`) the absolute precision of the result. If `None`, lifts to the maximum precision allowed.

Note

If setting `absprec` that high would violate the precision cap, raises a precision error.

Note that the new digits will not necessarily be zero.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(345, 17); a
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + O(w^
↪17)
sage: b = a.lift_to_precision(19); b  # indirect doctest
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + w^
↪17 + 2*w^18 + O(w^19)
sage: c = a.lift_to_precision(24); c
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + w^
↪17 + 2*w^18 + 4*w^19 + 4*w^20 + 2*w^21 + 4*w^23 + O(w^24)
sage: a._ntl_rep()
[345]
sage: b._ntl_rep()
[345]
sage: c._ntl_rep()
[345]
sage: a.lift_to_precision().precision_absolute() == W.precision_cap()
True
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(345), Integer(17)); a
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + O(w^
↪17)
>>> b = a.lift_to_precision(Integer(19)); b  # indirect doctest
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + w^
↪17 + 2*w^18 + O(w^19)
>>> c = a.lift_to_precision(Integer(24)); c
4*w^5 + 3*w^7 + w^9 + 3*w^10 + 2*w^11 + 4*w^12 + w^13 + 2*w^14 + 2*w^15 + w^
↪17 + 2*w^18 + 4*w^19 + 4*w^20 + 2*w^21 + 4*w^23 + O(w^24)
>>> a._ntl_rep()
[345]
>>> b._ntl_rep()
[345]
>>> c._ntl_rep()
[345]
>>> a.lift_to_precision().precision_absolute() == W.precision_cap()
True
```

matrix_mod_pn()

Return the matrix of right multiplication by the element on the power basis $1, x, x^2, \ldots, x^{d-1}$ for this extension field. Thus the *rows* of this matrix give the images of each of the x^i. The entries of the matrices are `IntegerMod` elements, defined modulo `p^(self.absprec() / e)`.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = (3+w)^7
sage: a.matrix_mod_pn()                                                    #␣
↪needs sage.geometry.polyhedron
[2757  333 1068  725 2510]
[  50 1507  483  318  725]
[ 500   50 3007 2358  318]
[1590 1375 1695 1032 2358]
[2415  590 2370 2970 1032]
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = (Integer(3)+w)**Integer(7)
>>> a.matrix_mod_pn()                                                      #␣
↪needs sage.geometry.polyhedron
[2757  333 1068  725 2510]
[  50 1507  483  318  725]
[ 500   50 3007 2358  318]
[1590 1375 1695 1032 2358]
[2415  590 2370 2970 1032]
```

polynomial(*var='x'*)

Return a polynomial over the base ring that yields this element when evaluated at the generator of the parent.

INPUT:

- `var` – string, the variable name for the polynomial

EXAMPLES:

```
sage: S.<x> = ZZ[]
sage: W.<w> = ZpCA(5).extension(x^2 - 5)
sage: (w + W(5, 7)).polynomial()
(1 + O(5^3))*x + 5 + O(5^4)
```

```
>>> from sage.all import *
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> W = ZpCA(Integer(5)).extension(x**Integer(2) - Integer(5), names=('w',));␣
↪(w,) = W._first_ngens(1)
>>> (w + W(Integer(5), Integer(7))).polynomial()
(1 + O(5^3))*x + 5 + O(5^4)
```

precision_absolute()

Return the absolute precision of `self`, ie the power of the uniformizer modulo which this element is defined.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75, 19); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
sage: a.valuation()
10
sage: a.precision_absolute()
19
sage: a.precision_relative()
9
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75), Integer(19)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
>>> a.valuation()
10
>>> a.precision_absolute()
19
>>> a.precision_relative()
9
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

precision_relative()

Return the relative precision of `self`, ie the power of the uniformizer modulo which the unit part of `self` is defined.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75, 19); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
sage: a.valuation()
10
sage: a.precision_absolute()
19
sage: a.precision_relative()
9
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75), Integer(19)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
>>> a.valuation()
10
>>> a.precision_absolute()
19
>>> a.precision_relative()
9
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

teichmuller_expansion(*n=None*)

Return a list $[a_0, a_1, \ldots, a_n]$ such that:

- $a_i^q = a_i$
- `self.unit_part()` = $\sum_{i=0}^{n} a_i \pi^i$, where π is a uniformizer of self.parent()
- if $a_i \neq 0$, the absolute precision of a_i is `self.precision_relative() - i`

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion

EXAMPLES:

```
sage: R.<a> = Zq(5^4,4)
sage: E = a.teichmuller_expansion(); E
5-adic expansion of a + O(5^4) (teichmuller)
sage: list(E)
[a + (2*a^3 + 2*a^2 + 3*a + 4)*5 + (4*a^3 + 3*a^2 + 3*a + 2)*5^2 + (4*a^2 +␣
↪2*a + 2)*5^3 + O(5^4),
 (3*a^3 + 3*a^2 + 2*a + 1) + (a^3 + 4*a^2 + 1)*5 + (a^2 + 4*a + 4)*5^2 + O(5^
↪3),
 (4*a^3 + 2*a^2 + a + 1) + (2*a^3 + 2*a^2 + 2*a + 4)*5 + O(5^2),
 (a^3 + a^2 + a + 4) + O(5)]
sage: sum([c * 5^i for i, c in enumerate(E)])
a + O(5^4)
sage: all(c^625 == c for c in E)
True

sage: S.<x> = ZZ[]
sage: f = x^3 - 98*x + 7
sage: W.<w> = ZpCA(7,3).ext(f)
sage: b = (1+w)^5; L = b.teichmuller_expansion(); L
[1 + O(w^9), 5 + 5*w^3 + w^6 + 4*w^7 + O(w^8), 3 + 3*w^3 + O(w^7),
 3 + 3*w^3 + O(w^6), O(w^5), 4 + 5*w^3 + O(w^4), 3 + O(w^3), 6 + O(w^2), 6 +␣
↪O(w)]
sage: sum([w^i*L[i] for i in range(9)]) == b
True
```

(continues on next page)

(continued from previous page)

```
sage: all(L[i]^(7^3) == L[i] for i in range(9))
True

sage: L = W(3).teichmuller_expansion(); L
[3 + 3*w^3 + w^7 + O(w^9), O(w^8), O(w^7), 4 + 5*w^3 + O(w^6), O(w^5),
 O(w^4), 3 + O(w^3), 6 + O(w^2)]
sage: sum([w^i*L[i] for i in range(len(L))])
3 + O(w^9)
```

```
>>> from sage.all import *
>>> R = Zq(Integer(5)**Integer(4),Integer(4), names=('a',)); (a,) = R._first_
↪ngens(1)
>>> E = a.teichmuller_expansion(); E
5-adic expansion of a + O(5^4) (teichmuller)
>>> list(E)
[a + (2*a^3 + 2*a^2 + 3*a + 4)*5 + (4*a^3 + 3*a^2 + 3*a + 2)*5^2 + (4*a^2 +␣
↪2*a + 2)*5^3 + O(5^4),
 (3*a^3 + 3*a^2 + 2*a + 1) + (a^3 + 4*a^2 + 1)*5 + (a^2 + 4*a + 4)*5^2 + O(5^
↪3),
 (4*a^3 + 2*a^2 + a + 1) + (2*a^3 + 2*a^2 + 2*a + 4)*5 + O(5^2),
 (a^3 + a^2 + a + 4) + O(5)]
>>> sum([c * Integer(5)**i for i, c in enumerate(E)])
a + O(5^4)
>>> all(c**Integer(625) == c for c in E)
True

>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(3) - Integer(98)*x + Integer(7)
>>> W = ZpCA(Integer(7),Integer(3)).ext(f, names=('w',)); (w,) = W._first_
↪ngens(1)
>>> b = (Integer(1)+w)**Integer(5); L = b.teichmuller_expansion(); L
[1 + O(w^9), 5 + 5*w^3 + w^6 + 4*w^7 + O(w^8), 3 + 3*w^3 + O(w^7),
 3 + 3*w^3 + O(w^6), O(w^5), 4 + 5*w^3 + O(w^4), 3 + O(w^3), 6 + O(w^2), 6 +␣
↪O(w)]
>>> sum([w**i*L[i] for i in range(Integer(9))]) == b
True
>>> all(L[i]**(Integer(7)**Integer(3)) == L[i] for i in range(Integer(9)))
True

>>> L = W(Integer(3)).teichmuller_expansion(); L
[3 + 3*w^3 + w^7 + O(w^9), O(w^8), O(w^7), 4 + 5*w^3 + O(w^6), O(w^5),
 O(w^4), 3 + O(w^3), 6 + O(w^2)]
>>> sum([w**i*L[i] for i in range(len(L))])
3 + O(w^9)
```

to_fraction_field()

Return `self` cast into the fraction field of `self.parent()`.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
```

(continues on next page)

(continued from previous page)

```
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: z = (1 + w)^5; z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 +␣
↪4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
sage: y = z.to_fraction_field(); y  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 +␣
↪4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
sage: y.parent()
5-adic Eisenstein Extension Field in w defined by x^5 + 75*x^3 - 15*x^2 +␣
↪125*x - 5
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1) + w)**Integer(5); z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 +␣
↪4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
>>> y = z.to_fraction_field(); y  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 +␣
↪4*w^16 + 4*w^17 + 4*w^20 + w^21 + 4*w^24 + O(w^25)
>>> y.parent()
5-adic Eisenstein Extension Field in w defined by x^5 + 75*x^3 - 15*x^2 +␣
↪125*x - 5
```

unit_part()

Return the unit part of `self`, ie `self / uniformizer^(self.valuation())`.

EXAMPLES:

```
sage: R = ZpCA(5,5)
sage: S.<x> = ZZ[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75, 19); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
sage: a.valuation()
10
sage: a.precision_absolute()
19
sage: a.precision_relative()
9
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

```
>>> from sage.all import *
>>> R = ZpCA(Integer(5),Integer(5))
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
```

(continues on next page)

(continued from previous page)

```
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75), Integer(19)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + O(w^19)
>>> a.valuation()
10
>>> a.precision_absolute()
19
>>> a.precision_relative()
9
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + O(w^9)
```

CHAPTER

TWENTYONE

p-ADIC ZZ_PX FM ELEMENT

This file implements elements of Eisenstein and unramified extensions of $\mathbf{Z}_p$ with fixed modulus precision.

For the parent class see `padic_extension_leaves.pyx`.

The underlying implementation is through NTL's `ZZ_pX` class. Each element contains the following data:

- `value` (`ZZ_pX_c`) – an ntl `ZZ_pX` storing the value. The variable x is the uniformizer in the case of Eisenstein extensions. This `ZZ_pX` is created with global ntl modulus determined by the parent's precision cap and shared among all elements.
- `prime_pow` (some subclass of *PowComputer_ZZ_pX*) – a class, identical among all elements with the same parent, holding common data.
  - `prime_pow.deg` – the degree of the extension
  - `prime_pow.e` – the ramification index
  - `prime_pow.f` – the inertia degree
  - `prime_pow.prec_cap` – the unramified precision cap: for Eisenstein extensions this is the smallest power of p that is zero
  - `prime_pow.ram_prec_cap` – the ramified precision cap: for Eisenstein extensions this will be the smallest power of x that is indistinguishable from zero
  - `prime_pow.pow_ZZ_tmp`, prime_pow.pow_mpz_t_tmp``, `prime_pow.pow_Integer` – functions for accessing powers of p. The first two return pointers. See `sage/rings/padics/pow_computer_ext` for examples and important warnings.
  - `prime_pow.get_context`, `prime_pow.get_context_capdiv`, `prime_pow.get_top_context` – obtain an `ntl_ZZ_pContext_class` corresponding to p^n. The capdiv version divides by `prime_pow.e` as appropriate. `top_context` corresponds to $p^{\texttt{prec_cap}}$.
  - `prime_pow.restore_context`, `prime_pow.restore_context_capdiv`, `prime_pow.restore_top_context` – restores the given context
  - `prime_pow.get_modulus`, `get_modulus_capdiv`, `get_top_modulus` – returns a `ZZ_pX_Modulus_c*` pointing to a polynomial modulus defined modulo p^n (appropriately divided by `prime_pow.e` in the capdiv case).

EXAMPLES:

An Eisenstein extension:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f); W
```

(continues on next page)

(continued from previous page)

```
5-adic Eisenstein Extension Ring in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: z = (1+w)^5; z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16 +␣
↪4*w^17 + 4*w^20 + w^21 + 4*w^24
sage: y = z >> 1; y
w^4 + w^5 + 2*w^6 + 4*w^7 + 3*w^9 + w^11 + 4*w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^19 + w^20 + 4*w^23 + 4*w^24
sage: y.valuation()
4
sage: y.precision_relative()
21
sage: y.precision_absolute()
25
sage: z - (y << 1)
1
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1); W
5-adic Eisenstein Extension Ring in w defined by x^5 + 75*x^3 - 15*x^2 + 125*x - 5
>>> z = (Integer(1)+w)**Integer(5); z
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16 +␣
↪4*w^17 + 4*w^20 + w^21 + 4*w^24
>>> y = z >> Integer(1); y
w^4 + w^5 + 2*w^6 + 4*w^7 + 3*w^9 + w^11 + 4*w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^19 + w^20 + 4*w^23 + 4*w^24
>>> y.valuation()
4
>>> y.precision_relative()
21
>>> y.precision_absolute()
25
>>> z - (y << Integer(1))
1
```

An unramified extension:

```
sage: g = x^3 + 3*x + 3
sage: A.<a> = R.ext(g)
sage: z = (1+a)^5; z
(2*a^2 + 4*a) + (3*a^2 + 3*a + 1)*5 + (4*a^2 + 3*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^3 +␣
↪(4*a^2 + 4*a + 4)*5^4
sage: z - 1 - 5*a - 10*a^2 - 10*a^3 - 5*a^4 - a^5
0
sage: y = z >> 1; y
(3*a^2 + 3*a + 1) + (4*a^2 + 3*a + 4)*5 + (4*a^2 + 4*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^
↪3
sage: 1/a
(3*a^2 + 4) + (a^2 + 4)*5 + (3*a^2 + 4)*5^2 + (a^2 + 4)*5^3 + (3*a^2 + 4)*5^4
```

```
>>> from sage.all import *
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)
>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> z = (Integer(1)+a)**Integer(5); z
(2*a^2 + 4*a) + (3*a^2 + 3*a + 1)*5 + (4*a^2 + 3*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^3 +␣
↪(4*a^2 + 4*a + 4)*5^4
>>> z - Integer(1) - Integer(5)*a - Integer(10)*a**Integer(2) -␣
↪Integer(10)*a**Integer(3) - Integer(5)*a**Integer(4) - a**Integer(5)
0
>>> y = z >> Integer(1); y
(3*a^2 + 3*a + 1) + (4*a^2 + 3*a + 4)*5 + (4*a^2 + 4*a + 4)*5^2 + (4*a^2 + 4*a + 4)*5^
↪3
>>> Integer(1)/a
(3*a^2 + 4) + (a^2 + 4)*5 + (3*a^2 + 4)*5^2 + (a^2 + 4)*5^3 + (3*a^2 + 4)*5^4
```

Different printing modes:

```
sage: R = ZpFM(5, print_mode='digits'); S.<x> = R[]; f = x^5 + 75*x^3 - 15*x^2 +␣
↪125*x -5; W.<w> = R.ext(f)
sage: z = (1+w)^5; repr(z)
'...411040311321031044222131124200011101120110200202330321433201121440323201314400140
↪444441030421100001'
sage: R = ZpFM(5, print_mode='bars'); S.<x> = R[]; g = x^3 + 3*x + 3; A.<a> = R.ext(g)
sage: z = (1+a)^5; repr(z)
'...[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 3, 4]|[1, 3, 3]|[0, 4, 2]'
sage: R = ZpFM(5, print_mode='terse'); S.<x> = R[]; f = x^5 + 75*x^3 - 15*x^2 + 125*x␣
↪-5; W.<w> = R.ext(f)
sage: z = (1+w)^5; z
6 + 95367431640505*w + 25*w^2 + 95367431640560*w^3 + 5*w^4
sage: R = ZpFM(5, print_mode='val-unit'); S.<x> = R[]; f = x^5 + 75*x^3 - 15*x^2 +␣
↪125*x -5; W.<w> = R.ext(f)
sage: y = (1+w)^5 - 1; y
w^5 * (2090041 + 19073486126901*w + 1258902*w^2 + 57220458985049*w^3 + 16785*w^4)
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5), print_mode='digits'); S = R['x']; (x,) = S._first_ngens(1);␣
↪f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); repr(z)
'...411040311321031044222131124200011101120110200202330321433201121440323201314400140
↪444441030421100001'
>>> R = ZpFM(Integer(5), print_mode='bars'); S = R['x']; (x,) = S._first_ngens(1); g␣
↪= x**Integer(3) + Integer(3)*x + Integer(3); A = R.ext(g, names=('a',)); (a,) = A._
↪first_ngens(1)
>>> z = (Integer(1)+a)**Integer(5); repr(z)
'...[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4, 4]|[4, 4,␣
↪4]|[4, 4, 4]|[4, 3, 4]|[1, 3, 3]|[0, 4, 2]'
>>> R = ZpFM(Integer(5), print_mode='terse'); S = R['x']; (x,) = S._first_ngens(1); f␣
↪= x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> z = (Integer(1)+w)**Integer(5); z
6 + 95367431640505*w + 25*w^2 + 95367431640560*w^3 + 5*w^4
>>> R = ZpFM(Integer(5), print_mode='val-unit'); S = R['x']; (x,) = S._first_ngens(1);
↪ f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x -Integer(5); W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> y = (Integer(1)+w)**Integer(5) - Integer(1); y
w^5 * (2090041 + 19073486126901*w + 1258902*w^2 + 57220458985049*w^3 + 16785*w^4)
```

AUTHORS:

- David Roe (2008-01-01) initial version

sage.rings.padics.padic_ZZ_pX_FM_element.**make_ZZpXFMElement**(*parent*, *f*)

Create a new pAdicZZpXFMElement out of an ntl_ZZ_pX f, with parent parent. For use with pickling.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: z = (1 + w)^5 - 1
sage: loads(dumps(z)) == z  # indirect doctest
True
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1) + w)**Integer(5) - Integer(1)
>>> loads(dumps(z)) == z  # indirect doctest
True
```

class sage.rings.padics.padic_ZZ_pX_FM_element.**pAdicZZpXFMElement**

Bases: *pAdicZZpXElement*

Create an element of a fixed modulus, unramified or eisenstein extension of $\mathbf{Z}_p$ or $\mathbf{Q}_p$.

INPUT:

- parent – either an EisensteinRingFixedMod or UnramifiedRingFixedMod
- x – integer, rational, p-adic element, polynomial, list, integer_mod, pari int/frac/poly_t/pol_mod, an ntl_ZZ_pX, an ntl_ZZX, an ntl_ZZ, or an ntl_ZZ_p
- absprec – not used
- relprec – not used
- empty – whether to return after initializing to zero (without setting anything)

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
```

(continues on next page)

(continued from previous page)

```
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: z = (1+w)^5; z  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^17 + 4*w^20 + w^21 + 4*w^24
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2) +␣
↪Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> z = (Integer(1)+w)**Integer(5); z  # indirect doctest
1 + w^5 + w^6 + 2*w^7 + 4*w^8 + 3*w^10 + w^12 + 4*w^13 + 4*w^14 + 4*w^15 + 4*w^16␣
↪+ 4*w^17 + 4*w^20 + w^21 + 4*w^24
```

add_bigoh(*absprec*)

Return a new element truncated modulo π^{absprec}.

This is only implemented for unramified extension at this point.

INPUT:

- `absprec` – integer

OUTPUT: a new element truncated modulo π^{absprec}

EXAMPLES:

```
sage: R = Zp(7,4,'fixed-mod')
sage: a = R(1 + 7 + 7^2)
sage: a.add_bigoh(1)
1
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7),Integer(4),'fixed-mod')
>>> a = R(Integer(1) + Integer(7) + Integer(7)**Integer(2))
>>> a.add_bigoh(Integer(1))
1
```

expansion(*n=None*, *lift_mode='simple'*)

Return a list giving a series representation of this element.

- If `lift_mode == 'simple' or 'smallest'`, the returned list will consist of
  - integers (in the eisenstein case) or
  - lists of integers (in the unramified case).
- this element can be reconstructed as
  - a sum of elements of the list times powers of the uniformiser (in the eisenstein case), or
  - as a sum of powers of the p times polynomials in the generator (in the unramified case).
- If `lift_mode == 'simple'`, all integers will be in the range $[0, p-1]$,
- If `lift_mode == 'smallest'` they will be in the range $[(1-p)/2, p/2]$.

- If `lift_mode == 'teichmuller'`, returns a list of `pAdicZZpXCRElements`, all of which are Teichmuller representatives and such that this element is the sum of that list times powers of the uniformizer.

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: y = W(775); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + w^20 + 2*w^21 + 3*w^
↪22 + w^23 + w^24
sage: (y>>9).expansion()
[0, 1, 0, 4, 0, 2, 1, 2, 4, 1, 0, 1, 2, 3, 1, 1, 4, 1, 2, 4, 1, 0, 0, 3]
sage: (y>>9).expansion(lift_mode='smallest')
[0, 1, 0, -1, 0, 2, 1, 2, 0, 1, 2, 1, 1, -1, -1, 2, -2, 0, -2, -2, -2, 0, -2,␣
↪-2, 2]
sage: w^10 - w^12 + 2*w^14 + w^15 + 2*w^16 + w^18 + 2*w^19 + w^20 + w^21 - w^
↪22 - w^23 + 2*w^24
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + w^20 + 2*w^21 + 3*w^
↪22 + w^23 + w^24
sage: g = x^3 + 3*x + 3

sage: A.<a> = R.ext(g)
sage: y = 75 + 45*a + 1200*a^2; y
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4
sage: E = y.expansion(); E
5-adic expansion of 4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4
sage: list(E)
[[], [0, 4], [3, 1, 3], [0, 0, 4], [0, 0, 1]]
sage: list(y.expansion(lift_mode='smallest'))
[[], [0, -1], [-2, 2, -2], [1], [0, 0, 2]]
sage: 5*((-2*5 + 25) + (-1 + 2*5)*a + (-2*5 + 2*125)*a^2)
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4
sage: W(0).expansion()
[]
sage: list(A(0,4).expansion())
[]
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> y = W(Integer(775)); y
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + w^20 + 2*w^21 + 3*w^
↪22 + w^23 + w^24
>>> (y>>Integer(9)).expansion()
[0, 1, 0, 4, 0, 2, 1, 2, 4, 1, 0, 1, 2, 3, 1, 1, 4, 1, 2, 4, 1, 0, 0, 3]
>>> (y>>Integer(9)).expansion(lift_mode='smallest')
```

(continues on next page)

(continued from previous page)

```
[0, 1, 0, -1, 0, 2, 1, 2, 0, 1, 2, 1, 1, -1, -1, 2, -2, 0, -2, -2, -2, 0, -2,␣
↪-2, 2]
>>> w**Integer(10) - w**Integer(12) + Integer(2)*w**Integer(14) +␣
↪w**Integer(15) + Integer(2)*w**Integer(16) + w**Integer(18) +␣
↪Integer(2)*w**Integer(19) + w**Integer(20) + w**Integer(21) -␣
↪w**Integer(22) - w**Integer(23) + Integer(2)*w**Integer(24)
w^10 + 4*w^12 + 2*w^14 + w^15 + 2*w^16 + 4*w^17 + w^18 + w^20 + 2*w^21 + 3*w^
↪22 + w^23 + w^24
>>> g = x**Integer(3) + Integer(3)*x + Integer(3)

>>> A = R.ext(g, names=('a',)); (a,) = A._first_ngens(1)
>>> y = Integer(75) + Integer(45)*a + Integer(1200)*a**Integer(2); y
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4
>>> E = y.expansion(); E
5-adic expansion of 4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4
>>> list(E)
[[], [0, 4], [3, 1, 3], [0, 0, 4], [0, 0, 1]]
>>> list(y.expansion(lift_mode='smallest'))
[[], [0, -1], [-2, 2, -2], [1], [0, 0, 2]]
>>> Integer(5)*((-Integer(2)*Integer(5) + Integer(25)) + (-Integer(1) +␣
↪Integer(2)*Integer(5))*a + (-Integer(2)*Integer(5) +␣
↪Integer(2)*Integer(125))*a**Integer(2))
4*a*5 + (3*a^2 + a + 3)*5^2 + 4*a^2*5^3 + a^2*5^4
>>> W(Integer(0)).expansion()
[]
>>> list(A(Integer(0),Integer(4)).expansion())
[]
```

Check that Issue #25879 has been resolved:

```
sage: K = ZpCA(3,5)
sage: R.<a> = K[]
sage: L.<a> = K.extension(a^2 - 3)
sage: a.residue()
0
```

```
>>> from sage.all import *
>>> K = ZpCA(Integer(3),Integer(5))
>>> R = K['a']; (a,) = R._first_ngens(1)
>>> L = K.extension(a**Integer(2) - Integer(3), names=('a',)); (a,) = L._
↪first_ngens(1)
>>> a.residue()
0
```

is_equal_to(*right*, *absprec=None*)

Return whether `self` is equal to `right` modulo `self.uniformizer()^absprec`.

If `absprec` is `None`, returns if `self` is equal to `right` modulo the precision cap.

EXAMPLES:

```
sage: R = Zp(5,5)
sage: S.<x> = R[]
```

(continues on next page)

(continued from previous page)

```
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(47); b = W(47 + 25)
sage: a.is_equal_to(b)
False
sage: a.is_equal_to(b, 7)
True
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(47)); b = W(Integer(47) + Integer(25))
>>> a.is_equal_to(b)
False
>>> a.is_equal_to(b, Integer(7))
True
```

is_zero(*absprec=None*)

Return whether the valuation of `self` is at least `absprec`; if `absprec` is `None`, return whether `self` is indistinguishable from zero.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: O(w^189).is_zero()
True
sage: W(0).is_zero()
True
sage: a = W(675)
sage: a.is_zero()
False
sage: a.is_zero(7)
True
sage: a.is_zero(21)
False
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> O(w**Integer(189)).is_zero()
True
>>> W(Integer(0)).is_zero()
True
```

(continues on next page)

(continued from previous page)

```
>>> a = W(Integer(675))
>>> a.is_zero()
False
>>> a.is_zero(Integer(7))
True
>>> a.is_zero(Integer(21))
False
```

lift_to_precision(*absprec=None*)

Return `self`.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: w.lift_to_precision(10000)
w
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> w.lift_to_precision(Integer(10000))
w
```

matrix_mod_pn()

Return the matrix of right multiplication by the element on the power basis $1, x, x^2, \ldots, x^{d-1}$ for this extension field.

The **rows** of this matrix give the images of each of the x^i. The entries of the matrices are `IntegerMod` elements, defined modulo `p^(self.absprec() / e)`.

Raises an error if `self` has negative valuation.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = (3+w)^7
sage: a.matrix_mod_pn()                                                    #␣
↪needs sage.geometry.polyhedron
[2757  333 1068  725 2510]
[  50 1507  483  318  725]
[ 500   50 3007 2358  318]
[1590 1375 1695 1032 2358]
[2415  590 2370 2970 1032]
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = (Integer(3)+w)**Integer(7)
>>> a.matrix_mod_pn()                                                    #␣
↪needs sage.geometry.polyhedron
[2757  333 1068  725 2510]
[  50 1507  483  318  725]
[ 500   50 3007 2358  318]
[1590 1375 1695 1032 2358]
[2415  590 2370 2970 1032]
```

norm(*base=None*)

Return the absolute or relative norm of this element.

Note

This is not the p-adic absolute value. This is a field theoretic norm down to a ground ring.

If you want the p-adic absolute value, use the `abs()` function instead.

If K is given then K must be a subfield of the parent L of `self`, in which case the norm is the relative norm from L to K. In all other cases, the norm is the absolute norm down to $\mathbf{Q}_p$ or $\mathbf{Z}_p$.

EXAMPLES:

```
sage: R = ZpCR(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: ((1+2*w)^5).norm()
1 + 5^2 + O(5^5)
sage: ((1+2*w)).norm()^5
1 + 5^2 + O(5^5)
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> ((Integer(1)+Integer(2)*w)**Integer(5)).norm()
1 + 5^2 + O(5^5)
>>> ((Integer(1)+Integer(2)*w)).norm()**Integer(5)
1 + 5^2 + O(5^5)
```

polynomial(*var='x'*)

Return a polynomial over the base ring that yields this element when evaluated at the generator of the parent.

INPUT:

- `var` – string, the variable name for the polynomial

EXAMPLES:

```
sage: S.<x> = ZZ[]
sage: W.<w> = ZpFM(5).extension(x^2 - 5)
sage: (w + 5).polynomial()
x + 5
```

```
>>> from sage.all import *
>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> W = ZpFM(Integer(5)).extension(x**Integer(2) - Integer(5), names=('w',));␣
↪(w,) = W._first_ngens(1)
>>> (w + Integer(5)).polynomial()
x + 5
```

precision_absolute()

Return the absolute precision of `self`, ie the precision cap of `self.parent()`.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + 3*w^19 + 2*w^21 + 3*w^22 +␣
↪3*w^23
sage: a.valuation()
10
sage: a.precision_absolute()
25
sage: a.precision_relative()
15
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + 3*w^9 + 2*w^11 + 3*w^12
 + 3*w^13 + w^15 + 4*w^16 + 2*w^17 + w^18 + 3*w^21 + w^22 + 3*w^24
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + 3*w^19 + 2*w^21 + 3*w^22 +␣
↪3*w^23
>>> a.valuation()
10
>>> a.precision_absolute()
25
>>> a.precision_relative()
15
>>> a.unit_part()
```

(continues on next page)

(continued from previous page)

```
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + 3*w^9 + 2*w^11 + 3*w^12
 + 3*w^13 + w^15 + 4*w^16 + 2*w^17 + w^18 + 3*w^21 + w^22 + 3*w^24
```

precision_relative()

Return the relative precision of `self`, ie the precision cap of `self.parent()` minus the `valuation of self`.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + 3*w^19 + 2*w^21 + 3*w^22 +␣
↪3*w^23
sage: a.valuation()
10
sage: a.precision_absolute()
25
sage: a.precision_relative()
15
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + 3*w^9 + 2*w^11 + 3*w^12
 + 3*w^13 + w^15 + 4*w^16 + 2*w^17 + w^18 + 3*w^21 + w^22 + 3*w^24
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + 3*w^19 + 2*w^21 + 3*w^22 +␣
↪3*w^23
>>> a.valuation()
10
>>> a.precision_absolute()
25
>>> a.precision_relative()
15
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + 3*w^9 + 2*w^11 + 3*w^12
 + 3*w^13 + w^15 + 4*w^16 + 2*w^17 + w^18 + 3*w^21 + w^22 + 3*w^24
```

teichmuller_expansion(*n=None*)

Return a list $[a_0, a_1, \ldots, a_n]$ such that.

- $a_i^q = a_i$
- `self.unit_part()` = $\sum_{i=0}^{n} a_i \pi^i$, where π is a uniformizer of `self.parent()`

INPUT:

- `n` – integer (default: `None`); if given, returns the corresponding entry in the expansion

EXAMPLES:

```
sage: R.<a> = ZqFM(5^4,4)
sage: E = a.teichmuller_expansion(); E
5-adic expansion of a (teichmuller)
sage: list(E)
[a + (2*a^3 + 2*a^2 + 3*a + 4)*5 + (4*a^3 + 3*a^2 + 3*a + 2)*5^2 + (4*a^2 +␣
↪2*a + 2)*5^3,
 (3*a^3 + 3*a^2 + 2*a + 1) + (a^3 + 4*a^2 + 1)*5 + (a^2 + 4*a + 4)*5^2 + (4*a^
↪2 + a + 3)*5^3,
 (4*a^3 + 2*a^2 + a + 1) + (2*a^3 + 2*a^2 + 2*a + 4)*5 + (3*a^3 + 2*a^2 + a +␣
↪1)*5^2 + (a^3 + a^2 + 2)*5^3,
 (a^3 + a^2 + a + 4) + (3*a^3 + 1)*5 + (3*a^3 + a + 2)*5^2 + (3*a^3 + 3*a^2 +␣
↪3*a + 1)*5^3]
sage: sum([c * 5^i for i, c in enumerate(E)])
a
sage: all(c^625 == c for c in E)
True

sage: S.<x> = ZZ[]
sage: f = x^3 - 98*x + 7
sage: W.<w> = ZpFM(7,3).ext(f)
sage: b = (1+w)^5; L = b.teichmuller_expansion(); L
[1,
 5 + 5*w^3 + w^6 + 4*w^7,
 3 + 3*w^3 + w^7,
 3 + 3*w^3 + w^7,
 0,
 4 + 5*w^3 + w^6 + 4*w^7,
 3 + 3*w^3 + w^7,
 6 + w^3 + 5*w^7,
 6 + w^3 + 5*w^7]
sage: sum([w^i*L[i] for i in range(len(L))]) == b
True
sage: all(L[i]^(7^3) == L[i] for i in range(9))
True

sage: L = W(3).teichmuller_expansion(); L
[3 + 3*w^3 + w^7,
 0,
 0,
 4 + 5*w^3 + w^6 + 4*w^7,
 0,
 0,
 3 + 3*w^3 + w^7,
 6 + w^3 + 5*w^7]
sage: sum([w^i*L[i] for i in range(len(L))])
3
```

```
>>> from sage.all import *
>>> R = ZqFM(Integer(5)**Integer(4),Integer(4), names=('a',)); (a,) = R._
↪first_ngens(1)
>>> E = a.teichmuller_expansion(); E
```

(continues on next page)

(continued from previous page)

```
5-adic expansion of a (teichmuller)
>>> list(E)
[a + (2*a^3 + 2*a^2 + 3*a + 4)*5 + (4*a^3 + 3*a^2 + 3*a + 2)*5^2 + (4*a^2 +␣
↪2*a + 2)*5^3,
 (3*a^3 + 3*a^2 + 2*a + 1) + (a^3 + 4*a^2 + 1)*5 + (a^2 + 4*a + 4)*5^2 + (4*a^
↪2 + a + 3)*5^3,
 (4*a^3 + 2*a^2 + a + 1) + (2*a^3 + 2*a^2 + 2*a + 4)*5 + (3*a^3 + 2*a^2 + a +␣
↪1)*5^2 + (a^3 + a^2 + 2)*5^3,
 (a^3 + a^2 + a + 4) + (3*a^3 + 1)*5 + (3*a^3 + a + 2)*5^2 + (3*a^3 + 3*a^2 +␣
↪3*a + 1)*5^3]
>>> sum([c * Integer(5)**i for i, c in enumerate(E)])
a
>>> all(c**Integer(625) == c for c in E)
True

>>> S = ZZ['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(3) - Integer(98)*x + Integer(7)
>>> W = ZpFM(Integer(7),Integer(3)).ext(f, names=('w',)); (w,) = W._first_
↪ngens(1)
>>> b = (Integer(1)+w)**Integer(5); L = b.teichmuller_expansion(); L
[1,
 5 + 5*w^3 + w^6 + 4*w^7,
 3 + 3*w^3 + w^7,
 3 + 3*w^3 + w^7,
 0,
 4 + 5*w^3 + w^6 + 4*w^7,
 3 + 3*w^3 + w^7,
 6 + w^3 + 5*w^7,
 6 + w^3 + 5*w^7]
>>> sum([w**i*L[i] for i in range(len(L))]) == b
True
>>> all(L[i]**(Integer(7)**Integer(3)) == L[i] for i in range(Integer(9)))
True

>>> L = W(Integer(3)).teichmuller_expansion(); L
[3 + 3*w^3 + w^7,
 0,
 0,
 4 + 5*w^3 + w^6 + 4*w^7,
 0,
 0,
 3 + 3*w^3 + w^7,
 6 + w^3 + 5*w^7]
>>> sum([w**i*L[i] for i in range(len(L))])
3
```

trace(*base=None*)

Return the absolute or relative trace of this element.

If K is given then K must be a subfield of the parent L of `self`, in which case the norm is the relative norm from L to K. In all other cases, the norm is the absolute norm down to $\mathbf{Q}_p$ or $\mathbf{Z}_p$.

EXAMPLES:

```
sage: R = ZpCR(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = (2+3*w)^7
sage: b = (6+w^3)^5
sage: a.trace()
3*5 + 2*5^2 + 3*5^3 + 2*5^4 + O(5^5)
sage: a.trace() + b.trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
sage: (a+b).trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
```

```
>>> from sage.all import *
>>> R = ZpCR(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = (Integer(2)+Integer(3)*w)**Integer(7)
>>> b = (Integer(6)+w**Integer(3))**Integer(5)
>>> a.trace()
3*5 + 2*5^2 + 3*5^3 + 2*5^4 + O(5^5)
>>> a.trace() + b.trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
>>> (a+b).trace()
4*5 + 5^2 + 5^3 + 2*5^4 + O(5^5)
```

unit_part()

Return the unit part of `self`, ie `self / uniformizer^(self.valuation())`

> **Warning**
>
> If this element has positive valuation then the unit part is not defined to the full precision of the ring. Asking for the unit part of `ZpFM(5)(0)` will not raise an error, but rather return itself.

EXAMPLES:

```
sage: R = ZpFM(5,5)
sage: S.<x> = R[]
sage: f = x^5 + 75*x^3 - 15*x^2 + 125*x - 5
sage: W.<w> = R.ext(f)
sage: a = W(75); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + 3*w^19 + 2*w^21 + 3*w^22 +␣
↪3*w^23
sage: a.valuation()
10
sage: a.precision_absolute()
25
sage: a.precision_relative()
15
```

(continues on next page)

(continued from previous page)

```
sage: a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + 3*w^9 + 2*w^11 + 3*w^12
 + 3*w^13 + w^15 + 4*w^16 + 2*w^17 + w^18 + 3*w^21 + w^22 + 3*w^24
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(5),Integer(5))
>>> S = R['x']; (x,) = S._first_ngens(1)
>>> f = x**Integer(5) + Integer(75)*x**Integer(3) - Integer(15)*x**Integer(2)␣
↪+ Integer(125)*x - Integer(5)
>>> W = R.ext(f, names=('w',)); (w,) = W._first_ngens(1)
>>> a = W(Integer(75)); a
3*w^10 + 2*w^12 + w^14 + w^16 + w^17 + 3*w^18 + 3*w^19 + 2*w^21 + 3*w^22 +␣
↪3*w^23
>>> a.valuation()
10
>>> a.precision_absolute()
25
>>> a.precision_relative()
15
>>> a.unit_part()
3 + 2*w^2 + w^4 + w^6 + w^7 + 3*w^8 + 3*w^9 + 2*w^11 + 3*w^12
 + 3*w^13 + w^15 + 4*w^16 + 2*w^17 + w^18 + 3*w^21 + w^22 + 3*w^24
```

The unit part inserts nonsense digits if this element has positive valuation:

```
sage: (a-a).unit_part()
0
```

```
>>> from sage.all import *
>>> (a-a).unit_part()
0
```

CHAPTER

TWENTYTWO

POWCOMPUTER

A class for computing and caching powers of the same integer.

This class is designed to be used as a field of p-adic rings and fields. Since elements of p-adic rings and fields need to use powers of p over and over, this class precomputes and stores powers of p. There is no reason that the base has to be prime however.

EXAMPLES:

```
sage: X = PowComputer(3, 4, 10)
sage: X(3)
27
sage: X(10) == 3^10
True
```

```
>>> from sage.all import *
>>> X = PowComputer(Integer(3), Integer(4), Integer(10))
>>> X(Integer(3))
27
>>> X(Integer(10)) == Integer(3)**Integer(10)
True
```

AUTHORS:

- David Roe

sage.rings.padics.pow_computer.**PowComputer**(*m*, *cache_limit*, *prec_cap*, *in_field=False*, *prec_type=None*)

Return a `PowComputer` that caches the values $1, m, m^2, \ldots, m^C$, where C is `cache_limit`.

Once you create a `PowComputer`, merely call it to get values out.

You can input any integer, even if it's outside of the precomputed range.

INPUT:

- `m` – integer; the base that you want to exponentiate
- `cache_limit` – positive integer; that you want to cache powers up to

EXAMPLES:

```
sage: PC = PowComputer(3, 5, 10)
sage: PC
PowComputer for 3
sage: PC(4)
81
```

(continues on next page)

(continued from previous page)

```
sage: PC(6)
729
sage: PC(-1)
1/3
```

```
>>> from sage.all import *
>>> PC = PowComputer(Integer(3), Integer(5), Integer(10))
>>> PC
PowComputer for 3
>>> PC(Integer(4))
81
>>> PC(Integer(6))
729
>>> PC(-Integer(1))
1/3
```

class sage.rings.padics.pow_computer.**PowComputer_base**

Bases: *PowComputer_class*

Initialization.

class sage.rings.padics.pow_computer.**PowComputer_class**

Bases: SageObject

Initialize `self`.

INPUT:

- `prime` – the prime that is the base of the exponentials stored in this `pow_computer`
- `cache_limit` – how high to cache powers of prime
- `prec_cap` – data stored for p-adic elements using this `pow_computer` (so they have C-level access to fields common to all elements of the same parent)
- `ram_prec_cap` – prec_cap * e
- `in_field` – same idea as prec_cap
- `poly` – same idea as prec_cap
- `shift_seed` – same idea as prec_cap

EXAMPLES:

```
sage: PC = PowComputer(3, 5, 10)
sage: PC.pow_Integer_Integer(2)
9
```

```
>>> from sage.all import *
>>> PC = PowComputer(Integer(3), Integer(5), Integer(10))
>>> PC.pow_Integer_Integer(Integer(2))
9
```

pow_Integer_Integer(n)

Test the `pow_Integer` function.

EXAMPLES:

```
sage: PC = PowComputer(3, 5, 10)
sage: PC.pow_Integer_Integer(4)
81
sage: PC.pow_Integer_Integer(6)
729
sage: PC.pow_Integer_Integer(0)
1
sage: PC.pow_Integer_Integer(10)
59049

sage: PC = PowComputer_ext_maker(3, 5, 10, 20, False, ntl.ZZ_pX([-3,0,1], 3^
↪10), 'big','e',ntl.ZZ_pX([1],3^10))
sage: PC.pow_Integer_Integer(4)
81
sage: PC.pow_Integer_Integer(6)
729
sage: PC.pow_Integer_Integer(0)
1
sage: PC.pow_Integer_Integer(10)
59049
```

```
>>> from sage.all import *
>>> PC = PowComputer(Integer(3), Integer(5), Integer(10))
>>> PC.pow_Integer_Integer(Integer(4))
81
>>> PC.pow_Integer_Integer(Integer(6))
729
>>> PC.pow_Integer_Integer(Integer(0))
1
>>> PC.pow_Integer_Integer(Integer(10))
59049

>>> PC = PowComputer_ext_maker(Integer(3), Integer(5), Integer(10),␣
↪Integer(20), False, ntl.ZZ_pX([-Integer(3),Integer(0),Integer(1)],␣
↪Integer(3)**Integer(10)), 'big','e',ntl.ZZ_pX([Integer(1)],
↪Integer(3)**Integer(10)))
>>> PC.pow_Integer_Integer(Integer(4))
81
>>> PC.pow_Integer_Integer(Integer(6))
729
>>> PC.pow_Integer_Integer(Integer(0))
1
>>> PC.pow_Integer_Integer(Integer(10))
59049
```

CHAPTER

TWENTYTHREE

POWCOMPUTER_EXT

The classes in this file are designed to be attached to p-adic parents and elements for Cython access to properties of the parent.

In addition to storing the defining polynomial (as an NTL polynomial) at different precisions, they also cache powers of p and data to speed right shifting of elements.

The hierarchy of PowComputers splits first at whether it's for a base ring (Qp or Zp) or an extension.

Among the extension classes (those in this file), they are first split by the type of NTL polynomial (ntl_ZZ_pX or ntl_ZZ_pEX), then by the amount and style of caching (see below). Finally, there are subclasses of the ntl_ZZ_pX PowComputers that cache additional information for Eisenstein extensions.

There are three styles of caching:

- FM: caches powers of p up to the cache_limit, only caches the polynomial modulus and the ntl_ZZ_pContext of precision prec_cap.
- small: Requires cache_limit = prec_cap. Caches p^k for every k up to the cache_limit and caches a polynomial modulus and a ntl_ZZ_pContext for each such power of p.
- big: Caches as the small does up to cache_limit and caches prec_cap. Also has a dictionary that caches values above the cache_limit when they are computed (rather than at ring creation time).

AUTHORS:

- David Roe (2008-01-01) initial version

class sage.rings.padics.pow_computer_ext.**PowComputer_ZZ_pX**

Bases: *PowComputer_ext*

polynomial()

Return the polynomial (with coefficient precision prec_cap) associated to this PowComputer.

The polynomial is output as an ntl_ZZ_pX.

EXAMPLES:

```
sage: PC = PowComputer_ext_maker(5, 5, 10, 20, False, ntl.ZZ_pX([-5,0,1],5^
↪10), 'FM', 'e',ntl.ZZ_pX([1],5^10))
sage: PC.polynomial()
[9765620 0 1]
```

```
>>> from sage.all import *
>>> PC = PowComputer_ext_maker(Integer(5), Integer(5), Integer(10),␣
↪Integer(20), False, ntl.ZZ_pX([-Integer(5),Integer(0),Integer(1)],
↪Integer(5)**Integer(10)), 'FM', 'e',ntl.ZZ_pX([Integer(1)],
```

(continues on next page)

(continued from previous page)

```
↪Integer(5)**Integer(10)))
>>> PC.polynomial()
[9765620 0 1]
```

speed_test(*n*, *runs*)

Run a speed test.

INPUT:

- `n` – input to a function to be tested (the function needs to be set in the source code)
- `runs` – the number of runs of that function

OUTPUT:

- The time in seconds that it takes to call the function on `n`, `runs` times.

EXAMPLES:

```
sage: PC = PowComputer_ext_maker(5, 10, 10, 20, False, ntl.ZZ_pX([-5, 0, 1],␣
↪5^10), 'small', 'e',ntl.ZZ_pX([1],5^10))
sage: PC.speed_test(10, 10^6)  # random
0.0090679999999991878
```

```
>>> from sage.all import *
>>> PC = PowComputer_ext_maker(Integer(5), Integer(10), Integer(10),␣
↪Integer(20), False, ntl.ZZ_pX([-Integer(5), Integer(0), Integer(1)],␣
↪Integer(5)**Integer(10)), 'small', 'e',ntl.ZZ_pX([Integer(1)],
↪Integer(5)**Integer(10)))
>>> PC.speed_test(Integer(10), Integer(10)**Integer(6))  # random
0.0090679999999991878
```

class sage.rings.padics.pow_computer_ext.**PowComputer_ZZ_pX_FM**

Bases: *PowComputer_ZZ_pX*

This class only caches a context and modulus for p^prec_cap.

Designed for use with fixed modulus p-adic rings, in Eisenstein and unramified extensions of $\mathbf{Z}_p$.

class sage.rings.padics.pow_computer_ext.**PowComputer_ZZ_pX_FM_Eis**

Bases: *PowComputer_ZZ_pX_FM*

This class computes and stores `low_shifter` and `high_shifter`, which aid in right shifting elements.

class sage.rings.padics.pow_computer_ext.**PowComputer_ZZ_pX_big**

Bases: *PowComputer_ZZ_pX*

This class caches all contexts and moduli between 1 and cache_limit, and also caches for prec_cap. In addition, it stores a dictionary of contexts and moduli of

reset_dictionaries()

Reset the dictionaries. Note that if there are elements lying around that need access to these dictionaries, calling this function and then doing arithmetic with those elements could cause trouble (if the context object gets garbage collected for example. The bugs introduced could be very subtle, because NTL will generate a new context object and use it, but there's the potential for the object to be incompatible with the different context object).

EXAMPLES:

```
sage: A = PowComputer_ext_maker(5, 6, 10, 20, False, ntl.ZZ_pX([-5,0,1],5^10),
↪ 'big','e',ntl.ZZ_pX([1],5^10))
sage: P = A._get_context_test(8)
sage: A._context_dict()
{8: NTL modulus 390625}
sage: A.reset_dictionaries()
sage: A._context_dict()
{}
```

```
>>> from sage.all import *
>>> A = PowComputer_ext_maker(Integer(5), Integer(6), Integer(10),␣
↪Integer(20), False, ntl.ZZ_pX([-Integer(5),Integer(0),Integer(1)],
↪Integer(5)**Integer(10)), 'big','e',ntl.ZZ_pX([Integer(1)],
↪Integer(5)**Integer(10)))
>>> P = A._get_context_test(Integer(8))
>>> A._context_dict()
{8: NTL modulus 390625}
>>> A.reset_dictionaries()
>>> A._context_dict()
{}
```

class sage.rings.padics.pow_computer_ext.**PowComputer_ZZ_pX_big_Eis**

Bases: *PowComputer_ZZ_pX_big*

This class computes and stores `low_shifter` and `high_shifter`, which aid in right shifting elements. These are only stored at maximal precision: in order to get lower precision versions just reduce mod p^n.

class sage.rings.padics.pow_computer_ext.**PowComputer_ZZ_pX_small**

Bases: *PowComputer_ZZ_pX*

This class caches contexts and moduli densely between 1 and cache_limit. It requires cache_limit == prec_cap.

It is intended for use with capped relative and capped absolute rings and fields, in Eisenstein and unramified extensions of the base p-adic fields.

class sage.rings.padics.pow_computer_ext.**PowComputer_ZZ_pX_small_Eis**

Bases: *PowComputer_ZZ_pX_small*

This class computes and stores `low_shifter` and `high_shifter`, which aid in right shifting elements. These are only stored at maximal precision: in order to get lower precision versions just reduce mod p^n.

class sage.rings.padics.pow_computer_ext.**PowComputer_ext**

Bases: *PowComputer_class*

sage.rings.padics.pow_computer_ext.**PowComputer_ext_maker**(*prime*, *cache_limit*, *prec_cap*, *ram_prec_cap*, *in_field*, *poly*, *prec_type='small'*, *ext_type='u'*, *shift_seed=None*)

Return a `PowComputer` that caches the values $1, p, p^2, \ldots, p^C$, where C is `cache_limit`.

Once you create a `PowComputer`, merely call it to get values out. You can input any integer, even if it's outside of the precomputed range.

INPUT:

- `prime` – integer; the base that you want to exponentiate
- `cache_limit` – positive integer that you want to cache powers up to

- `prec_cap` – the cap on precisions of elements. For ramified extensions, p^((prec_cap - 1) // e) will be the largest power of p distinguishable from zero.
- `in_field` – boolean indicating whether this PowComputer is attached to a field or not
- `poly` – an `ntl_ZZ_pX` or `ntl_ZZ_pEX` defining the extension. It should be defined modulo `p^((prec_cap - 1) // e + 1)`.
- `prec_type` – `'FM'`, `'small'`, or `'big'`, defining how caching is done
- `ext_type` – `'u'` = unramified, `'e'` = Eisenstein, `'t'` = two-step
- `shift_seed` – (required only for Eisenstein and two-step) for Eisenstein and two-step extensions, if f = a_n x^n - p a_{n-1} x^{n-1} - … - p a_0 with a_n a unit, then shift_seed should be 1/a_n (a_{n-1} x^{n-1} + … + a_0)

EXAMPLES:

```
sage: PC = PowComputer_ext_maker(5, 10, 10, 20, False, ntl.ZZ_pX([-5, 0, 1], 5^
↪10), 'small','e',ntl.ZZ_pX([1],5^10))
sage: PC
PowComputer_ext for 5, with polynomial [9765620 0 1]
```

```
>>> from sage.all import *
>>> PC = PowComputer_ext_maker(Integer(5), Integer(10), Integer(10), Integer(20),␣
↪False, ntl.ZZ_pX([-Integer(5), Integer(0), Integer(1)],␣
↪Integer(5)**Integer(10)), 'small','e',ntl.ZZ_pX([Integer(1)],
↪Integer(5)**Integer(10)))
>>> PC
PowComputer_ext for 5, with polynomial [9765620 0 1]
```

`sage.rings.padics.pow_computer_ext.`**`ZZ_pX_eis_shift_test`**`(`*_shifter*, *_a*, *_n*, *_finalprec*`)`

Shift `_a` right `_n` x-adic digits, where x is considered modulo the polynomial in `_shifter`.

EXAMPLES:

```
sage: from sage.rings.padics.pow_computer_ext import ZZ_pX_eis_shift_test
sage: A = PowComputer_ext_maker(5, 3, 10, 40, False, ntl.ZZ_pX([-5,75,15,0,1],5^
↪10), 'big', 'e',ntl.ZZ_pX([1,-15,-3],5^10))
sage: ZZ_pX_eis_shift_test(A, [0, 1], 1, 5)
[1]
sage: ZZ_pX_eis_shift_test(A, [0, 0, 1], 1, 5)
[0 1]
sage: ZZ_pX_eis_shift_test(A, [5], 1, 5)
[75 15 0 1]
sage: ZZ_pX_eis_shift_test(A, [1], 1, 5)
[]
sage: ZZ_pX_eis_shift_test(A, [17, 91, 8, -2], 1, 5)
[316 53 3123 3]
sage: ZZ_pX_eis_shift_test(A, [316, 53, 3123, 3], -1, 5)
[15 91 8 3123]
sage: ZZ_pX_eis_shift_test(A, [15, 91, 8, 3123], 1, 5)
[316 53 3123 3]
```

```
>>> from sage.all import *
>>> from sage.rings.padics.pow_computer_ext import ZZ_pX_eis_shift_test
```

(continues on next page)

(continued from previous page)

```
>>> A = PowComputer_ext_maker(Integer(5), Integer(3), Integer(10), Integer(40),␣
↪False, ntl.ZZ_pX([-Integer(5),Integer(75),Integer(15),Integer(0),Integer(1)],
↪Integer(5)**Integer(10)), 'big', 'e',ntl.ZZ_pX([Integer(1),-Integer(15),-
↪Integer(3)],Integer(5)**Integer(10)))
>>> ZZ_pX_eis_shift_test(A, [Integer(0), Integer(1)], Integer(1), Integer(5))
[1]
>>> ZZ_pX_eis_shift_test(A, [Integer(0), Integer(0), Integer(1)], Integer(1),␣
↪Integer(5))
[0 1]
>>> ZZ_pX_eis_shift_test(A, [Integer(5)], Integer(1), Integer(5))
[75 15 0 1]
>>> ZZ_pX_eis_shift_test(A, [Integer(1)], Integer(1), Integer(5))
[]
>>> ZZ_pX_eis_shift_test(A, [Integer(17), Integer(91), Integer(8), -Integer(2)],␣
↪Integer(1), Integer(5))
[316 53 3123 3]
>>> ZZ_pX_eis_shift_test(A, [Integer(316), Integer(53), Integer(3123),␣
↪Integer(3)], -Integer(1), Integer(5))
[15 91 8 3123]
>>> ZZ_pX_eis_shift_test(A, [Integer(15), Integer(91), Integer(8), Integer(3123)],
↪ Integer(1), Integer(5))
[316 53 3123 3]
```

CHAPTER

TWENTYFOUR

p-ADIC PRINTING

This file contains code for printing p-adic elements.

It has been moved here to prevent code duplication and make finding the relevant code easier.

AUTHORS:

- David Roe

sage.rings.padics.padic_printing.**pAdicPrinter**(*ring*, *options={}*)

Create a *pAdicPrinter*.

INPUT:

- `ring` – a p-adic ring or field
- `options` – dictionary, with keys in `'mode'`, `'pos'`, `'ram_name'`, `'unram_name'`, `'var_name'`, `'max_ram_terms'`, `'max_unram_terms'`, `'max_terse_terms'`, `'sep'`, `'alphabet'`; see *pAdicPrinter_class* for the meanings of these keywords.

EXAMPLES:

```
sage: from sage.rings.padics.padic_printing import pAdicPrinter
sage: R = Zp(5)
sage: pAdicPrinter(R, {'sep': '&'})
series printer for 5-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> from sage.rings.padics.padic_printing import pAdicPrinter
>>> R = Zp(Integer(5))
>>> pAdicPrinter(R, {'sep': '&'})
series printer for 5-adic Ring with capped relative precision 20
```

class sage.rings.padics.padic_printing.**pAdicPrinterDefaults**(*mode='series'*, *pos=True*, *max_ram_terms=-1*, *max_unram_terms=-1*, *max_terse_terms=-1*, *sep='|'*, *alphabet=None*)

Bases: `SageObject`

This class stores global defaults for p-adic printing.

allow_negatives(*neg=None*)

Controls whether or not to display a balanced representation.

`neg=None` returns the current value.

EXAMPLES:

```
sage: padic_printing.allow_negatives(True)
sage: padic_printing.allow_negatives()
True
sage: Qp(29)(-1)
-1 + O(29^20)
sage: Qp(29)(-1000)
-14 - 5*29 - 29^2 + O(29^20)
sage: padic_printing.allow_negatives(False)
```

```
>>> from sage.all import *
>>> padic_printing.allow_negatives(True)
>>> padic_printing.allow_negatives()
True
>>> Qp(Integer(29))(-Integer(1))
-1 + O(29^20)
>>> Qp(Integer(29))(-Integer(1000))
-14 - 5*29 - 29^2 + O(29^20)
>>> padic_printing.allow_negatives(False)
```

alphabet (*alphabet=None*)

Controls the alphabet used to translate p-adic digits into strings (so that no separator need be used in `'digits'` or `'digits-unicode'` mode).

`alphabet` should be passed in as a list or tuple.

`alphabet=None` returns the current value.

EXAMPLES:

```
sage: padic_printing.alphabet("abc")
sage: padic_printing.mode('digits-unicode')
sage: repr(Qp(3)(1234))
'…aaaaaaaaaaaaabcaacab'

sage: padic_printing.mode('series')
sage: padic_printing.alphabet(('0','1','2','3','4','5','6','7','8','9','A','B
↪','C','D','E','F','G','H','I','J','K','L','M','N','O','P','Q','R','S','T','U
↪','V','W','X','Y','Z','a','b','c','d','e','f','g','h','i','j','k','l','m','n
↪','o','p','q','r','s','t','u','v','w','x','y','z'))
```

```
>>> from sage.all import *
>>> padic_printing.alphabet("abc")
>>> padic_printing.mode('digits-unicode')
>>> repr(Qp(Integer(3))(Integer(1234)))
'…aaaaaaaaaaaaabcaacab'

>>> padic_printing.mode('series')
>>> padic_printing.alphabet(('0','1','2','3','4','5','6','7','8','9','A','B',
↪'C','D','E','F','G','H','I','J','K','L','M','N','O','P','Q','R','S','T','U',
↪'V','W','X','Y','Z','a','b','c','d','e','f','g','h','i','j','k','l','m','n',
↪'o','p','q','r','s','t','u','v','w','x','y','z'))
```

max_poly_terms (*max=None*)

Controls the number of terms appearing when printing polynomial representations in `'terse'` or `'val-unit'` modes.

`max=None` returns the current value.

`max=-1` encodes 'no limit.'

EXAMPLES:

```
sage: padic_printing.max_poly_terms(3)
sage: padic_printing.max_poly_terms()
3
sage: padic_printing.mode('terse')
sage: Zq(7^5, 5, names='a')([2,3,4])^8                                  #␣
↪needs sage.libs.ntl
2570 + 15808*a + 9018*a^2 + ... + O(7^5)

sage: padic_printing.max_poly_terms(-1)
sage: padic_printing.mode('series')
```

```
>>> from sage.all import *
>>> padic_printing.max_poly_terms(Integer(3))
>>> padic_printing.max_poly_terms()
3
>>> padic_printing.mode('terse')
>>> Zq(Integer(7)**Integer(5), Integer(5), names='a')([Integer(2),Integer(3),
↪Integer(4)])**Integer(8)                                  # needs sage.
↪libs.ntl
2570 + 15808*a + 9018*a^2 + ... + O(7^5)

>>> padic_printing.max_poly_terms(-Integer(1))
>>> padic_printing.mode('series')
```

max_series_terms(*max=None*)

Controls the maximum number of terms shown when printing in `'series'`, `'digits'`, `'digits-unicode'` or `'bars'` mode.

`max=None` returns the current value.

`max=-1` encodes 'no limit.'

EXAMPLES:

```
sage: padic_printing.max_series_terms(2)
sage: padic_printing.max_series_terms()
2
sage: Qp(31)(1000)
8 + 31 + ... + O(31^20)
sage: padic_printing.max_series_terms(-1)
sage: Qp(37)(100000)
26 + 37 + 36*37^2 + 37^3 + O(37^20)
```

```
>>> from sage.all import *
>>> padic_printing.max_series_terms(Integer(2))
>>> padic_printing.max_series_terms()
```

(continues on next page)

(continued from previous page)

```
2
>>> Qp(Integer(31))(Integer(1000))
8 + 31 + ... + O(31^20)
>>> padic_printing.max_series_terms(-Integer(1))
>>> Qp(Integer(37))(Integer(100000))
26 + 37 + 36*37^2 + 37^3 + O(37^20)
```

max_unram_terms(*max=None*)

For rings with non-prime residue fields, controls how many terms appear in the coefficient of each `pi^n` when printing in `'series'` or `'bar'` modes.

`max=None` returns the current value.

`max=-1` encodes 'no limit.'

EXAMPLES:

```
sage: padic_printing.max_unram_terms(2)
sage: padic_printing.max_unram_terms()
2
sage: Zq(5^6, 5, names='a')([1,2,3,-1])^17                                    #␣
↪needs sage.libs.ntl
(3*a^4 + ... + 3) + (a^5 + ... + a)*5 + (3*a^3 + ... + 2)*5^2 + (3*a^5 + ...␣
↪+ 2)*5^3 + (4*a^5 + ... + 4)*5^4 + O(5^5)

sage: padic_printing.max_unram_terms(-1)
```

```
>>> from sage.all import *
>>> padic_printing.max_unram_terms(Integer(2))
>>> padic_printing.max_unram_terms()
2
>>> Zq(Integer(5)**Integer(6), Integer(5), names='a')([Integer(1),Integer(2),
↪Integer(3),-Integer(1)])**Integer(17)                                        #␣
↪needs sage.libs.ntl
(3*a^4 + ... + 3) + (a^5 + ... + a)*5 + (3*a^3 + ... + 2)*5^2 + (3*a^5 + ...␣
↪+ 2)*5^3 + (4*a^5 + ... + 4)*5^4 + O(5^5)

>>> padic_printing.max_unram_terms(-Integer(1))
```

mode(*mode=None*)

Set the default printing mode.

`mode=None` returns the current value.

The allowed values for mode are: `'val-unit'`, `'series'`, `'terse'`, `'digits'`, `'digits-unicode'` and `'bars'`.

EXAMPLES:

```
sage: padic_printing.mode('terse')
sage: padic_printing.mode()
'terse'
sage: Qp(7)(100)
100 + O(7^20)
sage: padic_printing.mode('series')
```

(continues on next page)

(continued from previous page)

```
sage: Qp(11)(100)
1 + 9*11 + O(11^20)
sage: padic_printing.mode('val-unit')
sage: Qp(13)(130)
13 * 10 + O(13^21)
sage: padic_printing.mode('digits')
sage: repr(Qp(17)(100))
'...00000000000000005F'
sage: repr(Qp(17)(1000))
'...000000000000000037E'
sage: padic_printing.mode('digits-unicode')
sage: repr(Qp(17)(100))
'…00000000000000005F'
sage: repr(Qp(17)(1000))
'…000000000000000037E'
sage: padic_printing.mode('bars')
sage: repr(Qp(19)(1000))
'...2|14|12'

sage: padic_printing.mode('series')
```

```
>>> from sage.all import *
>>> padic_printing.mode('terse')
>>> padic_printing.mode()
'terse'
>>> Qp(Integer(7))(Integer(100))
100 + O(7^20)
>>> padic_printing.mode('series')
>>> Qp(Integer(11))(Integer(100))
1 + 9*11 + O(11^20)
>>> padic_printing.mode('val-unit')
>>> Qp(Integer(13))(Integer(130))
13 * 10 + O(13^21)
>>> padic_printing.mode('digits')
>>> repr(Qp(Integer(17))(Integer(100)))
'...00000000000000005F'
>>> repr(Qp(Integer(17))(Integer(1000)))
'...000000000000000037E'
>>> padic_printing.mode('digits-unicode')
>>> repr(Qp(Integer(17))(Integer(100)))
'…00000000000000005F'
>>> repr(Qp(Integer(17))(Integer(1000)))
'…000000000000000037E'
>>> padic_printing.mode('bars')
>>> repr(Qp(Integer(19))(Integer(1000)))
'...2|14|12'

>>> padic_printing.mode('series')
```

sep (*sep=None*)

Controls the separator used in `'bars'` mode.

`sep=None` returns the current value.

EXAMPLES:

```
sage: padic_printing.sep('][')
sage: padic_printing.sep()
']['
sage: padic_printing.mode('bars')
sage: repr(Qp(61)(-1))
'...
↪60][60][60][
```

```
sage: R = Zp(5,5); P = R._printer; a = R(-5); a
4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + O(5^6)
sage: P.repr_gen(a, False, pos=False)
'-5 + O(5^6)'
sage: P.repr_gen(a, False, ram_name='p')
'4*p + 4*p^2 + 4*p^3 + 4*p^4 + 4*p^5 + O(p^6)'
```

```
>>> from sage.all import *
>>> R = Zp(Integer(5),Integer(5)); P = R._printer; a = R(-Integer(5)); a
4*5 + 4*5^2 + 4*5^3 + 4*5^4 + 4*5^5 + O(5^6)
>>> P.repr_gen(a, False, pos=False)
'-5 + O(5^6)'
>>> P.repr_gen(a, False, ram_name='p')
'4*p + 4*p^2 + 4*p^3 + 4*p^4 + 4*p^5 + O(p^6)'
```

richcmp_modes(*other*, *op*)

Return a comparison of the printing modes of `self` and `other`.

Return 0 if and only if all relevant modes are equal (`max_unram_terms` is irrelevant if the ring is totally ramified over the base, for example). This does not check if the rings are equal (to prevent infinite recursion in the comparison functions of p-adic rings), but it does check if the primes are the same (since the prime affects whether `pos` is relevant).

EXAMPLES:

```
sage: R = Qp(7, print_mode='digits', print_pos=True)
sage: S = Qp(7, print_mode='digits', print_pos=False)
sage: R._printer == S._printer
True
sage: R = Qp(7)
sage: S = Qp(7, print_mode='val-unit')
sage: R == S
False
sage: R._printer < S._printer
True
```

```
>>> from sage.all import *
>>> R = Qp(Integer(7), print_mode='digits', print_pos=True)
>>> S = Qp(Integer(7), print_mode='digits', print_pos=False)
>>> R._printer == S._printer
True
>>> R = Qp(Integer(7))
>>> S = Qp(Integer(7), print_mode='val-unit')
>>> R == S
False
>>> R._printer < S._printer
True
```

CHAPTER

TWENTYFIVE

PRECISION ERROR

The errors in this file indicate various styles of precision problems that can go wrong for p-adics and power series.

AUTHORS:

- David Roe

exception sage.rings.padics.precision_error.**PrecisionError**

Bases: ArithmeticError

CHAPTER

TWENTYSIX

LATTICE PRECISION FOR THE PARENTS ZPLC/QPLC AND ZPLF/QPLF

AUTHOR:

- Xavier Caruso (2018-02): initial version

class sage.rings.padics.lattice_precision.**DifferentialPrecisionGeneric**(*p*, *label*)

Bases: SageObject

A generic class for precision objects obtained by automatic differentiation.

INPUT:

- p – a prime number
- label – string; the label of the parents to which the elements belong that are tracked by this precision module

Note

This object is used internally by the parent ring. You should not create instances of this class on your own.

EXAMPLES:

```
sage: R = ZpLC(2, label='init')
sage: R.precision()
Precision lattice on 0 objects (label: init)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='init')
>>> R.precision()
Precision lattice on 0 objects (label: init)
```

ambient_dimension()

Return the dimension of the vector space in which the precision module/lattice lives.

EXAMPLES:

```
sage: R = ZpLC(2, label='ambient_dim')
sage: prec = R.precision()

sage: x, y = R(1, 10), R(1, 5)
sage: prec.ambient_dimension()
2
sage: prec.dimension()
```

(continues on next page)

(continued from previous page)

```
2

sage: u = x + y
sage: prec.ambient_dimension()
3
sage: prec.dimension()
3
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='ambient_dim')
>>> prec = R.precision()

>>> x, y = R(Integer(1), Integer(10)), R(Integer(1), Integer(5))
>>> prec.ambient_dimension()
2
>>> prec.dimension()
2

>>> u = x + y
>>> prec.ambient_dimension()
3
>>> prec.dimension()
3
```

In the case of `ZpLC` (lattice-cap precision), it is always equal to the dimension of the lattice.

In the case of `ZpLF` (lattice-float precision), the precision object is not necessarily a lattice and then may have smaller dimension:

```
sage: R = ZpLF(2, label='ambient_dim')
sage: prec = R.precision()

sage: x, y = R(1, 10), R(1, 5)
sage: prec.ambient_dimension()
2
sage: prec.dimension()
2

sage: u = x + y
sage: prec.ambient_dimension()
3
sage: prec.dimension()
2
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(2), label='ambient_dim')
>>> prec = R.precision()

>>> x, y = R(Integer(1), Integer(10)), R(Integer(1), Integer(5))
>>> prec.ambient_dimension()
2
>>> prec.dimension()
```

(continues on next page)

(continued from previous page)

```
2

>>> u = x + y
>>> prec.ambient_dimension()
3
>>> prec.dimension()
2
```

del_elements(*threshold=None*)

Delete (or mark for future deletion) the columns of precision matrix corresponding to elements that were collected by the garbage collector.

INPUT:

- `threshold` – integer or `None` (default: `None`); a column whose distance to the right is greater than the threshold is not erased but marked for deletion. If `None`, always erase (never mark for deletion).

EXAMPLES:

```
sage: R = ZpLC(2, label='del_elements')
sage: prec = R.precision()

sage: x = R(1, 10)
sage: prec
Precision lattice on 1 object (label: del_elements)
sage: prec.precision_lattice()                                                  #␣
↪needs sage.geometry.polyhedron
[1024]

sage: del x
sage: prec
Precision lattice on 1 object (label: del_elements)
sage: prec.precision_lattice()                                                  #␣
↪needs sage.geometry.polyhedron
[1024]

sage: prec.del_elements()
sage: prec
Precision lattice on 0 objects (label: del_elements)
sage: prec.precision_lattice()                                                  #␣
↪needs sage.geometry.polyhedron
[]
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='del_elements')
>>> prec = R.precision()

>>> x = R(Integer(1), Integer(10))
>>> prec
Precision lattice on 1 object (label: del_elements)
>>> prec.precision_lattice()                                                    #␣
↪needs sage.geometry.polyhedron
[1024]
```

(continues on next page)

(continued from previous page)

```
>>> del x
>>> prec
Precision lattice on 1 object (label: del_elements)
>>> prec.precision_lattice()                                                #␣
↪needs sage.geometry.polyhedron
[1024]

>>> prec.del_elements()
>>> prec
Precision lattice on 0 objects (label: del_elements)
>>> prec.precision_lattice()                                                #␣
↪needs sage.geometry.polyhedron
[]
```

diffused_digits(*elements=None*)

Return the number of diffused digits of precision within a subset of elements.

A diffused digit of precision is a known digit which is not located on a single variable but only appears on a suitable linear combination of variables.

The number of diffused digits of precision quantifies the quality of the approximation of the lattice precision by a jagged precision (that is a precision which is split over all variables).

We refer to [CRV2018] for a detail exposition of the notion of diffused digits.

EXAMPLES:

```
sage: R = ZpLC(2)
sage: prec = R.precision()
sage: x = R(1, 10); y = R(1, 5)
sage: u = x + y
sage: v = x - y

sage: prec.diffused_digits([x, y])                                         #␣
↪needs sage.geometry.polyhedron
0
sage: prec.diffused_digits([u, v])                                         #␣
↪needs sage.geometry.polyhedron
6
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2))
>>> prec = R.precision()
>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> u = x + y
>>> v = x - y

>>> prec.diffused_digits([x, y])                                           #␣
↪needs sage.geometry.polyhedron
0
>>> prec.diffused_digits([u, v])                                           #␣
↪needs sage.geometry.polyhedron
6
```

The elements u and v are known at absolute precision $O(2^5)$. However, the sum $u + v = 2x$ is known at precision $O(2^1 1)$, that is with 6 more digits. That is where the 6 diffused digits of precision comes from.

Here is another example with matrices:

```
sage: M = matrix(R, 2, 2, [R(3, 5), R(7, 5), R(1, 5), R(11, 1)])                #␣
↪needs sage.modules
sage: N = M^10                                                                  #␣
↪needs sage.modules
```

```
>>> from sage.all import *
>>> M = matrix(R, Integer(2), Integer(2), [R(Integer(3), Integer(5)),␣
↪R(Integer(7), Integer(5)), R(Integer(1), Integer(5)), R(Integer(11),␣
↪Integer(1))])             # needs sage.modules
>>> N = M**Integer(10)                                                          ␣
↪       # needs sage.modules
```

The next syntax provides as easy way to select an interesting subset of variables (the selected subset consists of the four entries of the matrix `N`):

```
sage: prec.diffused_digits(N)                                                   #␣
↪needs sage.geometry.polyhedron sage.modules
17
```

```
>>> from sage.all import *
>>> prec.diffused_digits(N)                                                     #␣
↪needs sage.geometry.polyhedron sage.modules
17
```

Note that, in some cases, the number of diffused digits can be infinite:

```
sage: R = ZpLF(2)
sage: prec = R.precision()
sage: x = R(1, 10)
sage: y = x
sage: prec.diffused_digits([x, y])                                              #␣
↪needs sage.geometry.polyhedron
+Infinity
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(2))
>>> prec = R.precision()
>>> x = R(Integer(1), Integer(10))
>>> y = x
>>> prec.diffused_digits([x, y])                                                #␣
↪needs sage.geometry.polyhedron
+Infinity
```

dimension()

Return the dimension of this precision module.

EXAMPLES:

```
sage: R = ZpLC(5, label='dim')
sage: prec = R.precision()
sage: prec.dimension()
0

sage: x = R(1, 10)
sage: prec.dimension()
1
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(5), label='dim')
>>> prec = R.precision()
>>> prec.dimension()
0

>>> x = R(Integer(1), Integer(10))
>>> prec.dimension()
1
```

history(*compact=True*, *separate_reduce=False*, *timings=True*, *output_type='asciiart'*)

Show history.

The history records creations and deletions of elements attached to this precision lattice, together with many timings.

INPUT:

- `compact` – boolean (default: `True`); if `True`, all consecutive operations of the same type appear on a single row
- `separate_reduce` – boolean (default: `False`); specify whether partial/full Hermite reduction should be displayed separately
- `timings` – boolean (default: `True`); specify whether timings should be displayed
- `output_type` – only `asciiart` is implemented for now

IMPORTANT NOTE:

History is disabled by default. It should then be enabled (through a call to the method *history_enable()*) before use.

EXAMPLES:

```
sage: R = ZpLC(2, label='history_en')
sage: prec = R.precision()
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='history_en')
>>> prec = R.precision()
```

We first enable history:

```
sage: prec.history_enable()
```

```
>>> from sage.all import *
>>> prec.history_enable()
```

At the beginning, the history is of course empty:

```
sage: print(prec.history())
 Timings
   ---
```

```
>>> from sage.all import *
>>> print(prec.history())
 Timings
   ---
```

Now we start creating and deleting elements:

```
sage: L = [R.random_element() for _ in range(20)]
sage: for p in range(20):
....:     if is_prime(p): L[p] = None
sage: prec.del_elements()

sage: print(prec.history())  # somewhat random
 Timings
0.001108s  oooooooooooooooooo
0.000009s  oo~~o~o~ooo~o~ooo~o~
0.014250s  ooooooooooo
```

```
>>> from sage.all import *
>>> L = [R.random_element() for _ in range(Integer(20))]
>>> for p in range(Integer(20)):
...     if is_prime(p): L[p] = None
>>> prec.del_elements()

>>> print(prec.history())  # somewhat random
 Timings
0.001108s  oooooooooooooooooo
0.000009s  oo~~o~o~ooo~o~ooo~o~
0.014250s  ooooooooooo
```

The legend is the following:

- the symbol o represents a tracked element,
- the symbol ~ represents an element which is marked for deletion.

On the history, we see:

- 1st line: twenty new elements were created (this corresponds to the affectation of the list `L`);
- 2nd line: elements at prime positions were marked for deletion (this corresponds to the `for` loop);
- 3rd line: the above elements are indeed deleted (this corresponds to the call of the method *del_elements()*.

Here are some variants:

```
sage: print(prec.history(timings=False))
oooooooooooooooooo
oo~~o~o~ooo~o~ooo~o~
ooooooooooo
```

(continues on next page)

(continued from previous page)

```
sage: print(prec.history(separate_reduce=True))  # somewhat random
 Timings
0.001063s  oooooooooooooooooo
0.000014s  oo~~o~o~ooo~o~ooo~o~
0.000798s  oo~~o~o~ooo~ooooo
0.000233s  oo~~o~o~ooo~orrrr
0.000824s  oo~~o~o~oooooooo
0.000375s  oo~~o~o~ooorrrrr
0.001724s  oo~~o~oooooooo
0.001020s  oo~~o~orrrrrrrr
0.001989s  oo~~ooooooooo
0.001303s  oo~~orrrrrrrrr
0.002352s  oo~ooooooooo
0.001632s  oo~rrrrrrrrr
0.002265s  ooooooooooo
0.001630s  oorrrrrrrrrr
   ---     ooooooooooo
```

```
>>> from sage.all import *
>>> print(prec.history(timings=False))
ooooooooooooooooooo
oo~~o~o~ooo~o~ooo~o~
ooooooooooo

>>> print(prec.history(separate_reduce=True))  # somewhat random
 Timings
0.001063s  ooooooooooooooooooo
0.000014s  oo~~o~o~ooo~o~ooo~o~
0.000798s  oo~~o~o~ooo~ooooo
0.000233s  oo~~o~o~ooo~orrrr
0.000824s  oo~~o~o~oooooooo
0.000375s  oo~~o~o~ooorrrrr
0.001724s  oo~~o~oooooooo
0.001020s  oo~~o~orrrrrrrr
0.001989s  oo~~ooooooooo
0.001303s  oo~~orrrrrrrrr
0.002352s  oo~ooooooooo
0.001632s  oo~rrrrrrrrr
0.002265s  ooooooooooo
0.001630s  oorrrrrrrrrr
   ---     ooooooooooo
```

The symbol `r` represents a column of the precision matrix which is currently under partial Hermite reduction.

Timings for automatic reduction do not appear because they are included in the timings for deletion.

The symbol `R` is used to symbolize a column which is under full Hermite reduction. Note that full Hermite reduction are never performed automatically but needs to be called by hand:

```
sage: prec.reduce()
sage: print(prec.history(separate_reduce=True))  # somewhat random
 Timings
```

(continues on next page)

(continued from previous page)

```
0.001063s  oooooooooooooooooo
0.000014s  oo~~o~o~ooo~o~ooo~o~
0.000798s  oo~~o~o~ooo~ooooo
0.000233s  oo~~o~o~ooo~orrrr
0.000824s  oo~~o~o~oooooooo
0.000375s  oo~~o~o~ooorrrrr
0.001724s  oo~~o~ooooooooo
0.001020s  oo~~o~orrrrrrrr
0.001989s  oo~~ooooooooo
0.001303s  oo~~orrrrrrrrr
0.002352s  oo~ooooooooo
0.001632s  oo~rrrrrrrrr
0.002265s  ooooooooooo
0.001630s  oorrrrrrrrrr
0.001486s  RRRRRRRRRRR
   ---     ooooooooooo
```

```
>>> from sage.all import *
>>> prec.reduce()
>>> print(prec.history(separate_reduce=True))  # somewhat random
 Timings
0.001063s  oooooooooooooooooo
0.000014s  oo~~o~o~ooo~o~ooo~o~
0.000798s  oo~~o~o~ooo~ooooo
0.000233s  oo~~o~o~ooo~orrrr
0.000824s  oo~~o~o~oooooooo
0.000375s  oo~~o~o~ooorrrrr
0.001724s  oo~~o~ooooooooo
0.001020s  oo~~o~orrrrrrrr
0.001989s  oo~~ooooooooo
0.001303s  oo~~orrrrrrrrr
0.002352s  oo~ooooooooo
0.001632s  oo~rrrrrrrrr
0.002265s  ooooooooooo
0.001630s  oorrrrrrrrrr
0.001486s  RRRRRRRRRRR
   ---     ooooooooooo
```

Here is a more common example with matrices:

```
sage: R = ZpLC(3)
sage: prec = R.precision()
sage: prec.history_enable()
sage: M = random_matrix(R, 5)                                                  #␣
↪needs sage.geometry.polyhedron
sage: d = M.determinant()                                                      #␣
↪needs sage.geometry.polyhedron
sage: print(prec.history())  # somewhat random
   ---
0.004212s  oooooooooooooooooooooooooooooooooo
0.000003s  oooooooooooooooooooooooooooooooo~~
0.000010s  oooooooooooooooooooooooooooooooo
```

(continues on next page)

(continued from previous page)

```
0.001560s  oooooooooooooooooooooooooooooooooooooooo
0.000004s  oooooooooooooooooooooooooooo~oooo~oooo~o
0.002168s  oooooooooooooooooooooooooooooooooooooo
0.001787s  oooooooooooooooooooooooooooooooooooooooo
0.000004s  oooooooooooooooooooooooooooooooooooooo~~o
0.000198s  oooooooooooooooooooooooooooooooooooooo
0.001152s  oooooooooooooooooooooooooooooooooooooooo
0.000005s  oooooooooooooooooooooooooooooo~oooo~~o
0.000853s  oooooooooooooooooooooooooooooooooooo
0.000610s  oooooooooooooooooooooooooooooooooooooo
 [...]
0.003879s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000006s  ooooooooooooooooooooooooooooooooooooooooooooooooooooo~~~~~
0.000036s  oooooooooooooooooooooooooooooooooooooooooooooooooooo
0.006737s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000005s  ooooooooooooooooooooooooooooooooooooooooooooooooooo~~~~~ooooo
0.002637s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.007118s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000008s  ooooooooooooooooooooooooooooooooooooooooooooooooooooo~~~~o~~~~oooo
0.003504s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.005371s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000006s  ooooooooooooooooooooooooooooooooooooooooooooooooooo~~~o~~~ooo
0.001858s  ooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.003584s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000004s  ooooooooooooooooooooooooooooooooooooooooooooooooooo~~o~~oo
0.000801s  ooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.001916s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000022s  oooooooooooooooooooooooooo~~~~~~~~~~~~~~~~~~~~~~oooo~o~o
0.014705s  oooooooooooooooooooooooooooooooo
0.001292s  ooooooooooooooooooooooooooooooooo
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(3))
>>> prec = R.precision()
>>> prec.history_enable()
>>> M = random_matrix(R, Integer(5))
↪     # needs sage.geometry.polyhedron
>>> d = M.determinant()
↪needs sage.geometry.polyhedron
>>> print(prec.history())  # somewhat random
   ---
0.004212s  oooooooooooooooooooooooooooooooo
0.000003s  oooooooooooooooooooooooooooooooo~~
0.000010s  oooooooooooooooooooooooooooooo
0.001560s  oooooooooooooooooooooooooooooooooooooooo
0.000004s  oooooooooooooooooooooooooooo~oooo~oooo~o
0.002168s  oooooooooooooooooooooooooooooooooooooo
0.001787s  oooooooooooooooooooooooooooooooooooooooo
0.000004s  oooooooooooooooooooooooooooooooooooooo~~o
0.000198s  oooooooooooooooooooooooooooooooooooooo
0.001152s  oooooooooooooooooooooooooooooooooooooooo
0.000005s  oooooooooooooooooooooooooooooo~oooo~~o
```

(continues on next page)

(continued from previous page)

```
0.000853s  oooooooooooooooooooooooooooooooooo
0.000610s  ooooooooooooooooooooooooooooooooooo
 [...]
0.003879s  oooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000006s  oooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo~~~~~
0.000036s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.006737s  oooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000005s  oooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo~~~~~ooooo
0.002637s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.007118s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000008s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo~~~~o~~~~oooo
0.003504s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.005371s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000006s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo~~~o~~~ooo
0.001858s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.003584s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000004s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo~~o~~oo
0.000801s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.001916s  ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
0.000022s  oooooooooooooooooooooooooooo~~~~~~~~~~~~~~~~~~~~~~~~ooooo~o~o
0.014705s  oooooooooooooooooooooooooooooooooo
0.001292s  oooooooooooooooooooooooooooooooooo
```

We observe that deleted variables appear mostly on the right. This is the so-called principal of temporal locality.

See also

history_enable(), *history_disable()*, *history_clear()*

history_clear()

Clear history.

We refer to the documentation of the method *history()* for a complete documentation (including examples) about history.

See also

history(), *history_enable()*, *history_disable()*

history_disable()

Disable history.

We refer to the documentation of the method *history()* for a complete documentation (including examples) about history.

See also

history(), *history_enable()*, *history_clear()*

history_enable()

Enable history.

We refer to the documentation of the method *history()* for a complete documentation (including examples) about history.

See also

history(), *history_disable()*, *history_clear()*

precision_lattice(*elements=None*)

Return a lattice modeling the precision on the given set of elements or, if not given, on the whole set of elements tracked by the precision module.

INPUT:

- `elements` – list of elements or `None` (default: `None`)

EXAMPLES:

```
sage: R = ZpLC(2, label='precision_lattice')
sage: prec = R.precision()
sage: x = R(1, 10); y = R(1, 5)
sage: u = x + y
sage: v = x - y
sage: prec.precision_lattice()                                              #␣
↪needs sage.geometry.polyhedron
[         1024             0          1024          1024]
[            0            32            32 1099511627744]
[            0             0       2097152             0]
[            0             0             0 1099511627776]
sage: prec.precision_lattice([u, v])                                        #␣
↪needs sage.geometry.polyhedron
[  32 2016]
[   0 2048]
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='precision_lattice')
>>> prec = R.precision()
>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> u = x + y
>>> v = x - y
>>> prec.precision_lattice()                                                #␣
↪needs sage.geometry.polyhedron
[         1024             0          1024          1024]
[            0            32            32 1099511627744]
[            0             0       2097152             0]
[            0             0             0 1099511627776]
>>> prec.precision_lattice([u, v])                                          #␣
↪needs sage.geometry.polyhedron
[  32 2016]
[   0 2048]
```

If the precision module does not project to a lattice, an error is raised.

```
sage: R = ZpLF(2, label='precision_lattice')
sage: prec = R.precision()
sage: x = R(1, 10); y = R(1, 5)
sage: u = x + y
sage: v = x - y
sage: prec.precision_lattice([x,y,u,v])                                    #␣
↪needs sage.geometry.polyhedron
Traceback (most recent call last):
...
PrecisionError: the differential is not surjective
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(2), label='precision_lattice')
>>> prec = R.precision()
>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> u = x + y
>>> v = x - y
>>> prec.precision_lattice([x,y,u,v])                                      #␣
↪needs sage.geometry.polyhedron
Traceback (most recent call last):
...
PrecisionError: the differential is not surjective
```

prime()

Return the underlying prime number attached to this precision lattice.

EXAMPLES:

```
sage: R = ZpLC(2, label='mylabel')
sage: R.precision().prime()
2
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='mylabel')
>>> R.precision().prime()
2
```

threshold_deletion(*threshold=None*)

Return (and set) the threshold for column deletion.

When a variable dies, i.e., goes out of scope, the ambient space in which the precision module lives can be reduced (by projection onto the hyperplane defined by the dead variable). This reduction has a cost because it leads to re-echelonization of a part of the matrix that encodes the precision. The size of this part is roughly measured by the number of columns between the last column and the one corresponding to the dead variable.

This threshold returned by this method is the maximal distance until which a column of a dead variable is removed and the matrix re-echelonized. Beyond the threshold, the column of the dead variable is kept in this matrix as if the variable were not destroyed.

INPUT:

- `threshold` – nonnegative integer, `Infinity` or `None` (default: `None`); if not `None` set the threshold to the given value.

> **Note**
>
> Setting the threshold to `0` disables the dimension reduction.
>
> Setting the threshold to `Infinity` forces the dimension reduction after each deletion.

EXAMPLES:

```
sage: R = ZpLC(2, label='threshold_deletion')
sage: prec = R.precision()
sage: prec.threshold_deletion()
50

sage: prec.threshold_deletion(20)
20
sage: prec.threshold_deletion()
20

sage: prec.threshold_deletion(-2)
Traceback (most recent call last):
...
ValueError: The threshold must be a nonnegative integer or Infinity
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='threshold_deletion')
>>> prec = R.precision()
>>> prec.threshold_deletion()
50

>>> prec.threshold_deletion(Integer(20))
20
>>> prec.threshold_deletion()
20

>>> prec.threshold_deletion(-Integer(2))
Traceback (most recent call last):
...
ValueError: The threshold must be a nonnegative integer or Infinity
```

timings(*action=None*)

Return cumulated timings (grouped by actions) since the last time history has been cleared.

INPUT:

- `action` – `None` (default), `'add'`, `'mark'`, `'del'`, `'partial reduce'` or `'full reduce'`; if not `None`, return the cumulated timing corresponding to this action; otherwise, return a dictionary

Here are the meanings of the keywords above:

- `'add'`: time spent in adding new columns to the precision matrix (corresponding to the creation of new elements)
- `'mark'`: time spent in marking elements for deletion
- `'del'`: time spent in deleting columns of the precision matrix and re-echelonizing the matrix
- `'partial reduce'`: time spent in partial Hermite reduction

- `'full reduce'`: time spent in full Hermite reduction.

EXAMPLES:

```
sage: R = ZpLC(2, label='timings')
sage: prec = R.precision()
sage: prec.history_enable()
sage: M = random_matrix(R, 5, 5)                                                #␣
↪needs sage.geometry.polyhedron
sage: N = M^10                                                                  #␣
↪needs sage.geometry.polyhedron
sage: prec.timings()    # somewhat random
{'add': 1.0530245304107666,
 'del': 0.24358701705932617,
 'mark': 0.0013289451599121094,
 'partial reduce': 0.21604204177856445
 'full reduce': 0}
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='timings')
>>> prec = R.precision()
>>> prec.history_enable()
>>> M = random_matrix(R, Integer(5), Integer(5))                                ␣
↪              # needs sage.geometry.polyhedron
>>> N = M**Integer(10)                                                          ␣
↪      # needs sage.geometry.polyhedron
>>> prec.timings()    # somewhat random
{'add': 1.0530245304107666,
 'del': 0.24358701705932617,
 'mark': 0.0013289451599121094,
 'partial reduce': 0.21604204177856445
 'full reduce': 0}
```

tracked_elements(*values=True*, *dead=True*)

Return the list of tracked elements.

INPUT:

- `values` – boolean (default: `True`); if `False`, the method returns a list of weak references on tracked elements instead
- `dead` – boolean (default: `True`); whether dead elements for which the corresponding column is still not erased should be listed or not

EXAMPLES:

```
sage: R = ZpLC(2, label='tracked')
sage: prec = R.precision()
sage: x = R(1, 10); y = R(1, 5)
sage: prec.tracked_elements()
[1 + O(2^10), 1 + O(2^5)]
sage: prec.tracked_elements(values=False)
[WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...]
sage: prec.tracked_elements(values=False, dead=False)
```

(continues on next page)

(continued from previous page)

```
[WeakProxy#...,
 WeakProxy#...]

sage: u = x + y
sage: v = x - y
sage: prec.tracked_elements()
[1 + O(2^10), 1 + O(2^5), 2 + O(2^5), O(2^5)]
sage: prec.tracked_elements(values=False)
[WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...]
sage: del x; del y
sage: prec.tracked_elements()
[None, None, 2 + O(2^5), O(2^5), None]
sage: prec.tracked_elements(values=False)
[WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...]
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='tracked')
>>> prec = R.precision()
>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> prec.tracked_elements()
[1 + O(2^10), 1 + O(2^5)]
>>> prec.tracked_elements(values=False)
[WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...]
>>> prec.tracked_elements(values=False, dead=False)
[WeakProxy#...,
 WeakProxy#...]

>>> u = x + y
>>> v = x - y
>>> prec.tracked_elements()
[1 + O(2^10), 1 + O(2^5), 2 + O(2^5), O(2^5)]
>>> prec.tracked_elements(values=False)
[WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...]
>>> del x; del y
>>> prec.tracked_elements()
[None, None, 2 + O(2^5), O(2^5), None]
>>> prec.tracked_elements(values=False)
[WeakProxy#...,
 WeakProxy#...,
 WeakProxy#...]
```

class sage.rings.padics.lattice_precision.**PrecisionLattice**(*p*, *label*)

Bases: UniqueRepresentation, *DifferentialPrecisionGeneric*

A class for handling precision lattices which are used to track precision in the ZpLC model.

The precision lattice is stored as a triangular matrix whose rows are generators of the lattice.

INPUT:

- p – a prime number
- label – string; the label of the parents to which the elements tracked by this lattice belong

Note

You should not create instances of this class directly. The precision lattice is automatically initialized at the creation of the parent.

EXAMPLES:

```
sage: R = ZpLC(2, label='init')
sage: R.precision()
Precision lattice on 0 objects (label: init)
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='init')
>>> R.precision()
Precision lattice on 0 objects (label: init)
```

del_elements(*threshold=None*)

Erase columns of the lattice precision matrix corresponding to elements which are marked for deletion and echelonize the matrix in order to keep it upper triangular.

INPUT:

- threshold – integer or None (default: None); a column whose distance to the right is greater than the threshold is not erased

EXAMPLES:

```
sage: R = ZpLC(2, label='delelts')
sage: prec = R.precision()

sage: x = R(1, 10)
sage: prec
Precision lattice on 1 object (label: delelts)
sage: prec.precision_lattice()                                              #␣
↪needs sage.geometry.polyhedron
[1024]

sage: del x
sage: prec
Precision lattice on 1 object (label: delelts)
sage: prec.precision_lattice()                                              #␣
↪needs sage.geometry.polyhedron
[1024]
```

(continues on next page)

(continued from previous page)

```
sage: prec.del_elements()
sage: prec
Precision lattice on 0 objects (label: delelts)
sage: prec.precision_lattice()                                          #␣
↪needs sage.geometry.polyhedron
[]
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='delelts')
>>> prec = R.precision()

>>> x = R(Integer(1), Integer(10))
>>> prec
Precision lattice on 1 object (label: delelts)
>>> prec.precision_lattice()                                            #␣
↪needs sage.geometry.polyhedron
[1024]

>>> del x
>>> prec
Precision lattice on 1 object (label: delelts)
>>> prec.precision_lattice()                                            #␣
↪needs sage.geometry.polyhedron
[1024]

>>> prec.del_elements()
>>> prec
Precision lattice on 0 objects (label: delelts)
>>> prec.precision_lattice()                                            #␣
↪needs sage.geometry.polyhedron
[]
```

dimension()

Return the dimension of this lattice.

EXAMPLES:

```
sage: R = ZpLC(5, label='dimension')
sage: prec = R.precision()
sage: prec.dimension()
0

sage: x = R(1, 10)
sage: prec.dimension()
1
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(5), label='dimension')
>>> prec = R.precision()
>>> prec.dimension()
0
```

(continues on next page)

(continued from previous page)

```
>>> x = R(Integer(1), Integer(10))
>>> prec.dimension()
1
```

precision_lattice(*elements=None*)

Return a matrix representing the precision lattice on a subset of elements.

INPUT:

- `elements` – list of elements or `None` (default: `None`)
- `echelon` – boolean (default: `True`); whether the result should be in echelon form

EXAMPLES:

```
sage: R = ZpLC(2, label='preclattice')
sage: prec = R.precision()
sage: x = R(1, 10); y = R(1, 5)
sage: u = x + y
sage: v = x - y
sage: prec.precision_lattice()                                                  #␣
↪needs sage.geometry.polyhedron
[         1024             0          1024          1024]
[            0            32            32 1099511627744]
[            0             0       2097152             0]
[            0             0             0 1099511627776]
sage: prec.precision_lattice([u, v])                                            #␣
↪needs sage.geometry.polyhedron
[  32 2016]
[   0 2048]
```

```
>>> from sage.all import *
>>> R = ZpLC(Integer(2), label='preclattice')
>>> prec = R.precision()
>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> u = x + y
>>> v = x - y
>>> prec.precision_lattice()                                                    #␣
↪needs sage.geometry.polyhedron
[         1024             0          1024          1024]
[            0            32            32 1099511627744]
[            0             0       2097152             0]
[            0             0             0 1099511627776]
>>> prec.precision_lattice([u, v])                                              #␣
↪needs sage.geometry.polyhedron
[  32 2016]
[   0 2048]
```

Here is another example with matrices:

```
sage: M = matrix(R, 2, 2, [R(3, 5), R(7, 5), R(1, 5), R(11, 1)])               #␣
↪needs sage.modules
sage: N = M^10                                                                  #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
sage: prec.precision_lattice()                                                  #␣
↪needs sage.geometry.polyhedron sage.modules
23 x 23 dense matrix over Integer Ring (use the '.str()' method to see the␣
↪entries)
```

```
>>> from sage.all import *
>>> M = matrix(R, Integer(2), Integer(2), [R(Integer(3), Integer(5)),␣
↪R(Integer(7), Integer(5)), R(Integer(1), Integer(5)), R(Integer(11),␣
↪Integer(1))])            # needs sage.modules
>>> N = M**Integer(10)                                                          ␣
↪       # needs sage.modules
>>> prec.precision_lattice()                                                  #␣
↪needs sage.geometry.polyhedron sage.modules
23 x 23 dense matrix over Integer Ring (use the '.str()' method to see the␣
↪entries)
```

The next syntax provides as easy way to select an interesting subset of variables (the selected subset consists of the four entries of the matrix `N`):

```
sage: prec.precision_lattice(N)                                                 #␣
↪needs sage.modules
[  2048    512  28160 230400]
[     0   2048  14336 258048]
[     0      0  65536  65536]
[     0      0      0 262144]
```

```
>>> from sage.all import *
>>> prec.precision_lattice(N)                                                   #␣
↪needs sage.modules
[  2048    512  28160 230400]
[     0   2048  14336 258048]
[     0      0  65536  65536]
[     0      0      0 262144]
```

We can give a list of matrices as well:

```
sage: prec.precision_lattice([M, N])                                            #␣
↪needs sage.modules
[       32         0         0         0 226115584  96788480  52174848 ␣
↪82804736]
[        0        32         0         0  52174848 121765888  11829248 ␣
↪28516352]
[        0         0        32         0  96788480  42762240 121765888␣
↪199614464]
[        0         0         0         2   5175296  12475904   1782272  ␣
↪4045824]
[        0         0         0         0 268435456         0         0       ␣
↪ 0]
[        0         0         0         0         0 268435456         0       ␣
↪ 0]
[        0         0         0         0         0         0 268435456       ␣
```

(continues on next page)

(continued from previous page)

```
↪ 0]
[         0          0          0          0          0          0          0␣
↪268435456]
```

```
>>> from sage.all import *
>>> prec.precision_lattice([M, N])                                          #␣
↪needs sage.modules
[        32          0          0          0 226115584  96788480  52174848 ␣
↪82804736]
[         0         32          0          0  52174848 121765888  11829248 ␣
↪28516352]
[         0          0         32          0  96788480  42762240 121765888␣
↪199614464]
[         0          0          0          2   5175296  12475904   1782272  ␣
↪4045824]
[         0          0          0          0 268435456          0          0       ␣
↪ 0]
[         0          0          0          0          0 268435456          0       ␣
↪ 0]
[         0          0          0          0          0          0 268435456       ␣
↪ 0]
[         0          0          0          0          0          0          0␣
↪268435456]
```

reduce(*index=0*, *partial=False*)

Reduce the size of the entries above the diagonal of the precision matrix.

INPUT:

- `index` – integer; the starting row for which the reduction is performed
- `partial` – boolean (default: `False`); specifying whether a partial or a full Hermite reduction should be performed

NOTE:

The partial reduction has cost $O(m^2)$ where m is the number of rows that need to be reduced (that is the difference between the total number of rows and `index`).

The full Hermite reduction has cost $O(m^3)$.

Note

The software ensures that the precision lattice is always partially reduced. Calling the function manually with the argument `partial=True` should then just do nothing.

class sage.rings.padics.lattice_precision.**PrecisionModule**(*p*, *label*, *prec*)

Bases: `UniqueRepresentation`, *DifferentialPrecisionGeneric*

A class for handling precision modules which are used to track precision in the ZpLF model.

The precision module (which is not necessarily a lattice) is stored as a matrix whose rows are generators.

del_elements(*threshold=None*)

Erase columns of the lattice precision matrix corresponding to elements which were collected by the garbage collector. Then reduce the matrix in order to keep it in echelon form.

INPUT:

- `threshold` – integer or `None` (default: `None`); a non-pivot column whose distance to the right is greater than the threshold is not erased but only marked for future deletion

EXAMPLES:

```
sage: R = ZpLF(2, label='delelts')
sage: prec = R.precision()

sage: x = R(1, 10)
sage: prec
Precision module on 1 object (label: delelts)
sage: prec.precision_lattice()                                          #␣
↪needs sage.geometry.polyhedron
[1024]

sage: del x
sage: prec
Precision module on 1 object (label: delelts)
sage: prec.precision_lattice()                                          #␣
↪needs sage.geometry.polyhedron
[1024]

sage: prec.del_elements()
sage: prec
Precision module on 0 objects (label: delelts)
sage: prec.precision_lattice()                                          #␣
↪needs sage.geometry.polyhedron
[]
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(2), label='delelts')
>>> prec = R.precision()

>>> x = R(Integer(1), Integer(10))
>>> prec
Precision module on 1 object (label: delelts)
>>> prec.precision_lattice()                                            #␣
↪needs sage.geometry.polyhedron
[1024]

>>> del x
>>> prec
Precision module on 1 object (label: delelts)
>>> prec.precision_lattice()                                            #␣
↪needs sage.geometry.polyhedron
[1024]

>>> prec.del_elements()
```

(continues on next page)

(continued from previous page)

```
>>> prec
Precision module on 0 objects (label: delelts)
>>> prec.precision_lattice()                                                    #␣
↪needs sage.geometry.polyhedron
[]
```

dimension()

Return the dimension of this precision module.

EXAMPLES:

In general, the dimension increases by 1 when a new element with a given precision is created:

```
sage: R = ZpLF(2, label='dimension')
sage: prec = R.precision()

sage: prec.dimension()
0
sage: x = R.random_element(prec=10)
sage: prec.dimension()
1
sage: y = R.random_element(prec=10)
sage: prec.dimension()
2
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(2), label='dimension')
>>> prec = R.precision()

>>> prec.dimension()
0
>>> x = R.random_element(prec=Integer(10))
>>> prec.dimension()
1
>>> y = R.random_element(prec=Integer(10))
>>> prec.dimension()
2
```

However in general it does not increase while doing computations:

```
sage: u = x + y
sage: v = x^2 + 3*y + x*y + y^3
sage: prec.dimension()
2
```

```
>>> from sage.all import *
>>> u = x + y
>>> v = x**Integer(2) + Integer(3)*y + x*y + y**Integer(3)
>>> prec.dimension()
2
```

Of course, it may also decrease when a sufficient number of variables are collected:

```
sage: del x, y, u
sage: prec.del_elements()
sage: prec.dimension()
1

sage: del v
sage: prec.del_elements()
sage: prec.dimension()
0
```

```
>>> from sage.all import *
>>> del x, y, u
>>> prec.del_elements()
>>> prec.dimension()
1

>>> del v
>>> prec.del_elements()
>>> prec.dimension()
0
```

internal_prec()

Return the relative precision at which computations is handled internally.

It is slightly greater than the actual precision and increases a bit (at a logarithmic rate) when new elements are created and/or computed.

EXAMPLES:

```
sage: R = ZpLF(5, prec=20, label='internal_prec')
sage: prec = R.precision()

sage: prec.internal_prec()
25

sage: L = [R.random_element() for _ in range(50)]
sage: prec.internal_prec()
28
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(5), prec=Integer(20), label='internal_prec')
>>> prec = R.precision()

>>> prec.internal_prec()
25

>>> L = [R.random_element() for _ in range(Integer(50))]
>>> prec.internal_prec()
28
```

is_lattice()

Return `True` if this precision module is a lattice (i.e. has maximal dimension).

EXAMPLES:

```
sage: R = ZpLF(2, label='is_lattice')
sage: prec = R.precision()

sage: x = R(1, 10)
sage: y = R(1, 5)
sage: prec.is_lattice()
True

sage: u = x + y
sage: prec.is_lattice()
False
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(2), label='is_lattice')
>>> prec = R.precision()

>>> x = R(Integer(1), Integer(10))
>>> y = R(Integer(1), Integer(5))
>>> prec.is_lattice()
True

>>> u = x + y
>>> prec.is_lattice()
False
```

See also

dimension()

precision_lattice(*elements=None*)

Return a matrix representing the precision lattice on a subset of elements.

INPUT:

- `elements` – list of elements or `None` (default: `None`)

EXAMPLES:

```
sage: R = ZpLF(2, label='preclattice')
sage: prec = R.precision()
sage: x = R(1, 10); y = R(1, 5)
sage: prec.precision_lattice()                                              #␣
↪needs sage.geometry.polyhedron
[1024    0]
[   0   32]

sage: u = x + y
sage: v = x - y
sage: prec.precision_lattice([u, v])                                        #␣
↪needs sage.geometry.polyhedron
[  32 2016]
[   0 2048]
```

```
>>> from sage.all import *
>>> R = ZpLF(Integer(2), label='preclattice')
>>> prec = R.precision()
>>> x = R(Integer(1), Integer(10)); y = R(Integer(1), Integer(5))
>>> prec.precision_lattice()                                                #␣
↪needs sage.geometry.polyhedron
[1024    0]
[   0   32]

>>> u = x + y
>>> v = x - y
>>> prec.precision_lattice([u, v])                                          #␣
↪needs sage.geometry.polyhedron
[  32 2016]
[   0 2048]
```

If the precision module does not project to a lattice, an error is raised.

```
sage: prec.precision_lattice([x, y, u, v])                                  #␣
↪needs sage.geometry.polyhedron
Traceback (most recent call last):
...
PrecisionError: the differential is not surjective
```

```
>>> from sage.all import *
>>> prec.precision_lattice([x, y, u, v])                                    #␣
↪needs sage.geometry.polyhedron
Traceback (most recent call last):
...
PrecisionError: the differential is not surjective
```

Here is another example with matrices:

```
sage: M = matrix(R, 2, 2, [R(3, 5), R(7, 5), R(1, 5), R(11, 1)])            #␣
↪needs sage.modules
sage: N = M^10                                                              #␣
↪needs sage.modules
```

```
>>> from sage.all import *
>>> M = matrix(R, Integer(2), Integer(2), [R(Integer(3), Integer(5)),␣
↪R(Integer(7), Integer(5)), R(Integer(1), Integer(5)), R(Integer(11),␣
↪Integer(1))])            # needs sage.modules
>>> N = M**Integer(10)                                                      ␣
↪      # needs sage.modules
```

The next syntax provides as easy way to select an interesting subset of variables (the selected subset consists of the four entries of the matrix `N`):

```
sage: prec.precision_lattice(N)                                             #␣
↪needs sage.geometry.polyhedron sage.modules
[  2048    512  28160 230400]
[     0   2048  14336 258048]
```

(continues on next page)

(continued from previous page)

```
[     0      0  65536  65536]
[     0      0      0 262144]
```

```
>>> from sage.all import *
>>> prec.precision_lattice(N)                                                  #␣
↪needs sage.geometry.polyhedron sage.modules
[  2048    512  28160 230400]
[     0   2048  14336 258048]
[     0      0  65536  65536]
[     0      0      0 262144]
```

sage.rings.padics.lattice_precision.**list_of_padics**(*elements*)

Convert a list of p-adic composed elements (such as polynomials, matrices) to a list of weak references of their p-adic coefficients.

This is a helper function for the method *precision_lattice()*.

class sage.rings.padics.lattice_precision.**pAdicLatticeElementWeakProxy**(*element*, *callback=None*)

Bases: object

The implementations of *DifferentialPrecisionGeneric* hold weak references to pAdicLatticeElement. They are stored in dictionaries, e.g., a dictionary that maps an element to the corresponding column in the precision lattice matrix. However, weak references as implemented by Python are tricky to use as dictionary keys. Their equality depends on the equality of the element they point to (as long as that element is alive) and then on the equality by id. This means that statements such as: `ref in D == ref in D` could be false if the garbage collector kicks in between the two invocations. To prevent very subtle and hardly reproducible bugs, we wrap weak references in a proxy that gives every lattice element a unique increasing id and uses that id for comparisons.

EXAMPLES:

Proxy elements exist only internally and are not usually exposed to the user:

```
sage: from sage.rings.padics.lattice_precision import pAdicLatticeElementWeakProxy
sage: R = ZpLF(2, label='proxy')
sage: p = R(2)
sage: prec = R.precision()
sage: proxy = prec._elements[0]
sage: isinstance(proxy, pAdicLatticeElementWeakProxy)
True
```

```
>>> from sage.all import *
>>> from sage.rings.padics.lattice_precision import pAdicLatticeElementWeakProxy
>>> R = ZpLF(Integer(2), label='proxy')
>>> p = R(Integer(2))
>>> prec = R.precision()
>>> proxy = prec._elements[Integer(0)]
>>> isinstance(proxy, pAdicLatticeElementWeakProxy)
True
```

class sage.rings.padics.lattice_precision.**pRational**(*p*, *x*, *exponent=0*, *valuation=None*)

Bases: object

This class implements rational numbers viewed as elements of Qp. In particular, it provides additional methods which are specific to p-adics (as p-adic valuation).

Only for internal use.

INPUT:

- `p` – a prime number
- `x` – a rational number
- `exponent` – integer (default: 0)
- `valuation` – integer or `None` (default: `None`); the p-adic valuation of this element

If not `None`, this method trusts the given value to the attribute `valuation`.

is_p_power()

Return `True` if this element is a power of p.

is_zero()

Return `True` if this element vanishes.

list(*prec*)

Return the list of the digits of this element (written in radix p) up to position `prec`.

The first zeros are omitted.

normalize()

Normalize this element, i.e. write it as `p^v * u` where `u` is coprime to p.

reduce(*prec*)

Return this element reduced modulo `p^prec`.

INPUT:

- `prec` – integer

reduce_relative(*prec*)

Return this element reduced modulo `p^n` where `n = prec + val(x)`.

INPUT:

- `prec` – nonnegative integer

unit_part()

Return the unit part of this element, that is the part `u` in the writing `u * p^v` with `u` coprime to p.

valuation()

Return the p-adic valuation of this element.

value()

Return this element as a rational number.

xgcd(*other*)

Return the gcd of `self` and `other` together with two elements `u` and `v` such that `u*self + v*other = gcd`.

The `gcd` is normalized so that it is a power of p.

CHAPTER

TWENTYSEVEN

MISCELLANEOUS FUNCTIONS

This file contains several miscellaneous functions used by p-adics.

- `gauss_sum` – compute Gauss sums using the Gross-Koblitz formula.
- `min` – a version of `min` that returns ∞ on empty input.
- `max` – a version of `max` that returns $-\infty$ on empty input.

AUTHORS:

- David Roe
- Adriana Salerno
- Ander Steele
- Kiran Kedlaya (modified gauss_sum 2017/09)

`sage.rings.padics.misc.`**`gauss_sum`**(*a*, *p*, *f*, *prec=20*, *factored=False*, *algorithm='pari'*, *parent=None*)

Return the Gauss sum $g_q(a)$ as a p-adic number.

The Gauss sum $g_q(a)$ is defined by

$$g_q(a) = \sum_{u \in F_q^*} \omega(u)^{-a} \zeta_q^u,$$

where $q = p^f$, ω is the Teichmüller character and ζ_q is some arbitrary choice of primitive q-th root of unity. The computation is adapted from the main theorem in Alain Robert's paper *The Gross-Koblitz formula revisited*, Rend. Sem. Mat. Univ. Padova 105 (2001), 157–170.

Let p be a prime, f a positive integer, $q = p^f$, and π be the unique root of $f(x) = x^{p-1} + p$ congruent to $\zeta_p - 1$ modulo $(\zeta_p - 1)^2$. Let $0 \leq a < q - 1$. Then the Gross-Koblitz formula gives us the value of the Gauss sum $g_q(a)$ as a product of p-adic Gamma functions as follows:

$$g_q(a) = -\pi^s \prod_{0 \leq i < f} \Gamma_p(a^{(i)}/(q-1)),$$

where s is the sum of the digits of a in base p and the $a^{(i)}$ have p-adic expansions obtained from cyclic permutations of that of a.

INPUT:

- `a` – integer
- `p` – prime
- `f` – positive integer
- `prec` – positive integer (default: 20)

- `factored` – boolean (default: `False`)
- `algorithm` – flag passed to p-adic Gamma function (default: `'pari'`)

OUTPUT:

If `factored` is `False`, returns a p-adic number in an Eisenstein extension of $\mathbf{Q}_p$. This number has the form $pi^e * z$ where pi is as above, e is some nonnegative integer, and z is an element of $\mathbf{Z}_p$; if `factored` is `True`, the pair (e, z) is returned instead, and the Eisenstein extension is not formed.

Note

This is based on GP code written by Adriana Salerno.

EXAMPLES:

In this example, we verify that $g_3(0) = -1$:

```
sage: from sage.rings.padics.misc import gauss_sum
sage: -gauss_sum(0, 3, 1)                                                   #␣
↪needs sage.libs.ntl sage.rings.padics
1 + O(pi^40)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.misc import gauss_sum
>>> -gauss_sum(Integer(0), Integer(3), Integer(1))                          ␣
↪                    # needs sage.libs.ntl sage.rings.padics
1 + O(pi^40)
```

Next, we verify that $g_5(a)g_5(-a) = 5(-1)^a$:

```
sage: from sage.rings.padics.misc import gauss_sum
sage: gauss_sum(2,5,1)^2 - 5                                                #␣
↪needs sage.libs.ntl
O(pi^84)
sage: gauss_sum(1,5,1)*gauss_sum(3,5,1) + 5                                 #␣
↪needs sage.libs.ntl
O(pi^84)
```

```
>>> from sage.all import *
>>> from sage.rings.padics.misc import gauss_sum
>>> gauss_sum(Integer(2),Integer(5),Integer(1))**Integer(2) - Integer(5)    ␣
↪                                        # needs sage.libs.ntl
O(pi^84)
>>> gauss_sum(Integer(1),Integer(5),Integer(1))*gauss_sum(Integer(3),Integer(5),
↪Integer(1)) + Integer(5)                                    # needs sage.libs.
↪ntl
O(pi^84)
```

Finally, we compute a non-trivial value:

```
sage: from sage.rings.padics.misc import gauss_sum
sage: gauss_sum(2,13,2)                                                     #␣
↪needs sage.libs.ntl
```

(continues on next page)

(continued from previous page)

```
6*pi^2 + 7*pi^14 + 11*pi^26 + 3*pi^62 + 6*pi^74 + 3*pi^86 + 5*pi^98 +
pi^110 + 7*pi^134 + 9*pi^146 + 4*pi^158 + 6*pi^170 + 4*pi^194 +
pi^206 + 6*pi^218 + 9*pi^230 + O(pi^242)
sage: gauss_sum(2,13,2, prec=5, factored=True)                                  #␣
↪needs sage.rings.padics
(2, 6 + 6*13 + 10*13^2 + O(13^5))
```

```
>>> from sage.all import *
>>> from sage.rings.padics.misc import gauss_sum
>>> gauss_sum(Integer(2),Integer(13),Integer(2))                                ␣
↪                        # needs sage.libs.ntl
6*pi^2 + 7*pi^14 + 11*pi^26 + 3*pi^62 + 6*pi^74 + 3*pi^86 + 5*pi^98 +
pi^110 + 7*pi^134 + 9*pi^146 + 4*pi^158 + 6*pi^170 + 4*pi^194 +
pi^206 + 6*pi^218 + 9*pi^230 + O(pi^242)
>>> gauss_sum(Integer(2),Integer(13),Integer(2), prec=Integer(5), factored=True) ␣
↪                                  # needs sage.rings.padics
(2, 6 + 6*13 + 10*13^2 + O(13^5))
```

See also

- `sage.arith.misc.gauss_sum()` for general finite fields
- `sage.modular.dirichlet.DirichletCharacter.gauss_sum()` for prime finite fields
- `sage.modular.dirichlet.DirichletCharacter.gauss_sum_numerical()` for prime finite fields

`sage.rings.padics.misc.`**`max`**`(*L)`

Return the maximum of the inputs, where the maximum of the empty list is $-\infty$.

EXAMPLES:

```
sage: from sage.rings.padics.misc import max
sage: max()
-Infinity
sage: max(2,3)
3
```

```
>>> from sage.all import *
>>> from sage.rings.padics.misc import max
>>> max()
-Infinity
>>> max(Integer(2),Integer(3))
3
```

`sage.rings.padics.misc.`**`min`**`(*L)`

Return the minimum of the inputs, where the minimum of the empty list is ∞.

EXAMPLES:

```
sage: from sage.rings.padics.misc import min
sage: min()
```

(continues on next page)

(continued from previous page)

```
+Infinity
sage: min(2,3)
2
```

```
>>> from sage.all import *
>>> from sage.rings.padics.misc import min
>>> min()
+Infinity
>>> min(Integer(2),Integer(3))
2
```

sage.rings.padics.misc.**precprint**(*prec_type*, *prec_cap*, *p*)

String describing the precision mode on a p-adic ring or field.

EXAMPLES:

```
sage: from sage.rings.padics.misc import precprint
sage: precprint('capped-rel', 12, 2)
'with capped relative precision 12'
sage: precprint('capped-abs', 11, 3)
'with capped absolute precision 11'
sage: precprint('floating-point', 1234, 5)
'with floating precision 1234'
sage: precprint('fixed-mod', 1, 17)
'of fixed modulus 17^1'
```

```
>>> from sage.all import *
>>> from sage.rings.padics.misc import precprint
>>> precprint('capped-rel', Integer(12), Integer(2))
'with capped relative precision 12'
>>> precprint('capped-abs', Integer(11), Integer(3))
'with capped absolute precision 11'
>>> precprint('floating-point', Integer(1234), Integer(5))
'with floating precision 1234'
>>> precprint('fixed-mod', Integer(1), Integer(17))
'of fixed modulus 17^1'
```

sage.rings.padics.misc.**trim_zeros**(*L*)

Strip trailing zeros/empty lists from a list.

EXAMPLES:

```
sage: from sage.rings.padics.misc import trim_zeros
sage: trim_zeros([1,0,1,0])
[1, 0, 1]
sage: trim_zeros([[1],[],[2],[],[]])
[[1], [], [2]]
sage: trim_zeros([[],[]])
[]
sage: trim_zeros([])
[]
```

```
>>> from sage.all import *
>>> from sage.rings.padics.misc import trim_zeros
>>> trim_zeros([Integer(1),Integer(0),Integer(1),Integer(0)])
[1, 0, 1]
>>> trim_zeros([[Integer(1)],[],[Integer(2)],[],[]])
[[1], [], [2]]
>>> trim_zeros([[],[]])
[]
>>> trim_zeros([])
[]
```

Zeros are also trimmed from nested lists (one deep):

```
sage: trim_zeros([[1,0]])
[[1]]
sage: trim_zeros([[0],[1]])
[[], [1]]
```

```
>>> from sage.all import *
>>> trim_zeros([[Integer(1),Integer(0)]])
[[1]]
>>> trim_zeros([[Integer(0)],[Integer(1)]])
[[], [1]]
```

CHAPTER

TWENTYEIGHT

TOOLS FOR CREATION OF p-ADIC ELEMENTS

The functions in this file are used in creating new p-adic elements.

When creating a p-adic element, the user can specify that the absolute precision be bounded and/or that the relative precision be bounded. Moreover, different p-adic parents impose their own bounds on the relative or absolute precision of their elements. The precision determines to what power of p the defining data will be reduced, but the valuation of the resulting element needs to be determined before the element is created. Moreover, some defining data can impose their own precision bounds on the result.

AUTHORS:

- David Roe (2012-03-01)

CHAPTER

TWENTYNINE

FROBENIUS ENDOMORPHISMS ON p-ADIC FIELDS

class sage.rings.padics.morphism.**FrobeniusEndomorphism_padics**

Bases: RingHomomorphism

A class implementing Frobenius endomorphisms on p-adic fields.

is_identity()

Return True if this morphism is the identity morphism.

EXAMPLES:

```
sage: K.<a> = Qq(5^3)
sage: Frob = K.frobenius_endomorphism()
sage: Frob.is_identity()
False
sage: (Frob^3).is_identity()
True
```

```
>>> from sage.all import *
>>> K = Qq(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> Frob.is_identity()
False
>>> (Frob**Integer(3)).is_identity()
True
```

is_injective()

Return True since any power of the Frobenius endomorphism over an unramified p-adic field is always injective.

EXAMPLES:

```
sage: K.<a> = Qq(5^3)
sage: Frob = K.frobenius_endomorphism()
sage: Frob.is_injective()
True
```

```
>>> from sage.all import *
>>> K = Qq(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> Frob.is_injective()
True
```

is_surjective()

Return `True` since any power of the Frobenius endomorphism over an unramified p-adic field is always surjective.

EXAMPLES:

```
sage: K.<a> = Qq(5^3)
sage: Frob = K.frobenius_endomorphism()
sage: Frob.is_surjective()
True
```

```
>>> from sage.all import *
>>> K = Qq(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> Frob.is_surjective()
True
```

order()

Return the order of this endomorphism.

EXAMPLES:

```
sage: K.<a> = Qq(5^12)
sage: Frob = K.frobenius_endomorphism()
sage: Frob.order()
12
sage: (Frob^2).order()
6
sage: (Frob^9).order()
4
```

```
>>> from sage.all import *
>>> K = Qq(Integer(5)**Integer(12), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> Frob.order()
12
>>> (Frob**Integer(2)).order()
6
>>> (Frob**Integer(9)).order()
4
```

power()

Return the smallest integer n such that this endomorphism is the n-th power of the absolute (arithmetic) Frobenius.

EXAMPLES:

```
sage: K.<a> = Qq(5^12)
sage: Frob = K.frobenius_endomorphism()
sage: Frob.power()
1
sage: (Frob^9).power()
9
sage: (Frob^13).power()
1
```

```
>>> from sage.all import *
>>> K = Qq(Integer(5)**Integer(12), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> Frob.power()
1
>>> (Frob**Integer(9)).power()
9
>>> (Frob**Integer(13)).power()
1
```

CHAPTER

THIRTY

WITT VECTOR RINGS

This module provides the class *WittVectorRing* of rings of truncated Witt vectors.

AUTHORS:

- Jacob Dennerlein (2022-11-28): initial version
- Rubén Muñoz--Bertrand (2025-02-13, 2025-12-23): major refactoring and new features

class sage.rings.padics.witt_vector_ring.**WittVectorFrobeniusMorphism**(*domain*, *truncate=False*)

Bases: RingHomomorphism

A class implementing Frobenius homomorphisms on Witt vector rings.

Warning

This class should not be called directly, use *WittVectorRing.frobenius_morphism()* instead.

EXAMPLES:

```
sage: R.<x,y,z> = PolynomialRing(ZZ)
sage: W = WittVectorRing(R, p=3, prec=3)
sage: w = W([x, y, z])
sage: frob = W.frobenius_morphism()
sage: frob(w)
(x^3 + 3*y, -3*x^6*y - 9*x^3*y^2 - 8*y^3 + 3*z)

sage: W = WittVectorRing(R, p=3, prec=1)
sage: W.frobenius_morphism()
Traceback (most recent call last):
...
ValueError: the ring of Witt vectors must have precision at least 2
when truncate=True
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ, names=('x', 'y', 'z',)); (x, y, z,) = R._first_ngens(3)
>>> W = WittVectorRing(R, p=Integer(3), prec=Integer(3))
>>> w = W([x, y, z])
>>> frob = W.frobenius_morphism()
>>> frob(w)
(x^3 + 3*y, -3*x^6*y - 9*x^3*y^2 - 8*y^3 + 3*z)
```

(continues on next page)

(continued from previous page)

```
>>> W = WittVectorRing(R, p=Integer(3), prec=Integer(1))
>>> W.frobenius_morphism()
Traceback (most recent call last):
...
ValueError: the ring of Witt vectors must have precision at least 2
when truncate=True
```

class sage.rings.padics.witt_vector_ring.**WittVectorRing**(*coefficient_ring*, *prec*, *prime*, *algorithm*)

Bases: Parent, UniqueRepresentation

Return the appropriate p-typical truncated Witt vector ring.

INPUT:

- `coefficient_ring` – commutative ring of coefficients
- `prec` – integer (default: 1), length of the truncated Witt vectors in the ring
- `p` – a prime number (default: `None`); when it is not set, it defaults to the characteristic of `coefficient_ring` when it is prime.
- `algorithm` – the name of the algorithm to use for the ring laws (default: `None`); when it is not set, the most adequate algorithm is chosen

Available algorithms are:

- `standard` – the schoolbook algorithm;
- `finotti` – Finotti's algorithm; it can be used when the coefficient ring has characteristic p;
- `phantom` – computes the ring laws using the phantom components using a lift of `coefficient_ring`, assuming that it is either $\mathbf{F}_q$ for a power q of p, or a polynomial ring on that field;
- `p_invertible` – uses some optimisations when p is invertible in the coefficient ring.

EXAMPLES:

```
sage: WittVectorRing(QQ, p=5)
Ring of truncated 5-typical Witt vectors of length 1 over
Rational Field
```

```
>>> from sage.all import *
>>> WittVectorRing(QQ, p=Integer(5))
Ring of truncated 5-typical Witt vectors of length 1 over
Rational Field
```

```
sage: WittVectorRing(GF(3))
Ring of truncated 3-typical Witt vectors of length 1 over
Finite Field of size 3
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(3)))
Ring of truncated 3-typical Witt vectors of length 1 over
Finite Field of size 3
```

```
sage: WittVectorRing(GF(3)['t'])
Ring of truncated 3-typical Witt vectors of length 1 over
Univariate Polynomial Ring in t over Finite Field of size 3
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(3))['t'])
Ring of truncated 3-typical Witt vectors of length 1 over
Univariate Polynomial Ring in t over Finite Field of size 3
```

```
sage: WittVectorRing(Qp(7), prec=30, p=5)
Ring of truncated 5-typical Witt vectors of length 30 over
7-adic Field with capped relative precision 20
```

```
>>> from sage.all import *
>>> WittVectorRing(Qp(Integer(7)), prec=Integer(30), p=Integer(5))
Ring of truncated 5-typical Witt vectors of length 30 over
7-adic Field with capped relative precision 20
```

```
sage: p = 5
sage: W = WittVectorRing(ZZ, prec=3, p=p)
sage: W
Ring of truncated 5-typical Witt vectors of length 3 over Integer Ring
sage: V = W.verschiebung(extend=True)
sage: WW = V.codomain()
sage: WW
Ring of truncated 5-typical Witt vectors of length 4 over Integer Ring
sage: F = WW.frobenius_morphism()
sage: w = W.random_element(x=-100, y=100)
sage: w  # random
(-13, 20, -33)
sage: p*w == F(V(w))
True
```

```
>>> from sage.all import *
>>> p = Integer(5)
>>> W = WittVectorRing(ZZ, prec=Integer(3), p=p)
>>> W
Ring of truncated 5-typical Witt vectors of length 3 over Integer Ring
>>> V = W.verschiebung(extend=True)
>>> WW = V.codomain()
>>> WW
Ring of truncated 5-typical Witt vectors of length 4 over Integer Ring
>>> F = WW.frobenius_morphism()
>>> w = W.random_element(x=-Integer(100), y=Integer(100))
>>> w  # random
(-13, 20, -33)
>>> p*w == F(V(w))
True
```

```
sage: p = 7
sage: W = WittVectorRing(GF(p)['x,y'], prec=5)
```

(continues on next page)

(continued from previous page)

```
sage: W
Ring of truncated 7-typical Witt vectors of length 5 over Multivariate
Polynomial Ring in x, y over Finite Field of size 7
sage: V = W.verschiebung()
sage: F = W.frobenius_morphism()
sage: w = W.random_element()
sage: w  # random
(-3*x^2 + 2*y^2 - 3*y + 2, -2*x*y - y^2 - y - 2, -x^2 + y^2,
x^2 + 2*y^2 + 3*x + y, 2*x^2 - 3*x*y + y^2 + y + 1)
sage: p*w == F(V(w))
True
sage: p*w == V(F(w))
True
```

```
>>> from sage.all import *
>>> p = Integer(7)
>>> W = WittVectorRing(GF(p)['x,y'], prec=Integer(5))
>>> W
Ring of truncated 7-typical Witt vectors of length 5 over Multivariate
Polynomial Ring in x, y over Finite Field of size 7
>>> V = W.verschiebung()
>>> F = W.frobenius_morphism()
>>> w = W.random_element()
>>> w  # random
(-3*x^2 + 2*y^2 - 3*y + 2, -2*x*y - y^2 - y - 2, -x^2 + y^2,
x^2 + 2*y^2 + 3*x + y, 2*x^2 - 3*x*y + y^2 + y + 1)
>>> p*w == F(V(w))
True
>>> p*w == V(F(w))
True
```

cardinality()

Return the cardinality of `self`.

EXAMPLES:

```
sage: WittVectorRing(GF(17), prec=2).cardinality()
289
sage: WittVectorRing(QQ, p=2).cardinality()
+Infinity
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(17)), prec=Integer(2)).cardinality()
289
>>> WittVectorRing(QQ, p=Integer(2)).cardinality()
+Infinity
```

characteristic()

Return the characteristic of `self`.

EXAMPLES:

```
sage: WittVectorRing(GF(25), p=5, prec=3).characteristic()
125
sage: WittVectorRing(ZZ, p=2, prec=4).characteristic()
0
sage: WittVectorRing(Integers(18), p=3, prec=3).characteristic()
162
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(25)), p=Integer(5), prec=Integer(3)).
↪characteristic()
125
>>> WittVectorRing(ZZ, p=Integer(2), prec=Integer(4)).characteristic()
0
>>> WittVectorRing(Integers(Integer(18)), p=Integer(3), prec=Integer(3)).
↪characteristic()
162
```

coefficient_ring()

Return the coefficient ring of `self`.

EXAMPLES:

```
sage: W = WittVectorRing(Zp(5), p=5)
sage: W.coefficient_ring()
5-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> W = WittVectorRing(Zp(Integer(5)), p=Integer(5))
>>> W.coefficient_ring()
5-adic Ring with capped relative precision 20
```

frobenius_morphism(*truncate=False*)

Return the Frobenius homomorphism of this ring of Witt vectors.

INPUT:

- `truncate` – boolean (default: `False`); when the coefficient ring has positive characteristic p, chose whether to lower the precision of the codomain by 1. This is always the case when the characteristic of the coefficient ring is not p.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ)
sage: W = WittVectorRing(R, p=5, prec=2)
sage: w = W([x, y])
sage: frob = W.frobenius_morphism()
sage: frob(w)
(x^5 + 5*y)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> W = WittVectorRing(R, p=Integer(5), prec=Integer(2))
>>> w = W([x, y])
>>> frob = W.frobenius_morphism()
```

(continues on next page)

(continued from previous page)

```
>>> frob(w)
(x^5 + 5*y)
```

frobenius_polynomials(*variables=None*)

Return the Witt Frobenius polynomials.

INPUT:

- `variables` – names of the indeterminates (default: `None`), given as a string, or as a list of strings, whose length must be the precision of the ring. When nothing is given, variables indexed by X are used.

EXAMPLES:

```
sage: W = WittVectorRing(GF(5), prec=4)
sage: W.frobenius_polynomials()
[X0^5, X1^5, X2^5]

sage: W = WittVectorRing(ZZ, p=2, prec=3)
sage: W.frobenius_polynomials('T0, T1, T2')
[T0^2 + 2*T1, -2*T0^2*T1 - T1^2 + 2*T2]

sage: W = WittVectorRing(ZZ, p=3, prec=3)
sage: W.frobenius_polynomials('T0, T1, T2')
[T0^3 + 3*T1, -3*T0^6*T1 - 9*T0^3*T1^2 - 8*T1^3 + 3*T2]
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(5)), prec=Integer(4))
>>> W.frobenius_polynomials()
[X0^5, X1^5, X2^5]

>>> W = WittVectorRing(ZZ, p=Integer(2), prec=Integer(3))
>>> W.frobenius_polynomials('T0, T1, T2')
[T0^2 + 2*T1, -2*T0^2*T1 - T1^2 + 2*T2]

>>> W = WittVectorRing(ZZ, p=Integer(3), prec=Integer(3))
>>> W.frobenius_polynomials('T0, T1, T2')
[T0^3 + 3*T1, -3*T0^6*T1 - 9*T0^3*T1^2 - 8*T1^3 + 3*T2]
```

gen(*n=0*)

Return the `n`-th generator of `self`.

Warning

This is not the `n`-th generator of `self` as a `self.base_ring()`-algebra.

See also

gens()

EXAMPLES:

```
sage: R.<x, y> = ZZ[]
sage: W = WittVectorRing(R, p=3, prec=3)
sage: W.gen()
(x, 0, 0)
sage: W.gen(1)
(y, 0, 0)
sage: W.gen(n=2)
Traceback (most recent call last):
...
ValueError: Generator not defined.
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> W = WittVectorRing(R, p=Integer(3), prec=Integer(3))
>>> W.gen()
(x, 0, 0)
>>> W.gen(Integer(1))
(y, 0, 0)
>>> W.gen(n=Integer(2))
Traceback (most recent call last):
...
ValueError: Generator not defined.
```

gens()

Return a tuple of generators of `self`.

Warning

These are not the generators of `self` as a `self.base_ring()`-algebra. Rather, this set generates a sub-`self.base_ring()`-algebra A of `self` such that every element of `self` can be written as $\sum_{i=0}^{prec} V^i(a_i)$ where all $a_i \in A$.

EXAMPLES:

```
sage: W = WittVectorRing(GF(29), prec=3)
sage: W.gens()
((1, 0, 0),)

sage: F.<a> = GF(27)
sage: W = WittVectorRing(F, prec=3)
sage: W.gens()
((a, 0, 0),)

sage: R.<x> = ZZ[]
sage: W = WittVectorRing(R, p=7, prec=2)
sage: W.gens()
((x, 0),)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(29)), prec=Integer(3))
>>> W.gens()
```

(continues on next page)

(continued from previous page)

```
((1, 0, 0),)

>>> F = GF(Integer(27), names=('a',)); (a,) = F._first_ngens(1)
>>> W = WittVectorRing(F, prec=Integer(3))
>>> W.gens()
((a, 0, 0),)

>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> W = WittVectorRing(R, p=Integer(7), prec=Integer(2))
>>> W.gens()
((x, 0),)
```

is_exact()

Return whether the elements of `self` are represented exactly.

EXAMPLES:

```
sage: WittVectorRing(ZZ, p=3, prec=3).is_exact()
True
sage: WittVectorRing(Qp(7), p=7, prec=2).is_exact()
False
```

```
>>> from sage.all import *
>>> WittVectorRing(ZZ, p=Integer(3), prec=Integer(3)).is_exact()
True
>>> WittVectorRing(Qp(Integer(7)), p=Integer(7), prec=Integer(2)).is_exact()
False
```

is_field(*proof=True*)

Return `True` if this ring is a field.

INPUT:

- `proof` – boolean (default: `True`); if set to `True`, the returned value is correct but the method might throw an exception. Otherwise, if it is set to `False`, the method returns `True` if it can establish that `self` is a field, and `False` otherwise.

EXAMPLES:

```
sage: WittVectorRing(QQ, p=2).is_field()
True
sage: WittVectorRing(QQ, p=2, prec=2).is_field()
False
sage: WittVectorRing(GF(9, 'a')).is_field()
True
sage: WittVectorRing(GF(9, 'a'), prec=3).is_field()
False
sage: WittVectorRing(ZZ, p=5).is_field()
False
sage: L.<z> = LazyLaurentSeriesRing(ZZ)
sage: W = WittVectorRing(L, p=7, algorithm="standard")
sage: W.is_field(proof=True)
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
NotImplementedError: unable to determine whether or not Lazy
Laurent Series Ring in z over Integer Ring is a field.
sage: W.is_field(proof=False)
False
```

```
>>> from sage.all import *
>>> WittVectorRing(QQ, p=Integer(2)).is_field()
True
>>> WittVectorRing(QQ, p=Integer(2), prec=Integer(2)).is_field()
False
>>> WittVectorRing(GF(Integer(9), 'a')).is_field()
True
>>> WittVectorRing(GF(Integer(9), 'a'), prec=Integer(3)).is_field()
False
>>> WittVectorRing(ZZ, p=Integer(5)).is_field()
False
>>> L = LazyLaurentSeriesRing(ZZ, names=('z',)); (z,) = L._first_ngens(1)
>>> W = WittVectorRing(L, p=Integer(7), algorithm="standard")
>>> W.is_field(proof=True)
Traceback (most recent call last):
...
NotImplementedError: unable to determine whether or not Lazy
Laurent Series Ring in z over Integer Ring is a field.
>>> W.is_field(proof=False)
False
```

is_finite()

Return whether `self` is a finite ring.

EXAMPLES:

```
sage: WittVectorRing(GF(23)).is_finite()
True
sage: WittVectorRing(ZZ, p=2).is_finite()
False
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(23))).is_finite()
True
>>> WittVectorRing(ZZ, p=Integer(2)).is_finite()
False
```

is_integral_domain(*proof=True*)

Return `True` if this ring is an integral domain.

INPUT:

- `proof` – boolean (default: `True`); if set to `True`, the returned value is correct but the method might throw an exception. Otherwise, if it is set to `False`, the method returns `True` if it can establish that `self` is an integral domain, and `False` otherwise.

EXAMPLES:

```
sage: WittVectorRing(ZZ, p=2).is_integral_domain()
True
sage: WittVectorRing(ZZ, p=2, prec=2).is_integral_domain()
False
sage: WittVectorRing(GF(9, 'a')).is_integral_domain()
True
sage: WittVectorRing(GF(9, 'a'), prec=3).is_integral_domain()
False
sage: R.<x> = ZZ[]
sage: S.<y> = R.quo(x^2 + 7*x + 8)
sage: T.<t> = S[]
sage: Q.<z> = T.quo(t^2)
sage: W = WittVectorRing(Q, p=7, algorithm="standard")
sage: W.is_integral_domain(proof=True)
Traceback (most recent call last):
...
NotImplementedError: cannot rewrite Univariate Quotient
Polynomial Ring in y over Integer Ring with modulus
x^2 + 7*x + 8 as an isomorphic ring
sage: W.is_integral_domain(proof=False)
False
```

```
>>> from sage.all import *
>>> WittVectorRing(ZZ, p=Integer(2)).is_integral_domain()
True
>>> WittVectorRing(ZZ, p=Integer(2), prec=Integer(2)).is_integral_domain()
False
>>> WittVectorRing(GF(Integer(9), 'a')).is_integral_domain()
True
>>> WittVectorRing(GF(Integer(9), 'a'), prec=Integer(3)).is_integral_domain()
False
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> S = R.quo(x**Integer(2) + Integer(7)*x + Integer(8), names=('y',)); (y,)␣
↪= S._first_ngens(1)
>>> T = S['t']; (t,) = T._first_ngens(1)
>>> Q = T.quo(t**Integer(2), names=('z',)); (z,) = Q._first_ngens(1)
>>> W = WittVectorRing(Q, p=Integer(7), algorithm="standard")
>>> W.is_integral_domain(proof=True)
Traceback (most recent call last):
...
NotImplementedError: cannot rewrite Univariate Quotient
Polynomial Ring in y over Integer Ring with modulus
x^2 + 7*x + 8 as an isomorphic ring
>>> W.is_integral_domain(proof=False)
False
```

is_integrally_closed()

Return `True` if this ring is an integrally closed domain.

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: K.<a> = NumberField(x^2 + 189*x + 394)
```

(continues on next page)

(continued from previous page)

```
sage: R = K.order(2*a)
sage: WittVectorRing(R, p=79).is_integrally_closed()
False
sage: WittVectorRing(ZZ, p=101).is_integrally_closed()
True
sage: WittVectorRing(ZZ, p=101, prec=2).is_integrally_closed()
False
sage: S.<t> = ZZ[]
sage: WittVectorRing(S, p=101).is_integrally_closed()
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) + Integer(189)*x + Integer(394), names=('a',
↪)); (a,) = K._first_ngens(1)
>>> R = K.order(Integer(2)*a)
>>> WittVectorRing(R, p=Integer(79)).is_integrally_closed()
False
>>> WittVectorRing(ZZ, p=Integer(101)).is_integrally_closed()
True
>>> WittVectorRing(ZZ, p=Integer(101), prec=Integer(2)).is_integrally_closed()
False
>>> S = ZZ['t']; (t,) = S._first_ngens(1)
>>> WittVectorRing(S, p=Integer(101)).is_integrally_closed()
Traceback (most recent call last):
...
NotImplementedError
```

is_prime_field()

Return `True` if `self` is isomorphic to one of the prime fields $\mathbf{Q}$ or $\mathbf{F}_p$.

EXAMPLES:

```
sage: WittVectorRing(QQ, p=2).is_prime_field()
True
sage: WittVectorRing(QQ, p=2, prec=2).is_prime_field()
False
sage: WittVectorRing(GF(9, 'a')).is_prime_field()
False
sage: WittVectorRing(GF(7)).is_prime_field()
True
sage: WittVectorRing(GF(7), prec=3).is_prime_field()
False
```

```
>>> from sage.all import *
>>> WittVectorRing(QQ, p=Integer(2)).is_prime_field()
True
>>> WittVectorRing(QQ, p=Integer(2), prec=Integer(2)).is_prime_field()
False
>>> WittVectorRing(GF(Integer(9), 'a')).is_prime_field()
```

(continues on next page)

(continued from previous page)

```
False
>>> WittVectorRing(GF(Integer(7))).is_prime_field()
True
>>> WittVectorRing(GF(Integer(7)), prec=Integer(3)).is_prime_field()
False
```

ngens()

Return the number of generators of `self`.

Warning

This is not the number of generators of `self` as a `self.base_ring()`-algebra.

See also

gens()

EXAMPLES:

```
sage: W = WittVectorRing(GF(2), p=3, prec=3)
sage: W.ngens()
1
sage: W = WittVectorRing(GF(2)['t'], p=3, prec=3)
sage: W.ngens()
1
sage: W = WittVectorRing(GF(3), p=3, prec=3)
sage: W.ngens()
1
sage: W = WittVectorRing(GF(3)['t'], p=3, prec=3)
sage: W.ngens()
1
sage: W = WittVectorRing(GF(9,'a'), p=3, prec=3)
sage: W.ngens()
1
sage: W = WittVectorRing(GF(9,'a')['t'], p=3, prec=3)
sage: W.ngens()
1
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(2)), p=Integer(3), prec=Integer(3))
>>> W.ngens()
1
>>> W = WittVectorRing(GF(Integer(2))['t'], p=Integer(3), prec=Integer(3))
>>> W.ngens()
1
>>> W = WittVectorRing(GF(Integer(3)), p=Integer(3), prec=Integer(3))
>>> W.ngens()
1
>>> W = WittVectorRing(GF(Integer(3))['t'], p=Integer(3), prec=Integer(3))
```

(continues on next page)

(continued from previous page)

```
>>> W.ngens()
1
>>> W = WittVectorRing(GF(Integer(9),'a'), p=Integer(3), prec=Integer(3))
>>> W.ngens()
1
>>> W = WittVectorRing(GF(Integer(9),'a')['t'], p=Integer(3), prec=Integer(3))
>>> W.ngens()
1
```

precision()

Return the length of the truncated Witt vectors in `self`.

EXAMPLES:

```
sage: WittVectorRing(GF(9), p=3, prec=3).precision()
3
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(9)), p=Integer(3), prec=Integer(3)).precision()
3
```

prime()

Return the prime from which the truncated Witt vector ring has been constructed.

EXAMPLES:

```
sage: W = WittVectorRing(GF(81), prec=3)
sage: W.prime()
3

sage: W = WittVectorRing(ZZ, p=7, prec=2)
sage: W.prime()
7
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(81)), prec=Integer(3))
>>> W.prime()
3

>>> W = WittVectorRing(ZZ, p=Integer(7), prec=Integer(2))
>>> W.prime()
7
```

prod_polynomials(*variables=None*)

Return the Witt product polynomials.

INPUT:

- `variables` – names of the indeterminates (default: `None`), given as a string, or as a list of strings, whose length must be the double of the precision of the ring. When nothing is given, variables indexed by X and Y are used.

EXAMPLES:

```
sage: W = WittVectorRing(GF(5), prec=3)
sage: W.prod_polynomials()
[X0*Y0,
X1*Y0^5 + X0^5*Y1,
-X0^5*X1^4*Y0^20*Y1 - 2*X0^10*X1^3*Y0^15*Y1^2 - 2*X0^15*X1^2*Y0^10*Y1^3 - X0^
↪20*X1*Y0^5*Y1^4 + X2*Y0^25 + X0^25*Y2 + X1^5*Y1^5]

sage: W = WittVectorRing(ZZ, p=2, prec=2)
sage: W.prod_polynomials('T0, T1, U0, U1')
[T0*U0, T1*U0^2 + T0^2*U1 + 2*T1*U1]
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(5)), prec=Integer(3))
>>> W.prod_polynomials()
[X0*Y0,
X1*Y0^5 + X0^5*Y1,
-X0^5*X1^4*Y0^20*Y1 - 2*X0^10*X1^3*Y0^15*Y1^2 - 2*X0^15*X1^2*Y0^10*Y1^3 - X0^
↪20*X1*Y0^5*Y1^4 + X2*Y0^25 + X0^25*Y2 + X1^5*Y1^5]

>>> W = WittVectorRing(ZZ, p=Integer(2), prec=Integer(2))
>>> W.prod_polynomials('T0, T1, U0, U1')
[T0*U0, T1*U0^2 + T0^2*U1 + 2*T1*U1]
```

random_element(**args*, ***kwds*)

Return a random truncated Witt vector.

Extra arguments are passed to the random generator of the coefficient ring.

EXAMPLES:

```
sage: WittVectorRing(GF(27,'t'), prec=2).random_element()  # random
(z3, 2*z3^2 + 1)

sage: W = WittVectorRing(PolynomialRing(ZZ,'x'), p=3, prec=3)
sage: W.random_element(5)  # random
(x^5 - 2*x^4 - 4*x^3 - 2*x^2 + 1, -x^5 + 2*x^4 - x - 1,
-x^5 + 7*x^4 + 3*x^3 - 24*x^2 - 1)
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(27),'t'), prec=Integer(2)).random_element()  #␣
↪random
(z3, 2*z3^2 + 1)

>>> W = WittVectorRing(PolynomialRing(ZZ,'x'), p=Integer(3), prec=Integer(3))
>>> W.random_element(Integer(5))  # random
(x^5 - 2*x^4 - 4*x^3 - 2*x^2 + 1, -x^5 + 2*x^4 - x - 1,
-x^5 + 7*x^4 + 3*x^3 - 24*x^2 - 1)
```

sum_polynomials(*variables=None*)

Return the Witt sum polynomials.

INPUT:

- `variables` – names of the indeterminates (default: `None`), given as a string, or as a list of strings, whose length must be the double of the precision of the ring. When nothing is given, variables indexed by X

and Y are used.

EXAMPLES:

```
sage: W = WittVectorRing(GF(5), prec=2)
sage: W.sum_polynomials(['T0', 'T1', 'U0', 'U1'])
[T0 + U0, -T0^4*U0 - 2*T0^3*U0^2 - 2*T0^2*U0^3 - T0*U0^4 + T1 + U1]

sage: W = WittVectorRing(ZZ, p=2, prec=3)
sage: W.sum_polynomials()
[X0 + Y0,
-X0*Y0 + X1 + Y1,
-X0^3*Y0 - 2*X0^2*Y0^2 - X0*Y0^3 + X0*X1*Y0 + X0*Y0*Y1 - X1*Y1 + X2 + Y2]
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(5)), prec=Integer(2))
>>> W.sum_polynomials(['T0', 'T1', 'U0', 'U1'])
[T0 + U0, -T0^4*U0 - 2*T0^3*U0^2 - 2*T0^2*U0^3 - T0*U0^4 + T1 + U1]

>>> W = WittVectorRing(ZZ, p=Integer(2), prec=Integer(3))
>>> W.sum_polynomials()
[X0 + Y0,
-X0*Y0 + X1 + Y1,
-X0^3*Y0 - 2*X0^2*Y0^2 - X0*Y0^3 + X0*X1*Y0 + X0*Y0*Y1 - X1*Y1 + X2 + Y2]
```

teichmuller_lift(*x*)

Return the Teichmüller lift of `x` in `self`.

This lift is sometimes known as the multiplicative lift of `x`.

EXAMPLES:

```
sage: WittVectorRing(GF(125,'t'), prec=2).teichmuller_lift(3)
(3, 0)
```

```
>>> from sage.all import *
>>> WittVectorRing(GF(Integer(125),'t'), prec=Integer(2)).teichmuller_
↪lift(Integer(3))
(3, 0)
```

verschiebung(*extend=False*)

Return the Verschiebung map of this ring of Witt vectors.

INPUT:

- `extend` – boolean (default: `False`); whether the codomain of the map has precision 1 more than `self`.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ)
sage: W = WittVectorRing(R, p=5, prec=2)
sage: w = W([x, y])
sage: V = W.verschiebung()
sage: V(w)
(0, x)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> W = WittVectorRing(R, p=Integer(5), prec=Integer(2))
>>> w = W([x, y])
>>> V = W.verschiebung()
>>> V(w)
(0, x)
```

class sage.rings.padics.witt_vector_ring.**WittVectorRing_finotti**(*coefficient_ring*, *prec*, *prime*)

Bases: *WittVectorRing*

Child class for truncated Witt vectors using Finotti's algorithm.

Warning

This class should never be called directly, use `WittVectorRing` instead.

EXAMPLES:

```
sage: W = WittVectorRing(GF(49), prec=3, algorithm='finotti')
sage: W
Ring of truncated 7-typical Witt vectors of length 3 over Finite Field in z2 of␣
↪size 7^2

sage: W = WittVectorRing(ZZ, p=11, prec=3, algorithm='finotti')
Traceback (most recent call last):
...
ValueError: the 'finotti' algorithm only works for coefficients rings of␣
↪characteristic p
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(49)), prec=Integer(3), algorithm='finotti')
>>> W
Ring of truncated 7-typical Witt vectors of length 3 over Finite Field in z2 of␣
↪size 7^2

>>> W = WittVectorRing(ZZ, p=Integer(11), prec=Integer(3), algorithm='finotti')
Traceback (most recent call last):
...
ValueError: the 'finotti' algorithm only works for coefficients rings of␣
↪characteristic p
```

Element

alias of *WittVector_finotti*

class sage.rings.padics.witt_vector_ring.**WittVectorRing_phantom**(*coefficient_ring*, *prec*, *prime*)

Bases: *WittVectorRing*

Child class for truncated Witt vectors using the `phantom` algorithm.

> **Warning**
>
> This class should never be called directly, use `WittVectorRing` instead.

EXAMPLES:

```
sage: W = WittVectorRing(GF(19), prec=20)
sage: W
Ring of truncated 19-typical Witt vectors of length 20 over Finite Field of size␣
↪19

sage: W = WittVectorRing(QQ, p=23, prec=3, algorithm='phantom')
Traceback (most recent call last):
...
ValueError: the 'phantom' algorithm only works when the coefficient ring is a␣
↪finite field of char. p, or a polynomial ring on that field
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(19)), prec=Integer(20))
>>> W
Ring of truncated 19-typical Witt vectors of length 20 over Finite Field of size␣
↪19

>>> W = WittVectorRing(QQ, p=Integer(23), prec=Integer(3), algorithm='phantom')
Traceback (most recent call last):
...
ValueError: the 'phantom' algorithm only works when the coefficient ring is a␣
↪finite field of char. p, or a polynomial ring on that field
```

Element

alias of *WittVector_phantom*

class sage.rings.padics.witt_vector_ring.**WittVectorRing_pinvertible**(*coefficient_ring*, *prec*, *prime*)

Bases: *WittVectorRing*

Child class for truncated Witt vectors using the `p_invertible` algorithm.

> **Warning**
>
> This class should never be called directly, use `WittVectorRing` instead.

EXAMPLES:

```
sage: W = WittVectorRing(QQ, p=31, prec=20)
sage: W
Ring of truncated 31-typical Witt vectors of length 20 over Rational Field

sage: W = WittVectorRing(GF(3), prec=3, algorithm='p_invertible')
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: the 'p_invertible' algorithm only works when p is a unit in the ring␣
↪of coefficients
```

```
>>> from sage.all import *
>>> W = WittVectorRing(QQ, p=Integer(31), prec=Integer(20))
>>> W
Ring of truncated 31-typical Witt vectors of length 20 over Rational Field

>>> W = WittVectorRing(GF(Integer(3)), prec=Integer(3), algorithm='p_invertible')
Traceback (most recent call last):
...
ValueError: the 'p_invertible' algorithm only works when p is a unit in the ring␣
↪of coefficients
```

Element

alias of *WittVector_pinvertible*

class sage.rings.padics.witt_vector_ring.**WittVectorRing_standard**(*coefficient_ring*, *prec*, *prime*)

Bases: *WittVectorRing*

Child class for truncated Witt vectors using the `standard` algorithm.

Warning

This class should never be called directly, use `WittVectorRing` instead.

EXAMPLES:

```
sage: W = WittVectorRing(GF(3), prec=3, algorithm='standard')
sage: W
Ring of truncated 3-typical Witt vectors of length 3 over Finite Field of size 3
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(3)), prec=Integer(3), algorithm='standard')
>>> W
Ring of truncated 3-typical Witt vectors of length 3 over Finite Field of size 3
```

Element

alias of *WittVector_standard*

class sage.rings.padics.witt_vector_ring.**WittVectorVerschiebung**(*domain*, *extend=False*)

Bases: `RingMap`

A class implementing Veschiebung maps on Witt vector rings.

Warning

This class should not be called directly, use *WittVectorRing.verschiebung()* instead.

EXAMPLES:

```
sage: R.<x,y,z> = PolynomialRing(ZZ)
sage: W = WittVectorRing(R, p=3, prec=3)
sage: w = W([x, y, z])
sage: V = W.verschiebung()
sage: V(w)
(0, x, y)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ, names=('x', 'y', 'z',)); (x, y, z,) = R._first_ngens(3)
>>> W = WittVectorRing(R, p=Integer(3), prec=Integer(3))
>>> w = W([x, y, z])
>>> V = W.verschiebung()
>>> V(w)
(0, x, y)
```

sage.rings.padics.witt_vector_ring.**fast_char_p_power**(*x*, *n*, *p=None*)

Return x^n assuming that x lives in a ring of characteristic p.

If x is not an element of a ring of characteristic p, this throws an error.

EXAMPLES:

```
sage: from sage.rings.padics.witt_vector_ring import fast_char_p_power
sage: t = GF(1913)(33)
sage: fast_char_p_power(t, 77)
1371
```

```
>>> from sage.all import *
>>> from sage.rings.padics.witt_vector_ring import fast_char_p_power
>>> t = GF(Integer(1913))(Integer(33))
>>> fast_char_p_power(t, Integer(77))
1371
```

```
sage: K.<t> = GF(5^3)
sage: fast_char_p_power(t, 385)
4*t^2 + 1
sage: t^385
4*t^2 + 1
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('t',)); (t,) = K._first_ngens(1)
>>> fast_char_p_power(t, Integer(385))
4*t^2 + 1
>>> t**Integer(385)
4*t^2 + 1
```

```
sage: A.<x> = K[]
sage: fast_char_p_power(x + 1, 10)
x^10 + 2*x^5 + 1
```

```
>>> from sage.all import *
>>> A = K['x']; (x,) = A._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> fast_char_p_power(x + Integer(1), Integer(10))
x^10 + 2*x^5 + 1
```

```
sage: B.<u,v> = K[]
sage: fast_char_p_power(u + v, 1250)
u^1250 + 2*u^625*v^625 + v^1250
```

```
>>> from sage.all import *
>>> B = K['u, v']; (u, v,) = B._first_ngens(2)
>>> fast_char_p_power(u + v, Integer(1250))
u^1250 + 2*u^625*v^625 + v^1250
```

CHAPTER

THIRTYONE

WITT VECTORS

Implementation of the class *WittVector* of truncated Witt vectors.

AUTHORS:

- Jacob Dennerlein (2022-11-28): initial version
- Rubén Muñoz--Bertrand (2025-02-13, 2025-12-23): major refactoring and new features

class sage.rings.padics.witt_vector.**WittVector**(*parent*, *vec=None*)

Bases: CommutativeRingElement

Base class for truncated Witt vectors.

EXAMPLES:

```
sage: W = WittVectorRing(GF(25), p=5, prec=3)
sage: W(12)
(2, 1, 3)

sage: W = WittVectorRing(Integers(6), p=3, prec=4)
sage: w = W([1,2,3,4]) * W([4,5,0,0])
sage: w
(4, 1, 3, 4)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(25)), p=Integer(5), prec=Integer(3))
>>> W(Integer(12))
(2, 1, 3)

>>> W = WittVectorRing(Integers(Integer(6)), p=Integer(3), prec=Integer(4))
>>> w = W([Integer(1),Integer(2),Integer(3),Integer(4)]) * W([Integer(4),
↪Integer(5),Integer(0),Integer(0)])
>>> w
(4, 1, 3, 4)
```

additive_order()

Return the additive order of `self`.

EXAMPLES:

```
sage: W = WittVectorRing(Integers(10), p=2, prec=4)
sage: w = W([1, 3, 0, 7])
sage: w.additive_order()
```

(continues on next page)

(continued from previous page)

```
80
sage: W = WittVectorRing(ZZ, p=13, prec=3)
sage: w = W([0, 7, -3])
sage: w.additive_order()
+Infinity
```

```
>>> from sage.all import *
>>> W = WittVectorRing(Integers(Integer(10)), p=Integer(2), prec=Integer(4))
>>> w = W([Integer(1), Integer(3), Integer(0), Integer(7)])
>>> w.additive_order()
80
>>> W = WittVectorRing(ZZ, p=Integer(13), prec=Integer(3))
>>> w = W([Integer(0), Integer(7), -Integer(3)])
>>> w.additive_order()
+Infinity
```

coordinates()

Return the underlying tuple of the truncated Witt vector.

EXAMPLES:

```
sage: W = WittVectorRing(GF(7), p=7, prec=3)
sage: v = W([1,2,3])
sage: v.coordinates()
(1, 2, 3)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(7)), p=Integer(7), prec=Integer(3))
>>> v = W([Integer(1),Integer(2),Integer(3)])
>>> v.coordinates()
(1, 2, 3)
```

is_unit()

Return `True` if `self` has a multiplicative inverse.

EXAMPLES:

```
sage: W = WittVectorRing(GF(11), prec=4)
sage: w = W([1,2,10,0])
sage: w.is_unit()
True
sage: w = W([0,2,10,5])
sage: w.is_unit()
False
sage: W = WittVectorRing(ZZ, p=7, prec=3)
sage: w = W([2,-2,11])
sage: w.is_unit()
False
sage: w = W([-1,0,0])
sage: w.is_unit()
True
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(11)), prec=Integer(4))
>>> w = W([Integer(1),Integer(2),Integer(10),Integer(0)])
>>> w.is_unit()
True
>>> w = W([Integer(0),Integer(2),Integer(10),Integer(5)])
>>> w.is_unit()
False
>>> W = WittVectorRing(ZZ, p=Integer(7), prec=Integer(3))
>>> w = W([Integer(2),-Integer(2),Integer(11)])
>>> w.is_unit()
False
>>> w = W([-Integer(1),Integer(0),Integer(0)])
>>> w.is_unit()
True
```

class sage.rings.padics.witt_vector.**WittVector_finotti**(*parent*, *vec=None*)

Bases: *WittVector*

Child class for truncated Witt vectors using Finotti's algorithm.

EXAMPLES:

```
sage: W = WittVectorRing(GF(7), prec=4, algorithm='finotti')
sage: 49*W.one()
(0, 0, 1, 0)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(7)), prec=Integer(4), algorithm='finotti')
>>> Integer(49)*W.one()
(0, 0, 1, 0)
```

class sage.rings.padics.witt_vector.**WittVector_phantom**(*parent*, *vec=None*, *phantom=None*)

Bases: *WittVector*

Child class for truncated Witt vectors using the `phantom` algorithm.

Here, a Witt vector with coefficients in $\mathbf{F}_q$ (respectively in a polynomial ring over that field), is lifted to another Witt vector with coefficients in $\mathbf{Q}_q$ (respectively in the corresponding polynomial ring with coefficients in that field), whose phantom components are stored. Computations are done with these phantom components, and the corresponding Witt vectors in $\mathbf{F}_q$ (respectively in the polynomial ring) are computed from them only when needed.

EXAMPLES:

```
sage: W = WittVectorRing(GF(7), prec=5)
sage: t = W.one()
sage: t
(1, 0, 0, 0, 0)
sage: t.phantom()
(1, 1, 1, 1, 1)
sage: u = 7*t
sage: u.phantom(lift=True)
(7, 7, 7, 7, 7)
sage: u[1]
1
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(7)), prec=Integer(5))
>>> t = W.one()
>>> t
(1, 0, 0, 0, 0)
>>> t.phantom()
(1, 1, 1, 1, 1)
>>> u = Integer(7)*t
>>> u.phantom(lift=True)
(7, 7, 7, 7, 7)
>>> u[Integer(1)]
1
```

coordinates()

Return the underlying tuple of the truncated Witt vector.

EXAMPLES:

```
sage: W = WittVectorRing(GF(7), p=7, prec=3)
sage: v = W([1,2,3])
sage: v.coordinates()
(1, 2, 3)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(7)), p=Integer(7), prec=Integer(3))
>>> v = W([Integer(1),Integer(2),Integer(3)])
>>> v.coordinates()
(1, 2, 3)
```

phantom(*lift=False*)

Return the phantom components of the lift of `self`.

INPUT:

- `lift` – a Boolean (default: `False`). When `True`, return the phantom components in the lift of the coefficient ring.

EXAMPLES:

```
sage: W = WittVectorRing(GF(5,'t'), prec=3)
sage: t = W([1,1,3])
sage: t.phantom()
(1, 1, 1)
sage: t.phantom(lift=True)
(1, 1 + 5, 1 + 5 + 3*5^2)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(5),'t'), prec=Integer(3))
>>> t = W([Integer(1),Integer(1),Integer(3)])
>>> t.phantom()
(1, 1, 1)
>>> t.phantom(lift=True)
(1, 1 + 5, 1 + 5 + 3*5^2)
```

class sage.rings.padics.witt_vector.**WittVector_pinvertible**(*parent*, *vec=None*)

Bases: *WittVector*

Child class for truncated Witt vectors using the `p_invertible` algorithm.

EXAMPLES:

```
sage: W = WittVectorRing(QQ, p=3, prec=3)
sage: w = W.random_element()
sage: w - w
(0, 0, 0)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(QQ, p=Integer(3), prec=Integer(3))
>>> w = W.random_element()
>>> w - w
(0, 0, 0)
```

class sage.rings.padics.witt_vector.**WittVector_standard**(*parent*, *vec=None*)

Bases: *WittVector*

Child class for truncated Witt vectors using the `standard` algorithm.

EXAMPLES:

```
sage: W = WittVectorRing(GF(5), prec=3, algorithm='standard')
sage: 5 * W.one()
(0, 1, 0)
```

```
>>> from sage.all import *
>>> W = WittVectorRing(GF(Integer(5)), prec=Integer(3), algorithm='standard')
>>> Integer(5) * W.one()
(0, 1, 0)
```

CHAPTER

THIRTYTWO

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

r

sage.rings.padics.common_conversion, 451
sage.rings.padics.eisenstein_extension_generic, 191
sage.rings.padics.factory, 9
sage.rings.padics.generic_nodes, 149
sage.rings.padics.lattice_precision, 417
sage.rings.padics.local_generic, 107
sage.rings.padics.local_generic_element, 217
sage.rings.padics.misc, 445
sage.rings.padics.morphism, 453
sage.rings.padics.padic_base_generic, 173
sage.rings.padics.padic_base_leaves, 203
sage.rings.padics.padic_capped_absolute_element, 301
sage.rings.padics.padic_capped_relative_element, 273
sage.rings.padics.padic_ext_element, 343
sage.rings.padics.padic_extension_generic, 179
sage.rings.padics.padic_extension_leaves, 213
sage.rings.padics.padic_fixed_mod_element, 321
sage.rings.padics.padic_generic, 131
sage.rings.padics.padic_generic_element, 231
sage.rings.padics.padic_printing, 407
sage.rings.padics.padic_ZZ_pX_CA_element, 365
sage.rings.padics.padic_ZZ_pX_CR_element, 349
sage.rings.padics.padic_ZZ_pX_element, 347
sage.rings.padics.padic_ZZ_pX_FM_element, 381
sage.rings.padics.pow_computer, 397
sage.rings.padics.pow_computer_ext, 401
sage.rings.padics.precision_error, 415
sage.rings.padics.tutorial, 1
sage.rings.padics.unramified_extension_generic, 197
sage.rings.padics.witt_vector, 477
sage.rings.padics.witt_vector_ring, 457

INDEX

A

abs() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 233
absolute_degree() (*sage.rings.padics.local_generic.LocalGeneric method*), 107
absolute_discriminant() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 173
absolute_e() (*sage.rings.padics.eisenstein_extension_generic.EisensteinExtensionGeneric method*), 191
absolute_e() (*sage.rings.padics.local_generic.LocalGeneric method*), 108
absolute_f() (*sage.rings.padics.local_generic.LocalGeneric method*), 109
absolute_f() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 197
absolute_inertia_degree() (*sage.rings.padics.local_generic.LocalGeneric method*), 109
absolute_ramification_index() (*sage.rings.padics.local_generic.LocalGeneric method*), 110
add_bigoh() (*sage.rings.padics.local_generic_element.LocalGenericElement method*), 217
add_bigoh() (*sage.rings.padics.padic_capped_absolute_element.CAElement method*), 301
add_bigoh() (*sage.rings.padics.padic_capped_relative_element.CRElement method*), 273
add_bigoh() (*sage.rings.padics.padic_fixed_mod_element.FMElement method*), 322
add_bigoh() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 385
additive_order() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 234
additive_order() (*sage.rings.padics.witt_vector.WittVector method*), 477
algdep() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 235
algebraic_dependency() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 235
allow_negatives() (*sage.rings.padics.padic_printing.pAdicPrinterDefaults method*), 407
alphabet() (*sage.rings.padics.padic_printing.pAdicPrinterDefaults method*), 408
ambient_dimension() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 417
ap_element() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 161
artin_hasse_exp() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 236

B

base_p_list() (*in module sage.rings.padics.padic_capped_relative_element*), 282

C

CAElement (*class in sage.rings.padics.padic_capped_absolute_element*), 301
CappedAbsoluteGeneric (*class in sage.rings.padics.generic_nodes*), 149
CappedRelativeFieldGeneric (*class in sage.rings.padics.generic_nodes*), 149
CappedRelativeGeneric (*class in sage.rings.padics.generic_nodes*), 150
CappedRelativeRingGeneric (*class in sage.rings.padics.generic_nodes*), 150
cardinality() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 460
change() (*sage.rings.padics.local_generic.LocalGeneric method*), 110
characteristic() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 133
characteristic() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 460
coefficient_ring() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 461
composite() (*sage.rings.padics.generic_nodes.pAdicFieldBaseGeneric method*), 152
construction() (*sage.rings.padics.generic_nodes.pAdicFieldBaseGeneric method*), 152

construction() (*sage.rings.padics.generic_nodes.pAdicRingBaseGeneric method*), 169
construction() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 181
convert_multiple() (*sage.rings.padics.generic_nodes.pAdicLatticeGeneric method*), 155
coordinates() (*sage.rings.padics.witt_vector.WittVector method*), 478
coordinates() (*sage.rings.padics.witt_vector.WittVector_phantom method*), 480
create_key() (*sage.rings.padics.factory.Qp_class method*), 43
create_key() (*sage.rings.padics.factory.Zp_class method*), 85
create_key_and_extra_args() (*sage.rings.padics.factory.pAdicExtension_class method*), 105
create_object() (*sage.rings.padics.factory.pAdicExtension_class method*), 105
create_object() (*sage.rings.padics.factory.Qp_class method*), 43
create_object() (*sage.rings.padics.factory.Zp_class method*), 86
CRElement (*class in sage.rings.padics.padic_capped_relative_element*), 273

D

default_prec() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 162
defining_polynomial() (*sage.rings.padics.local_generic.LocalGeneric method*), 115
defining_polynomial() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 182
DefPolyConversion (*class in sage.rings.padics.padic_extension_generic*), 179
degree() (*sage.rings.padics.local_generic.LocalGeneric method*), 115
del_elements() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 419
del_elements() (*sage.rings.padics.lattice_precision.PrecisionLattice method*), 433
del_elements() (*sage.rings.padics.lattice_precision.PrecisionModule method*), 437
dict() (*sage.rings.padics.padic_printing.pAdicPrinter_class method*), 412
DifferentialPrecisionGeneric (*class in sage.rings.padics.lattice_precision*), 417
diffused_digits() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 420
dimension() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 421
dimension() (*sage.rings.padics.lattice_precision.PrecisionLattice method*), 434
dimension() (*sage.rings.padics.lattice_precision.PrecisionModule method*), 439
discriminant() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 173
discriminant() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 198
dwork_expansion() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 238
dwork_mahler_coeffs() (*in module sage.rings.padics.padic_generic_element*), 231

E

e() (*sage.rings.padics.local_generic.LocalGeneric method*), 116
EisensteinExtensionFieldCappedRelative (*class in sage.rings.padics.padic_extension_leaves*), 213
EisensteinExtensionGeneric (*class in sage.rings.padics.eisenstein_extension_generic*), 191
EisensteinExtensionRingCappedAbsolute (*class in sage.rings.padics.padic_extension_leaves*), 213
EisensteinExtensionRingCappedRelative (*class in sage.rings.padics.padic_extension_leaves*), 213
EisensteinExtensionRingFixedMod (*class in sage.rings.padics.padic_extension_leaves*), 214
Element (*sage.rings.padics.witt_vector_ring.WittVectorRing_finotti attribute*), 472
Element (*sage.rings.padics.witt_vector_ring.WittVectorRing_phantom attribute*), 473
Element (*sage.rings.padics.witt_vector_ring.WittVectorRing_pinvertible attribute*), 474
Element (*sage.rings.padics.witt_vector_ring.WittVectorRing_standard attribute*), 474
euclidean_degree() (*sage.rings.padics.local_generic_element.LocalGenericElement method*), 218
evaluate_dwork_mahler() (*in module sage.rings.padics.padic_generic_element*), 232
exact_field() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 173
exact_field() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 183
exact_ring() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 174
exact_ring() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*),

184
exp() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 239
expansion() (*sage.rings.padics.padic_capped_absolute_element.pAdicTemplateElement method*), 313
expansion() (*sage.rings.padics.padic_capped_relative_element.pAdicTemplateElement method*), 293
expansion() (*sage.rings.padics.padic_fixed_mod_element.pAdicTemplateElement method*), 334
expansion() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 370
expansion() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 355
expansion() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 385
ExpansionIter (*class in sage.rings.padics.padic_capped_absolute_element*), 305
ExpansionIter (*class in sage.rings.padics.padic_capped_relative_element*), 281
ExpansionIter (*class in sage.rings.padics.padic_fixed_mod_element*), 321
ExpansionIterable (*class in sage.rings.padics.padic_capped_absolute_element*), 305
ExpansionIterable (*class in sage.rings.padics.padic_capped_relative_element*), 281
ExpansionIterable (*class in sage.rings.padics.padic_fixed_mod_element*), 321
ext() (*sage.rings.padics.local_generic.LocalGeneric method*), 116
extension() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 134

F

f() (*sage.rings.padics.local_generic.LocalGeneric method*), 117
fast_char_p_power() (*in module sage.rings.padics.witt_vector_ring*), 475
FixedModGeneric (*class in sage.rings.padics.generic_nodes*), 150
FloatingPointFieldGeneric (*class in sage.rings.padics.generic_nodes*), 151
FloatingPointGeneric (*class in sage.rings.padics.generic_nodes*), 151
FloatingPointRingGeneric (*class in sage.rings.padics.generic_nodes*), 152
FMElement (*class in sage.rings.padics.padic_fixed_mod_element*), 322
fraction_field() (*sage.rings.padics.padic_extension_leaves.EisensteinExtensionRingFixedMod method*), 214
fraction_field() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 134
free_module() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 185
frobenius() (*sage.rings.padics.padic_ext_element.pAdicExtElement method*), 343
frobenius_endomorphism() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 135
frobenius_morphism() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 461
frobenius_polynomials() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 462
FrobeniusEndomorphism_padics (*class in sage.rings.padics.morphism*), 453

G

gamma() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 242
gauss_sum() (*in module sage.rings.padics.misc*), 445
gauss_table() (*in module sage.rings.padics.padic_generic_element*), 232
gcd() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 243
gen() (*sage.rings.padics.eisenstein_extension_generic.EisensteinExtensionGeneric method*), 192
gen() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 174
gen() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 198
gen() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 462
gens() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 136
gens() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 463
get_key_base() (*in module sage.rings.padics.factory*), 102
ground_ring() (*sage.rings.padics.local_generic.LocalGeneric method*), 118
ground_ring() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 186

ground_ring_of_tower() (*sage.rings.padics.local_generic.LocalGeneric method*), 118
ground_ring_of_tower() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 186

H

halting_prec() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 162
has_pth_root() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 174
has_pth_root() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 198
has_root_of_unity() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 174
has_root_of_unity() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 199
history() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 422
history_clear() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 427
history_disable() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 427
history_enable() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 427

I

inertia_degree() (*sage.rings.padics.local_generic.LocalGeneric method*), 118
inertia_subring() (*sage.rings.padics.eisenstein_extension_generic.EisensteinExtensionGeneric method*), 192
inertia_subring() (*sage.rings.padics.local_generic.LocalGeneric method*), 119
integer_ring() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 137
internal_prec() (*sage.rings.padics.lattice_precision.PrecisionModule method*), 440
inverse_of_unit() (*sage.rings.padics.local_generic_element.LocalGenericElement method*), 219
is_abelian() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 175
is_capped_absolute() (*sage.rings.padics.generic_nodes.CappedAbsoluteGeneric method*), 149
is_capped_absolute() (*sage.rings.padics.local_generic.LocalGeneric method*), 119
is_capped_relative() (*sage.rings.padics.generic_nodes.CappedRelativeGeneric method*), 150
is_capped_relative() (*sage.rings.padics.local_generic.LocalGeneric method*), 120
is_eisenstein() (*in module sage.rings.padics.factory*), 103
is_equal_to() (*sage.rings.padics.padic_capped_absolute_element.CAElement method*), 302
is_equal_to() (*sage.rings.padics.padic_capped_relative_element.CRElement method*), 274
is_equal_to() (*sage.rings.padics.padic_fixed_mod_element.FMElement method*), 322
is_equal_to() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 372
is_equal_to() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 356
is_equal_to() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 387
is_exact() (*sage.rings.padics.local_generic.LocalGeneric method*), 120
is_exact() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 464
is_field() (*sage.rings.padics.generic_nodes.pAdicFieldGeneric method*), 154
is_field() (*sage.rings.padics.generic_nodes.pAdicRingGeneric method*), 170
is_field() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 464
is_finite() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 465
is_fixed_mod() (*sage.rings.padics.generic_nodes.FixedModGeneric method*), 150
is_fixed_mod() (*sage.rings.padics.local_generic.LocalGeneric method*), 121
is_floating_point() (*sage.rings.padics.generic_nodes.FloatingPointGeneric method*), 151
is_floating_point() (*sage.rings.padics.local_generic.LocalGeneric method*), 121
is_galois() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 199
is_identity() (*sage.rings.padics.morphism.FrobeniusEndomorphism_padics method*), 453
is_injective() (*sage.rings.padics.morphism.FrobeniusEndomorphism_padics method*), 453
is_injective() (*sage.rings.padics.padic_capped_absolute_element.pAdicCoercion_CA_frac_field method*), 310
is_injective() (*sage.rings.padics.padic_capped_relative_element.pAdicCoercion_CR_frac_field method*), 288
is_injective() (*sage.rings.padics.padic_exten-*

sion_generic.pAdicModuleIsomorphism method), 188
is_injective() (sage.rings.padics.padic_fixed_mod_element.pAdicCoercion_FM_frac_field method), 326
is_injective() (sage.rings.padics.padic_generic.ResidueReductionMap method), 132
is_integral() (sage.rings.padics.local_generic_element.LocalGenericElement method), 221
is_integral_domain() (sage.rings.padics.witt_vector_ring.WittVectorRing method), 465
is_integrally_closed() (sage.rings.padics.witt_vector_ring.WittVectorRing method), 466
is_isomorphic() (sage.rings.padics.padic_base_generic.pAdicBaseGeneric method), 175
is_lattice() (sage.rings.padics.lattice_precision.PrecisionModule method), 440
is_lattice_prec() (sage.rings.padics.generic_nodes.pAdicLatticeGeneric method), 157
is_lattice_prec() (sage.rings.padics.local_generic.LocalGeneric method), 122
is_normal() (sage.rings.padics.padic_base_generic.pAdicBaseGeneric method), 176
is_p_power() (sage.rings.padics.lattice_precision.pRational method), 444
is_padic_unit() (sage.rings.padics.local_generic_element.LocalGenericElement method), 221
is_prime() (sage.rings.padics.padic_generic_element.pAdicGenericElement method), 246
is_prime_field() (sage.rings.padics.witt_vector_ring.WittVectorRing method), 467
is_relaxed() (sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method), 163
is_relaxed() (sage.rings.padics.local_generic.LocalGeneric method), 123
is_secure() (sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method), 163
is_square() (sage.rings.padics.padic_generic_element.pAdicGenericElement method), 247
is_squarefree() (sage.rings.padics.padic_generic_element.pAdicGenericElement method), 247
is_surjective() (sage.rings.padics.morphism.FrobeniusEndomorphism_padics method), 453
is_surjective() (sage.rings.padics.padic_capped_absolute_element.pAdicCoercion_CA_frac_field method), 310
is_surjective() (sage.rings.padics.padic_capped_relative_element.pAdicCoercion_CR_frac_field method), 288
is_surjective() (sage.rings.padics.padic_extension_generic.pAdicModuleIsomorphism method), 188
is_surjective() (sage.rings.padics.padic_fixed_mod_element.pAdicCoercion_FM_frac_field method), 326
is_surjective() (sage.rings.padics.padic_generic.ResidueReductionMap method), 132
is_unit() (sage.rings.padics.local_generic_element.LocalGenericElement method), 222
is_unit() (sage.rings.padics.witt_vector.WittVector method), 478
is_unramified() (in module sage.rings.padics.factory), 104
is_zero() (sage.rings.padics.lattice_precision.pRational method), 444
is_zero() (sage.rings.padics.padic_capped_absolute_element.CAElement method), 302
is_zero() (sage.rings.padics.padic_capped_relative_element.CRElement method), 276
is_zero() (sage.rings.padics.padic_fixed_mod_element.FMElement method), 323
is_zero() (sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method), 372
is_zero() (sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method), 357
is_zero() (sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method), 388

K

krasner_check() (in module sage.rings.padics.factory), 104
krull_dimension() (sage.rings.padics.generic_nodes.pAdicRingGeneric method), 171

L

label() (sage.rings.padics.generic_nodes.pAdicLatticeGeneric method), 157
lift() (sage.rings.padics.padic_capped_absolute_element.pAdicCappedAbsoluteElement method), 307
lift() (sage.rings.padics.padic_capped_relative_element.pAdicCappedRelativeElement method), 286
lift() (sage.rings.padics.padic_fixed_mod_element.pAdicFixedModElement method), 331
lift_to_precision() (sage.rings.padics.padic_capped_absolute_element.pAdicTemplateElement method), 317
lift_to_precision() (sage.rings.padics.padic_capped_relative_element.pAdicTemplateElement method), 296
lift_to_precision() (sage.rings.padics.padic_fixed_mod_element.pAdicTemplateElement method), 337
lift_to_precision() (sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method), 373

lift_to_precision() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 358
lift_to_precision() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 389
list() (*sage.rings.padics.lattice_precision.pRational method*), 444
list_of_padics() (*in module sage.rings.padics.lattice_precision*), 443
local_print_mode() (*in module sage.rings.padics.padic_generic*), 133
LocalGeneric (*class in sage.rings.padics.local_generic*), 107
LocalGenericElement (*class in sage.rings.padics.local_generic_element*), 217
log() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 248

M

make_pAdicCappedAbsoluteElement() (*in module sage.rings.padics.padic_capped_absolute_element*), 306
make_pAdicFixedModElement() (*in module sage.rings.padics.padic_fixed_mod_element*), 325
make_ZZpXCAElement() (*in module sage.rings.padics.padic_ZZ_pX_CA_element*), 369
make_ZZpXCRElement() (*in module sage.rings.padics.padic_ZZ_pX_CR_element*), 353
make_ZZpXFMElement() (*in module sage.rings.padics.padic_ZZ_pX_FM_element*), 384
MapFreeModuleToOneStep (*class in sage.rings.padics.padic_extension_generic*), 180
MapFreeModuleToTwoStep (*class in sage.rings.padics.padic_extension_generic*), 180
MapOneStepToFreeModule (*class in sage.rings.padics.padic_extension_generic*), 180
MapTwoStepToFreeModule (*class in sage.rings.padics.padic_extension_generic*), 181
matrix_mod_pn() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 374
matrix_mod_pn() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 359
matrix_mod_pn() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 389
max() (*in module sage.rings.padics.misc*), 447
max_poly_terms() (*sage.rings.padics.padic_printing.pAdicPrinterDefaults method*), 408
max_series_terms() (*sage.rings.padics.padic_printing.pAdicPrinterDefaults method*), 409
max_unram_terms() (*sage.rings.padics.padic_printing.pAdicPrinterDefaults method*), 410
maximal_unramified_subextension() (*sage.rings.padics.local_generic.LocalGeneric method*), 123
min() (*in module sage.rings.padics.misc*), 447
minimal_polynomial() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 255
mode() (*sage.rings.padics.padic_printing.pAdicPrinterDefaults method*), 410
module
    sage.rings.padics.common_conversion, 451
    sage.rings.padics.eisenstein_extension_generic, 191
    sage.rings.padics.factory, 9
    sage.rings.padics.generic_nodes, 149
    sage.rings.padics.lattice_precision, 417
    sage.rings.padics.local_generic, 107
    sage.rings.padics.local_generic_element, 217
    sage.rings.padics.misc, 445
    sage.rings.padics.morphism, 453
    sage.rings.padics.padic_base_generic, 173
    sage.rings.padics.padic_base_leaves, 203
    sage.rings.padics.padic_capped_absolute_element, 301
    sage.rings.padics.padic_capped_relative_element, 273
    sage.rings.padics.padic_ext_element, 343
    sage.rings.padics.padic_extension_generic, 179
    sage.rings.padics.padic_extension_leaves, 213
    sage.rings.padics.padic_fixed_mod_element, 321
    sage.rings.padics.padic_generic, 131
    sage.rings.padics.padic_generic_element, 231
    sage.rings.padics.padic_printing, 407
    sage.rings.padics.padic_ZZ_pX_CA_element, 365
    sage.rings.padics.padic_ZZ_pX_CR_element, 349
    sage.rings.padics.padic_ZZ_pX_element, 347
    sage.rings.padics.padic_ZZ_pX_FM_element, 381
    sage.rings.padics.pow_computer, 397

sage.rings.padics.pow_computer_ext, 401
sage.rings.padics.precision_error, 415
sage.rings.padics.tutorial, 1
sage.rings.padics.unramified_extension_generic, 197
sage.rings.padics.witt_vector, 477
sage.rings.padics.witt_vector_ring, 457
modulus() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 176
modulus() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 186
multiplicative_order() (*sage.rings.padics.padic_capped_absolute_element.pAdicCappedAbsoluteElement method*), 308
multiplicative_order() (*sage.rings.padics.padic_fixed_mod_element.pAdicFixedModElement method*), 332
multiplicative_order() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 256

N

ngens() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 138
ngens() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 468
norm() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 258
norm() (*sage.rings.padics.padic_ZZ_pX_element.pAdicZZpXElement method*), 347
norm() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 390
normalize() (*sage.rings.padics.lattice_precision.pRational method*), 444
normalized_valuation() (*sage.rings.padics.local_generic_element.LocalGenericElement method*), 223
nth_root() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 258

O

order() (*sage.rings.padics.morphism.FrobeniusEndomorphism_padics method*), 454
ordp() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 261

P

pAdicBaseGeneric (*class in sage.rings.padics.padic_base_generic*), 173
pAdicCappedAbsoluteElement (*class in sage.rings.padics.padic_capped_absolute_element*), 306
pAdicCappedAbsoluteRingGeneric (*class in sage.rings.padics.generic_nodes*), 152
pAdicCappedRelativeElement (*class in sage.rings.padics.padic_capped_relative_element*), 282
pAdicCappedRelativeFieldGeneric (*class in sage.rings.padics.generic_nodes*), 152
pAdicCappedRelativeRingGeneric (*class in sage.rings.padics.generic_nodes*), 152
pAdicCoercion_CA_frac_field (*class in sage.rings.padics.padic_capped_absolute_element*), 309
pAdicCoercion_CR_frac_field (*class in sage.rings.padics.padic_capped_relative_element*), 288
pAdicCoercion_FM_frac_field (*class in sage.rings.padics.padic_fixed_mod_element*), 326
pAdicCoercion_QQ_CR (*class in sage.rings.padics.padic_capped_relative_element*), 289
pAdicCoercion_ZZ_CA (*class in sage.rings.padics.padic_capped_absolute_element*), 311
pAdicCoercion_ZZ_CR (*class in sage.rings.padics.padic_capped_relative_element*), 290
pAdicCoercion_ZZ_FM (*class in sage.rings.padics.padic_fixed_mod_element*), 327
pAdicConvert_CA_frac_field (*class in sage.rings.padics.padic_capped_absolute_element*), 312
pAdicConvert_CA_ZZ (*class in sage.rings.padics.padic_capped_absolute_element*), 312
pAdicConvert_CR_frac_field (*class in sage.rings.padics.padic_capped_relative_element*), 291
pAdicConvert_CR_QQ (*class in sage.rings.padics.padic_capped_relative_element*), 290
pAdicConvert_CR_ZZ (*class in sage.rings.padics.padic_capped_relative_element*), 291
pAdicConvert_FM_frac_field (*class in sage.rings.padics.padic_fixed_mod_element*), 328
pAdicConvert_FM_ZZ (*class in sage.rings.padics.padic_fixed_mod_element*), 328
pAdicConvert_QQ_CA (*class in sage.rings.padics.padic_capped_absolute_element*), 313

pAdicConvert_QQ_CR (*class in sage.rings.padics.padic_capped_relative_element*), 292
pAdicConvert_QQ_FM (*class in sage.rings.padics.padic_fixed_mod_element*), 328
pAdicExtElement (*class in sage.rings.padics.padic_ext_element*), 343
pAdicExtension (*in module sage.rings.padics.factory*), 31
pAdicExtension_class (*class in sage.rings.padics.factory*), 105
pAdicExtensionGeneric (*class in sage.rings.padics.padic_extension_generic*), 181
pAdicFieldBaseGeneric (*class in sage.rings.padics.generic_nodes*), 152
pAdicFieldCappedRelative (*class in sage.rings.padics.padic_base_leaves*), 207
pAdicFieldFloatingPoint (*class in sage.rings.padics.padic_base_leaves*), 207
pAdicFieldGeneric (*class in sage.rings.padics.generic_nodes*), 154
pAdicFieldLattice (*class in sage.rings.padics.padic_base_leaves*), 207
pAdicFieldRelaxed (*class in sage.rings.padics.padic_base_leaves*), 209
pAdicFixedModElement (*class in sage.rings.padics.padic_fixed_mod_element*), 329
pAdicFixedModRingGeneric (*class in sage.rings.padics.generic_nodes*), 154
pAdicFloatingPointFieldGeneric (*class in sage.rings.padics.generic_nodes*), 155
pAdicFloatingPointRingGeneric (*class in sage.rings.padics.generic_nodes*), 155
pAdicGeneric (*class in sage.rings.padics.padic_generic*), 133
pAdicGenericElement (*class in sage.rings.padics.padic_generic_element*), 233
pAdicLatticeElementWeakProxy (*class in sage.rings.padics.lattice_precision*), 443
pAdicLatticeGeneric (*class in sage.rings.padics.generic_nodes*), 155
pAdicModuleIsomorphism (*class in sage.rings.padics.padic_extension_generic*), 188
pAdicPrinter() (*in module sage.rings.padics.padic_printing*), 407
pAdicPrinter_class (*class in sage.rings.padics.padic_printing*), 412
pAdicPrinterDefaults (*class in sage.rings.padics.padic_printing*), 407
pAdicRelaxedGeneric (*class in sage.rings.padics.generic_nodes*), 161
pAdicRingBaseGeneric (*class in sage.rings.padics.generic_nodes*), 169
pAdicRingCappedAbsolute (*class in sage.rings.padics.padic_base_leaves*), 209
pAdicRingCappedRelative (*class in sage.rings.padics.padic_base_leaves*), 210
pAdicRingFixedMod (*class in sage.rings.padics.padic_base_leaves*), 210
pAdicRingFloatingPoint (*class in sage.rings.padics.padic_base_leaves*), 210
pAdicRingGeneric (*class in sage.rings.padics.generic_nodes*), 170
pAdicRingLattice (*class in sage.rings.padics.padic_base_leaves*), 210
pAdicRingRelaxed (*class in sage.rings.padics.padic_base_leaves*), 211
pAdicTemplateElement (*class in sage.rings.padics.padic_capped_absolute_element*), 313
pAdicTemplateElement (*class in sage.rings.padics.padic_capped_relative_element*), 292
pAdicTemplateElement (*class in sage.rings.padics.padic_fixed_mod_element*), 334
pAdicZZpXCAElement (*class in sage.rings.padics.padic_ZZ_pX_CA_element*), 369
pAdicZZpXCRElement (*class in sage.rings.padics.padic_ZZ_pX_CR_element*), 354
pAdicZZpXElement (*class in sage.rings.padics.padic_ZZ_pX_element*), 347
pAdicZZpXFMElement (*class in sage.rings.padics.padic_ZZ_pX_FM_element*), 384
phantom() (*sage.rings.padics.witt_vector.WittVector_phantom method*), 480
plot() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 176
polylog() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 262
polynomial() (*sage.rings.padics.padic_capped_absolute_element.CAElement method*), 303
polynomial() (*sage.rings.padics.padic_capped_relative_element.CRElement method*), 277
polynomial() (*sage.rings.padics.padic_fixed_mod_element.FMElement method*), 323
polynomial() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 375
polynomial() (*sage.rings.padics.padic_ZZ_pX_CR_ele-*

ment.pAdicZZpXCRElement method), 360
polynomial() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 390
polynomial() (*sage.rings.padics.pow_computer_ext.PowComputer_ZZ_pX method*), 401
polynomial_ring() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 187
pow_Integer_Integer() (*sage.rings.padics.pow_computer.PowComputer_class method*), 398
PowComputer() (*in module sage.rings.padics.pow_computer*), 397
PowComputer_ (*class in sage.rings.padics.padic_capped_absolute_element*), 306
PowComputer_ (*class in sage.rings.padics.padic_capped_relative_element*), 282
PowComputer_ (*class in sage.rings.padics.padic_fixed_mod_element*), 325
PowComputer_base (*class in sage.rings.padics.pow_computer*), 398
PowComputer_class (*class in sage.rings.padics.pow_computer*), 398
PowComputer_ext (*class in sage.rings.padics.pow_computer_ext*), 403
PowComputer_ext_maker() (*in module sage.rings.padics.pow_computer_ext*), 403
PowComputer_ZZ_pX (*class in sage.rings.padics.pow_computer_ext*), 401
PowComputer_ZZ_pX_big (*class in sage.rings.padics.pow_computer_ext*), 402
PowComputer_ZZ_pX_big_Eis (*class in sage.rings.padics.pow_computer_ext*), 403
PowComputer_ZZ_pX_FM (*class in sage.rings.padics.pow_computer_ext*), 402
PowComputer_ZZ_pX_FM_Eis (*class in sage.rings.padics.pow_computer_ext*), 402
PowComputer_ZZ_pX_small (*class in sage.rings.padics.pow_computer_ext*), 403
PowComputer_ZZ_pX_small_Eis (*class in sage.rings.padics.pow_computer_ext*), 403
power() (*sage.rings.padics.morphism.FrobeniusEndomorphism_padics method*), 454
pRational (*class in sage.rings.padics.lattice_precision*), 443
precision() (*sage.rings.padics.generic_nodes.pAdicLatticeGeneric method*), 159
precision() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 469
precision_absolute() (*sage.rings.padics.padic_capped_absolute_element.CAElement method*), 303
precision_absolute() (*sage.rings.padics.padic_capped_relative_element.CRElement method*), 277
precision_absolute() (*sage.rings.padics.padic_fixed_mod_element.FMElement method*), 324
precision_absolute() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 375
precision_absolute() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 360
precision_absolute() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 391
precision_cap() (*sage.rings.padics.generic_nodes.pAdicLatticeGeneric method*), 159
precision_cap() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 164
precision_cap() (*sage.rings.padics.local_generic.LocalGeneric method*), 123
precision_cap_absolute() (*sage.rings.padics.generic_nodes.pAdicLatticeGeneric method*), 160
precision_cap_relative() (*sage.rings.padics.generic_nodes.pAdicLatticeGeneric method*), 160
precision_lattice() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 428
precision_lattice() (*sage.rings.padics.lattice_precision.PrecisionLattice method*), 435
precision_lattice() (*sage.rings.padics.lattice_precision.PrecisionModule method*), 441
precision_relative() (*sage.rings.padics.padic_capped_absolute_element.CAElement method*), 304
precision_relative() (*sage.rings.padics.padic_capped_relative_element.CRElement method*), 278
precision_relative() (*sage.rings.padics.padic_fixed_mod_element.FMElement method*), 324
precision_relative() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 376
precision_relative() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 361
precision_relative() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 392
PrecisionError, 415
PrecisionLattice (*class in sage.rings.padics.lat-*

tice_precision), 432
PrecisionModule (*class in sage.rings.padics.lattice_precision*), 437
precprint() (*in module sage.rings.padics.misc*), 448
prime() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 429
prime() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 138
prime() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 469
primitive_root_of_unity() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 138
print_mode() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 140
prod_polynomials() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 469

Q

Qp (*in module sage.rings.padics.factory*), 9
Qp_class (*class in sage.rings.padics.factory*), 33
QpCR() (*in module sage.rings.padics.factory*), 31
QpER() (*in module sage.rings.padics.factory*), 32
QpFP() (*in module sage.rings.padics.factory*), 32
QpLC() (*in module sage.rings.padics.factory*), 32
QpLF() (*in module sage.rings.padics.factory*), 32
Qq() (*in module sage.rings.padics.factory*), 43
QqCR() (*in module sage.rings.padics.factory*), 58
QqFP() (*in module sage.rings.padics.factory*), 58
quo_rem() (*sage.rings.padics.local_generic_element.LocalGenericElement method*), 223

R

ramification_index() (*sage.rings.padics.local_generic.LocalGeneric method*), 124
random_element() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 164
random_element() (*sage.rings.padics.generic_nodes.pAdicRingBaseGeneric method*), 170
random_element() (*sage.rings.padics.padic_base_leaves.pAdicFieldCappedRelative method*), 207
random_element() (*sage.rings.padics.padic_base_leaves.pAdicFieldLattice method*), 208
random_element() (*sage.rings.padics.padic_base_leaves.pAdicRingLattice method*), 211
random_element() (*sage.rings.padics.padic_extension_generic.pAdicExtensionGeneric method*), 187
random_element() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 470
rational_reconstruction() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 264
reduce() (*sage.rings.padics.lattice_precision.pRational method*), 444
reduce() (*sage.rings.padics.lattice_precision.PrecisionLattice method*), 437
reduce_relative() (*sage.rings.padics.lattice_precision.pRational method*), 444
relative_degree() (*sage.rings.padics.local_generic.LocalGeneric method*), 125
relative_e() (*sage.rings.padics.local_generic.LocalGeneric method*), 125
relative_f() (*sage.rings.padics.local_generic.LocalGeneric method*), 126
relative_inertia_degree() (*sage.rings.padics.local_generic.LocalGeneric method*), 126
relative_ramification_index() (*sage.rings.padics.local_generic.LocalGeneric method*), 127
repr_gen() (*sage.rings.padics.padic_printing.pAdicPrinter_class method*), 412
reset_dictionaries() (*sage.rings.padics.pow_computer_ext.PowComputer_ZZ_pX_big method*), 402
residue() (*sage.rings.padics.padic_capped_absolute_element.pAdicCappedAbsoluteElement method*), 308
residue() (*sage.rings.padics.padic_capped_absolute_element.pAdicTemplateElement method*), 318
residue() (*sage.rings.padics.padic_capped_relative_element.pAdicCappedRelativeElement method*), 286
residue() (*sage.rings.padics.padic_capped_relative_element.pAdicTemplateElement method*), 297
residue() (*sage.rings.padics.padic_ext_element.pAdicExtElement method*), 344
residue() (*sage.rings.padics.padic_fixed_mod_element.pAdicFixedModElement method*), 333
residue() (*sage.rings.padics.padic_fixed_mod_element.pAdicTemplateElement method*), 338
residue_characteristic() (*sage.rings.padics.local_generic.LocalGeneric method*), 128
residue_characteristic() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 140
residue_class_field() (*sage.rings.padics.eisenstein_extension_generic.EisensteinExtensionGeneric method*), 193
residue_class_field() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 140
residue_class_field() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 200
residue_field() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 140
residue_ring() (*sage.rings.padics.eisenstein_ex-*

tension_generic.EisensteinExtensionGeneric method), 193
residue_ring() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 141
residue_ring() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 200
residue_system() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 141
ResidueLiftingMap (*class in sage.rings.padics.padic_generic*), 131
ResidueReductionMap (*class in sage.rings.padics.padic_generic*), 131
richcmp_modes() (*sage.rings.padics.padic_printing.pAdicPrinter_class method*), 413
roots_of_unity() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 141

S

sage.rings.padics.common_conversion
module, 451
sage.rings.padics.eisenstein_extension_generic
module, 191
sage.rings.padics.factory
module, 9
sage.rings.padics.generic_nodes
module, 149
sage.rings.padics.lattice_precision
module, 417
sage.rings.padics.local_generic
module, 107
sage.rings.padics.local_generic_element
module, 217
sage.rings.padics.misc
module, 445
sage.rings.padics.morphism
module, 453
sage.rings.padics.padic_base_generic
module, 173
sage.rings.padics.padic_base_leaves
module, 203
sage.rings.padics.padic_capped_absolute_element
module, 301
sage.rings.padics.padic_capped_relative_element
module, 273
sage.rings.padics.padic_ext_element
module, 343
sage.rings.padics.padic_extension_generic
module, 179
sage.rings.padics.padic_extension_leaves
module, 213
sage.rings.padics.padic_fixed_mod_element
module, 321
sage.rings.padics.padic_generic
module, 131
sage.rings.padics.padic_generic_element
module, 231
sage.rings.padics.padic_printing
module, 407
sage.rings.padics.padic_ZZ_pX_CA_element
module, 365
sage.rings.padics.padic_ZZ_pX_CR_element
module, 349
sage.rings.padics.padic_ZZ_pX_element
module, 347
sage.rings.padics.padic_ZZ_pX_FM_element
module, 381
sage.rings.padics.pow_computer
module, 397
sage.rings.padics.pow_computer_ext
module, 401
sage.rings.padics.precision_error
module, 415
sage.rings.padics.tutorial
module, 1
sage.rings.padics.unramified_extension_generic
module, 197
sage.rings.padics.witt_vector
module, 477
sage.rings.padics.witt_vector_ring
module, 457
section() (*sage.rings.padics.padic_capped_absolute_element.pAdicCoercion_CA_frac_field method*), 311
section() (*sage.rings.padics.padic_capped_absolute_element.pAdicCoercion_ZZ_CA method*), 311
section() (*sage.rings.padics.padic_capped_relative_element.pAdicCoercion_CR_frac_field method*), 289
section() (*sage.rings.padics.padic_capped_relative_element.pAdicCoercion_QQ_CR method*), 290
section() (*sage.rings.padics.padic_capped_relative_element.pAdicCoercion_ZZ_CR method*), 290
section() (*sage.rings.padics.padic_capped_relative_element.pAdicConvert_QQ_CR method*), 292
section() (*sage.rings.padics.padic_fixed_mod_element.pAdicCoercion_FM_frac_field method*), 327
section() (*sage.rings.padics.padic_fixed_mod_element.pAdicCoercion_ZZ_FM method*), 327
section() (*sage.rings.padics.padic_generic.ResidueReductionMap method*), 132
sep() (*sage.rings.padics.padic_printing.pAdicPrinterDefaults method*), 411

slice() (*sage.rings.padics.local_generic_element.LocalGenericElement method*), 224
some_elements() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 165
some_elements() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 143
speed_test() (*sage.rings.padics.pow_computer_ext.PowComputer_ZZ_pX method*), 402
split() (*in module sage.rings.padics.factory*), 105
sqrt() (*sage.rings.padics.local_generic_element.LocalGenericElement method*), 226
square_root() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 265
str() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 268
subfield() (*sage.rings.padics.generic_nodes.pAdicFieldBaseGeneric method*), 153
subfields_of_degree() (*sage.rings.padics.generic_nodes.pAdicFieldBaseGeneric method*), 154
sum_polynomials() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 470

T

teichmuller() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 166
teichmuller() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 143
teichmuller_expansion() (*sage.rings.padics.padic_capped_absolute_element.pAdicTemplateElement method*), 318
teichmuller_expansion() (*sage.rings.padics.padic_capped_relative_element.pAdicTemplateElement method*), 298
teichmuller_expansion() (*sage.rings.padics.padic_fixed_mod_element.pAdicTemplateElement method*), 339
teichmuller_expansion() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 377
teichmuller_expansion() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 362
teichmuller_expansion() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 392
teichmuller_lift() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 471
teichmuller_system() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 167
teichmuller_system() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 145
threshold_deletion() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 429
timings() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 430
to_fraction_field() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 378
trace() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 268
trace() (*sage.rings.padics.padic_ZZ_pX_element.pAdicZZpXElement method*), 348
trace() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 394
tracked_elements() (*sage.rings.padics.lattice_precision.DifferentialPrecisionGeneric method*), 431
trim_zeros() (*in module sage.rings.padics.misc*), 448
truncate_to_prec() (*in module sage.rings.padics.factory*), 106

U

uniformiser() (*sage.rings.padics.local_generic.LocalGeneric method*), 128
uniformiser_pow() (*sage.rings.padics.local_generic.LocalGeneric method*), 128
uniformizer() (*sage.rings.padics.eisenstein_extension_generic.EisensteinExtensionGeneric method*), 194
uniformizer() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 177
uniformizer() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 201
uniformizer_pow() (*sage.rings.padics.eisenstein_extension_generic.EisensteinExtensionGeneric method*), 194
uniformizer_pow() (*sage.rings.padics.padic_base_generic.pAdicBaseGene method*), 177
uniformizer_pow() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 146
uniformizer_pow() (*sage.rings.padics.unramified_extension_generic.UnramifiedExtensionGeneric method*), 201
unit_part() (*sage.rings.padics.lattice_precision.pRational method*), 444
unit_part() (*sage.rings.padics.padic_capped_absolute_element.CAElement method*), 304
unit_part() (*sage.rings.padics.padic_capped_absolute_element.pAdicTemplateElement method*), 319
unit_part() (*sage.rings.padics.padic_capped_relative_element.CRElement method*), 279

unit_part() (*sage.rings.padics.padic_capped_relative_element.pAdicTemplateElement method*), 299
unit_part() (*sage.rings.padics.padic_fixed_mod_element.FMElement method*), 324
unit_part() (*sage.rings.padics.padic_fixed_mod_element.pAdicTemplateElement method*), 340
unit_part() (*sage.rings.padics.padic_ZZ_pX_CA_element.pAdicZZpXCAElement method*), 379
unit_part() (*sage.rings.padics.padic_ZZ_pX_CR_element.pAdicZZpXCRElement method*), 363
unit_part() (*sage.rings.padics.padic_ZZ_pX_FM_element.pAdicZZpXFMElement method*), 395
unknown() (*sage.rings.padics.generic_nodes.pAdicRelaxedGeneric method*), 167
unpickle_cae_v2() (*in module sage.rings.padics.padic_capped_absolute_element*), 320
unpickle_cre_v2() (*in module sage.rings.padics.padic_capped_relative_element*), 299
unpickle_fme_v2() (*in module sage.rings.padics.padic_fixed_mod_element*), 340
unpickle_pcre_v1() (*in module sage.rings.padics.padic_capped_relative_element*), 299
UnramifiedExtensionFieldCappedRelative (*class in sage.rings.padics.padic_extension_leaves*), 214
UnramifiedExtensionFieldFloatingPoint (*class in sage.rings.padics.padic_extension_leaves*), 215
UnramifiedExtensionGeneric (*class in sage.rings.padics.unramified_extension_generic*), 197
UnramifiedExtensionRingCappedAbsolute (*class in sage.rings.padics.padic_extension_leaves*), 215
UnramifiedExtensionRingCappedRelative (*class in sage.rings.padics.padic_extension_leaves*), 215
UnramifiedExtensionRingFixedMod (*class in sage.rings.padics.padic_extension_leaves*), 216
UnramifiedExtensionRingFloatingPoint (*class in sage.rings.padics.padic_extension_leaves*), 216

V

val_unit() (*sage.rings.padics.padic_capped_absolute_element.CAElement method*), 304
val_unit() (*sage.rings.padics.padic_capped_relative_element.CRElement method*), 280
val_unit() (*sage.rings.padics.padic_fixed_mod_element.FMElement method*), 325
val_unit() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 269
valuation() (*sage.rings.padics.lattice_precision.pRational method*), 444
valuation() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 269
valuation() (*sage.rings.padics.padic_generic.pAdicGeneric method*), 146
value() (*sage.rings.padics.lattice_precision.pRational method*), 444
verschiebung() (*sage.rings.padics.witt_vector_ring.WittVectorRing method*), 471

W

WittVector (*class in sage.rings.padics.witt_vector*), 477
WittVector_finotti (*class in sage.rings.padics.witt_vector*), 479
WittVector_phantom (*class in sage.rings.padics.witt_vector*), 479
WittVector_pinvertible (*class in sage.rings.padics.witt_vector*), 480
WittVector_standard (*class in sage.rings.padics.witt_vector*), 481
WittVectorFrobeniusMorphism (*class in sage.rings.padics.witt_vector_ring*), 457
WittVectorRing (*class in sage.rings.padics.witt_vector_ring*), 458
WittVectorRing_finotti (*class in sage.rings.padics.witt_vector_ring*), 472
WittVectorRing_phantom (*class in sage.rings.padics.witt_vector_ring*), 472
WittVectorRing_pinvertible (*class in sage.rings.padics.witt_vector_ring*), 473
WittVectorRing_standard (*class in sage.rings.padics.witt_vector_ring*), 474
WittVectorVerschiebung (*class in sage.rings.padics.witt_vector_ring*), 474

X

xgcd() (*sage.rings.padics.lattice_precision.pRational method*), 444
xgcd() (*sage.rings.padics.padic_generic_element.pAdicGenericElement method*), 269

Z

zeta() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 177
zeta_order() (*sage.rings.padics.padic_base_generic.pAdicBaseGeneric method*), 178
Zp (*in module sage.rings.padics.factory*), 19
Zp_class (*class in sage.rings.padics.factory*), 74
ZpCA() (*in module sage.rings.padics.factory*), 59
ZpCR() (*in module sage.rings.padics.factory*), 59
ZpER() (*in module sage.rings.padics.factory*), 59
ZpFM() (*in module sage.rings.padics.factory*), 65
ZpFP() (*in module sage.rings.padics.factory*), 65
ZpLC() (*in module sage.rings.padics.factory*), 66
ZpLF() (*in module sage.rings.padics.factory*), 73

Zq() (*in module sage.rings.padics.factory*), 86
ZqCA() (*in module sage.rings.padics.factory*), 101
ZqCR() (*in module sage.rings.padics.factory*), 101
ZqFM() (*in module sage.rings.padics.factory*), 102
ZqFP() (*in module sage.rings.padics.factory*), 102
ZZ_pX_eis_shift_test() (*in module sage.rings.padics.pow_computer_ext*), 404